

Manual for Package PGFPLOTS

2D/3D Plots in L^AT_EX, Version 1.8

<http://sourceforge.net/projects/pgfplots>

Dr. Christian Feuersänger

`cfeuersaenger@users.sourceforge.net`

Revision 1.8 (2013/03/17)

Abstract

PGFPLOTS draws high-quality function plots in normal or logarithmic scaling with a user-friendly interface directly in T_EX. The user supplies axis labels, legend entries and the plot coordinates for one or more plots and PGFPLOTS applies axis scaling, computes any logarithms and axis ticks and draws the plots. It supports line plots, scatter plots, piecewise constant plots, bar plots, area plots, mesh- and surface plots, patch plots, contour plots, quiver plots, histogram plots, box plots, polar axes, ternary diagrams, smith charts and some more. It is based on Till Tantau's package PGF/TikZ.

Contents

1	Introduction	6
2	About PGFPlots: Preliminaries	7
2.1	Components	7
2.2	Upgrade remarks	7
2.2.1	New Optional Features	7
2.2.2	Old Features Which May Need Attention	8
2.3	The Team	9
2.4	Acknowledgements	10
2.5	Installation and Prerequisites	10
2.5.1	Licensing	10
2.5.2	Prerequisites	10
2.5.3	Installation in Windows	10
2.5.4	Installation of Linux Packages	11
2.5.5	Installation in Any Directory - the <code>TEXINPUTS</code> Variable	11
2.5.6	Installation Into a Local TDS Compliant <code>texmf</code> -Directory	11
2.5.7	Installation If Everything Else Fails...	11
2.6	Troubleshooting – Error Messages	12
2.6.1	Problems with available Dimen-registers	12
2.6.2	Dimension Too Large Errors	12
2.6.3	Restrictions for DVI-Viewers and <code>dvipdfm</code>	12
2.6.4	Problems with $\TeX$'s Memory Capacities	13
2.6.5	Problems with Language Settings and Active Characters	13
2.6.6	Other Problems	13
3	User's Guide: Drawing Axes and Plots	14
3.1	$\TeX$ -dialects: $\LaTeX$, Con $\TeX$ t, plain $\TeX$	14
3.2	A First Plot	16
3.3	Two Plots in the Same Axis	16
3.4	Logarithmic Plots	17
3.5	Cycling Line Styles	19
3.6	Scaling Plots	20
4	The Reference	22
4.1	The Axis-Environments	22
4.2	The <code>\addplot</code> Command: Coordinate Input	23
4.2.1	Coordinate Lists	27
4.2.2	Reading Coordinates From Tables	29
4.2.3	Computing Coordinates with Mathematical Expressions	33
4.2.4	Mathematical Expressions And File Data	37
4.2.5	Computing Coordinates with Mathematical Expressions (<code>gnuplot</code>)	38
4.2.6	Computing Coordinates with External Programs (<code>shell</code>)	40
4.2.7	Using External Graphics as Plot Sources	41
4.2.8	Keys To Configure Plot Graphics	44
4.2.9	Reading Coordinates From Files	52
4.3	About Options: Preliminaries	53
4.3.1	PGFPLOTS and TikZ Options	56

4.4	Two Dimensional Plot Types	56
4.4.1	Linear Plots	56
4.4.2	Smooth Plots	57
4.4.3	Constant Plots	57
4.4.4	Bar Plots	59
4.4.5	Histograms	67
4.4.6	Comb Plots	68
4.4.7	Quiver Plots (Arrows)	68
4.4.8	Stacked Plots	72
4.4.9	Area Plots	74
4.4.10	Scatter Plots	78
4.4.11	1D Colored Mesh Plots	87
4.4.12	Interrupted Plots	88
4.4.13	Patch Plots	90
4.5	Three Dimensional Plot Types	90
4.5.1	Before You Start With 3D	90
4.5.2	The <code>\addplot3</code> Command: Three Dimensional Coordinate Input	90
4.5.3	Line Plots	95
4.5.4	Scatter Plots	96
4.5.5	Mesh Plots	98
4.5.6	Surface Plots	101
4.5.7	Surface Plots with Explicit Color	108
4.5.8	Contour Plots	115
4.5.9	Parameterized Plots	126
4.5.10	3D Quiver Plots (Arrows)	127
4.5.11	About 3D Const Plots and 3D Bar Plots	127
4.5.12	Patch Plots	127
4.6	Markers, Linestyles, (Background-) Colors and Colormaps	134
4.6.1	Markers	134
4.6.2	Line Styles	139
4.6.3	Edges and Their Parameters	140
4.6.4	Font Size and Line Width	140
4.6.5	Colors	141
4.6.6	Color Maps	143
4.6.7	Cycle Lists – Options Controlling Line Styles	147
4.6.8	Axis Background	156
4.7	Providing Color Data - Point Meta	156
4.8	Axis Descriptions	161
4.8.1	Placement of Axis Descriptions	161
4.8.2	Alignment of Axis Descriptions	167
4.8.3	Labels	172
4.8.4	Legends	175
4.8.5	Legend Appearance	177
4.8.6	Legends with <code>\label</code> and <code>\ref</code>	186
4.8.7	Legends Outside Of an Axis	187
4.8.8	Legends with Customized Texts or Multiple Lines	189
4.8.9	Axis Lines	190
4.8.10	Two Ordinates	195
4.8.11	Axis Discontinuities	196
4.8.12	Color Bars	198
4.8.13	Color Bars Outside Of an Axis	207
4.9	Scaling Options	208
4.9.1	Common Scaling Options	209
4.9.2	Scaling Descriptions: Predefined Styles	220
4.9.3	Scaling Strategies	223
4.10	3D Axis Configuration	225
4.10.1	View Configuration	225
4.10.2	Styles Used Only For 3D Axes	227

4.10.3	Appearance Of The 3D Box	228
4.10.4	Axis Line Variants	231
4.11	Error Bars	231
4.11.1	Input Formats of Error Coordinates	234
4.12	Number Formatting Options	234
4.12.1	Frequently Used Number Printing Settings	235
4.12.2	PGFPlots-specific Number Formatting	236
4.13	Specifying the Plotted Range	239
4.14	Tick Options	245
4.14.1	Tick Coordinates and Label Texts	245
4.14.2	Tick Alignment: Positions and Shifts	256
4.14.3	Tick Scaling - Common Factors In Ticks	257
4.14.4	Tick Fine-Tuning	261
4.15	Grid Options	262
4.16	Custom Annotations	263
4.16.1	Accessing Axis Coordinates in Graphical Elements	264
4.16.2	Placing Nodes on Coordinates of a Plot	268
4.16.3	Placing Decorations on Top of a Plot	272
4.17	Style Options	273
4.17.1	All Supported Styles	273
4.17.2	(Re)Defining Own Styles	280
4.18	Alignment Options	280
4.18.1	Basic Alignment	280
4.18.2	Vertical Alignment with <code>baseline</code>	283
4.18.3	Horizontal Alignment	285
4.18.4	Alignment In Array Form (Subplots)	285
4.18.5	Miscellaneous for Alignment	289
4.19	The Picture's Size: Bounding Box and Clipping	289
4.19.1	Bounding Box Restrictions	289
4.19.2	Clipping	292
4.20	Closing Plots (Filling the Area Under Plots)	293
4.21	Symbolic Coordinates and User Transformations	295
4.21.1	String Symbols as Input Coordinates	296
4.21.2	Dates as Input Coordinates	297
4.22	Skipping Or Changing Coordinates – Filters	299
4.23	Transforming Coordinate Systems	303
4.24	Fitting Lines – Regression	305
4.25	Miscellaneous Options	308
4.26	TikZ Interoperability	313
4.27	Layers	316
4.27.1	Summary	316
4.27.2	Using Predefined Layers	316
4.27.3	Changing the Layer of Graphical Elements	319
4.28	Technical Internals	319
5	Related Libraries	321
5.1	Clickable Plots	321
5.1.1	Overview	321
5.1.2	Requirements for the Library	324
5.1.3	Customization	325
5.1.4	Using the Clickable Library in Other Contexts	327
5.2	Colormaps	327
5.3	Dates as Input Coordinates	332
5.4	Image Externalization	332
5.5	Grouping plots	332
5.5.1	Grouping options	335
5.6	Patchplots Library	338
5.6.1	Additional Patch Types	339

5.6.2	Automatic Patch Refinement and Triangulation	349
5.6.3	Peculiarities of Flat Shading and High Order Patches	351
5.6.4	Drawing Grids	352
5.7	Polar Axes	354
5.7.1	Polar Axes	354
5.7.2	Using Radians instead of Degrees	356
5.7.3	Mixing With Cartesian Coordinates	357
5.7.4	Special Polar Plot Types	357
5.7.5	Partial Polar Axes	358
5.8	Smith Charts	360
5.8.1	Smith Chart Axes	360
5.8.2	Size Control	362
5.8.3	Working with Prepared Data	367
5.8.4	Appearance Control and Styles	368
5.8.5	Controlling Arcs and Their Stop Points	370
5.9	Statistics	372
5.9.1	Box Plots	373
5.9.2	Histograms	383
5.10	Ternary Diagrams	388
5.10.1	Ternary Axis	388
5.10.2	Tieline Plots	397
5.11	Units in Labels	399
5.11.1	Preset SI prefixes	401
6	Memory and Speed considerations	403
6.1	Memory Limits of $\text{\TeX}$	403
6.2	Memory Limitations	404
6.2.1	LuaLa $\text{\TeX}$	404
6.2.2	Mik $\text{\TeX}$	404
6.2.3	$\text{\TeX}$ Live or similar installations	405
6.3	Reducing Typesetting Time	406
7	Import/Export From Other Formats	407
7.1	Export to pdf/eps	407
7.1.1	Using the Automatic Externalization Framework of $\text{\textit{TikZ}}$	407
7.1.2	Using the Externalization Framework of PGF By Hand	413
7.2	Importing From Matlab	414
7.2.1	Importing Mesh Data From Matlab To PGFPlots	414
7.2.2	matlab2pgfplots.m	415
7.2.3	matlab2pgfplots.sh	415
7.2.4	Importing Colormaps From Matlab	415
7.3	SVG Output	415
7.4	Generate PGFPLOTS Graphics Within Python	416
8	Utilities and Basic Level Commands	417
8.1	Utility Commands	417
8.2	Commands Inside Of PGFPlots Axes	419
8.3	Path Operations	420
8.4	Specifying Basic Coordinates	421
8.5	Accessing Axis Limits	425
8.6	Layer Access	425
	Index	426

Chapter 1

Introduction

This package provides tools to generate plots and labeled axes easily. It draws normal plots, logplots and semi-logplots, in two and three dimensions. Axis ticks, labels, legends (in case of multiple plots) can be added with key-value options. It supports line plots, scatter plots, piecewise constant plots, bar plots, area plots, mesh- and surface plots, patch plots, contour plots, quiver plots, histogram plots, box plots, polar axes, ternary diagrams, smith charts and some more. It can cycle through a set of predefined line/marker/color specifications.

In summary, its purpose is to simplify the generation of high-quality function and/or data plots, and solving the problems of

- consistency of document and font type and font size,
- direct use of $\text{\TeX}$ math mode in axis descriptions,
- consistency of data and figures (no third party tool necessary),
- inter-document consistency using preamble configurations and styles.

Although not necessary, separate `.pdf` or `.eps` graphics can be generated using the [external](#) library developed as part of `TikZ`.

You are invited to use PGFPLOTS for visualization of medium sized data sets in two and three dimensions. It is based on Till Tantau's package `PGF/TikZ`.

Chapter 2

About PGFPLOTS: Preliminaries

This section contains information about upgrades, the team, the installation (in case you need to do it manually) and troubleshooting. You may skip it completely except for the upgrade remarks.

PGFPLOTS is built completely on *TikZ*/PGF. Knowledge of *TikZ* will simplify the work with PGFPLOTS, although it is not required.

However, note that this library requires at least PGF version 2.10. At the time of this writing, many *T_EX*-distributions still contain the older PGF version 1.18, so it may be necessary to install a recent PGF prior to using PGFPLOTS.

2.1 Components

PGFPLOTS comes with two components:

1. the plotting component (which you are currently reading) and
2. the `PGFPLOTS`TABLE component which simplifies number formatting and postprocessing of numerical tables. It comes as a separate package and has its own manual [pgfplotstable.pdf](#).

2.2 Upgrade remarks

This release provides a lot of improvements which can be found in all detail in **ChangeLog** for interested readers. However, some attention is useful with respect to the following changes.

2.2.1 New Optional Features

PGFPLOTS has been written with backwards compatibility in mind: old *T_EX* files should compile without modifications and without changes in the appearance. However, new features occasionally lead to a different behavior. In such a case, PGFPLOTS will deactivate the new feature¹.

Any new features or bugfixes which cause backwards compatibility problems need to be activated *manually* and *explicitly*. In order to do so, you should use

```
\usepackage{pgfplots}  
\pgfplotsset{compat=1.8}
```

in your preamble. This will configure the compatibility layer.

You should have at least `compat=1.3`. The suggested value is printed to the `.log` file after running *T_EX*. Here is a list of changes introduced in recent versions of PGFPLOTS:

1. PGFPLOTS 1.8 comes with a new revision for alignment of label- and tick scale label alignment. Furthermore, it improves the bounding box for `hide axis`. This revision is enabled with `compat=1.8` or higher.
The configuration `compat=1.8` is *nessecary* to repair `axis lines=center` in three-dimensional axes.
2. PGFPLOTS 1.7 added new options for bar widths defined in terms of axis units. These are enabled with `compat=1.7` or higher.

¹In case of broken backwards compatibility, we apologize – and ask you to submit a bug report. We will take care of it.

3. PGFPLOTS 1.6 added new options for more accurate scaling and more scaling options for `\addplot3` `graphics`. These are enabled with `compat=1.6` or higher.
4. PGFPLOTS 1.5.1 interpretes circle- and ellipse radii as PGFPLOTS coordinates (older versions used PGF unit vectors which have no direct relation to PGFPLOTS). In other words: starting with version 1.5.1, it is possible to write `\draw circle[radius=5]` inside of an axis. This requires `\pgfplotsset{compat=1.5.1}` or higher.

Without this compatibility setting, circles and ellipses use low-level canvas units of PGF as in earlier versions.

5. PGFPLOTS 1.5 uses `log origin=0` as default (which influences logarithmic bar plots or stacked logarithmic plots). Older versions keep `log origin=infty`. This requires `\pgfplotsset{compat=1.5}` or higher.
6. PGFPLOTS 1.4 has fixed several smaller bugs which might produce differences of about 1–2pt compared to earlier releases. This requires `\pgfplotsset{compat=1.4}` or higher.
7. PGFPLOTS 1.3 comes with user interface improvements. The technical distinction between “behavior options” and “style options” of older versions is no longer necessary (although still fully supported). This is always activated.
8. PGFPLOTS 1.3 has a new feature which allows to *move axis labels tight to tick labels* automatically. This is strongly recommended. It requires `\pgfplotsset{compat=1.3}` or higher.

Since this affects the spacing, it is not enabled by default.

9. PGFPLOTS 1.3 supports reversed axes. It is no longer necessary to use workarounds with negative units.

Take a look at the `x dir=reverse` key.

Existing workarounds will still function properly. Use `\pgfplotsset{compat=1.3}` or higher together with `x dir=reverse` to switch to the new version.

2.2.2 Old Features Which May Need Attention

1. The `scatter/classes` feature produces proper legends as of version 1.3. This may change the appearance of existing legends of plots with `scatter/classes`.
2. Starting with PGFPLOTS 1.1, `\tikzstyle` should *no longer be used* to set PGFPLOTS options.

Although `\tikzstyle` is still supported for some older PGFPLOTS options, you should replace any occurrence of `\tikzstyle` with `\pgfplotsset{<style name>/.style={<key-value-list>}}` or the associated `/.append style` variant. See Section 4.17 for more detail.

I apologize for any inconvenience caused by these changes.

`/pgfplots/compat=1.8|1.7|1.6|1.5.1|1.5|1.4|1.3|pre 1.3|default` (initially default)

The preamble configuration

```
\usepackage{pgfplots}
\pgfplotsset{compat=1.8}
```

allows to choose between backwards compatibility and most recent features.

Occasionally, you might want to use different versions in the same document. Then, provide

```
\begin{figure}
  \pgfplotsset{compat=1.4}
  ...
  \caption{...}
\end{figure}
```

in order to restrict the compatibility setting to the actual context (in this case, the `figure` environment).

The the output of your `.log` file to see the suggested value for `compat`.

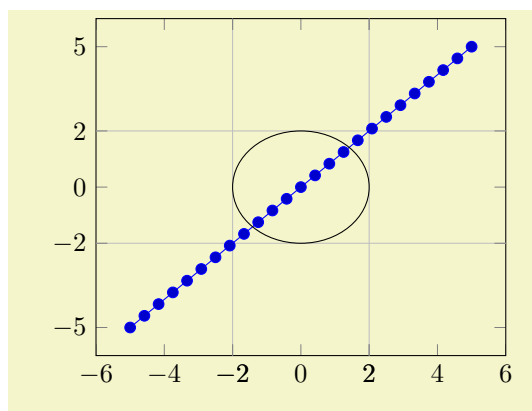
Use `\pgfplotsset{compat=default}` to restore the factory settings.

Although typically unnecessary, it is also possible to activate only selected changes and keep compatibility to older versions in general:

```
/pgfplots/compat/path replacement=<version>
/pgfplots/compat/labels=<version>
/pgfplots/compat/scaling=<version>
/pgfplots/compat/scale mode=<version>
/pgfplots/compat/empty line=<version>
/pgfplots/compat/plot3graphics=<version>
/pgfplots/compat/general=<version>
```

Let us assume that we have a document with `\pgfplotsset{compat=1.3}` and you want to keep it this way.

In addition, you realized that version 1.5.1 supports circles and ellipses. Then, use



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% preamble:
\pgfplotsset{compat=1.3,compat/path replacement=1.5.1}
\begin{tikzpicture}
\begin{axis}[
    extra x ticks={-2,2},
    extra y ticks={-2,2},
    extra tick style={grid=major}]
\addplot {x};
\draw (axis cs:0,0) circle[radius=2];
\end{axis}
\end{tikzpicture}
```

All of these keys accept the possible values of the `compat` key.

The `compat/path replacement` key controls how radii of circles and ellipses are interpreted.

The `compat/labels` key controls how axis labels are aligned: either uses adjacent to ticks or with an absolute offset. As of 1.8, it also enables an entirely new revision of the axis label styles. In most cases, you will see no difference – but it repairs `axis lines=center` in three-dimensional axes.

The `compat/scaling` key controls some bugfixes introduced in version 1.4 and 1.6: they might introduce slight scaling differences in order to improve the accuracy.

The `compat/plot3graphics` controls new features for `\addplot3 graphics`.

The `compat/scale mode` allows to enable/disable the warning “The content of your 3d axis has CHANGED compared to previous versions” because the `axis equal` and `unit vector ratio` features were broken for all versions before 1.6 and have been fixed in 1.6.

The `compat/empty line` allows to write empty lines into input files in order to generate a jump. This requires `compat=1.4` or newer. See `empty line` for details.

The `compat/general` key currently only activates `log origin`.

The detailed effects can be seen on the beginning of this section.

The value `<version>` can be `default`, a version number, and `newest`. The value `default` is the same as `pre 1.3` (up to insignificant changes). The use of `newest` is strongly *discouraged*: it might cause changes in your document, depending on the current version of PGFPLOTS. Please inspect your `.log` file to see suggestions for the best possible version.

2.3 The Team

PGFPLOTS has been written mainly by Christian Feuersänger with many improvements of Pascal Wolkotte and Nick Papior Andersen as a spare time project. We hope it is useful and provides valuable plots.

If you are interested in writing something but don’t know how, consider reading the auxiliary manual

[TeX-programming-notes.pdf](#) which comes with PGFPLOTS. It is far from complete, but maybe it is a good starting point (at least for more literature).

2.4 Acknowledgements

I thank God for all hours of enjoyed programming. I thank Pascal Wolkotte and Nick Papior Andersen for their programming efforts and contributions as part of the development team. I thank Jürnjakob Dugge for his contribution of [hist/density](#), matlab scripts for [\addplot3 graphics](#), excellent user forum help and helpful bug reports. I thank Stefan Tibus, who contributed the [plot shell](#) feature. I thank Tom Cashman for the contribution of the [reverse legend](#) feature. Special thanks go to Stefan Pinnow whose tests of PGFPLOTS lead to numerous quality improvements. Furthermore, I thank Dr. Schweitzer for many fruitful discussions and Dr. Meine for his ideas and suggestions. Special thanks go to Markus Böhning for proof-reading all the manuals of PGF, PGFPLOTS, and [PGFPLOTS TABLE](#). Thanks as well to the many international contributors who provided feature requests or identified bugs or simply improvements of the manual!

Last but not least, I thank Till Tantau and Mark Wibrow for their excellent graphics (and more) package PGF and *TikZ*, which is the base of PGFPLOTS.

2.5 Installation and Prerequisites

2.5.1 Licensing

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

A copy of the GNU General Public License can be found in the package file

`doc/latex/pgfplots/gpl-3.0.txt`

You may also visit <http://www.gnu.org/licenses>.

2.5.2 Prerequisites

PGFPLOTS requires PGF. You should generally use the most recent stable version of PGF. PGFPLOTS is used with

```
\usepackage{pgfplots}
\pgfplotsset{compat=yourversion}
```

in your preamble (see Section 3.1 for information about how to use it with ConTeXt and plain TeX).

The `compat=<yourversion>` entry should be added to activate new features, see the documentation of the [compat](#) key for more details.

There are several ways how to teach TeX where to find the files. Choose the option which fits your needs best.

2.5.3 Installation in Windows

Windows users often use MikTeX which downloads the latest stable package versions automatically. You do not need to install anything manually here.

However, MikTeX provides a feature to install packages locally in its own TeX-Directory-Structure (TDS). This is the preferred way if you like to install newer version than those of MikTeX. The basic idea is to unzip PGFPLOTS in a directory of your choice and configure the MikTeX Package Manager to use this specific directory with higher priority than its default paths. If you want to do this, start the MikTeX Settings using “Start » Programs » MikTeX » Settings”. There, use the “Roots” menu section. It contains the MikTeX Package directory as initial configuration. Use “Add” to select the directory in which the unzipped PGFPLOTS tree resides. Then, move the newly added path to the list’s top using the “Up” button. Then press “Ok”. For MikTeX 2.8, you may need to uncheck the “Show MikTeX-maintained root directories” button to see the newly installed path.

MikTeX complains if the provided directory is not TDS conform (see Section 2.5.6 for details), so you can't provide a wrong directory here. This method does also work for other packages, but some packages may need some directory restructuring before MikTeX accepts them.

2.5.4 Installation of Linux Packages

At the time of this writing, I am unaware of PGFPLOTS packages for recent stable Linux distributions. For Ubuntu, there are unofficial Ubuntu Package Repositories which can be added to the Ubuntu Package Tools. The idea is: add a simple URL to the Ubuntu Package Tool, run update and the installation takes place automatically. These URLs are maintained as PPA on Ubuntu Servers.

The PGFPLOTS download area on sourceforge contains recent links about Ubuntu Package Repositories, go to <http://sourceforge.net/projects/pgfplots/files> and download the readme files with recent links.

2.5.5 Installation in Any Directory - the TEXINPUTS Variable

You can simply install PGFPLOTS anywhere on your harddrive, for example into

```
/foo/bar/pgfplots.
```

Then, you set the TEXINPUTS variable to

```
TEXINPUTS=/foo/bar/pgfplots//:
```

The trailing ':' tells TeX to check the default search paths after /foo/bar/pgfplots. The double slash '/' tells TeX to search all subdirectories.

If the TEXINPUTS variable already contains something, you can append the line above to the existing TEXINPUTS content.

Furthermore, you should set TEXDOCS as well,

```
TEXDOCS=/foo/bar/pgfplots//:
```

so that the TeX-documentation system finds the files `pgfplots.pdf` and `pgfplotstable.pdf` (on some systems, it is then enough to use `texdoc pgfplots`).

Please refer to your operating systems manual for how to set environment variables.

2.5.6 Installation Into a Local TDS Compliant texmf-Directory

PGFPLOTS comes in a "TeX Directory Structure" (TDS) conforming directory structure, so you can simply unpack the files into a directory which is searched by TeX automatically. Such directories are `~/texmf` on Linux systems, for example.

Copy PGFPLOTS to a local `texmf` directory like `~/texmf`. You need at least the PGFPLOTS directories `tex/generic/pgfplots` and `tex/latex/pgfplots`. Then, run `texhash` (or some equivalent path-updating command specific to your TeX distribution).

The TDS consists of several sub directories which are searched separately, depending on what has been requested: the sub directories `doc/latex/⟨package⟩` are used for (La)TeX documentation, the sub-directories `doc/generic/⟨package⟩` for documentation which apply to LaTeX and other TeX dialects (like plain TeX and ConTeXt which have their own, respective sub-directories) as well.

Similarly, the `tex/latex/⟨package⟩` sub-directories are searched whenever LaTeX packages are requested. The `tex/generic/⟨package⟩` sub-directories are searched for packages which work for LaTeX and other TeX dialects.

Do not forget to run `texhash`.

2.5.7 Installation If Everything Else Fails...

If TeX still doesn't find your files, you can copy all `.sty` and all `.code.tex`-files (perhaps all `.def` files as well) into your current project's working directory. In fact, you need everything which is in the `tex/latex/pgfplots` and `tex/generic/pgfplots` sub directories.

Please refer to <http://www.ctan.org/installationadvice/> for more information about package installation.

2.6 Troubleshooting – Error Messages

This section discusses some problems which may occur when using PGFPLOTS. Some of the error messages are shown in the index, take a look at the end of this manual (under “Errors”).

2.6.1 Problems with available Dimen-registers

To avoid problems with the many required $\TeX$ -registers for PGF and PGFPLOTS, you may want to include

```
\usepackage{etex}
```

as first package. This avoids problems with “no room for a new dimen” in most cases. It should work with any modern installation of $\TeX$ (it activates the e- $\TeX$ extensions).

2.6.2 Dimension Too Large Errors

The core mathematical engine of PGF relies on $\TeX$ registers to perform fast arithmetics. To compute $50 + 299$, it actually computes $50\text{pt}+299\text{pt}$ and strips the pt suffix of the result. Since $\TeX$ registers can only contain numbers up to ± 16384 , overflow error messages like “Dimension too large” occur if the result leaves the allowed range. Normally, this should never happen – PGFPLOTS uses a floating point unit with data range $\pm 10^{324}$ and performs all mappings automatically. However, there are some cases where this fails. Some of these cases are:

1. The axis range (for example, for x) becomes *relatively* small. It’s no matter if you have absolutely small ranges like $[10^{-17}, 10^{-16}]$. But if you have an axis range like $[1.99999999, 2]$, where a lot of significant digits are necessary, this may be problematic.

I guess I can’t help here: you may need to prepare the data somehow before PGFPLOTS processes it.

2. This may happen as well if you only view a very small portion of the data range.

This happens, for example, if your input data ranges from $x \in [0, 10^6]$, and you say `xmax=10`.

Consider using the `restrict x to domain*= $\langle min \rangle$ : $\langle max \rangle$` key in such a case, where the $\langle min \rangle$ and $\langle max \rangle$ should be (say) four times of your axis limits (see page 302 for details).

3. The `axis equal` key will be confused if x and y have a very different scale.
4. You may have found a bug – please contact the developers.

2.6.3 Restrictions for DVI-Viewers and dvipdfm

PGF is compatible with

- `latex/dvips`,
- `latex/dvipdfm`,
- `pdflatex`,
- $\vdots$

However, there are some restrictions: I don’t know any DVI viewer which is capable of viewing the output of PGF (and therefor PGFPLOTS as well). After all, DVI has never been designed to draw something different than text and horizontal/vertical lines. You will need to view the postscript file or the pdf-file.

Then, the DVI/pdf combination doesn’t support all types of shadings (for example, the `shader=interp` is only available for `dvips`, `pdftex`, `dvipdfmx`, and `xetex` drivers).

Furthermore, PGF needs to know a *driver* so that the DVI file can be converted to the desired output. Depending on your system, you need the following options:

- `latex/dvips` does not need anything special because `dvips` is the default driver if you invoke `latex`.
- `pdflatex` will also work directly because `pdflatex` will be detected automatically.
- `latex/dvipdfm` requires to use

```
\def\pgfsysdriver{pgfsys-dvipdfm.def}  
%\def\pgfsysdriver{pgfsys-pdftex.def}  
%\def\pgfsysdriver{pgfsys-dvips.def}  
%\def\pgfsysdriver{pgfsys-dvipdfmx.def}  
%\def\pgfsysdriver{pgfsys-xetex.def}  
\usepackage{pgfplots}.
```

The uncommented commands could be used to set other drivers explicitly.

Please read the corresponding sections in [5, Section 7.2.1 and 7.2.2] if you have further questions. These sections also contain limitations of particular drivers.

The choice which won't produce any problems at all is `pdflatex`.

2.6.4 Problems with T_EX's Memory Capacities

PGFPLOTS can handle small up to medium sized plots. However, T_EX has never been designed for data plots – you will eventually face the problem of small memory capacities. See Section 6.1 for how to enlarge them.

2.6.5 Problems with Language Settings and Active Characters

Both PGF and PGFPLOTS use a lot of active characters – which may lead to incompatibilities with other packages which define active characters. Compatibility is better than in earlier versions, but may still be an issue. The manual compiles with the `babel` package for english and french, the `german` package does also work. If you experience any trouble, let me know. Sometimes it may work to disable active characters temporarily (`babel` provides such a command).

2.6.6 Other Problems

Please read the mailing list at

<http://sourceforge.net/projects/pgfplots/support>.

Perhaps someone has also encountered your problem before, and maybe he came up with a solution.

Please write a note on the mailing list if you have a different problem. In case it is necessary to contact the authors directly, consider the addresses shown on the title page of this document.

Chapter 3

User's Guide: Drawing Axes and Plots

The user interface of PGFPLOTS consists of three components: a `tikzpicture` environment, an `axis` and the `\addplot` command.

Each axis is generated as part of a picture environment (which can be used to annotate plots afterwards, for example). The axis environment encapsulates one or more `\addplot` commands and controls axis-wide settings (like limits, legends, and descriptions). The `\addplot` command supports several coordinate input methods (like table input or mathematical expressions) and allows various sorts of visualization options with straight lines as initial configuration.

The rest of PGFPLOTS is a huge set of key-value options to modify the initial configuration or to select plot types. The reference manual has been optimized for electronical display: a lot of examples illustrate the features, and reference documentation can be found by clicking into the sourcecode text fields. Note that most pdf viewers also support to jump back from a hyperlink: for Acrobat Reader, open the menu View»Toolbars»More Tools and activate the “Previous View” and “Next View” buttons (which are under “Page Navigation Toolbar”). Thus, knowledge of all keys is unnecessary; you can learn them when it is necessary.

To learn PGFPLOTS, you should learn about the `\addplot` command and its coordinate input methods. The most important input methods are `\addplot table` and `\addplot expression`.

The following sections explain the basics of PGFPLOTS, namely how to work with the `\addplot` commands and `axis` environments and how line styles are assigned automatically.

3.1 T_EX-dialects: L^AT_EX, ConT_EXt, plain T_EX

The starting point for PGFPLOTS is an `axis` environment like `axis` or the logarithmic variants `semilogxaxis`, `semilogyaxis` or `loglogaxis`.

Each environment is available for L^AT_EX, ConT_EXt and plain T_EX:

L^AT_EX: `\usepackage{pgfplots}` and

```
\begin{tikzpicture}
\begin{axis}
...
\end{axis}
\end{tikzpicture}
```

```
\begin{tikzpicture}
\begin{semilogxaxis}
...
\end{semilogxaxis}
\end{tikzpicture}
```

```

\documentclass[a4paper]{article}

% for dvipdfm:
% \def\pgfsysdriver{pgfsys-dvipdfm.def}
\usepackage{pgfplots}
\pgfplotsset{compat=1.6}% <-- moves axis labels near ticklabels (respects tick label widths)

\begin{document}
\begin{figure}
  \centering
  \begin{tikzpicture}
    \begin{loglogaxis}[xlabel=Cost,ylabel=Error]
      \addplot coordinates {
        (5,      8.31160034e-02)
        (17,     2.54685628e-02)
        (49,     7.40715288e-03)
        (129,    2.10192154e-03)
        (321,    5.87352989e-04)
        (769,    1.62269942e-04)
        (1793,   4.44248889e-05)
        (4097,   1.20714122e-05)
        (9217,   3.26101452e-06)
      };
      \addplot coordinates {
        (7,      8.47178381e-02)
        (31,     3.04409349e-02)
        (111,    1.02214539e-02)
        (351,    3.30346265e-03)
        (1023,   1.03886535e-03)
        (2815,   3.19646457e-04)
        (7423,   9.65789766e-05)
        (18943,  2.87339125e-05)
        (47103,  8.43749881e-06)
      };
      \legend{Case 1,Case 2}
    \end{loglogaxis}
  \end{tikzpicture}
  \caption{A larger example}
\end{figure}
\end{document}

```

C_ON_TE_XT: `\usemodule[pgfplots]` and

<code>\starttikzpicture</code>	<code>\starttikzpicture</code>
<code>\startaxis</code>	<code>\startsemilogxaxis</code>
<code>...</code>	<code>...</code>
<code>\stopaxis</code>	<code>\stopsemilogxaxis</code>
<code>\stoptikzpicture</code>	<code>\stoptikzpicture</code>

A complete C_ON_TE_XT-example file can be found in

```
doc/context/pgfplots/pgfplotsexample.tex.
```

p^LA^IN T_EX: `\input pgfplots.tex` and

<code>\tikzpicture</code>	<code>\tikzpicture</code>
<code>\axis</code>	<code>\semilogxaxis</code>
<code>...</code>	<code>...</code>
<code>\endaxis</code>	<code>\endsemilogxaxis</code>
<code>\endtikzpicture</code>	<code>\endtikzpicture</code>

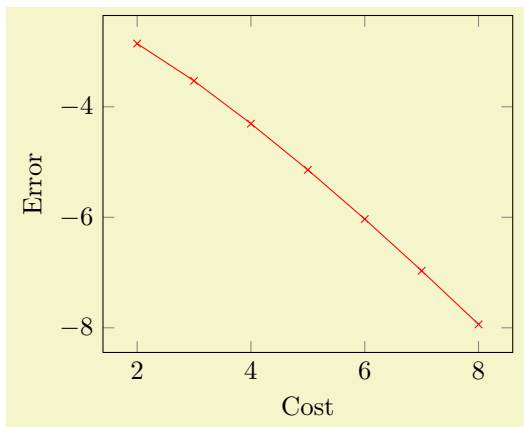
A complete p^LA^IN-T_EX-example file can be found in

```
doc/plain/pgfplots/pgfplotsexample.tex.
```

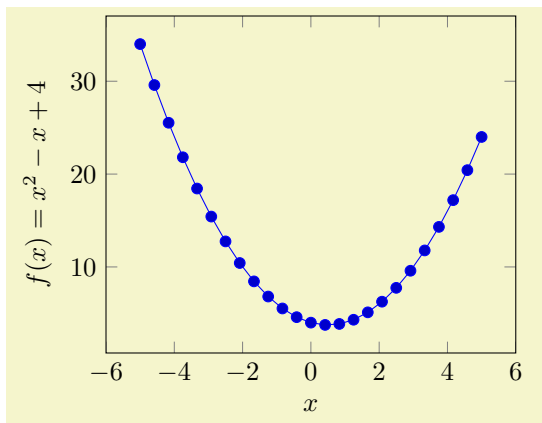
If you use `latex` / `dvips` or `pdflatex`, no further modifications are necessary. For `dvipdfm`, you should use the `\def\pgfsysdriver` line as indicated above in the examples (see also Section 2.6.3).

3.2 A First Plot

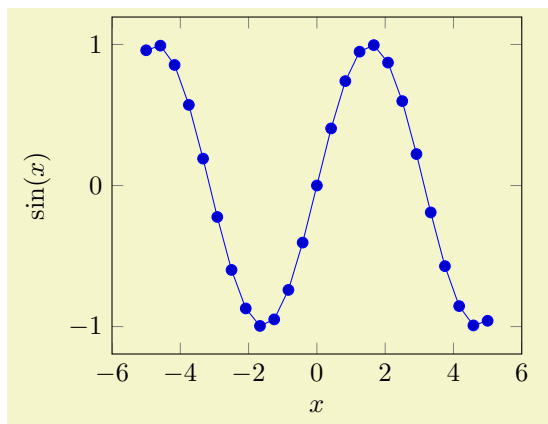
Plotting is done using `\begin{axis} ... \addplot ...; \end{axis}`, where `\addplot` is the main interface to perform plotting operations.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xlabel=Cost,
    ylabel=Error
  ]
    \addplot[color=red,mark=x] coordinates {
      (2,-2.8559703)
      (3,-3.5301677)
      (4,-4.3050655)
      (5,-5.1413136)
      (6,-6.0322865)
      (7,-6.9675052)
      (8,-7.9377747)
    };
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xlabel=$x$,
    ylabel={$f(x) = x^2 - x + 4$}
  ]
    % use TeX as calculator:
    \addplot {x^2 - x + 4};
  \end{axis}
\end{tikzpicture}
```



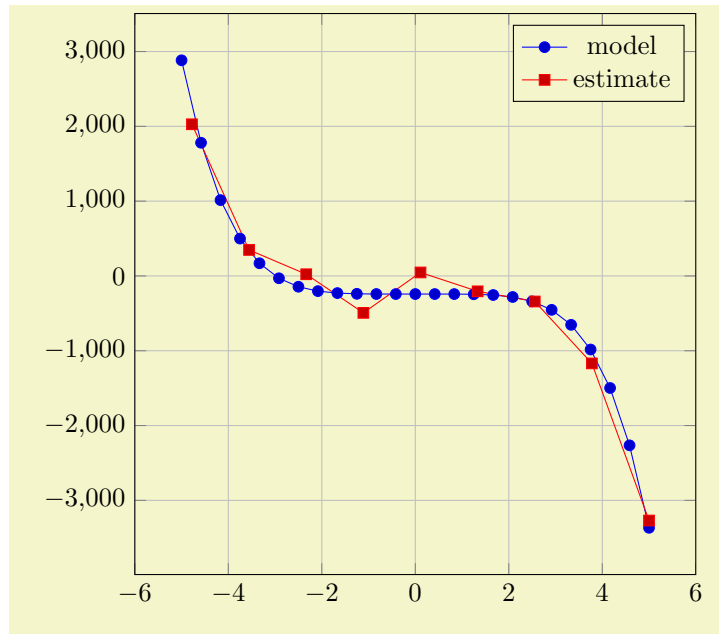
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xlabel=$x$,
    ylabel=$\sin(x)$
  ]
    % invoke external gnuplot as
    % calculator:
    \addplot gnuplot[id=sin]{sin(x)};
  \end{axis}
\end{tikzpicture}
```

The `plot coordinates`, `plot expression` and `plot gnuplot` commands are three of the several supported ways to create plots, see Section 4.2 for more details¹ and the remaining ones (`plot file`, `plot shell`, `plot table` and `plot graphics`). The options ‘`xlabel`’ and ‘`ylabel`’ define axis descriptions.

3.3 Two Plots in the Same Axis

Multiple `\addplot`-commands can be placed into the same axis, and a `cycle list` is used to automatically select different line styles:

¹Please note that you need `gnuplot` installed to use `plot gnuplot`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    height=9cm,
    width=9cm,
    grid=major,
  ]

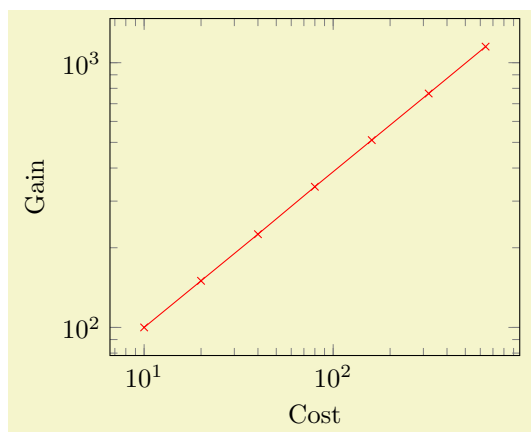
    \addplot {-x^5 - 242};
    \addlegendentry{model}

    \addplot coordinates {
      (-4.77778,2027.60977)
      (-3.55556,347.84069)
      (-2.33333,22.58953)
      (-1.11111,-493.50066)
      (0.11111,46.66082)
      (1.33333,-205.56286)
      (2.55556,-341.40638)
      (3.77778,-1169.24780)
      (5.00000,-3269.56775)
    };
    \addlegendentry{estimate}
  \end{axis}
\end{tikzpicture}
```

A legend entry is generated if there are `\addlegendentry` commands (or one `\legend` command).

3.4 Logarithmic Plots

Logarithmic plots show $\log x$ versus $\log y$ (or just one logarithmic axis). PGFPLOTS normally uses the natural logarithm, i.e. basis $e \approx 2.718$ (see the key `\log basis x`). Now, the axis description also contains minor ticks and the labels are placed at 10^i .

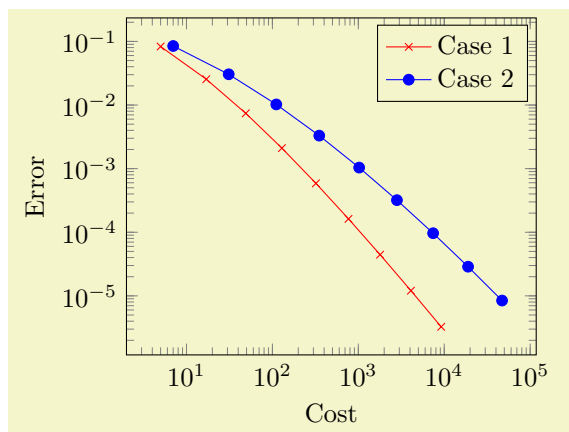


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[xlabel=Cost,ylabel=Gain]
\addplot[color=red,mark=x] coordinates {
(10,100)
(20,150)
(40,225)
(80,340)
(160,510)
(320,765)
(640,1150)
};
\end{loglogaxis}
\end{tikzpicture}
```

A common application is to visualise scientific data. This is often provided in the format $1.42 \cdot 10^4$, usually written as $1.42\text{e}+04$. Suppose we have a numeric table named `pgfplots.testtable`, containing

Level	Cost	Error
1	7	8.471e-02
2	31	3.044e-02
3	111	1.022e-02
4	351	3.303e-03
5	1023	1.038e-03
6	2815	3.196e-04
7	7423	9.657e-05
8	18943	2.873e-05
9	47103	8.437e-06

then we can plot `Cost` versus `Error` using



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
xlabel=Cost,
ylabel=Error]
\addplot[color=red,mark=x] coordinates {
(5, 8.31160034e-02)
(17, 2.54685628e-02)
(49, 7.40715288e-03)
(129, 2.10192154e-03)
(321, 5.87352989e-04)
(769, 1.62269942e-04)
(1793, 4.44248889e-05)
(4097, 1.20714122e-05)
(9217, 3.26101452e-06)
};
\addplot[color=blue,mark=*)
table[x=Cost,y=Error] {pgfplots.testtable};
\legend{Case 1,Case 2}
\end{loglogaxis}
\end{tikzpicture}
```

The first plot employs inline coordinates; the second one reads numerical data from file and plots column 'Cost' versus 'Error'.

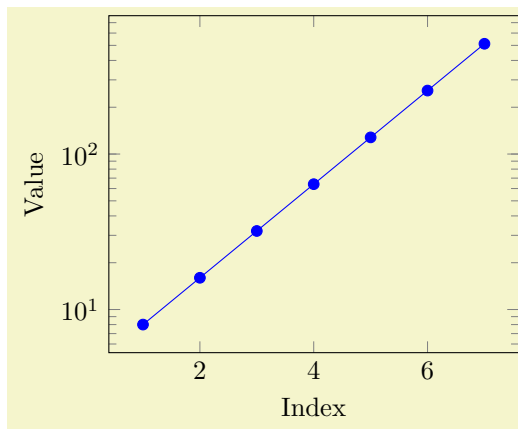
Besides the environment "loglogaxis" you can use

- `\begin{axis}...\end{axis}` for normal plots,
- `\begin{semilogxaxis}...\end{semilogxaxis}` for plots which have a normal y axis and a logarithmic x axis,
- `\begin{semilogyaxis}...\end{semilogyaxis}` the same with x and y switched,
- `\begin{loglogaxis}...\end{loglogaxis}` for double-logarithmic plots.

You can also use

```
\begin{axis}[xmode=normal,ymode=log]
...
\end{axis}
```

which is the same as `\begin{semilogyaxis}...\end{semilogyaxis}`.

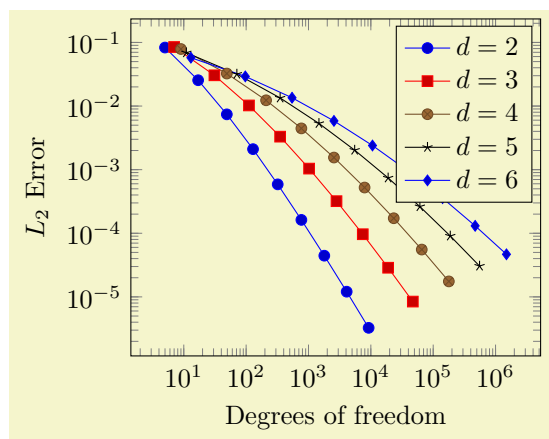


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  xlabel=Index,ylabel=Value]

\addplot[color=blue,mark=*] coordinates {
  (1,8)
  (2,16)
  (3,32)
  (4,64)
  (5,128)
  (6,256)
  (7,512)
};
\end{semilogyaxis}%
\end{tikzpicture}%
```

3.5 Cycling Line Styles

You can skip the style arguments for `\addplot[...]` to determine plot specifications from a predefined list:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel={Degrees of freedom},
  ylabel={$L_2$ Error}
]
\addplot coordinates {
  (5,8.312e-02) (17,2.547e-02) (49,7.407e-03)
  (129,2.102e-03) (321,5.874e-04) (769,1.623e-04)
  (1793,4.442e-05) (4097,1.207e-05) (9217,3.261e-06)
};

\addplot coordinates{
  (7,8.472e-02) (31,3.044e-02) (111,1.022e-02)
  (351,3.303e-03) (1023,1.039e-03) (2815,3.196e-04)
  (7423,9.658e-05) (18943,2.873e-05) (47103,8.437e-06)
};

\addplot coordinates{
  (9,7.881e-02) (49,3.243e-02) (209,1.232e-02)
  (769,4.454e-03) (2561,1.551e-03) (7937,5.236e-04)
  (23297,1.723e-04) (65537,5.545e-05) (178177,1.751e-05)
};

\addplot coordinates{
  (11,6.887e-02) (71,3.177e-02) (351,1.341e-02)
  (1471,5.334e-03) (5503,2.027e-03) (18943,7.415e-04)
  (61183,2.628e-04) (187903,9.063e-05) (553983,3.053e-05)
};

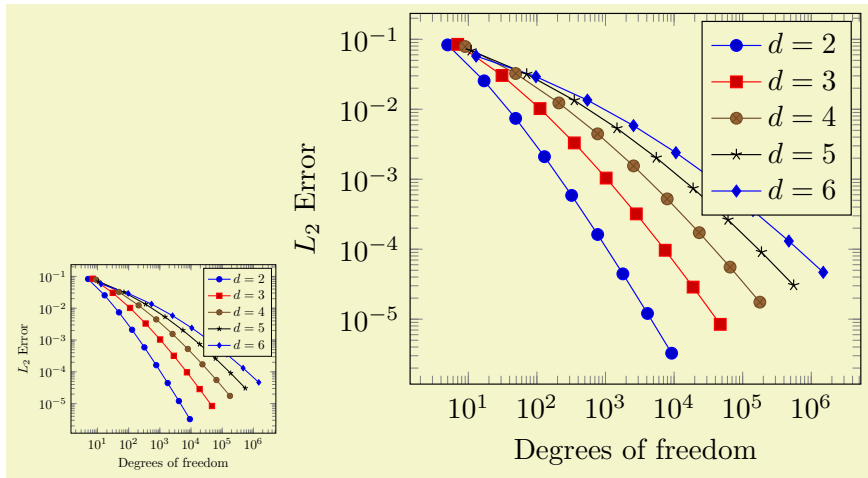
\addplot coordinates{
  (13,5.755e-02) (97,2.925e-02) (545,1.351e-02)
  (2561,5.842e-03) (10625,2.397e-03) (40193,9.414e-04)
  (141569,3.564e-04) (471041,1.308e-04) (1496065,4.670e-05)
};
};
\legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
\end{loglogaxis}
\end{tikzpicture}

```

The `cycle list` can be modified, see the reference below.

3.6 Scaling Plots

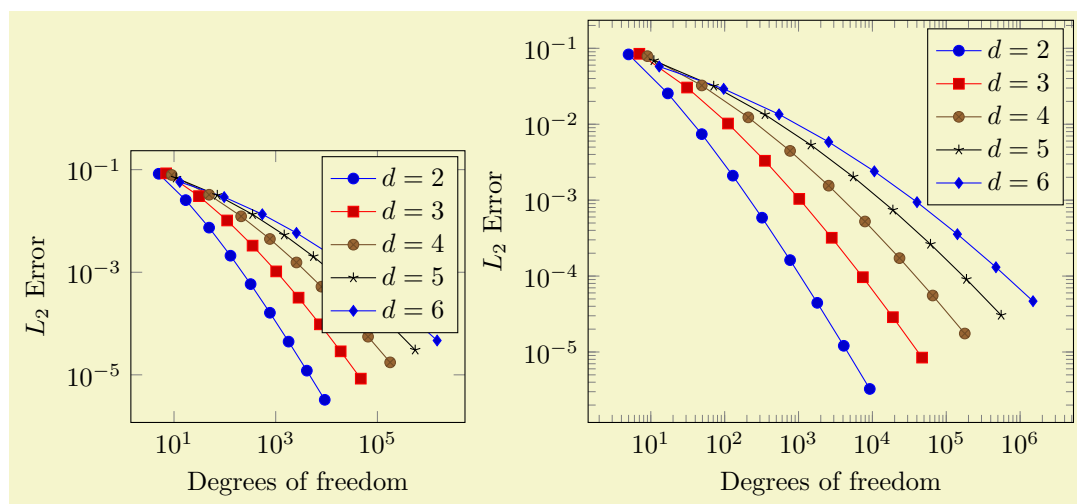
You can use any of the TikZ options to modify the appearance. For example, the “`scale`” transformation takes the picture as such and scales it (just like `\includegraphics`):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}[scale=0.5]
  \begin{loglogaxis}[
    xlabel={Degrees of freedom},
    ylabel={$L_2$ Error}
  ]
    \plotcoords
    \legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
  \end{loglogaxis}
\end{tikzpicture}

\begin{tikzpicture}[scale=1.1]
  \begin{loglogaxis}[
    xlabel={Degrees of freedom},
    ylabel={$L_2$ Error}
  ]
    \plotcoords
    \legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
  \end{loglogaxis}
\end{tikzpicture}
```

However, you can also scale plots by assigning a `width=5cm` and/or `height=3cm` argument. This only affects the distance of point coordinates, no font sizes or axis descriptions:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    width=6cm,
    xlabel={Degrees of freedom},
    ylabel={$L_2$ Error}
  ]
    \plotcoords
    \legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
  \end{loglogaxis}
\end{tikzpicture}

\begin{tikzpicture}
  \begin{loglogaxis}[
    width=8cm,
    xlabel={Degrees of freedom},
    ylabel={$L_2$ Error}
  ]
    \plotcoords
    \legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
  \end{loglogaxis}
\end{tikzpicture}
```

Use the predefined styles `normalsize`, `small`, `footnotesize` to adopt font sizes and ticks automatically. Use the `/pgfplots/scale` key to rescale the axis without affecting fonts.

Chapter 4

The Reference

4.1 The Axis-Environments

There is an axis environment for linear scaling, two for semi-logarithmic scaling and one for double-logarithmic scaling.

```
\begin{tikzpicture}[\langle options of tikz \rangle]
  \langle environment contents \rangle
\end{tikzpicture}
```

This is the graphics environment of TikZ. It produces a single picture and encloses also every axis.

Instead of using the environment version, there is also a shortcut command

```
\tikz{\langle picture content \rangle}
```

which can be used alternatively.

```
\begin{axis}[\langle options \rangle]
  \langle environment contents \rangle
\end{axis}
```

The axis environment for normal plots with linear axis scaling.

The ‘`every linear axis`’ style key can be modified with

```
\pgfplotsset{every linear axis/.append style={...}}
```

to install styles specifically for linear axes. These styles can contain both TikZ- and PGFLOTS options.

```
\begin{semilogxaxis}[\langle options \rangle]
  \langle environment contents \rangle
\end{semilogxaxis}
```

The axis environment for logarithmic scaling of x and normal scaling of y . Use

```
\pgfplotsset{every semilogx axis/.append style={...}}
```

to install styles specifically for the case with `xmode=log`, `y mode=normal`.

The logarithmic scaling means to apply the natural logarithm (base e) to each x coordinate. Furthermore, ticks will be typeset as $10^{\langle exponent \rangle}$, see Section 4.12 for more details.

```
\begin{semilogyaxis}[\langle options \rangle]
  \langle environment contents \rangle
\end{semilogyaxis}
```

The axis environment for normal scaling of x and logarithmic scaling of y ,

The style ‘`every semilogy axis`’ will be installed for each such plot.

The same remarks as for `semilogxaxis` apply here as well.

```
\begin{loglogaxis}[\langle options \rangle]
  \langle environment contents \rangle
```

`\end{loglogaxis}`

The axis environment for logarithmic scaling of both, x and y axes. As for the other axis possibilities, there is a style ‘`every loglog axis`’ which is installed at the environment’s beginning.

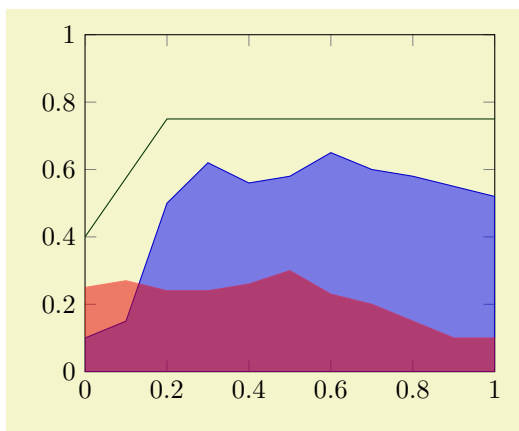
The same remarks as for `semilogxaxis` apply here as well.

They are all equivalent to

```
\begin{axis}[
  xmode=log|normal,
  ymode=log|normal]
...
\end{axis}
```

with properly set variables ‘`xmode`’ and ‘`ymode`’ (see below).

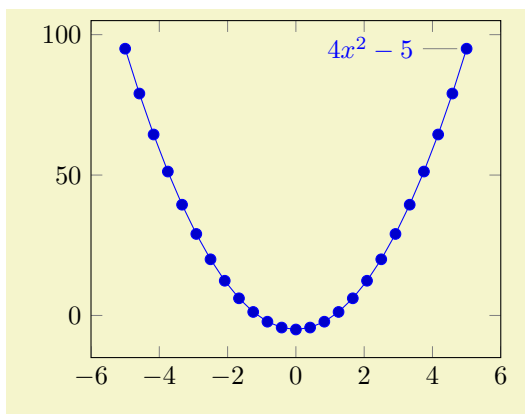
4.2 The `\addplot` Command: Coordinate Input



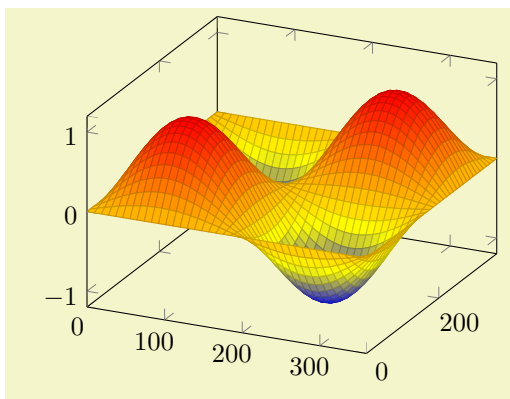
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[ymin=0,ymax=1,enlargelimits=false]
\addplot
  [blue!80!black,fill=blue,fill opacity=0.5]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
 (0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
 (0.8,0.58) (0.9,0.55) (1,0.52)}
|- (axis cs:0,0) -- cycle;

\addplot
  [red,fill=red!90!black,opacity=0.5]
coordinates
{(0,0.25) (0.1,0.27) (0.2,0.24) (0.3,0.24)
 (0.4,0.26) (0.5,0.3) (0.6,0.23) (0.7,0.2)
 (0.8,0.15) (0.9,0.1) (1,0.1)}
|- (axis cs:0,0) -- cycle;

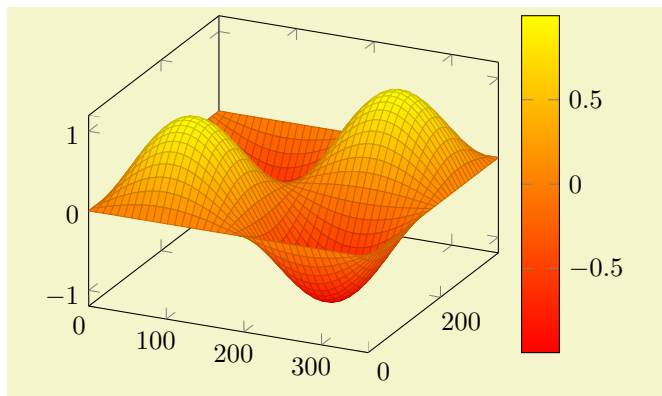
\addplot[green!20!black] coordinates
{(0,0.4) (0.2,0.75) (1,0.75)};
\end{axis}
\end{tikzpicture}
```



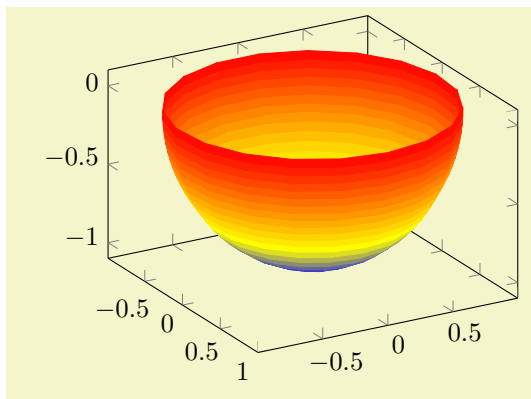
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[id=parable,domain=-5:5]
  gnuplot{4*x**2 - 5}
  node[pin=180:{$4x^2-5$}]{};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,domain=0:360,samples=40]
      {sin(x)*sin(y)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colormap/redyellow,colorbar]
    \addplot3[surf,
      domain=0:360,samples=40]
      {sin(x)*sin(y)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={60}{30}]
    \addplot3[surf,shader=flat,
      samples=20,
      domain=-1:0,y domain=0:2*pi,
      z buffer=sort]
      ({sqrt(1-x^2) * cos(deg(y))},
      {sqrt(1-x^2) * sin(deg(y))},
      x);
  \end{axis}
\end{tikzpicture}
```

Inside of an axis environment, the `\addplot` command is the main user interface. It comes in two variants: `\addplot` for two-dimensional visualization and `\addplot3` for three-dimensional visualization.

`\addplot` [*options*] *input data* *trailing path commands*;

This is the main plotting command, available within each axis environment. It can be used one or more times within an axis to add plots to the current axis. There is also an `\addplot3` command which is described in Section 4.5.

It reads point coordinates from one of the available input sources specified by *input data*, updates limits, remembers *options* for use in a legend (if any) and applies any necessary coordinate transformations (or logarithms).

The *options* can be omitted in which case the next entry from the `cycle list` will be inserted as *options*. These keys characterize the plot's type like linear interpolation with `sharp plot`, `smooth`

plot, constant interpolation with `const plot`, bar plot, `mesh` plots, `surface` plots or whatever and define `colors`, `markers` and line specifications¹. Plot variants like error bars, the number of `samples` or a sample `domain` can also be configured in `\options`.

The `\input data` is one of several coordinate input tools which are described in more detail below. Finally, if `\addplot` successfully processed all coordinates from `\input data`, it generates TikZ paths to realize the drawing operations. Any `\trailing path commands` are appended to the final drawing command, allowing to continue the TikZ path (from the last plot coordinate).

Some more details:

- The style `/pgfplots/every axis plot` will be installed at the beginning of `\options`. That means you can use

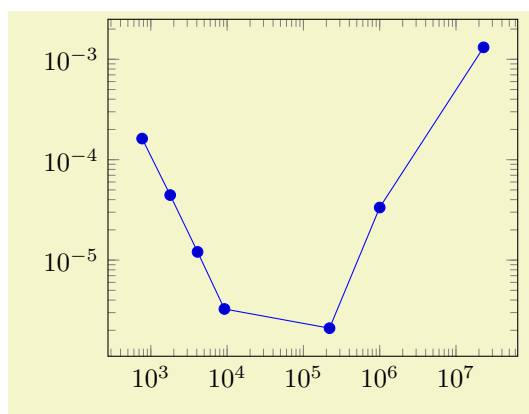
```
\pgfplotsset{every axis plot/.append style={...}}
```

to add options to all your plots - maybe to set line widths to `thick`. Furthermore, if you have more than one plot inside of an axis, you can also use

```
\pgfplotsset{every axis plot no 3/.append style={...}}
```

to modify options for the plot with number 3 only. The first plot in an axis has number 0.

- The `\options` are remembered for the legend. They are available as ‘`current plot style`’ as long as the path is not yet finished or in associated error bars.
- See Subsection 4.6 for a list of available markers and line styles.
- For log plots, PGFPLOTS will compute the natural logarithm `log(.)` numerically using a floating point unit developed for this purpose². For example, the following numbers are valid input to `\addplot`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}
\addplot coordinates {
(769, 1.6227e-04)
(1793, 4.4425e-05)
(4097, 1.2071e-05)
(9217, 3.2610e-06)
(2.2e5, 2.1E-6)
(1e6, 0.00003341)
(2.3e7, 0.00131415)
};
\end{loglogaxis}
\end{tikzpicture}
```

You can represent arbitrarily small or very large numbers as long as its logarithm can be represented as a $\text{\TeX}$ -length (up to about 16384). Of course, any coordinate $x \leq 0$ is not possible since the logarithm of a non-positive number is not defined. Such coordinates will be skipped automatically (using the initial configuration `unbounded coords=discard`).

- For normal (non-logarithmic) axes, PGFPLOTS applies floating point arithmetics to support large or small numbers like 0.00000001234 or $1.234 \cdot 10^{24}$. Its number range is much larger than $\text{\TeX}$'s native support for numbers. The relative precision is between 4 and 7 significant decimal digits for the mantissa.

As soon as the axes limits are completely known, PGFPLOTS applies a transformation which maps these floating point numbers into $\text{\TeX}$ -precision using transformations

$$T_x(x) = 10^{s_x} \cdot x - a_x \text{ and } T_y(y) = 10^{s_y} \cdot y - a_y \text{ and (for 3D plots) } T_z(z) = 10^{s_z} \cdot z - a_z$$

with properly chosen integers $s_x, s_y, s_z \in \mathbb{Z}$ and shifts $a_x, a_y, a_z \in \mathbb{R}$. Section 4.25 contains a description of `disabledatascaling` and provides more details about the transformation.

¹In version 1.2.2 and earlier, there was an explicit distinction between “behaviour” options like error bars, domain, number of samples etc. and “style options” like color, line width, markers etc. This distinction is obsolete now, simply collect everything into `\options`.

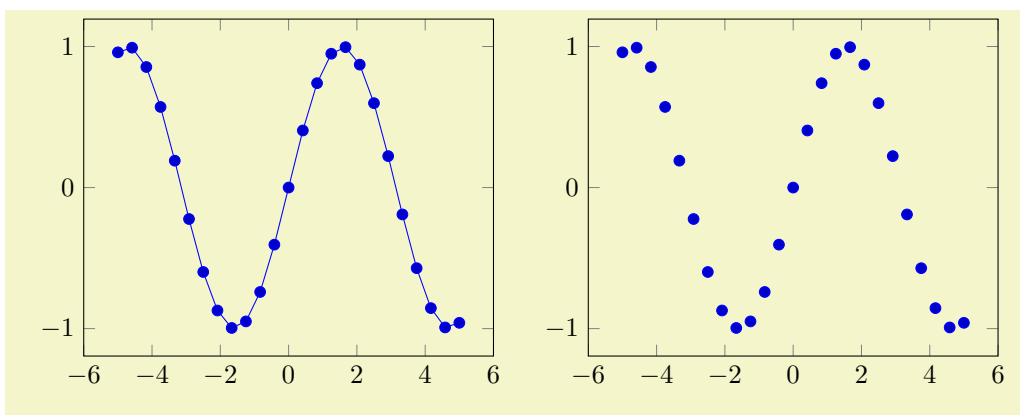
²This floating point unit is available as TikZ library as part of TikZ.

- Some of the coordinate input routines use the powerful `\pgfmathparse` feature of PGF to read their coordinates, among them `plot coordinates`, `plot expression` and `plot table`. This allows to use mathematical expressions as coordinates which will be evaluated using the floating point routines (this applies to logarithmic and linear scales).
- PGFPLOTS automatically computes missing axis limits. The automatic computation of axis limits works as follows:
 1. Every coordinate will be checked. Care has been taken to avoid T_EX's limited numerical capabilities.
 2. Since more than one `\addplot` command may be used inside of `\begin{axis}...\end{axis}`, all drawing commands will be postponed until `\end{axis}`.

`\addplot+[<options>] ...;`

Does the same like `\addplot[<options>] ...;` except that `<options>` are *appended* to the arguments which would have been taken for `\addplot ...` (the element of the default list).

Thus, you can combine `cycle list` and `<options>`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot {sin(deg(x))};
\end{axis}
\end{tikzpicture}

\begin{tikzpicture}
\begin{axis}
\addplot+[only marks] {sin(deg(x))};
\end{axis}
\end{tikzpicture}
```

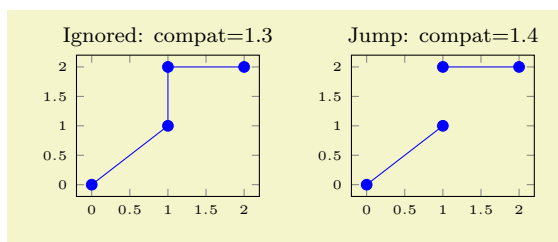
The distinction is as follows: `\addplot ...` (without options) lets PGFPLOTS select colors, markers and linestyles automatically (using `cycle list`). The variant `\addplot+[<option>] ...` will use the same automatically determined styles, but in addition it uses `<options>`. Finally, `\addplot[<options>]` (without the +) uses only the manually provided `<options>`.

`/pgfplots/empty line=auto|none|scanline|jump` (initially `auto`)

Controls how empty lines in the input coordinate stream are to be interpreted. You should ensure that you have `\pgfplotsset{compat=1.4}` or newer in your preamble and leave this key at its default `empty line=auto`.

Empty lines can occur between the coordinates of `\addplot coordinates` or successive rows of the data file input routines `\addplot table` (and `\addplot file`).

The choice `auto` checks if the current plot type is `mesh` or `surf`. If so, it uses `scanline`. If the current plot type is some other plot type (like a standard line plot), it uses `jump`. Note that the value `auto` for non-mesh plots results in `none` if `compat=1.3` or older is used. In other words: you have to write `\pgfplotsset{compat=1.4}` or newer to let PGFPLOTS interpret empty lines as `jump` in standard line plots:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[tiny,
    title={Ignored: compat=1.3},
    compat=1.3]
    \addplot table {
      A B
      0 0
      1 1

      1 2
      2 2
    };
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[tiny,
    title={Jump: compat=1.4},
    compat=1.4]
    \addplot table {
      A B
      0 0
      1 1

      1 2
      2 2
    };
  \end{axis}
\end{tikzpicture}
```

The choice `scanline` is only useful for `mesh` and `surf`: it is used to decode a matrix from a coordinate stream. If an empty line occurs once every N data points, the “scanline” length is N . This information, together with `mesh/ordering` and the total number of points, allows to deduce the matrix size. However, the distance between empty lines has to be consistent: if the first two empty lines have a distance of 2 and the next comes after 5, PGFPLOTS will ignore the information and will expect explicit matrix sizes using `mesh/rows` and/or `mesh/cols`. The choice `scanline` is ignored if `mesh input=patches`. It has no effect for other plot types.

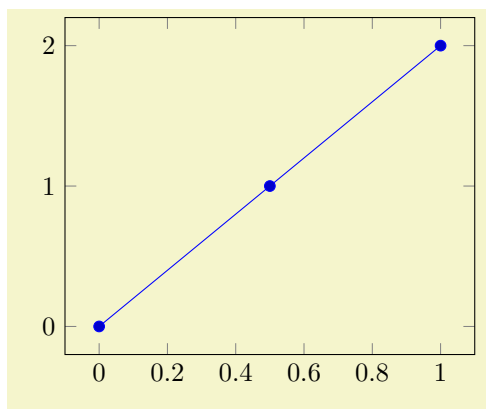
The choice `none` will silently discard any empty line in the input stream.

The choice `jump` tells PGFPLOTS to generate a jump.

4.2.1 Coordinate Lists

```
\addplot coordinates {<coordinate list>;}
\addplot[<options>] coordinates {<coordinate list>} <trailing path commands>;
\addplot3 ...
```

The ‘`\addplot coordinates`’ command is like that provided by TikZ and reads its input data from a sequence of point coordinates, encapsulated in round braces.

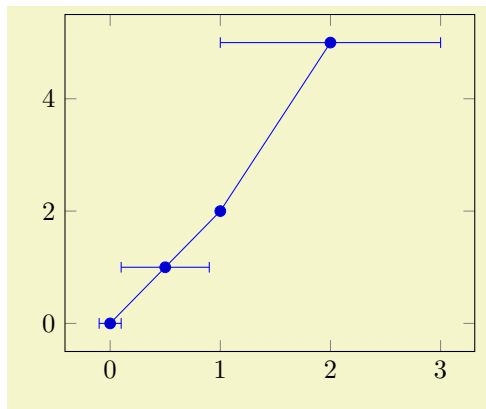


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot coordinates {
      (0,0)
      (0.5,1)
      (1,2)
    };
  \end{axis}
\end{tikzpicture}
```

You should only use this input format if you have short diagrams and you want to provide mathematical expressions for each of the involved coordinates. Any data plots are typically easier to handle using a table format and `\addplot table`.

The coordinates can be numbers, but they can also contain mathematical expressions like `\sin(0.5)` or `\h*8` (assuming you defined `\h` somewhere). However, expressions which involve round braces need to be encapsulated in a further set of curly braces, for example `(\sin(0.5)),\{\cos(0.1)\}`.

You can also supply error coordinates (reliability bounds) if you are interested in error bars. Simply append the error coordinates with `'+- (\langle ex,ey \rangle)'` (or `+- (\langle ex,ey,ez \rangle)`) to the associated coordinate:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[error bars/.cd,x dir=both,x explicit]
coordinates {
(0,0) +- (0.1,0)
(0.5,1) +- (0.4,0.2)
(1,2)
(2,5) +- (1,0.1)
};
\end{axis}
\end{tikzpicture}
```

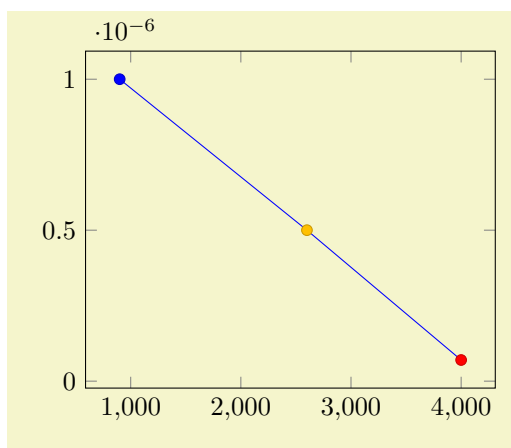
or

```
\addplot coordinates {
(900,1e-6) +- (0.1,0.2)
(2600,5e-7) +- (0.2,0.5)
(4000,7e-8) +- (0.1,0.01)
};
```

These error coordinates are only used in case of error bars, see Section 4.11. You will also need to configure whether these values denote absolute or relative errors.

The coordinates as such can be numbers as `+5`, `-1.2345e3`, `35.0e2`, `0.00000123` or `1e2345e-8`. They are not limited to T_EX's precision.

Furthermore, `coordinates` allows to define “meta data” for each coordinate. The interpretation of meta data depends on the visualization technique: for scatter plots, meta data can be used to define colors or style associations for every point (see page 81 for an example). Meta data (if any) must be provided after the coordinates and after error bar bounds (if any) in square brackets:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[scatter,scatter src=explicit] coordinates {
(900,1e-6) [1]
(2600,5e-7) [2]
(4000,7e-8) [3]
};
\end{axis}
\end{tikzpicture}
```

Please refer to the documentation of `point meta` on page 156 for more information about per point meta data.

The coordinate stream can contain `empty lines` to tell PGFPLOTS that the function has jumps. To use it, simply insert an empty line (and ensure that you have `\pgfplotsset{compat=1.4}` or newer in your preamble). See the documentation of `empty line` for details.

`/pgfplots/plot coordinates/math parser=true|false` (initially `true`)

Allows to turn off support for mathematical expressions in every coordinate inside of `plot coordinates`. This might be necessary if coordinates are not in numerical form (or if you'd like to improve speed).

It is necessary to disable `plot coordinates/math parser` if you use some sort of symbolic transformations (i.e. text coordinates).

4.2.2 Reading Coordinates From Tables

```
\addplot table [<column selection>]{<file or inline table>};
\addplot[<options>] table [<column selection>]{<file or inline table>} <trailing path commands>;
\addplot3 ...
```

This input method is the main input format for any data-based function. It accepts either a file containing data or an inline table provided in curly braces.

Given a data file like

dof	L2	Lmax	maxlevel
5	8.31160034e-02	1.80007647e-01	2
17	2.54685628e-02	3.75580565e-02	3
49	7.40715288e-03	1.49212716e-02	4
129	2.10192154e-03	4.23330523e-03	5
321	5.87352989e-04	1.30668515e-03	6
769	1.62269942e-04	3.88658098e-04	7
1793	4.44248889e-05	1.12651668e-04	8
4097	1.20714122e-05	3.20339285e-05	9
9217	3.26101452e-06	8.97617707e-06	10

one may want to plot ‘dof’ versus ‘L2’ or ‘dof’ versus ‘Lmax’. This can be done by

```
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel=Dof,
  ylabel=$L_2$ error]
\addplot table[x=dof,y=L2] {datafile.dat};
\end{loglogaxis}
\end{tikzpicture}
```

or, for the Lmax column, using

```
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel=Dof,
  ylabel=$L_{\infty}$ error]
\addplot table[x=dof,y=Lmax] {datafile.dat};
\end{loglogaxis}
\end{tikzpicture}
```

It is also possible to provide the data inline, i.e. directly as argument in curly braces:

```
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel=Dof,
  ylabel=$L_{\infty}$ error]
\addplot table[x=dof,y=Lmax] {
  dof      L2      Lmax      maxlevel
  5         8.31160034e-02  1.80007647e-01  2
  17        2.54685628e-02  3.75580565e-02  3
  49        7.40715288e-03  1.49212716e-02  4
  129       2.10192154e-03  4.23330523e-03  5
  321       5.87352989e-04  1.30668515e-03  6
  769       1.62269942e-04  3.88658098e-04  7
  1793      4.44248889e-05  1.12651668e-04  8
  4097      1.20714122e-05  3.20339285e-05  9
  9217      3.26101452e-06  8.97617707e-06  10
};
\end{loglogaxis}
\end{tikzpicture}
```

Inline table may be convenient together with ‘\\’ and `row sep=\\`, see below for more information.

Alternatively, you can load the table *once* into an internal structure and use it *multiple* times³:

³In earlier versions, there was an addition keyword ‘from’ before the argument like `\addplot table from {\loadedtable}`. This keyword is still accepted, but no longer required.

```

\pgfplotstableread{datafile.dat}\loadedtable % use any custom name in place of '\loadedtable'
...
\addplot table[x=dof,y=L2] {\loadedtable};
...
\addplot table[x=dof,y=Lmax] {\loadedtable};
...

```

I am not really sure how much time can be saved, but it works anyway. The `\pgfplotstableread` command is documented in all detail in the manual for `PGFPLOTS`TABLE. As a rule of thumb, decide as follows:

1. If tables contain few rows and many columns, the $\langle\text{macro}\rangle$ framework will be more efficient.
2. If tables contain more than 200 data points (rows), you should always use file input (and reload if necessary).

Occasionally, it might be handy to load a table, apply manual preparation steps (for example `\pgfplotstabletranspose`) and plot the result tables afterwards.

If you do prefer to access columns by column indices instead of column names (or your tables do not have column names), you can also use

```

\addplot table[x index=2,y index=3] {datafile.dat};
\addplot table[x=dof,y index=2] {datafile.dat};

```

Summary and remarks:

- Use `\addplot table[x={\langle column name \rangle},y={\langle column name \rangle}]` to access column names. Those names are case sensitive and need to exist.
- Use `\addplot table[x index={\langle column index \rangle},y index={\langle column index \rangle}]` to access column indices. Indexing starts with 0. You may also use an index for x and a column name for y .
- Use `\addplot table[x expr=\coordindex,y={\langle column name \rangle}]` to plot the coordinate index versus some y data.
- Use `\addplot table[header=false] {\langle file name \rangle}` if your input file has no column names. Otherwise, the first non-comment line is checked for column names: if all entries are numbers, they are treated as numerical data; if one of them is not a number, all are treated as column names.
- It is possible to read error coordinates from tables as well. Simply add options '`x error`', '`y error`' or '`x error index`'/'`y error index`' to $\langle\text{source columns}\rangle$. See Section 4.11 for details about error bars.
- It is possible to read per point meta data (usable in `scatter src`, see page 79) as has been discussed for `plot coordinates` and `plot file` above. The meta data column can be provided using the `meta` key (or the `meta index` key).
- Use `\addplot table[\langle source columns \rangle] {\langle macro \rangle}` to use a pre-read table. Tables can be read using

```

\pgfplotstableread{datafile.dat}\macroname.

```

If you like, you can insert the optional keyword '`from`' before `\macroname`.

- The accepted input format of tables is as follows:
 - Rows are separated by new line characters.
Alternatively, you can use `row sep=\\` which enables '`\\`' as row separator. This might become necessary for inline table data, more precisely: if newline characters have been converted to white spaces by $\text{\TeX}$'s character processing before `PGFPLOTS` had a chance to see them. This happens if inline tables are provided inside of macros. Use `row sep=\\` and separate the rows by '`\\`' if you experience such problems.
 - Columns are usually separated by white spaces (at least one tab or space).
If you need other column separation characters, you can use the `col sep=space|tab|comma|colon|semicolon|braces|&|ampersand` option documented in all detail in the manual for `PGFPLOTS`TABLE which is part of `PGFPLOTS`.
 - Any line starting with '`#`' or '`%`' is ignored.

- The first line will be checked if it contains numerical data. If there is a column in the first line which is *no* number, the complete line is considered to be a header which contains column names. Otherwise it belongs to the numerical data and you need to access column indices instead of names.
- There is future support for a second header line which must start with ‘`$flags`’. Currently, such a line is ignored. It may be used to provide number formatting hints like precision and number format if those tables shall be typeset using `\pgfplotstabletypeset` (see the manual for `PGFPLOTS`TABLE).
- The accepted number format is the same as for ‘`plot coordinates`’, see above.
- If you omit column selectors, the default is to plot the first column against the second. That means `plot table` does exactly the same job as `plot file` for this case.
- If you need unbalanced columns, simply use `nan` as “empty cell” placeholder. These coordinates will be skipped in plots.
- It is also possible to use **mathematical expressions** together with ‘`plot table`’. This is documented in all detail in Section 4.2.4, but the key idea is to use one of `x expr`, `y expr`, `z expr` or `meta expr` as in ‘`plot table[x expr=\thisrow{maxlevel}+3,y=L2]`’.
- The `PGFPLOTS`TABLE package coming with `PGFPLOTS` has a the feature “Postprocessing Data in New Columns” (see its manual).
This allows to compute new columns based on existing data. One of these features is `create col/linear regression` (described in Section 4.24).
You can invoke all the `create col/⟨key name⟩` features directly in `\addplot table` using `\addplot table[x={create col/⟨key name⟩=⟨arguments⟩}]`.
In this case, a new column will be created using the functionality of `⟨key name⟩`. This column generation is described in all detail in `PGFPLOTS`TABLE. Finally, the resulting data is available as `x` coordinate (the same holds for `y=` or `z=`).
One application (with several examples how to use this syntax) is line fitting with `create col/linear regression`, see Section 4.24 for details.
- The table can contain `empty lines` to tell `PGFPLOTS` that the function has jumps. To use it, simply insert an empty line (and ensure that you have `\pgfplotsset{compat=1.4}` or newer in your preamble). See the documentation of `empty line` for details.
- Technical note: every opened file will be protocolled into your log file.

Keys To Configure Table Input

The following list of keys allow different methods to select input data or different input formats. Note that the common prefix ‘`table/`’ can be omitted if these keys are set after `\addplot table[⟨options⟩]`. The `/pgfplots/` prefix can always be omitted when used in a `PGFPLOTS` method.

`/pgfplots/table/header=true|false` (initially `true`)

Allows to disable header identification for `plot table`. See above.

```
/pgfplots/table/x={⟨column name⟩}
/pgfplots/table/y={⟨column name⟩}
/pgfplots/table/z={⟨column name⟩}
/pgfplots/table/x index={⟨column index⟩}
/pgfplots/table/y index={⟨column index⟩}
/pgfplots/table/z index={⟨column index⟩}
```

These keys define the sources for `plot table`. If both column names and column indices are given, column names are preferred. Column indexing starts with 0. The initial setting is to use `x index=0` and `y index=1`.

Please note that column *aliases* will be considered if unknown column names are used. Please refer to the manual of `PGFPLOTS`TABLE which comes with this package.

```
/pgfplots/table/x expr={⟨expression⟩}
/pgfplots/table/y expr={⟨expression⟩}
/pgfplots/table/z expr={⟨expression⟩}
```


`/pgfplots/table/meta expr={\langle expression \rangle}`

These keys allow to combine the mathematical expression parser with file input. They are listed here to complete the list of table keys, but they are described in all detail in Section 4.2.4.

The key idea is to provide an $\langle expression \rangle$ which depends on table data (possibly on all columns in one row). Only data within the same row can be used where columns are referenced with `\thisrow{\langle column name \rangle}` or `\thisrowno{\langle column index \rangle}`.

Please refer to Section 4.2.4 for details.

```
/pgfplots/table/x error={\langle column name \rangle}
/pgfplots/table/y error={\langle column name \rangle}
/pgfplots/table/z error={\langle column name \rangle}
/pgfplots/table/x error index={\langle column index \rangle}
/pgfplots/table/y error index={\langle column index \rangle}
/pgfplots/table/z error index={\langle column index \rangle}
/pgfplots/table/x error expr={\langle math expression \rangle}
/pgfplots/table/y error expr={\langle math expression \rangle}
/pgfplots/table/z error expr={\langle math expression \rangle}
```

These keys define input sources for error bars with explicit error values.

The `x error` method provides an input column name (or alias), the `x error index` method provides input column *indices* and `x error expr` works just as `table/x expr`: it allows arbitrary mathematical expressions which may depend on any number of table columns using `\thisrow{\langle col name \rangle}`.

Please see Section 4.11 for details about the usage of error bars.

```
/pgfplots/table/meta={\langle column name \rangle}
/pgfplots/table/meta index={\langle column index \rangle}
```

These keys define input sources for per point meta data. Please see page 79 for details about meta data or the documentation for `plot coordinates` and `plot file` for further information.

```
/pgfplots/table/row sep=newline|\\ (initially newline)
```

Configures the character to separate rows.

The choice `newline` uses the end of line as it appears in the table data (i.e. the input file or any inline table data).

The choice `\\` uses ‘\\’ to indicate the end of a row.

Note that `newline` for inline table data is “fragile”: you can’t provide such data inside of $\TeX$ macros (this does not apply to input files). Whenever you experience problems, proceed as follows:

1. First possibility: call `\pgfplotstableread{\langle data \rangle}\yourmacro` *outside* of any macro declaration.
2. Use `row sep=\\`.

The same applies if you experience problems with inline data and special `col sep` choices (like `col sep=tab`).

The reasons for such problems is that $\TeX$ scans the macro bodies and replaces newlines by white spaces. It does other substitutions of this sort as well, and these substitutions can’t be undone (maybe not even found).

```
/pgfplots/table/col sep=space|tab|comma|semicolon|colon|braces|&|ampersand (initially space)
```

Allows to choose column separators for `plot table`. Please refer to the manual of `PGFplotsTable` which comes with this package for details about `col sep`.

```
/pgfplots/table/read completely={\langle auto,true,false \rangle} (initially auto)
```

Allows to customize `\addplot table{\langle file name \rangle}` such that it always reads the entire table into memory.

This key has just one purpose, namely to create postprocessing columns on-the-fly and to plot those columns afterwards. This “lazy evaluation” which creates missing columns on-the-fly is documented in the `PGFplotsTable` manual (in section “Postprocessing Data in New Columns”).

The initial configuration `auto` checks whether one of the keys `table/x`, `table/y`, `table/z` or `table/meta` contains a `create on use` column. If so, it enables `read completely`, otherwise it prefers to load the file in the normal way.

Attention: Usually, `\addplot table` only picks required entries, requiring linear runtime complexity. As soon as `read completely` is activated, tables are loaded completely into memory. Due to datastructures issues (“macro append runtime”), the runtime complexity for `read completely` is $O(N^2)$ where N is the number of rows. Thus: use this feature only for “small” tables⁴.

`/pgfplots/table/ignore chars={⟨comma-separated-list⟩}` (initially empty)

Allows to silently remove a set of single characters from input files. The characters are separated by commas. The documentation for this command, including cases like ‘`\%`’, ‘`\#`’, ‘`\`’ or binary character codes like ‘`\^ff`’ can be found in the manual for `PGFPLOTS`TABLE.

This setting applies to `\addplot file` as well.

`/pgfplots/table/white space chars={⟨comma-separated-list⟩}` (initially empty)

Allows to define a list of single characters which are actually treated like white spaces (in addition to tabs and spaces). Please refer to the manual of `PGFPLOTS`TABLE for details.

This setting applies to `\addplot file` as well.

`/pgfplots/table/comment chars={⟨comma-separated-list⟩}` (initially empty)

Allows to add one or more *additional* comment characters. Each of these characters has a similar effect as the `#` character, i.e. all following characters of that particular input line are skipped.

For example, `comment chars=!` uses ‘`!`’ as additional comment character (which allows to parse Touchstone files).

Please refer to the manual of `PGFPLOTS`TABLE for details.

`/pgfplots/table/skip first n={⟨integer⟩}` (initially 0)

Allows to skip the first `⟨integer⟩` lines of an input file. The lines will not be processed.

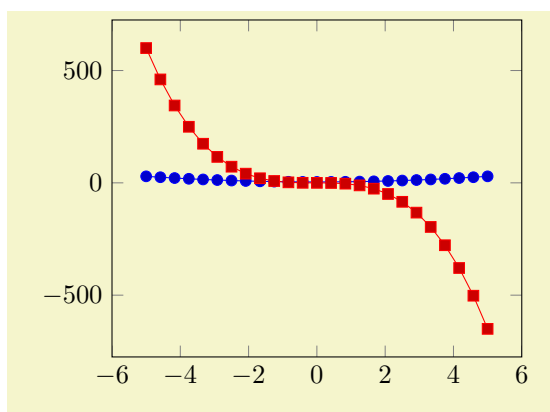
Please refer to the manual of `PGFPLOTS`TABLE for details.

4.2.3 Computing Coordinates with Mathematical Expressions

```
\addplot {⟨math expression⟩} ;
\addplot[⟨options⟩] {⟨math expression⟩} ⟨trailing path commands⟩;
\addplot3 ...
```

This input method allows to provide mathematical expressions which will be sampled. But unlike `plot gnuplot`, the expressions are evaluated using the math parser of PGF, no external program is required.

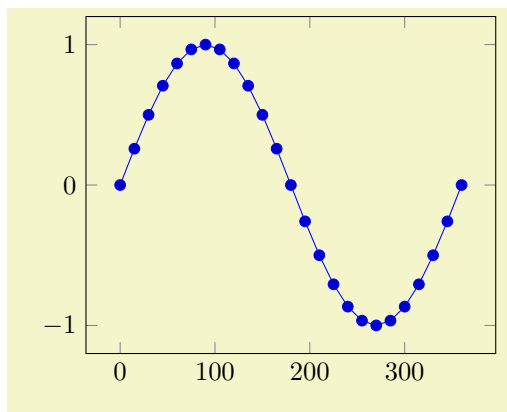
Plot expression samples x from the interval $[a, b]$ where a and b are specified with the `domain` key. The number of samples can be configured with `samples=⟨N⟩` as for `plot gnuplot`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot {x^2 + 4};
\addplot {-5*x^3 - x^2};
\end{axis}
\end{tikzpicture}
```

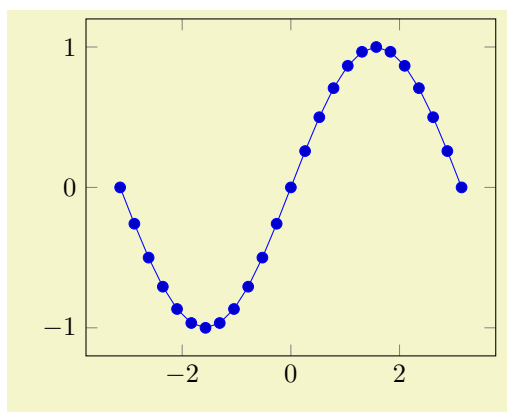
Please note that PGF’s math parser uses degrees for trigonometric functions:

⁴This remark might be deprecated; many of the slow routines have been optimized in the meantime to have at least pseudo-linear runtime.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[domain=0:360]
{sin(x)};
\end{axis}
\end{tikzpicture}
```

If you want to use radians, use



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[domain=-pi:pi]
{sin(deg(x))};
\end{axis}
\end{tikzpicture}
```

to convert the radians to degrees. The plot expression parser also accepts some more options like `samples at={⟨coordinate list⟩}` or `domain=⟨first⟩:⟨last⟩` which are described below.

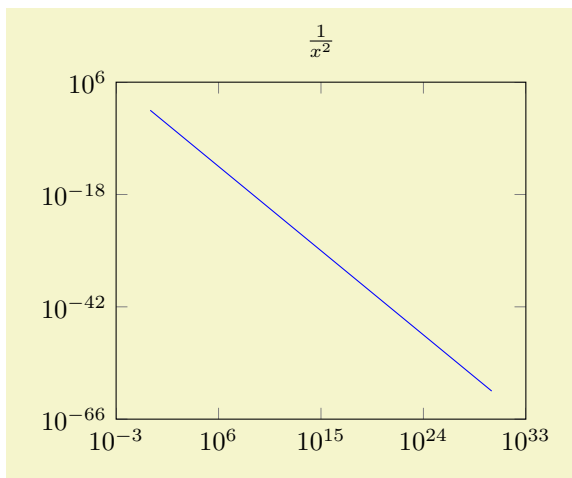
Remarks

1. What really goes on is a loop which assigns the current sample coordinate to the macro `\x`. PGFLOTS defines a math constant `x` which always has the same value as `\x`.
In short: it is the same whether you write `\x` or just `x` inside of math expressions.
The variable name can be customized using `variable=\t` (the backslash is necessary!). Then, `t` will be the same as `\t`.
2. The complete set of math expressions can be found in the PGF manual. The most important mathematical operations are `+`, `-`, `*`, `/`, `abs`, `round`, `floor`, `mod`, `<`, `>`, `max`, `min`, `sin`, `cos`, `tan`, `deg` (conversion from radians to degrees), `rad` (conversion from degrees to radians), `atan`, `asin`, `acos`, `cot`, `sec`, `cosec`, `exp`, `ln`, `sqrt`, the constants `pi` and `e`, `^` (power operation), `factorial`⁵, `rand` (random between -1 and 1), `rnd` (random between 0 and 1), number format conversions `hex`, `Hex`, `oct`, `bin` and some more. The math parser has been written by Mark Wibrow and Till Tantau [5], the FPU routines have been developed as part of PGFLOTS. The documentation for both parts can be found in [5].
Please note, however, that trigonometric functions are defined in degrees. The character `^` is used for exponentiation (not `**` as in gnuplot).
3. If the x axis is logarithmic, samples will be drawn logarithmically.
4. Please note that plot expression does not allow separate per point meta data (color data). You can, of course, use `point meta=f(x)` or `point meta=x`.

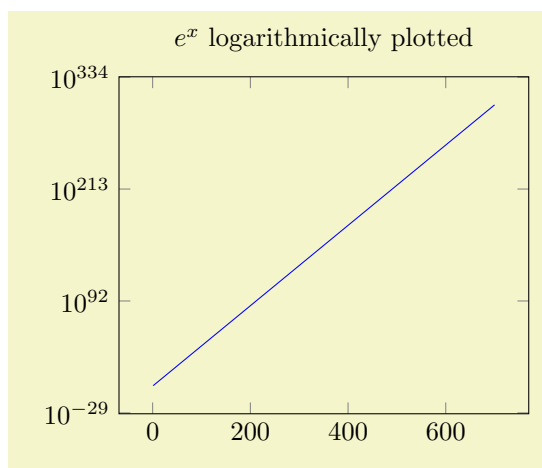
About the precision and number range: Starting with version 1.2, `plot expression` uses a floating point unit. The FPU provides the full data range of scientific computing with a relative precision between 10^{-4} and 10^{-6} . The `/pgf/fpu` key provides some more details.

⁵Starting with PGF versions newer than 2.00, you can use the postfix operator `!` instead of `factorial`.

In case the `fpu` does not provide the desired mathematical function or is too slow⁶, you should consider using the `plot gnuplot` method which invokes the external, freely available program `gnuplot` as desktop calculator.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    title={\frac{1}{x^2}}
    \addplot[blue,domain=1:1e30]
      {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{semilogyaxis}[
    title={e^x logarithmically plotted}
    \addplot[blue,domain=1:700]
      {exp(x)};
  \end{semilogyaxis}
\end{tikzpicture}
```

```
\addplot expression {\math expr};
\addplot[<options>] expression {\math expr} <trailing path commands>;
\addplot3 ...
```

The syntax

```
\addplot {\math expression};
```

as short-hand equivalent for

```
\addplot expression {\math expression};
```

```
\addplot (<x expression>,<y expression>) ;
\addplot[<options>] (<x expression>,<y expression>) <trailing path commands>;
\addplot3 ...
```

A variant of `\addplot expression` which allows to provide different coordinate expressions for the x and y coordinates. This can be used to generate parametrized plots.

Please note that `\addplot (x,x^2)` is equivalent to `\addplot expression {x^2}`.

Note further that since the complete point expression is surrounded by round braces, round braces for either $\langle x \text{ expression} \rangle$ or $\langle y \text{ expression} \rangle$ need special attention. You will need to introduce curly braces additionally to allow round braces:

```
\addplot ({\math expr}, {\math expr}, {\math expr});
```

```
/pgfplots/domain= $\langle x_1 \rangle$ : $\langle x_2 \rangle$ 
```

(initially $[-5:5]$)

⁶Or in case you find a bug...

```
/pgfplots/y domain=<y1>:<y2>
/pgfplots/domain y=<y1>:<y2>
```

Sets the function's domain(s) for `plot expression` and `plot gnuplot`. Two dimensional plot expressions are defined as functions $f: [x_1, x_2] \rightarrow \mathbb{R}$ and $\langle x_1 \rangle$ and $\langle x_2 \rangle$ are set with `domain`. Three dimensional plot expressions use functions $f: [x_1, x_2] \times [y_1, y_2] \rightarrow \mathbb{R}$ and $\langle y_1 \rangle$ and $\langle y_2 \rangle$ are set with `y domain`. If `y domain` is empty, $[y_1, y_2] = [x_1, x_2]$ is assumed for three dimensional plots (see page 93 for details about three dimensional plot expressions).

The keys `y domain` and `domain y` are the same.

The `domain` key won't be used if `samples at` is specified; `samples at` has higher precedence.

Please note that `domain` is not necessarily the same as the axis limits (which are configured with the `xmin/xmax` options).

The `domain` keys are *only* relevant for `gnuplot` and `plot expression`. In case you'd like to plot only a subset of other coordinate input routines, consider using the coordinate filter `restrict x to domain`.

Remark for TikZ-users: `/pgfplots/domain` and `/tikz/domain` are independent options. Please prefer the PGFLOTS variant (i.e. provide `domain` to an axis, `\pgfplotsset` or a plot command). Since older versions also accepted something like `\begin{tikzpicture}[domain=...]`, this syntax is also accepted as long as no PGFLOTS `domain` key is set.

```
/pgfplots/samples={<number>} (initially 25)
/pgfplots/samples y={<number>}
```

Sets the number of sample points for `plot expression` and `plot gnuplot`. The `samples` key defines the number of samples used for line plots while the `samples y` key is used for mesh plots (three dimensional visualisation, see page 93 for details). If `samples y` is not set explicitly, it uses the value of `samples`.

The `samples` key won't be used if `samples at` is specified; `samples at` has higher precedence.

The same special treatment of `/tikz/samples` and `/pgfplots/samples` as for the `domain` key applies here. See above for details.

```
/pgfplots/samples at={<coordinate list>}
```

Sets the x coordinates for `plot expression` explicitly. This overrides `domain` and `samples`.

The $\langle \text{coordinate list} \rangle$ is a `\foreach` expression, that means it can contain a simple list of coordinates (comma-separated), but also complex ... expressions like⁷

```
\pgfplotsset{samples at={5e-5,7e-5,10e-5,12e-5}}
\pgfplotsset{samples at={-5,-4.5,...,5}}
\pgfplotsset{samples at={-5,-3,-1,-0.5,0,...,5}}
```

The same special treatment of `/tikz/samples at` and `/pgfplots/samples at` as for the `domain` key applies here. See above for details.

Attention: `samples at` overrides `domain`, even if `domain` has been set *after* `samples at`! Use `samples at={}` to clear $\langle \text{coordinate list} \rangle$ and re-activate `domain`.

```
/pgfplots/variable={<variable name>} (initially x)
/pgfplots/variable y={<variable name>} (initially y)
```

Defines the variables names which will be sampled in `domain` (with `variable`) and in `domain y` (with `variable y`).

The same variables are used for parametric and for non-parametric plots. Use `variable=t` to change them if you like (for `gnuplot`, there is such a distinction; see `parametric/var id`).

Technical remark: TikZ also uses the `variable` key. However, it expects a *macro* name, i.e. `\x` instead of just `x`. Both possibilities are accepted here.

⁷Unfortunately, the ... is somewhat restrictive when it comes to extended accuracy. So, if you have particularly small or large numbers (or a small distance), you have to provide a comma-separated list (or use the `domain` key).

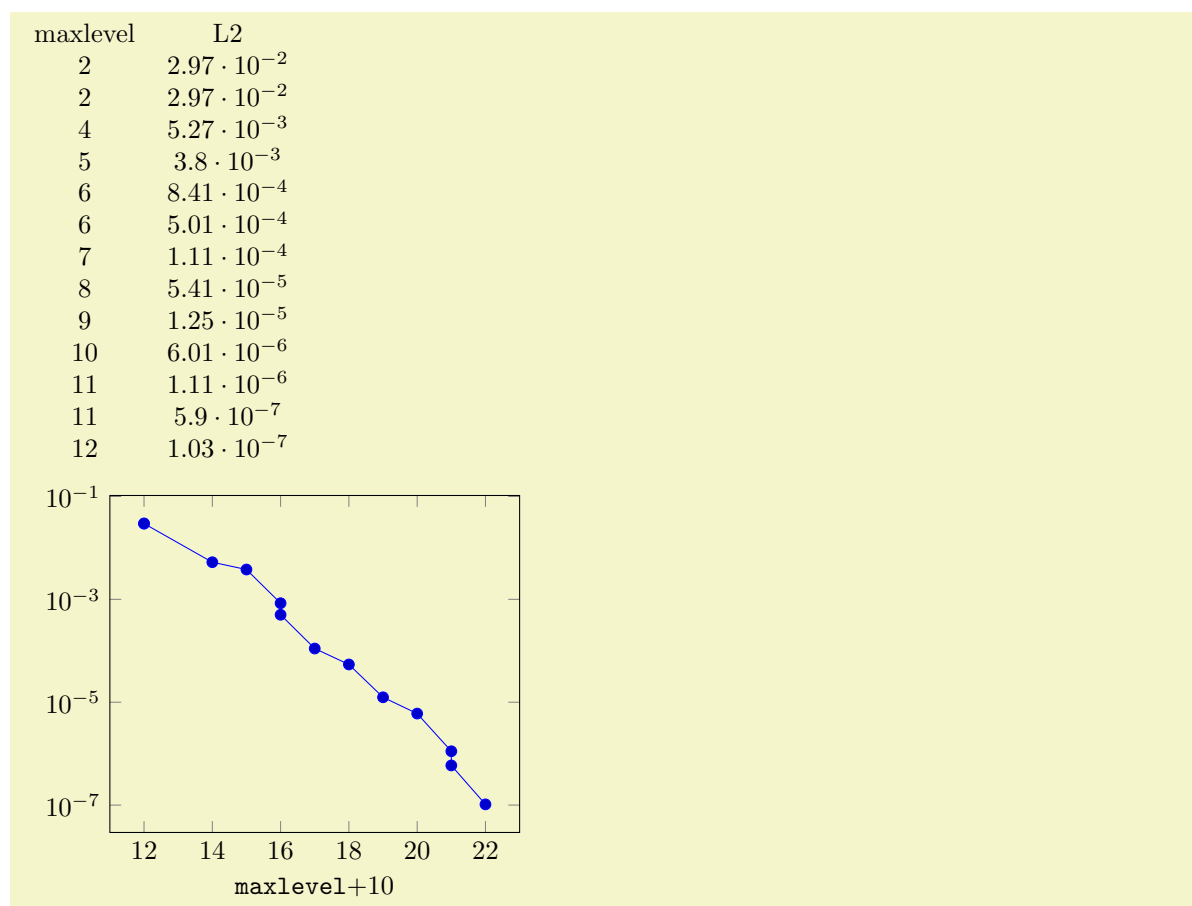
4.2.4 Mathematical Expressions And File Data

PGFPLOTS allows to combine ‘`plot table`’ and ‘`plot expression`’ to get both file input and modifications by means of mathematical expressions.

```
\addplot table [<column selection and expressions>]{<file>};
\addplot[<options>] table [<column selection and expressions>]{<file>} <trailing path commands>;
\addplot3 ...
```

Besides the already discussed possibility to provide a column selection by means of column names (`x=<name>` or `x index=<index>`, see Section 4.2.2), it is also possible to provide mathematical expressions as arguments.

Mathematical expressions are specified with `x expr=<expression>` inside of `<column selection and expressions>`. They can depend on zero, one or more columns of the input file. A column is referenced using the special command ‘`\thisrow{<column name>}`’ within `<expression>` (or `\thisrowno{<column index>}`).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotstabletypeset[columns={maxlevel,L2}]{plotdata/newexperiment1.dat}

\begin{tikzpicture}
  \begin{semilogyaxis}[
    xlabel=\texttt{maxlevel}$ + 10$
  ]
    \addplot table
      [x expr=\thisrow{maxlevel}+10, y=L2]
      {plotdata/newexperiment1.dat};
  \end{semilogyaxis}
\end{tikzpicture}
```

Besides `x expr`, there are keys `y expr`, `z expr` and `meta expr` where the latter allows to provide point meta data (which is used as `scatter src` or color data for surface plots etc.).

Inside of `<expression>`, the following macros can be used to access numerical data cells inside of the input file:

`\thisrow{⟨column name⟩}`

Yields the value of the column designated by $\langle column name \rangle$. There is no limit on the number of columns which can be part of a mathematical expression, but only values inside of the currently processed *table row* can be used.

It is possible to provide column aliases for $\langle column name \rangle$ as described in the manual of [PGFPLOTS](#)TABLE.

The argument $\langle column name \rangle$ has to denote either an existing column or one for which a column alias exists (see the manual of [PGFPLOTS](#)TABLE). If it can't be resolved, the math parser yields an "Unknown function" error message.

`\thisrowno{⟨column index⟩}`

Similar to `\thisrow`, this command yields the value of the column with index $\langle column index \rangle$ (starting with 0).

`\coordindex`

Yields the current index of the table row (starting with 0). This does *not* count header or comment lines.

`\lineno`

Yields the current line number (starting with 0). This does also count header and comment lines.

If x *index*, x and x *expr* (or the corresponding keys for y , z or *meta*) are combined, this is how they interact:

1. Column access via x has higher precedence than index access via x *index*.
2. Even if x *expr* is provided, the values of x *index* and x are still checked. Any value found using column name access or column index access is made available as `\columnx` (or `\columny`, `\columnz`, `\columnmeta`, resp.). However, the result of x *expr* is used as plot coordinate.

This allows to access the cell values identified by x or x *index* using the "pointer" `\columnx`. I am not sure if this yields any advantage, but it is possible nevertheless. If in doubt, prefer using `\thisrow{⟨column name⟩}`.

Attention: If your table has less than two rows, you may need to set x *index*=`{}`, y *index*=`{}` explicitly. This is a consequence of the fact that column name/index access is still applied even if an expression is provided.

4.2.5 Computing Coordinates with Mathematical Expressions (gnuplot)

```
\addplot gnuplot [⟨further options⟩]{⟨gnuplot code⟩};
\addplot[⟨options⟩] gnuplot [⟨further options⟩]{⟨gnuplot code⟩} ⟨trailing path commands⟩;
\addplot3 ...
```

In contrast to *plot expression*, the *plot gnuplot* command⁸ employs the external program *gnuplot* to compute coordinates. The resulting coordinates are written to a text file which will be plotted with *plot file*. PGF checks whether coordinates need to be re-generated and calls *gnuplot* whenever necessary (this is usually the case if you change the number of samples, the argument to *plot gnuplot* or the plotted domain⁹).

The differences between *plot expression* and *plot gnuplot* are:

- *plot expression* does not require any external programs and requires no additional command line options.
- *plot expression* does not produce a lot of temporary files.
- *plot gnuplot* uses radians for trigonometric functions while *plot expression* has degrees.
- *plot gnuplot* is faster.

⁸Note that *plot gnuplot* is actually a re-implementation of the *plotfunction* method known from PGF. It also invokes PGF basic layer commands.

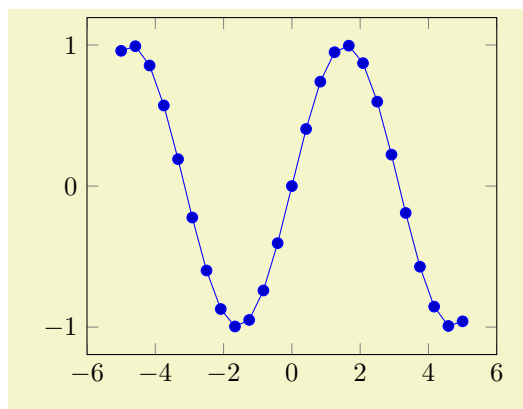
⁹Please note that PGFPLOTS produces slightly different files than TikZ when used with *plot gnuplot* (it configures high precision output). You should use different *id* for PGFPLOTS and TikZ to avoid conflicts in such a case.

- `plot gnuplot` has a larger mathematical library.
- `plot gnuplot` has a higher accuracy. However, starting with version 1.2, this is no longer a great problem. The new floating point unit for $\text{\TeX}$ provides reasonable accuracy and the same data range as `gnuplot`.

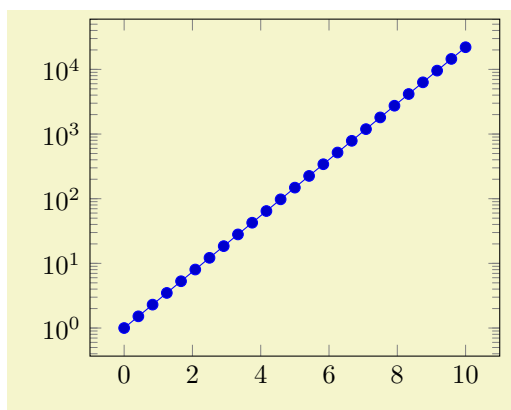
Since system calls are a potential danger, they need to be enabled explicitly using command line options, for example

```
pdflatex -shell-escape filename.tex.
```

Sometimes it is called `shell-escape` or `enable-write18`. Sometimes one needs two hyphens – that all depends on your $\text{\TeX}$ distribution.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot
    gnuplot[id=sin]{sin(x)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}
\addplot gnuplot
    [id=exp,domain=0:10]{exp(x)};
\end{semilogyaxis}
\end{tikzpicture}
```

The $\langle options \rangle$ determine the appearance of the plotted function; these parameters also affect the legend. There is also a set of options which are specific to the `gnuplot` interface. These options are described in all detail in [5, section 18.6]. A short summary is shown below.

Some remarks:

- The independent variable for one-dimensional plots can be changed with the `variable` option, just as for `plot expression`. Similarly, the second variable for two dimensional plots can be changed with `variable y`.
For `parametric` plots, the variable names need to be adjusted with `parametric/var 1d` and `parametric/var 2d` (since `gnuplot` uses `t` and `u,v` as initial values for `parametric` plots).
- Please note that `plot gnuplot` does not allow separate per point meta data (color data for each coordinate). You can, however, use `point meta=f(x)` or `point meta=x`.
- The generated output file name can be customized with `id`, see below.

Please refer to [5, section 18.6] for more details about `plot function` and the `gnuplot` interaction.

```
\addplot function {\gnuplot code};
\addplot[ $\langle options \rangle$ ] function {\gnuplot code}  $\langle trailing path commands \rangle$ ;
```

`\addplot3 ...`

Use

`\addplot function {\gnuplot code};`

as alias for

`\addplot gnuplot {\gnuplot code};`

`/pgfplots/translate gnuplot=true|false` (initially true)

Enables or disables automatic translation of the exponentiation operator ‘`^`’ to ‘`**`’.

This features allows to use `^` in `plot gnuplot` instead of gnuplot’s `**`.

`/pgfplots/parametric=true|false` (initially false)

Set this to true if you’d like to use parametric plots with `gnuplot`. Parametric plots use a comma separated list of expressions to make up $x(t)$, $y(t)$ for a line plot or $x(u, v)$, $y(u, v)$ $z(u, v)$ for a mesh plot (refer to the gnuplot manual for more information about its input methods for parametric plots).

`/pgfplots/parametric/var 1d={\variable name}` (initially `t`)

`/pgfplots/parametric/var 2d={\variable name,variable name}` (initially `u,v`)

Allows to change the dummy variables used by `parametric gnuplot` plots. The initial setting is the one of `gnuplot`: to use the dummy variable ‘`t`’ for parametric line plots and ‘`u,v`’ for parametric mesh plots.

These keys are quite the same as `variable` and `variable y`, only for parametric plots. If you like to change variables for non-parametric plots, use `variable` and/or `variable y`.

In case you don’t want the distinction between parametric and non-parametric plots, use

`\pgfplotsset{parametric/var 1d=,parametric/var 2d=}`.

`/tikz/id={\unique string identifier}`

A unique identifier for the current plot. It is used to generate temporary filenames for `gnuplot` output.

`/tikz/prefix={\file name prefix}`

A common path prefix for temporary filenames (see [5, section 18.6] for details).

`/tikz/raw gnuplot` (no value)

Disables the use of `samples` and `domain`.

4.2.6 Computing Coordinates with External Programs (shell)

`\addplot shell [\further options]{\shell commands};`

`\addplot[\options] shell [\further options]{\shell commands} \trailing path commands;`

`\addplot3 ...`

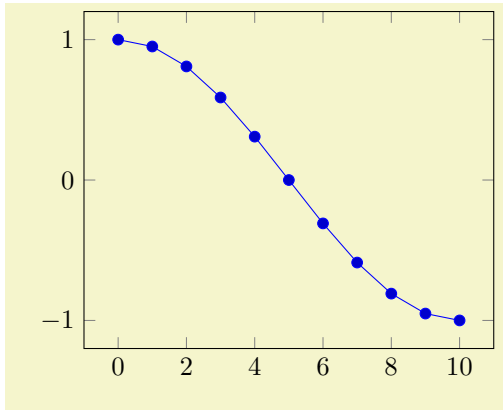
An extension by Stefan Tibus

In contrast to `plot gnuplot`, the `plot shell` command allows execution of arbitrary shell commands to compute coordinates. The resulting coordinates are written to a text file which will be plotted with `plot file`. PGF checks whether coordinates need to be re-generated and executes the `\shell commands` whenever necessary.

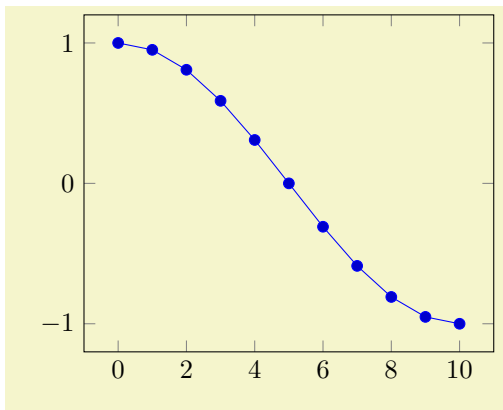
Since system calls are a potential danger, they need to be enabled explicitly using command line options, for example

```
pdflatex -shell-escape filename.tex.
```

Sometimes it is called `shell-escape` or `enable-write18`. Sometimes one needs two slashes – that all depends on your T_EX distribution.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot
    shell[prefix=pgfshell_,id=cos]{awk 'BEGIN{
        pi=3.14159; N=10;
        for(i=0;i<=N;i++) print i,cos(i/N*pi);}'};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[prefix=pgfshell_,id=replot]
    shell{cat pgfshell_cos.out};
    % just reprint the result from above
\end{axis}
\end{tikzpicture}
```

The *options* determine the appearance of the plotted function; these parameters also affect the legend. There is also a set of options which are specific to the gnuplot and the shell interface. These options are described in all detail in [5, section 19.6]. A short summary is shown below.

`/tikz/id={unique string identifier}`

A unique identifier for the current plot. It is used to generate temporary filenames for `shell` output.

`/tikz/prefix={file name prefix}`

A common path prefix for temporary filenames (see [5, section 19.6] for details).

4.2.7 Using External Graphics as Plot Sources

```
\addplot graphics {file name};
\addplot[options] graphics {file name} trailing path commands;
\addplot3 ...
```

This plot type allows to extend the plotting capabilities of PGFLOTS beyond its own limitations. The idea is to generate the graphics as such (for example, a contour plot, a complicated shaded surface¹⁰ or a large point cluster) with an external program like Matlab (®) or `gnuplot`. The graphics, however, should *not* contain an axis or descriptions. Then, we use `\includegraphics` and a PGFLOTS axis which fits exactly on top of the imported graphics.

Of course, one could do this manually by providing proper scales and such. The operation `plot graphics` is intended so simplify this process. However the *main difficulty* is to get images with correct bounding box. Typically, you will have to adjust bounding boxes manually.

Let's start with an example: Suppose we use, for example, matlab to generate a surface plot like

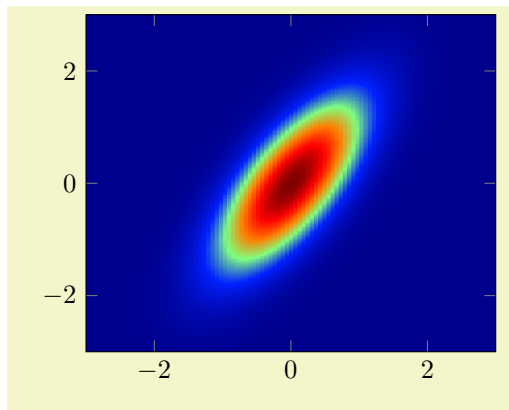
```
[X,Y] = meshgrid( linspace(-3,3,500) );
surf( X,Y, exp(-(X - Y).^2 - X.^2) );
shading flat; view(0,90); axis off;
print -dpng external1
```

¹⁰See also Section 4.5.6 for an overview of PGFLOTS methods to draw shaded surfaces.

which is then found in `external1.png`. The `surf` command of Matlab generates the surface, the following commands disable the axis descriptions, initialise the desired view and export it. Viewing the image in any image tool, we see a lot of white space around the surface – Matlab has a particular weakness in producing tight bounding boxes, as far as I know. Well, no problem: use your favorite image editor and crop the image (most image editors can do this automatically). We could use the free ImageMagick command

```
convert -trim external1.png external1.png
```

to get a tight bounding box. Then, we use



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[enlargelimits=false,axis on top]
    \addplot graphics
      [xmin=-3,xmax=3,ymin=-3,ymax=3]
      {external1};
  \end{axis}
\end{tikzpicture}
```

to load the graphics¹¹ just as if we would have drawn it with PGFPLOTS. The `axis on top` simply tells PGFPLOTS to draw the axis on top of any plots (see its description).

Please note that PGFPLOTS offers support for smaller surface plots as well which might be an option – unless the number of samples is too large. See Section 4.5.6 for details.

However, external programs have the following advantages here: they are faster, allow more complexity and provide real z buffering which is currently only simulated by PGFPLOTS. Thus, it may help to consider `plot graphics` for complicated surface plots.

Our first test was successful – and not difficult at all because graphics programs can automatically compute the bounding box. There are a couple of free tools available which can compute tight bounding boxes for `.eps` or `.pdf` graphics:

1. The free vector graphics program **inkscape** can help here. Its feature “File >> Document Properties: Fit page to selection” computes a tight bounding box around every picture element. However, some images may contain a rectangular path which is as large as the bounding box (Matlab (®) computes such `.eps` images). In this case, use the “Ungroup” method (context menu of **inkscape**) as often as necessary and remove such a path. Finally, save as `.eps`. However, **inkscape** appears to have problems with postscript fonts – it substitutes them. This doesn’t pose problems in this application because fonts shouldn’t be part of such images – the descriptions will be drawn by PGFPLOTS.
2. The tool **pdfcrop** removes surrounding whitespace in `.pdf` images and produces quite good bounding boxes.

Adjusting bounding boxes manually

In case you don’t have tools at hand to provide correct bounding boxes, you can still use $\text{\TeX}$ to set the bounding box manually. Some viewers like **gv** provide access to low-level image coordinates. The idea is to determine the number of units which need to be removed and communicate these units to `\includegraphics`.

I am aware of the following methods to determine bounding boxes manually:

inkscape I am pretty sure that **inkscape** can do it.

gv The ghost script viewer **gv** always shows the postscript units under the mouse cursor.

¹¹Please note that I don’t have a Matlab license, so I used **gnuplot** to produce an equivalent replacement graphics.

gimp The graphics program **gimp** usually shows the cursor position in pixels, but it can be configured to display postscript points (**pt**) instead.

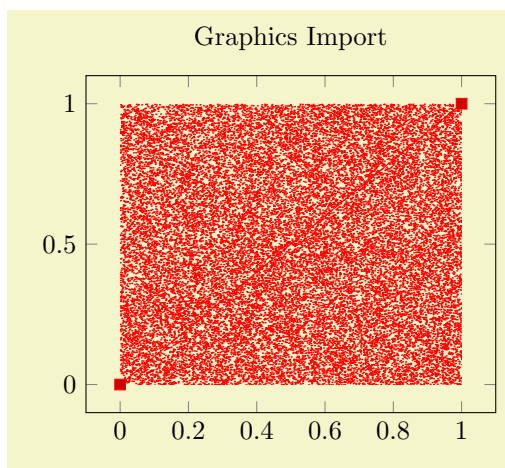
Let's follow this approach in a further example.

We use **gnuplot** to draw a (relatively stupid) example data set. The gnuplot script

```
set samples 30000
set parametric
unset border
unset xtics
unset ytics
set output "external2.eps"
set terminal postscript eps color
plot [t=0:1] rand(0),rand(0) with dots notitle lw 5
```

generates `external2.eps` with a uniform random sample of size 30000. As before, we import this scatter plot into PGFPLOTS using `plot graphics`. Again, the bounding box is too large, so we need to adjust it (**gnuplot** can do this automatically, but we do it anyway to explain the mechanisms):

Using **gv**, I determined that the bounding box needs to be shifted 12 units to the left and 9 down. Furthermore, the right end is 12 units too far off and the top area has about 8 units space wasted. This can be provided to the `trim` option of `\includegraphics`, and we use `clip` to clip the rest away:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[axis on top,title=Graphics Import]
    \addplot graphics
      [xmin=0,xmax=1,ymin=0,ymax=1,
      % trim=left bottom right top
      \includegraphics={trim=12 9 12 8,clip}}
      {external2};
    \addplot coordinates {(0,0) (1,1)};
  \end{axis}
\end{tikzpicture}
```

So, `plot graphics` takes a graphics file along with options which can be passed to `\includegraphics`. Furthermore, it provides the information how to embed the graphics into an axis. The axis can contain any other `\addplot` command as well and will be resized properly.

Details about `plot graphics`:

The loaded graphics file is drawn with

```
\node[/pgfplots/plot graphics/node] {\includegraphics[<options>]{<file name>}};
```

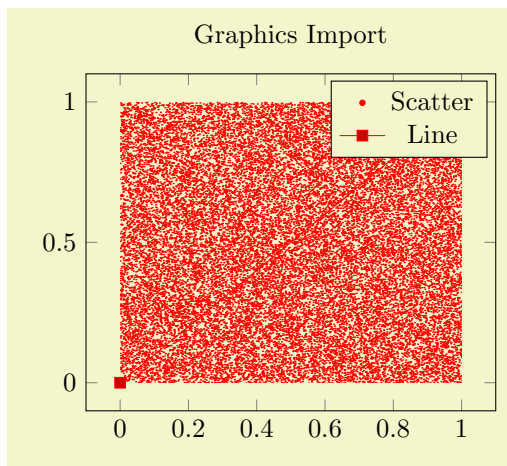
where the `node` style is a configurable style. The node is placed at the coordinate designated by `xmin`, `ymin`.

The `<options>` are any arguments provided to the `\includegraphics` key (see below) and `width` and `height` determined such that the graphics fits exactly into the rectangle denoted by the `xmin`, `ymin` and `xmax`, `ymax` coordinates.

The scaling will thus ignore the aspect ratio of the external image and prefer the one used by PGFPLOTS. You will need to provide `width` and `height` to the PGFPLOTS axis to change its scaling. Use the `scale only axis` key in such a case.

Legends in `plot graphics`:

A legend for `plot graphics` uses the current plot handler and the current plot `mark`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis on top,title=Graphics Import]
% provide options for the legend:
\addplot[red,only marks,mark=*,mark size=1pt]
graphics
[xmin=0,xmax=1,ymin=0,ymax=1,
% trim=left bottom right top
includegraphics={trim=12 9 12 8,clip}]
{external2};

\addplot coordinates {(0,0) (1,1)};

\legend{Scatter,Line}
\end{axis}
\end{tikzpicture}
```

4.2.8 Keys To Configure Plot Graphics

The following list of keys configure `\addplot graphics`. Note that the common prefix `'plot graphics/'` can be omitted if these keys are set after `\addplot graphics[<options>]`. The `/pgfplots/` prefix can always be omitted when used in a PGFLOTS method.

```
/pgfplots/plot graphics/xmin={<coordinate>}
/pgfplots/plot graphics/ymin={<coordinate>}
/pgfplots/plot graphics/zmin={<coordinate>}
/pgfplots/plot graphics/xmax={<coordinate>}
/pgfplots/plot graphics/ymax={<coordinate>}
/pgfplots/plot graphics/zmax={<coordinate>}
```

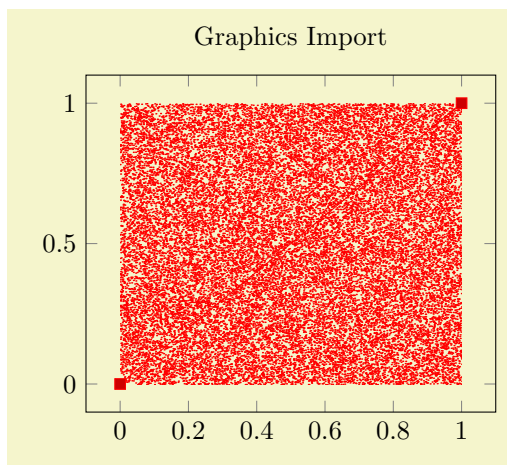
These keys are required for `plot graphics` and provide information about the external data range. The graphics will be squeezed between these coordinates. The arguments are axis coordinates; they are only useful if you provide each of them.

Alternatively, you can also use the `plot graphics/points` feature to provide the external data range, see below.

```
/pgfplots/plot graphics/points={<list of coordinates>} (initially empty)
```

This key also allows to provide the external data range. It constitutes an alternative to `plot graphics/xmin` (and its variants): simply provide at least two coordinates in *<list of coordinates>*. Their bounding box is used to determine the external data range, and the graphics is squeezed between these coordinates.

The example from above can be written equivalently as



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis on top,title=Graphics Import]
\addplot graphics
% instead of the min/max things:
[points={(0,1) (1,0)},
% trim=left bottom right top
includegraphics={trim=12 9 12 8,clip}]
{external2};

\addplot coordinates {(0,0) (1,1)};
\end{axis}
\end{tikzpicture}
```

The *<list of coordinates>* is a sequence of the form (x,y) for two-dimensional plots and (x,y,z) for three-dimensional ones, the ordering is irrelevant. The single elements are separated by white space.

It is possible to mix `plot graphics/xmin` and variants with `plot graphics/points`.

The `plot graphics/points` key has further functionality for inclusion of three-dimensional graphics which is discussed at the end of this section (on page 45). Here is a short reference on the accepted syntax for three-dimensional plot graphics: in addition to the `(x,y,z)` syntax, you can provide arguments of the form `(x,y,z) => (X,Y)`. Here, the first (three-dimensional) coordinate is a logical coordinate and the second (two-dimensional) coordinate denotes the coordinates of the very same point, but inside of the included image (relative to the lower left corner of the image). Applications and examples for this syntax can be found in the section for three-dimensional plot graphics (see page 45).

`/pgfplots/plot graphics/includegraphics={\options}`

A list of options which will be passed as-is to `\includegraphics`. Interesting options include the `trim=<left> <bottom> <right> <top>` key which reduces the bounding box and `clip` which discards everything outside of the bounding box. The scaling options won't have any effect, they will be overwritten by PGFPLOTS.

`/pgfplots/plot graphics/includegraphics cmd={\macro}` (initially `\includegraphics`)

Allows to use a different graphics routine. A possible choice could be `\pgfimage`. The macro should accept the `width` and `height` arguments (in brackets) and the file name as first argument.

`/pgfplots/plot graphics/node` (style, no value)

A predefined style used for the TikZ node containing the graphics. The predefined value is

```
\pgfplotsset{
  plot graphics/node/.style={
    transform shape,
    inner sep=0pt,
    outer sep=0pt,
    every node/.style={},
    anchor=south west,
    at=((0pt,0pt)),
    rectangle
  }
}
```

`/pgfplots/plot graphics` (no value)

This key belongs to the public low-level plotting interface. You won't need it in most cases.

This key is similar to `sharp plot` or `smooth` or `const plot`: it installs a low-level plot-handler which expects exactly two points: the lower left corner and the upper right one. The graphics will be drawn between them. The graphics file name is expected as value of the `/pgfplots/plot graphics/src` key. The other keys described above need to be set correctly (excluding the limits, these are ignored at this level of abstraction). This key can be used independently of an axis.

`/pgfplots/plot graphics/lowlevel draw={\width}{\height}`

A low-level interface for `plot graphics` which actually invokes `\includegraphics`. But there is no magic involved: the command is simply expected to draw a box of dimensions $\langle width \rangle \times \langle height \rangle$. The coordinate system has already been shifted correctly.

The initial configuration is

```
\includegraphics[{value of "plot graphics/includegraphics"},width=#1,height=#2]
  {{value of "plot graphics/src"}}.
```

Thus, you can tweak `plot graphics` to place any T_EX box of the desired dimensions into an axis between the provided minimum and maximum coordinates. It is not necessary to make use of the graphics file name or the options in the `'includegraphics'` key if you overwrite this lowlevel interface with

```
plot graphics/lowlevel draw/.code 2 args={\code which depends on #1 and #2}.
```

Support for External Three-Dimensional Graphics

PGFPLOTS offers several visualization techniques for three dimensional graphics. Nevertheless, complex visualizations or specialized applications are beyond the scope of PGFPLOTS and you might want to use other tools to generate such figures.

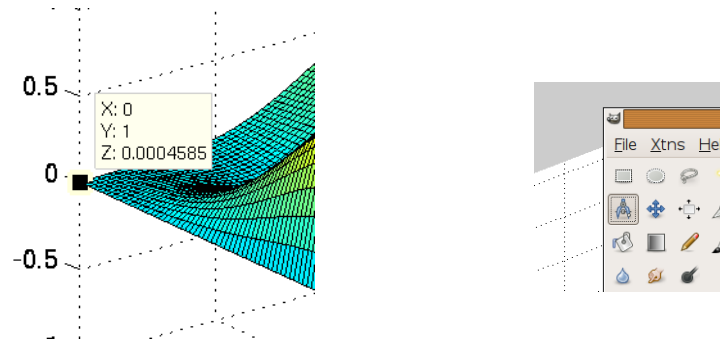


Figure 4.1: Using Matlab to extract image coordinates (left) and Gimp to measure distances (right).

The `plot_graphics` tool of PGFPLOTS allows to include three-dimensional external graphics: it generates a three-dimensional axis on its own. The idea is to provide a graphics (without descriptions) and use PGFPLOTS to overlay a three-dimensional axis automatically. This allows to maintain document consistency (making it unnecessary to use different programs within the same document).

You are probably wondering how this is possible. Well, it needs more user input than two-dimensional external graphics. The cost to include external three dimensional images into PGFPLOTS is essentially control of a graphics program like `gimp`: you need to identify the 3D coordinates of a couple of points in your image. PGFPLOTS will then squeeze the graphics correctly, and it reconfigures the axis to ensure a correct display of the result.

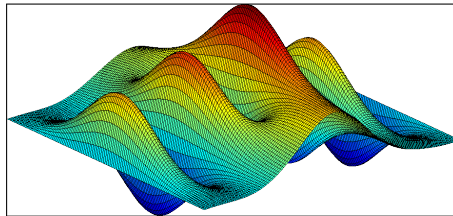
Matlab versus other tools: Although this section is based on Matlab images, the technique to import three-dimensional graphics is independent of Matlab. Thus, if you have a different tool, you need to read all that follows. However, users of Matlab *can use a simplified export mechanism* which has been contributed by Jürnjakob Dugge. Please skip to section 4.2.8 on page 50 if you use Matlab to generate the graphics files (although you may want to take a brief look at the examples on the following pages to learn about flexibility or legends).

Let's start with two examples. Suppose you generate a surface plot with Matlab and want to include it in PGFPLOTS. We have the matlab script

```
[x,y]=meshgrid(linspace(0,1,120));
surf(x,y,sin(8*pi*x).* exp(-20*(y-0.5).^2) + exp(-(x-0.5).^2*30 - (y-0.25).^2 - (x-0.5).*(y-0.25)))
xlabel('x'), ylabel('y')
axis off
print -dpng plotgraphics3dsurf
```

which generates the figure in question.

After automatically computing a tight bounding box for `plotgraphics3dsurf.png` (I used `gimp`'s Image>>Autocrop feature), and making the background color transparent (`gimp`: select the outer white space with the magic wand, then use¹² Layer>>Transparency>>Color to Transparency) we get:

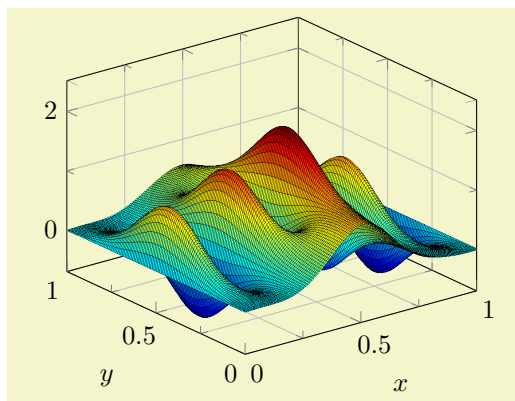


The key idea is now to identify several points in the image, and assign *both* their logical three-dimensional coordinates *and* the corresponding two-dimensional canvas coordinates in image coordinates. How? Well, the three-dimensional coordinates are known to Matlab, it can display them for you if you click somewhere into the image, compare Figure 4.1 (left).

The two-dimensional canvas coordinates need work; they need to be provided relative to the *lower left corner* of the image. I used `gimp` and activated “Points” as units (lower left corner). The lower left corner now displays the image coordinates in `pt` which is compatible with PGFPLOTS. An alternative to pointing onto coordinates is a measurement tool; compare Figure 4.1 (right) for the “Measure” tool in `gimp` which allows to compute the length of a line (in our case, the length of the lower left corner to the point of interest).

¹²I have a german version, I am not sure if the translation is correct.

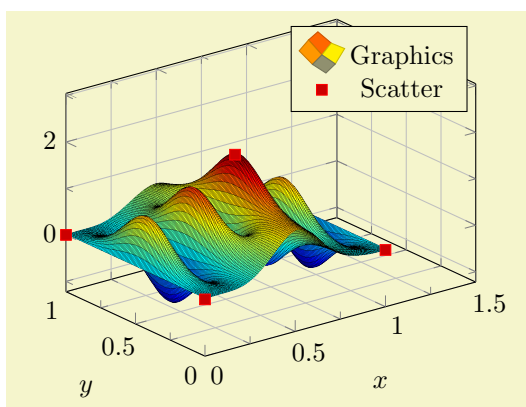
I selected four points in the graphics and noted their 2d image coordinates and their 3d logical coordinates as follows:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  grid=both,minor tick num=1,
  xlabel=$x$,ylabel=$y$,
]
\addplot3 graphics[
  points={% important
    (0,1,0) => (0,207-112)
    (1,0,0) => (446,207-133)
    (0.5546,0.5042,1.825) => (236,207)
    (0,0,0) => (194,207-202)
  }] {plotdata/plotgraphics3dsurf.png};
\end{axis}
\end{tikzpicture}
```

Here, the `points` key gets our collected coordinates as argument. It accepts a sequence of maps of the form $\langle 3d \text{ logical coordinate} \rangle \Rightarrow \langle 2d \text{ canvas coordinate} \rangle$. In our case, $(0,1,0)$ has been found in the `.png` file at $(0,207-112)$. Note that I introduced the difference since `gimp` counts from the upper left, but `PGFPLOTS` counts from the lower left.

Once these four point coordinates are gathered, we find Matlab's surface plot in a `PGFPLOTS` axis. You can modify any appearance options, including different axis limits or further `\addplot` commands:

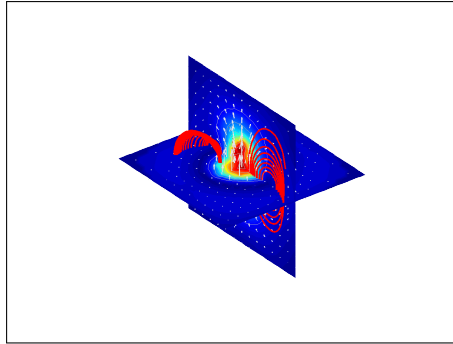


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xmax=1.5,% extra limits
  grid=both,minor tick num=1,
  xlabel=$x$,ylabel=$y$,
]
\addplot3[surf] % 'surf' is only used for the legend.
graphics[
  points={
    (0,1,0) => (0,207-112)
    (1,0,0) => (446,207-133)
    (0.5546,0.5042,1.825) => (236,207)
    (0,0,0) => (194,207-202)
  }]
{plotdata/plotgraphics3dsurf.png};
\addlegendentry{Graphics}

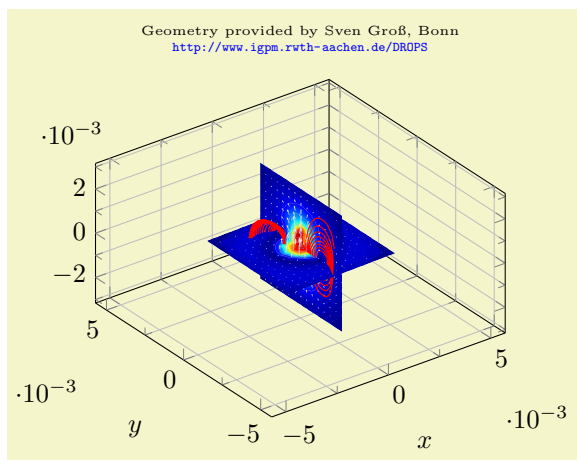
\addplot3+[only marks] coordinates {
  (0,1,0) (1,0,0)
  (0.5546,0.5042,1.825) (0,0,0)
};
\addlegendentry{Scatter}
\end{axis}
\end{tikzpicture}
```

`PGFPLOTS` uses the four input points to compute appropriate `x`, `y` and `z` unit vectors (and the origin in graphics coordinates). These four vectors (with two components each) can be computed as a result of a linear system of size 8×8 , that is why you need to provide four input points (each has two coordinates). `PGFPLOTS` computes the unit vectors of the imported graphics, and afterwards it rescales the result such that it fits into the specified `width` and `height`. This rescaling respects the `unit vector ratio` (more precisely, it uses `scale mode=scale uniformly` instead of `scale mode=stretch to fill`). Consequently, the freedom to change the view of a three-dimensional axis which contains a projected graphics is considerably smaller than before. Surprisingly, you can still change axis limits and `width` and `height` – `PGFPLOTS` will take care of a correct display of your imported graphics. Since version 1.6, you can also change `zmin` and/or `zmax` – `PGFPLOTS` will respect your changes as good as it can.

Here is a further example. Suppose we are given the three-dimensional visualization



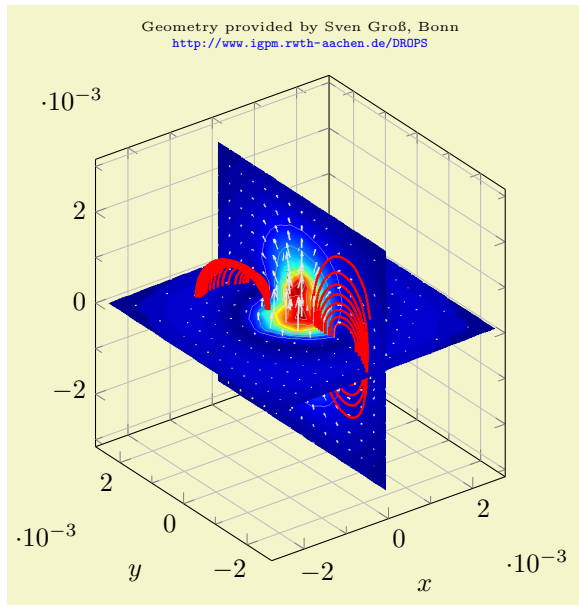
It has been generated by matlab (I only added transparency to the background with `gimp`). Besides advanced visualization techniques, it uses `axis equal`, i.e. `unit vector ratio=1 1 1`. As before, we need to identify four points, each with its 3d logical coordinates (from matlab) and the associated 2d canvas coordinates relative to the lower left corner of the graphics (note that there is a lot of white space around the graphics). Here is the output of PGFPLOTS when you import the resulting graphics:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  grid=both,minor tick num=1,
  xlabel=$x$,ylabel=$y$,
  title={\centering
    Geometry provided by Sven Groß, Bonn\\
    \url{http://www.igpm.rwth-aachen.de/DROPS}}},
  title style={text width=6cm,font=\tiny},
]
\addplot3 graphics[
  points={
    (-0.002625,0.002625,0) => (140,234)
    (0,0.00263,0.00263) => (230,364)
    (0,-0.00263,-0.00263) => (366,81)
    (0,-0.00263,0.00263) => (366,276)
    (0.002625,0.002625,0.002625)
  }
]{plotdata/risingdrop3d.png};
\end{axis}
\end{tikzpicture}
```

Note that I provided *five* three-dimensional coordinates here, but the last entry has no `=>` mapping to two-dimensional canvas coordinates. Thus, it is only used to update the bounding box (see the reference manual for the `points` key for details).

The example above leads to a relatively small image and much “empty space”. This is due to the `scale mode=scale` uniformly implementation of PGFPLOTS: it decided that the best way is to enlarge the involved axis limits. Here, “best way” means to satisfy `width/height` constraints combined with minimally enlarged (never shrunk) axis limits. The remaining degrees of freedom are `width`, `height`, and the axis limits. In our case, changing the ratio between `width` and `height` improves the display:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    height=8cm,width=7cm,% improve scaling manually
    grid=both,minor tick num=1,
    xlabel=$x$,ylabel=$y$,
    title={\centering
        Geometry provided by Sven Groß, Bonn\\
        \url{http://www.igpm.rwth-aachen.de/DROPS}\},
    title style={text width=6cm,font=\tiny},
]
\addplot3 graphics[
    points={
        (-0.002625,0.002625,0) => (140,234)
        (0,0.00263,0.00263) => (230,364)
        (0,-0.00263,-0.00263) => (366,81)
        (0,-0.00263,0.00263) => (366,276)
        (0.002625,0.002625,0.002625)
    }
]
{plotdata/risingdrop3d.png};
\end{axis}
\end{tikzpicture}
```

What happens is that PGFPLOTS selects a *single* scaling factor which is applied to all units as they have been deduced from the `points` key. This ensures that the imported graphics fits correctly into the axis. In addition, PGFPLOTS does its best to satisfy the remaining constraints.

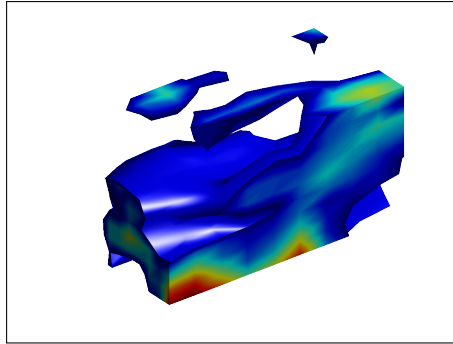
The complete description of how PGFPLOTS scales the axis can be found in the documentation for `scale mode=scale` uniformly. Here is just a brief summary: PGFPLOTS assumes that the prescribed `width` and `height` have to be satisfied. To this end, it rescales the projected unit vectors (i.e. the space which is taken up for each unit in x , y , and z) and it can modify the axis limits. In the default configuration `scale uniformly strategy=auto`, PGFPLOTS will *never* shrink axis limits.

Compatibility remark: Note that the scaling capabilities have been improved for PGFPLOTS version 1.6. In previous versions, only `scale uniformly strategy=change vertical limits` was available which lead to clipped axes. In short: please consider writing `\pgfplotsset{compat=1.6}` or newer into your document to benefit from the improved scaling. If you have `\pgfplotsset{compat=1.5}` or older, the outcome for `\addplot3 graphics` will be different.

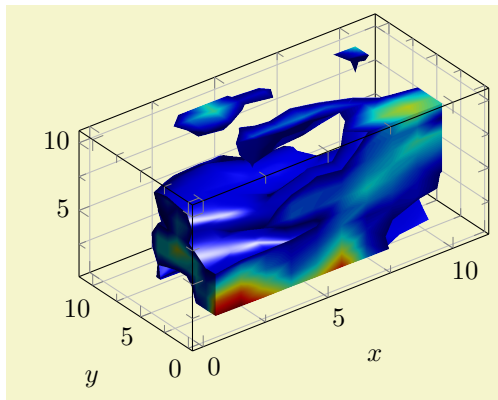
We consider a third example which has been generated by the Matlab code

```
clear all
close all
seed = sum(clock)
rand('seed',seed);
X = rand(10,10,10);
data = smooth3(X,'box',5);
p1 = patch(isosurface(data,.5), ...
    'FaceColor','blue','EdgeColor','none');
p2 = patch(isocaps(data,.5), ...
    'FaceColor','interp','EdgeColor','none');
isonormals(data,p1)
daspect([1 2 2])
view(3); axis vis3d tight
camlight; lighting phong
% print -dpng plotgraphics3withaxis
axis off
print -dpng plotgraphics3
save plotgraphics3.seed seed -ASCII % to reproduce the result
```

I only added background transparency with gimp and got the following graphics:



We proceed as before and collect four points, each with 3d logical coordinates (by clicking into the matlab figure) and their associated 2d canvas (graphics) coordinates using the measure tool of gimp. The result is shown in the code example below.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    grid=both,minor tick num=1,
    xlabel=$x$,ylabel=$y$,
    3d box,
]
    \addplot3 graphics[
        points={
            (1,1,1)    => (205,48)
            (10,1,10) => (503,324)
            (1,1,4.044)=> (206,102)
            (10,10,10) => (390,398)
        }
    ]
    {plotdata/plotgraphics3.png};
\end{axis}
\end{tikzpicture}
```

Note that it has non-standard data aspect ratio which is respected by PGFPLOTS automatically.

External Three-Dimensional Graphics and Matlab

An extension by Jörn Jakob Dugge

The procedure to map three-dimensional logical coordinates to two-dimensional canvas coordinates is tedious.

Jörn Jakob Dugge contributed a script which does most of the logic and your work is reduced to a copy-paste job. With his permission, I post the contribution here.

The idea is to start a simple script which *records* mappings for any coordinates which have been clicked by the user. It works as follows:

1. Create the Matlab plot, say, using

```
hist3(randn(10000,2)) % some random data
set(get(gca,'child'),'FaceColor','interp','CDataMode','auto'); % colors
% make sure the "print" paper format is the same as the screen paper format:
set(gcf,'PaperPositionMode','auto')
```

2. Save the following code as `pgfplotsconversion.m`:

```
function pgfplotsconversion

% Hook into the Data Cursor "click" event
h = datacursormode(gcf);
set(h,'UpdateFcn',@myupdatefcn,'SnapToDataVertex','off');
datacursormode on

% select four points in plot using mouse

% The function that gets called on each Data Cursor click
function [txt] = myupdatefcn(obj,event_obj)

% Get the screen resolution, in dots per inch
dpi = get(0,'ScreenPixelsPerInch');

% Get the click position in pixels, relative to the lower left of the
% screen
screen_location=get(0,'PointerLocation');

% Get the position of the plot window, relative to the lower left of
% the screen
figurePos = get(gcf,'Position');

% Get the data coordinates of the cursor
pos = get(event_obj,'Position');

% Format the data and figure coordinates. The factor "72.27/dpi" is
% necessary to convert from pixels to TeX points (72.27 points per inch)
display(['(',num2str(pos(1))',' ',num2str(pos(2))',' ',num2str(pos(3))',' ') => (' , ...
    num2str((screen_location(1)-figurePos(1))*72.27/dpi)',' ', ...
    num2str((screen_location(2)-figurePos(2))*72.27/dpi)',' ')]

% Format the tooltip display
txt = {'X: ',num2str(pos(1))',' ', 'Y: ',num2str(pos(2))',' ', 'Z: ',num2str(pos(3))'};
```

Run `pgfplotsconversion`, click on four points in your plot. Preferably select non-colinear points near the edges of the plot. Copy and paste the four lines that were written to the Matlab command window.

Make sure that the first two points have different X and Y values on screen (i.e. image canvas coordinates).

3. Export the plot as an image

```
axis off
print -dpng matlabout -r400 % PNG called "matlabout.png" with 400 dpi resolution
```

If you want to export vectors graphics, you should note that pdf output of Matlab is clumsy. It might be best to export to `eps` first, followed by a conversion from `eps` to `pdf`.

If you really want to use pdf output of Matlab, you may need to set the paper size to match the figure size by yourself, since the PDF driver does not automatically adjust the size:

```
% It might be better to use print -depssc followed by epstopdf.
% Use this if you (really) want to use print -dpdf:
currentScreenUnits=get(gcf,'Units') % Get current screen units
currentPaperUnits=get(gcf,'PaperUnits') % Get current paper units
set(gcf,'Units',currentPaperUnits) % Set screen units to paper units
plotPosition=get(gcf,'Position') % Get the figure position and size
set(gcf,'PaperSize',plotPosition(3:4)) % Set the paper size to the figure size
set(gcf,'Units',currentScreenUnits) % Restore the screen units

print -dpdf matlabout % PDF called "matlabout.pdf"
```

4. Include the image in your PGFPLOTS axis. If you selected points on the plot corners, your `xmin`, `xmax`, `ymin` and `ymax` should be set automatically, otherwise you may want to provide those yourself. Also, adjustments of `width` and `height` might be of interest to get the right vertical placement of the plot. Consider changing `zmin` and/or `zmax` to fit your needs (preferably only one of them; otherwise PGFPLOTS may be unable to fix the `height`).

This contribution is from
<http://tex.stackexchange.com/questions/52987/3-dimensional-histogram-in-pgfplots> .

Summary: External Three-Dimensional Graphics

As has been shown in the previous sections, `\addplot3 graphics` allows to include three-dimensional graphics and PGFPLOTS overlays a flexible axis with all its power. The cost to do so is

1. collect both logical three-dimensional coordinates *and* image-internal two-dimensional coordinates for *four points* of your graphics.
 In Matlab, this can be simplified by the tool mentioned on page 50.
2. If your axes form a right-handed-coordinate system, that is all. If not, also add `x dir=reverse` for any reversed axes.

Consider the following list of you encounter problems while working with `\addplot3 graphics`:

- It must be possible to deduce the origin and the three (two-dimensional) unit vectors from the four provide `points`; otherwise the algorithm will fail.
 The algorithm should detect any deficiencies. However, if you encounter strange “Dimension too large” messages here, you can try other arguments in `points`. Take a look into your log file, it will probably indicate the source of problems (or use the `debug` key).
- Ensure that the external graphics has an orthogonal axis. In fact, the axis may be skewed (just like a PGFPLOTS axis can be created by means of custom `x`, `y`, and `z` vectors). However, the external image must not have perspective projection as this is unsupported by PGFPLOTS. The `points` command needs to receive four points which belong to linearly independent position vectors.
- PGFPLOTS uses the first two points to squeeze the graphics into the desired coordinates (which implies that they should not have the same canvas *X* or *Y* coordinates). It verifies that the remaining `points` arguments are projected correctly.
- The resulting scaling by means of `scale mode=scale uniformly` will try to satisfy all scaling constraints. You can change these constraints by modifying `width`, `height`, `xmin`, `xmax`, `ymin`, `ymax`, `zmin`, `zmax` and/or any combination of these parameters. See also `unit rescale keep size` which controls the flexibility of limit changes. There is also a key `scale uniformly strategy` which allows to select a different scaling strategy.
- The image should have a “right-handed-coordinate system”: you should be able to take your right hand, point your thumb in direction of the *x* axis, your first finger in direction of *y*, and your second finger in direction of the *z* axis. If that is impossible, once of your axes is reversed and you need to communicate that to PGFPLOTS explicitly by means of the `x dir=reverse` key (and its variants).
- Note that this feature has been verified with standard cartesian axes only.
- There is a `debug` key to investigate what the algorithm is doing:

`/pgfplots/plot graphics/debug=true|false|visual` (initially `false`)

If you provide `\addplot3 graphics[debug,points={...}]`, PGFPLOTS will provide debug information onto your terminal and into the logfile. It will also generate extra files containing the determined unit vectors and the linear system used to derive them (one such file for every `\addplot3 graphics` statement, the filename will be the graphics file name and `.dat` appended).

Without the `debug` key, only the log file will contain brief information what PGFPLOTS is doing behind the scenes.

The choice `true` activates log messages. The choice `visual` activates log messages *and* places some filled circles at the provided `points`. The choice `false` disables all `debug` features.

4.2.9 Reading Coordinates From Files

```
\addplot file {<name>};
\addplot[<options>] file {<name>} <trailing path commands>;
\addplot3 ...
```

Deprecation note: If you have data files, you should generally use `\addplot table`. The input type `\addplot file` is almost the same, but considerably less powerful. It is only kept for backwards compatibility.

The `\addplot file` input mechanism is similar to the TikZ-command ‘`plot file`’. It is to be used like

```
\addplot file {datafile.dat};
```

where $\langle name \rangle$ is a text file with at least two columns which will be used as x and y coordinates. Lines starting with ‘%’ or ‘#’ are ignored. Such files are often generated by GNUPLLOT:

```
#Curve 0, 20 points
#x y type
0.00000 0.00000 i
0.52632 0.50235 i
1.05263 0.86873 i
1.57895 0.99997 i
...
9.47368 -0.04889 i
10.00000 -0.54402 i
```

This listing has been copied from [5, section 16.4].

Plot file accepts one optional argument,

```
\addplot file[skip first] {datafile.dat};
```

which allows to skip over a non-comment header line. This allows to read the same input files as `plot table` by skipping over column names. Please note that comment lines do not count as lines here.

The input method `plot file` can also read meta data for every coordinate. As already explained for `plot coordinates` (see above), meta data can be used to change colors or other style parameters for every marker separately. Now, if `point meta` is set to `explicit` or to `explicit symbolic` and the input method is `plot file`, one further element will be read from disk – for every line. Meta data is always the last element which is read. See page 79 for information and examples about per point meta data and page 81 for an application example using `scatter/classes`.

Plot file is very similar to `plot table`: you can achieve the same effect with

```
\addplot table[x index=0,y index=1,header=false] {datafile.dat};
```

Due to its simplicity, `plot file` is slightly faster while `plot table` allows higher flexibility.

Technical note: every opened file will be protocolled into your log file.

The file can contain `empty lines` to tell PGFPLOTS that the function has jumps. To use it, simply insert an empty line (and ensure that you have `\pgfplotsset{compat=1.4}` or newer in your preamble). See the documentation of `empty line` for details.

```
/pgfplots/plot file/skip first=true|false (initially false)
/pgfplots/plot file/ignore first=true|false (initially false)
```

The two keys can be provided as arguments to `\addplot file[ $\langle options \rangle$ ] { $\langle filename \rangle$ }; to skip the first non-comment entry in the file. They are equivalent. If you provide them in this context, the prefix /pgfplots/plot file can be omitted.`

4.3 About Options: Preliminaries

PGFPLOTS knows a whole lot of key–value options which can be (re)defined to activate desired features or modified to apply some fine-tuning.

A key usually has a value (like a number, a string, or perhaps some macro code). You can assign values to keys (“set keys”) in many places in a L^AT_EX document. The value will remain effective until it is changed or until the current T_EX scope ends (which happens after a closing curly brace ‘}’, after `\end{ $\langle name \rangle$ }` or, for example, after `\addplot`).

Most keys can be used like

```

\begin{tikzpicture}
\begin{axis}[key=value,key2=value2] % axis-wide keys
...
\end{axis}
\end{tikzpicture}

```

which changes them for the complete axis. A key in this context can be any option defined in this manual, no matter if it has the `/pgfplots/` or the `/tikz/` key prefix. Note that key prefixes can be omitted in almost all cases.

A value can usually be provided without curly braces. For example, if the manual contains something like `\xmin={\langle x coordinate \rangle}`, you can safely skip the curly braces. The curly braces are mandatory if values contain something which would otherwise confuse the key setup (for example an equal sign ‘=’ or a comma ‘,’).

Some keys can be changed individually for each plot:

```

\begin{tikzpicture}
\begin{axis}
% keys valid for single plots:
\addplot ...; % uses the "cycle list" to determine keys
\addplot[key=value,key2=value2] ... ; % uses the provided keys (not the "cycle list")
\addplot+[key=value,key2=value2] ... ; % appends something to the "cycle list"
\end{axis}
\end{tikzpicture}

```

Besides these two possibilities, it is also possible to work with document-wide keys:

```

\chapter{My Section}
\pgfplotsset{
  key=value,
  key2=value2,
}
This section has a common key configuration:
\begin{tikzpicture}
  \begin{axis}% uses the key config from above
  ...
  \end{axis}
\end{tikzpicture}

```

In the example above, the `\pgfplotsset` command changes keys. The changes are permanent and will be used until

- you redefine them or
- the current environment (like `\end{figure}`) is ended or
- $\text{\TeX}$ encounters a closing brace ‘}’.

This includes document-wide preamble configurations like

```

\documentclass{article}

\usepackage{pgfplots}
\pgfplotsset{
  xticklabel={\mathsf{\pgfmathprintnumber{\tick}}},
  every axis/.append style={
    font=\sfamily,
  },
}
...

```

The basic engine to manage key–value pairs is `pgfkeys` which is part of PGF. This engine always has a key name and a key “path”, which is somehow similar to file name and directory of files. The common “directory” (key path) of PGFPLOTS is `/pgfplots/`. Although the key definitions below provide this full path, it is always (well, almost always) safe to skip this prefix – PGFPLOTS uses it automatically. The same holds for the prefixes `/tikz/` which are common for all TikZ drawing options and `/pgf/` which are for the (more or less) low-level commands of PGF. All these prefixes can be omitted.

One important concept is the concept of *styles*. A style is a key which contains one or more other keys. It can be redefined or modified until it is actually used by the internal routines. Each single component of TikZ and PGFPLOTS can be configured with styles.

For example,

```
\pgfplotsset{legend style={line width=1pt}}
```

sets the line width for every legend to 1pt by appending ‘line width=1pt’ to the existing style for legends.

There are keys like `legend style`, `ticklabel style`, and `label style` which allow to modify the predefined styles (in this case the styles for legends, ticklabels and axis labels, respectively). They are, in general, equivalent to a `<style name>/.<append style>={}` command (the only difference is that the `./append style` thing is a little bit longer). There is also the possibility to define a new style (or to overwrite an already existing one) using `./style={}`.

There are several other styles predefined to modify the appearance, see Section 4.17.

```
\pgfplotsset{<key-value-list>}
```

Defines or sets all options in `<key-value-list>`. The `<key-value-list>` can contain any of the options in this manual which have the prefix `/pgfplots/` (however, you do not need to type that prefix).

Inside of `<key-value-list>`, the prefixes `/pgfplots/` which are commonly presented in this manual can be omitted (they are checked automatically).

This command can be used to define default options for the complete document or a part of the document. For example,

```
\pgfplotsset{
  cycle list={%
    {red, mark=*}, {blue, mark=*},
    {red, mark=x}, {blue, mark=x},
    {red, mark=square*}, {blue, mark=square*},
    {red, mark=triangle*}, {blue, mark=triangle*},
    {red, mark=diamond*}, {blue, mark=diamond*},
    {red, mark=pentagon*}, {blue, mark=pentagon*}
  },
  legend style={
    at={(0.5,-0.2)},
    anchor=north,
    legend columns=2,
    cells={anchor=west},
    font=\footnotesize,
    rounded corners=2pt,
  },
  xlabel=$x$, ylabel=$f(x)$
}
```

can be used to set document-wise styles for line specifications, the legends’ style and axis labels. The settings remain in effect until the end of the current environment (like `\end{figure}`) or until you redefine them or until the next closing curly brace ‘}’ (whatever comes first).

You can also define new styles (collections of key–value–pairs) with `./style` and `./append style`.

```
\pgfplotsset{
  My Style 1/./style={xlabel=$x$, legend entries={1,2,3} },
  My Style 2/./style={xlabel=$X$, legend entries={4,5,6} }
}
```

The `./style` and `./append style` key handlers are described in Section 4.17 in more detail.

Key handler `<key>/.<code>={<TEX code>}`

Occasionally, the PGFLOTS user interface offers to replace parts of its routines. This is accomplished using so called “code keys”. What it means is to replace the original key and its behavior with new `<TEX code>`. Inside of `<TEX code>`, any command can be used. Furthermore, the `#1` pattern will be the argument provided to the key.

I’ve been invoked with ‘this here’

```
\pgfplotsset{
  My Code/./code={I’ve been invoked with ‘#1’}
}\pgfplotsset{My Code={this here}}
```

The example defines a (new) key named `My Code`. Essentially, it is nothing else but a `\newcommand`, plugged into the key-value interface. The second statement “invokes” the code key.

Key handler `<key>/.<code> 2 args={<TEX code>}`

As `./code`, but this handler defines a key which accepts two arguments. When the so defined key is used, the two arguments are available as `#1` and `#2`.

Key handler `<key>/.cd`

Each key has a fully qualified name with a (long) prefix, like `/pgfplots/xmin`. However, if the “current directory” is `/pgfplots`, it suffices to write just `xmin`. The `/.cd` key handler changes the “current directory” in this way.

The prefixes `/tikz/` and `/pgfplots/` are checked automatically for any argument provided to `\begin{axis}[<options>]` or `\addplot`. So, you won’t need to worry about them, just omit them – and look closer in case the package doesn’t identify the option.

4.3.1 Pgfplots and TikZ Options

This section is more or less technical and can be skipped unless one really wants to know more about this topic.

TikZ options and PGFPLOTS options can be mixed inside of the axis arguments and in any of the associated styles. For example,

```
\pgfplotsset{every axis legend/.append style={
  legend columns=3,font=\Large}}
```

assigns the ‘`legend columns`’ option (a PGFPLOTS option) and uses ‘`font`’ for drawing the legend (a TikZ option). The point is: `legend columns` needs to be known *before* the legend is typeset whereas `font` needs to be active when the legend is typeset. PGFPLOTS sorts out any key dependencies automatically:

The axis environments will process any known PGFPLOTS options, and all ‘`every`’-styles will be parsed for PGFPLOTS options. Every unknown option is assumed to be a TikZ option and will be forwarded to the associated TikZ drawing commands. For example, the ‘`font=\Large`’ above will be used as argument to the legend matrix, and the ‘`font=\Large`’ argument in

```
\pgfplotsset{every axis label/.append style={
  ylabel=Error,xlabel=Dof,font=\Large}}
```

will be used in the nodes for axis labels (but not the axis title, for example).

It is an error if you assign incompatible options to axis labels, for example ‘`xmin`’ and ‘`xmax`’ can’t be set inside of ‘`every axis label`’.

4.4 Two Dimensional Plot Types

PGFPLOTS supports several two-dimensional line plots like piecewise linear line plots, piecewise constant plots, smoothed plots, bar plots and comb plots. Most of them use the PGF plot handler library directly, see [5, section 18.8].

Plot types are part of the plot style, so they are set with options. Most of the basic 2d plot types are part of TikZ, see [5, section 18.8], and are probably known to users of TikZ. They are documented here as well.

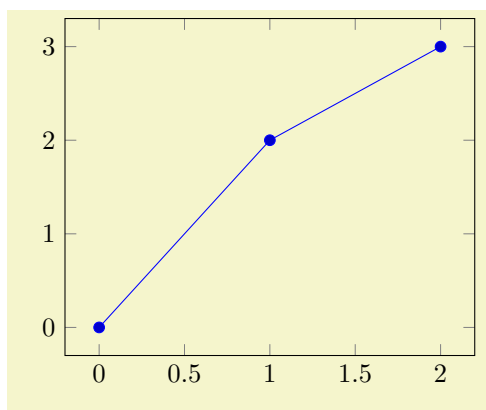
4.4.1 Linear Plots

`/tikz/sharp plot`

(no value)

`\addplot+[sharp plot]`

Linear (‘sharp’) plots are the default. Point coordinates are simply connected by straight lines.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
  \addplot+[sharp plot] coordinates
    {(0,0) (1,2) (2,3)};
\end{axis}
\end{tikzpicture}
```


The ‘+’ here means to use the normal plot cycle list and append ‘**sharp plot**’ to its option list.

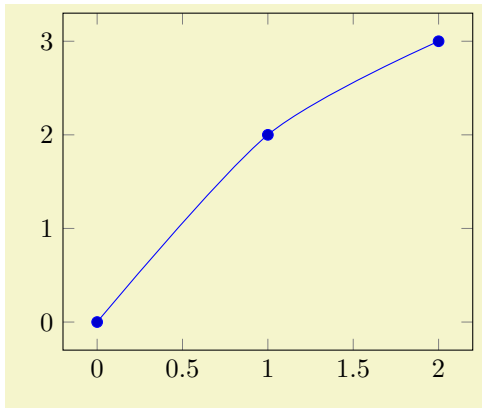
4.4.2 Smooth Plots

`/tikz/smooth`

(no value)

`\addplot+[smooth]`

Smooth plots interpolate smoothly between successive points.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[smooth] coordinates
{(0,0) (1,2) (2,3)};
\end{axis}
\end{tikzpicture}
```

4.4.3 Constant Plots

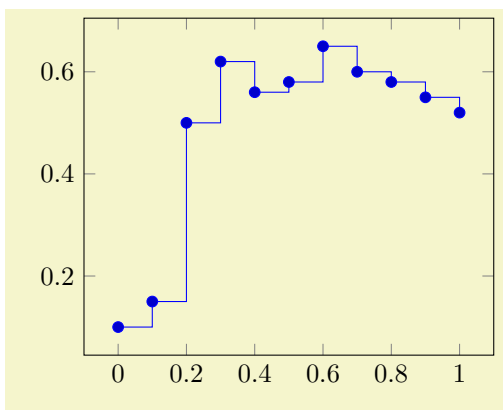
Constant plots draw lines parallel to the x -axis to connect coordinates. The discontinuous edges may be drawn or not, and marks may be placed on left or right ends.

`/tikz/const plot`

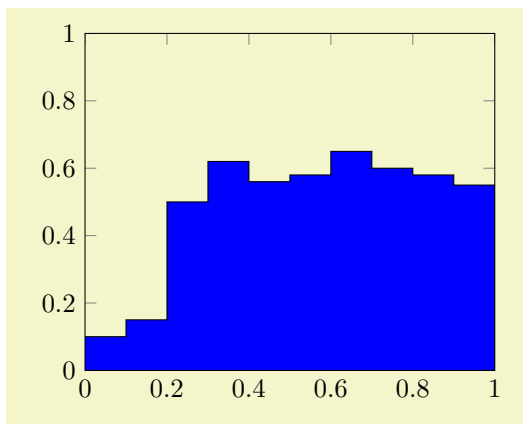
(no value)

`\addplot+[const plot]`

Connects all points with horizontal and vertical lines. Marks are placed left-handed on horizontal line segments, causing the plot to be right-sided continuous at all data points.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[const plot]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
(0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
(0.8,0.58) (0.9,0.55) (1,0.52)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[ymin=0,ymax=1,enlargelimits=false]
\addplot
[const plot,fill=blue,draw=black]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
(0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
(0.8,0.58) (0.9,0.55) (1,0.52)}
\closedcycle;
\end{axis}
\end{tikzpicture}
```

`/tikz/const plot mark left`

(no value)

`\addplot+[const plot mark left]`

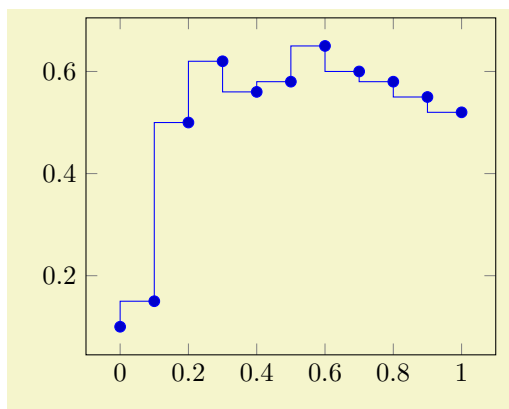
An alias for ‘const plot’.

`/tikz/const plot mark right`

(no value)

`\addplot+[const plot mark right]`

A variant which places marks on the right of each line segment, causing plots to be left-sided continuous at the given coordinates.



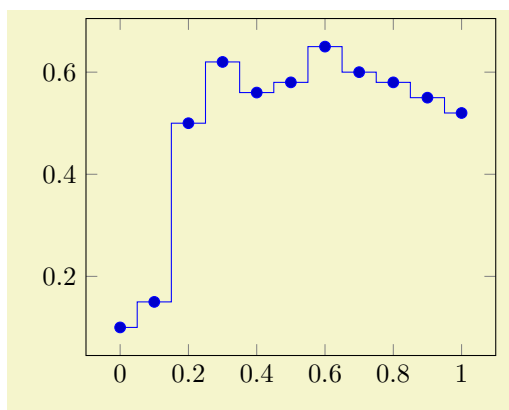
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[const plot mark right]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
(0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
(0.8,0.58) (0.9,0.55) (1,0.52)};
\end{axis}
\end{tikzpicture}
```

`/tikz/const plot mark mid`

(no value)

`\addplot+[const plot mark mid]`

A variant which places marks in the middle of each line segment, causing plots to be symmetric around its data points.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[const plot mark mid]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
(0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
(0.8,0.58) (0.9,0.55) (1,0.52)};
\end{axis}
\end{tikzpicture}
```

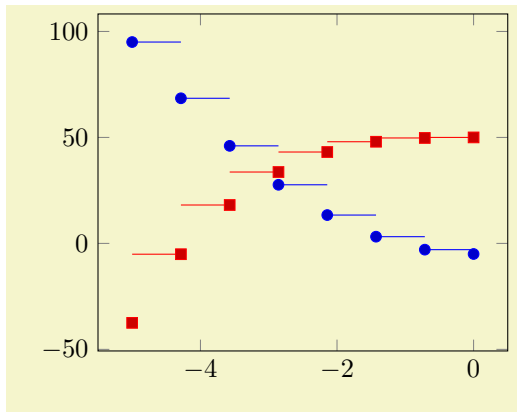
Note that “symmetric” is only true for constant mesh width: if the x -distances between adjacent data points differ, `const plot mark mid` will produce vertical lines in the middle between each pair of consecutive points.

`/tikz/jump mark left`

(no value)

`\addplot+[jump mark left]`

A variant of ‘`const plot mark left`’ which does not draw vertical lines.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[samples=8]
\addplot+[jump mark left,domain=-5:0]
{4*x^2 - 5};

\addplot+[jump mark right,domain=-5:0]
{0.7*x^3 + 50};
\end{axis}
\end{tikzpicture}
```

`/tikz/jump mark right`

(no value)

`\addplot+[jump mark right]`

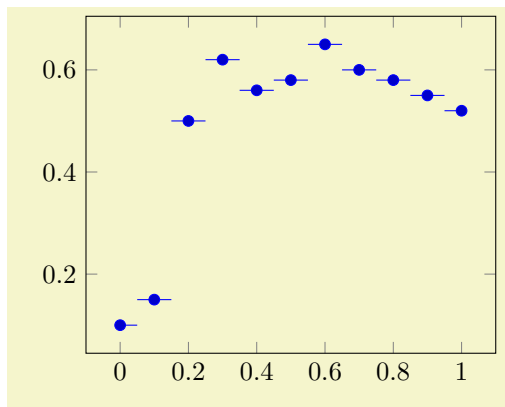
A variant of ‘`const plot mark right`’ which does not draw vertical lines.

`/tikz/jump mark mid`

(no value)

`\addplot+[jump mark mid]`

A variant of ‘`const plot mark mid`’ which does not draw vertical lines.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[jump mark mid]
coordinates
{(0,0.1) (0.1,0.15) (0.2,0.5) (0.3,0.62)
(0.4,0.56) (0.5,0.58) (0.6,0.65) (0.7,0.6)
(0.8,0.58) (0.9,0.55) (1,0.52)};
\end{axis}
\end{tikzpicture}
```

4.4.4 Bar Plots

Bar plots place horizontal or vertical bars at coordinates. Multiple bar plots in one axis can be stacked on top of each other or aligned next to each other.

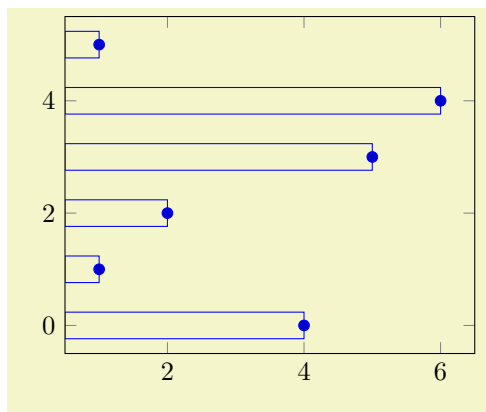
`/tikz/xbar`

(no value)

`\addplot+[xbar]`

Places horizontal bars between the ($y = 0$) line and each coordinate.

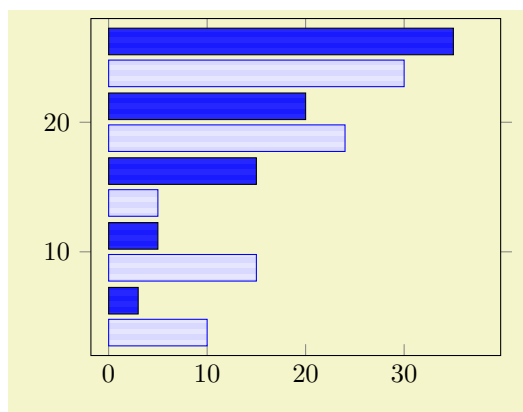
This option is used on a per-plot basis and configures only the visualization of coordinates. The figure-wide style `/pgfplots/xbar` also sets reasonable options for ticks, legends and multiple plots.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[xbar] coordinates
{(4,0) (1,1) (2,2)
(5,3) (6,4) (1,5)};
\end{axis}
\end{tikzpicture}
```

Bars are centered at plot coordinates with width `bar width`. Using bar plots usually means more than just a different way of how to connect coordinates, for example to draw ticks outside of the axis, change the legend's appearance or introduce shifts if multiple `\addplot` commands appear.

There is a preconfigured style for `xbar` which is installed automatically if you provide `xbar` as argument to the axis environment which provides this functionality.

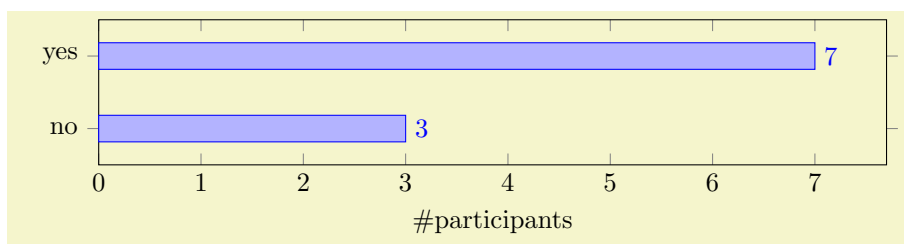


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[xbar,enlargelimits=0.15]
\addplot
[draw=blue,pattern=horizontal lines light blue]
coordinates
{(10,5) (15,10) (5,15) (24,20) (30,25)};

\addplot
[draw=black,pattern=horizontal lines dark blue]
coordinates
{(3,5) (5,10) (15,15) (20,20) (35,25)};
\end{axis}
\end{tikzpicture}
```

Here `xbar` yields `/pgfplots/xbar` because it is an argument to the axis, not to a single plot.

For bar plots, it is quite common to provide textual coordinates or even descriptive nodes near the bars. This can be implemented using the keys `symbolic y coords` and `nodes near coords`, respectively:

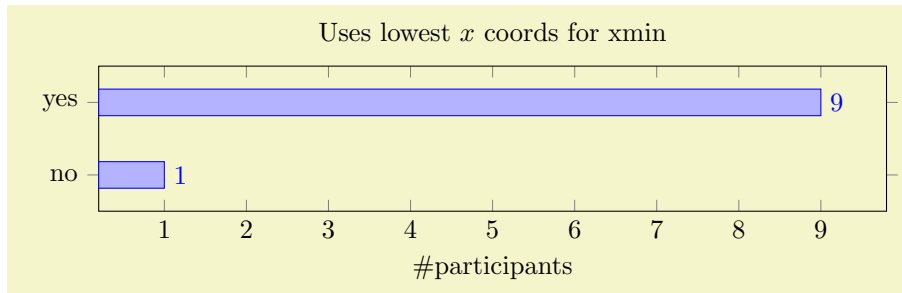


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
xbar, xmin=0,
width=12cm, height=3.5cm, enlarge y limits=0.5,
xlabel={\#participants},
symbolic y coords={no,yes},
ytick=data,
nodes near coords, nodes near coords align={horizontal},
]
\addplot coordinates {(3,no) (7,yes)};
\end{axis}
\end{tikzpicture}
```

The `symbolic y coords` defines a dictionary of accepted coordinates which are then expected in y coordinates and the `nodes near coords` key displays values as extra nodes (see their reference docu-

mentations for details). The example employs `enlarge y limits` in order to get some more free space since the default spacing is not always appropriate for bar plots.

Note that it might be quite important to include `xmin=0` explicitly as in the example above. Without it, the lower bound will be used:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title=Uses lowest $x$ coords for xmin,
    xbar,
    width=12cm, height=3.5cm, enlarge y limits=0.5,
    xlabel={\#participants},
    symbolic y coords={no,yes},
    ytick=data,
    nodes near coords, nodes near coords align={horizontal},
  ]
    \addplot coordinates {(1,no) (9,yes)};
  \end{axis}
\end{tikzpicture}
```

Besides line, fill, and colorstyles, bars can be configured with `bar width` and `bar shift`, see below.

`/pgfplots/xbar={\langle shift for multiple plots \rangle}` (style, default 2pt)

This style sets `/tikz/xbar` and some commonly used options concerning horizontal bars for the complete axis. This is automatically done if you provide `xbar` as argument to an axis argument, see above.

The `xbar` style defines shifts if multiple plots are placed into one axis. It draws bars adjacent to each other, separated by `\langle shift for multiple plots \rangle`. Furthermore, it sets the style `bar cycle list` and sets tick and legend appearance options.

The style is defined as follows.

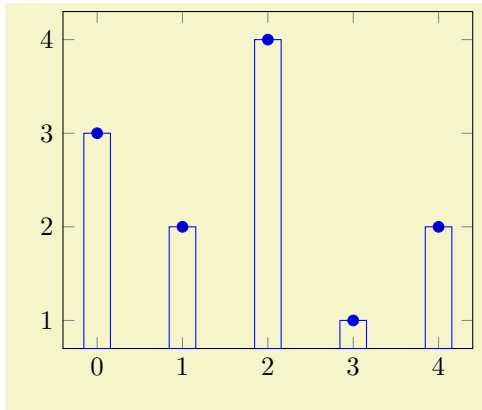
```
\pgfplotsset{
  /pgfplots/xbar/.style={
    /tikz/xbar,
    bar cycle list,
    tick align=outside,
    xbar legend,
    /pgf/bar shift={%
      % total width = n*w + (n-1)*skip
      % i.e. subtract half for centering
      -0.5*(\numplotsactualtype*\pgfplotbarwidth + (\numplotsactualtype-1)*\pgfplotbarwidth) +
      % the '0.5*w' is for centering
      (.5*\plotnumofactualtype)*\pgfplotbarwidth + \plotnumofactualtype*1%
    },
  },
}
```

The formula for `bar shift` assigns shifts dependent on the total number of plots and the current plot's number. It is designed to fill a total width of $n \cdot \text{bar width} + (n - 1) \cdot \langle \text{shift for multiple plots} \rangle$. The 0.5 compensates for centering.

`/tikz/ybar` (no value)

`\addplot+[ybar]`

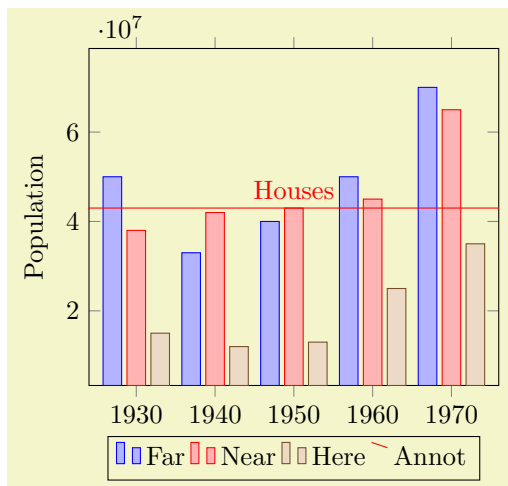
Like `xbar`, this option generates bar plots. It draws vertical bars between the ($x = 0$) line and each input coordinate.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[ybar] plot coordinates
    {(0,3) (1,2) (2,4) (3,1) (4,2)};
\end{axis}
\end{tikzpicture}
```

The example above simply changes how input coordinates shall be visualized. As mentioned for `xbar`, one usually needs modified legends and shifts for multiple bars in the same axis.

There is a predefined style which installs these customizations when provided to the axis environment:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    x tick label style={
        /pgf/number format/1000 sep=},
    ylabel=Population,
    enlargelimits=0.15,
    legend style={at={(0.5,-0.15)},
        anchor=north,legend columns=-1},
    ybar,
    bar width=7pt,
]
\addplot
    coordinates {(1930,50e6) (1940,33e6)
        (1950,40e6) (1960,50e6) (1970,70e6)};

\addplot
    coordinates {(1930,38e6) (1940,42e6)
        (1950,43e6) (1960,45e6) (1970,65e6)};

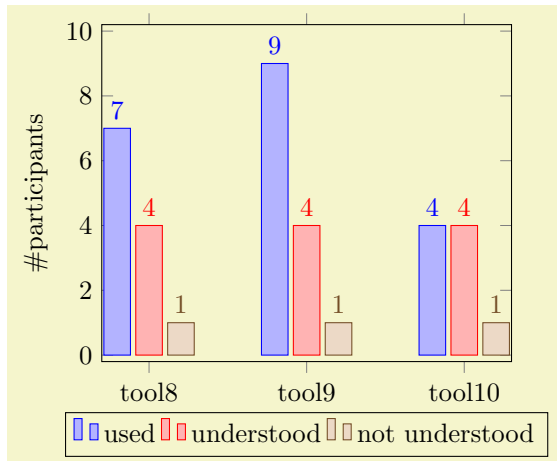
\addplot
    coordinates {(1930,15e6) (1940,12e6)
        (1950,13e6) (1960,25e6) (1970,35e6)};

\addplot[red,sharp plot,update limits=false]
    coordinates {(1910,4.3e7) (1990,4.3e7)}
    node[above] at (axis cs:1950,4.3e7) {Houses};

\legend{Far,Near,Here,Annot}
\end{axis}
\end{tikzpicture}
```

Here, `ybar` yields `/pgfplots/ybar` because it is an argument to the axis, not to a single plot. The style affects the first three `\addplot` commands. Note that it shifts them horizontally around the plot coordinates. The fourth `\addplot` command is some kind of annotation which doesn't `update limits`.

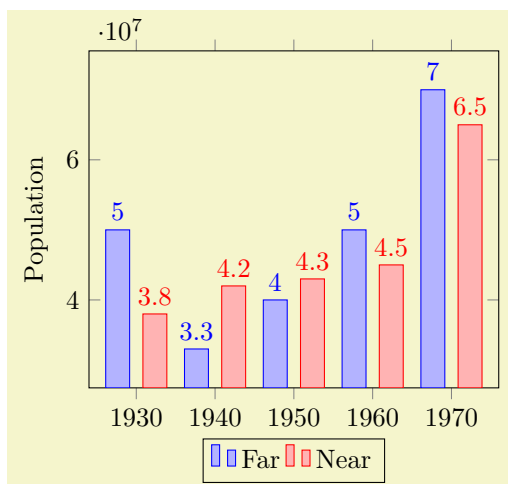
The `ybar` style can be combined with `symbolic x coords` in a similar way as described for `xbar`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ybar,
  enlargelimits=0.15,
  legend style={at={(0.5,-0.15)},
    anchor=north,legend columns=-1},
  ylabel={\#participants},
  symbolic x coords={tool8,tool9,tool10},
  xtick=data,
  nodes near coords,
  nodes near coords align={vertical},
]
\addplot coordinates {(tool8,7) (tool9,9) (tool10,4)};
\addplot coordinates {(tool8,4) (tool9,4) (tool10,4)};
\addplot coordinates {(tool8,1) (tool9,1) (tool10,1)};
\legend{used,understood,not understood}
\end{axis}
\end{tikzpicture}
```

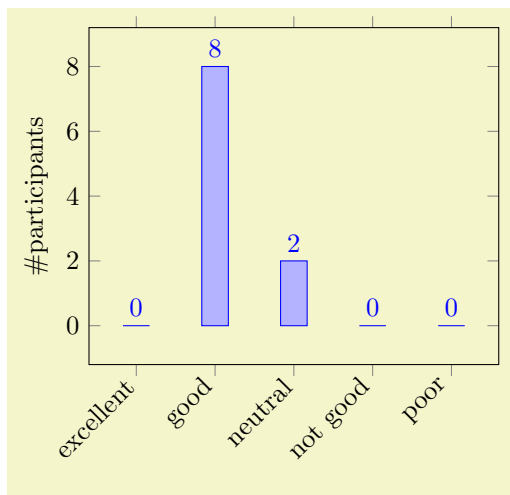
As for `xbar`, the bar width and shift can be configured with `bar width` and `bar shift`. However, the bar shift is better provided as argument to `/pgfplots/ybar` since this style will overwrite the bar shift. Thus, prefer `/pgfplots/ybar=4pt` to set the bar shift.

Sometimes it is useful to write the y values directly near the bars. This can be realized using the `nodes near coords` method:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  x tick label style={
    /pgf/number format/1000 sep=},
  ylabel=Population,
  enlargelimits=0.15,
  legend style={at={(0.5,-0.15)},
    anchor=north,legend columns=-1},
  ybar=5pt,% configures 'bar shift'
  bar width=9pt,
  nodes near coords,
  point meta=y *10^-7 % the displayed number
]
\addplot
  coordinates {(1930,50e6) (1940,33e6)
    (1950,40e6) (1960,50e6) (1970,70e6)};
\addplot
  coordinates {(1930,38e6) (1940,42e6)
    (1950,43e6) (1960,45e6) (1970,65e6)};
\legend{Far,Near}
\end{axis}
\end{tikzpicture}
```

Any support style changes are possible, of course. A useful example for bar plots might be to use rotated tick labels:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ybar,
  enlargelimits=0.15,
  legend style={at={(0.5,-0.2)},
    anchor=north,legend columns=-1},
  ylabel={\#participants},
  symbolic x coords={excellent,good,neutral,%
    not good,poor},
  xtick=data,
  nodes near coords,
  nodes near coords align={vertical},
  x tick label style={rotate=45,anchor=east},
]
\addplot coordinates {(excellent,0) (good,8)
  (neutral,2) (not good,0) (poor,0)};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/ybar={⟨shift for multiple plots⟩}` (style, default 2pt)

As `/pgfplots/xbar`, this style sets the `/tikz/ybar` option to draw vertical bars, but it also provides commonly used options for vertical bars.

If you supply `ybar` to an axis environment, `/pgfplots/ybar` will be chosen instead of `/tikz/ybar`.

It changes the legend, draws ticks outside of the axis lines and draws multiple `\addplot` arguments adjacent to each other; block-centered at the x coordinate and separated by *⟨shift for multiple plots⟩*. It will also install the `bar shift` for *every node near coord*. Furthermore, it installs the style `bar cycle list`. It is defined similarly to `/pgfplots/xbar`.

`/pgfplots/bar cycle list` (no value)

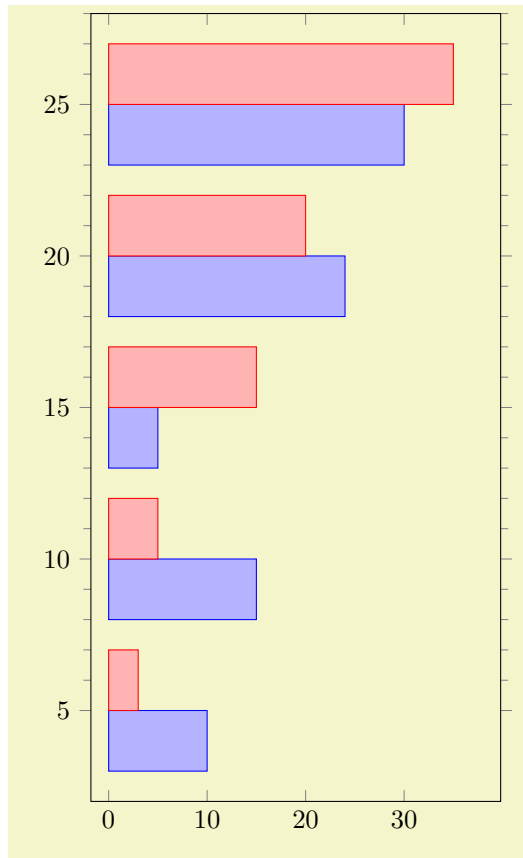
A style which installs cycle lists for multiple bar plots.

```
\pgfplotsset{
  /pgfplots/bar cycle list/.style={/pgfplots/cycle list={%
    {blue,fill=blue!30!white,mark=none},%
    {red,fill=red!30!white,mark=none},%
    {brown!60!black,fill=brown!30!white,mark=none},%
    {black,fill=gray,mark=none},%
  }
},
}
```

`/pgf/bar width={⟨dimension or unit⟩}` (initially 10pt)

Configures the width used by `xbar` and `ybar`. It is accepted to provide mathematical expressions.

As of PGFPLOTS 1.7, it allows to provide an *unit* as `bar width`. In this case, the `bar width` will be interpreted as axis unit:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xbar=0pt,% space of 0pt between adjacent bars
  bar width=2,
  width=7cm,
  height=12cm,
  minor y tick num=4,
  ytick=data,
  enlargelimits=0.15]
\addplot
coordinates
  {(10,5) (15,10) (5,15) (24,20) (30,25)};
\addplot
coordinates
  {(3,5) (5,10) (15,15) (20,20) (35,25)};
\end{axis}
\end{tikzpicture}
```

In order to interpret arguments as units, you have to write `\pgfplotsset{compat=1.7}` (or newer) into your preamble. Older versions will implicitly append the `pt` suffix if the argument is no dimension.

`\pgfplotbarwidth`

A mathematical expression which results in the fully computed value of `bar width` (i.e. it includes any unit computations).

Note that you may need to `enlargelimits` in order to see the complete bar – PGFPLOTS will not automatically update the axis limits to respect `bar width`.

`/pgf/bar shift={\langle dimension or unit \rangle}` (initially 0pt)

Configures a shift for `xbar` and `ybar`. Use `bar shift` together with `bar width` to draw multiple bar plots into the same axis. It is accepted to provide mathematical expressions.

As of PGFPLOTS 1.7, it allows to provide an *unit* as `bar shift`. In this case, the `bar shift` will be interpreted as axis unit.

`\pgfplotbarshift`

A mathematical expression which results in the fully computed value of `bar shift` (i.e. it includes any unit computations).

Note that you may need to `enlargelimits` in order to see the complete bar – PGFPLOTS will not automatically update the axis limits to respect `bar shift`.

`/pgfplots/bar direction=auto|x|y` (initially auto)

If PGFPLOTS encounters a value `bar width=1` (i.e. *without* dimension like `1pt`), it attempts to evaluate the bar's direction.

The default configuration `auto` assumes that you write something like `ybar, bar width=1`. In this case, it is clear that you have a *y* bar and PGFPLOTS assumes `bar direction=y`.

However, this context information is unavailable. In this case, you can use the choice `x` if PGFPLOTS is unaware that it works on an `xbar` plot or `y` if PGFPLOTS is unaware that you meant an `ybar` plot.

`/tikz/ybar interval`

(no value)

`\addplot+[ybar interval]`

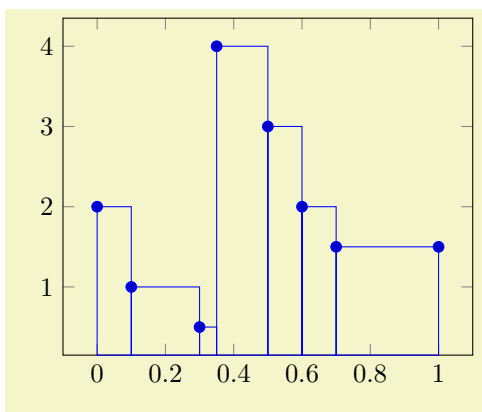
This plot type produces vertical bars with width (and shift) relative to intervals of coordinates.

There is one conceptional difference when working with intervals: an interval is defined by *two* coordinates. Since `ybar` has one value for each interval, the i th bar is defined by

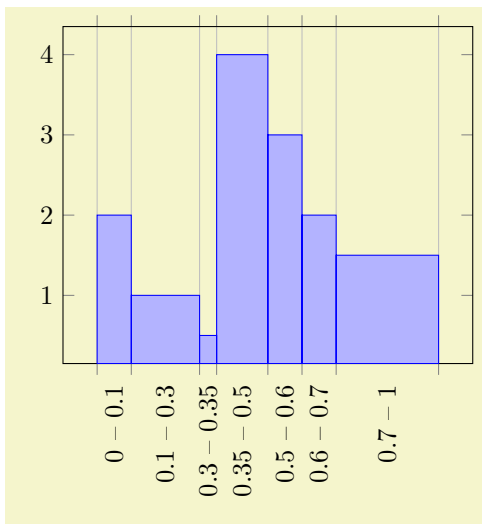
1. the y value of the i th coordinates,
2. the x value of the i th coordinate as left interval boundary,
3. the x value of the $(i + 1)$ th coordinate as right interval boundary.

Consequently, there is *one coordinate too much*: the last coordinate will *only* be used to determine the interval width; its y value doesn't influence the bar appearance.

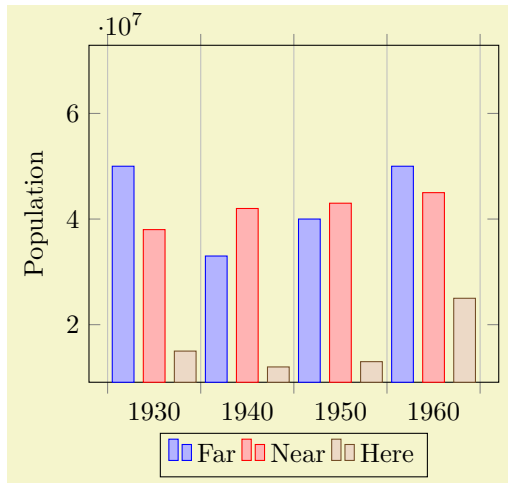
It is installed on a per-plot basis and configures *only* the visualization of coordinates. See the style `/pgfplots/ybar interval` which configures the appearance of the complete figure.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[ybar interval] plot coordinates
  {(0,2) (0.1,1) (0.3,0.5) (0.35,4) (0.5,3)
   (0.6,2) (0.7,1.5) (1,1.5)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[ybar interval,
  xtick=data,
  xticklabel interval boundaries,
  x tick label style=
    {rotate=90,anchor=east}
]
\addplot coordinates
  {(0,2) (0.1,1) (0.3,0.5) (0.35,4) (0.5,3)
   (0.6,2) (0.7,1.5) (1,1.5)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  x tick label style={
    /pgf/number format/1000 sep=},
  ylabel=Population,
  enlargelimits=0.05,
  legend style={at={(0.5,-0.15)},
    anchor=north,legend columns=-1},
  ybar interval=0.7,
]
\addplot
  coordinates {(1930,50e6) (1940,33e6)
    (1950,40e6) (1960,50e6) (1970,70e6)};

\addplot
  coordinates {(1930,38e6) (1940,42e6)
    (1950,43e6) (1960,45e6) (1970,65e6)};

\addplot
  coordinates {(1930,15e6) (1940,12e6)
    (1950,13e6) (1960,25e6) (1970,35e6)};
\legend{Far,Near,Here}
\end{axis}
\end{tikzpicture}
```

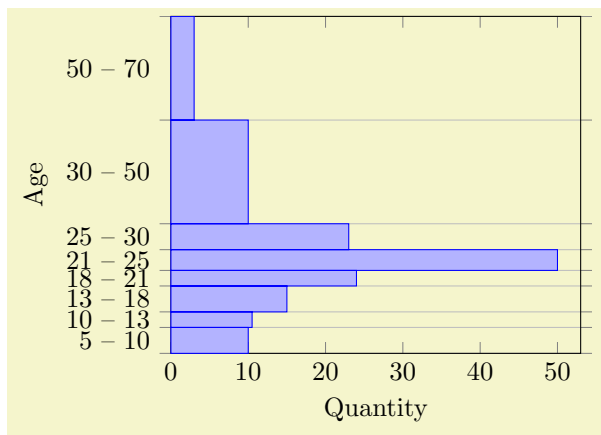
`/pgfplots/ybar interval={\relative width}` (style, default 1)

A style which is intended to install options for `ybar interval` for a complete figure. This includes tick and legend appearance, management of multiple bar plots in one figure and a more adequate `cycle list` using the style `bar cycle list`.

`/tikz/xbar interval` (no value)

`\addplot+[xbar interval]`

As `ybar interval`, just for horizontal bars.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xmin=0,xmax=53,
  ylabel=Age,
  xlabel=Quantity,
  enlargelimits=false,
  ytick=data,
  yticklabel interval boundaries,
  xbar interval,
]
\addplot
  coordinates {(10,5) (10.5,10) (15,13)
    (24,18) (50,21) (23,25) (10,30)
    (3,50) (3,70)};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/xbar interval={\relative width}` (style, default 1)

A style which is intended to install options for `xbar interval` for a complete figure, see the style `/pgfplots/ybar interval` for details.

`/pgfplots/xticklabel interval boundaries` (no value)

`/pgfplots/yticklabel interval boundaries` (no value)

`/pgfplots/zticklabel interval boundaries` (no value)

These are style keys which set `x tick label as interval` (see page 254 for details) and configure the tick appearance to be `<start> - <end>` for each tick interval.

4.4.5 Histograms

This section has been moved to the `statistics` library, see Section 5.9.2 on page 383.

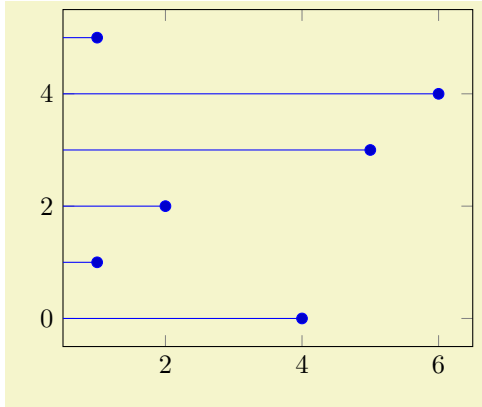
4.4.6 Comb Plots

Comb plots are very similar to bar plots except that they employ single horizontal/vertical lines instead of rectangles.

`/tikz/xcomb`

(no value)

`\addplot+[xcomb]`

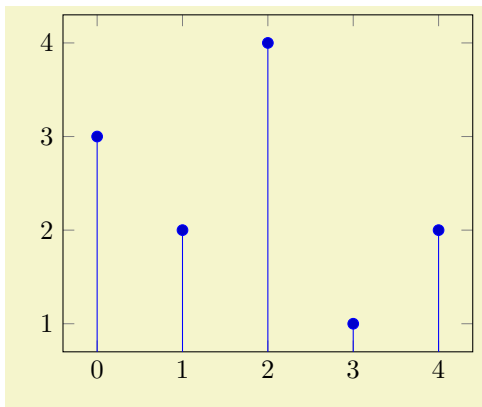


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[xcomb] coordinates
    {(4,0) (1,1) (2,2)
     (5,3) (6,4) (1,5)};
\end{axis}
\end{tikzpicture}
```

`/tikz/ycomb`

(no value)

`\addplot+[ycomb]`



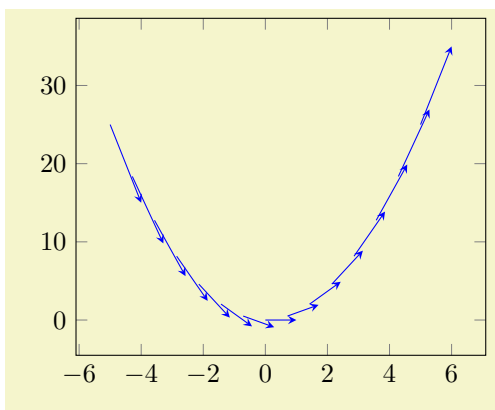
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[ycomb] plot coordinates
    {(0,3) (1,2) (2,4) (3,1) (4,2)};
\end{axis}
\end{tikzpicture}
```

4.4.7 Quiver Plots (Arrows)

`/pgfplots/quiver={options with 'quiver/' prefix}}`

`\addplot+[quiver={options with 'quiver/' prefix}]`

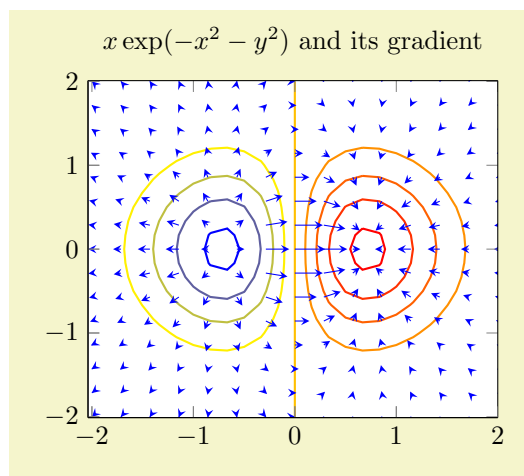
A plot type which draws small arrows, starting at (x, y) , in direction of (u, v) .



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot[blue,
    quiver={u=1,v=2*x},
    -stealth,samples=15] {x^2};
\end{axis}
\end{tikzpicture}
```

The base point (x, y) is provided as before; in the example above, it is generated by `plot expression` and yields (x, x^2) . The vector direction (u, v) needs to be given in addition. Our example with `quiver/u=1` and `quiver/v=2*x` results in $u = 1$ and $v = 2x$. Thus, we have defined and visualized a vector field for the derivative of $f(x) = x^2$.

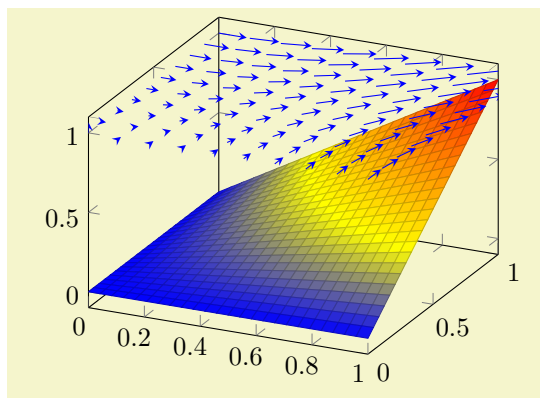
A common example is to visualize the gradient $(\partial_x f, \partial_y f)(x, y)$ of a two-dimensional function $f(x, y)$:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
\addplot3[contour gnuplot={number=9,
labels=false},thick]
{exp(0-x^2-y^2)*x};
\addplot3[blue,
quiver={
u={exp(0-x^2-y^2)*(1-2*x^2)},
v={exp(0-x^2-y^2)*(-2*x*y)},
scale arrows=0.3,
},
-stealth,samples=15]
{exp(0-x^2-y^2)*x};
\end{axis}
\end{tikzpicture}
```

The example visualizes $f(x, y) = x \exp(-x^2 - y^2)$ using `contour gnuplot` as first step. The options `contour/number` and `contour/labels` provide fine-tuning for the contour and are not of interest here (so is the `axis background` which just improves visibility of contour lines). What we are interested in is the `quiver=` style: it defines u and v to some two-dimensional expressions. Furthermore, we used `quiver/scale arrows` to reduce the arrow size. The `-stealth` is a TikZ style which configures outgoing arrow styles of type ‘stealth’. The `samples=15` key configures how we get our input data. In our case, we have input data $(x_i, y_j, f(x_i, y_j))$ with 15 samples for each, i and j .

It is also possible to place quiver plots on a prescribed z value:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
domain=0:1,
xmax=1,
ymax=1,
]
\addplot3[surf] {x*y};
\addplot3[blue,/pgfplots/quiver,
quiver/u=y,
quiver/v=x,
quiver/w=0,
quiver/scale arrows=0.1,
-stealth,samples=10] {1};
\end{axis}
\end{tikzpicture}
```

Here, the quiver plots is placed on top of a `surf`. It visualizes the gradient (using a common scale factor of $1/10$ to reduce the arrow lengths). The `quiver/w=0` means that arrows have no z difference, and the `{1}` argument indicates that all start at $(x_i, y_j, 1)$. Here, the values (x_i, y_j) are sampled in the `domain=0:1` argument (with `samples=10`), i.e. arrows start at $(x_i, y_j, 1)$ and end at $(x_i + y_j/10, y_j + x_i/10, 1)$.

So far, quiver plots do not assume a special sequence of input points. This has two consequences: first, you can plot any vector field by considering just $(x, y) + (u, v)$ (or $(x, y, z) + (u, v, w)$) – the data doesn’t necessarily need to be a two-dimensional function (as opposed to `surf` etc). On the other hand, you need to provide `quiver/scale arrows` manually since `quiver` doesn’t know the mesh width in case you provide matrix data¹³.

Note that quiver plots are currently not available together with logarithmic axes.

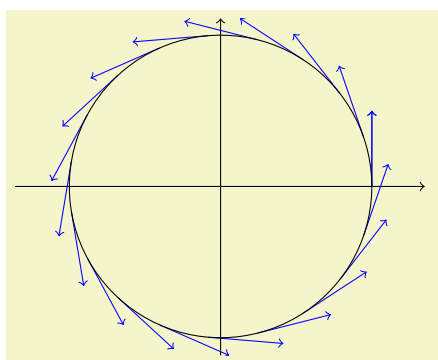
¹³Actually, I might add something like `quiver/scale arrows=auto` in the future, I don’t know yet. Loops through input data are slow in TeX, automatic mesh widths computation even more...

`/pgfplots/quiver/u=<expression>` (initially 0)
`/pgfplots/quiver/v=<expression>` (initially 0)
`/pgfplots/quiver/w=<expression>` (initially 0)

These keys define how the vector directions (u, v) (or, for three dimensional plots, (u, v, w)) shall be set.

The $\langle expression \rangle$ can be a constant expression like `quiver/u=1` or `quiver/u=42*5`. It may also depend on the final base point values using the values `x`, `y` or `z` as in the example above. In this context, `x` yields the x coordinate of the point where the vector starts, `y` the y coordinate and so on.

Attention: the fact that `x` refers to the final x coordinate means that parametric plots should use t as variable¹⁴. Consider the following example:



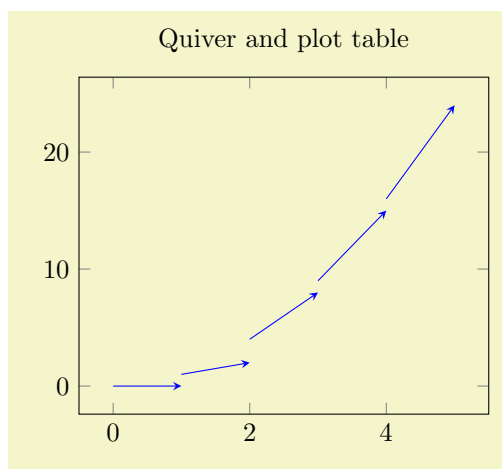
```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis equal,
axis lines=middle,
axis line style={->},
tick style={color=black},
xtick=\empty,
ytick=\empty]
\addplot[samples=20, domain=0:2*pi,
% the default choice 'variable=x' leads to
% unexpected results here!
variable=\t,
quiver={
u={-sin(deg(t))},
v={cos(deg(t))},
scale arrows=0.5,
->,blue}
({cos(deg(t))}, {sin(deg(t))});
\addplot[samples=100, domain=0:2*pi]
({cos(deg(x))}, {sin(deg(x))});
\end{axis}
\end{tikzpicture}

```

Here, a parametric plot is used to draw a circle and tangent vectors. The choice `variable=\t` plays a functional role besides naming conventions: it allows to access the parametric variable within the expressions for both `u` and `v`. On the other hand, we could have used `u=y` and `v=-x` since `x` expands to the x coordinate with value `sin(deg(t))` and `y` expands to the y coordinate `cos(deg(t))`.

Another important application is to use *table column references* like `quiver/u=\thisrow{col}` in conjunction with `\addplot table`:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[title=Quiver and plot table]
\addplot[blue,
quiver={u=\thisrow{u},v=\thisrow{v}},
-stealth]
table
{
x y u v
0 0 1 0
1 1 1 1
2 4 1 4
3 9 1 6
4 16 1 8
};
\end{axis}
\end{tikzpicture}

```

Here, the $\langle expression \rangle$ employs `\thisrow` which always refers to the actual row of `\addplot table`.

¹⁴Sorry for this usability issue.

Note that $\langle expression \rangle$ should always be of numeric type (no symbolic input extensions are supported currently).

```
/pgfplots/quiver/u value={\langle value \rangle} (initially 0)
/pgfplots/quiver/v value={\langle value \rangle} (initially 0)
/pgfplots/quiver/w value={\langle value \rangle} (initially 0)
```

These keys have the *same function* as `quiver/u` and its variants. However, they don't call the math parser, so only single values are allowed (including something like `\thisrow{columnname}`).

```
/pgfplots/quiver/colored (no value)
/pgfplots/quiver/colored={\langle color \rangle}
```

Allows to define an individual color for each arrow. Omitting the argument ' $\langle color \rangle$ ' is identical to `quiver/colored=mapped color` which uses the `point meta` to get colors for every arrow.

If you just want to set the same color for every arrow, prefer using `\addplot[blue,quiver]` which is more efficient.

```
/pgfplots/quiver/scale arrows={\langle scale \rangle} (initially 1)
```

Allows to rescale the arrows by a factor. This may be necessary if the arrow length is longer than the distance between adjacent base points (x_i, y_i) . There may come a feature to rescale them automatically.

```
/pgfplots/quiver/update limits=true|false (initially true)
```

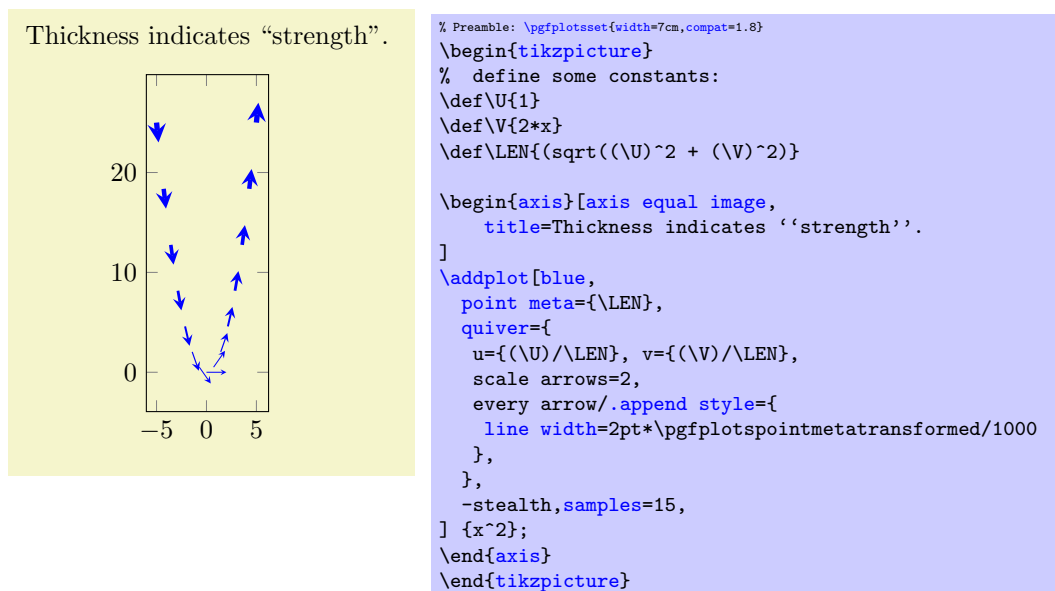
A boolean indicating whether points $(x, y) + (u, v)$ shall contribute to the axis limits.

```
/pgfplots/quiver/every arrow (style, initially empty)
```

Allows to provide individual arrow styles.

The style can contain any TikZ drawing option. It will be evaluated for every individual arrow and may depend upon anything which is available at visualization time.

In particular, this includes `point meta` data, typically using `\pgfplotspointmetatransformed` $\in [0, 1000]$ where 0 corresponds to `point meta min` and 1000 corresponds to `point meta max`:



In the example, we have some 2d vector field stored in helper constants $\U$ and $\V$. The length of each vector is stored in $\LEN$ here. The `quiver` plot as such contains unit length vectors – and the $\LEN$ enters an `every arrow` style to get varying `line width`.

An `every arrow` style might also depend upon mapped `color` (provided `point meta` has been set).

Again, if you do not need individual arrow styles, prefer using a plot style (`cycle list` or argument to `\addplot`) which is more efficient.

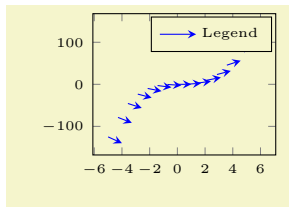
```
/pgfplots/quiver/before arrow/.code={\langle ... \rangle}
```

`/pgfplots/quiver/after arrow/.code={\<...>}`

Advanced keys for more fine tuning of the display. They allow to install some T_EX code manually before or after the drawing operations for single arrows. Both are initially empty.

`/pgfplots/quiver/quiver legend` (style, no value)

A style which redefines `legend image code` in order to produce a suitable legend for `quiver` plots. It is implicitly activated whenever `quiver` plot handlers are selected.

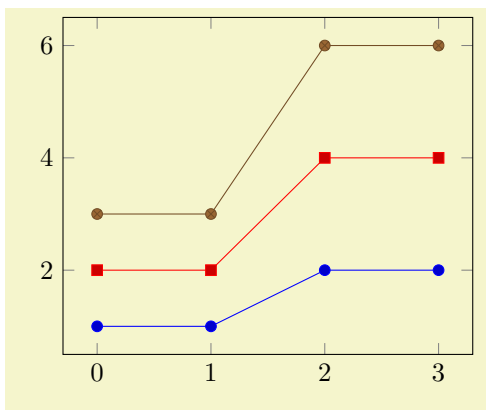


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny]
\addplot[blue,
  quiver={u=1,v=3*x},
  -stealth,
  samples=15]
  {x^3};
\addlegendentry{Legend}
\end{axis}
\end{tikzpicture}
```

4.4.8 Stacked Plots

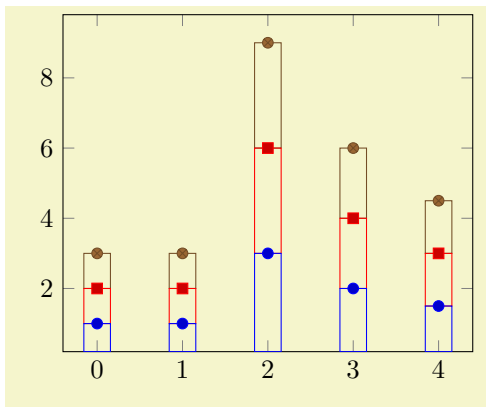
`/pgfplots/stack plots=x|y|false` (initially false)

Allows stacking of plots in either x or y direction. Stacking means to add either x - or y coordinates of successive `\addplot` commands on top of each other.

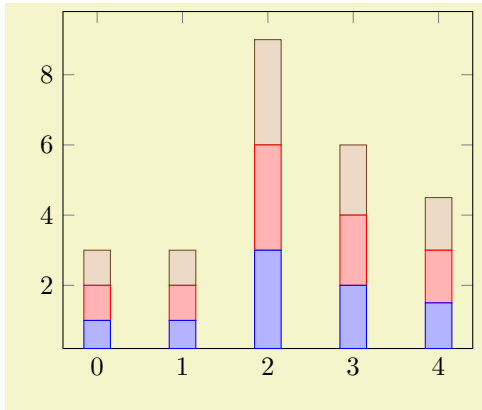


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[stack plots=y]
\addplot coordinates
  {(0,1) (1,1) (2,2) (3,2)};
\addplot coordinates
  {(0,1) (1,1) (2,2) (3,2)};
\addplot coordinates
  {(0,1) (1,1) (2,2) (3,2)};
\end{axis}
\end{tikzpicture}
```

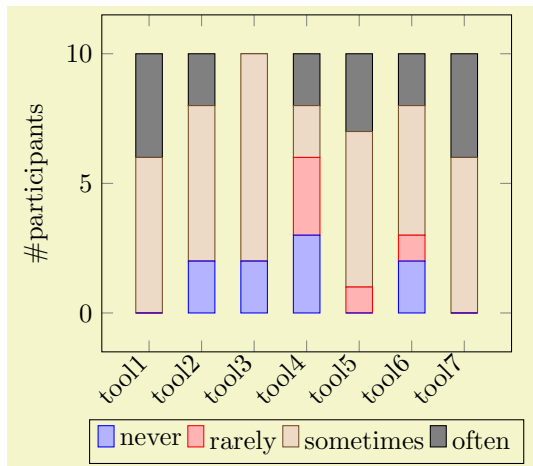
`stack plots` is particularly useful for bar plots. The following examples demonstrate its functionality. Normally, it is advisable to use the styles `ybar stacked` and `xbar stacked` which also set some other options.



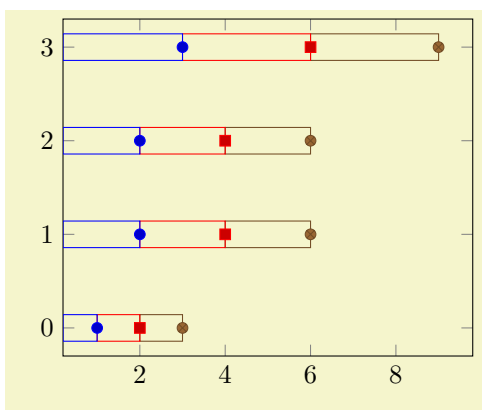
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[stack plots=y,/tikz/ybar]
\addplot coordinates
  {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
\addplot coordinates
  {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
\addplot coordinates
  {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
\end{axis}
\end{tikzpicture}
```

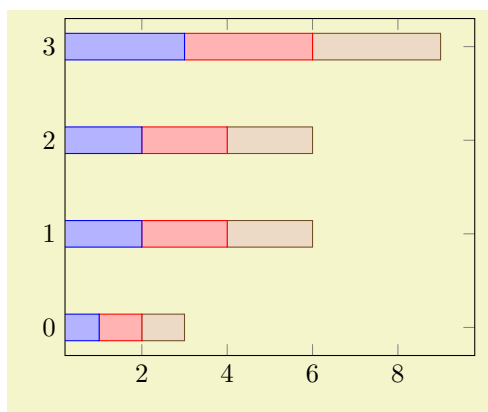
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[ybar stacked]
    \addplot coordinates
      {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
    \addplot coordinates
      {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
    \addplot coordinates
      {(0,1) (1,1) (2,3) (3,2) (4,1.5)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    ybar stacked,
    enlargelimits=0.15,
    legend style={at={(0.5,-0.20)},
      anchor=north,legend columns=-1},
    ylabel={\#participants},
    symbolic x coords={tool1, tool2, tool3, tool4,
      tool5, tool6, tool7},
    xtick=data,
    x tick label style={rotate=45,anchor=east},
  ]
    \addplot+[ybar] plot coordinates {(tool1,0) (tool2,2)
      (tool3,2) (tool4,3) (tool5,0) (tool6,2) (tool7,0)};
    \addplot+[ybar] plot coordinates {(tool1,0) (tool2,0)
      (tool3,0) (tool4,3) (tool5,1) (tool6,1) (tool7,0)};
    \addplot+[ybar] plot coordinates {(tool1,6) (tool2,6)
      (tool3,8) (tool4,2) (tool5,6) (tool6,5) (tool7,6)};
    \addplot+[ybar] plot coordinates {(tool1,4) (tool2,2)
      (tool3,0) (tool4,2) (tool5,3) (tool6,2) (tool7,4)};
  \legend{never, rarely, sometimes, often}
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[stack plots=x,/tikz/xbar]
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}

\begin{tikzpicture}
  \begin{axis}[xbar stacked]
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
    \addplot coordinates
      {(1,0) (2,1) (2,2) (3,3)};
  \end{axis}
\end{tikzpicture}
```

The current implementation for `stack plots` does *not* interpolate missing coordinates. That means stacking will fail if the plots have different grids.

`/pgfplots/stack dir=plus|minus` (initially plus)

Configures the direction of `stack plots`. The value `plus` will add coordinates of successive plots while `minus` subtracts them.

`/pgfplots/reverse stacked plots=true|false` (initially true, default true)

Configures the sequence in which stacked plots are drawn. This is more or less a technical detail which should not be changed in any normal case.

The motivation is as follows: suppose multiple `\addplot` commands are stacked on top of each other and they are processed in the order of appearance. Then, the second plot could easily draw its lines (or fill area) on top of the first one - hiding its marker or line completely. Therefore, PGFPLOTS reverses the sequence of drawing commands.

This has the side-effect that any normal TikZ-paths inside of an axis will also be processed in reverse sequence.

`/pgfplots/xbar stacked=plus|minus` (style, default plus)

A figure-wide style which enables stacked horizontal bars (i.e. `xbar` and `stack plots=x`). It also adjusts the legend and tick appearance and assigns a useful `cycle list`.

`/pgfplots/ybar stacked=plus|minus` (style, default plus)

A figure-wide style which enables stacked vertical bars (i.e. `ybar` and `stack plots=y`). It also adjusts the legend and tick appearance and assigns a useful `cycle list`.

`/pgfplots/xbar interval stacked=plus|minus` (style, default plus)

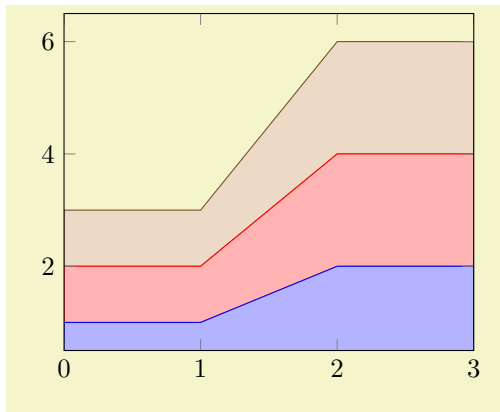
A style similar to `/pgfplots/xbar stacked` for the interval based bar plot variant.

`/pgfplots/ybar interval stacked=plus|minus` (style, default plus)

A style similar to `/pgfplots/ybar stacked` for the interval based bar plot variant.

4.4.9 Area Plots

Area plots are a combination of `\closedcycle` and `stack plots`. They can be combined with any other plot type.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    stack plots=y,
    area style,
    enlarge x limits=false]
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
  \end{axis}
\end{tikzpicture}
```

Area plots may need modified legends, for example using the [area legend](#) key. Furthermore, one may want to consider the [axis on top](#) key such that filled areas do not overlap ticks and grid lines.

`/pgfplots/area style`

(style, no value)

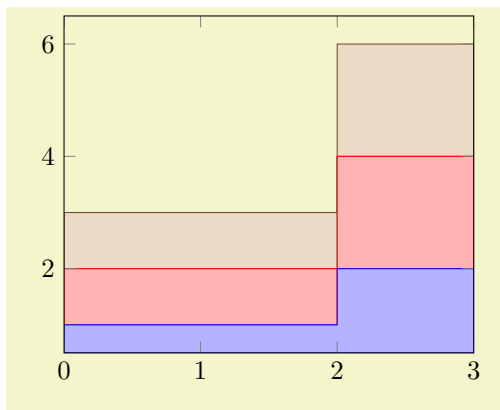
A style which sets

```
\pgfplotsset{
  /pgfplots/area style/.style={%
    area cycle list,
    area legend,
    axis on top,
  }}
}
```

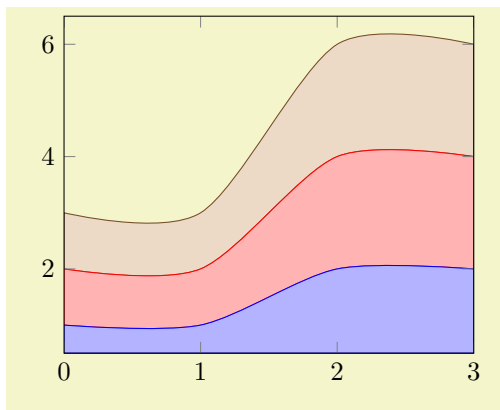
`/pgfplots/area cycle list`

(style, no value)

A style which installs a [cycle list](#) suitable for area plots. The initial configuration of this style simply invokes the [bar cycle list](#) which does also provide filled plot styles.



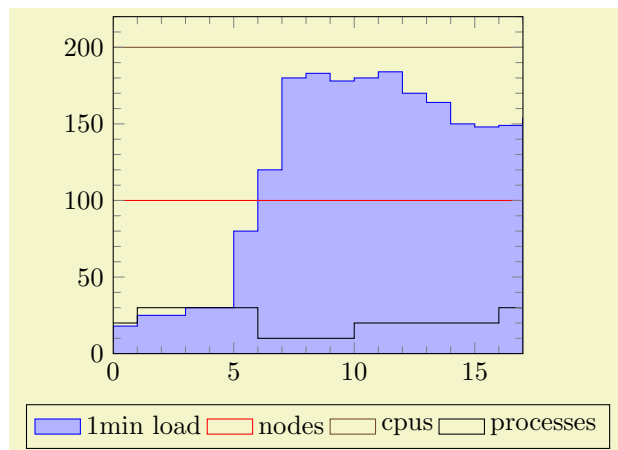
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    const plot,
    stack plots=y,
    area style,
    enlarge x limits=false]
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    smooth,
    stack plots=y,
    area style,
    enlarge x limits=false]
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
    \addplot coordinates
      {(0,1) (1,1) (2,2) (3,2)}
    \closedcycle;
  \end{axis}
\end{tikzpicture}
```

time	1minload	nodes	cpus	processes	memused	memcached	membuf	memtotal
0	18	100	200	20	15	45	1	150
1	25	100	200	30	20	45	2	150
2	25	100	200	30	21	42	2	150
3	30	100	200	30	20	40	2	150
4	30	100	200	30	19	40	1	150
5	80	100	200	30	20	40	3	150
6	120	100	200	10	3	40	3	150
7	180	100	200	10	4	41	3	150
8	183	100	200	10	3	42	2	150
9	178	100	200	10	2	41	1	150
10	180	100	200	20	15	45	2	150
11	184	100	200	20	20	45	3	150
12	170	100	200	20	22	47	4	150
13	164	100	200	20	24	50	4	150
14	150	100	200	20	25	52	3	150
15	148	100	200	20	26	53	2	150
16	149	100	200	30	30	54	2	150
17	154	100	200	30	35	55	1	150

```
\pgfplotstableread{pgfplots.timeseries.dat}\loadedtable
\pgfplotstabletypeset\loadedtable
```



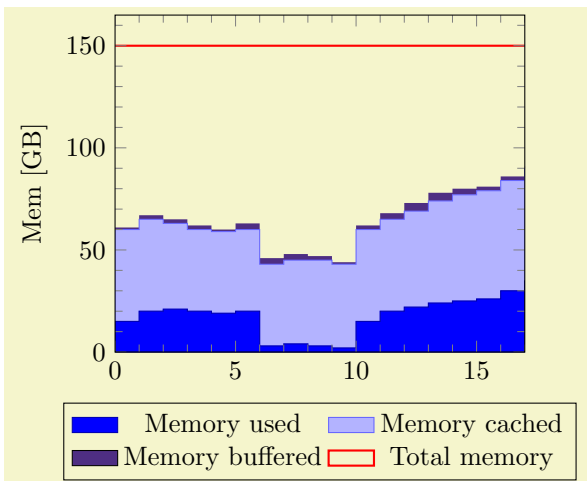
```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotstableread
{pgfplots.timeseries.dat}
{\loadedtable}

\begin{tikzpicture}
  \begin{axis}[
    ymin=0,
    minor tick num=4,
    enlarge x limits=false,
    axis on top,
    every axis plot post/.append style={mark=none},
    const plot,
    legend style={
      area legend,
      at={(0.5,-0.15)},
      anchor=north,
      legend columns=-1}]

    \addplot[draw=blue,fill=blue!30!white]
      table[x=time,y=iminload] from \loadedtable
      \closedcycle;
    \addplot table[x=time,y=nodes] from \loadedtable;
    \addplot table[x=time,y=cpus] from \loadedtable;
    \addplot table[x=time,y=processes]
      from \loadedtable;
    \legend{imin load,nodes,cpus,processes}
  \end{axis}
\end{tikzpicture}

```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotstableread{pgfplots.timeseries.dat}\loadedtable

\begin{tikzpicture}
  \begin{axis}[
    ymin=0,
    minor tick num=4,
    enlarge x limits=false,
    const plot,
    axis on top,
    stack plots=y,
    cycle list={%
      {blue!70!black,fill=blue},%
      {blue!60!white,fill=blue!30!white},%
      {draw=none,fill={rgb:red,138;green,82;blue,232}},%
      {red,thick}%
    },
    ylabel={Mem [GB]},
    legend style={
      area legend,
      at={(0.5,-0.15)},
      anchor=north,
      legend columns=2}
  ]

  \addplot table[x=time,y=memused]      from \loadedtable \closedcycle;
  \addplot table[x=time,y=memcached]   from \loadedtable \closedcycle;
  \addplot table[x=time,y=membuf]      from \loadedtable \closedcycle;
  \addplot+[stack plots=false]
    table[x=time,y=memtotal]          from \loadedtable;
  \legend{Memory used,Memory cached,Memory buffered,Total memory}
\end{axis}
\end{tikzpicture}
```

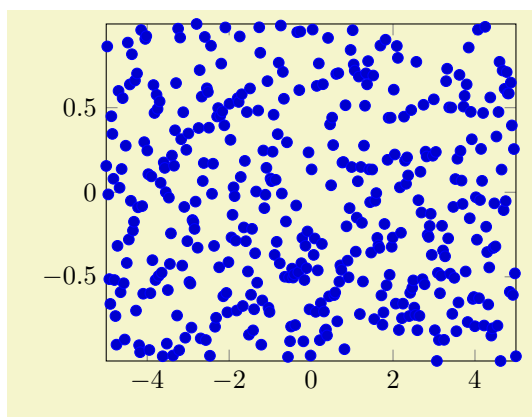
4.4.10 Scatter Plots

The most simple scatter plots produce the same as the line plots above – but they contain only markers. They are enabled using the `only marks` key of TikZ.

`/tikz/only marks` (no value)

`\addplot+[only marks]`

Draws a simple scatter plot: all markers have the same appearance.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[enlargelimits=false]
    \addplot+[only marks,samples=400]
      {rand};
  \end{axis}
\end{tikzpicture}
```

The `only marks` visualization style simply draws marks at every coordinate. Marks can be set with `mark=<mark name>` and marker options like size and color can be specified using the `mark options=<style options>` key (or by modifying the `every mark` style). The available markers along with the accepted style options can be found in Section 4.6 on page 134.

More sophisticated scatter plots change the marker appearance for each data point. An example is that marker colors depend on the magnitude of function values $f(x)$ or other provided coordinates. The term “scatter plot” will be used for this type of plot in the following sections.

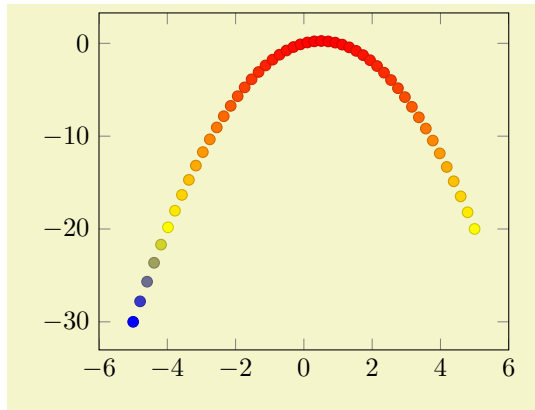
Scatter plots require “source” coordinates. These source coordinates can be the y coordinate, or explicitly provided additional values.

`/pgfplots/scatter`

(no value)

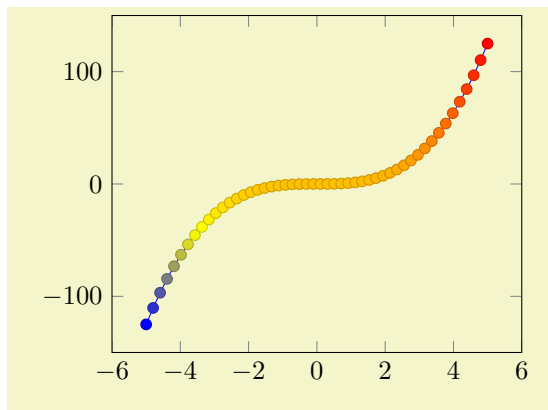
`\addplot+[scatter]`

Enables marker appearance modifications. The default implementation acquires “source coordinates” for every data point (see `scatter src` below) and maps them linearly into the current color map. The resulting color is used as draw and fill color of the marker.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot+[scatter,only marks,
      samples=50,scatter src=y]
      {x-x^2};
  \end{axis}
\end{tikzpicture}
```

The key `scatter` is simply a boolean variable which enables marker modifications. It applies only to markers and it can be combined with any other plot type.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot+[scatter,
      samples=50,scatter src=y]
      {x^3};
  \end{axis}
\end{tikzpicture}
```

Scatter plots can be configured using a set of options. One of them is mandatory, the rest allows fine grained control over marker appearance options.

`/pgfplots/scatter src=none|<expression>|x|y|z|f(x)|explicit|explicit symbolic` (initially none)

This key is necessary for any scatter plot and it is set to $f(x)$ as soon as `scatter` is activated and no different choice has been made. It needs to be provided as *<option>* for `\addplot` to configure the value used to determine marker appearances. Actually, `scatter src` is nothing but an alias for `point meta`, so the main documentation for this key is on page 157. However, we summarize the choices here together with scatter plot examples.

Usually, `scatter src` provides input data (numeric or string type) which is used to determine colors and other style options for markers. The default configuration expects numerical data which is mapped linearly into the current color map.

The value of `scatter src` determines how to get this data: the choices `x`, `y` and `z` will use either the x , y or z coordinates to determine marker options. Any coordinate filters, logarithms or stacked-plot computations have already been applied to these values (use `rawx`, `rawy` and `rawz` for unprocessed values). The special choice $f(x)$ is the same as `y` for two dimensional plots and the same as `z` for three dimensional plots. The choice `explicit` expects the scatter source data as additional coordinate from the coordinate input streams (see Section 4.2.1 for how to provide input meta data or below for some

small examples). They will be treated as numerical data. The choice `explicit symbolic` also expects scatter source data as additional meta information for each input coordinate, but it treats them as strings, not as numerical data. Consequently, no arithmetics is performed. It is the task of the scatter plot style to do something with it. See, for example, the `scatter/classes` style below. Finally, it is possible to provide an arbitrary mathematical expression which involves zero, one or more of the values `x` (the current x coordinate), `y` (the current y coordinate) or `z` (the current z coordinate, if any).

If data is read from tables, mathematical expressions might also involve `\thisrow{<column name>}` or `\thisrowno{<column index>}` to access any of the table cells in the current row.

Here are examples for how to provide data for the choices `explicit` and `explicit symbolic`.

```
\begin{tikzpicture}
  \begin{axis}
    % provide color data explicitly using [<data>]
    % behind coordinates:
    \addplot+[scatter,scatter src=explicit]
      coordinates {
        (0,0) [1.0e10]
        (1,2) [1.1e10]
        (2,3) [1.2e10]
        (3,4) [1.3e10]
        % ...
      };

    % Assumes a datafile.dat like
    % xcolname ycolname colordata
    % 0        0        0.001
    % 1        2        0.3
    % 2        2.1      0.4
    % 3        3        0.5
    % ...
    % the file may have more columns.
    \addplot+[scatter,scatter src=explicit]
      table[x=xcolname,y=ycolname,meta=colordata]
        {datafile.dat};

    % Same data as last example:
    \addplot+[scatter,scatter src=\thisrow{colordata}+\thisrow{ycolname}]
      table[x=xcolname,y=ycolname]
        {datafile.dat};

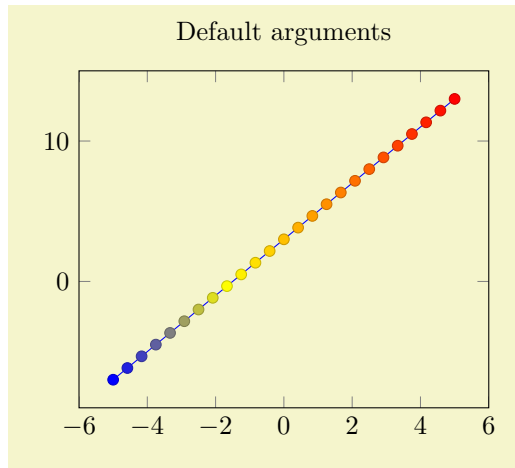
    % Assumes a datafile.dat like
    % 0        0        0.001
    % 1        2        0.3
    % 2        2.1      0.4
    % 3        3        0.5
    % ...
    % the first three columns will be used here:
    \addplot+[scatter,scatter src=explicit]
      file {datafile.dat};
  \end{axis}
\end{tikzpicture}
```

Please note that `scatter src≠none` results in computational work even if `scatter=false`.

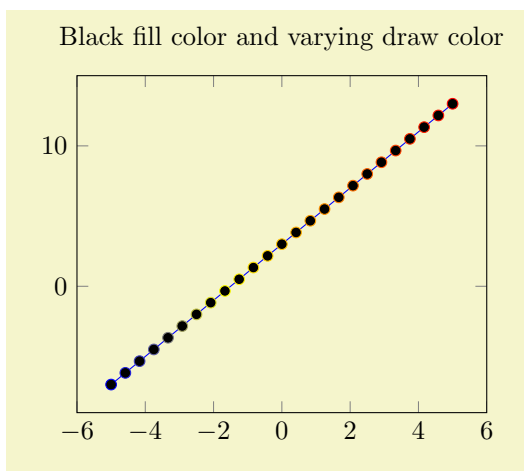
`/pgfplots/scatter/use mapped color={<options for each marker>}` (style, initially `draw=mapped color!80!black,fill=mapped color`)

This style is installed by default. When active, it recomputes the color `mapped color` for every processed point coordinate by transforming the `scatter src` coordinates into the current color map linearly. Then, it evaluates the options provided as `<options for each marker>` which are expected to depend on `mapped color`.

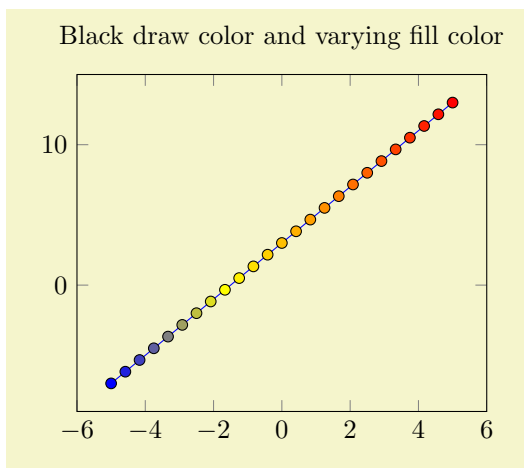
The user interface for color maps is described in Section 4.6.6.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[title=Default arguments]
\addplot+[scatter,scatter src=y]
{2*x+3};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
title=Black fill color and varying draw color,
scatter/use mapped color=
{draw=mapped color,fill=black}]
\addplot+[scatter,scatter src=y]
{2*x+3};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
title=Black draw color and varying fill color,
scatter/use mapped color=
{draw=black,fill=mapped color}]
\addplot+[scatter,scatter src=y]
{2*x+3};
\end{axis}
\end{tikzpicture}
```

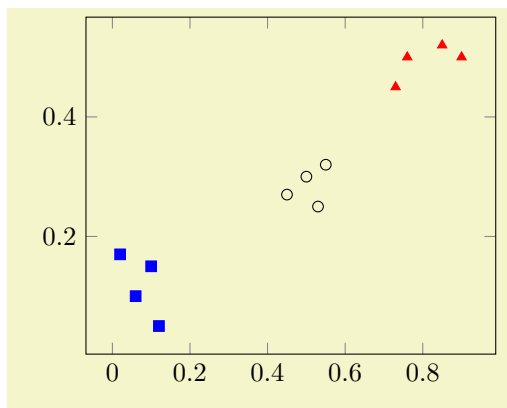
This key is actually a style which redefines `@pre marker code` and `@post marker code` (see below).

Remark: The style `use mapped color` redefines `@pre marker code` and `@post marker code`. There is a starred variant `use mapped color*` which *appends* the functionality while keeping the old marker code.

`/pgfplots/scatter/classes={\styles for each class name}`

A scatter plot style which visualizes points using several classes. The style assumes that every point coordinate has a class label attached, that means the choice `scatter src=explicit symbolic` is as-

sumed¹⁵. A class label can be a number, but it can also be a symbolic constant. Given class labels for every point, `<styles for each class name>` contains a comma-separated list which associates appearance options to each class label.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[scatter/classes={
  a={mark=square*,blue},%
  b={mark=triangle*,red},%
  c={mark=o,draw=black}}]

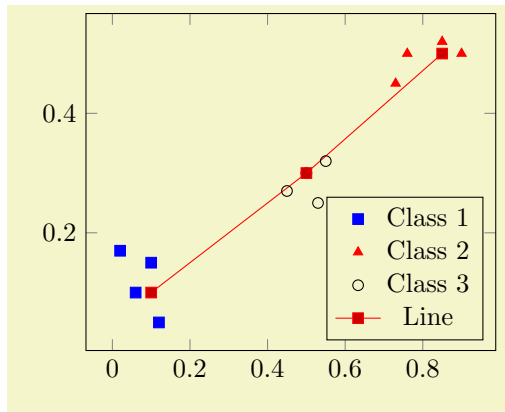
% \addplot[] is better than \addplot+[] here:
% it avoids scalings of the cycle list
\addplot[scatter,only marks,
  scatter src=explicit symbolic]
  coordinates {
    (0.1,0.15) [a]
    (0.45,0.27) [c]
    (0.02,0.17) [a]
    (0.06,0.1) [a]
    (0.9,0.5) [b]
    (0.5,0.3) [c]
    (0.85,0.52) [b]
    (0.12,0.05) [a]
    (0.73,0.45) [b]
    (0.53,0.25) [c]
    (0.76,0.5) [b]
    (0.55,0.32) [c]
  };
\end{axis}
\end{tikzpicture}
```

In this example, the coordinate (0.1,0.15) has the associated label ‘a’ while (0.45,0.27) has the label ‘c’ (see Section 4.2 for details about specifying point meta data). Now, the argument to `scatter/classes` contains styles for every label – for label ‘a’, square markers will be drawn in color blue.

The generation of a legend works as for a normal plot – but `scatter/classes` requires one legend entry for every provided class. It communicates the class labels to the legend automatically. It works as if there had been different `\addplot` commands, one for every class label.

It is also possible to provide `scatter/classes` as argument to a single plot, allowing different scatter plots in one axis.

¹⁵If `scatter src` is not `explicit symbolic`, we expect a numeric argument which is rounded to the nearest integer. The resulting integer is used as a class label. If that fails, the numeric argument is truncated to the nearest integer. If that fails as well, the point has no label.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=south east]
% The data file contains:
% x      y      label
% 0.1    0.15   a
% 0.45   0.27   c
% 0.02   0.17   a
% 0.06   0.1    a
% 0.9    0.5    b
% 0.5    0.3    c
% 0.85   0.52   b
% 0.12   0.05   a
% 0.73   0.45   b
% 0.53   0.25   c
% 0.76   0.5    b
% 0.55   0.32   c
\addplot[
% clickable coords={\thisrow{label}},
scatter/classes={
a={mark=square*,blue},%
b={mark=triangle*,red},%
c={mark=o,draw=black,fill=black}%
},
scatter,only marks,
scatter src=explicit symbolic]
table[x=x,y=y,meta=label]
{plotdata/scattercl.dat};

\addplot coordinates
{(0.1,0.1) (0.5,0.3) (0.85,0.5)};
\legend{Class 1,Class 2,Class 3,Line}
\end{axis}
\end{tikzpicture}
```

In general, the format of $\langle \text{styles for each class name} \rangle$ is a comma separated list of $\langle \text{label} \rangle = \{ \langle \text{style options} \rangle \}$.

Attention: The keys `every mark` and `mark options` have *no effect* when used inside of $\langle \text{styles for each class name} \rangle$! So, instead of assigning `mark options`, you can simply provide the options directly. They apply only to markers anyway.

Remark: To use `\label` and `\ref` in conjunction with `scatter/classes`, you can provide the class labels as optional arguments to `\label` in square brackets:

```
\addplot[
  scatter/classes={
    a={mark=square*,blue},%
    b={mark=triangle*,red},%
    c={mark=o,draw=black,fill=black}%
  },
  scatter,only marks,
  scatter src=explicit symbolic]
% [and coordinate input here... ]
;

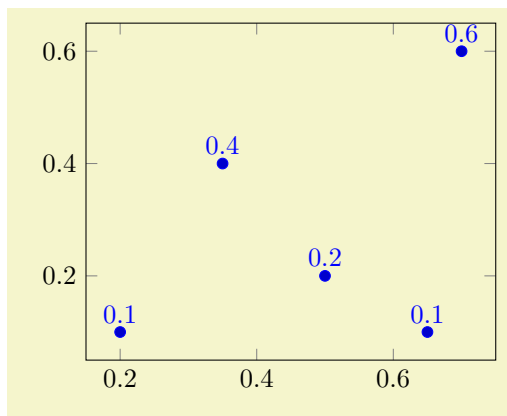
\label[a]{label:for:first:class}
\label[b]{label:for:second:class}
\label[c]{label:for:third:class}
...
First class is \ref{label:for:first:class}, second is \ref{label:for:second:class}.
```

Remark: It is possible to click into the plot to display labels with mouse popups, see the `clickable coords` key of the `clickable` library.

Remark: The style `scatter/classes` redefines `@pre` marker code and `@post` marker code. There is a starred variant `scatter/classes*` which *appends* the functionality while keeping the old marker code.

```
/pgfplots/nodes near coords={\langle content \rangle} (default \pgfmathprintnumber\pgfplotspointmeta)
/pgfplots/nodes near coords*={\langle content \rangle} (default \pgfmathprintnumber\pgfplotspointmeta)
```

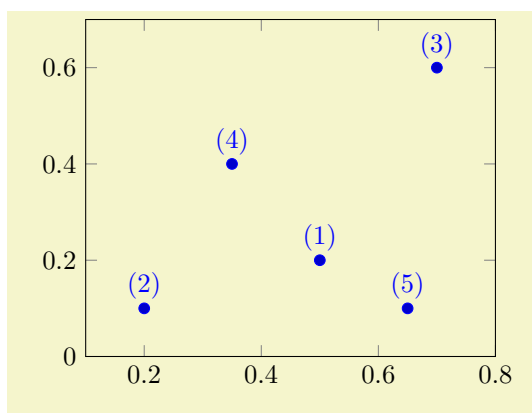
A `scatter` plot style which places text nodes near every coordinate.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords]
\addplot+[only marks] coordinates {
(0.5,0.2) (0.2,0.1) (0.7,0.6)
(0.35,0.4) (0.65,0.1)};
\end{axis}
\end{tikzpicture}
```

The $\langle content \rangle$ is, if nothing else has been specified, the content of the “point meta”, displayed using the default $\langle content \rangle = \texttt{\backslash pgfmathprintnumber\pgfplotspointmeta}$. The macro `\pgfplotspointmeta` contains whatever has been selected by the `point meta` key, it defaults to the y coordinate for two dimensional plots and the z coordinate for three dimensional plots.

Since `point meta=explicit symbolic` allows to treat string data, you can provide textual descriptions which will be shown inside of the generated nodes¹⁶:

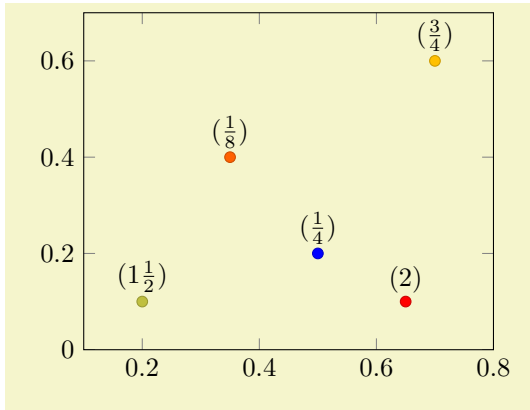


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords,enlargelimits=0.2]
\addplot+[only marks,
point meta=explicit symbolic]
coordinates {
(0.5,0.2) [(1)]
(0.2,0.1) [(2)]
(0.7,0.6) [(3)]
(0.35,0.4) [(4)]
(0.65,0.1) [(5)]
};
\end{axis}
\end{tikzpicture}
```

The square brackets are the way to provide explicit `point meta` for `plot coordinates`. Please refer to the documentation of `plot file` and `plot table` for how to get point meta from files.

The $\langle content \rangle$ can also depend on something different than `\pgfplotspointmeta`. But since $\langle content \rangle$ is evaluated during `\end{axis}`, PGFLOTS might not be aware of any special information inside of $\langle content \rangle$ – you’ll need to communicate it to PGFLOTS with the `visualization depends on` key as follows:

¹⁶In this case, the `\pgfmathprintnumber` will be skipped automatically.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[enlargelimits=0.2]
    \addplot[
      scatter,mark=*,only marks,
      % we use 'point meta' as color data...
      point meta=\thisrow{color},
      % ... therefore, we can't use it as argument for nodes near coords ...
      nodes near coords*={\pgfmathprintnumber[frac]{\myvalue}$},
      % ... which requires to define a visualization dependency:
      visualization depends on={\thisrow{myvalue} \as \myvalue},
    ]
    table {
      x      y      color  myvalue
      0.5    0.2    1      0.25
      0.2    0.1    2      1.5
      0.7    0.6    3      0.75
      0.35   0.4    4      0.125
      0.65   0.1    5      2
    };
  \end{axis}
\end{tikzpicture}
```

The example uses a `scatter` plot to get different colors, where the `scatter src` (or, equivalently, `point meta`) is already used to define the markers color. In addition to the colored `scatter` plot, we'd like to add `nodes near coords`, where the displayed nodes should contain `\thisrow{myvalue}`. To do so, we define `scatter,point meta=\thisrow{color}` (just as described in the previous sections). Furthermore, we use `nodes near coords*` in order to *combine* different `scatter` styles (see below for details). The value for `nodes near coords*` depends on `\thisrow{myvalue}`, but we can't use `\pgfplotspointmeta` (which is already occupied). Thus, we communicate the additional input data by means of `visualization depends on={\thisrow{myvalue} \as \myvalue}`. The statement defines a new macro, `\myvalue`, and assigns the value `\thisrow{myvalue}`. Furthermore, it configures PGF-PLOTS to remember this particular macro and its contents until `\end{axis}` (see the documentation for `visualization depends on` for details).

The style `nodes near coords` might be useful for bar plots, see `ybar` for an example of `nodes near coords`.

Remarks and Details:

- `nodes near coords` uses the same options for line styles and colors as the current plot. This may be changed using the style `every node near coord`, see below.
- `nodes near coords` is actually one of the `scatter` plot styles. It redefines `scatter/@pre marker code` to generate several TikZ `\node` commands.

In order to use `nodes near coords` together with other `scatter` plot styles (like `scatter/use mapped color` or `scatter/classes`), you may append a star to each of these keys. The variant `nodes near coords*` will *append* code to `scatter/@pre marker code` without overwriting the previous value.

- Consider using `enlargelimits` together with `nodes near coords` if text is clipped away.

- Currently `nodes near coords` does not work satisfactorily for `ybar interval` or `xbar interval`, sorry.

`/pgfplots/every node near coord` (style, no value)

A style used for every node generated by `nodes near coords`. It is initially empty.

`/pgfplots/nodes near coords align={alignment method}` (initially auto)

Specifies how to align nodes generated by `nodes near coords`.

Possible choices for *alignment method* are

auto uses **horizontal** if the x coordinates are shown or **vertical** in all other cases. This checks the current value of `point meta`.

horizontal uses **left** if `\pgfplotspointmeta` < 0 and **right** otherwise.

vertical uses **below** if `\pgfplotspointmeta` < 0 and **above** otherwise.

It is also possible to provide any TikZ alignment option such as `anchor=north east`, `below` or something like that. It is also allowed to provide multiple options.

`/pgfplots/scatter/@pre marker code/.code={...}`

`/pgfplots/scatter/@post marker code/.code={...}`

These two keys constitute the public interface which determines the marker appearance depending on scatter source coordinates.

Redefining them allows fine grained control even over marker types, line styles and colors.

The scatter plot algorithm works as follows:

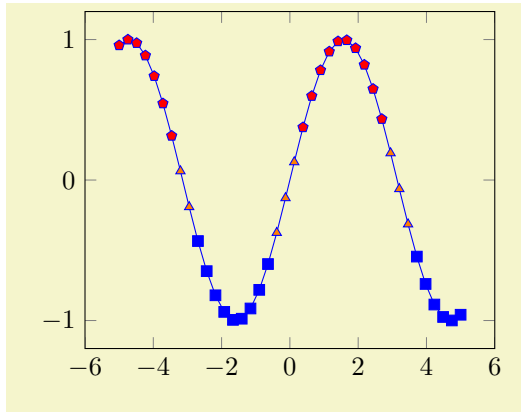
1. The scatter source coordinates form a data stream whose data limits are computed additionally to the axis limits. This step is skipped for **symbolic** meta data.
2. Before any markers are drawn, a linear coordinate transformation from these data limits to the interval $[0.0, 1000.0]$ is initialised.
3. Every scatter source coordinate¹⁷ will be transformed linearly and the result is available as macro `\pgfplotspointmetatransformed` $\in [0.0, 1000.0]$.
The decision is thus based on per thousands of the data range. The transformation is skipped for **symbolic** meta data (and the meta data is simply contained in the mentioned macro).
4. The PGF coordinate system is translated such that `(0pt,0pt)` is the plot coordinate.
5. The code of `scatter/@pre marker code` is evaluated (without arguments).
6. The standard code which draws markers is evaluated.
7. The code of `scatter/@post marker code` is evaluated (without arguments).

The idea is to generate a set of appearance keys which depends on `\pgfplotspointmetatransformed`. Then, a call to `\scope[generated keys]` as `@pre` code and the associated `\endscope` as `@post` code will draw markers individually using `[generated keys]`.

A technical example is shown below. It demonstrates how to write user defined routines, in this case a three-class system¹⁸.

¹⁷During the evaluation, the public macros `\pgfplotspointmeta` and `\pgfplotspointmetarange` indicate the source coordinate and the source coordinate range in the format $a : b$ (for log-axis, they are given in fixed-point representation and for linear axes in floating point).

¹⁸Please note that you don't need to copy this particular example: the multiple-class example is also available as predefined style `scatter/classes`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
% Low-Level scatter plot interface Example:
% use three different marker classes
% 0% - 30% : first class
% 30% - 60% : second class
% 60% - 100% : third class
\begin{axis}[
scatter/@pre marker code/.code={%
\ifdim\pgfplotspointmetatransformed pt<300pt
\def\markopts{mark=square*,fill=blue}%
\else
\ifdim\pgfplotspointmetatransformed pt<600pt
\def\markopts{mark=triangle*,fill=orange}%
\else
\def\markopts{mark=pentagon*,fill=red}%
\fi
\fi
\expandafter\scope\expandafter[\markopts]
},%
scatter/@post marker code/.code={%
\endscope
}]

\addplot+[scatter,scatter src=y,
samples=40]
{sin(deg(x))};

\end{axis}
\end{tikzpicture}
```

Please note that `\ifdim` compares TeX lengths, so the example employs the suffix `pt` for any number used in this context. That doesn't change the semantics. The two (!) `\expandafter` constructions make sure that `\scope` is invoked with the *content* of `\markopts` instead of the macro name `\markopts`.

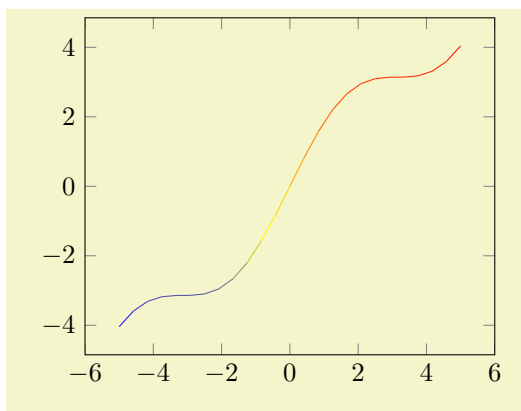
4.4.11 1D Colored Mesh Plots

`/pgfplots/mesh`

(no value)

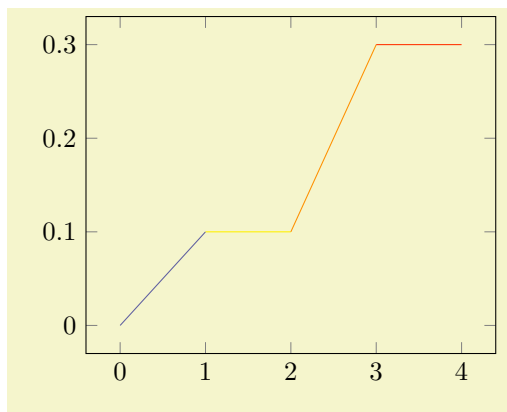
`\addplot+[mesh]`

Uses the current color map to determine colors for each fixed line segment. Each line segment will get the same color.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot[mesh] {x+sin(deg(x))};
\end{axis}
\end{tikzpicture}
```

The color data is per default the y value of the plot. It can be reconfigured using the `point meta` key (which is actually the same as `scatter src`). The following example provides the color data explicitly for `plot coordinates`, using the square bracket notation.



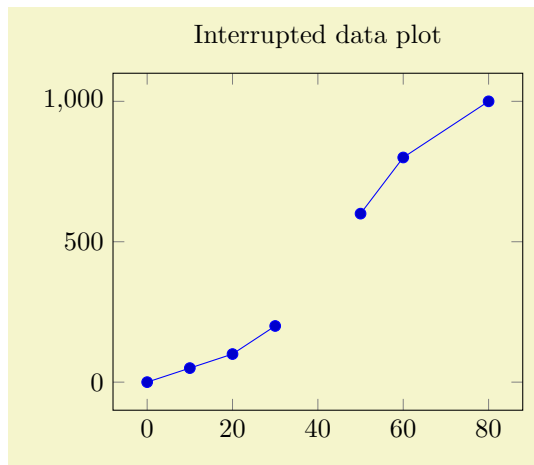
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[mesh,point meta=explicit]
      coordinates {
        (0,0) [0]
        (1,0.1) [1]
        (2,0.1) [2]
        (3,0.3) [3]
        (4,0.3) [4]
      };
  \end{axis}
\end{tikzpicture}
```

This one-dimensional `mesh` plot is actually a special case of the twodimensional mesh plots, so more detailed configuration, including how to change the color data, can be found in Section 4.5.5.

4.4.12 Interrupted Plots

Sometimes it is desirable to draw parts of a single plot separately, without connection between the parts (discontinuities). PGFLOTS offers two ways to generate interrupted plots: either using `empty lines` or by providing `unbounded coords`.

The first way is simple; it needs no extra key (only `\pgfplotsset{compat=1.4}` or newer in your preamble):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title=Interrupted data plot

  \addplot coordinates {
    (0,0) (10,50) (20,100) (30,200)

    (50,600) (60,800) (80,1000)
  };
\end{axis}
\end{tikzpicture}
```

Here, PGFLOTS runs with the default configuration `empty line=auto` which interpretes empty lines as “jump” markers. This works for any data input method, i.e. using `\addplot coordinates`, `\addplot table`, and `\addplot file`.

The second way to generate interrupted plots addresses the case where `empty lines` are unavailable or impossible (due to limitations of the tool generating the data file, for example). In this case, interrupted plots can be achieved using the `unbounded coords` key combined with coordinate values `nan`, `inf` or `-inf`.

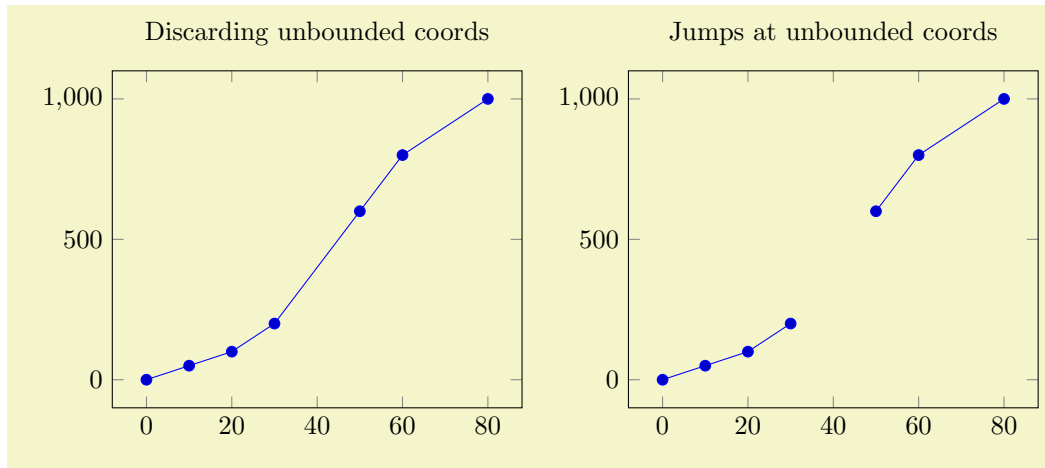
`/pgfplots/unbounded coords=discard|jump` (initially `discard`)

This key configures what to do if one or more coordinates of a single point are unbounded. Here, unbounded means it is either $\pm\infty$ (`+inf` or `-inf`) or it has the special “not-a-number” value `nan`.

The initial setting `discard` discards the complete point and a warning is issued in the log file¹⁹. This setting has the same effect as if the unbounded point did not occur: PGFLOTS will interpolate between the bounded adjacent points.

The alternative `jump` allows interrupted plots: it provides extra checking for these coordinates and does not interpolate over them; only those line segments which are adjacent to unbounded coordinates will be skipped.

¹⁹The warning can be disabled with `filter discard warning=false`.



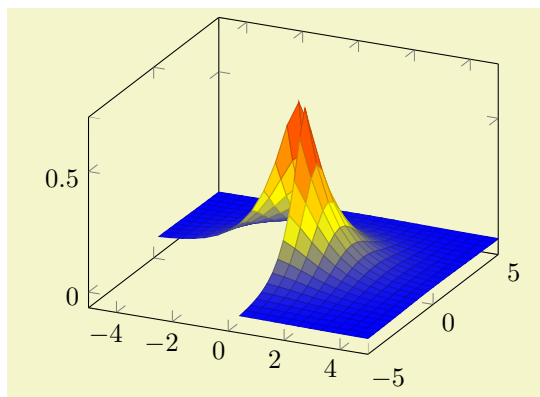
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title=Discarding unbounded coords,
  unbounded coords=discard

  \addplot coordinates {
    (0,0) (10,50) (20,100) (30,200)
    (40,inf) (50,600) (60,800) (80,1000)
  };
\end{axis}
\end{tikzpicture}
\begin{tikzpicture}
\begin{axis}[
  title=Jumps at unbounded coords,
  unbounded coords=jump

  \addplot coordinates {
    (0,0) (10,50) (20,100) (30,200)
    (40,inf) (50,600) (60,800) (80,1000)
  };
\end{axis}
\end{tikzpicture}
```

For plot expression and its friends, it is more likely to get very large floating point numbers instead of `inf`. In this case, consider using the `restrict x to domain` key described on page 302.

The `unbounded coords=jump` method does also work for mesh/surface plots: every face adjacent to an unbounded coordinate will be discarded in this case. The following example sets up a (cryptic) coordinate filter which cuts out a quarter of the domain and replaces its values with `nan`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  unbounded coords=jump,
  % A technical filter to cut out
  % the x<0 and y<0 edge.
  filter point/.code={%
    \pgfmathparse
      {\pgfkeysvalueof{/data point/x}<0}%
    \ifpgfmathfloatcomparison
      \pgfmathparse
        {\pgfkeysvalueof{/data point/y}<0}%
      \ifpgfmathfloatcomparison
        \pgfkeyssetvalue{/data point/x}{nan}%
      \fi
    \fi
  },
]
\addplot3[surf] {exp(-sqrt(x^2 + y^2))};
\end{axis}
\end{tikzpicture}
```

More about this coordinate filtering can be found in Section 4.22 “Skipping Or Changing Coordinates – Filters”.

4.4.13 Patch Plots

Patch Plots visualize a sequence of one or more triangles (or other sorts of patches). These triangles can be drawn with a single color (`shader=flat` and `shader=faceted interp`) or with interpolated colors (`shader=interp`).

There are both two- and three-dimensional patch plots, both with the same interface and the same keys. Therefore, the reference documentation for patch plots can be found in Section 4.5.12 together with three-dimensional patch plots.

4.5 Three Dimensional Plot Types

PGFPLOTS provides three dimensional visualizations like scatter, line, mesh or surface plots. This section explains the methods to provide input coordinates and how to use the different plot types.

4.5.1 Before You Start With 3D

Before we delve into the capabilities of PGFPLOTS for three dimensional visualization, let me start with some preliminary remarks. The reason to use PGFPLOTS for three dimensional plots are similar to those of normal, two dimensional plots: the possibility to get consistent fonts and document consistent styles combined with high-quality output.

While this works very nice for (not too complex) two dimensional plots, it requires considerably more effort than non-graphical documents. This is even more so for three dimensional plots. In other words: PGFPLOTS' three dimensional routines are slow. There are reasons for this and some of them may vanish in future versions. But one of these reasons is that T_EX has never been designed for complex visualisation techniques. Consider the image externalization routines mentioned in Section 7.1, in particular the `external` library to reduce typesetting time. Besides the speed limitations, three dimensional plots reach memory limits easily. Therefore, the plot complexity of three dimensional plots is limited to relatively coarse resolutions. Section 7.1 also discusses methods to extend the initial T_EX memory limits.

Another issue which arises in three dimensional visualization is depth. PGFPLOTS supports *z* buffering techniques up to a certain extend. It works pretty well for single scatter plots (`z buffer=sort`), mesh or surface plots (`z buffer=auto`) or parametric mesh and surface plots (`z buffer=sort`). However, it can't combine different `\addplot` commands, those will be drawn in the order of appearance. You may encounter the limitations sometimes. Maybe it will be improved in future versions.

If you decide that you need high complexity, speed and 100% reliable *z* buffers (depth information), you should consider using other visualization tools and return to PGFPLOTS in several years. If you can wait for a complex picture and you don't even see the limitations arising from *z* buffering limitations, you should use PGFPLOTS. Again, consider using the automatic picture externalization with the `external` library discussed in Section 7.1.

Enough for now, let's continue.

4.5.2 The `\addplot3` Command: Three Dimensional Coordinate Input

`\addplot3`[*options*] *input data* *trailing path commands*;

The `\addplot3` command is the main interface for any three dimensional plot. It works in the same way as its two dimensional variant `\addplot` which has been described in all detail in Section 4.2 on page 24.

The `\addplot3` command accepts the same input methods as the `\addplot` variant, including expression plotting, coordinates, files and tables. However, a third coordinate is necessary for each of these methods which is usually straight-forward and is explained in all detail in the following.

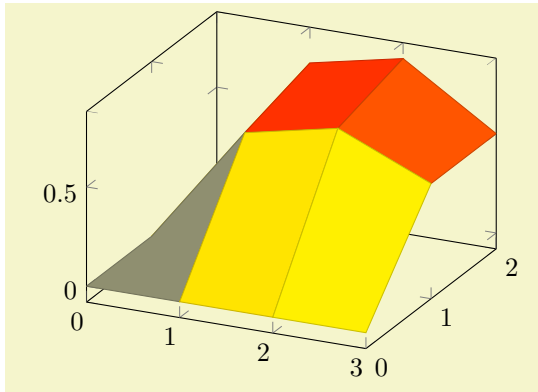
Furthermore, `\addplot3` has a way to decide whether a *line* visualization or a *mesh* visualization has to be done. The first one is a map from one dimension into $\mathbb{R}^3$ and the latter one a map from two dimensions to $\mathbb{R}^3$. Here, the keys `mesh/rows` and `mesh/cols` are used to define mesh sizes (matrix sizes). Usually, you don't have to care about that because the coordinate input routines already allow either one- or two-dimensional structure.

`\addplot3` *coordinates* *{(coordinate list)}*;

`\addplot3[options] coordinates {coordinate list} (trailing path commands);`

The `\addplot3 coordinates` method works like its two-dimensional variant, `\addplot coordinates` which is described in all detail on page 27:

A long list of coordinates ($\langle x \rangle, \langle y \rangle, \langle z \rangle$) is expected, separated by white spaces. The input list can be either an unordered series of coordinates, for example for scatter or line plots. It can also have matrix structure, in which case an **empty line** (which is equivalent to “`\par`”) marks the end of one matrix row. Matrix structure can also be provided if one of `mesh/rows` or `mesh/cols` is provided explicitly.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    % this yields a 3x4 matrix:
    \addplot3[surf] coordinates {
      (0,0,0) (1,0,0) (2,0,0) (3,0,0)

      (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)

      (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
    };
  \end{axis}
\end{tikzpicture}
```

Here, `\addplot3` reads a matrix with three rows and four columns. The **empty lines** separate one row from the following.

As for the two-dimensional `plot coordinates`, it is possible to provide (constant) mathematical expressions inside of single coordinates. The syntax ($\langle x \rangle, \langle y \rangle, \langle z \rangle$) [*meta*] can be used just as for two dimensional `plot coordinates` to provide explicit color data; error bars are also supported.

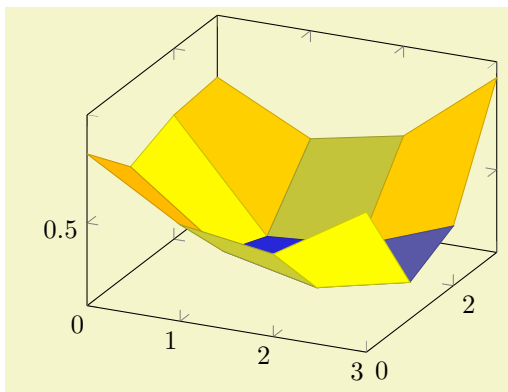
`\addplot3 file {name};`

`\addplot3[options] file {name} (trailing path commands);`

The `\addplot3 file` input method is the same as `\addplot file` – it only expects one more coordinate. Thus, the input file contains x_i in the first column, y_i in the second column and z_i in the third.

A further column is read after z_i if `point meta=explicit` has been requested, see the documentation of `\addplot file` on page 52 for details.

As for `\addplot3 coordinates`, an **empty line** in the file marks the end of one matrix row.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    % We have 'plotdata/first3d.dat' with
    % -----
    % 0 0 0.8
    % 1 0 0.56
    % 2 0 0.5
    % 3 0 0.75
    %
    % 0 1 0.6
    % 1 1 0.3
    % 2 1 0.21
    % 3 1 0.3
    %
    % 0 2 0.68
    % 1 2 0.22
    % 2 2 0.25
    % 3 2 0.4
    %
    % 0 3 0.7
    % 1 3 0.5
    % 2 3 0.58
    % 3 3 0.9
    % -> yields a 4x4 matrix:
    \addplot3[surf] file {plotdata/first3d.dat};
  \end{axis}
\end{tikzpicture}
```

For matrix data in files, it is important to specify the ordering in which the matrix entries have been written. The default configuration is `mesh/ordering=x varies`, so you need to change it to `mesh/ordering=y varies` in case you have columnwise ordering.

```
\addplot3 table [⟨column selection⟩]{⟨file⟩};
\addplot3[⟨options⟩] table [⟨column selection⟩]{⟨file⟩} ⟨trailing path commands⟩;
```

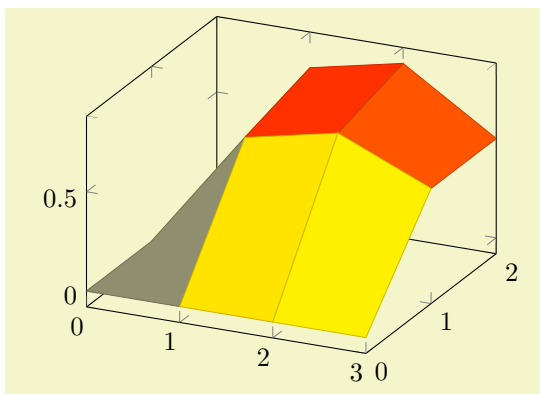
The `\addplot3 table` input works in the same way as its two dimensional counterpart `\addplot table`. It only expects a column for the z coordinates. Furthermore, it interprets empty input lines as end-of-row (more generally, end-of-scanline) markers, just as for `plot file`. The remark above about the `mesh/ordering` applies here as well.

```
/pgfplots/mesh/rows=⟨integer⟩
/pgfplots/mesh/cols=⟨integer⟩
```

For visualization of mesh or surface plots which need some sort of matrix input, the dimensions of the input matrix need to be known in order to visualize the plots correctly. The matrix structure may be known from end-of-row marks (empty lines as general end-of-scanline markers in the input stream) as has been described above.

If the matrix structure is not yet known, it is necessary to provide at least one of `mesh/rows` or `mesh/cols` where `mesh/rows` indicates the number of samples for y coordinates whereas `mesh/cols` is the number of samples used for x coordinates (see also `mesh/ordering`).

Thus, the following example is also a valid method to define an input matrix.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    % this yields also a 3x4 matrix:
    \addplot3[surf,mesh/rows=3] coordinates {
      (0,0,0) (1,0,0) (2,0,0) (3,0,0)
      (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
      (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
    };
  \end{axis}
\end{tikzpicture}
```

It is enough to supply one of `mesh/rows` or `mesh/cols` – the missing value will be determined automatically.

If you provide one of `mesh/rows` or `mesh/cols`, any end-of-row marker seen inside of input files or coordinate streams will be ignored.

```
/pgfplots/mesh/scanline verbose=true|false (initially false)
```

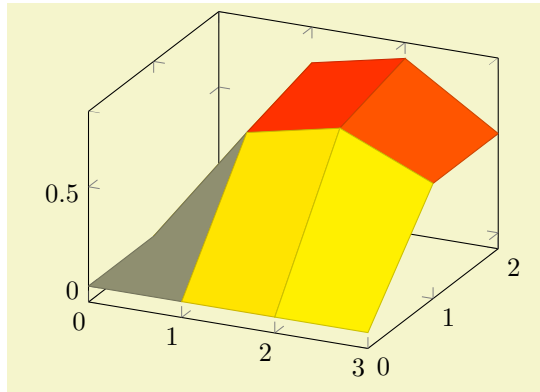
Provides debug messages in the L^AT_EX output about end-of-scanline markers.

The message will tell whether end-of-scanlines have been found and if they are the same.

```
/pgfplots/mesh/ordering=x varies|y varies|rowwise|colwise (initially x varies)
```

Allows to configure the sequence in which matrices (meshes) are read from `\addplot3 coordinates`, `\addplot3 file` or `\addplot3 table`.

Here, `x varies` means a sequence of points where $n=\text{mesh/cols}$ successive points have the y coordinate fixed. This is intuitive when you write down a function because x is horizontal and y vertical. Note that in matrix terminology, x refers to *column indices* whereas y refers to *row indices*. Thus, `x varies` is equivalent to `rowwise` ordering in this sense. This is the initial configuration.



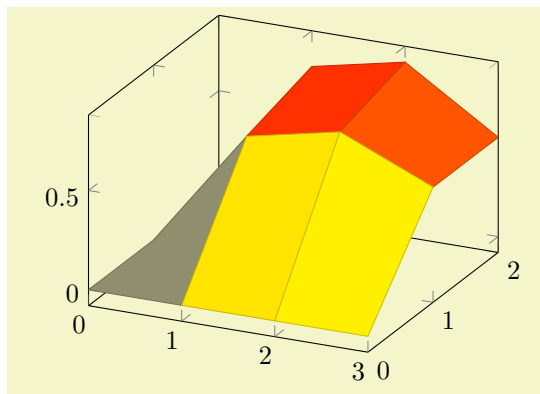
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[mesh/ordering=x varies]
% this yields a 3x4 matrix in 'x varies'
% ordering:
\addplot3[surf] coordinates {
(0,0,0) (1,0,0) (2,0,0) (3,0,0)

(0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)

(0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
};
\end{axis}
\end{tikzpicture}
```

Note that `mesh/ordering` is mandatory, even though the size of the matrix can be provided in different ways. The example above uses `empty lines` to mark scanlines. One could also say `mesh/rows=3` and omit the `empty lines`.

Consequently, `mesh/ordering=y varies` provides points such that successive $m=\text{mesh/rows}$ points form a column, i.e. the x coordinate is fixed and the y coordinate changes. In this sense, `y varies` is equivalent to `colwise` ordering, it is actually a matrix transposition.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[mesh/ordering=y varies]
% this yields a 3x4 matrix in colwise ordering:
\addplot3[surf] coordinates {
(0,0,0) (0,1,0) (0,2,0)

(1,0,0) (1,1,0.6) (1,2,0.7)

(2,0,0) (2,1,0.7) (2,2,0.8)

(3,0,0) (3,1,0.5) (3,2,0.5)
};
\end{axis}
\end{tikzpicture}
```

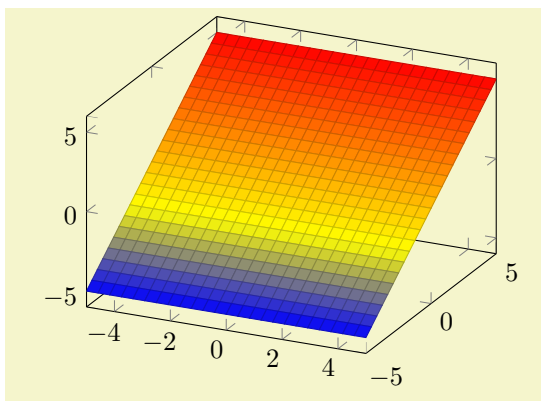
Again, note the subtle difference to the common matrix indexing where a column has the second index fixed. PGFLOTS refers to the way one would write down a function on a sheet of paper (this is consistent with how Matlab (®) displays discrete functions with matrices).

```
\addplot3 { $\langle\text{math expression}\rangle$ } ;
\addplot3[ $\langle\text{options}\rangle$ ] { $\langle\text{math expression}\rangle$ }  $\langle\text{trailing path commands}\rangle$ ;
```

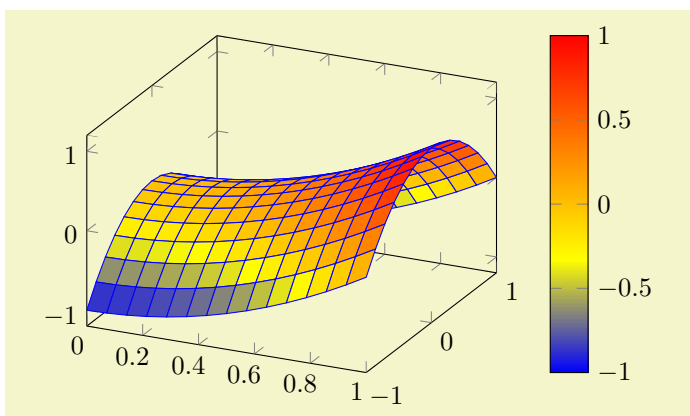
Expression plotting also works in the same way as for two dimensional plots. Now, however, a two dimensional mesh is sampled instead of a single line, which may depend on x and y .

The method `\addplot3 { $\langle\text{math expr}\rangle$ }` visualizes the function $f(x, y) = \langle\text{math expr}\rangle$ where $f: [x_1, x_2] \times [y_1, y_2] \rightarrow \mathbb{R}$. The interval $[x_1, x_2]$ is determined using the `domain` key, for example using `domain=0:1`. The interval $[y_1, y_2]$ is determined using the `y domain` key. If `y domain` is empty, $[y_1, y_2] = [x_1, x_2]$ will be assumed. If `y domain=0:0` (or any other interval of length zero), it is assumed that the plot does not depend on y (thus, it is a line plot).

The number of samples in x direction is set using the `samples` key. The number of samples in y direction is set using the `samples y` key. If `samples y` is not set, the same value as for x is used. If `samples y` ≤ 1 , it is assumed that the plot does not depend on y (meaning it is a line plot).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf] {y};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar]
    \addplot3
      [surf,faceted color=blue,
       samples=15,
       domain=0:1,y domain=-1:1]
      {x^2 - y^2};
  \end{axis}
\end{tikzpicture}
```

Expression plotting sets `mesh/rows` and `mesh/cols` automatically; these settings don't have any effect for expression plotting.

```
\addplot3 expression { $\langle\text{math expression}\rangle$ };
\addplot3[options] expression { $\langle\text{math expression}\rangle$ } trailing path commands;
```

The syntax

```
\addplot3 { $\langle\text{math expression}\rangle$ };
```

as short-hand equivalent for

```
\addplot3 expression { $\langle\text{math expression}\rangle$ };
```

```
\addplot3 (x expression),(y expression),(z expression) ;
\addplot3[options] (x expression),(y expression),(z expression) trailing path commands;
```

A variant of `\addplot3 expression` which allows to provide different coordinate expressions for the x , y and z coordinates. This can be used to generate parametrized plots.

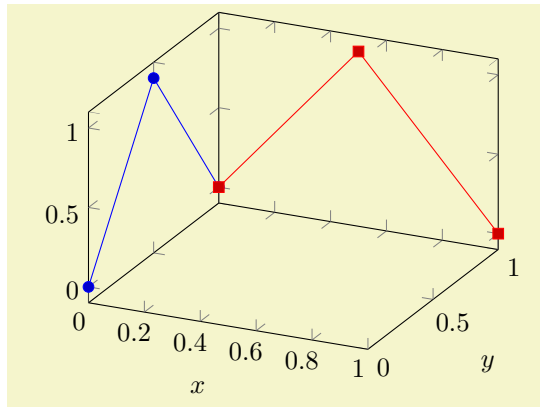
Please note that `\addplot3 (x,y,x^2)` is equivalent to `\addplot3 expression {x^2}`.

Note further that since the complete point expression is surrounded by round braces, round braces inside of $\langle x \text{ expression} \rangle$, $\langle y \text{ expression} \rangle$ or $\langle z \text{ expression} \rangle$ need to be treated specially. Surround the expressions (which contain round braces) with curly braces:

```
\addplot3 ({ $\langle x \text{ expr} \rangle$ }, { $\langle y \text{ expr} \rangle$ }, { $\langle z \text{ expr} \rangle$ });
```

4.5.3 Line Plots

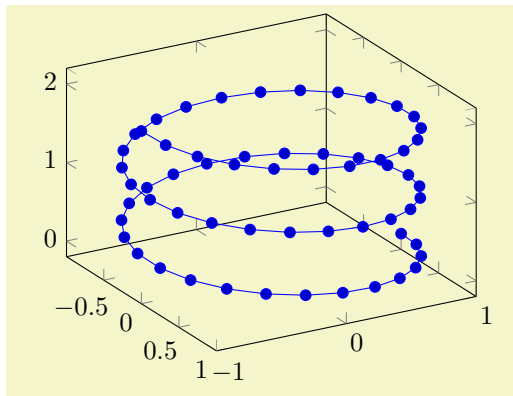
Three dimensional line plots are generated if the input source has no matrix structure. Line plots take the input coordinates and connect them in the order of appearance.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[xlabel=$x$,ylabel=$y$]
\addplot3 coordinates {(0,0,0) (0,0.5,1) (0,1,0)};
\addplot3 coordinates {(0,1,0) (0.5,1,1) (1,1,0)};
\end{axis}
\end{tikzpicture}
```

If there is no value for neither `mesh/rows` nor `mesh/cols` or if one of them is 1, PGFPLOTS will draw a line plot. This is also the case if there is no end-of-scanline marker (`empty line`) in the input stream.

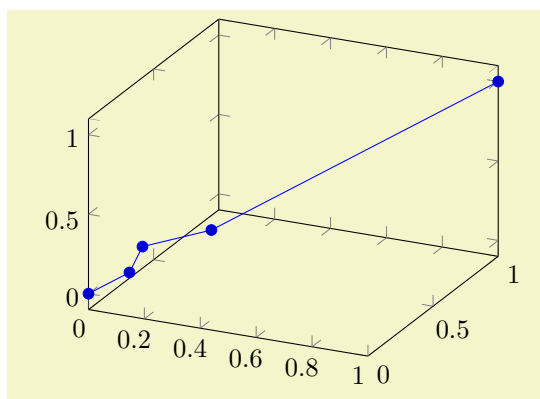
For `\addplot3 expression`, this requires to set `samples y=0` to disable the generation of a mesh.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view={60}{30}]
\addplot3+[domain=0:5*pi,samples=60,samples y=0]
({sin(deg(x))},
{cos(deg(x))},
{2*x/(5*pi)});
\end{axis}
\end{tikzpicture}
```

The example above is a parametric plot by expression, i.e. it has three distinct expressions for x , y , and z .

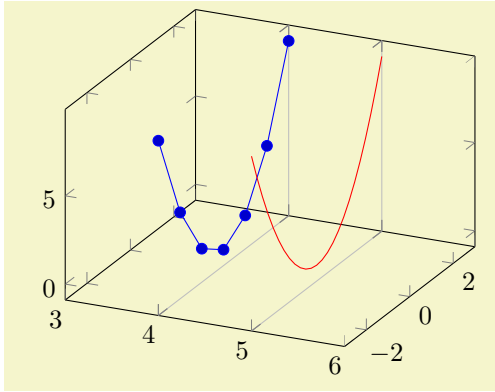
Line plots in three dimensions are also possible for data plots (tables). The most simple case is if you simply provide a series of three-dimensional coordinates which will be connected in the order of appearance:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot3 table {
x y z
0 0 0
0.1 0.1 0.1
0.1 0.2 0.2
0.3 0.3 0.3
1 1 1
};
\end{axis}
\end{tikzpicture}
```

Note that this plot implicitly has `mesh/rows=1` because it has no end-of-scanline markers (`empty lines`). If in doubt, you can set `mesh/rows=1` explicitly to tell PGFPLOTS that you have one-dimensional data (and not a matrix).

Line plots from data files are also possible if the data files only contains two coordinates – and the third should be provided somehow. In this case, the `table/x expr` feature comes into play: it allows to combine data plots and math expressions:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xmin=3,xmax=6,
  extra x ticks={4,5},
  extra x tick style={xticklabel=\empty,grid=major}
]
\addplot3 table[x expr=4,y=a,z=b] {
a b
-3 9
-2 4
-1 1
0 0
1 1
2 4
3 9
};
\addplot3[red,domain=-3:3,samples y=0] (5,x,x^2);
\end{axis}
\end{tikzpicture}
```

Here, we have two plots in one axis: one data plot from a data table with just two coordinates and one parametric plot. Both denote the same two functions. For the data plot, `x expr=4` assigns the x coordinate, and `y=a,z=b` define how the input columns map to coordinates. Again, the plot implicitly uses `mesh/rows=1` since there is no end-of-scanline marker. The second plot does the same with the short-handed notation `(5,x,x^2)`. It only samples one-dimensional data due to `samples y=0`. Finally, `extra x ticks` configures two additional ticks for the x axis; this is used to display grid lines for these specific ticks. The `xticklabel=\empty` argument avoids overprinted x tick labels at positions $x \in \{4, 5\}$.

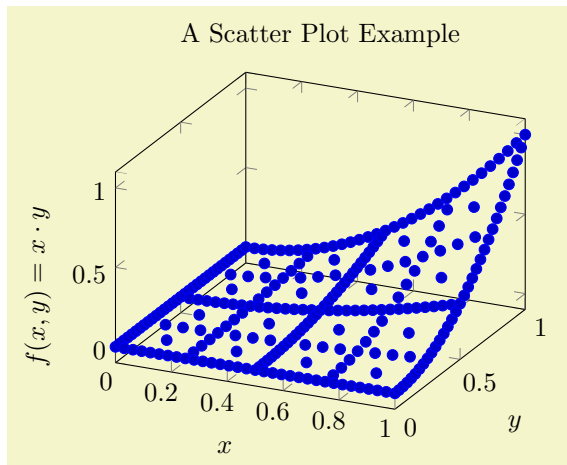
Three dimensional line plots will usually employ lines to connect points (i.e. the initial `sharp plot` handler of TikZ). The `smooth` method of TikZ might also prove be an option. Note that no piecewise constant plot, comb or bar plot handler is supported for three dimensional axes.

4.5.4 Scatter Plots

Three dimensional scatter plots have the same interface as for two dimensional scatter plots, so all examples of Section 4.4.10 can be used for the three dimensional case as well. The key features are to use `only marks` and/or `scatter` as plot styles.

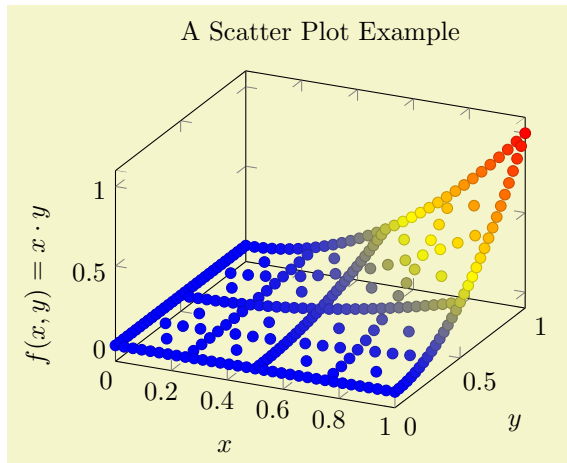
We provide some more examples which are specific for the three dimensional case.

Our first example uses `only marks` to place the current plot `mark` at each input position:



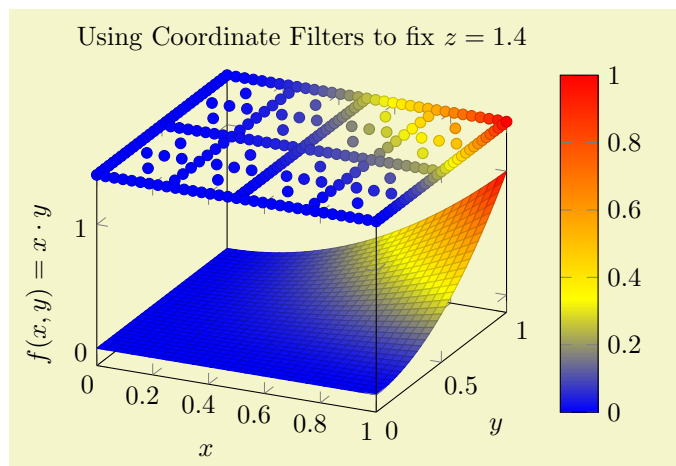
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xlabel=$x$,
  ylabel=$y$,
  zlabel={$f(x,y) = x \cdot y$},
  title=A Scatter Plot Example
]
% 'pgfplotsexample4_grid.dat' contains a
% large sequence of input points of the form
% x_0 x_1 f(x)
% 0 0 0
% 0 0.03125 0
% 0 0.0625 0
% 0 0.09375 0
% 0 0.125 0
% 0 0.15625 0
\addplot3+[only marks] table
{plotdata/pgfplotsexample4_grid.dat};
\end{axis}
\end{tikzpicture}
```

If we add the key `scatter`, the plot mark will also use the colors of the current `colormap`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xlabel=$x$,
    ylabel=$y$,
    zlabel={$f(x,y) = x \cdot y$},
    title=A Scatter Plot Example]
    \addplot3+[only marks,scatter] table
      {plotdata/pgfplotsexample4_grid.dat};
  \end{axis}
\end{tikzpicture}
```

A more sophisticated example is to draw the approximated function as a `surf` plot (which requires matrix data) and the underlying grid (which is `scattered` data) somewhere into the same axis. We choose to place the (x, y) grid points at $z = 1.4$. Furthermore, we want the grid points to be colored according to the value of column $f(x)$ in the input table:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    3d box,
    zmax=1.4,
    colorbar,
    xlabel=$x$,
    ylabel=$y$,
    zlabel={$f(x,y) = x \cdot y$},
    title={Using Coordinate Filters to fix $z=1.4$}]
    % 'pgfplotsexample4.dat' contains similar data as in
    % 'pgfplotsexample4_grid.dat', but it uses a uniform
    % matrix structure (same number of points in every scanline).
    % See examples above for extracts.
    \addplot3[surf,mesh/ordering=y varies]
      table {plotdata/pgfplotsexample4.dat};
    \addplot3[scatter,scatter src=\thisrow{f(x)},only marks, z filter/.code={\def\pgfmathresult{1.4}}]
      table {plotdata/pgfplotsexample4_grid.dat};
  \end{axis}
\end{tikzpicture}
```

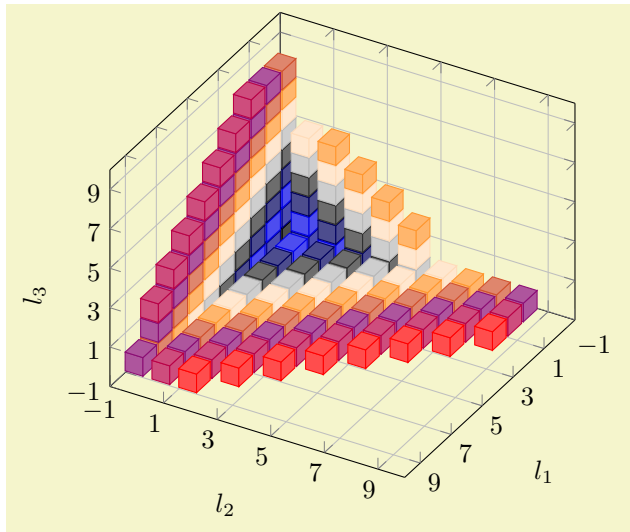
We used `z filter` to fix the z coordinate to 1.4. We could also have used the `table/z expr=1.4` feature

```
\addplot3[scatter,scatter src=\thisrow{f(x)},only marks]
  table[z expr=1.4] {plotdata/pgfplotsexample4_grid.dat};
```

to get exactly the same effect. Choose whatever you like best. The `z filter` works for every coordinate input routine, the `z expr` feature is only available for `plot table`.

The following example uses `mark=cube*` and `z buffer=sort` to place boxes at each input coordinate. The color for each box is determined by `point meta={x+y+3}`. The remaining keys are just for pretty

printing.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    view={120}{40},
    width=220pt,
    height=220pt,
    grid=major,
    z buffer=sort,
    xmin=-1,xmax=9,
    ymin=-1,ymax=9,
    zmin=-1,zmax=9,
    enlargelimits=upper,
    xtick={-1,1,...,19},
    ytick={-1,1,...,19},
    ztick={-1,1,...,19},
    xlabel={\mathit{l}_1\$},
    ylabel={\mathit{l}_2\$},
    zlabel={\mathit{l}_3\$},
    point meta={x+y+z+3},
    colormap={summap}{
        color=(black); color=(blue);
        color=(black); color=(white)
        color=(orange) color=(violet)
        color=(red)
    },
    scatter/use mapped color={
        draw=mapped color,fill=mapped color!70},
    ]
    % 'pgfplots_scatter4.dat' contains a large sequence of
    % the form
    % l_0 l_1 l_2
    % 1 6 -1
    % -1 -1 -1
    % 0 -1 -1
    % -1 0 -1
    % -1 -1 0
    % 1 -1 -1
    % 0 0 -1
    % 0 -1 0
    \addplot3[only marks,scatter,mark=cube*,mark size=7]
        table {plotdata/pgfplots_scatterdata4.dat};
\end{axis}
\end{tikzpicture}
```

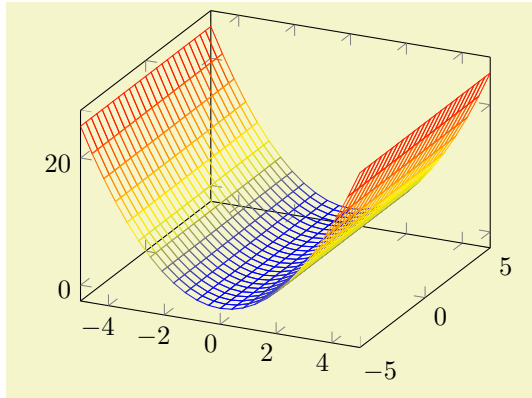
4.5.5 Mesh Plots

/pgfplots/**mesh**

(no value)

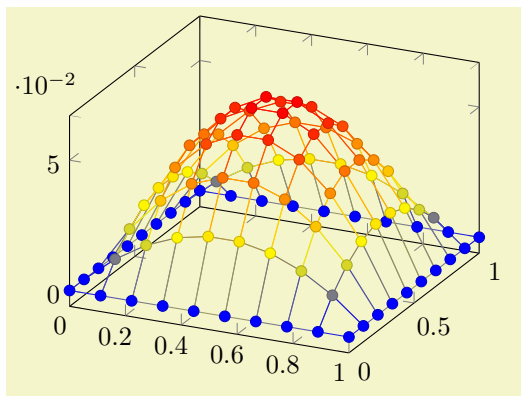
`\addplot+[mesh]`

A mesh plot uses different colors for each mesh segment. The color is determined using a “color coordinate” which is also called “meta data” throughout this document. It is the same data which is used for surface and scatter plots as well, see Section 4.7. In the initial configuration, the “color coordinate” is the z axis (or the y axis for two dimensional plots). This color coordinate is mapped linearly into the current color map to determine the color for each mesh segment. Thus, if the smallest occurring color data is, say, -1 and the largest is 42 , points with color data -1 will get the color at the lower end of the color map and points with color data 42 the color of the upper end of the color map.

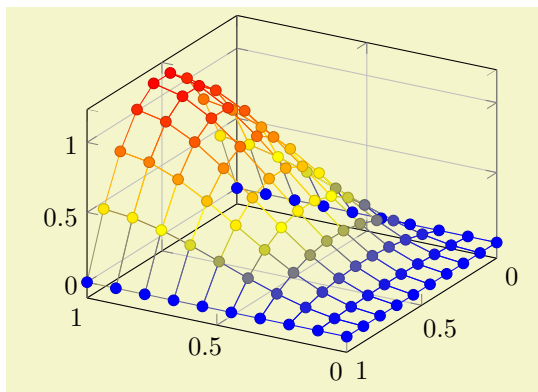


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[mesh] {x^2};
  \end{axis}
\end{tikzpicture}
```

A mesh plot can be combined with markers or with the `scatter` key which also draws markers in different colors.

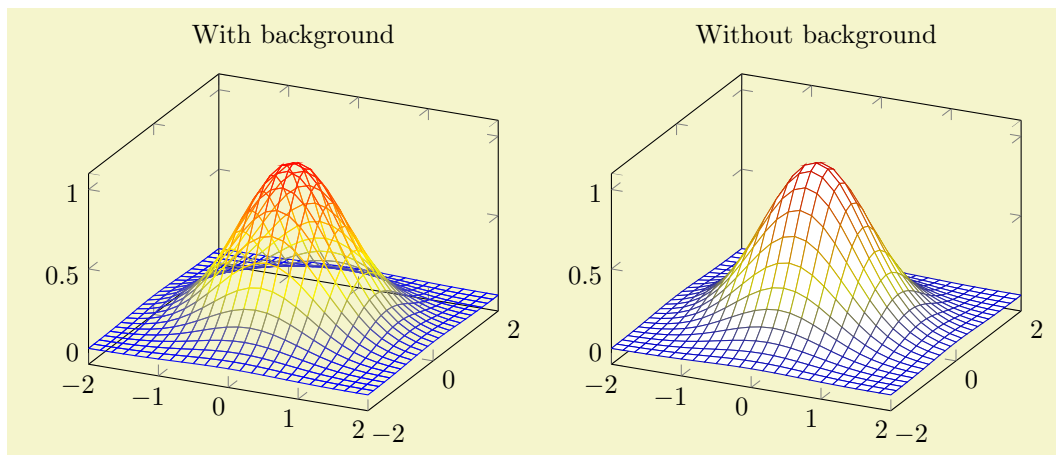


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3+[mesh,scatter,samples=10,domain=0:1]
      {x*(1-x)*y*(1-y)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[grid=major,view={210}{30}]
    \addplot3+[mesh,scatter,samples=10,domain=0:1]
      {5*x*sin(2*deg(x)) * y*(1-y)};
  \end{axis}
\end{tikzpicture}
```

Occasionally, one may want to hide the background mesh segments. This can be realized using the `surf` plot handler (see below) and a specific fill color:

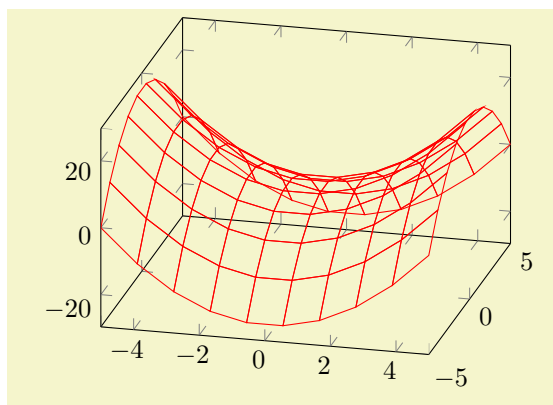


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[title=With background]
    \addplot3[mesh,domain=-2:2] {exp(-x^2-y^2)};
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[title=Without background]
    \addplot3[surf,fill=white,domain=-2:2] {exp(-x^2-y^2)};
  \end{axis}
\end{tikzpicture}
```

The fill color needs to be provided explicitly.

Details:

- A mesh plot uses the same implementation as `shader=flat` to get one color for each single segment. Thus, if `shader=flat mean`, the color for a segment is determined using the *mean* of the color data of adjacent vertices. If `shader=flat corner`, the color of a segment is the color of *one* adjacent vertex.
- As soon as `mesh` is activated, `color=mapped color` is installed. This is *necessary* unless one needs a different color – but `mapped color` is the only color which reflects the color data. It is possible to use a different color using the `color=<color name>` as for any other plot.
- It is easily possible to add `mark=<marker name>` to mesh plots, `scatter` is also possible. Scatter plots will use the same color data as for the mesh.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view/az=14]
    \addplot3[mesh,draw=red,samples=10] {x^2-y^2};
  \end{axis}
\end{tikzpicture}
```

Mesh plots use the `mesh legend` style to typeset legend images.

`/pgfplots/mesh/check=false|warning|error`

(initially `error`)

Allows to configure whether an error is generated if `mesh/rows` $\times$ `mesh/cols` does not equal the total number of coordinates.

If you know exactly what you are doing, it may be useful to disable the check. If you are unsure, it is best to leave the initial setting.

`/pgfplots/z buffer=default|none|auto|sort|reverse x seq|reverse y seq|reverse xy seq` (initially default)

This key allows to choose between different z buffering strategies. A z buffer determines which parts of an image should be drawn in front of other parts. Since both, the graphics packages PGF and the final document format `.pdf` are inherently two dimensional, this work has to be done in $\text{\TeX}$. Currently, several (fast) heuristics can be used which work reasonably well for simple mesh- and surface plots. Furthermore, there is a (time consuming) sorting method which also works if the fast heuristics fails.

The z buffering algorithms of PGFPLOTS apply only to a single `\addplot` command. Different `\addplot` commands will be drawn on top of each other, in the order of appearance.

The choice `default` checks if we are currently working with a mesh or surface plot and uses `auto` in this case. If not, it sets `z buffer=none`.

The choice `none` disables z buffering. This is also the case for two dimensional axes which don't need z buffering.

The choice `auto` is the initial value for any mesh or surface plot: it uses a very fast heuristics to decide how to realize z buffering for mesh and surface plots. The idea is to reverse either the sequence of all x coordinates, or those of all y coordinates, or both. For regular meshes, this suffices to provide z buffering. In other words: the choice `auto` will use one of the three reverse strategies `reverse * seq` (or none at all). The choice `auto`, applied to `patch` plots, uses `z buffer=sort` since `patch` plots have no matrix structure.

The choice `sort` can be used for scatter, line, mesh, surface and patch plots. It sorts according to the depth of each point (or mesh segment). Sorting in $\text{\TeX}$ uses a slow algorithm and may require a lot of memory (although it has the expected runtime asymptotics $\mathcal{O}(N \log N)$). The depth of a mesh segment is just *one* number, currently determined as *mean* over the vertex depths. Since `z buffer=sort` is actually just a more intelligent way of drawing mesh segments on top of each other, it may still fail. Failure can occur if mesh segments are large and overlap at different parts of the segment (see Wikipedia "Painter's algorithm"). If you experience problems of this sort, consider reducing the mesh width (the mesh element size) such that they can be sorted independently (for example automatically using `patch refines=2`, see the `patchplots` library).

The remaining choices apply only to mesh/surface plots (i.e. for matrix data) and do nothing more then their name indicates: they reverse the coordinate sequences of the input matrix (using quasi linear runtime). They should only be used in conjunction by `z buffer=auto`.

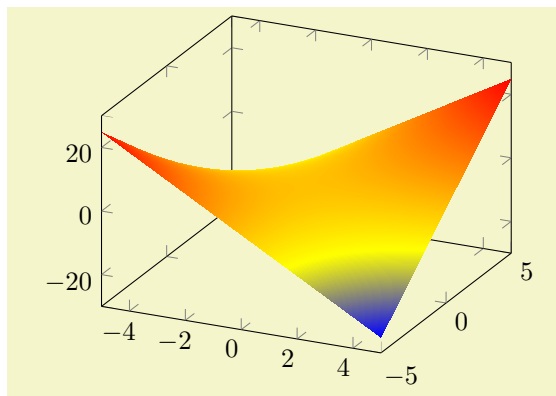
4.5.6 Surface Plots

`/pgfplots/surf`

(no value)

`\addplot+[surf]`

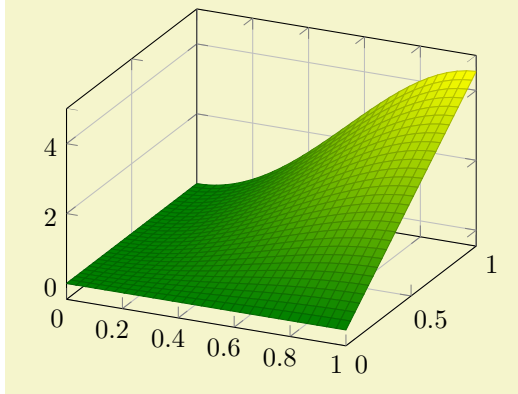
A surface plot visualizes a two dimensional, single patch using different fill colors for each patch segment. Each patch segment is a (pseudo) rectangle, that means input data is given in form of a data matrix as is discussed in the introductory section about three dimensional coordinates, 4.5.2.



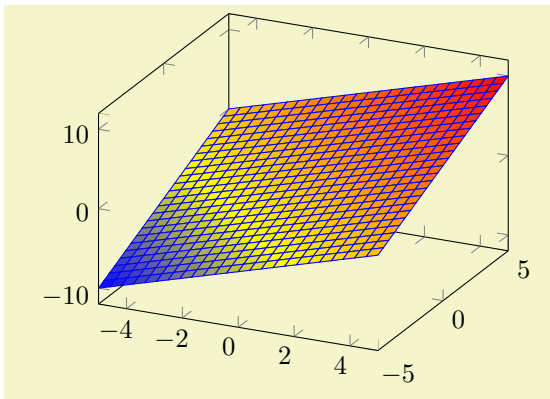
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=interp] {x*y};
  \end{axis}
\end{tikzpicture}
```

The simplest way to generate surface plots is to use the plot expression feature, but – as discussed in Section 4.5.2 – other input methods like `\addplot3 table` or `\addplot3 coordinates` are also possible.

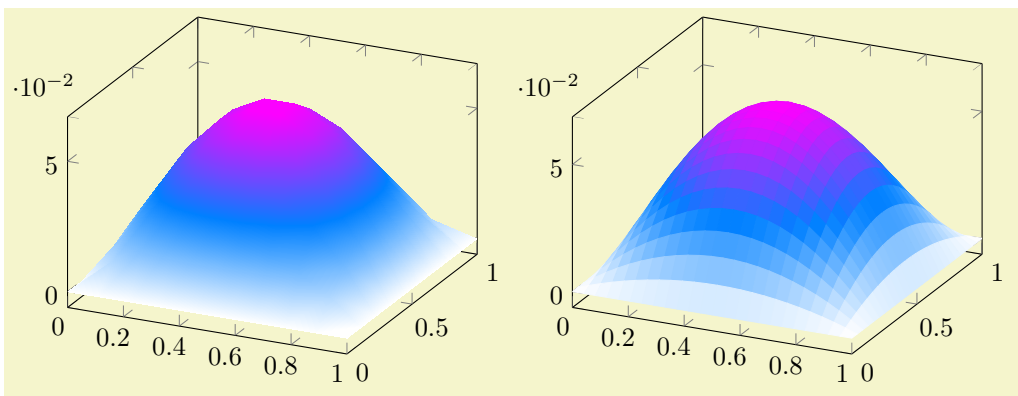
The appearance can be configured using `colormaps`, the value of the `shader`, `faceted color` keys and the current `color` and/or `draw/fill` color. As for `mesh` plots, the special `color=mapped color` is installed for the faces. The stroking color for faceted plots can be set with `faceted color` (see below for details).



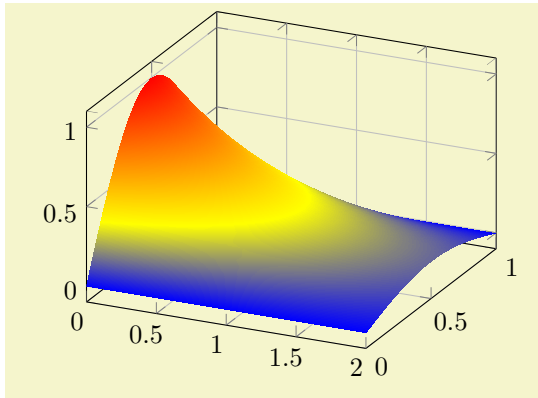
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    grid=major,
    colormap/greenyellow]
    \addplot3[surf,samples=30,domain=0:1]
      {5*x*sin(2*deg(x)) * y};
  \end{axis}
\end{tikzpicture}
```



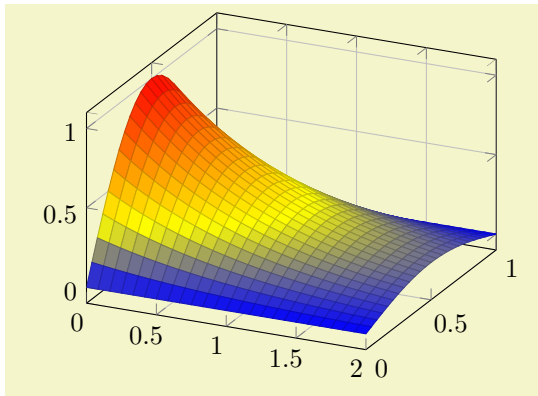
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,faceted color=blue] {x+y};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colormap/cool]
    \addplot3[surf,samples=10,domain=0:1,
      shader=interp]
      {x*(1-x)*y*(1-y)};
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[colormap/cool]
    \addplot3[surf,samples=25,domain=0:1,
      shader=flat]
      {x*(1-x)*y*(1-y)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[grid=major]
    \addplot3[surf,shader=interp,
      samples=25,domain=0:2,y domain=0:1]
      {exp(-x) * sin(pi*deg(y))};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[grid=major]
    \addplot3[surf,shader=faceted,
      samples=25,domain=0:2,y domain=0:1]
      {exp(-x) * sin(pi*deg(y))};
  \end{axis}
\end{tikzpicture}
```

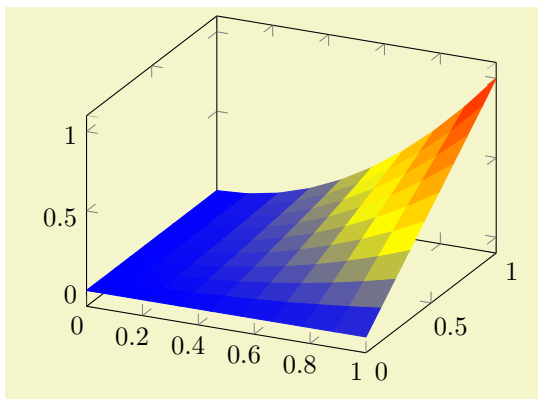
Details about the shading algorithm are provided below in the documentation of [shader](#).

Surface plots use the [mesh legend](#) style to create legend images.

`/pgfplots/shader=flat|interp|faceted|flat corner|flat mean|faceted interp` (initially [faceted](#))

Configures the shader used for surface plots. The shader determines how the color data available at each single vertex is used to fill the surface patch.

The simplest choice is to use one fill color for each segment, the choice [flat](#).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=flat,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```

There are (currently) two possibilities to determine the single color for every segment:

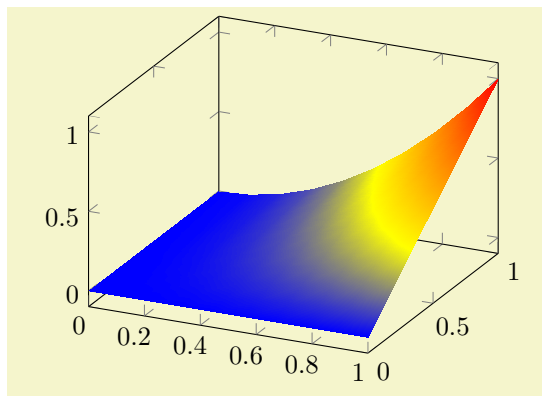
[flat corner](#) Uses the color data of one vertex to color the segment. It is not defined which vertex is used here²⁰.

[flat mean](#) Uses the mean of all four color data values as segment color. This is the initial value as it provides symmetric colors for symmetric functions.

The choice [flat](#) is actually the same as [flat mean](#). Please note that [shader=flat mean](#) and [shader=flat corner](#) also influence mesh plots – the choices determine the mesh segment color.

²⁰PGFLOTS just uses the last vertex encountered in its internal processings – but after any *z* buffer re-orderings.

Another choice is `shader=interp` which uses Gouraud shading (smooth linear interpolation of two triangles approximating rectangles) to fill the segments.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=interp,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```

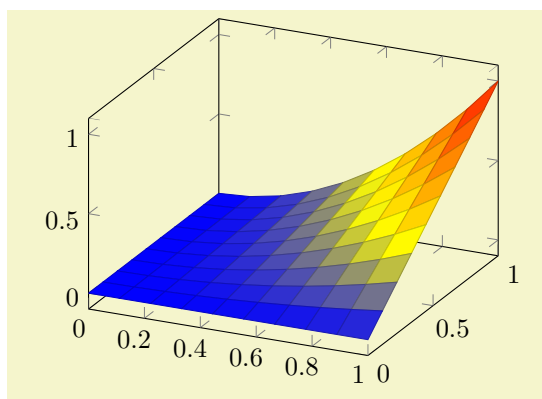
The `shader=interp` employs a low-level shading implementation which is currently available for the following drivers:

- the postscript driver `\def\pgfsysdriver{pgfsys-dvips.def}`,
- the pdflatex driver `\def\pgfsysdriver{pgfsys-pdftex.def}`,
- the lualatex driver `\def\pgfsysdriver{pgfsys-pdftex.def}` (yes, it is the same),
- the dvipdfmx driver `\def\pgfsysdriver{pgfsys-dvipdfmx.def}`.

For other drivers, the choice `shader=interp` will result in a warning and is equivalent to `shader=flat mean`. See also below for detail remarks.

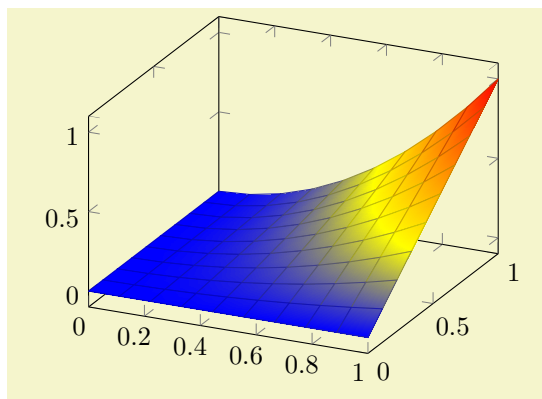
Note that `shader=interp,patch type=bilinear` allows real bilinear interpolation, see the `patchplots` library.

The choice `shader=faceted` uses a constant fill color for every mesh segment (as for `flat`) and the value of the key `/pgfplots/faceted color` to draw the connecting mesh elements:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=faceted,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```

The last choice is `shader=faceted interp`. As the name suggests, it is a mixture of `interp` and `faceted` in the sense that each element is shaded using linear triangle interpolation (see also the `patchplots` library for bilinear interpolation) in the same way as for `interp`, but additionally, the edges are colored in `faceted color`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=faceted interp,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```

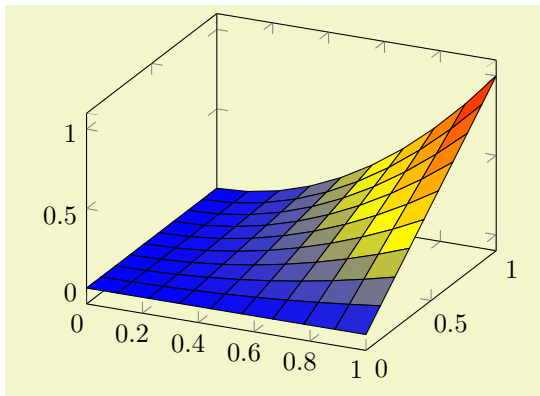
In principle, there is nothing wrong with the idea as such, and it looks quite good – but it enlarges the resulting pdf document considerably and might take a long time to render. It works as follows: for every mesh element (either triangle for `patch` plots or rectangle for lattice plots), it creates a low level shading. It then fills the single mesh element with that shading, and strokes the edges with `faceted color`. The declaration of that many low level shadings is rather inefficient in terms of pdf objects (large output files) and might render slowly²¹. For orthogonal plots (like `view={0}{90}`), the effect of `faceted interp` can be gained with less cost if one uses two separate `\addplot` commands: one with `surf` and one with `mesh`. Handle this choice with care.

Details:

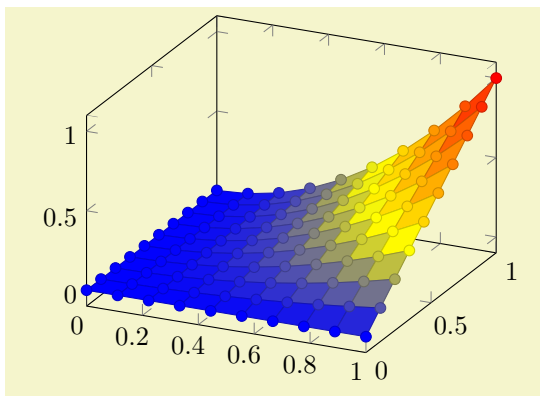
- All shaders support `z buffer=sort` (starting with version 1.4)
- The choice `shader=faceted` is the same as `shader=flat` – except that it uses a special draw color. So, `shader=faceted` has the same effect as `shader=flat,draw=\pgfkeysvalueof{/pgfplots/faceted color}`.
- The `flat` shader uses the current `draw` and `fill` colors. They are set with `color=mapped color` and can be overruled with `draw=<draw color>` and `fill=<fill color>`. The mapped color always contains the color of the color map.
- You easily add `mark=<plot mark>` to mesh and/or surface plots or even colored plot marks with `scatter`. The scatter plot feature will use the same color data as for the surface. But: Markers and surfaces do not share the same depth information. They are drawn on top of each other.
- Remarks on `shader=interp`:
 - It uses the current color map in any case, ignoring `draw` and `fill`.
 - For surface plots with lots of points, `shader=interp` produces smaller pdf documents, requires less compilation time in T_EX and requires less time to display in Acrobat Reader than `shader=flat`.
 - The postscript driver *truncates* coordinates to 24 bit – which might result in a loss of precision (the truncation is not very intelligent). See the `surf shading/precision` key for details. To improve compatibility, this 24 bit truncation algorithm is enabled by default also for pdf documents.
 - The choice `shader=interp` works well with either Acrobat Reader or recent versions of free viewers²². However, some free viewers show colors incorrectly (like evince). I hope this message will soon become outdated... if not, provide bug reports to the Linux community to communicate the need to improve support for Type 4 (`patch`) and Type 5 pdf (`surf`) and Type 7 (`patch` and elements of the `patchplots` library) shadings.
 - The `interp` shader yields the same outcome as `faceted interp,faceted color=none`, although `faceted interp` requires much more resources.

²¹My experience is as follows: Acrobat reader can efficiently render huge `interp` shadings. But it is very slow for `faceted interp` shadings. Linux viewers like xpdf are reasonably efficient for `interp` (at least with my bugfixes to libpoppler) and are also fast for `faceted interp` shadings.

²²The author of this package has submitted bugfixes to xpdf/libpoppler which should be part of the current stable versions of many viewers.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=flat,
      draw=black,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[surf,shader=faceted,
      scatter,mark=*,
      samples=10,domain=0:1]
      {x^2*y};
  \end{axis}
\end{tikzpicture}
```

`/pgfplots/faceted color={\color name}` (initially mapped color!80!black)

Defines the color to be used for meshes of faceted surface plots.

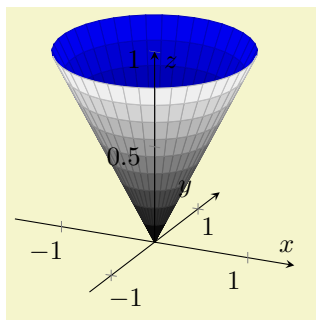
Set `faceted color=none` to disable edge colors.

`/pgfplots/mesh/interior colormap={\map name}\{colormap specification}`

`/pgfplots/mesh/interior colormap name={\map name}`

Allows to use a different `colormap` for the “other side” of the surface.

Each mesh has two sides: one which “points” to the view’s origin and one which points away from it. This key allows to define a second `colormap` for the side which points away from the view’s origin. The motivation is to distinguish between the outer side and the interior parts of a surface:

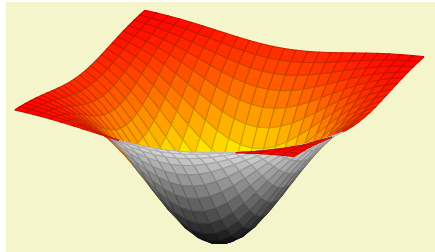


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    axis lines=center,
    axis on top,
    xlabel={\$x\$}, ylabel={\$y\$}, zlabel={\$z\$},
    domain=0:1,
    y domain=0:2*pi,
    xmin=-1.5, xmax=1.5,
    ymin=-1.5, ymax=1.5, zmin=0.0,
    mesh/interior colormap=
      {blueblack}{color=(black) color=(blue)},
    colormap/blackwhite,
    samples=10,
    samples y=40,
    z buffer=sort,
  ]
    \addplot3[surf]
      ({x*cos(deg(y))},{x*sin(deg(y))},{x});
  \end{axis}
\end{tikzpicture}
```

The `interior colormap` is often the one for the “inner side”. However, the orientation of the surface depends on its normal vectors: PGFLOTS computes them using the right-hand-rule. The right-hand-rule applied to a triangle means to take the first encountered point, point the thumb in direction of the

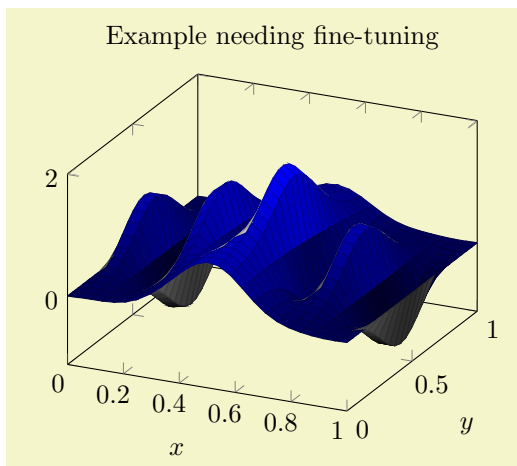
second point and the first finger in direction of the third point. Then, the normal for that triangle is the third finger (i.e. the cross-product of the involved oriented edges). For rectangular patches, PGFPLOTS uses the normal of one of its triangles²³. Consequently, `mesh/interior colormap` will only work if the involved patch segments are consistently oriented.

A patch whose normal vector points into the same direction as the view direction uses the standard `colormap name`. A patch whose normal vector points into the opposite direction (i.e. in direction of the viewport) uses `mesh/interior colormap`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    hide axis,
    xlabel=$x$,ylabel=$y$,
    mesh/interior colormap name=hot,
    colormap/blackwhite,
]
\addplot3[domain=-1.5:1.5,surf]
    {-exp(-x^2-y^2)};
\end{axis}
\end{tikzpicture}
```

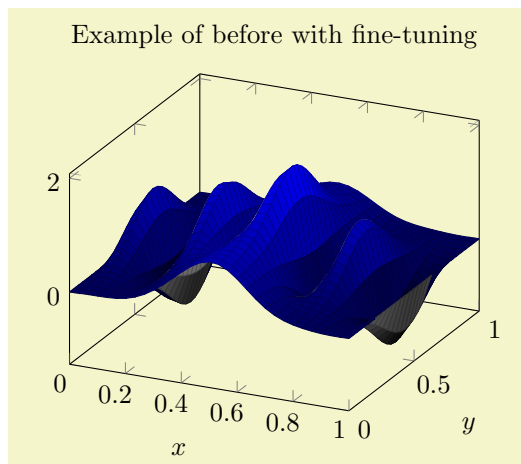
The implementation of `mesh/interior colormap` works well for most examples; in particular, if the number of samples is large enough to resolve the boundary between inner and outer colormap. However, it might still produce spurious artifacts:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title=Example needing fine-tuning,
    xlabel=$x$,
    ylabel=$y$]
\addplot3[surf,
    mesh/interior colormap=
    {blueblack}{color=(black) color=(blue)},
    colormap/blackwhite,
    domain=0:1]
    {sin(deg(8*pi*x))* exp(-20*(y-0.5)^2)
    + exp(-(x-0.5)^2*30
    - (y-0.25)^2 - (x-0.5)*(y-0.25))};
\end{axis}
\end{tikzpicture}
```

The previous example has need for improvement with respect to a couple of aspects: first, it has small overshoots near some of the meshes vertices (especially on top of the hills). These can be fixed using `miter limit=1`. Second, the boundary between blue and black is incorrect. This can be improved by means of an increased sampling density (`samples=31`). In addition, we can configure PGFPLOTS to move the boundary between the two colormaps in favor of the blue region using `mesh/interior colormap thresh` as follows:

²³This may change in future versions.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title=Example of before with fine-tuning,
    xlabel=$x$,
    ylabel=$y$]
\addplot3[surf,
    mesh/interior colormap=
        {blueblack}{color=(black) color=(blue)},
    % slightly increase sampling quality (was 25):
    samples=31,
    % avoids overshooting corners:
    miter limit=1,
    % move boundary between inner and outer:
    mesh/interior colormap thresh=0.1,
    colormap/blackwhite,
    domain=0:1]
    {sin(deg(8*pi*x))* exp(-20*(y-0.5)^2)
    + exp(-(x-0.5)^2*30
    - (y-0.25)^2 - (x-0.5)*(y-0.25));}
\end{axis}
\end{tikzpicture}
```

This improves the display.

Call for volunteers: it would be nice if the fine-tuning of these keys would be unnecessary. If someone has well-founded suggestions (like knowledge and perhaps exhaustive experiments) on how to improve the feature, let me know.

Note that `mesh/interior colormap` cannot be combined with `mesh/refines` currently.

Note that `mesh/interior colormap` will increase compilation times due to the computation of normal vectors.

`/pgfplots/mesh/interior colormap thresh={\langle Number between -1.0 and +1.0 \rangle}` (initially 0)

A threshold which moves the boundary between the `colormap` and `interior colormap` in favor of `colormap` (if the value is negative) or in favor of `interior colormap` (if the value is positive).

The extreme value `-1` essentially deactivates `interior colormap` whereas the other extreme `+1` deactivates `colormap`.

See above for an example.

`/pgfplots/surf shading/precision=pdf|postscript|ps` (initially `postscript`)

A key to configure how the low level driver for `shader=interp` writes its data. The choice `pdf` uses 32 bit binary coordinates (which is lossless). The resulting `.pdf` files appear to be correct, but they can't be converted to postscript – the converter software always complains about an error.

The choice `postscript` (or, in short, `ps`) uses 24 bit truncated binary coordinates. This results in both, readable `.ps` and `.pdf` files. However, the truncation is lossy.

If anyone has ideas how to fix this problem: let me know. As far as I know, Postscript should accept 32 bit coordinates, so it might be a mistake in the shading driver.

4.5.7 Surface Plots with Explicit Color

The surface plots described in Section 4.5.6 are all based on `colormaps`. This section introduces a different type of surface plot. In fact, it uses the very same plot handlers: it applies to `mesh`, `surf`, and `patch` plots. However, the way colors are provided and the way PGFPLOTS interpolates colors is substantially different.

This section describes surface plots with explicit colors. These expect colors like `red`, `green`, or `rgb=(0.5,0.2,1)` for every vertex of the `mesh` – and interpolates smoothly between these vertices. This appears to be simpler, perhaps even more straight-forward than `surface` plots based on `colormaps`. It is not. Surface plots with explicit color are more difficult to define, and they are more difficult to read.

If you are in doubt of whether to use a surface colored by a `colormap` or explicit colors, you should prefer `colormaps` for reasons discussed below.

`/pgfplots/mesh/color input=colormap|explicit|explicit mathparse` (initially `colormap`)

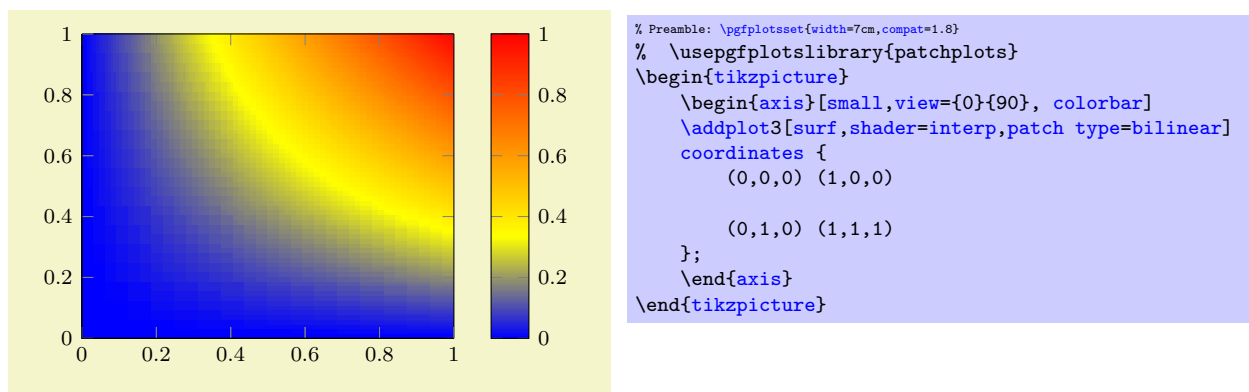
Allows to configure how PGFPLOTS expects color input for surface plots.

The choice `colormap` uses the standard colormaps. This particular choice expects scalar values of `point meta` which are mapped linearly into the colormap. It resembles the surface plots which are explained in more detail in Section 4.5.6. It is the default configuration and covers (probably) most common use-cases.

The choice `explicit` expects explicitly provided `point meta` of symbolic form: every coordinate of your input coordinate stream is supposed to have an explicitly defined color as `point meta`. Here, “explicitly provided” refers to `point meta=explicit symbolic`. This choice and the available color formats are explained in all detail in the following sub-sections.

The choice `explicit mathparse` is similar to `explicit`, but it allows to provide just one math expression which is evaluated for every coordinate. The math expression can be of the form `rgb=x,y,0.5` in which case `x` is used to define the “red” component, `y` is used to define the “green” component and the “blue” component is fixed to 0.5. This key is also explained in more detail in the following sub-sections.

The main use-case of `mesh/color input=colormap` is to allow a map between the interpolated colors and some value of interest. This map can be shown as `colorbar`:

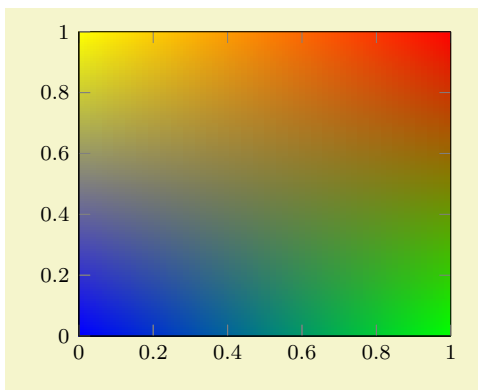


Note that the preceding example is a standard `surf` plot except for `patch type=bilinear` which controls how color is to be interpolated. There is a 2×2 matrix, and its z values are used as color data. Clearly, value $z = 0$ corresponds to blue and $z = 1$ corresponds to red – and all other colors in-between are not directly related to blue and red; they are taken from the colormap. The colormap defines which colors appear: those which make up the colormap and those which can occur as interpolated colors between the colors of the color map. The pairwise mixture of colors is a property of `mesh/color input=colormap`, not of `mesh/color input=explicit` (where more than two colors are mixed together). Furthermore, the surface indicates contours of constant z level. Take, for example, the yellow contour. We know that it has some value between 0.3 and 0.4, say 0.35. Since these shadings are continuous, we know that the point $z = 0.35$ occurs between $z = 0$ and $z = 1$ – at every point of the surface. Due to the colormap, each point on the surface which has $z = 0.35$ will receive the yellow color. This is because the interpolation is carried out on the scalar `point meta` value, which is afterwards mapped into the `colormap`. This contour-property is also unique for colormap surfaces.

The other two choices are explained in all detail in the next sub-sections.

Providing Colors Explicitly For Each Coordinate

Here is a simple approach with the same vertices as the colormap-example above, but with explicit colors:



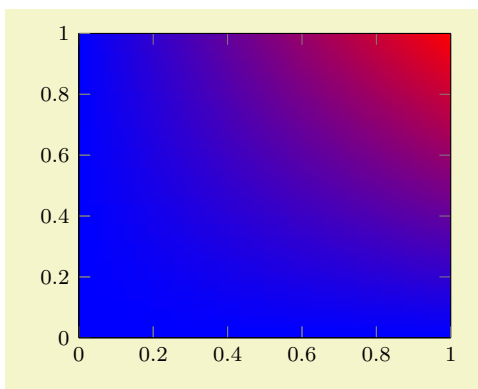
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepgfplotslibrary{patchplots}
\begin{tikzpicture}
  \begin{axis}[small,view={0}{90}]
    \addplot3[surf,shader=interp,patch type=bilinear,
      mesh/color input=explicit]
      coordinates {
        (0,0,0) [color=blue] (1,0,0) [color=green]

        (0,1,0) [color=yellow] (1,1,1) [color=red]
      };
  \end{axis}
\end{tikzpicture}
```

The coordinates and the view is the same, even the way colors are being interpolated bilinearly. However, we have four different colors in the corners. We see these corners in the output, and we see that they are smoothly mixed together. However, the mix contains all four colors, not just two. As a direct consequence, there are no contour lines.

The absence of direct information how to map color information to “some information of the data visualization” implies that *if* you want to use `surface` plots with explicit color, you have to state clearly what you want to show. This is considerably simpler for `colormaps`.

The following example configures $z = 0$ to receive `blue` and $z = 1$ to receive `red` as in our preceding `colormap` example (see above) using `mesh/color input=explicit`.



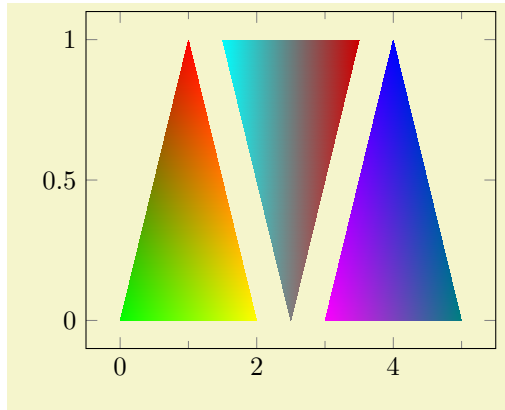
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepgfplotslibrary{patchplots}
\begin{tikzpicture}
  \begin{axis}[small,view={0}{90}]
    \addplot3[surf,shader=interp,patch type=bilinear,
      mesh/color input=explicit]
      coordinates {
        (0,0,0) [color=blue] (1,0,0) [color=blue]

        (0,1,0) [color=blue] (1,1,1) [color=red]
      };
  \end{axis}
\end{tikzpicture}
```

We see the bilinear nature of the interpolation; it is related to that of `mesh/color input=colormap` above (compare the contour lines in-between). In most cases, you simply want to show some contour lines. And for such cases, a `colormap` is the way to go.

There might be cases where `mesh/color input=explicit` is adequate. However, you will need to think it through properly. And you need to explain clearly what you did because your audience will also have to think a lot before they make sense of any data visualization based on explicit color interpolation.

The choice `mesh/color input=explicit` expects a choice of `point meta` which results in symbolic values. In this context, “symbolic” refers to a special color definition:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[minor x tick num=1]
\addplot[
    patch,
    shader=interp,
    mesh/color input=explicit,
]
table[meta=c] {
    x y c
    0 0 color=green
% default color model is rgb:
    1 1 1,0,0
    2 0 1,1,0

    1.5 1 cmyk=1,0,0,0
    2.5 0 gray=0.5
    3.5 1 color=red!80!black

    3 0 1,0,1
    4 1 0,0,1
    5 0 rgb255=0,128,128
};
\end{axis}
\end{tikzpicture}
```

The previous example defines a `patch` plot with three triangle patches, each made up of three vertices which are placed as-is into the input coordinate stream. Each vertex has its color data in column `c`. The format of color specifications is explained in more detail in the following paragraph.

As soon as you write `mesh/color input=explicit`, PGFPLOTS checks the current value of `point meta`. If the current value of `point meta` is `none`, it is set to `point meta=explicit symbolic` (that is what happened in our example above). If the current value of `point meta` is some choice which yields numeric output (like `point meta=x` or `point meta=\thisrow{x}+1`), it is set to `point meta=explicit symbolic`. If the current value of `point meta` is already of symbolic form, it is left unchanged.

Consequently, our example above sets `point meta=explicit symbolic` as soon as it encounters `mesh/color input=explicit`. The explicit symbolic input handler in turn expects the coordinate stream to provide point meta data for every streamed coordinate. We use a table here, and a table reads its color data from the column name provided in the `table/meta` key.

The accepted format of colors is quite similar to that of Colormap definitions (compare Section 4.6.6 on page 144). The common format is `<color model>=<arguments>`. In contrast to the similar input format inside of colormap definitions, the syntax here has no round braces and does not have the `<length>` argument. Nevertheless, the same `<color model>`s with the same `<arguments>` are accepted. The choices are

`rgb=<red>,<green>,<blue>` where each component is in the interval $[0, 1]$,

`rgb255=<red>,<green>,<blue>` is similar to `rgb` except that each component is expected in the interval $[0, 255]$,

`gray=<value>` with `<value>` in the interval $[0, 1]$,

`color=<named color>` where `<named color>` is a predefined (named) color like `'red'` or a color expression like `'red!50'`,

`cmyk=<cyan>,<magenta>,<yellow>,<black>` where each component is in the interval $[0, 1]$,

`cmyk255=<cyan>,<magenta>,<yellow>,<black>` is the same as `cmyk` but expects components in the interval $[0, 255]$,

`cmy=<cyan>,<magenta>,<yellow>` where each component is in the interval $[0, 1]$,

`hsb=<hue>,<saturation>,<brightness>` where each component is in the interval $[0, 1]$,

`Hsb=<hue>,<saturation>,<brightness>` is the same as `hsb` except that `<hue>` is accepted in the interval $[0, 360]$ (degree),

`HTML=<hex red><hex green><hex blue>` where component is expected to be a hex number between 00 and FF (a variant of `rgb255`),

`wave=<wave length>` which expects a single wave length as numeric argument in the range $[363, 814]$.

`/pgfplots/mesh/colourspace explicit color input=rgb|rgb255|cmy|cmyk|cmyk255|gray|wave|hsb|Hsb|HTML` (initially `rgb`)

If the input color has *no color model*, the color components are interpreted as color in the color model specified as argument to this key.

This key has just one purpose: to omit the *color model* if it is the same for lots of points anyway.

`/pgfplots/mesh/colourspace explicit color output=rgb|cmyk|gray` (initially `rgb`)

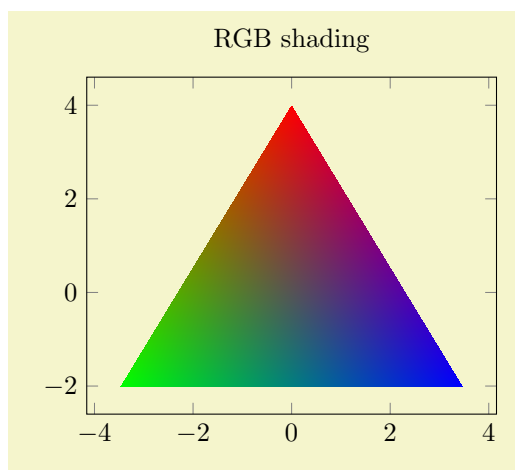
Any color which is encountered by the survey phase (i.e. while inspecting the `point meta` value) is immediately transformed into the color space configured as `mesh/colourspace explicit color output`.

The transformed color is used for any color interpolation. In most cases, this is done by the `shader`, but it applies to `patch refines` and `patch to triangles` as well.

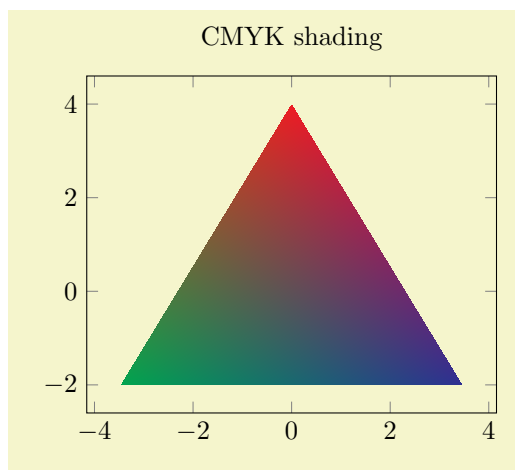
Because the transformed color is used for color interpolation, the list of available *output* color spaces is considerably smaller than the available *input* color spaces. Only the device color spaces `rgb`, `gray`, and `cmyk` are available as value for `mesh/colourspace explicit color output`.

Any necessary colorspace transformations rely on the `xcolor` package²⁴. Note that colorspace transformations are subject to nearest-color-matching, i.e. they are less accurate. This is typically far beyond pure rounding issues; it is caused by the fact that these color spaces are actually so different that transformations are hard to hard to accomplish. If you can specify colors immediately in the correct color space, you can eliminate such transformations.

Here is the same shading, once with CMYK output and once with RGB output. Depending on your output media (screen or paper), you will observe slightly different colors when comparing the pictures.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[title=RGB shading]
\addplot[
    patch,
    shader=interp,
    mesh/color input=explicit,
    mesh/colourspace explicit color output=rgb,
    data cs=polar,
]
coordinates {
    (90,4) [color=red]
    (210,4) [color=green]
    (-30,4) [color=blue]
};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[title=CMYK shading]
\addplot[
    patch,
    shader=interp,
    mesh/color input=explicit,
    mesh/colourspace explicit color output=cmyk,
    data cs=polar,
]
coordinates {
    (90,4) [color=red]
    (210,4) [color=green]
    (-30,4) [color=blue]
};
\end{axis}
\end{tikzpicture}
```

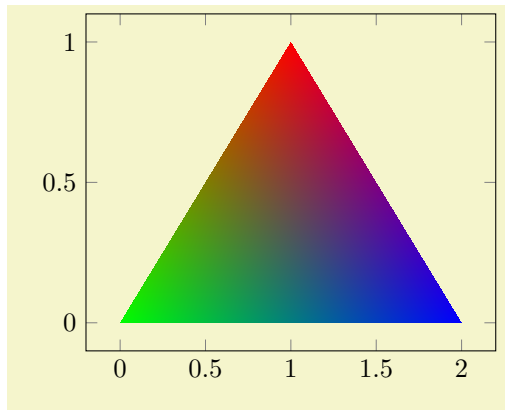
²⁴Colorspace transformations are unavailable for plain $\text{\TeX}$ and $\text{\ConTeXt}$. For these cases, you have to ensure that input and output color model are the same.

This key is similar to the related key `colormap default colorspace`, although the values can be chosen independently.

Providing Color Components as Table

The previous section shows how to provide a single symbolic color expression for each coordinate, namely using `point meta=explicit symbolic`.

Another use-case might be to provide a table containing both the coordinate values and one column by color component. In order to assemble the color specification from the input table, you can provide a symbolic expression:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot[
    patch,
    shader=interp,
    mesh/color input=explicit,
    point meta={symbolic=
        {\thisrow{R},\thisrow{G},\thisrow{B}}
    },
]
table {
    x y R G B
    0 0 0 1 0
    1 1 0 0 0
    2 0 0 0 1
};
\end{axis}
\end{tikzpicture}
```

The preceding example employs a `patch` plot with triangular elements as we have seen before. Furthermore, it uses explicit color input – but combined with `point meta={symbolic={\langle value \rangle}}` (note the extra pair of braces). This choice accepts arbitrary symbols on input which will be reevaluated (expanded) for every coordinate. In our example, we simply read the values from table columns using `\thisrow`. Since the default input colorspace is RGB, this results in the expected triangle with red, green, and blue corners. The result has to form a valid color specification.

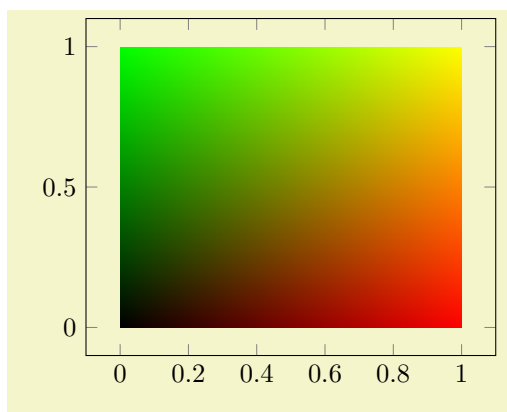
Note that the symbolic expression is purely string-based in this context. If you plan to use math expressions, you have to use `mesh/color input=explicit mathparse` as explained in the following section.

Providing Colors as Math Expression

The key `mesh/color input` has two choices for explicit color input. The choice `explicit` has been discussed in the preceding paragraphs, it expects one color specification for every node for which colors are needed. It also accepts a kind of string-based expressions to concatenate the expected color specification in a suitable way.

The second choice `mesh/color input=explicit mathparse` is almost the same – with one major difference: it allows to provide math expressions inside of the `point meta` value. However, the provided math expressions need to form a color specification which typically has more than one color component.

With this choice, the value of `point meta` is of symbolic form, but the color components are reevaluated with the math parser for every input point which has color data. The most convenient way to provide such expressions is `point meta={symbolic={\langle R \rangle,\langle G \rangle,\langle B \rangle}}` (again, note the extra set of braces for the argument).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot[
    surf,
    shader=interp,
    mesh/color input=explicit mathparse,
    point meta={symbolic={x,\thisrow{y},0}},
]
table {
    x y
    0 0
    1 0

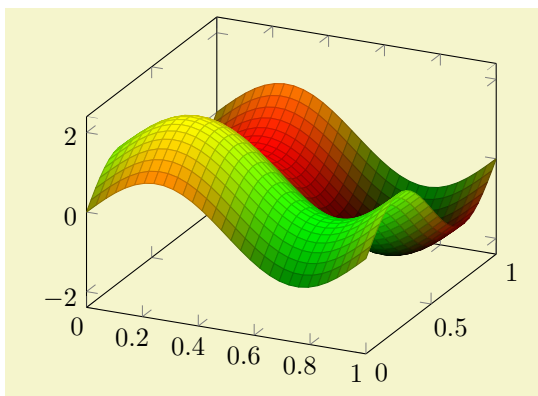
    0 1
    1 1
};
\end{axis}
\end{tikzpicture}
```

The preceding example is a **surface** plot with a 2×2 input matrix. Note that the table has no explicit point meta data. The **point meta** data is acquired from a common math expression which uses the final x coordinate as *red* component, the value seen in the current row and column y as *green* value and constant value *blue*=0. Consequently, the output is black in the lower left corner since black is (0,0,0), red in the lower right corner, green in the upper left corner, and a mixture of both along the diagonal.

The value provided as **point meta**=**{symbolic={value}}** is of the same form as for **mesh/color input=explicit**, i.e. it is supposed to be of the form $\langle \text{color model} \rangle = \langle \text{color components} \rangle$. If the $\langle \text{color model} \rangle$ is omitted, it defaults to **mesh/colourspace explicit color input** (which is **rgb** by default).

Since the math expression can be anything, it can safely be combined with plot by expression.

A potential use-case could be to show a surface with (x, y, z) and some two-dimensional quantity which is encoded as mixture of red and green. The following example relies on **mesh/color input=explicit mathparse** and **point meta=symbolic** to provide a vector of math expressions:



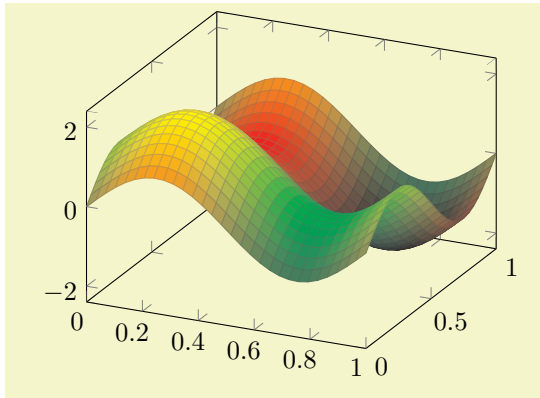
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepgfplotslibrary{patchplots}
\begin{tikzpicture}
\begin{axis}
% this example burns colors if opacity
% is active in the document.
\addplot3[
    patch,
    patch type=bilinear,
    mesh/color input=explicit mathparse,
    domain=0:1,
    samples=30,
    point meta={symbolic={%
        (sin(deg(x*pi*2))+1)/2,% R
        (sin(deg(y*pi*2))+1)/2,% G
        0% B
    }}
]
{sin(deg(x*pi*2))+sin(deg(y*pi*2))};
\end{axis}
\end{tikzpicture}
```

Note that the preceding example suffers from color burning²⁵: the green areas become too bright and the black areas become too dark. Note that the picture is entirely acceptable if it is written as stand-alone picture. But as soon as you import the picture (either as **.pdf** or as **.png**) into a document for which **opacity** is active, it suffers from burned colors.

The color burning is caused by the combination of RGB colourspace, the special color set in this example, and the color blending which is activated by **opacity**. Note that it is enough to activate **opacity** somewhere in the document.

In order to repair the problem for the picture at hand, one has to choose a different output colourspace:

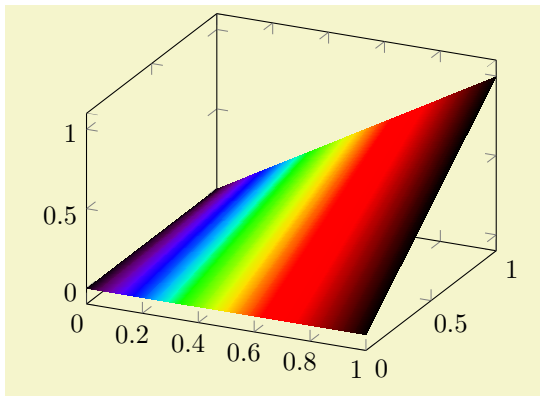
²⁵At least in Acrobat Reader.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepgfplotslibrary{patchplots}
\begin{tikzpicture}
\begin{axis}
\addplot3[
    patch,
    patch type=bilinear,
    mesh/color input=explicit mathparse,
    %
    % CMYK produces better quality here
    % since the manual has opacity enabled
    mesh/colourspace explicit color output=cmyk,
    domain=0:1,
    samples=30,
    point meta={symbolic={%
        (sin(deg(x*pi*2))+1)/2,% R
        (sin(deg(y*pi*2))+1)/2,% G
        0% B
    }}
    ],
    {sin(deg(x*pi*2))+sin(deg(y*pi*2))};
\end{axis}
\end{tikzpicture}
```

The key `mesh/colourspace explicit color output` transforms every input RGB color to a matching CMYK color. This, in turn, is a lossy transformation which seems to lack a trivial solution²⁶.

Math expressions can also be used for complicated input color spaces, for example the `wave` colorspace.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot3[
    surf,
    shader=interp,
    mesh/color input=explicit mathparse,
    domain=0:1,
    samples y=2,
    point meta={symbolic={wave=363+x*(814-363)}},
    ],
    {x*y};
\end{axis}
\end{tikzpicture}
```

Note that you have to take care that the color components are within the expected bounds.

4.5.8 Contour Plots

PGFPLOTS supports visualization of contour plots whose coordinates have been computed by *external tools*. The `contour prepared` plot handler coming with PGFPLOTS takes precomputed contour line coordinates and handles their visualization (`contour/draw color`, `contour/labels` etc.). The `contour gnuplot` style takes matrix input in the same format as for `mesh` or `surf` (that includes any of the PGFPLOTS matrix input methods). It then writes the matrix data to a file and invokes `gnuplot` (or other, user customizable external programs) to compute contour coordinates. Finally, the computed contours are visualized with the `contour prepared` algorithm. Thus, external programs need to compute the contour coordinates and PGFPLOTS visualizes the result.

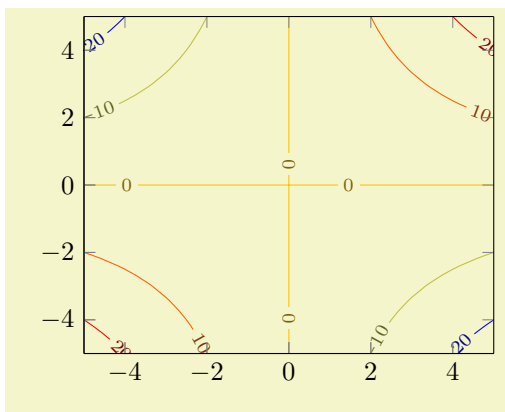
We discuss the high level interface to external programs first and continue with `contour prepared` later-on.

```
/pgfplots/contour gnuplot={options with 'contour/' or 'contour external/' prefix}}
```

```
\addplot+[contour gnuplot={options with 'contour/' or 'contour external/' prefix}]
```

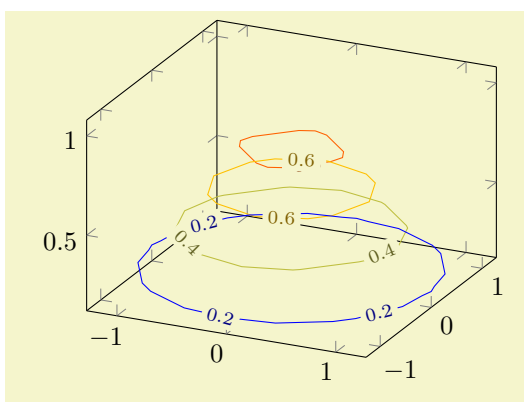
This is a high level contour plot interface. It expects matrix data in the same way as two dimensional `surf` or `mesh` plots do. It then computes contours and visualizes them.

²⁶If some expert in color space operations can contribute best-practices here, feel free to contact me.



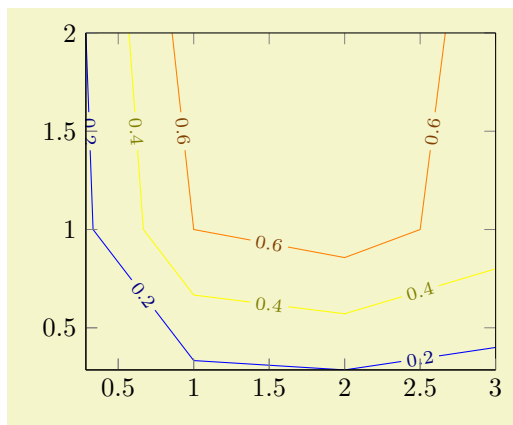
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={0}{90}]
    \addplot3[contour gnuplot]
      {x*y};
  \end{axis}
\end{tikzpicture}
```

The example uses `\addplot3` together with `expression` plotting, that means the input data is of the form $(x_i, y_i, f(x_i, y_i))$. The `view={0}{90}` flag means “view from top”, otherwise the contour lines would have been drawn as z value:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[contour gnuplot]
      {exp(0-x^2-y^2)};
  \end{axis}
\end{tikzpicture}
```

As mentioned, you can use any of the PGFPLOTS input methods as long as it yields matrix output. Thus, we can re-use our introductory example of matrix data, this time with inline data:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}%
\begin{tikzpicture}%
  \begin{axis}[view={0}{90}]%
    \addplot3[contour gnuplot]%
      coordinates {
        (0,0,0) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
        (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
      };
  \end{axis}%
\end{tikzpicture}%
```

What happens behind the scenes is that PGFPLOTS takes the input matrix and writes all encountered coordinates to a temporary file, including the end-of-scanline markers. Then, it generates a small `gnuplot` script and invokes `gnuplot` to compute the contour coordinates, writing everything into a temporary output file. Afterwards, it includes `gnuplot`'s output file just as if you'd write `\addplot3[contour prepared] file {<temporaryfile>};`.

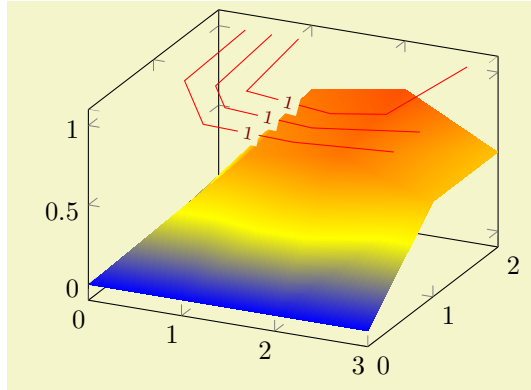
All this invocation of `gnuplot`, including input/output file management is transparent to the user. It only requires two things: first of all, it requires matrix data as input²⁷. Second, it requires you to enable

²⁷Note that `contour gnuplot` processes the input stream only once. Consequently, the temporary file will contain only information which was available before the first point has been seen. The example above works because it contains `emptylines` as end-of-scanline markers. If you do not provide such markers, you may need to provide two of the three options `mesh/rows`, `mesh/cols`, or `mesh/num points`.

system calls. Consider the documentation for `plot gnuplot` for how to enable system calls.

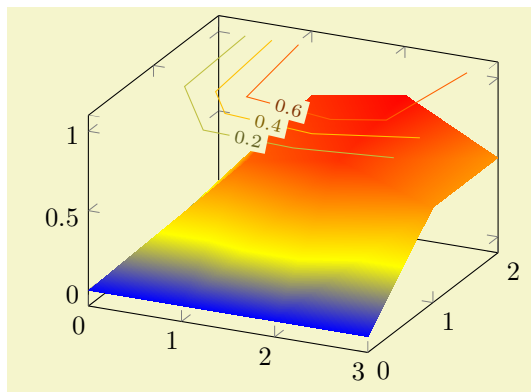
Note that the z coordinate of the data which is communicated to `gnuplot` is the current value of `point meta`. This allows to generate contours on two columns only and has more freedom. See also the `contour external/output point meta` key.

The resulting data can be projected onto a separate slice, for example using `z filter`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}%
\begin{tikzpicture}%
  \begin{axis}%
    \addplot3[surf,shader=interp]%
      coordinates {
        (0,0,0) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
        (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
      };
    \addplot3[
      contour gnuplot,
      z filter/.code={\def\pgfmathresult{1}},
    ]%
      coordinates {
        (0,0,0) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
        (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
      };
  \end{axis}%
\end{tikzpicture}%
```

An unexpected side-effect of a `z filter` combined with `contour gnuplot` is that the color information and the label sizes are essentially lost. To compensate this effect, we have to assign a new value for `point meta`. But since `point meta` defines the values for contour levels (see above), there is a special key named `output point meta` which can be used here:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}%
\begin{tikzpicture}%
  \begin{axis}%
    \addplot3[surf,shader=interp]%
      coordinates {
        (0,0,0) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
        (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
      };
    \addplot3[
      contour gnuplot={
        output point meta=rawz,
      },
      z filter/.code={\def\pgfmathresult{1}},
    ]%
      coordinates {
        (0,0,0) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0.6) (2,1,0.7) (3,1,0.5)
        (0,2,0) (1,2,0.7) (2,2,0.8) (3,2,0.5)
      };
  \end{axis}%
\end{tikzpicture}%
```

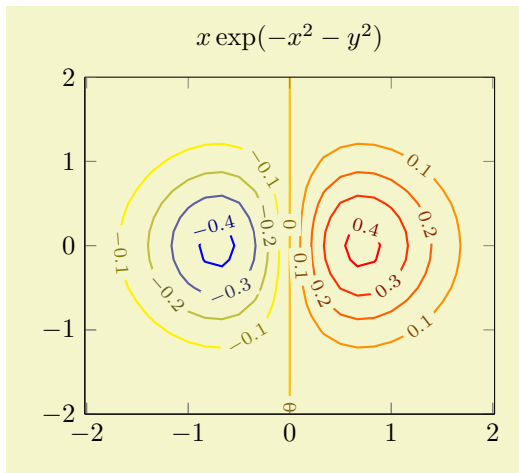
Here, `output point meta=rawz` allows to assign point meta to the *output* of the contouring algorithm, i.e. something which is handed over to `contour prepared`. The choice `rawz` means to use the unfiltered z coordinate which is what we want here.

There are several fine-tuning parameters of the input/output file management, and it is even possible to invoke different programs than `gnuplot` (even `matlab`). These details are discussed at the end of this section, see below at page 126.

`/pgfplots/contour/number={integer}`

(initially 5)

Configures the number of contour lines which should be produced by any external contouring algorithm.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,enlarge x limits,
    view={0}{90},
  ]
  \addplot3[contour gnuplot={number=14},thick]
    {exp(-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

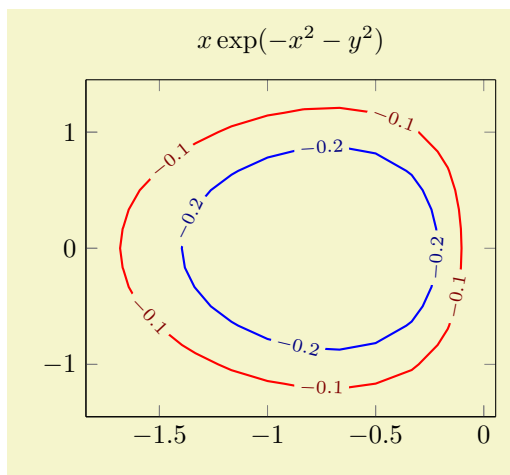
It is also possible to change the `/pgf/number` format settings, see the documentation for the `contour/every contour label` style below.

Note that `contour/number` has no effect on `contour prepared`.

`/pgfplots/contour/levels={list of levels}`

(initially empty)

Configures the number of contour lines which should be produced by any external contouring algorithm by means of a list of discrete levels.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,
    enlargelimits,
    view={0}{90},
  ]
  \addplot3[
    contour gnuplot={levels={-0.1,-0.2,-0.6}},
    thick
  ]
    {exp(-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

It is also possible to change the `/pgf/number` format settings, see the documentation for the `contour/every contour label` style below.

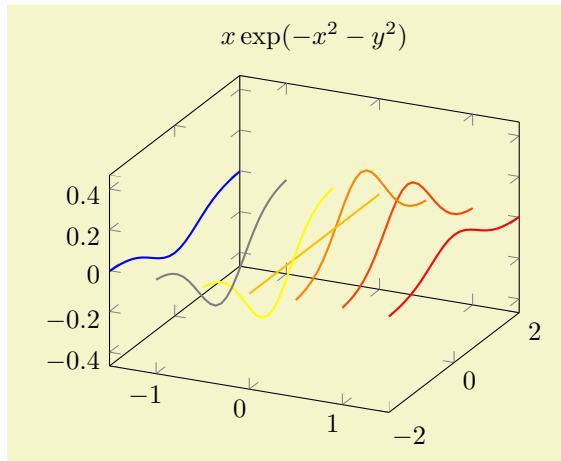
This key has higher precedence than `contour/number`, i.e. if both are given, `contour/levels` will be active.

Note that `contour/levels` has no effect on `contour prepared`.

`/pgfplots/contour/contour dir=x|y|z`

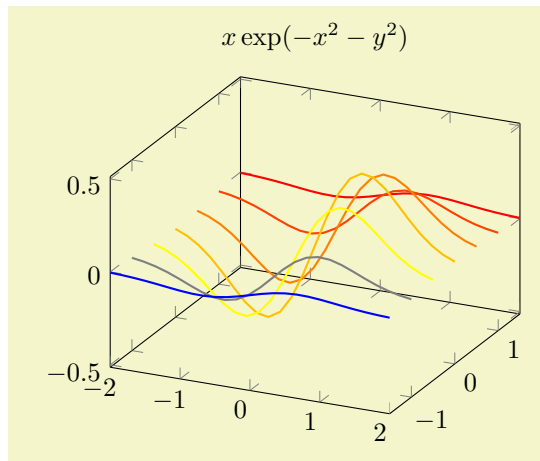
(initially z)

Allows to generate contours with respect to another direction.



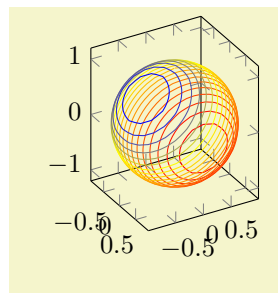
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,
  ]
  \addplot3[
    contour gnuplot={
      contour dir=x,labels=false,number=15},
    thick]
    {exp(-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

The input data is the same as before – it has to be given in matrix form. The key `contour dir` configures the algorithm to compute contours along the provided direction.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,
  ]
  \addplot3[
    contour gnuplot={
      contour dir=y,labels=false,number=15},
    thick]
    {exp(-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

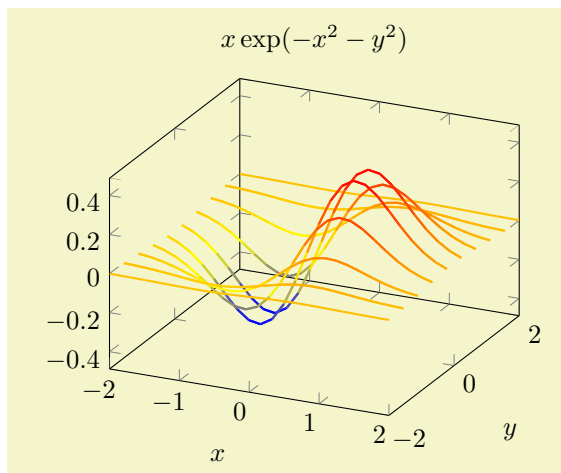
This function is also available for parameterized surfaces.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view={60}{30}, axis equal image]
  \addplot3[
    contour gnuplot={
      contour dir=x,
      number=20,
      labels=false,
    },
    samples=30,domain=-1:1,y domain=0:2*pi]
    ({sqrt(1-x^2) * cos(deg(y))},
    {sqrt(1-x^2) * sin(deg(y))},
    x);
\end{axis}
\end{tikzpicture}
```

Note, however, that each contour line receives a single color. This is what one expects for a contour plot: it has a single style, and a single contour level. Note furthermore that the color which is assigned to a contour plot with `contour dir=x` is *different* compared with the color assigned to a contour plot with `contour dir=z`: the argument of `contour dir` implicitly defines the argument for `point meta` (also known as color data). More precisely, a contour plot with `contour dir=x` has `point meta=x` whereas a contour plot with `contour dir=z` uses `point meta=z`.

If you would like to have individually colored segments inside of contours, you have to use a different plot handler. There is a simple alternative which works well in many cases: you can use a standard `mesh` plot combined with `patch type=line`:

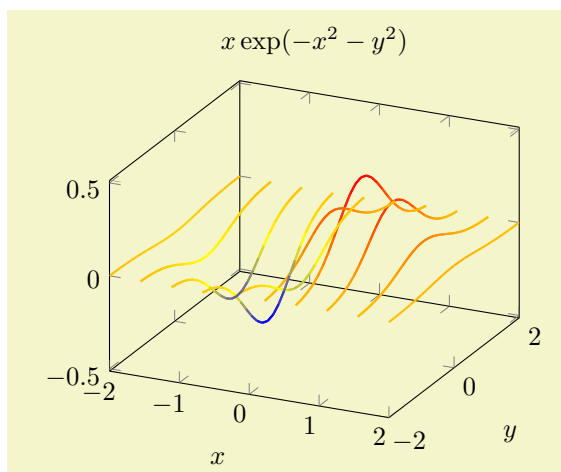


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,
    xlabel=$x$, ylabel=$y$,
  ]
  \addplot3[
    samples y=10, samples=25,
    mesh, patch type=line,
    thick]
    {exp(-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

Here, we did *not* generate a contour plot. We generated a `mesh` plot with `patch type=line`. The choice `patch type=line` causes an inherently one-dimensional plot as opposed to the default matrix-style visualization which would be generated by `mesh` in different cases. Since a mesh plot uses one color for every patch segment, we have a lot of freedom to color the segments. In the example above, we have the default configuration `point meta=z`, i.e. the z value defines the color.

The fact that a `mesh` plot with `patch type=line` yields almost the same output as `contour dir=y` is an artifact of the scanline encoding. Our example uses `\addplot3` expression which relies on `mesh/ordering=x` varies. If we visualize the resulting matrix by means of `patch type=line`, the visualization follows the scanlines which vary along the x axis. In our example, we used `samples y=10` to control the number of “contour lines”.

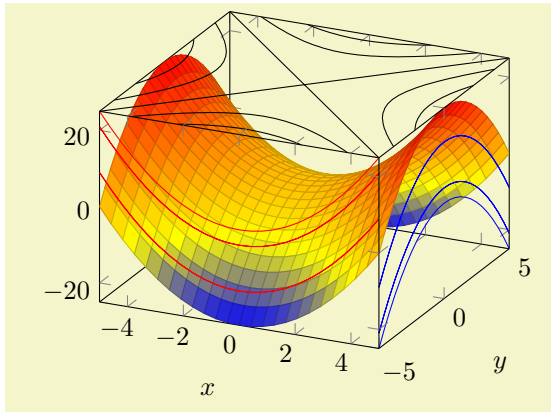
A consequence of the previous paragraph is that we have a more challenging task at hand if we want to get the same effect as `contour dir=x`: we would need `mesh/ordering=y` varies. In our case, we would need to transpose the data matrix. For `\addplot3` expression, this is relatively simple: we can exchange the meaning of x and y to get a transposition:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title={$x \exp(-x^2-y^2)$},
    domain=-2:2,
    xlabel=$x$, ylabel=$y$,
  ]
  \addplot3[
    samples y=10, samples=25,
    mesh, patch type=line,
    thick]
    (y,x,{exp(-x^2-y^2)*y});
  \end{axis}
\end{tikzpicture}
```

This is the same example as above – but as you noted, the meaning of x and y in the expression has been exchanged and the notation has been switched to a parametric plot. Such an approach is also possible for data files, but PGFPLOTS cannot transpose the matrix ordering on its own.

Coming back to `contour dir`, we can also use its output to generate several contour projections using coordinate filters.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    xlabel=$x$,ylabel=$y$,
    enlargelimits=false,
    3d box=complete]
\addplot3[surf]
    {x^2-y^2};

\addplot3[
    contour gnuplot={contour dir=y,
        draw color=red,labels=false},
    y filter/.code=\def\pgfmathresult{-5}
] {x^2-y^2};

\addplot3[
    contour gnuplot={contour dir=x,
        draw color=blue,labels=false},
    x filter/.code=\def\pgfmathresult{5}
] {x^2-y^2};

\addplot3[
    contour gnuplot={contour dir=z,
        draw color=black,labels=false},
    z filter/.code=\def\pgfmathresult{25}
] {x^2-y^2};

\end{axis}
\end{tikzpicture}
```

The preceding example uses a fixed `draw color` combined with `x filter`, `y filter`, and `z filter` to fix the contours in one of the axis planes.

Technical background: This section is probably unnecessary and can be skipped. The key `contour dir` is implemented by means of coordinate permutations. Since contouring algorithms always support `contour dir=z`, it is relatively simple to compute z -contour lines from input matrixes X, Y, Z . The choice `contour dir=z` key simply takes the input as-is. The choice `contour dir=x` reorders the input coordinates to yzx . The choice `contour dir=y` reorders the input coordinates to xzy . All this reordering is applied before coordinates are handed over to the contouring algorithm (see `contour external`) and is undone when reading the results back from the contouring algorithm. That means that `contour dir` is also available for `contour prepared`. In this context, `contour prepared` is supposed to be the output of some contouring algorithm. Its input coordinates are automatically reordered according to the inverse permutation. This allows to draw x or y contours which are given in the prepared format.

`/pgfplots/contour prepared={⟨options with ‘contour/’ prefix⟩}`

`\addplot+[contour prepared={⟨options with ‘contour/’ prefix⟩}]`

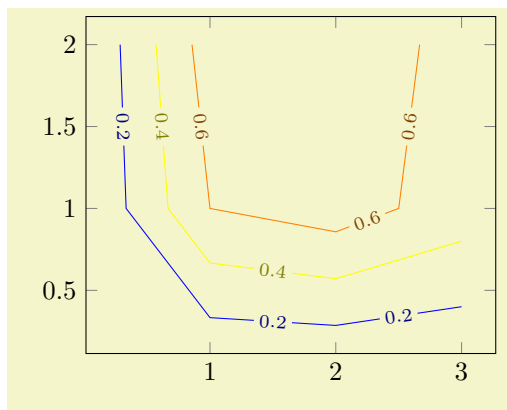
A plot handler which expects already computed contours on input and visualizes them. It cannot compute contours on its own.

`/pgfplots/contour prepared format=standard|matlab` (initially `standard`)

There are two accepted input formats. The first is a long sequence of coordinates of the form (x, y, z) where all successive coordinates with the same z value make up a contour level (this is only part of complete truth, see below). The end-of-scanline markers (`empty lines` in the input) mark an interruption in one contour level.

For example, `contour prepared format=standard` could be²⁸

²⁸This is actually the output from our `\addplot3[contour gnuplot]` coordinates example from above.



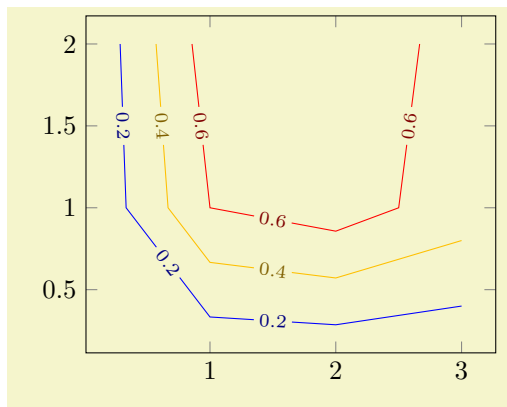
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[contour prepared]
      table {
        2 2 0.8
        0.857143 2 0.6
        1 1 0.6
        2 0.857143 0.6
        2.5 1 0.6
        2.66667 2 0.6

        0.571429 2 0.4
        0.666667 1 0.4
        1 0.666667 0.4
        2 0.571429 0.4
        3 0.8 0.4

        0.285714 2 0.2
        0.333333 1 0.2
        1 0.333333 0.2
        2 0.285714 0.2
        3 0.4 0.2
      };
  \end{axis}
\end{tikzpicture}
```

Note that the `empty lines` are not necessary in this case: `empty lines` make only a difference if they occur within the same contour level (i.e. if the same z value appears above and below of them).

The choice `contour prepared format=matlab` expects two-dimensional input data where the contour level and the number of elements of the contour line are provided as x and y coordinates, respectively, of a leading point. Such a format is used by `matlab`'s contour algorithms, i.e. it resembles the output of the `matlab` commands `data=contour(...)` or `data=contourc(...)`.



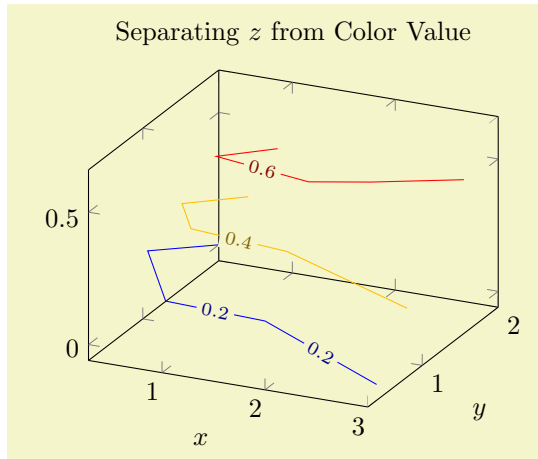
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[contour prepared,
      contour prepared format=matlab]
      table {
        % (0.2,5) ==> contour '0.2' (x), 5 points follow (y):
        2.0000000e-01 5.0000000e+00
        3.0000000e+00 4.0000000e-01
        2.0000000e+00 2.8571429e-01
        1.0000000e+00 3.3333333e-01
        3.3333333e-01 1.0000000e+00
        2.8571429e-01 2.0000000e+00
        % (0.4,5) ==> contour '0.4', consists of 5 points
        4.0000000e-01 5.0000000e+00
        3.0000000e+00 8.0000000e-01
        2.0000000e+00 5.7142857e-01
        1.0000000e+00 6.6666667e-01
        6.6666667e-01 1.0000000e+00
        5.7142857e-01 2.0000000e+00
        % (0.6,6) ==> contour '0.6', has 6 points
        6.0000000e-01 6.0000000e+00
        2.6666667e+00 2.0000000e+00
        2.5000000e+00 1.0000000e+00
        2.0000000e+00 8.5714286e-01
        1.0000000e+00 1.0000000e+00
        1.0000000e+00 1.0000000e+00
        8.5714286e-01 2.0000000e+00
      };
  \end{axis}
\end{tikzpicture}
```

In case you use `matlab`, you can generate such data with

```
[x,y]=meshgrid(linspace(0,1,15));
data=contour(x,y,x.*y);
```

```
data=data';
save 'exporteddata.dat' data -ASCII
```

As already mentioned in the beginning, the z coordinate is not necessarily the coordinate used to delimit contour levels. In fact, the `point meta` data is acquired here, i.e. you are free to use whatever z coordinate you want as long as you have a correct `point meta` value. The example from above could be modified as follows:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
title=Separating  $z$  from Color Value,
xlabel= $x$ ,
ylabel= $y$ ,
]
\addplot3[contour prepared,
point meta=\thisrow{level}]
table {
x      y      z      level
0.857143  2    0.4    0.6
1        1    0.6    0.6
2    0.857143  0.6    0.6
2.5    1      0.6    0.6
2.66667  2    0.4    0.6

0.571429  2    0.2    0.4
0.666667  1    0.4    0.4
1    0.666667  0.4    0.4
2    0.571429  0.4    0.4
3    0.8      0.2    0.4

0.285714  2    0      0.2
0.333333  1    0.2    0.2
1    0.333333  0.2    0.2
2    0.285714  0.2    0.2
3    0.4      0      0.2
};
\end{axis}
\end{tikzpicture}
```

The example above uses different z coordinates for each first and each last point on contour lines. The contour lines as such are defined by the `level` column since we wrote `point meta=\thisrow{level}`. Such a feature also allows `contour prepared` for nonstandard axes, compare the examples for the `ternary` lib on page 393.

`/pgfplots/contour/draw color={ $\langle color \rangle$ }` (initially mapped color)

Defines the draw color for every contour. Note that only mapped color actually depends on the contour level.

`/pgfplots/contour/labels={ $\langle true, false \rangle$ }` (initially true)

Configures whether contour labels shall be drawn or not.

`/pgfplots/contour/label distance={ $\langle dimension \rangle$ }` (initially 70pt)

Configures the distance between adjacent contour labels within the same contour level.

`/pgfplots/contour/every contour label` (style, no value)

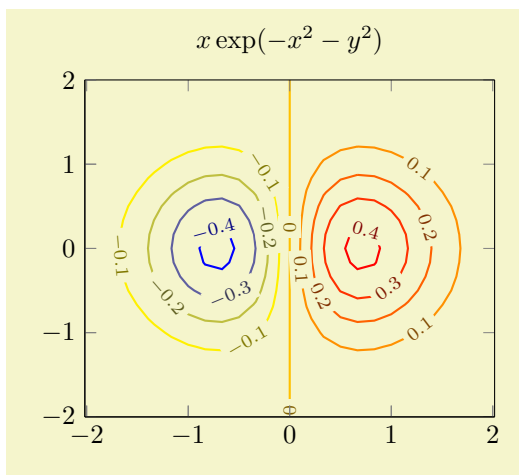
Allows to customize contour labels. The preferred way to change this style is the `contour label style={ $\langle options \rangle$ }` method, see below.

The initial value is

```
\pgfplotsset{
  contour/every contour label/.style={
    sloped,
    transform shape,
    inner sep=2pt,
    every node/.style={mapped color!50!black,fill=white},
    /pgf/number format/relative={\pgfplotspointmetarangeexponent},
  }
}
```

Note that `\pgfplotspointmetarangeexponent = e` where $\pm m \cdot 10^e$ is the largest occurring label value (technically, it is the largest occurring value of `point meta`).

The following example modifies the `/pgf/number format` styles for contour labels:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    \addplot3[
      contour gnuplot={
        scanline marks=required,
        number=14,
        contour label style={
          /pgf/number format/fixed,
          /pgf/number format/precision=1,
        },
      },thick
    ]
      {exp(0-x^2-y^2)*x};
  \end{axis}
\end{tikzpicture}
```

`/pgfplots/contour/contour label style={⟨key-value-list⟩}`

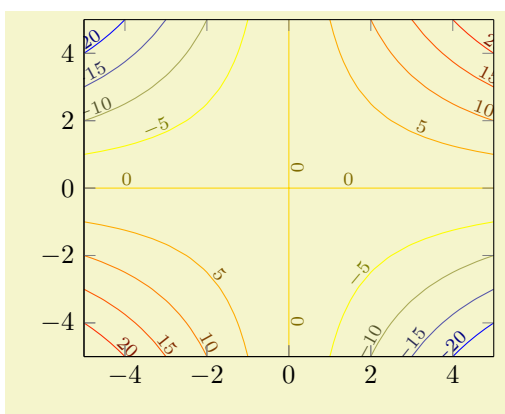
An abbreviation for `contour/every contour label/.append style={⟨key-value-list⟩}`.

It appends options to the already existing style `contour/every contour label`.

`/pgfplots/contour/labels over line`

(style, no value)

A style which changes `every contour label` such that labels are right over the lines, without fill color.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={0}{90}]
    \addplot3[contour gnuplot={
      labels over line,number=9]
      {x*y};
  \end{axis}
\end{tikzpicture}
```

`/pgfplots/contour/handler`

(style, no value)

Allows to modify the plot handler which connects the points of a single contour level.

The initial value is

```
\pgfplotsset{contour/handler/.style={/tikz/sharp plot}}
```

but a useful alternative might be the `smooth` handler.

`/pgfplots/contour/label node code/.code={\<...>}`

A lowlevel interface to modify how contour labels are placed.

The initial value is

```
\pgfplotsset{
  contour/label node code/.code={\node {\pgfmathprintnumber{#1}};};
}
```

`/pgfplots/contour external={\<options with ‘contour/’ or ‘contour external/’ prefix>}`

`\addplot+[contour external={\<options with ‘contour/’ or ‘contour external/’ prefix>}]`

This handler constitutes a generic interface to external programs to compute contour lines. The `contour gnuplot` method is actually a special case of `contour external`.

`/pgfplots/contour external/file={\<base file name>}` (initially empty)

The initial configuration is to automatically generate a unique file name.

`/pgfplots/contour external/scanline marks=false|if in input|required>true` (initially if in input)

Controls how `contour external` writes end-of-scanline markers.

The choice `false` writes no such markers at all. In this case, `script` should contain `mesh/rows` and/or `mesh/cols`.

The choice `if in input` generates end-of-scanline markers if they appear in the provided input data (either as `empty lines` or if the user provided at least two of the three options `mesh/rows`, `mesh/cols`, or `mesh/num points` explicitly).

The choice `required` works like `if in input`, but it will fail unless there really was such a marker.

The choice `true` is an alias for `required`.

`/pgfplots/contour external/script={\<Code for external program>}` (initially empty)

Provides template code to generate a script for the external program. Inside of `\<Code for external program>`, the placeholder `\infile` will expand to the temporary input file and `\outfile` to the temporary output file. The temporary `\infile` is a text file containing one point on each line, in the form `x y meta meta`, separated by tabstops. Whenever a scanline is complete, an `empty line` is issued (but only if these scanline markers are found in the input stream as well). The complete set of scanlines forms a matrix. There are no additional comments or extra characters in the file. The macro `\ordering` will expand to 0 if the matrix is stored in `mesh/ordering=x varies` and `\ordering` will be 1 for `mesh/ordering=y varies`.

Inside of `\<Code for external program>`, you can also use `\pgfkeysvalueof{/pgfplots/mesh/rows}` and `\pgfkeysvalueof{/pgfplots/mesh/cols}`; they expand to the matrix' size. Similarly, `\pgfkeysvalueof{/pgfplots/mesh/num points}` expands to the total number of points.

Inside of `\<Code for external program>`, the macro `\thecontournumber` is defined to be the value `\pgfkeysvalueof{/pgfplots/contour/number}` and `\thecontourlevels` contains the value `\pgfkeysvalueof{/pgfplots/contour/levels}`. These two macros simplify conditional code.

If you need one of the characters `"|;:##'‘` and some macro package already uses the character for other purposes, you can prepend them with a backslash, i.e. write `\` instead of `"`.

`/pgfplots/contour external/script extension={\<extension>}` (initially `script`)

The file name extension for the temporary script.

`/pgfplots/contour external/cmd={\<system call>}` (initially empty)

A template to generate system calls for the external program. Inside of `\<system call>`, you may use `\script` as placeholder for the filename which contains the result of `contour external/script`.

`/pgfplots/contour external/output point meta={\<point meta read from result of external tool>}` (initially empty)

Allows to customize the `point meta` configuration which is applied to the result of the external tool.

In `contour external`, the value of `point meta` is used to generate the *input* z coordinate for the external tool.

As soon as the external tool computed contour lines, its output is read and interpreted as contour lines – and the value of `output point meta` determines the value of `point meta` which will be used to visualize the result.

An empty value means to use the z coordinate returned by the external tool.

Any other value is interpreted as a valid choice of `point meta`.

`/pgfplots/contour gnuplot`

(style, no value)

The initial configuration is

```
\pgfplotsset{
  contour gnuplot/.style={
    contour external={
      script={
        unset surface;
        \ifx\thecontourlevels\empty
          set cncparam levels \thecontournumber;
        \else
          set cncparam levels discrete \thecontourlevels;
        \fi
        set contour;
        set table "\outfile\";
        splot "\infile\";
      },
      cmd={gnuplot "\script\"},
      #1,%
    },
  },
}
```

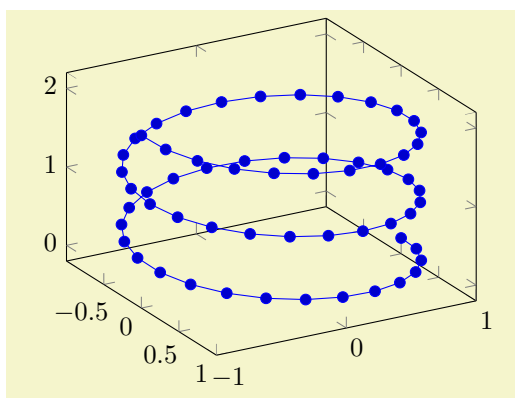
Note that `contour gnuplot` requires explicit scanline markers in the input stream, and it assumes `mesh/ordering=x` varies.

Note that `contour external` lacks the intelligence to detect changes; it will always re-generate the output (unless the `-shell-escape` feature is not active).

4.5.9 Parameterized Plots

Parameterized plots use the same plot types as documented in the preceding sections: both `mesh` and `surface` plots are actually special parametrized plots where x and y are on cartesian grid points.

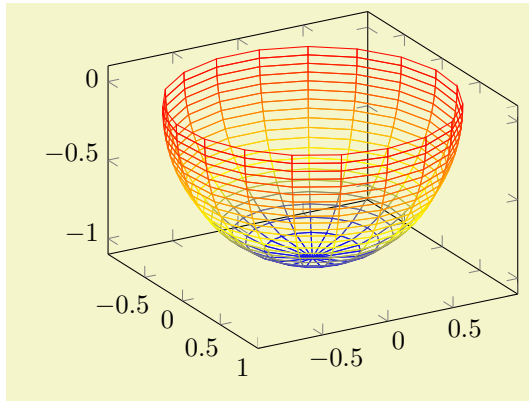
Parameterized plots just need a special way to provide the coordinates:



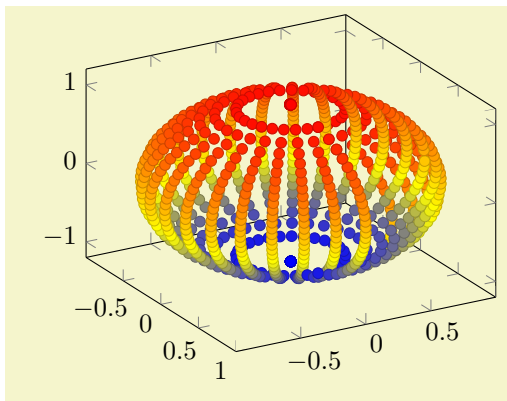
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={60}{30}]
    \addplot3+[domain=0:5*pi,samples=60,samples y=0]
      ({sin(deg(x))},
       {cos(deg(x))},
       {2*x/(5*pi)});
  \end{axis}
\end{tikzpicture}
```

The preceding example uses `samples y=0` to indicate that a line shall be sampled instead of a matrix. The curly braces are necessary because T_EX can't nest round braces. The single expressions here are used to parametrize the helix.

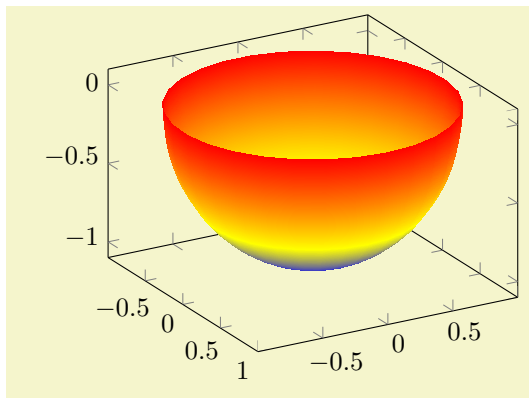
Another example follows. Note that `z buffer=sort` is a necessary method here.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view={60}{30}]
\addplot3[mesh,z buffer=sort,
samples=20,domain=-1:0,y domain=0:2*pi]
({sqrt(1-x^2) * cos(deg(y))},
{sqrt(1-x^2) * sin(deg(y))},
x);
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view={60}{30}]
\addplot3[mesh,z buffer=sort,
scatter,only marks,scatter src=z,
samples=30,domain=-1:1,y domain=0:2*pi]
({sqrt(1-x^2) * cos(deg(y))},
{sqrt(1-x^2) * sin(deg(y))},
x);
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view={60}{30}]
\addplot3[surf,shader=interp,z buffer=sort,
samples=30,domain=-1:0,y domain=0:2*pi]
({sqrt(1-x^2) * cos(deg(y))},
{sqrt(1-x^2) * sin(deg(y))},
x);
\end{axis}
\end{tikzpicture}
```

4.5.10 3D Quiver Plots (Arrows)

Three dimensional **quiver** plots are possible with the same interface as their two-dimensional counterparts, simply provide the third coordinate using **quiver/w**. Please refer to Section 4.4.7 for details and examples.

4.5.11 About 3D Const Plots and 3D Bar Plots

There are currently *no* equivalents of **const plot** and its variants or the bar plot types like **ybar** for three dimensional axes, sorry.

4.5.12 Patch Plots

/pgfplots/**patch**

(no value)

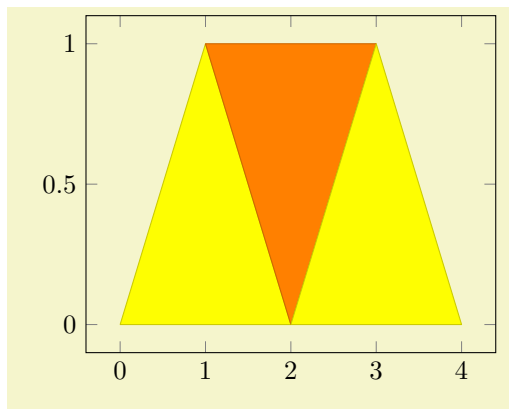
\addplot+**[patch]**

Patch plots are similar to **mesh** and **surf** plots in that they describe a filled area by means of a geometry.

However, `patch` plots are defined by *explicitly* providing the elements of the geometry: they expect a sequence of triangles (or other `patch types`) which make up the mesh.

There are two dimensional and three dimensional patch plots, both with the same interfaces which are explained in the following sections.

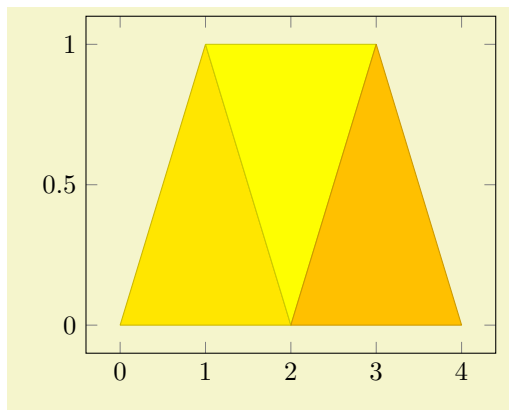
The standard input format (constituted by `mesh input=patches`) is to provide a sequence of coordinates (either two- or three-dimensional) as usual. Each consecutive set of points makes up a patch element, which is often a triangle:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[patch]
      table {
        x y
        0 0
        1 1
        2 0
      }
      % empty lines do not hurt, they are ignored here:
      1 1
      2 0
      3 1

      2 0
      3 1
      4 0
    };
  \end{axis}
\end{tikzpicture}
```

Patch plots use `point meta` to determine fill colors. In its initial configuration, `point meta` will be set to the y coordinate (or the z coordinate for three dimensional `patch` plots). Set `point meta` somehow to color the patches:

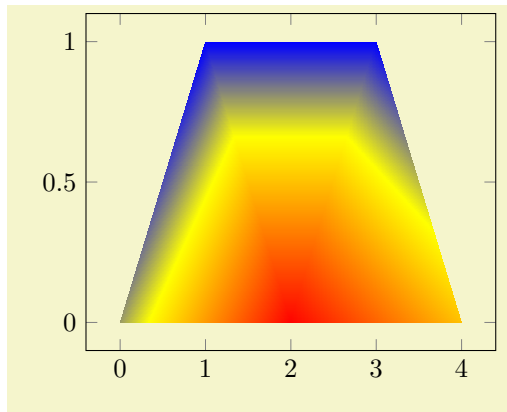


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[patch]
      table[point meta=\thisrow{c}] {
        x y c
        0 0 0.2
        1 1 0
        2 0 1

        1 1 0
        2 0 1
        3 1 0

        2 0 1
        3 1 0
        4 0 0.5
      }
    };
  \end{axis}
\end{tikzpicture}
```

Patch plots make use of the `mesh` configuration, including the `shader`. Thus, the example above uses the initial `shader=faceted` (which uses the *mean* color data to determine a triangle's color and a related stroke color). The `shader=interp` yields the following result:

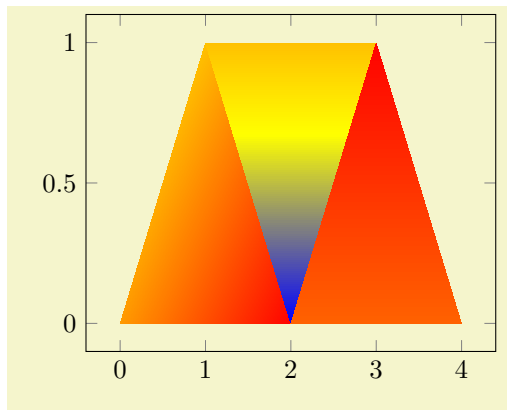


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[patch,shader=interp]
      table[point meta=\thisrow{c}] {
        x y c
        0 0 0.2
        1 1 0
        2 0 1

        1 1 0
        2 0 1
        3 1 0

        2 0 1
        3 1 0
        4 0 0.5
      };
  \end{axis}
\end{tikzpicture}
```

For triangles, `shader=interp` results in linearly interpolated `point meta` values throughout each individual triangle, which are then mapped to the color map (a technique also known as Gouraud shading). The color data does not need to be continuous, it is associated to triangle vertices. Thus, changing some of the color values allows individually shaded regions:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[patch,shader=interp]
      table[point meta=\thisrow{c}] {
        x y c
        0 0 0.2
        1 1 0
        2 0 1

        1 1 0
        2 0 -1
        3 1 0

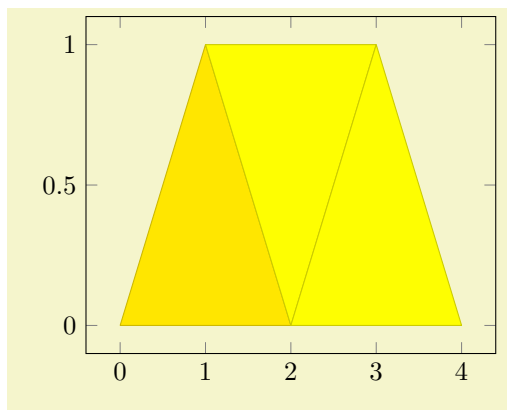
        2 0 0.5
        3 1 1
        4 0 0.5
      };
  \end{axis}
\end{tikzpicture}
```

Two dimensional `patch` plots simply draw triangles in their order of appearance. In three dimensions, single elements are sorted according to their view depth, with foreground elements drawn on top of background elements (“Painter’s algorithm”, see `z buffer=sort`).

```
/pgfplots/patch table={⟨table file name or inline table⟩} (initially empty)
/pgfplots/patch table with point meta={⟨table file name or inline table⟩} (initially empty)
/pgfplots/patch table with individual point meta={⟨table file name or inline table⟩} (initially empty)
```

Allows to provide patch connectivity data stored in an input table.

A non-empty argument for `patch table` enables patch input mode. Now, the standard input stream is a long list of vertices which are stored in an array using their `\coordindex` as key. Each row of `⟨table file name or inline table⟩` makes up one patch, defined by indices into the vertex array:

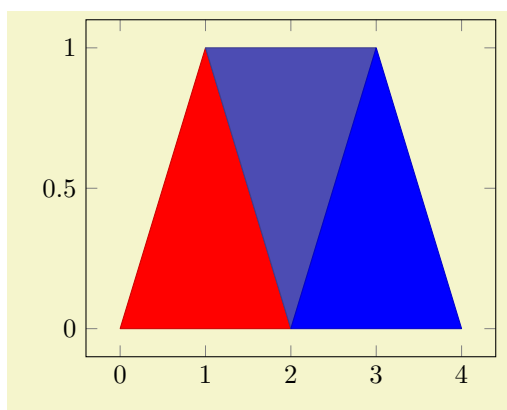


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[patch,table/row sep=\\,patch table={%
      0 1 2\\
      1 2 3\\
      4 3 5\\
    }]
    table[row sep=\\,point meta=\thisrow{c}] {
      x y c \\
      0 0 0.2\\% 0
      1 1 0 \\% 1
      2 0 1 \\% 2
      3 1 0 \\% 3
      2 0 0.5\\% 4
      4 0 0.5\\% 5
    };
  \end{axis}
\end{tikzpicture}
```

The example consists of *two separate* tables. The `patch table` argument is a table, provided inline where rows are separated by `\\` (which is the purpose of the `row sep=\\` key as you guessed²⁹). The `patch table` here declares three triangles: the triangle made up by vertex #0, #1 and #2, the triangle made up by #1, #2 and #3 and finally the one using the vertices #4, #3 and #5. The vertices as such are provided using the standard input methods of PGFLOTS; in our case using a table as well. The standard input simply provides coordinates (and `point meta`) which are stored in the vertex array; you could also have used `plot coordinates` to provide them (or `plot expression`).

The argument to `patch table` needs to be a table – either a file name or an inline table as in the example above. The first n columns of this table are assumed to contain indices into the vertex array (which is made up using all vertices of the standard input as explained in the previous paragraph). The entries in this table can be provided in floating point, just make sure they are not rounded. The variable n is the number of vertices required to make up a single patch. For triangular patches, it is $n = 3$, for `patch type=bilinear` it is $n = 4$ and similar for other choices of `patch type`.

The alternative `patch table with point meta` is almost the same as `patch table` – but it allows to provide (a single) `point meta` (color data) per patch instead of per vertex. Here, a further column of the argument table is interpreted as color data:

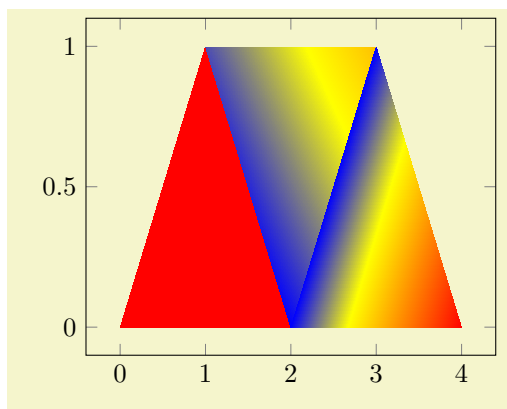


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    % this uses per-patch color data:
    \addplot[patch,table/row sep=\\,
      patch table with point meta={%
        0 1 2 100\\
        1 2 3 10\\
        4 3 5 0\\
      }]
    table[row sep=\\] {
      x y \\
      0 0 \\% 0
      1 1 \\% 1
      2 0 \\% 2
      3 1 \\% 3
      2 0 \\% 4
      4 0 \\% 5
    };
  \end{axis}
\end{tikzpicture}
```

The `patch table with point meta` always prefers `point meta` data from the provided table argument. However, it is still supported to write `point meta=\thisrow{<colname>}` or similar constructs – but now, `<colname>` refers to the provided table argument. More precisely, `point meta` is evaluated in a context where the patch connectivity has been resolved and the `patch table with point meta` is loaded.

²⁹Note that the choice `row sep=\\` is much more robust here: newlines would be converted to spaces by T_EX before PGFLOTS had a chance to see them.

The other alternative `patch table with individual point meta` is very similar, but instead of a flat color per patch, it allows to write one color value for every patch:



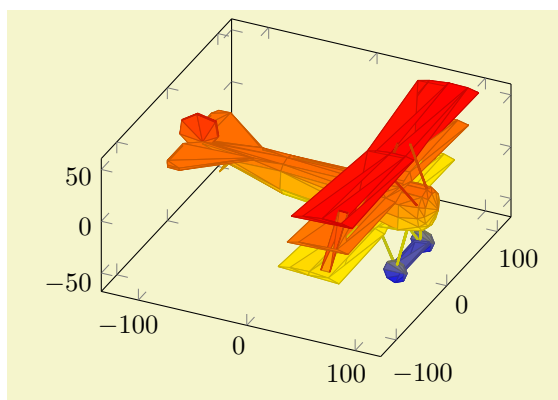
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
% this uses n per-patch color values:
\addplot[patch,shader=interp,
table/row sep=\\,
patch table with individual point meta={%
0 1 2 100 100 100\\% V_0 V_1 V_2 C_0 C_1 C_2
1 2 3 10 0 50\\
4 3 5 0 0 100\\
}]
table[row sep=\\] {
x y \\
0 0 \\% 0
1 1 \\% 1
2 0 \\% 2
3 1 \\% 3
2 0 \\% 4
4 0 \\% 5
};
\end{axis}
\end{tikzpicture}
```

To find the `point meta` data for vertex $\#i$, $i = 0, 1, 2$, PGFPLOTS searches in column $i + n$ where n is the number of vertices for `patch type` (in our case, $n = 3$).

Technical remark: The key `patch table with individual point meta` automatically installs `point meta=explicit` as well. It might be confusing to override the value of `point meta` here (although it is allowed).

The `patch table` input type allows to reduce the size of geometries since vertices are stored just once. PGFPLOTS unpacks them into memory into the redundant format in order to work with single patch elements³⁰. In case you experience TEX memory problems with this connectivity input, consider using the redundant format. It uses other types of memory limits.

A more involved example is shown below; it uses `\addplot3[patch]` to visualize a three dimensional `patch` plot, provided by means of a long sequence of patches:



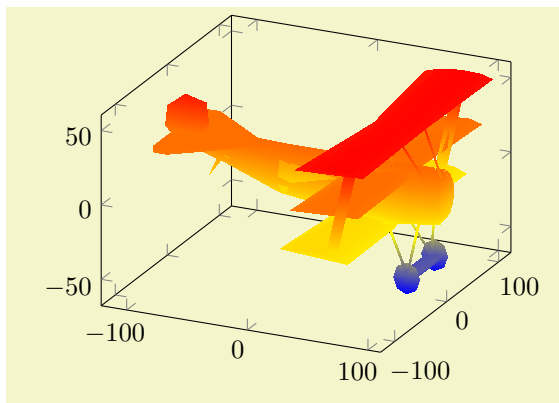
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis equal]
% FokkerDrI_layer_0.patches.dat contains:
% # each row is one vertex; three consecutive
% # vertices make one triangle (patch)
% 105.577 -19.7332 2.85249
% 88.9233 -21.1254 13.0359
% 89.2104 -22.1547 1.46467
% # end of facet 0
% 105.577 -19.7332 2.85249
% 105.577 -17.2161 12.146
% 88.9233 -21.1254 13.0359
% # end of facet 1
\addplot3[patch]
file
{plotdata/FokkerDrI_layer_0.patches.dat};
\end{axis}
\end{tikzpicture}
```

The ordering in which triangles are specified is irrelevant, three-dimensional patch plots use `z buffer=sort` to sort patches according to their depth (defined as mean depth over each vertex), where foreground patches are drawn on top of background patches. This so-called “Painter’s algorithm” works well for most meshes. If it fails, consider using `patch refines=1` or `patch refines=2` to split larger elements into small ones automatically.

³⁰The reason for such an approach is that TEX doesn’t really know what an array is – and according to my experience, arrays implemented by macros tend to blow up TEX’s memory limits even faster than the alternative.

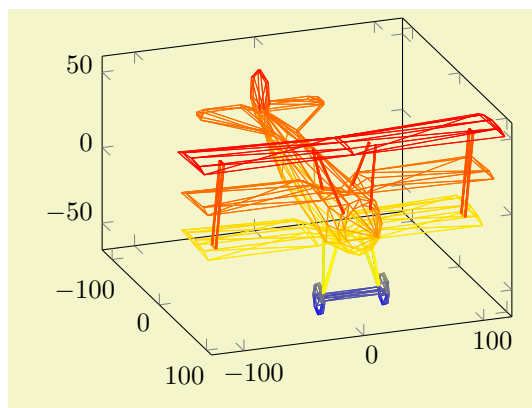
The drawing color associated to single vertices can be changed using the `point meta` key (which is the common method to configure color data in PGFplots). The initial configuration is `point meta=z` for three dimensional `patch` plots, i.e. to use the z coordinate also as color data. Use `point meta=\thisrow{colname}` in conjunction with `\addplot3[patch]` table to load a selected table column.

Patch plots are (almost) the same as `mesh` or `surf` plots, they only have more freedom in their input format (and a more complicated geometry). Actually, “`patch`” is just a style for `surf,mesh input=patches`. In other words, `patch` is the same as `surf`, it even shares the same internal implementation. Thus, most of the keys to configure `mesh` or `surf` plots apply to `patch` as well, especially `shader` and `z buffer`. As already mentioned, `\addplot3[patch]` automatically activates `z buffer=sort` to ensure a good drawing sequence. The `shader` can be used to modify the appearance:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
% FokkerDrI_layer_0.facetIdx.dat contains:
% # each row makes up one facet; it
% # consists of 0-based indices into
% # the vertex array
% 0 1 2 % triangle of vertices #0,#1 and #2
% 0 3 1 % triangle of vertices #0,#3 and #1
% 3 4 1
% 5 6 7
% 6 8 7
% 8 9 7
% 8 10 9
% ...
% while FokkerDrI_layer_0.vertices.dat contains
% 105.577 -19.7332 2.85249 % vertex #0
% 88.9233 -21.1254 13.0359 % vertex #1
% 89.2104 -22.1547 1.46467 % vertex #2
% 105.577 -17.2161 12.146
% 105.577 -10.6054 18.7567
% 105.577 7.98161 18.7567
% 105.577 14.5923 12.146
% ...
\addplot3[patch,shader=interp,
patch table=
{plotdata/FokkerDrI_layer_0.facetIdx.dat}]
file
{plotdata/FokkerDrI_layer_0.vertices.dat};
\end{axis}
\end{tikzpicture}
```

See the description of `shader=interp` for details and remarks. The example above makes use of the alternative syntax to provide a geometry: the `patch table` input. It allows to provide vertices separate from patch connectivity, where each patch is defined using three indices into the vertex array as discussed above.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view/h=70]
% FokkerDrI_layer_0.patches.dat contains:
% # each row is one vertex; three consecutive
% # vertices make one triangle (patch)
% 105.577 -19.7332 2.85249
% 88.9233 -21.1254 13.0359
% 89.2104 -22.1547 1.46467
% # end of facet 0
% 105.577 -19.7332 2.85249
% 105.577 -17.2161 12.146
% 88.9233 -21.1254 13.0359
% # end of facet 1
\addplot3[patch,mesh]
file
{plotdata/FokkerDrI_layer_0.patches.dat};
\end{axis}
\end{tikzpicture}
```

/pgfplots/mesh input=lattice|patches

This key controls how input coordinates are decoded to get patches. It is used only if `patch table` is empty (`patch table` has its own way to decode input coordinates). Usually, you won't need to bother with this key as it is set implicitly.

The choice `mesh input=lattice` is the initial configuration for `mesh` and `surf` plots: it expects input in a compact matrix form as described at the beginning of this section starting with page 90 and requires a `mesh/ordering` and perhaps end-of-scanline markers. It yields patches with exactly four corners and is compatible with `patch type=rectangle` and `patch type=bilinear` (the latter requiring to load the `patchplots` library).

The choice `mesh input=patches` is implicitly set when you use the `patch` style (remember that `surf` is actually some sort of patch plot on its own). It expects the input format as described for `patch` plots, i.e. n consecutive coordinates make up the vertices of a single patch where n is the expected number of vertices for the configured `patch type`.

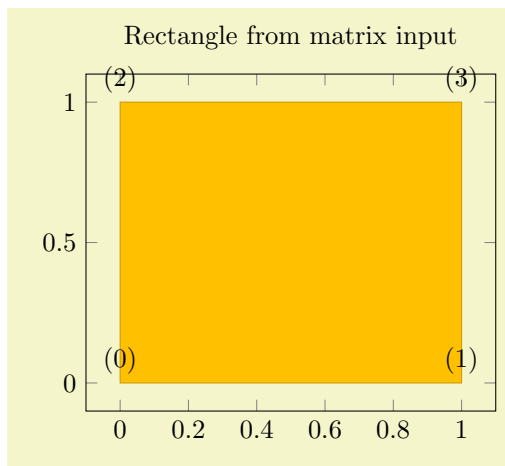
Note that a non-empty `patch table` implies `mesh input=patches`.

/pgfplots/patch type=default|rectangle|triangle|line (initially default)

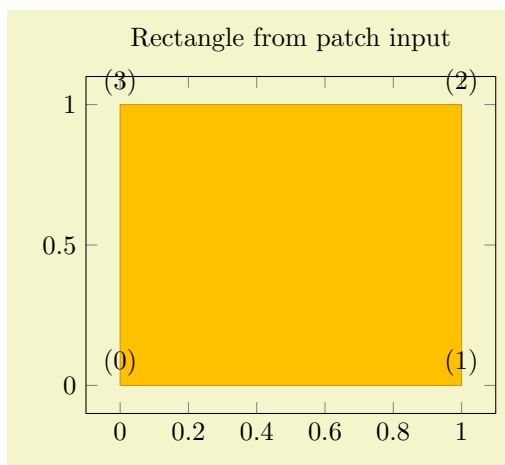
Defines the type of patch.

The initial configuration `patch type=default` checks the configuration of `mesh input`: for `mesh input=patches`, it uses `triangle`. For `mesh input=lattice`, it checks if there is just one row or just one col and uses `patch type=line` in such a case, otherwise it uses `patch type=rectangle`.

The choice `patch type=rectangle` expects $n = 4$ vertices. The vertices can be encoded as a matrix or, using `mesh input=patches`, in the sequence in which you would connect the vertices:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[nodes near coords={(\coordindex)},
    title=Rectangle from matrix input]
    % note that surf implies 'patch type=rectangle'
    \addplot[surf,mesh/rows=2,patch type=rectangle]
      coordinates {
        (0,0) (1,0)
        (0,1) (1,1)
      };
  \end{axis}
\end{tikzpicture}
```

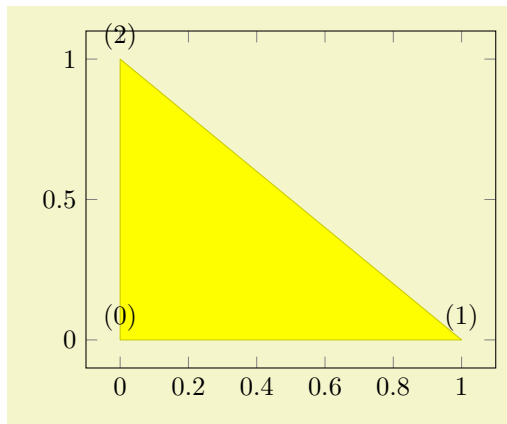


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[nodes near coords={(\coordindex)},
    title=Rectangle from patch input]
    \addplot[patch,patch type=rectangle]
      coordinates {
        (0,0) (1,0) (1,1) (0,1)
      };
  \end{axis}
\end{tikzpicture}
```

As for all other `patch type` values, the vertices can be arbitrary two- or three-dimensional points, there may be even two on top of each other (resulting in a triangle). When used together with `shader=interp`, `patch type=rectangle` is visualized using two Gouraud shaded triangles (see

below for `triangle`). It is the *most efficient* representation for interpolated shadings together with `mesh input=lattice` since the input lattice is written directly into the pdf. Use `patch type=rectangle` if you want rectangular elements and perhaps “some sort” of smooth shading. Use `patch type=bilinear` of the `patchplots` library in case you need real bilinear shading.

The choice `patch type=triangle` expects $n = 3$ vertices which make up a triangle. The ordering of the vertices is irrelevant:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[nodes near coords={(\coordindex)}]
    \addplot[patch,patch type=triangle]
      coordinates {
        (0,0) (1,0) (0,1)
      };
  \end{axis}
\end{tikzpicture}
```

The use of `shader=interp` is realized by means of linear interpolation of the three color values (specified with the `point meta` key) between the corners; the resulting interpolated `point meta` values are then mapped into the actual `colormap`. This type of interpolation is called Gouraud shading.

The choice `patch type=line` expects $n = 2$ vertices which make up a line. It is used for one-dimensional `mesh` plots (see Section 4.4.11 for examples).

There are more values for `patch type` like `bilinear`, `triangle quadr`, `biquadratic`, `coons`, `polygon` and `tensor bezier`. Please refer to the separate `patchplots` library in Section 5.6.

`/pgfplots/every patch`

(style, no value)

This style will be installed as soon as the `patch` plot handler is activated.

The initial configuration is

```
\pgfplotsset{
  every patch/.style={miter limit=1}
}
```

which improves display of sharp triangle corners significantly (see the TikZ manual for details about `miter limit` and line join parameters).

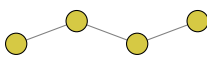
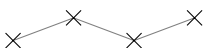
There is much more to say about patch plots, like `patch type` which allows triangles, bilinear elements, quadratic triangles, biquadratic quadrilaterals, coons patches; the `patch refines` key which allows automatic refinement, `patch to triangles` which triangulates higher order elements; how matrix data can be used for rectangular shapes and more. These details are subject of the `patchplots` library in Section 5.6.

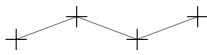
4.6 Markers, Linestyles, (Background-) Colors and Colormaps

The following options of TikZ are available to plots.

4.6.1 Markers

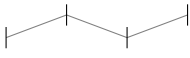
This list is copied from [5, section 29]:

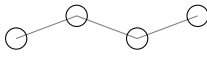
```
mark=* 
mark=x 
```

mark=+ 

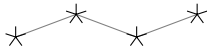
And with `\usetikzlibrary{plotmarks}`:

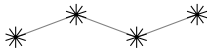
mark=- 

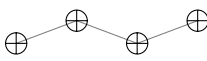
mark=| 

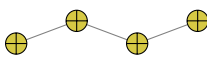
mark=o 

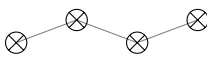
mark=asterisk 

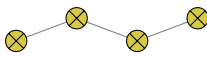
mark=star 

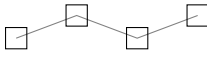
mark=10-pointed star 

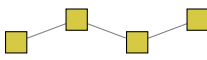
mark=oplus 

mark=oplus* 

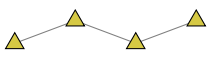
mark=otimes 

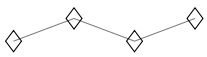
mark=otimes* 

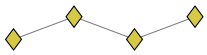
mark=square 

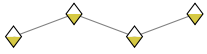
mark=square* 

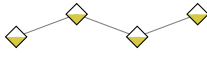
mark=triangle 

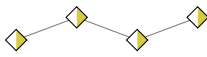
mark=triangle* 

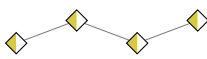
mark=diamond 

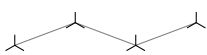
mark=diamond* 

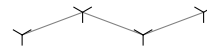
mark=halfdiamond* 

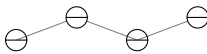
mark=halfsquare* 

mark=halfsquare right* 

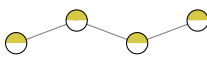
mark=halfsquare left* 

mark=Mercedes star 

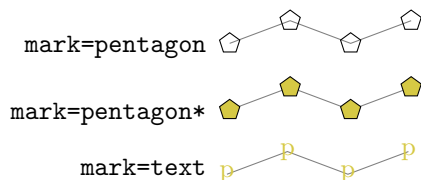
mark=Mercedes star flipped 

mark=halfcircle 

One half is filled with white (more precisely, with `mark color`).

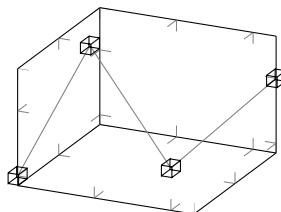
mark=halfcircle* 

One half is filled with white (more precisely, with `mark color`) and the other half is filled with the actual `fill` color.



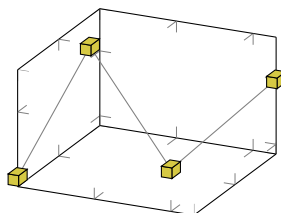
This marker is special as it can be configured freely. The character (or even text) used is configured by a set of variables, see below.

`mark=cube`



This marker is only available inside of a PGFPLOTS axis, it draws a cube with axis parallel faces. Its dimensions can be configured separately, see below.

`mark=cube*`



User defined It is possible to define new markers with `\pgfdeclareplotmark`, see below.

All these options have been drawn with the additional options

```
\draw[
  gray,
  thin,
  mark options={%
    scale=2,fill=yellow!80!black,draw=black
  }
]
```

Please see Section 4.6.5 for how to change `draw` and `fill` colors. Note that each of the provided marks can be rotated freely by means of `mark options={rotate=90}` or `every mark/.append style={rotate=90}`.

`/tikz/mark size={⟨dimension⟩}`

(initially 2pt)

This TikZ option allows to set marker sizes to `⟨dimension⟩`. For circular markers, `⟨dimension⟩` is the radius, for other plot marks it is about half the width and height.

`/pgfplots/cube/size x={⟨dimension⟩}`

(initially `\pgfplotmarksize=2pt`)

`/pgfplots/cube/size y={⟨dimension⟩}`

(initially `\pgfplotmarksize=2pt`)

`/pgfplots/cube/size z={⟨dimension⟩}`

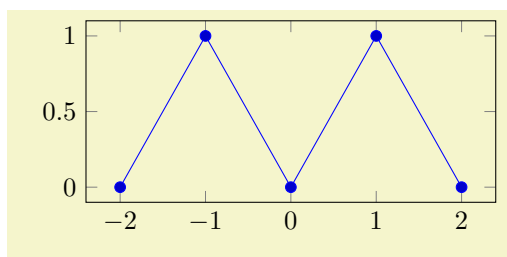
(initially `\pgfplotmarksize=2pt`)

Sets the size for `mark=cube` separately for every axis.

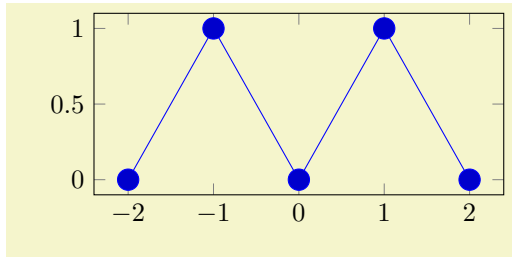
`/tikz/every mark`

(no value)

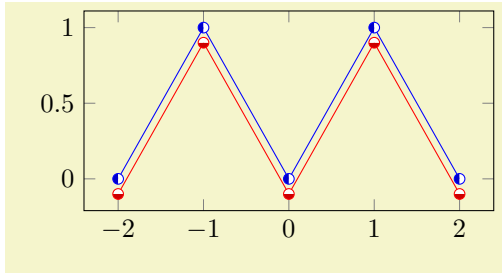
This TikZ style can be reconfigured to set marker appearance options like colors or transformations like scaling or rotation. PGFPLOTS appends its `cycle list` options to this style.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=2cm]
  \addplot coordinates
    {(-2,0) (-1,1) (0,0) (1,1) (2,0)};
\end{axis}
\end{tikzpicture}
```

```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\tikzset{every mark/.append style={scale=2}}
\begin{tikzpicture}
\begin{axis}[y=2cm]
\addplot coordinates
{(-2,0) (-1,1) (0,0) (1,1) (2,0)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=2cm]
\addplot+[
mark=halfcircle*,
every mark/.append style={rotate=90}]
coordinates
{(-2,0) (-1,1) (0,0) (1,1) (2,0)};

\addplot+[
mark=halfcircle*,
every mark/.append style={rotate=180}]
coordinates
{(-2,-0.1) (-1,0.9) (0,-0.1) (1,0.9) (2,-0.1)};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/no markers` (style, no value)

A key which overrides any `mark` value set by `cycle list` of option lists after `\addplot`.

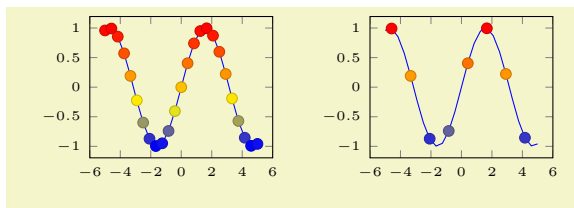
If this style is provided as argument to a complete axis, it is appended to `every axis plot post` such that it disables markers even for `cycle lists` which contain markers.

`/tikz/mark repeat={⟨integer⟩}` (initially empty)

Allows to draw only each n th `mark` where n is provided as `⟨integer⟩`.

`/tikz/mark phase={⟨integer p⟩}` (initially 1)

This option allows to control which markers are drawn. It is primarily used together with the `TikZ` option `mark repeat=r`: it tells `TikZ` that the first mark to be draw should be the p th, followed by the $(p+r)$ th, then the $(p+2r)$ th, and so on.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny]
\addplot+[scatter] {sin(deg(x))};
\end{axis}
\end{tikzpicture}
\begin{tikzpicture}
\begin{axis}[tiny]
\addplot+[scatter,
mark repeat=3,mark phase=2]
{sin(deg(x))};
\end{axis}
\end{tikzpicture}
```

Here, $p = 1$ is the first point (the one with `\coordindex=0`).

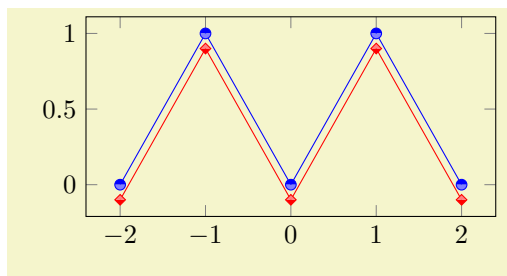
`/tikz/mark indices={⟨index list⟩}` (initially empty)

Allows to draw only the marker whose index numbers are in the argument list.

`/pgf/mark color={⟨color⟩}` (initially empty)

Defines the *additional* fill color for the `halfcircle`, `halfcircle*`, `halfdiamond*` and `halfsquare*` markers. An empty value uses `white` (which is the initial configuration). The value `none` disables filling for this part.

These markers have two distinct fill colors, one is determined by `fill` as for any other marker and the other one is `mark color`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=2cm]
\addplot[
  blue,mark color=blue!50!white,
  mark=halfcircle*]
  coordinates
  {(-2,0) (-1,1) (0,0) (1,1) (2,0)};

\addplot[
  red,mark color=red!50!white,
  mark=halfsquare*]
  coordinates
  {(-2,-0.1) (-1,0.9) (0,-0.1) (1,0.9) (2,-0.1)};
\end{axis}
\end{tikzpicture}
```

Note that this key requires PGF 2.10 or later.

`/tikz/mark options={⟨options⟩}`

Resets `every mark` to `{⟨options⟩}`.

`/pgf/text mark={⟨text⟩}`

(initially p)

Changes the text shown by `mark=text`.

With `/pgf/text mark=m`: 

With `/pgf/text mark=A`: 

There is no limitation about the number of characters or whatever. In fact, any T_EX material can be inserted as `⟨text⟩`, including images.

`/pgf/text mark style={⟨options for mark=text⟩}`

Defines a set of options which control the appearance of `mark=text`.

If `/pgf/text mark as node=false` (the default), `⟨options⟩` is provided as argument to `\pgftext` – which provides only some basic keys like `left`, `right`, `top`, `bottom`, `base` and `rotate`.

If `/pgf/text mark as node=true`, `⟨options⟩` is provided as argument to `\node`. This means you can provide a very powerful set of options including `anchor`, `scale`, `fill`, `draw`, `rounded corners` etc.

`/pgf/text mark as node=true|false`

(initially false)

Configures how `mark=text` will be drawn: either as `\node` or as `\pgftext`.

The first choice is highly flexible and possibly slow, the second is very fast and usually enough.

`\pgfdeclareplotmark{⟨plot mark name⟩}{⟨code⟩}`

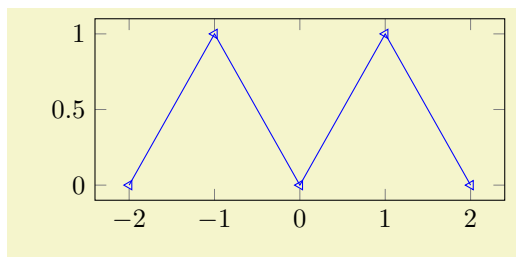
Defines a new marker named `⟨plot mark name⟩`. Whenever it is used, `⟨code⟩` will be invoked. It is supposed to contain (preferable PGF basic level) drawing commands. During `⟨code⟩`, the coordinate system's origin denotes the coordinate where the marker shall be placed.

Please refer to [5] section “Mark Plot Handler” for more detailed information.

`/pgfplots/every axis plot post`

(style, initially)

The `every axis plot post` style can be used to overwrite parts (or all) of the drawing styles which are assigned for plots.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% Overwrite any cycle list:
\pgfplotsset{
  every axis plot post/.append style={
    mark=triangle,
    every mark/.append style={rotate=90}}
\begin{tikzpicture}
\begin{axis}[y=2cm]
\addplot coordinates
  {(-2,0) (-1,1) (0,0) (1,1) (2,0)};
\end{axis}
\end{tikzpicture}
```

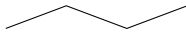
Markers paths are not subjected to clipping as other parts of the figure. Markers are either drawn completely or not at all.

TikZ offers more options for marker fine tuning, please refer to [5] for details.

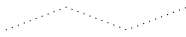
4.6.2 Line Styles

The following line styles are predefined in TikZ.

`/tikz/solid` (style, no value)



`/tikz/dotted` (style, no value)



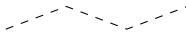
`/tikz/densely dotted` (style, no value)



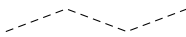
`/tikz/loosely dotted` (style, no value)



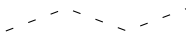
`/tikz/dashed` (style, no value)



`/tikz/densely dashed` (style, no value)



`/tikz/loosely dashed` (style, no value)



`/tikz/dashdotted` (style, no value)



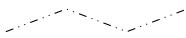
`/tikz/densely dashdotted` (style, no value)



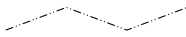
`/tikz/loosely dashdotted` (style, no value)



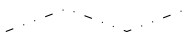
`/tikz/dashdotdotted` (style, no value)



`/tikz/densely dashdotdotted` (style, no value)



`/tikz/loosely dashdotdotted` (style, no value)



since these styles apply to markers as well, you may want to consider using

```
\pgfplotsset{
  every mark/.append style={solid}
}
```

in marker styles.

Besides linestyles, PGF also offers (a lot of) arrow heads. Please refer to [5] for details.

4.6.3 Edges and Their Parameters

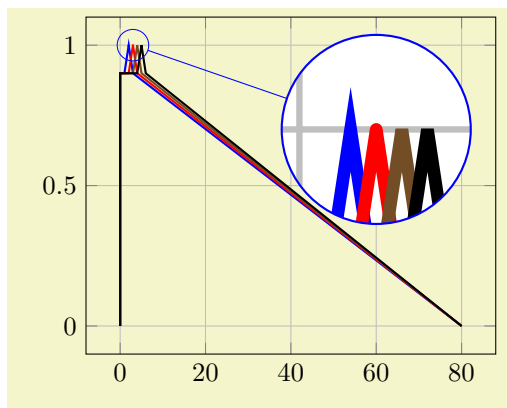
When PGFLOTS connects points, it relies on PGF drawing parameters to create proper edges (and it only changes them in the `every patch` style).

It might occasionally be necessary to change these parameters:

```
/tikz/line cap=round|rect|butt (initially butt)
/tikz/line join=round|bevel|miter (initially miter)
/tikz/miter limit={factor} (initially 10)
```

These keys control how lines are joined at edges. Their description is beyond the scope of this manual, so interested readers should consult [5].

Here is just an example illustrating why it might be of interest to study these parameters:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usetikzlibrary{spy}
\begin{tikzpicture}[spy using outlines=
  {circle, magnification=6, connect spies}]
\begin{axis}[no markers,grid=major,
  every axis plot post/.append style={thick}]
\addplot coordinates
  {(0, 0.0) (0, 0.9) (1, 0.9) (2, 1) (3, 0.9) (80, 0)};
\addplot+[line join=round] coordinates
  {(0, 0.0) (0, 0.9) (2, 0.9) (3, 1) (4, 0.9) (80, 0)};
\addplot+[line join=bevel] coordinates
  {(0, 0.0) (0, 0.9) (3, 0.9) (4, 1) (5, 0.9) (80, 0)};
\addplot+[miter limit=5] coordinates
  {(0, 0.0) (0, 0.9) (4, 0.9) (5, 1) (6, 0.9) (80, 0)};

\coordinate (spypoint) at (axis cs:3,1);
\coordinate (magnifyglass) at (axis cs:60,0.7);
\end{axis}

\spy [blue, size=2.5cm] on (spypoint)
  in node[fill=white] at (magnifyglass);
\end{tikzpicture}
```

4.6.4 Font Size and Line Width

Often, one wants to change line width and font sizes for plots. This can be done using the following options of TikZ.

```
/tikz/font={font name} (initially \normalfont)
```

Sets the font which is to be used for text in nodes (like tick labels, legends or descriptions).

A font can be any L^AT_EX argument like `\footnotesize` or `\small\bfseries`³¹.

It may be useful to change fonts only for specific axis descriptions, for example using

```
\pgfplotsset{
  tick label style={font=\small},
  label style={font=\small},
  legend style={font=\footnotesize}
}
```

See also the predefined styles `normalsize`, `small` and `footnotesize` in Section 4.9.2.

```
/tikz/line width={dimension} (initially 0.4pt)
```

Sets the line width. Please note that line widths for tick lines and grid lines are predefined, so it may be necessary to override the styles `every tick` and `every axis grid`.

The `line width` key is changed quite often in TikZ. You should use

```
\pgfplotsset{every axis/.append style={line width=1pt}}
```

or

³¹ConT_EXt and plain T_EX users need to provide other statements, of course.

```
\pgfplotsset{every axis/.append style={thick}}
```

to change the overall line width. To also adjust ticks and grid lines, one can use

```
\pgfplotsset{every axis/.append style={
  line width=1pt,
  tick style={line width=0.6pt}}}
```

or styles like

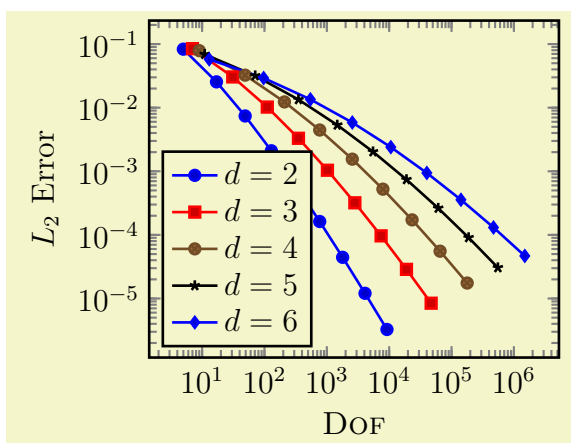
```
\pgfplotsset{every axis/.append style={
  thick,
  tick style={semithick}}}
```

The ‘every axis plot’ style can be used to change line widths for plots only.

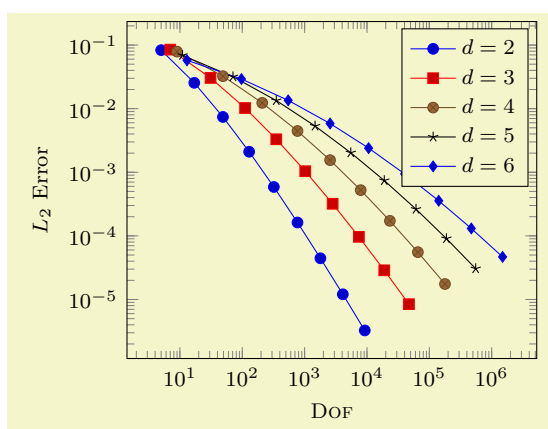
/tikz/ thin	(no value)
/tikz/ ultra thin	(no value)
/tikz/ very thin	(no value)
/tikz/ semithick	(no value)
/tikz/ thick	(no value)
/tikz/ very thick	(no value)
/tikz/ ultra thick	(no value)

These TikZ styles provide different predefined line widths.

This example shows the same plots as on page 19 (using `\plotcoords` as place holder for the commands on page 19), with different line widths and font sizes.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={
  font=\large,
  line width=1pt,
  tick style={line width=0.8pt}}}
\begin{tikzpicture}
  \begin{loglogaxis}[
    legend style={at={(0.03,0.03)},
      anchor=south west},
    xlabel=\textsc{Dof},
    ylabel=$L_2$ Error
  ]
    % see above for this macro:
    \plotcoords
    \legend{$d=2$,$d=3$,$d=4$,$d=5$,$d=6$}
  \end{loglogaxis}
\end{tikzpicture}
```



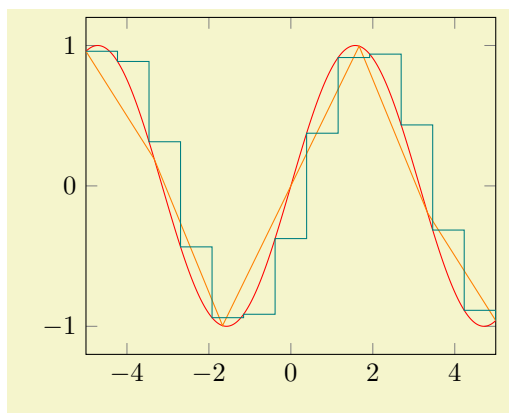
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={
  font=\footnotesize,
  thin,
  tick style={ultra thin}}}
\begin{tikzpicture}
  \begin{loglogaxis}[
    xlabel=\textsc{Dof},
    ylabel=$L_2$ Error
  ]
    % see above for this macro:
    \plotcoords
    \legend{$d=2$,$d=3$,$d=4$,$d=5$,$d=6$}
  \end{loglogaxis}
\end{tikzpicture}
```

4.6.5 Colors

PGF uses the color support of `xcolor`. Therefore, the main reference for how to specify colors is the `xcolor` manual [3]. The PGF manual [5] is the reference for how to select colors for specific purposes like drawing,

filling, shading, patterns etc. This section contains a short overview over the specification of colors in [3] (which is not limited to PGFPLOTS).

The package `xcolor` defines a set of predefined colors, namely ■ `red`, ■ `green`, ■ `blue`, ■ `cyan`, ■ `magenta`, ■ `yellow`, ■ `black`, ■ `gray`, ■ `white`, ■ `darkgray`, ■ `lightgray`, ■ `brown`, ■ `lime`, ■ `olive`, ■ `orange`, ■ `pink`, ■ `purple`, ■ `teal`, ■ `violet`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[enlarge x limits=false]
    \addplot[red,samples=500] {sin(deg(x))};

    \addplot[orange,samples=7] {sin(deg(x))};

    \addplot[teal,const plot,
      samples=14] {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

Besides predefined colors, it is possible to *mix* two (or more) colors. For example, ■ `red!30!white` contains 30% of ■ `red` and 70% of ■ `white`. Consequently, one can build ■ `red!70!white` to get 70% red and 30% white or ■ `red!10!white` for 10% red and 90% white. This mixing can be done with any color, for example ■ `red!50!green`, ■ `blue!50!yellow` or ■ `green!60!black`.

A different type of color mixing is supported, which allows to take 100% of *each* component. For example, ■ `rgb,2:red,1;green,1` will add 1/2 part ■ `red` and 1/2 part ■ `green` and we reproduced the example from above. Using the denominator 1 instead of 2 leads to ■ `rgb,1:red,1;green,1` which uses 1 part ■ `red` and 1 part ■ `green`. Many programs allow to select pieces between 0, ..., 255, so a denominator of 255 is useful. Consequently, ■ `rgb,255:red,231;green,84;blue,121` uses 231/255 red, 84/255 green and 121/255. This corresponds to the standard RGB color (231, 84, 121). Other examples are ■ `rgb,255:red,32;green,127;blue,43`, ■ `rgb,255:red,178;green,127;blue,43`, ■ `rgb,255:red,169;green,178;blue,43`.

It is also possible to use RGB values, the HSV color model, the CMY (or CMYK) models, or the HTML color syntax directly. However, this requires some more programming. I suppose this is the fastest (and probably the most uncomfortable) method to use colors. For example,

```
■ \definecolor{color1}{rgb}{1,1,0}
\tikz \fill[color1]
(0,0) rectangle (1em,0.6em);
```

creates the color with 100% ■ `red`, 100% ■ `green` and 0% ■ `blue`;

```
■ \definecolor{color1}{cmyk}{0.6,0.9,0.5,0.1}
\tikz \fill[color1]
(0,0) rectangle (1em,0.6em);
```

creates the color with 60% ■ `cyan`, 90% ■ `magenta`, 50% ■ `yellow` and 10% ■ `black`;

```
■ \definecolor{color1}{HTML}{D0B22B}
\tikz \fill[color1]
(0,0) rectangle (1em,0.6em);
```

creates the color with 208/255 pieces red, 178/255 pieces green and 43 pieces blue, specified in standard HTML notation. Please refer to the `xcolor` manual [3] for more details and color models.

The `xcolor` package provides even more methods to combine colors, among them the prefix ‘-’ (minus) which changes the color into its complementary color (■ `-black`, ■ `-white`, ■ `-red`) or color wheel calculations. Please refer to the `xcolor` manual [3].

```
/tikz/color={\langle a color \rangle}
/tikz/draw={\langle stroke color \rangle}
/tikz/fill={\langle fill color \rangle}
```

These keys are (generally) used to set colors. Use `color` to set the color for both drawing and filling. Instead of `color={\langle color name \rangle}` you can simply write `\langle color name \rangle`. The `draw` and `fill` keys only set colors for stroking and filling, respectively.

Use `draw=none` to disable drawing and `fill=none` to disable filling³².

Since these keys belong to TikZ, the complete documentation can be found in the TikZ manual [5, Section “Specifying a Color”].

4.6.6 Color Maps

`/pgfplots/colormap name={⟨color map name⟩}` (initially `hot`)

Changes the current color map to the already defined map named `⟨color map name⟩`. The predefined color map is



The definition can be found in the documentation for `colormap/hot`. This, and further color maps, are described below.

Colormaps can be used, for example, in scatter plots (see Section 4.4.10).

You can use `colormap` to create new color maps (see below).

`/pgfplots/colormap={⟨name⟩}{⟨color specification⟩}`

Defines a new colormap named `⟨name⟩` according to `⟨color specification⟩` and activates it using `colormap name={⟨name⟩}`.

The `⟨color specification⟩` is a sequence of positions and associated colors where linear interpolation is applied in-between. The syntax is very similar as the one used for PGF shadings described in [5, VIII – Shadings]: it is a semicolon-separated series of

`⟨color type⟩(⟨offset⟩)=(⟨color value⟩); :`

```
% possibility 1: like PGF shadings:
rgb(0cm)=(1,0,0); rgb(1cm)=(0,1,0); rgb255(2cm)=(0,0,255); gray(3cm)=(0.3); color(4cm)=(green)
```



If the distance between successive colors is the same, the `⟨offset⟩` can be omitted. The ‘;’ separators are not necessary either:

```
% (simplified) possibility 2: skip ‘;’ and length arguments:
rgb=(1,0,0) rgb=(0,1,0) rgb255=(0,0,255) gray=(0.3) color=(green)
```



It is also possible to provide non-uniform distances between the different colors – if all single positions can be projected onto a uniform grid. PGFPLOTS will perform this interpolation automatically:

```
% non uniform spacing example: the mesh width is provided as first
% part of the specification.
\pgfplotsset{colormap={violetnew}
  {[1cm] rgb255(0cm)=(25,25,122) color(1cm)=(white) rgb255(5cm)=(238,140,238)}}

```



In this last example, the mesh width has been provided explicitly and PGFPLOTS interpolates the missing grid points on its own. It is an error if the provided positions are not multiple of the mesh width. The `\pgfplotsset` employs the public user interface to create a new color map named ‘violetnew’.

The single colors can be separated by semicolons ‘;’. The (optional) length describes how much of the bar is occupied by the interval, it is interpreted relative to the complete length. If the length argument is missing, it is taken to be the last specified length plus the last length difference (the first color defaults to 1cm in this case).

³²Up to now, plot marks always have a stroke color (some also have a fill color). This restriction may be lifted in upcoming versions.

Colormap Input Format

Each entry in $\langle color\ specification \rangle$ has the form $\langle color\ model \rangle(\langle length \rangle)=(\langle arguments \rangle)$. Here, the $\langle length \rangle$ argument is optional as discussed above. The entries can be separated by semicolons ‘;’ or by white spaces. The leftmost entry *must* have $\langle length \rangle=0pt$. As discussed, all entries will be placed on a uniform grid, i.e. the distance between adjacent $\langle length \rangle$ arguments has to be the same (see the previous paragraph for automatic generation of intermediate points). The complete length of a color map is irrelevant: it will be mapped linearly to an internal range anyway (for efficient interpolation). The only requirement is that the left end must be at 0.

Available choices for $\langle color\ model \rangle$ are

rgb which expects $\langle arguments \rangle$ of the form $(\langle red \rangle, \langle green \rangle, \langle blue \rangle)$ where each component is in the interval $[0, 1]$,

rgb255 which is similar to **rgb** except that each component is expected in the interval $[0, 255]$,

gray in which case $\langle arguments \rangle$ is a single number in the interval $[0, 1]$,

color in which case $\langle arguments \rangle$ contains a predefined (named) color like ‘red’ or a color expression like ‘red!50’,

cmyk which expects $\langle arguments \rangle$ of the form $(\langle cyan \rangle, \langle magenta \rangle, \langle yellow \rangle, \langle black \rangle)$ where each component is in the interval $[0, 1]$,

cmyk255 which is the same as **cmyk** but expects components in the interval $[0, 255]$,

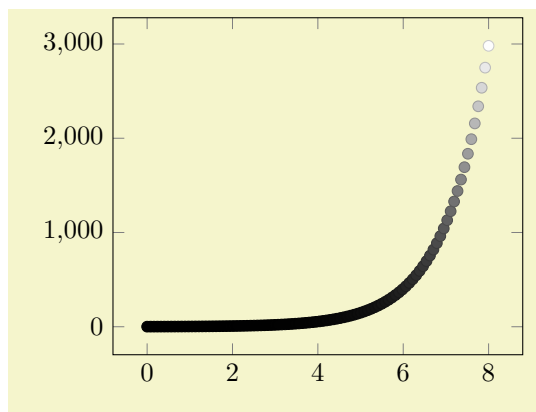
cmy which expects $\langle arguments \rangle$ of the form $(\langle cyan \rangle, \langle magenta \rangle, \langle yellow \rangle)$ where each component is in the interval $[0, 1]$,

hsb which expects $\langle arguments \rangle$ of the form $(\langle hue \rangle, \langle saturation \rangle, \langle brightness \rangle)$ where each component is in the interval $[0, 1]$,

Hsb which is the same as **hsb** except that $\langle hue \rangle$ is accepted in the interval $[0, 360]$ (degree),

HTML which is similar to **rgb255** except that each component is expected to be a hex number between 00 and FF,

wave which expects a single wave length as numeric argument in the range $[363, 814]$.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    colormap={bw}{gray(0cm)=(0); gray(1cm)=(1)}]
    \addplot+[scatter,only marks,
      domain=0:8,samples=100]
      {exp(x)};
  \end{axis}
\end{tikzpicture}
```

The Colorspace of a Colormap

Even though a **colormap** accepts lots of color spaces on *input* (in fact, it accepts most or all that **xcolor** provides), the *output* color of a colorspace has strict limitations. The output colorspace is the one in which PGFLOTS interpolates between two other colors. To this end, it transforms input colors to the output color space. The output colorspace is also referred to as “the colorspace of a colormap”.

There are three supported color spaces for a **colormap**: the GRAY, RGB, and CMYK color spaces. Each access into a **colormap** requires linear interpolation which is performed in its color space. Color spaces make a difference: colors in different color spaces may be represented differently, depending on the output device. Many printers use CMYK for color printing, so providing CMYK colors might improve the printing quality on a color printer. The RGB color space is often used for display devices. The predefined **colormaps** in PGFLOTS all use RGB.

Whenever a new `colormap` is created, PGFPLOTS determines an associated color space. Then, each color in this specific `colormap` will be represented in its associated color space (converting colors automatically if necessary). Furthermore, every access into the `colormap` will be performed in its associated color space and every returned mapped color will be represented with respect to this color space. Furthermore, every shading generated by `shader=interp` will be represented with respect to the `colormap`'s associated color space.

The color space is chosen as follows: in case `colormap default colorspace=auto` (the initial configuration), the color space depends on the *first* encountered color in `<color specification>`. For `rgb` or `gray` or `color`, the associated color space will be RGB (as it was in all earlier versions of PGFPLOTS). For `cmlyk`, the associated color space will be CMYK. If `colormap default colorspace` is either `gray`, `rgb` or `cmlyk`, this specific color space is used and every color is converted automatically.

`/pgfplots/colormap default colorspace=auto|gray|rgb|cmlyk` (initially auto)

Allows to set the color space of every *newly created* `colormap`. The choices are explained in the previous paragraph.

It is (not yet) possible to change the color space of an existing `colormap`; re-create it if conversion is required.

The macro `\pgfplotscolormapgetcolorspace{<name>}` defines `\pgfplotsretval` to contain the color space of an existing `colormap name`, if you are in doubt.

Predefined Colormaps

Available color maps are shown below.

`/pgfplots/colormap/hot` (style, no value)

A style which installs the colormap

```
\pgfplotsset{
  colormap={hot}{color(0cm)=(blue); color(1cm)=(yellow); color(2cm)=(orange); color(3cm)=(red)}
}
```



This is the preconfigured color map.

`/pgfplots/colormap/hot2` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={hot2}{[1cm]rgb255(0cm)=(0,0,0) rgb255(3cm)=(255,0,0)
    rgb255(6cm)=(255,255,0) rgb255(8cm)=(255,255,255)}
}
```



Note that this particular choice ships directly with PGFPLOTS, you do not need to load the `colormaps` library for this value.

This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/jet` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={jet}{rgb255(0cm)=(0,0,128) rgb255(1cm)=(0,0,255)
    rgb255(3cm)=(0,255,255) rgb255(5cm)=(255,255,0) rgb255(7cm)=(255,0,0) rgb255(8cm)=(128,0,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/blackwhite**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={blackwhite}{gray(0cm)=(0); gray(1cm)=(1)}
}
```

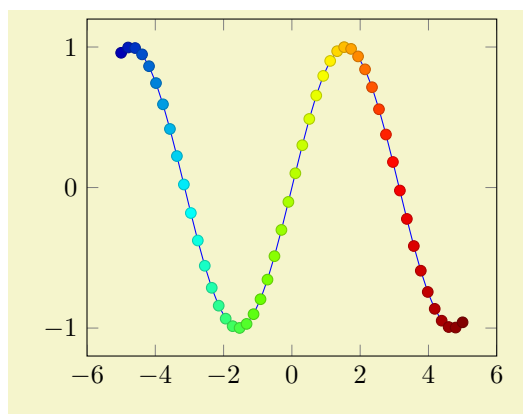


/pgfplots/**colormap/bluered**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={bluered}{
    rgb255(0cm)=(0,0,180); rgb255(1cm)=(0,255,255); rgb255(2cm)=(100,255,0);
    rgb255(3cm)=(255,255,0); rgb255(4cm)=(255,0,0); rgb255(5cm)=(128,0,0)}
}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colormap/bluered]
    \addplot+[scatter,
      scatter src=x,samples=50]
      {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

Remark: The style **bluered** (re-)defines the color map and activates it. T_EX will be slightly faster if you call `\pgfplotsset{colormap/bluered}` in the preamble (to create the color map once) and use `colormap name=bluered` whenever you need it. This remark holds for every color map style which follows. But you can simply ignore this remark.

/pgfplots/**colormap/cool**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={cool}{rgb255(0cm)=(255,255,255); rgb255(1cm)=(0,128,255); rgb255(2cm)=(255,0,255)}
}
```



/pgfplots/**colormap/greenyellow**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={greenyellow}{rgb255(0cm)=(0,128,0); rgb255(1cm)=(255,255,0)}
}
```



/pgfplots/**colormap/redyellow**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={redyellow}{rgb255(0cm)=(255,0,0); rgb255(1cm)=(255,255,0)}
}
```



/pgfplots/**colormap/violet**

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  colormap={violet}{rgb255=(25,25,122) color=(white) rgb255=(238,140,238)}
}
```



\pgfplotscolormaptoashadingspec{*colormap name*}{*right end size*}{*\macro*}

A command which converts a colormap into a PGF shading's color specification. It can be used in commands like `\pgfdeclare*shading` (see the PGF manual [5] for details).

The first argument is the name of a (defined) colormap, the second the rightmost dimension of the specification. The result will be stored in *\macro*.



```
% convert 'hot' -> \result
\pgfplotscolormaptoashadingspec{hot}{8cm}\result
% define and use a shading in pgf:
\def\tempb{\pgfdeclarehorizontalshading{tempshading}{1cm}}%
% where '\result' is inserted as last argument:
\expandafter\tempb\expandafter{\result}%
\pgfusesshading{tempshading}%
```

The usage of the result *\macro* is a little bit low-level.

Attention: PGF shadings are always represented with respect to the RGB color space. Consequently, even CMYK *colormap name*s will result in an RGB shading specification when using this method³³.

Note that there *more available choices* in the `colormaps` library which needs to be loaded by means of `\usepgfplotslibrary{colormaps}`.

4.6.7 Cycle Lists – Options Controlling Line Styles

/pgfplots/**cycle list**={*list*}

/pgfplots/**cycle list name**={*\macro*}

Allows to specify a list of plot specifications which will be used for each `\addplot` command without explicit plot specification. Thus, the currently active `cycle list` will be used if you write either `\addplot+[{keys}] ...`; or if you *don't* use square brackets as in `\addplot[explicit plot specification]` ...;.

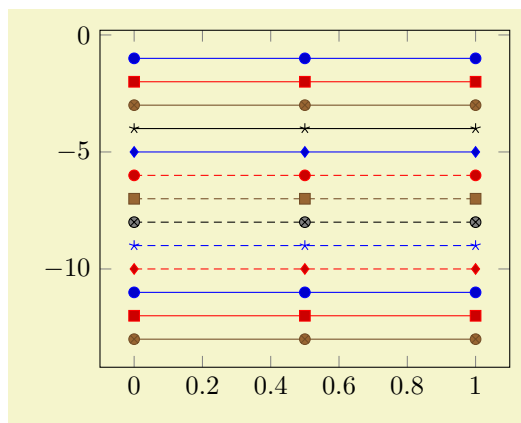
³³In case PGF should someday support CMYK shadings and you still see this remark, you can add the macro definition `\def\pgfplotscolormaptoashadingspectorgb{0}` to your preamble.

The list element with index i will be chosen where i is the index of the current `\addplot` command (see also the `cycle list shift` key which allows to use $i+n$ instead). This indexing does also include plot commands which don't use the `cycle list`.

There are several possibilities to change the currently active `cycle list`:

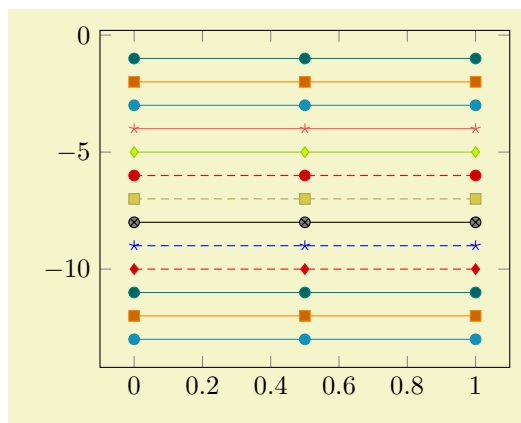
1. Use one of the predefined lists³⁴,

- `color` (from top to bottom)



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    stack plots=y,stack dir=minus,
    cycle list name=color]
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\end{axis}
\end{tikzpicture}
```

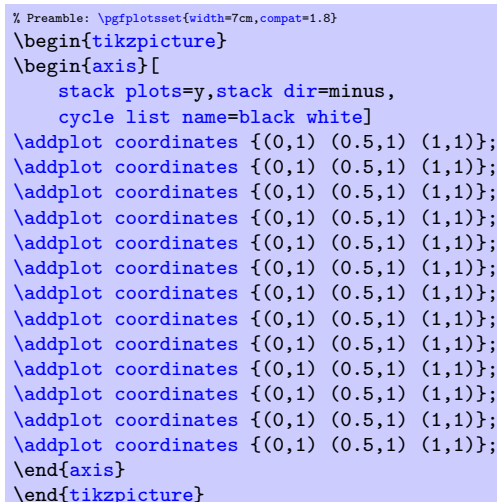
- `exotic` (from top to bottom)



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    stack plots=y,stack dir=minus,
    cycle list name=exotic]
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\addplot coordinates {(0,1) (0.5,1) (1,1)};
\end{axis}
\end{tikzpicture}
```

- `black white` (from top to bottom)

³⁴In an early version, these lists were called `\coloredplotspeclist` and `\blackwhiteplotspeclist` which appeared to be unnecessarily long, so they have been renamed. The old names are still accepted, however.



- [illegible]

[illegible]

- **auto** The `cycle list name=auto` always denotes the most recently used cycle list activated by `cycle list` or `cycle list name`.

The definitions of all predefined cycle lists follow (see the end of this paragraph for a syntax description).

```
\pgfplotscreateplotcyclelist{color}{%
  blue, every mark/.append style={fill=blue!80!black}, mark=*\\%
  red, every mark/.append style={fill=red!80!black}, mark=square*\\%
  brown!60!black, every mark/.append style={fill=brown!80!black}, mark=otimes*\\%
  black, mark=star\\%
  blue, every mark/.append style={fill=blue!80!black}, mark=diamond*\\%
  red, densely dashed, every mark/.append style={solid, fill=red!80!black}, mark=*\\%
  brown!60!black, densely dashed, every mark/.append style={
    solid, fill=brown!80!black}, mark=square*\\%
  black, densely dashed, every mark/.append style={solid, fill=gray}, mark=otimes*\\%
  blue, densely dashed, mark=star, every mark/.append style=solid\\%
  red, densely dashed, every mark/.append style={solid, fill=red!80!black}, mark=diamond*\\%
}
```

```
\pgfplotscreateplotcyclelist{black white}{%
  every mark/.append style={fill=gray}, mark=*\\%
  every mark/.append style={fill=gray}, mark=square*\\%
  every mark/.append style={fill=gray}, mark=otimes*\\%
  mark=star\\%
  every mark/.append style={fill=gray}, mark=diamond*\\%
  densely dashed, every mark/.append style={solid, fill=gray}, mark=*\\%
  densely dashed, every mark/.append style={solid, fill=gray}, mark=square*\\%
  densely dashed, every mark/.append style={solid, fill=gray}, mark=otimes*\\%
  densely dashed, every mark/.append style={solid}, mark=star\\%
  densely dashed, every mark/.append style={solid, fill=gray}, mark=diamond*\\%
}
```

```
\pgfplotscreateplotcyclelist{exotic}{%
  teal, every mark/.append style={fill=teal!80!black}, mark=*\\%
  orange, every mark/.append style={fill=orange!80!black}, mark=square*\\%
  cyan!60!black, every mark/.append style={fill=cyan!80!black}, mark=otimes*\\%
  red!70!white, mark=star\\%
  lime!80!black, every mark/.append style={fill=lime}, mark=diamond*\\%
  red, densely dashed, every mark/.append style={solid, fill=red!80!black}, mark=*\\%
  yellow!60!black, densely dashed,
    every mark/.append style={solid, fill=yellow!80!black}, mark=square*\\%
  black, every mark/.append style={solid, fill=gray}, mark=otimes*\\%
  blue, densely dashed, mark=star, every mark/.append style=solid\\%
  red, densely dashed, every mark/.append style={solid, fill=red!80!black}, mark=diamond*\\%
}
```

```
% note that "." is the currently defined Tikz color.
\pgfplotscreateplotcyclelist{mark list}{%
  every mark/.append style={solid, fill=!.!80!black}, mark=*\\%
  every mark/.append style={solid, fill=!.!80!black}, mark=square*\\%
  every mark/.append style={solid, fill=!.!80!black}, mark=triangle*\\%
  every mark/.append style={solid}, mark=star\\%
  every mark/.append style={solid, fill=!.!80!black}, mark=diamond*\\%
  every mark/.append style={solid, fill=!.!80!black!40}, mark=otimes*\\%
  every mark/.append style={solid}, mark=\\%
  every mark/.append style={solid, fill=!.!80!black}, mark=pentagon*\\%
  every mark/.append style={solid}, mark=text, text mark=p\\%
  every mark/.append style={solid}, mark=text, text mark=a\\%
}
```

This is not the complete truth: the actual implementation of `mark list` allows to customize the `fill` value:

`/pgfplots/mark list fill={color}` (initially `!.!80!black`)

Allows to customize the fill color for the `mark list` and `mark list*`.

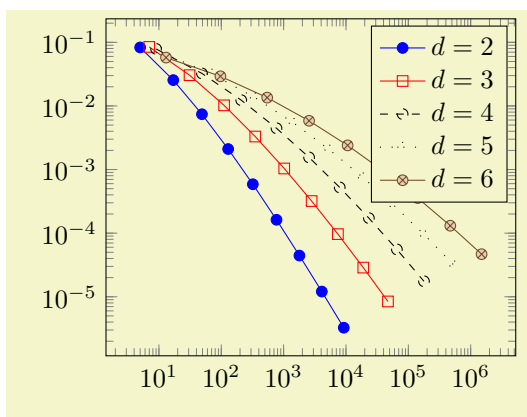
For example, if you have `black` as color, the alternative choice `mark list fill=!.!50!white` will produce much better results.

```
% note that "." is the currently defined Tikz color.
\pgfplotscreateplotcyclelist{mark list*}{%
  every mark/.append style={solid,fill=!.80!black},mark=*\\%
  every mark/.append style={solid,fill=!.80!black},mark=square*\\%
  every mark/.append style={solid,fill=!.80!black},mark=triangle*\\%
  every mark/.append style={solid,fill=!.80!black},mark=halfsquare*\\%
  every mark/.append style={solid,fill=!.80!black},mark=pentagon*\\%
  every mark/.append style={solid,fill=!.80!black},mark=halfcircle*\\%
  every mark/.append style={solid,fill=!.80!black,rotate=180},mark=halfdiamond*\\%
  every mark/.append style={solid,fill=!.80!black!40},mark=otimes*\\%
  every mark/.append style={solid,fill=!.80!black},mark=diamond*\\%
  every mark/.append style={solid,fill=!.80!black},mark=halfsquare right*\\%
  every mark/.append style={solid,fill=!.80!black},mark=halfsquare left*\\%
}
```

```
\pgfplotscreateplotcyclelist{color list}{%
  red,blue,black,yellow,brown,teal,orange,violet,cyan,green!70!black,magenta,gray}
```

```
\pgfplotscreateplotcyclelist{linestyles}{solid,dashed,dotted}
\pgfplotscreateplotcyclelist{linestyles*}{solid,dashed,dotted,dashdotted,dashdotdotted}
```

2. The second choice for cycle lists is to provide each entry directly as argument to `cycle list`,



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[cycle list={%
  {blue,mark=*},
  {red,mark=square},
  {dashed,mark=o},
  {loosely dotted,mark=+},
  {brown!60!black,
    mark options={fill=brown!40},
    mark=otimes*}}
]
\plotcoords
\legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
\end{loglogaxis}
\end{tikzpicture}
```

(This example list requires `\usetikzlibrary{plotmarks}`).

The input format is described below in more detail.

3. The last method is to combine 1. and 2.: Define named cycle lists in the preamble and use them with ‘`cycle list name`’:

```
\pgfplotscreateplotcyclelist{<name>}{<list>}
```

```
\pgfplotscreateplotcyclelist{mylist}{%
  {blue,mark=*},
  {red,mark=square},
  {dashed,mark=o},
  {loosely dotted,mark=+},
  {brown!60!black,mark options={fill=brown!40},mark=otimes*}}
...
\begin{axis}[cycle list name=mylist]
...
\end{axis}
```

The format of `<list>`: The argument `<list>` is usually a comma separated list of lists of style keys like colors, line styles, marker types and marker styles. This “comma list of comma lists” structure requires to encapsulate the inner list using curly braces:

```
\pgfplotscreateplotcyclelist{mylist}{%
  {blue,mark=*},
  {red,mark=square},
  {dashed,mark=o},
  {loosely dotted,mark=+},
  {brown!60!black,mark options={fill=brown!40},mark=otimes*}}
```


Alternatively, one can terminate the inner lists (i.e. those for one single plot) with ‘\\’:

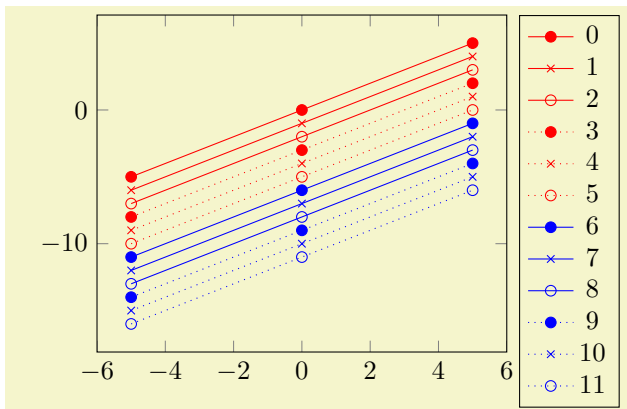
```
\begin{axis}[cycle list={%
  blue,mark=*\\%
  red,mark=square\\%
  dashed,mark=o\\%
  loosely dotted,mark=+\\%
  brown!60!black,mark options={fill=brown!40},mark=otimes*\\%
}]
...
\end{axis}
```

In this case, the *last* entry also needs a terminating ‘\\’, but one can omit braces around the single entries.

Remark: It is possible to call `\pgfplotsset{cycle list={⟨a list⟩}}` or `cycle list name` *between* plots. Such a setting remains effective until the end of the current T_EX group (that means curly braces). Every `\addplot` command queries the `cycle list` using the plot index; it doesn’t hurt if `cycle lists` have changed in the meantime.

`/pgfplots/cycle multi list=⟨list 1⟩\nextlist⟨list 2⟩\nextlist...`

Allows to supply more than one `cycle list` in a way such that each one contributes to the plot style. This is probably best explained using an example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  cycle multi list={
    red,blue\nextlist
    solid,{dotted,mark options={solid}}\nextlist
    mark=*,mark=x,mark=o
  },
  samples=3,
  legend entries={0,...,20},
  legend pos=outer north east
]
  \addplot {x};
  \addplot {x-1};
  \addplot {x-2};
  \addplot {x-3};
  \addplot {x-4};
  \addplot {x-5};
  \addplot {x-6};
  \addplot {x-7};
  \addplot {x-8};
  \addplot {x-9};
  \addplot {x-10};
  \addplot {x-11};
\end{axis}
\end{tikzpicture}
```

The provided `cycle multi list` consists of three lists. The style for a single plot is made up using elements of each of the three lists: the first plot has style `red,solid,mark=*`, the second has

`red,solid,mark=x`, the third has `red,solid,mark=o`. The fourth plot restarts the third list and uses the next one of list 2: it has `red,dotted,mark options={solid},mark=*` and so on.

The last list will always be advanced for a new plot. The list before the last (in our case the second list) will be advanced after the last one has been reset. In other words: `cycle multi list` allows a composition of different `cycle list` in a lexicographical way³⁵.

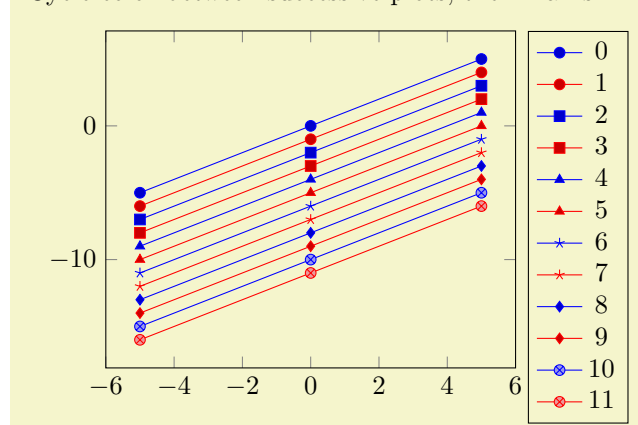
The argument for `cycle multi list` is a sequence of arguments as they would have been provided for `cycle list`, separated by `\nextlist`. In addition to providing a new cycle list, the $\langle list\ i \rangle$ elements can also denote `cycle list name` values (including the special `auto` cycle list which is the most recently assigned `cycle list` or `cycle list name`). The final `\nextlist` is optional.

The list in our example above could have been written as

```
\begin{axis}[
  cycle multi list={
    red\\blue\\nextlist
    solid\\dotted,mark options={solid}\\nextlist
    mark=*\mark=x\mark=o\\
  }
]
```

as well (note the terminating `\\` commands!).

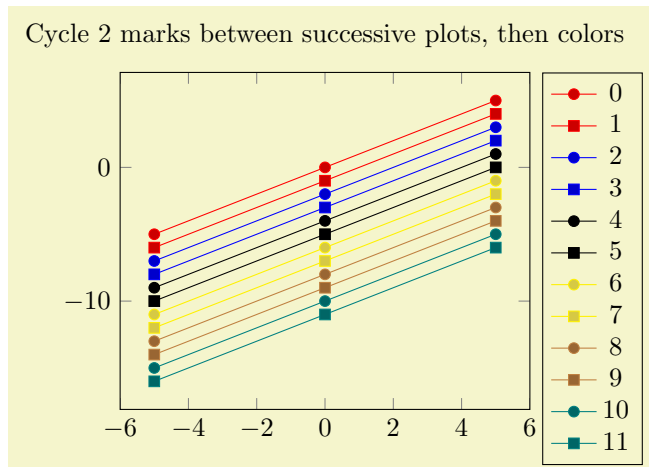
Cycle color between successive plots, then marks



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title={Cycle color between successive plots, then marks},
  cycle multi list={
    mark list\nextlist
    blue,red%
  },
  samples=3,
  legend entries={0,...,20},
  legend pos=outer north east
]
  \addplot {x};
  \addplot {x-1};
  \addplot {x-2};
  \addplot {x-3};
  \addplot {x-4};
  \addplot {x-5};
  \addplot {x-6};
  \addplot {x-7};
  \addplot {x-8};
  \addplot {x-9};
  \addplot {x-10};
  \addplot {x-11};
\end{axis}
\end{tikzpicture}
```

³⁵For those who prefer formulas: The plot with index $0 \leq i < N$ will use cycle list offsets $i_0, i_1, \dots, i_k$, $0 \leq i_m < N_m$ where k is the number of arguments provided to `cycle multi list` and N_m is the number of elements in the m th cycle list. The offsets i_m are computed in a loop `{ int tmp=i; for( int m=k-1; m>=0; m=m-1 ) { i_m = tmp%N_m; tmp = tmp/N_m; }}`.

Using Sub-Lists The $\langle list\ i \rangle$ entry can also contain just the first n elements of an already known cycle list name using the syntax $[\langle number \rangle\ of]\langle cycle\ list\ name \rangle$. For example `[2 of]mark list` will use the first 2 elements of `mark list`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title={Cycle 2 marks between successive plots, then colors},
  cycle multi list={%
    color list\nextlist
    [2 of]mark list
  },
  samples=3,
  legend entries={0,...,20},
  legend pos=outer north east
]
\addplot {x};
\addplot {x-1};
\addplot {x-2};
\addplot {x-3};
\addplot {x-4};
\addplot {x-5};
\addplot {x-6};
\addplot {x-7};
\addplot {x-8};
\addplot {x-9};
\addplot {x-10};
\addplot {x-11};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/cycle list shift={ $\langle integer \rangle$ }` (initially empty)

Allows to *shift* the index into the `cycle list`. If $\langle integer \rangle$ is n , the list element $i + n$ will be taken instead of the i th one. Remember that i is the index of the current `\addplot` command (starting with 0). Since a `cycle list` is queried *immediately* when `\addplot` (or `\addplot+`) is called, you can adjust the `cycle list shift` for selected plots:

```
\pgfplotsset{cycle list shift=3}
\addplot ....

\pgfplotsset{cycle list shift=-1}
\addplot ....
```

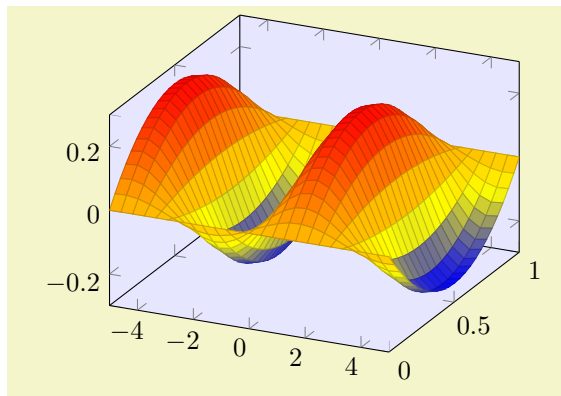
Special case: If the result is negative, $i + n < 0$, the list index $-(i + n)$ will be taken. For example, `cycle list shift=-10` and $i < 10$ will result in list index $10 - i$. Note that you can use `reverse legend` to reverse legends, so this feature is probably never needed.

4.6.8 Axis Background

/pgfplots/**axis background**

(initially empty)

This is a style to configure the appearance of the axis as such. It can be defined and/or changed using the **axis background/.style={⟨options⟩}** method. A background path will be generated with ⟨options⟩, which may contain fill colors or shadings.

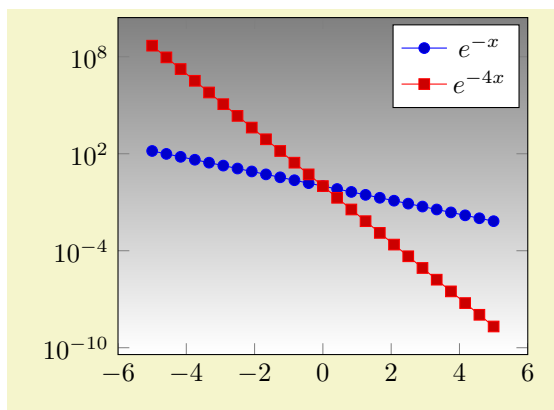


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
axis background/.style={fill=blue!10}]

\addplot3[surf,y domain=0:1]
{sin(deg(x)) * y*(1-y)};

\end{axis}
\end{tikzpicture}
```

Please note that legends are filled with white in the default configuration.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
axis background/.style={
shade,top color=gray,bottom color=white},
legend style={fill=white}]

\addplot {exp(-x)};
\addplot {exp(-4*x)};
\legend{$e^{-x}$,$e^{-4x}$}

\end{semilogyaxis}
\end{tikzpicture}
```

Details about **fill** and **shade** can be found in the TikZ manual, [5].

4.7 Providing Color Data - Point Meta

PGFplots provides features which modify plots depending on a special coordinate, the “point meta data”. For example, scatter plots may vary marker colors, size or appearance depending on this special data. Surface and mesh plots are another example: here, the color of a surface patch (or mesh part) depends on “point meta”. The “meta data” of a node is not the position (which is given as (x, y) or (x, y, z)). It is “some information” about that node; you could say: it is a specific property of the node. This is commonly called “meta data”.

In PGFplots, every node has its coordinate *and* its meta data. Thus, two-dimensional plots have three values: x , y , and the **point meta** data. Three-dimensional plots have four values for each coordinate: x , y , z , and the **point meta**.

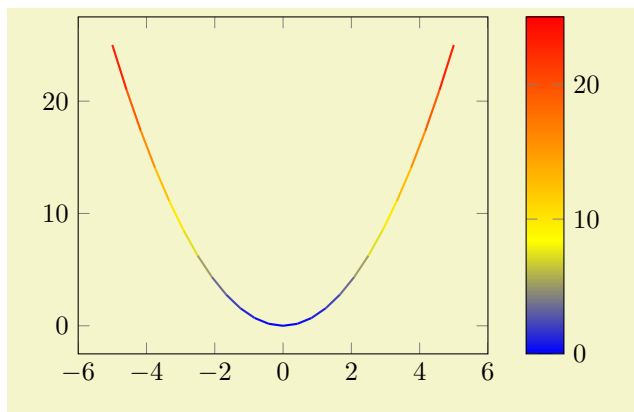
In many cases, **point meta** is interpreted to be color data. To be more precise: it is interpreted to be *scalar* color data which is mapped into the **colormap** (more about this approach in the next paragraphs). However, **point meta** can be anything. Often, it is a single number as in the case of color data. That number is mapped linearly to the interval $[0, 1000]$ such that 0 corresponds to the smallest encountered value and 1000 corresponds to the largest encountered scalar value. The mapped value is available as **\pgfplotspointmetatransformed**. This special value allows to define some property of the plot: it can be the color (together with **colormap**). It can also be the **line width** in a **mesh** plot (more precisely: the **line width** could be defined to depend on the transformed meta data). The value can also be used to define **mark size**. However, **point meta** data does not necessarily need to be a number. It can be a text label

(any text that you like). This is used by `nodes near coords`, for example. It could also contain a tuple like RGB color information (which is not commonly used, however). Thus, `point meta` is really some abstract information about individual coordinates.

Note that there is only *one* `point meta` per point. See the key `visualization depends on` if you need more than one meta data value per coordinate.

The common idea is to tell PGFPLOTS how to get the meta data. It is not necessary to provide data explicitly – in many cases, the data which is used to color surface patches or marker colors is the plot’s y or z coordinate. The method used to tell PGFPLOTS where to find “point meta data” is the `point meta` key.

The most common use-case of `point meta` is color information: if the point meta data is in the interval $[m_{\min}, m_{\max}]$, the point meta coordinate $m = m_{\min}$ will get the lowest color provided by the color map while $m = m_{\max}$ will get the highest color provided by the color map. As already mentioned, this is accomplished using `\pgfplotspointmetatransformed` $\in [0, 1000]$ (per convention). Any coordinate between the smallest and largest values will be mapped linearly: for example, the mean $m = 1/2(m_{\max} + m_{\min})$ will get the middle color of the color map (it will have `\pgfplotspointmetatransformed` = 500). This is why “point meta” is sometimes called “color data” in this manual.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[colorbar]
\addplot[mesh,point meta=y,thick] {x^2};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/point meta=none|<expression>|x|y|z|f(x)|explicit|explicit symbolic` (initially none)

The `point meta` key tells PGFPLOTS where to get the special point meta data. Please note that `point meta` and `scatter src` is actually the same – `scatter src` is an alias for `point meta`. Thus, the summary provided for `scatter src` on page 79 covers the same topics. However, the main reference for `point meta` is here.

none The initial choice `none` disables point meta data, resulting in no computational work. Any other choice will activate the computation of upper and lower ranges for point meta data, i.e. the computation of $[m_{\min}, m_{\max}]$.

x The choice `x` uses the already available x coordinates as point meta data. This does always refer to the *final* x coordinates after any user transformations, logarithms, stacked plot computations etc. have been applied. Consider using `rawx` if you need the unprocessed coordinate value here.

y

z The choices `y` and `z` are similar: they use the y or z coordinates respectively as point meta data. Consequently, these three choices do *not* need any extra data. As for `x`, there are math constants `rawy` and `rawz` which yield the unprocessed y and z value, respectively.

f(x) This will use the last available coordinate, in other words: it is the same as `y` for two dimensional plots and `z` for three dimensional ones.

explicit This choice tells PGFPLOTS to expect *numerical* point meta data which is provided explicitly in the coordinate input streams. This data will be transformed linearly into the current color map as it has been motivated above.

How point meta data is provided for `plot coordinates`, `plot table` and the other input methods is described in all detail in Section 4.2.1 – but we provide small examples here to summarize the possibilities:

```
% for 'coordinates':
% provide color data explicitly using [<data>]
% behind coordinates:
\addplot+[point meta=explicit]
  coordinates {
    (0,0) [1.0e10]
    (1,2) [1.1e10]
    (2,3) [1.2e10]
    (3,4) [1.3e10]
    % ...
  };
```

```
% for 'table':
% Assumes a datafile.dat like
% xcolname ycolname colordata
% 0        0        0.001
% 1        2        0.3
% 2        2.1      0.4
% 3        3        0.5
% ...
% the file may have more columns.
\addplot+[point meta=explicit]
  table[x=xcolname,y=ycolname,meta=colordata]
    {datafile.dat};
% or, equivalently (perhaps a little bit slower):
\addplot+[point meta=\thisrow{colordata}]
  table[x=xcolname,y=ycolname]
    {datafile.dat};
```

```
% for 'file':
% Assumes a datafile.dat like
% 0        0        0.001
% 1        2        0.3
% 2        2.1      0.4
% 3        3        0.5
% ...
% the first three columns will be used here as x,y and meta,
% resp.
\addplot+[point meta=explicit]
  file {datafile.dat};
```

```
% 'table' using expressions which may depend on all
% columns:
% Assumes a datafile.dat like
% xcolname ycolname anything othercol
% 0        0        4        15
% 1        2        5        20
% 2        2.1      8        30
% 3        3        42       40
% ...
% the file may have more columns.
\addplot+[point meta={0.5*(\thisrow{anything} + sqrt(\thisrow{othercol}))}]
  table[x=xcolname,y=ycolname]
    {datafile.dat};
```

Thus, there are several methods to provide point meta (color data). The key for the choice `explicit` is that some data is provided explicitly – although `point meta` doesn't know how. The data is expected to be of numerical type and is mapped linearly into the range $[0, 1000]$ (maybe for use in the current color map).

explicit symbolic The choice `explicit symbolic` is very similar to `explicit` in that it expects extra data by the coordinate input routines. However, `explicit symbolic` does not necessarily expect numerical data: you can provide any sort of symbols. One might provide a set of styles, one for each class in a scatter plot. This is realised using `scatter/classes`, see page 81. Input data is provided in the same fashion as mentioned above for the choice `explicit`.

Currently, this choice can only be used for scatter plots.

<expression> This choice allows to compute point meta data using a mathematical expression. The *<expression>* may depend on **x**, **y**, **z** which yield the current *x*, *y* or *z* coordinate, respectively. The coordinates are completely processed (transformations, logs) as mentioned above for the choice **x**. Furthermore, the *<expression>* may depend on commands which are valid during `\addplot` like `\plotnum` or `\coordindex` (see Section 4.25 for details). Computations are performed using the floating point unit of PGF, and all supported arithmetical operations can be used.

In essence, the *<expression>* may depend on everything which is known to all `\addplot` commands: the *x*, *y* and (if any) *z* coordinates. In addition, it may depend upon **rawx**, **rawy** or **rawz**. These three expressions yield the unprocessed *x*, *y* or *z* value as it has been found in the input stream (no logs, no user transformations)³⁶. If used together with `plot table`, you may also access other table columns (for example with `\thisrow{<colname>}`).

TeX code=*<code>* A rather low level choice which allows to provide TeX *<code>* to compute a numerical value. The *<code>* should define the macro `\pgfplotspointmeta`. It is evaluated in a locally scoped environment (it's local variables are freed afterwards). It may depend on the same values as described for *<expression>* above, especially on `\thisrow{<colname>}` for table input.

Note that the math parser will be configured to use the **fpu** at this time, so `\pgfmathparse` yields floats.

Note that you need an extra pair of braces to provide this key, i.e.

`point meta={TeX code={<code>}}.`

TeX code symbolic=*<code>* Just as **TeX code**, you can provide *<code>* which defines the macro `\pgfplotspointmeta`, but the result is not interpreted as a number. It is like the **explicit symbolic** choice.

Note that you need an extra pair of braces to provide this key, i.e.

`point meta={TeX code symbolic={<code>}}.`

symbolic=*<symbol>* A choice which accepts some arbitrary *<symbol>* which is used for every coordinate. As **explicit symbolic** and **TeX code symbolic**, this choice yields symbolic representations, i.e. it is kept as-is without mapping the result.

The difference to **explicit symbolic** is that *<symbol>* is a common symbol for all points whereas **explicit symbolic** expects the input data stream (like a table) to provide individual symbols.

The different to **TeX code symbolic** is marginal: **symbolic** is actually the same as `point meta/TeX code symbolic={\def\pgfplotspointmeta{<symbol>}}.`

A use-case for **symbolic** is `mesh/color input=explicit mathparse`, see the documentation therein.

Note that you need an extra pair of braces to provide this key, i.e.

`point meta={symbolic={<symbol>}}.`

As already mentioned, a main application of point meta data is to determine (marker/face/edge) colors using a linear map into the range $[0, 1000]$ (maybe for use in the current color map). This map works as follows: it is a function

$$\phi: [m_{\min}, m_{\max}] \rightarrow [0, 1000]$$

with

$$\phi(m) = \frac{m - m_{\min}}{1000}$$

such that $\phi(m_{\min}) = 0$ and $\phi(m_{\max}) = 1000$. The value 1000 is – per convention – the upper limit of all color maps. Now, if a coordinate (or edge/face) has the point meta data *m*, its color will be determined using $\phi(m)$: it is the color at $\phi(m)\%$ of the current color map.

This transformation depends on the interval $[m_{\min}, m_{\max}]$ which, in turn, can be modified using the keys **point meta rel**, **point meta min** and **point meta max** described below.

The untransformed point meta data is available in the macro `\pgfplotspointmeta` (only in the correct context, for example the scatter plot styles or the `scatter/@pre marker code` interface). This macro contains a low level floating point number (unless it is non-parsed string data). The transformed data will be available in the macro `\pgfplotspointmetatransformed` and is in fixed point representation. It is expected to be in the range $[0, 1000]$.

³⁶In rare circumstances, it might be interesting to apply a math expression to another source of point meta (one of the other choices. To this end, the *<expression>* is checked after the other possible choices have already been evaluated. In other words, the statement `point meta=explicit, point meta=meta*meta+3` will evaluate the expression with **meta** set to whatever data has been provided explicitly.

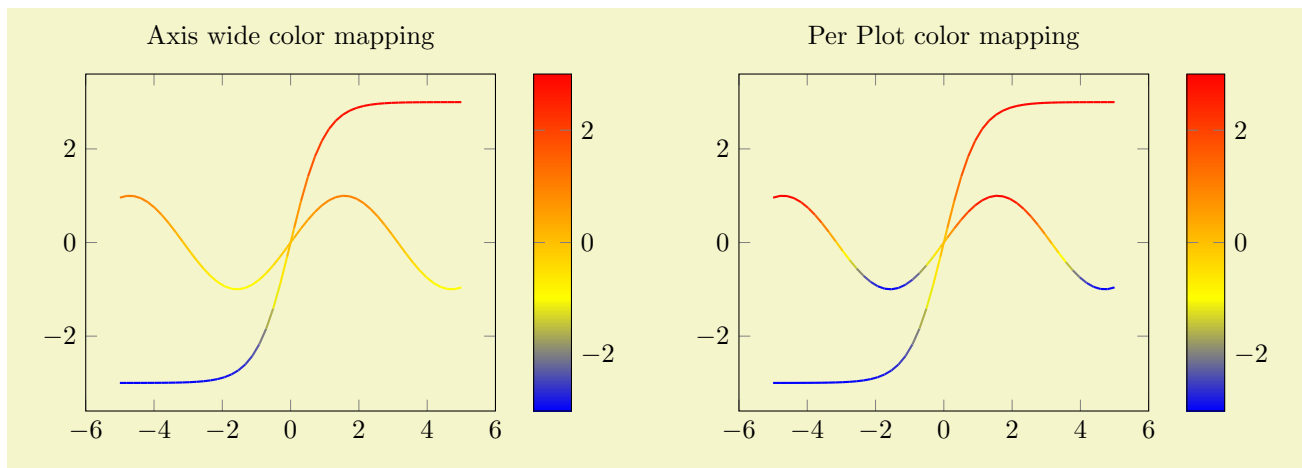
/pgfplots/**set point meta if empty**= $\{\langle point meta source \rangle\}$

Sets `point meta`= $\langle point meta source \rangle$, but only if `point meta`=none currently. This is used for `scatter`, `mesh` and `surf` with `set point meta if empty`= $f(x)$.

/pgfplots/**point meta rel**=axis wide|per plot (initially axis wide)

As already explained in the documentation for `point meta`, one application for point meta data is to determine colors using the current color map and a linear map from point meta data into the current color map. The question is how this linear map is computed.

The key `point meta rel` configures whether the interval of all point meta coordinates, $[m_{\min}, m_{\max}]$ is computed as maximum over all plots in the complete axis (the choice `axis wide`) or only for one particular plot (the choice `per plot`).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title=Axis wide color mapping,
    colorbar,
    samples=50,point meta rel=axis wide,
    point meta=y]

    \addplot[mesh,thick] {sin(deg(x))};
    \addplot[mesh,thick] {3*tanh(x)};
  \end{axis}
\end{tikzpicture}
~
\begin{tikzpicture}
  \begin{axis}[
    title=Per Plot color mapping,
    colorbar,
    samples=50,
    point meta rel=per plot,
    point meta=y]

    \addplot[mesh,thick] {sin(deg(x))};
    \addplot[mesh,thick] {3*tanh(x)};
  \end{axis}
\end{tikzpicture}
```

Note that a `colorbar` will still use the axis wide point meta limits. Consider the `colorbar source` key if you want the color data limits of a *particular* plot for your color bar. The `point meta rel` key configures how point meta maps to colors in the `colormap`.

/pgfplots/**point meta min**= $\{\langle number \rangle\}$

/pgfplots/**point meta max**= $\{\langle number \rangle\}$

These keys allow to define the range required for the linear map of point meta data into the range $[0, 1000]$ (for example, for current maps) explicitly. This is necessary if the same mapping shall be used for more than one axis.

Remarks about special cases:

- It is possible to provide limits partially; in this case, only the missing limit will be computed.
- If point meta data falls outside of these limits, the linear transformation is still well defined which is acceptable (unless the interval is of zero length). However, color data can't be outside of these limits, so color bars perform a truncation.
- This key can be provided for single plots as well as for the complete axis (or for both).
- If meta limits are provided for a single plot, these limits may also contribute to the axis wide meta interval.

`/pgfplots/colormap access=map|direct` (initially `map`)

This key configures how point meta data is used to determine colors from a color map. The initial configuration `map` performs the linear mapping operation explained above. The choice `direct` does not perform any transformation; it takes the point meta as integer indices into the current color map.

Consequently, there is no interpolation between colors in the color map, there will only be as many colors as the color map contains explicitly.

Some more details:

- If there are m colors in the color map and the color data falls outside of $[0, m - 1]$, it will be pruned to either the first or the last color.
- If color data is a real number, it will be truncated to the next smaller integer.
- This key does not work for `shader=interp` (note that this shader will always interpolate in the color map).

Attention: This feature is experimental and has never been used or tested so far. If you believe that it should receive more attention, let us know.

4.8 Axis Descriptions

Axis descriptions are labels for x and y axis, titles, legends and the like. Axis descriptions are drawn after the plot is finished and they are not subjected to clipping.

4.8.1 Placement of Axis Descriptions

This section describes how to *modify* the placement of titles, labels, legends and other axis descriptions. It may be skipped at first reading.

There are different methods to place axis descriptions. One of them is to provide coordinates relative to the axis' rectangle such that $(0,0)$ is the lower left corner and $(1,1)$ is the upper right corner – this is very useful for figure titles or legends. Coordinates of this type, i.e. without unit like $(0,0)$ or $(1.03,1)$, are called `axis description cs` (the `cs` stands for “coordinate system”). One other method is of primary interest for axis labels – they should be placed near the tick labels, but in a way that they don't overlap or obscure tick labels. Furthermore, axis labels shall be placed such that they are automatically moved if the axis is rotated (or tick labels are moved to the right side of the figure). There is a special coordinate system to realize these two demands, the `ticklabel cs`.

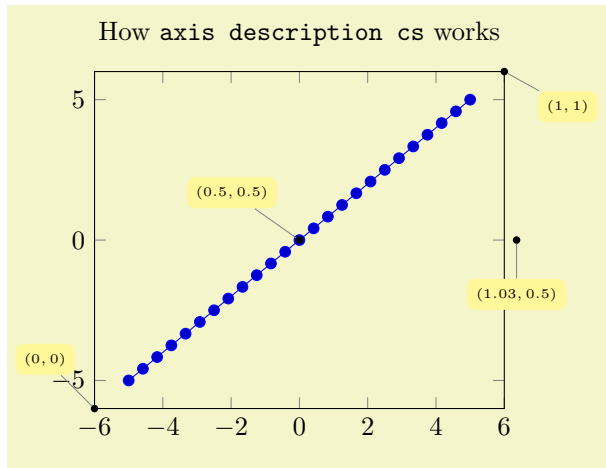
In the following, the two coordinate systems `axis description cs` and `ticklabel cs` are described in more detail. It should be noted that `axis description cs` is used automatically, so it might never be necessary to use it explicitly.

Coordinate system `axis description cs`

A coordinate system which is used to place axis descriptions. Whenever the option `'at={(\langle x \rangle, \langle y \rangle)}` occurs in `label style`, `legend style` or any other axis description, $(\langle x \rangle, \langle y \rangle)$ is interpreted to be a coordinate in `axis description cs`.

The point $(0,0)$ is always the lower left corner of the tightest bounding box around the axes (without any descriptions or ticks) while the point $(1,1)$ is the upper right corner of this bounding box.

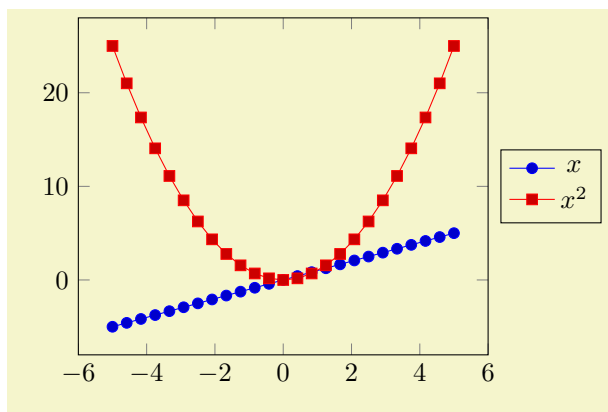
In most cases, it is *not* necessary to explicitly write `axis description cs` as it is the default coordinate system for any axis description. An example for how coordinates are placed is shown below.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% [See the TikZ manual if you'd like to learn about nodes and pins]
\begin{tikzpicture}
  \tikzset{
    every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
    small dot/.style={fill=black,circle,scale=0.3}
  }
  \begin{axis}[
    clip=false,
    title=How \texttt{axis description cs} works
  ]
    \addplot {x};

    \node[small dot,pin=120:{$(0,0)$}] at (axis description cs:0,0) {};
    \node[small dot,pin=-30:{$(1,1)$}] at (axis description cs:1,1) {};
    \node[small dot,pin=-90:{$(1.03,0.5)$}] at (axis description cs:1.03,0.5) {};
    \node[small dot,pin=125:{$(0.5,0.5)$}] at (axis description cs:0.5,0.5) {};
  \end{axis}
\end{tikzpicture}
```

Axis descriptions are TikZ nodes, that means all placement and detail options of [5] apply. The point on the node's boundary which is actually shifted to the `at` coordinate needs to be provided with an anchor (cf [5, Nodes and Edges]):

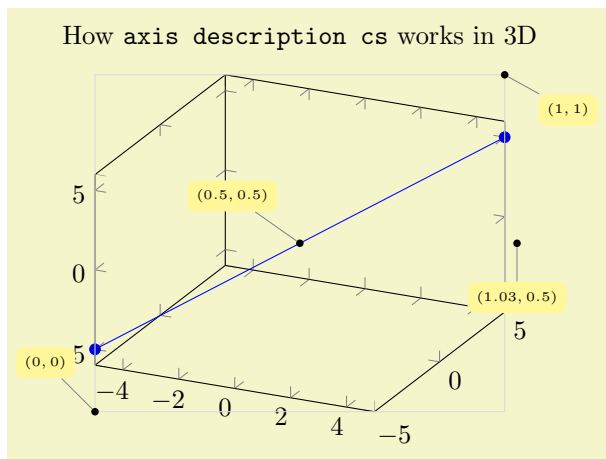


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    legend entries={$x$, $x^2$},
    legend style={
      at={(1.03,0.5)},
      anchor=west
    }
  ]
    \addplot {x};
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

Standard anchors of nodes are **north**, **east**, **south**, **west** and mixed components like **north east**. Please refer to [5] for a complete documentation of anchors.

Remarks:

- Each of the anchors described in Section 4.18 can be described by **axis description cs** as well.
- The **axis description cs** is independent of axis reversals or skewed axes. Only for the default configuration of boxed axes is it the same as **rel axis cs**, i.e. (0,0) is the same as the smallest axis coordinate and (1,1) is the largest one in case of standard boxed axes³⁷.
- Even for three dimensional axes, the **axis description cs** is still two-dimensional: it always refers to coordinates relative to the tightest bounding box around the axis (without any descriptions or ticks).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% the same as above for 3D ...
% [See the TikZ manual if you'd like to learn about nodes and pins]
\begin{tikzpicture}
  \tikzset{
    every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
    small dot/.style={fill=black,circle,scale=0.3}
  }
  \begin{axis}[
    clip=false,
    title=How \texttt{axis description cs} works in 3D
  ]
  \addplot3 coordinates {(-5,-5,-5) (5,5,5)};

  \draw[black!15] (axis description cs:0,0) rectangle (axis description cs:1,1);

  \node[small dot,pin=120:{{(0,0)}}] at (axis description cs:0,0) {};
  \node[small dot,pin=-30:{{(1,1)}}] at (axis description cs:1,1) {};
  \node[small dot,pin=-90:{{(1.03,0.5)}}] at (axis description cs:1.03,0.5) {};
  \node[small dot,pin=125:{{(0.5,0.5)}}] at (axis description cs:0.5,0.5) {};
  \end{axis}
\end{tikzpicture}
```

- Since the view does not influence these positions, **axis description cs** might not be a good choice for axis labels in 3D. The **ticklabel cs** is used in this case.

Coordinate system **xticklabel cs**
 Coordinate system **yticklabel cs**
 Coordinate system **zticklabel cs**
 Coordinate system **ticklabel cs**
 Coordinate system **xticklabel* cs**
 Coordinate system **yticklabel* cs**
 Coordinate system **zticklabel* cs**

³⁷This was different in versions before 1.3: earlier versions did not have the distinction between **axis description cs** and **rel axis cs**.

Coordinate system `ticklabel* cs`

A set of special coordinate systems intended to place axis descriptions (or any other drawing operation) besides tick labels, in a way such that neither tick labels nor the axis as such are obscured.

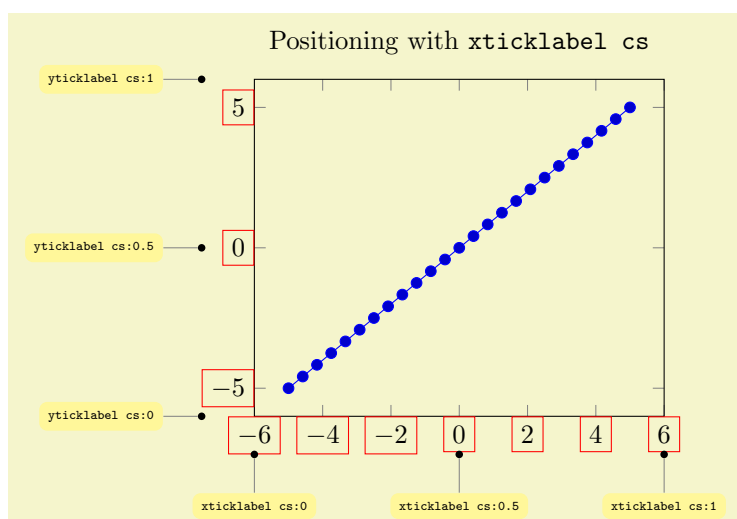
See also `xlabel near ticks` as one main application of `ticklabel cs`.

The `xticklabel cs` (and its variants) always refer to one, uniquely identified axis: the one which is (or would be) annotated with tick labels.

The `ticklabel cs` (without explicit `x`, `y` or `z`) can only be used in contexts where the axis character is known from context (for example, inside of `xlabel style` – there, the `ticklabel cs` is equivalent to `xticklabel cs`).

The starred variants `xticklabel* cs` and its friends do not take the size of any tick labels into account.

Each of these coordinate systems allows to specify points on a straight line which is placed parallel to an axis containing tick labels, moved away just far enough to avoid overlaps with the tick labels:



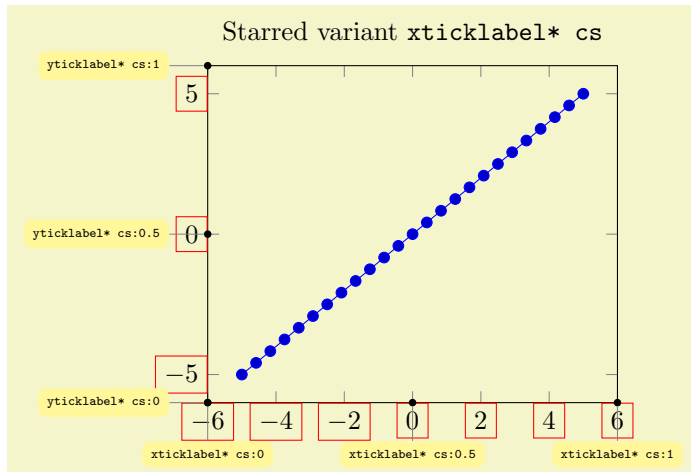
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\tikzset{
  every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
  small dot/.style={fill=black,circle,scale=0.3}
}
\begin{tikzpicture}
\begin{axis}[
  clip=false,
  ticklabel style={draw=red},
  title=Positioning with \texttt{xticklabel cs}
\addplot {x};
\node[small dot,pin=-90:\texttt{xticklabel cs:0}] at (xticklabel cs:0) {};
\node[small dot,pin=-90:\texttt{xticklabel cs:0.5}] at (xticklabel cs:0.5) {};
\node[small dot,pin=-90:\texttt{xticklabel cs:1}] at (xticklabel cs:1) {};

\node[small dot,pin=180:\texttt{yticklabel cs:0}] at (yticklabel cs:0) {};
\node[small dot,pin=180:\texttt{yticklabel cs:0.5}] at (yticklabel cs:0.5) {};
\node[small dot,pin=180:\texttt{yticklabel cs:1}] at (yticklabel cs:1) {};
\end{axis}
\end{tikzpicture}
```

The basic idea is to place coordinates on a straight line which is parallel to the axis containing tick labels – but shifted such that the line does not cut through tick labels.

Note that an axis description which has been placed with `xticklabel cs` or its friends is also useful for skewed axes or the `axis x line` variants – it is often the same value for all these variants. In particular, it is useful for three-dimensional axes, see below.

Typically, `xticklabel cs` places nodes exactly at the position where the largest associated tick label is finished. While this is very useful, it might be undesired – for example if one wants to move into the opposite direction (here, the special anchor `near ticklabel opposite` might be of interest). To this end, there are the starred variants, i.e. `xticklabel* cs` and its friends:

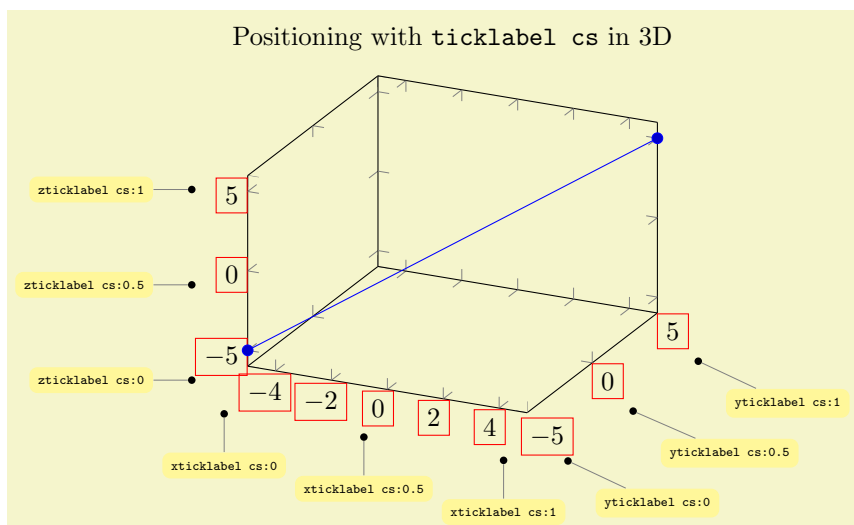


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\tikzset{
  every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
  small dot/.style={fill=black,circle,scale=0.3}
}
\begin{tikzpicture}
\begin{axis}[
  clip=false,
  ticklabel style={draw=red},
  title=Starred variant \texttt{xticklabel* cs}
]
\addplot {x};
\node[small dot,pin=-90:{\texttt{xticklabel* cs:0}}] at (xticklabel* cs:0) {};
\node[small dot,pin=-90:{\texttt{xticklabel* cs:0.5}}] at (xticklabel* cs:0.5) {};
\node[small dot,pin=-90:{\texttt{xticklabel* cs:1}}] at (xticklabel* cs:1) {};

\node[small dot,pin=180:{\texttt{yticklabel* cs:0}}] at (yticklabel* cs:0) {};
\node[small dot,pin=180:{\texttt{yticklabel* cs:0.5}}] at (yticklabel* cs:0.5) {};
\node[small dot,pin=180:{\texttt{yticklabel* cs:1}}] at (yticklabel* cs:1) {};
\end{axis}
\end{tikzpicture}
```

The preceding example places all the additional anchors precisely onto the axis on which tick labels are drawn. The starred version `xticklabel* cs` ignores the size of tick labels.

Of course, it is relatively simple to get the same coordinates as in the two dimensional example above with `axis description cs`, except that `ticklabel cs` always respects the tick label sizes appropriately. However, `ticklabel cs` becomes far superior when it comes to three dimensional positioning:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% the same as above for 3D ...
\begin{tikzpicture}
  \tikzset{
    every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
    small dot/.style={fill=black,circle,scale=0.3}
  }
  \begin{axis}[
    ticklabel style={draw=red},
    clip=false,
    title=Positioning with \texttt{ticklabel cs} in 3D
  ]
  \addplot3 coordinates {(-5,-5,-5) (5,5,5)};

  \node[small dot,pin=-90:{\texttt{xticklabel cs:0}}] at (xticklabel cs:0) {};
  \node[small dot,pin=-90:{\texttt{xticklabel cs:0.5}}] at (xticklabel cs:0.5) {};
  \node[small dot,pin=-90:{\texttt{xticklabel cs:1}}] at (xticklabel cs:1) {};

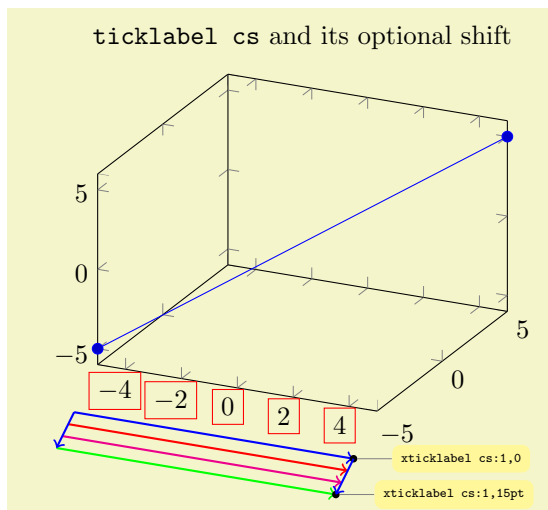
  \node[small dot,pin=-45:{\texttt{yticklabel cs:0}}] at (yticklabel cs:0) {};
  \node[small dot,pin=-45:{\texttt{yticklabel cs:0.5}}] at (yticklabel cs:0.5) {};
  \node[small dot,pin=-45:{\texttt{yticklabel cs:1}}] at (yticklabel cs:1) {};

  \node[small dot,pin=180:{\texttt{zticklabel cs:0}}] at (zticklabel cs:0) {};
  \node[small dot,pin=180:{\texttt{zticklabel cs:0.5}}] at (zticklabel cs:0.5) {};
  \node[small dot,pin=180:{\texttt{zticklabel cs:1}}] at (zticklabel cs:1) {};
  \end{axis}
\end{tikzpicture}

```

The coordinate `ticklabel cs:0` is associated with the lower axis limit while `ticklabel cs:1` is near the upper axis limit. The value 0.5 is in the middle of the axis, any other values (including negative values or values beyond 1) are linearly interpolated inbetween.

All coordinate systems like `ticklabel cs` also accepts a second (optional) argument: a shift “away” from the tick labels. The shift points to a vector which is orthogonal³⁸ the associated axis, away from the tick labels. A shift of `0pt` is directly at the edge of the tick labels in direction of the normal vector, positive values move the position away and negative closer to the tick labels.



³⁸Actually, the outer normal has the impression of being “orthogonal” to its axis, which appears to be sufficient.

```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\tikzset{
  every pin/.style={fill=yellow!50!white,rectangle,rounded corners=3pt,font=\tiny},
  small dot/.style={fill=black,circle,scale=0.3}
}
\begin{tikzpicture}
  \begin{axis}[
    xticklabel style={draw=red},
    clip=false,
    title=\texttt{xticklabel cs} and its optional shift
  ]
    \addplot3 coordinates {(-5,-5,-5) (5,5,5)};

    \draw[blue,thick,->] (xticklabel cs:0,0) -- (xticklabel cs:1,0);
    \draw[red,thick,->] (xticklabel cs:0,5pt) -- (xticklabel cs:1,5pt);
    \draw[magenta,thick,->] (xticklabel cs:0,10pt) -- (xticklabel cs:1,10pt);
    \draw[green,thick,->] (xticklabel cs:0,15pt) -- (xticklabel cs:1,15pt);
    \node[small dot,pin=0:{\texttt{xticklabel cs:1,0}}] at (xticklabel cs:1,0) {};
    \node[small dot,pin=0:{\texttt{xticklabel cs:1,15pt}}] at (xticklabel cs:1,15pt) {};

    \draw[blue,thick,->] (xticklabel cs:0,0) -- (xticklabel cs:0,15pt);
    \draw[blue,thick,->] (xticklabel cs:1,0) -- (xticklabel cs:1,15pt);
  \end{axis}
\end{tikzpicture}

```

Whenever the `ticklabel cs` is used, the anchor should be set to `anchor=near ticklabel` (see below).

Whenever the starred version `ticklabel* cs` is used, both anchors `anchor=near ticklabel` and `anchor=near ticklabel opposite` are useful choices.

There is one specialty: if you reverse an axis (with `x dir=reverse`), points provided by `ticklabel cs` will be *unaffected* by the axis reversal. This is intended to provide consistent placement even for reversed axes. Use `allow reversal of rel axis cs=false` to disable this feature.

Besides the mentioned positioning methods, there is also the predefined node `current axis`. The anchors of `current axis` can also be used to place descriptions: At the time when axis descriptions are drawn, all anchors which refer to the axis origin (that means the “real” point (0,0)) or any of the axis corners can be referenced using `current axis.<anchor name>`. Please see Section 4.18, Alignment, for further details.

4.8.2 Alignment of Axis Descriptions

This section describes how to modify the default alignment of axis descriptions. It can be skipped at first reading.

The two topics positioning and alignment always work together: *positioning* means to select an appropriate coordinate and *alignment* means to select an anchor inside of the description which will actually be moved to the desired position.

TikZ uses many anchors to provide alignment; most of them are named like `north`, `north east` etc. These names hold for any axis description as well (as axis descriptions are TikZ nodes). Readers can learn details about this topic in the TikZ manual [5] or some more advice in Section 4.18.

When it comes to axis descriptions, PGFLOTS offers some specialized anchors and alignment methods which are described below.

Anchor `near xticklabel`

Anchor `near yticklabel`

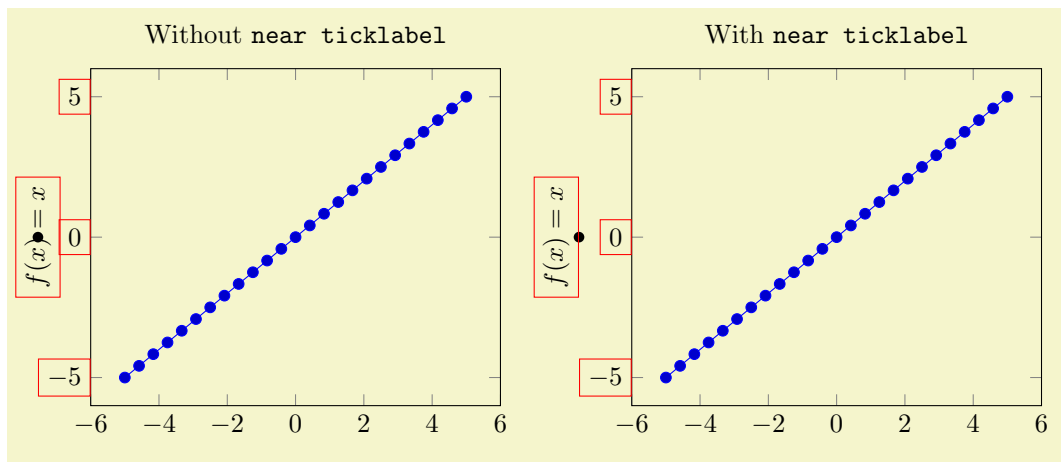
Anchor `near zticklabel`

Anchor `near ticklabel`

These anchors can be used to align at the part of a node (for example, an axis description) which is *nearest* to the tick labels of a particular axis (or nearest to the position where tick labels would have been drawn if there were any).

These anchors are used for axis labels, especially for three dimensional axes. Furthermore, they are used for every tick label.

Maybe it is best to demonstrate it by example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    title=Without \texttt{near ticklabel},
    ylabel={\mathit{f}(x)=x\$},
    every axis y label/.style={
      {at={(ticklabel cs:0.5)},rotate=90,anchor=center},
      clip=false,% to display the \path below
    },
    ylabel style={draw=red},
    yticklabel style={draw=red}
  ]

    \addplot {x};

    % visualize the position:
    \fill (yticklabel cs:0.5) circle(2pt);
  \end{axis}
\end{tikzpicture}%
~
\begin{tikzpicture}
  \begin{axis}[
    title=With \texttt{near ticklabel},
    ylabel={\mathit{f}(x)=x\$},
    every axis y label/.style={
      {at={(ticklabel cs:0.5)},rotate=90,anchor=near ticklabel},
      clip=false,
    },
    ylabel style={draw=red},
    yticklabel style={draw=red}
  ]

    \addplot {x};
    \fill (yticklabel cs:0.5) circle(2pt);
  \end{axis}
\end{tikzpicture}
```

The motivation is to place nodes such that they are anchored next to the tick label, regardless of the node's rotation or the position of ticks. The special anchor `near ticklabel` is only available for axis labels (as they have a uniquely identified axis, either x , y or z).

In more detail, the anchor is placed such that first, the node's center is on a line starting in the node's `at` position going in direction of the inwards normal vector of the axis line which contains the tick labels and second, the node does not intrude the axis (but see also the key `near ticklabel align` and the details in the lengthy elaboration in the documentation for `near xticklabel opposite` below). This normal vector is the same which is used for the `shift` argument in `ticklabel cs`: it is orthogonal to the tick label axis. Furthermore, `near ticklabel` inverts the transformation matrix before it computes this intersection point.

The `near ticklabel` anchor and its friends will be added temporarily to any shape used inside of an axis. This includes axis descriptions, but it is not limited to them: it applies to every `TikZ \node[anchor=near xticklabel] ...` setting.

Note that it is not necessary at all to *have* tick labels in an axis. The anchor will be placed such that it is near the axis on which tick labels *would* be drawn. In fact, every tick label uses `anchor=near`

`ticklabel` as initial configuration.

Anchor `near xticklabel opposite`

Anchor `near yticklabel opposite`

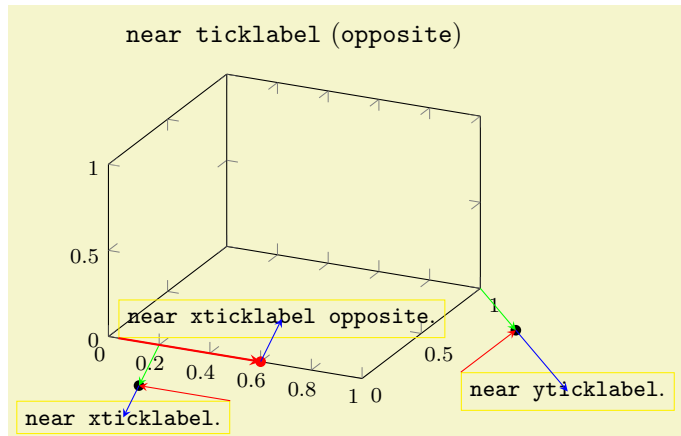
Anchor `near zticklabel opposite`

Anchor `near ticklabel opposite`

These anchors are similar to `near xticklabel` and its variants, except that they align at the *opposite* direction.

Mathematically speaking, the only difference to `near xticklabel` and its variants is the sign in front of the normal vector.

But it is probably best explained by means of an example.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    clip=false,
    small,
    title=\texttt{near ticklabel} (\texttt{opposite}),
    min=0, max=1
  ]
    \node[draw=yellow,anchor=near xticklabel,font=\small]
      (name) at (xticklabel cs:0.2) {\texttt{near xticklabel}.};
    \fill (xticklabel cs:0.2) circle(2pt);
    \draw[green,-stealth] (xticklabel* cs:0.2) -- (xticklabel cs:0.2);
    \draw[blue,-stealth] (xticklabel cs:0.2) -- (name.center);
    \draw[red,-stealth] (name.north east) -- (xticklabel cs:0.2);

    \node[draw=yellow,anchor=near xticklabel opposite,font=\small]
      (name) at (xticklabel* cs:0.6) {\texttt{near xticklabel opposite}.};
    \fill (xticklabel* cs:0.6) circle(2pt);
    \draw[blue,-stealth] (xticklabel* cs:0.6) -- (name.center);
    \draw[red,thick,-stealth] (name.south west) -- (xticklabel* cs:0.6);

    \node[draw=yellow,anchor=near yticklabel,font=\small]
      (name) at (yticklabel cs:1) {\texttt{near yticklabel}.};
    \fill (yticklabel cs:1) circle(2pt);
    \draw[green,-stealth] (yticklabel* cs:1) -- (yticklabel cs:1);
    \draw[blue,-stealth] (yticklabel cs:1) -- (name.center);
    \draw[red,-stealth] (name.north west) -- (yticklabel cs:1);
  \end{axis}
\end{tikzpicture}
```

The figure is a boxed three-dimensional axis with standard ranges. It has three manually placed nodes. The nodes are placed at `(xticklabel cs:0.2)`, at `(xticklabel* cs:0.6)`, and at `(yticklabel cs:1)`, respectively. The ‘at’ locations are visually emphasized using filled circles.

Despite the different locations, we clearly see the effect of `near xticklabel opposite`: it causes the node to be aligned into the box rather than outside of the box. Note that this is the *only* essential difference between the two nodes ‘`near xticklabel opposite`’ and ‘`near xticklabel`’.

The figure also shows the difference between `xticklabel cs` and `xticklabel* cs` when we compare

the ‘at’ locations. Take, for example, the two nodes on the x axis. The position at (`xticklabel cs:0.2`) is shifted by the green arrow. The length of this arrow is precisely the length of the largest x tick label. The position at (`xticklabel* cs:0.6`) is exactly on the axis; it ignores the size of any tick labels. Note that the direction of the green arrow is the “outer normal vector in x direction” (in our case, it is the vector sum of the z and y unit vectors with appropriate signs).

The difference between `near xticklabel opposite` and `near xticklabel` is that the direction of the green arrow (the “outer normal”) is flipped.

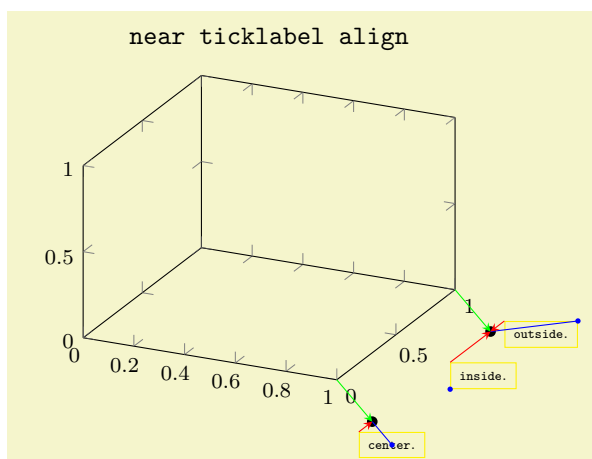
The nodes also highlight how the anchoring works. This technique is almost the same for both `anchor=near xticklabel` and `anchor=near xticklabel opposite`. Let us discuss the technique for the node with text ‘`near yticklabel`’. The black circle is placed at (`yticklabel cs:1`). This position has been computed by starting at 100% of the y axis³⁹ and moving along the green vector whose magnitude is the size of the bounding box of the largest y tick label. As soon as the ‘at’ location is fixed, the algorithm for `near yticklabel` starts. First, it computes the anchor inside of the node for which we do not penetrate the y axis. To this end, it checks the direction of the green vector. It came up with `north west`. Then, it considers the red line which starts at (`name.north west`) and has the same direction as the y axis. Note that the red line and the y axis are parallel. It also considers the blue line. This line points into the direction of the green line and is fixed by the current node’s `center`. The precise intersection point of the red line and the blue line are the result of `anchor=near yticklabel`. The same applies for the other two nodes as well: the red line is always parallel to the axis under consideration and is anchored at the “snap-to-nearest-anchor” of the node. The blue line is always parallel to the outer normal vector of the axis under consideration, and is anchored at the current node’s `center`.

`/pgfplots/near ticklabel align=inside|center|outside` (initially `center`)

Allows to change the alignment algorithm of `anchor=near ticklabel`.

In the default configuration, `anchor=near ticklabel` and its variants take the node’s `center` in order to derive the final anchor. This corresponds to the target of the blue line in the illustration for `near xticklabel opposite`, see above.

This key changes the node’s anchor which is used here. The choice `center` can move half of the node’s bounding box beyond the associated axis. The choice `inside` moves the complete node’s bounding box in a way such that it is not beyond the associated axis. The choice `outside` moves the entire node’s bounding box in a way that it is beyond the associated axis:



³⁹In our example, percentages and absolute values are accidentally the same.

```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    clip=false,
    small,
    title=\texttt{near ticklabel align},
    min=0, max=1,
    /tikz/node style/.style={
      draw=yellow,
      anchor=near yticklabel,
      font=\tiny,
    },
  ]
    \node[node style,
      /pgfplots/near ticklabel align=center]
      (C) at (yticklabel cs:0) {\texttt{center}.};

    \node[node style,
      /pgfplots/near ticklabel align=inside]
      (I) at (yticklabel cs:1) {\texttt{inside}.};

    \node[node style,
      /pgfplots/near ticklabel align=outside]
      (O) at (yticklabel cs:1) {\texttt{outside}.};

    \fill (yticklabel cs:0) circle(2pt);
    \fill (yticklabel cs:1) circle(2pt);

    \fill[blue] (C.center) circle(1pt);
    \fill[blue] (I.south west) circle(1pt);
    \fill[blue] (O.north east) circle(1pt);

    \draw[green,-stealth] (yticklabel* cs:0) -- (yticklabel cs:0);
    \draw[blue] (yticklabel cs:0) -- (C.center);
    \draw[red,-stealth] (C.north west) -- (yticklabel cs:0);

    \draw[green,-stealth] (yticklabel* cs:1) -- (yticklabel cs:1);
    \draw[blue] (yticklabel cs:1) -- (O.north east);
    \draw[red,-stealth] (O.north west) -- (yticklabel cs:1);
    \draw[red,-stealth] (I.north west) -- (yticklabel cs:1);
  \end{axis}
\end{tikzpicture}

```

The example is similar to the one above: it generates an empty axis with a default range. Then, it creates three nodes: one for choice **center**, one for choice **inside**, and one for choice **outside**. The node for **center** is placed at `(yticklabel cs:0)` (lower black circle). We see that its bounding box extends the size of the y axis. This is because its anchor `C.center` is used for the alignment. Both the node for **inside** and the node for **outside** are placed at `(yticklabel cs:1)` (upper black circle). Their only difference is the choice for `near ticklabel align`. The node for **inside** does not extend the size of the y axis; it is placed within its boundaries – because its internal anchor `(I.north east)` (blue) has automatically been used in order to align the node. The node for **outside** is completely outside of the extends for the y axis because its (blue) anchor `(O.south west)` has been chosen.

Note that the red lines are always the same. They are the “snap-to-nearest” anchor such that the node is outside of the axis. Only the location of the blue anchors is affected by this key.

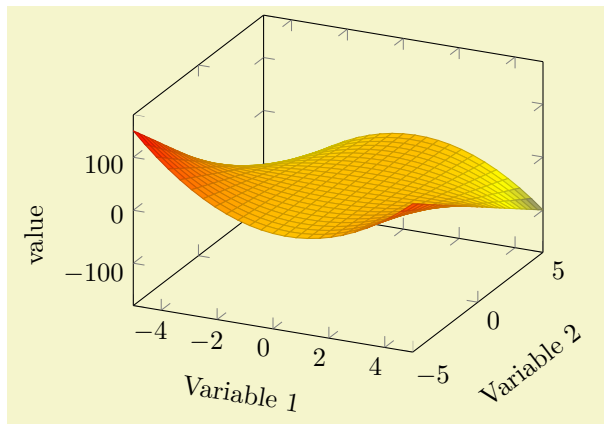
Note that `near ticklabel align` always results in the same alignment, independent of the actual *position* of the node. This is because an `anchor` is independent of the `at` location of a node. In this context, the names “inside” and “outside” might be a bad choice: they stress the intended meaning if the node is chosen at the upper end of the axis. However, if you say `at (yticklabel cs:0)`, `near ticklabel align=inside`, it will actually end up *outside* of the axis. This is because the “inside” anchor has been computed without considering where the node is.

```

/tikz/sloped like x axis (no value)
/tikz/sloped like y axis (no value)
/tikz/sloped like z axis (no value)
/tikz/sloped like x axis={\options}
/tikz/sloped like y axis={\options}
/tikz/sloped like z axis={\options}

```

A key which replaces the rotational / scaling parts of the transformation matrix such that the node is sloped like the provided axis. For two dimensional plots, `sloped like y axis` is effectively the same as `rotate=90`. For a three dimensional axis, this will lead to a larger difference:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xlabel=Variable 1,
    ylabel=Variable 2,
    zlabel=value,
    xlabel style={sloped like x axis},
    ylabel style={sloped}
  ]
    \addplot3[surf] {y*x*(1-x)};
  \end{axis}
\end{tikzpicture}
```

Inside of axis labels, `sloped` is an alias for `sloped like <char> axis` with the correct `<char>` chosen automatically.

Please note that rotated text might not look very good (neither on screen nor printed).

It is possible to customize `sloped like x axis` by means of the following keys, which need to be provided as `<options>` (simply ignore the lengthy gray key prefixes):

`/pgfplots/sloped/allow upside down=true|false` (initially false)

Use `sloped like x axis=allow upside down` to enable upside down labels.

`/pgfplots/sloped/execute for upside down=code` (initially empty)

Use `sloped like x axis={execute for upside down=\tikzset{anchor=north}}` or something like that to handle upside down text nodes in a customized way (this is used by the `smithchart` library).

`/pgfplots/sloped/reset nontranslations=true|false` (initially true)

Use `sloped like x axis={reset nontranslations=false}` to *append* the transformations to the actual transformation matrix (instead of replacing it).

4.8.3 Labels

```
/pgfplots/xlabel={<text>}
```

```
/pgfplots/ylabel={<text>}
```

```
/pgfplots/zlabel={<text>}
```

These options set axis labels to `<text>` which is any T_EX text.

To include special characters, you can use curly braces: “`xlabel={, = characters}`”. This is necessary if characters like ‘=’ or ‘,’ need to be included literally.

Use `xlabel/.add={<prefix>}{<suffix>}` to modify an already assigned label.

Labels are TikZ-nodes which are placed with

```
% for x:
\node
  [style=every axis label,
   style=every axis x label]

% for y:
\node
  [style=every axis label,
   style=every axis y label]
```

so their position and appearance can be customized.

For example, a multiline `xlabel` can be configured using

```
\begin{axis}[xlabel style={align=right,text width=3cm},xlabel=A quite long label with a line break]
...
\end{axis}
```

See [5] to learn more about `align` and `text width`.

Upgrade notice: Since version 1.3, label placement *can* respect the size of adjacent tick labels. Use `\pgfplotsset{compat=1.3}` (or newer) in the preamble to activate this feature. See `xlabel near ticks` for details.

```
/pgfplots/xlabel shift={⟨dimension⟩} (initially 0pt)
/pgfplots/ylabel shift={⟨dimension⟩} (initially 0pt)
/pgfplots/zlabel shift={⟨dimension⟩} (initially 0pt)
/pgfplots/label shift={⟨dimension⟩}
```

Shifts labels in direction of the outer normal vector of the axis by an amount of $\langle dimension \rangle$. The `label shift` sets all three label shifts to the same value.

Attention: This does only work if `\pgfplotsset{compat=1.3}` (or newer) has been called (more precisely: if `xlabel near ticks` is active for the respective axis).

```
/pgfplots/xlabel near ticks (no value)
/pgfplots/ylabel near ticks (no value)
/pgfplots/zlabel near ticks (no value)
/pgfplots/compat=1.3
```

These keys place axis labels (like `xlabel`) near the tick labels. If tick labels are small, labels will move closer to the axis. If tick labels are large, axis labels will move away from the axis. This is the default for every three dimensional plot, but it *won't* be used initially for two-dimensional plots for backwards compatibility. Take a look at the definition of `near ticklabel` on page 167 for an example.

The definition of these styles is

```
\pgfplotsset{
  /pgfplots/xlabel near ticks/.style={
    /pgfplots/every axis x label/.style={
      at={((ticklabel cs:0.5))},anchor=near ticklabel
    }
  },
  /pgfplots/ylabel near ticks/.style={
    /pgfplots/every axis y label/.style={
      at={((ticklabel cs:0.5))},rotate=90,anchor=near ticklabel
    }
  }
}
```

It is encouraged to write

```
\pgfplotsset{compat=1.3} % or newer
```

in your preamble to install the styles document-wide – it leads to the best output (it avoids unnecessary space). It is not activated initially for backwards compatibility with older versions which used fixed distances from the tick labels.

```
/pgfplots/xlabel absolute (no value)
/pgfplots/ylabel absolute (no value)
/pgfplots/zlabel absolute (no value)
/pgfplots/compat=pre 1.3
```

Installs placement styles for axis labels such that `xlabel` yields a description of absolute, fixed distance to the axis. This is the initial configuration (for backwards compatibility with versions before 1.3). Use `compat=1.3` to get the most recent, more flexible configuration. Take a look at the definition of `near ticklabel` on page 167 for an example.

These styles are defined by

```

\pgfplotsset{
  /pgfplots/xlabel absolute/.style={%
    /pgfplots/every axis x label/.style={at={(0.5,0)},below,yshift=-15pt},%
    /pgfplots/every x tick scale label/.style={
      at={(1,0)},yshift=-2em,left,inner sep=0pt
    },
  },
  /pgfplots/ylabel absolute/.style={%
    /pgfplots/every axis y label/.style={at={(0,0.5)},xshift=-35pt,rotate=90},%
    /pgfplots/every y tick scale label/.style={
      at={(0,1)},above right,inner sep=0pt,yshift=0.3em
    },
  },
}

```

There is no predefined absolute placement style for three dimensional axes.

Whenever possible, consider using `/append style` instead of overwriting the default styles to ensure compatibility with future versions.

```

\pgfplotsset{every axis label/.append style={...}}
\pgfplotsset{every axis x label/.append style={...}}
\pgfplotsset{every axis y label/.append style={...}}

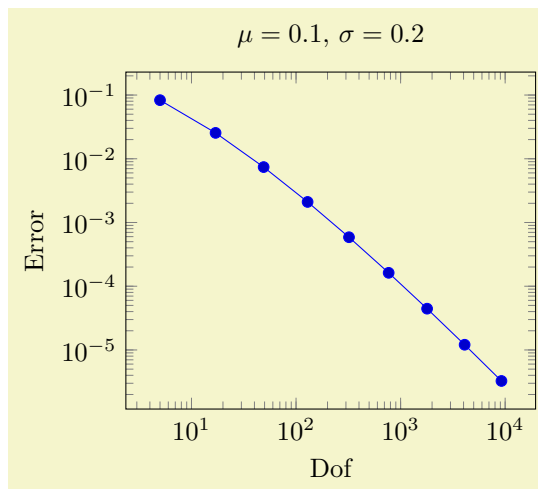
```

`/pgfplots/title={⟨text⟩}`

Adds a caption to the plot. This will place a *TikZ*-node with

```
\node[every axis title] {text};
```

to the current axis.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel=Dof,ylabel=Error,
  title={\mu=0.1$, \sigma=0.2$}]

  \addplot coordinates {
    (5, 8.312e-02)
    (17, 2.547e-02)
    (49, 7.407e-03)
    (129, 2.102e-03)
    (321, 5.874e-04)
    (769, 1.623e-04)
    (1793, 4.442e-05)
    (4097, 1.207e-05)
    (9217, 3.261e-06)
  };
\end{loglogaxis}
\end{tikzpicture}%

```

The title's appearance and/or placement can be reconfigured with

```

\pgfplotsset{title style={at={(0.75,1)}}}
% or, equivalently,
\pgfplotsset{every axis title/.append style={at={(0.75,1)}}}

```

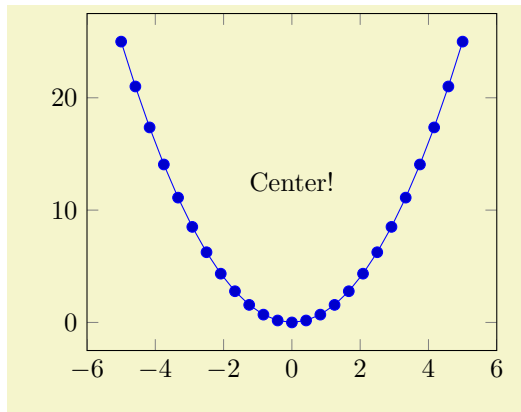
This will place the title at 75% of the x -axis. The coordinate (0,0) is the lower left corner and (1,1) the upper right one (see [axis description cs](#) for details).

Use `title/.add={⟨prefix⟩}{⟨suffix⟩}` to modify an already assigned title.

`/pgfplots/extra description/.code={⟨...⟩}`

Allows to insert $\langle commands \rangle$ after axis labels, titles and legends have been typeset.

As all other axis descriptions, the code can use (0,0) to access the lower left corner and (1,1) to access the upper right one. It won't be clipped.



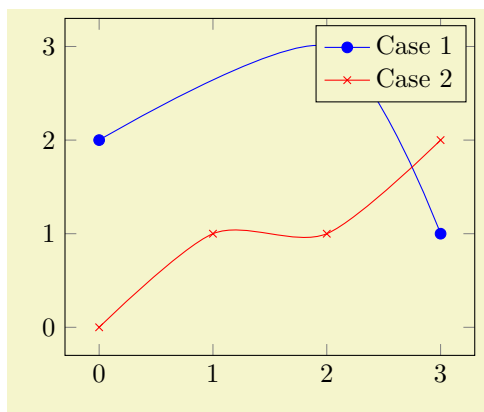
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={
  extra description/.code={
    \node at (0.5,0.5) {Center!};
  }}}
\begin{tikzpicture}
  \begin{axis}
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

4.8.4 Legends

Legends can be generated in two ways: the first is to use `\addlegendentry` or `\legend` inside of an axis. The other method is to use the key `legend entries`.

`\addlegendentry``[<options>]{<name>}`

Adds a single legend entry to the legend list. This will also enable legend drawing.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[smooth,mark=*,blue] coordinates {
      (0,2)
      (2,3)
      (3,1)
    };
    \addlegendentry{Case 1}

    \addplot[smooth,color=red,mark=x]
      coordinates {
        (0,0)
        (1,1)
        (2,1)
        (3,2)
      };
    \addlegendentry{Case 2}
  \end{axis}
\end{tikzpicture}
```

It does not matter where `\addlegendentry` commands are placed, only the sequence matters. You will need one `\addlegendentry` for every `\addplot` command (unless you prefer an empty legend).

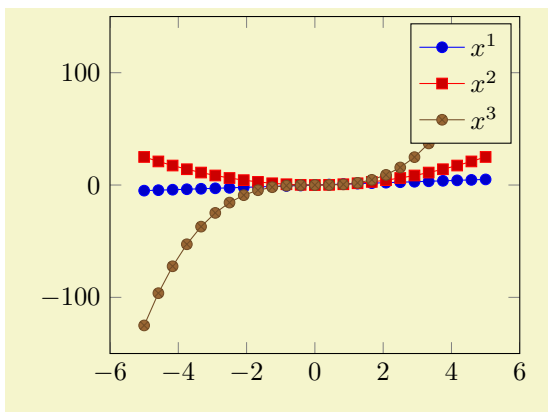
The optional *<options>* affect how the text is drawn; they apply only for this particular description text. For example, `\addlegendentry[red]{Text}` would yield a red legend text. Behind the scenes, the text is placed with `\node[<options>]{<name>}`; so *<options>* can be any TikZ option which affects nodes.

Using `\addlegendentry` disables the key `legend entries`.

`\addlegendentryexpanded``[<options>]{<TEX text>}`

A variant of `\addlegendentry` which provides a method to deal with macros inside of *<TEX text>*.

Suppose *<TEX text>* contains some sort of parameter which varies for every plot. Moreover, you like to use a loop to generate the plots. Then, it is simpler to use `\addlegendentryexpanded`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\foreach \p in {1,2,3} {
\addplot {x^\p};
\addlegendentryexpanded{$x^\p$}
}
\end{axis}
\end{tikzpicture}
```

Note that this example wouldn't have worked with `\addlegendentry{$x^\p$}` because the macro `\p` is no longer defined when PGFLOTS attempts to draw the legend.

The invocation `\addlegendentryexpanded{$x^\p$}` is equivalent to calling `\addlegendentry{$x^2$}` if `\p` expands to 2.

The argument $\langle T_{\text{E}}X \text{ text} \rangle$ is expanded until nothing but un-expandable material remains (i.e. it uses the $T_{\text{E}}X$ primitive `\edef`). Occasionally, $\langle T_{\text{E}}X \text{ text} \rangle$ contains parts which should be expanded (like `\p`) and other parts which should be left unexpanded (for example `\pgfmathprintnumber{\p}`). Then, use

`\noexpand\pgfmathprintnumber{\p}`

or, equivalently

`\protect\pgfmathprintnumber{\p}`

to avoid expansion of the macro which follows the `\protect` immediately.

`\legend{\langle list \rangle}`

You can use `\legend{\langle list \rangle}` to assign a complete legend.

```
\legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
```

The argument of `\legend` is a list of entries, one for each plot.

Two different delimiters are supported:

1. There are comma-separated lists like

```
\legend{$d=2$, $d=3$, $d=4$, $d=5$, $d=6$}
```

These lists are processed using the PGF `\foreach` command and are quite powerful.

The `\foreach` command supports a dots-notation to denote ranges like `\legend{1,2,...,5}` or even `\legend{$x^1$, $x^...$, $x^d$}`.

Attention with periods: to avoid confusion with the dots $\dots$ notation, you may need to encapsulate a legend entry containing periods by curly braces: `\legend{{ML spcm.},{CW spcm.},{ML AC}}` (or use the `\` delimiter, see below).

2. It is also possible to delimit the list by `\`. In this case, the *last element must be terminated* by `\` as well:

```
\legend{$a=1, b=2\\, $a=2, b=3\\ $a=3, b=5\\}
```

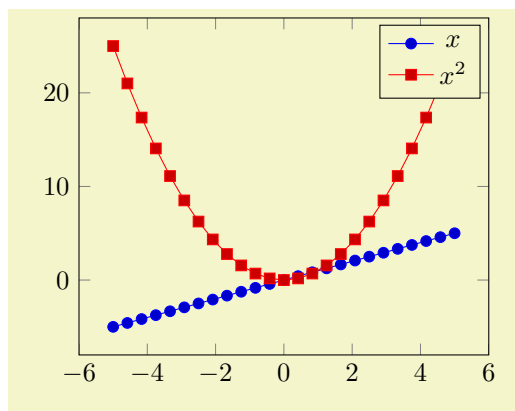
This syntax simplifies the use of `,` inside of legend entries, but it does not support the dots-notation.

The short marker/line combination shown in legends is acquired from the $\langle style options \rangle$ argument of `\addplot`.

Using `\legend` overwrites any other existing legend entries.

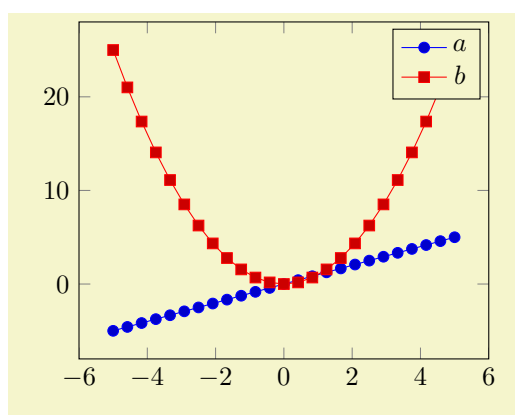
`/pgfplots/legend entries={\langle comma separated list \rangle}`

This key can be used to assign legend entries just like the commands `\addlegendentry` and `\legend`. Again, the positioning is relative to the axis rectangle (unless units like `cm` or `pt` are specified explicitly).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend entries={x$,x^2$}]
    \addplot {x};
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

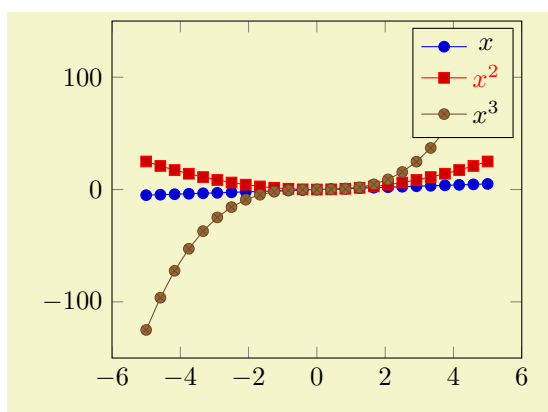
The commands for legend creation take precedence: the key `legend entries` is only considered if there is no `legend` command in the current axis.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend entries={x$,x^2$}]
    \addplot {x};
    \addplot {x^2};
    \legend{$a$, $b$}% overrides the option
  \end{axis}
\end{tikzpicture}
```

Please be careful with whitespaces in `<comma separated list>`: they will contribute to legend entries. Consider using ‘%’ at the end of each line in multiline arguments (the end of line character is also a whitespace in $\text{\TeX}$).

Just as for `\addlegendentry`, it is possible to provide `[<options>]` to single descriptions. To do so, place the options in square brackets right before the text:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend entries={x$, [red]x^2$, $x^3$}]
    \addplot {x};
    \addplot {x^2};
    \addplot {x^3};
  \end{axis}
\end{tikzpicture}
```

If the square brackets contain a comma, you can enclose the complete entry in curly braces like `{[red,font=\Huge]Text}` (or you can use the ‘\’ delimiters).

4.8.5 Legend Appearance

`/pgfplots/every axis legend`

(style, no value)

The style “`every axis legend`” determines the legend’s position and outer appearance:

```
\pgfplotsset{every axis legend/.append style={
  at={(0,0)},
  anchor=south west}}
```

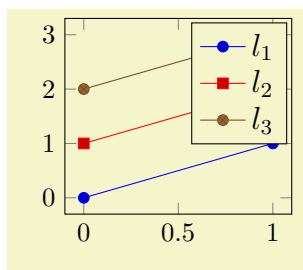
will draw it at the lower left corner of the axis while

```
\pgfplotsset{every axis legend/.append style={
  at={(1,1)},
  anchor=north east}}
```

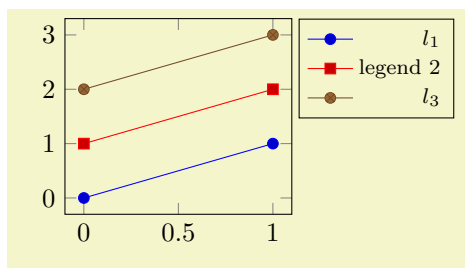
means the upper right corner. The ‘`anchor`’ option determines which point *of the legend* will be placed at (0,0) or (1,1).

The legend is a TikZ-matrix, so one can use any TikZ option which affects nodes and matrices (see [5, section 13 and 14]). The matrix is created by something like

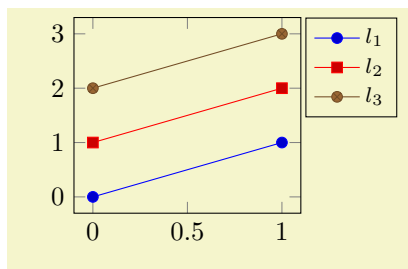
```
\matrix[style=every axis legend] {
  draw plot specification 1 & \node{legend 1}\\
  draw plot specification 2 & \node{legend 2}\\
  ...
};
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  % this modifies 'every axis legend':
  legend style={font=\large}
]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, $l_2$, $l_3$}
\end{axis}
\end{tikzpicture}
```

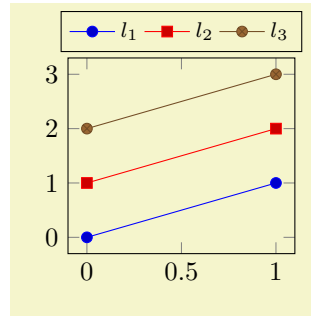


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  % align right:
  legend style={
    cells={anchor=east},
    legend pos=outer north east,
  }
]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, legend $2$, $l_3$}
\end{axis}
\end{tikzpicture}
```



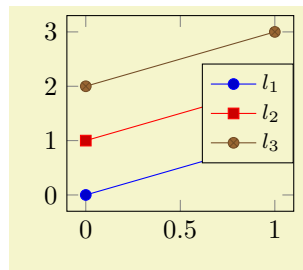
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% similar placement as previous example:
\pgfplotsset{every axis legend/.append style={
  at={(1.02,1)},
  anchor=north west}}
\begin{tikzpicture}
\begin{axis}
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, $l_2$, $l_3$}
\end{axis}
\end{tikzpicture}
```

Use `legend columns={number}` to configure the number of horizontal legend entries.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\pgfplotsset{every axis legend/.append style={
    at={(0.5,1.03)},
    anchor=south}}
\begin{axis}[legend columns=4]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, $l_2$, $l_3$}
\end{axis}
\end{tikzpicture}
```

Instead of the `/.append style`, it is possible to use `legend style` as in the following example. It has the same effect.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    legend style={
        at={(1,0.5)},
        anchor=east}]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, $l_2$, $l_3$}
\end{axis}
\end{tikzpicture}
```

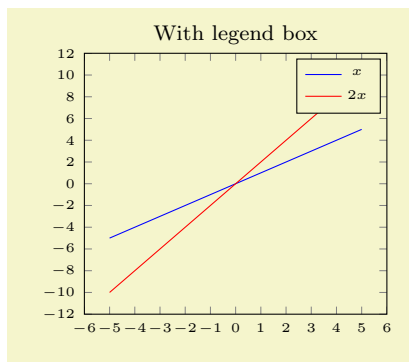
The default `every axis legend` style is

```
\pgfplotsset{every axis legend/.style={
    cells={anchor=center},% Centered entries
    inner xsep=3pt,inner ysep=2pt,nodes={inner sep=2pt,text depth=0.15em},
    anchor=north east,
    shape=rectangle,
    fill=white,
    draw=black,
    at={(0.98,0.98)}}
}
```

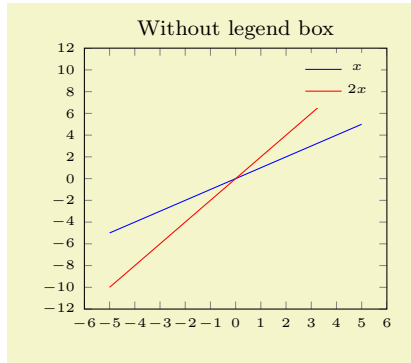
Whenever possible, consider using `/.append style` to keep the default styles active. This ensures compatibility with future versions.

```
\pgfplotsset{every axis legend/.append style={...}}
```

Note that in order to disable drawing of the legend box, you can use `draw=none` as style argument:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny,title=With legend box]
\addplot[blue]{x};
\addplot[red]{2*x};
\legend{$x$, $2x$}
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny,title=Without legend box,
  legend style={draw=none}]
\addplot[blue]{x};
\addplot[red]{2*x};
\legend{$x$,$2x$}
\end{axis}
\end{tikzpicture}
```

`/pgfplots/legend style={⟨key-value-list⟩}`

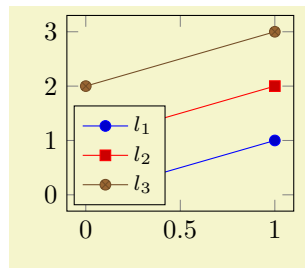
An abbreviation for `every axis legend/.append style={⟨key-value-list⟩}`.

It appends options to the already existing style `every axis legend`.

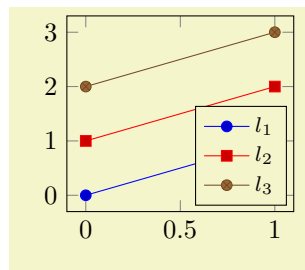
`/pgfplots/legend pos=south west|south east|north west|north east|outer north east`

A style which provides shorthand access to some commonly used legend positions.

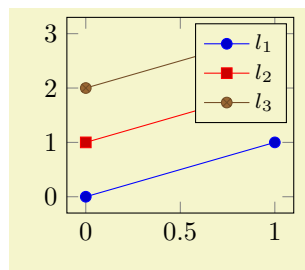
Each of these styles appends `at={⟨⟨x⟩,⟨y⟩⟩}`, `anchor=⟨name⟩` values to `every axis legend`.



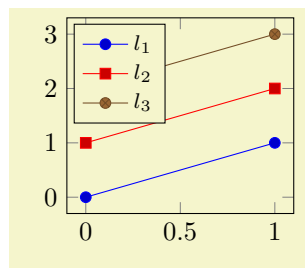
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=south west]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$,$l_2$,$l_3$}
\end{axis}
\end{tikzpicture}
```



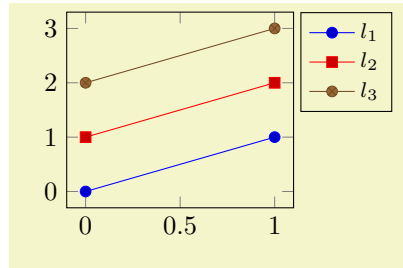
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=south east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$,$l_2$,$l_3$}
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=north east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$,$l_2$,$l_3$}
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=north west]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$,$l_2$,$l_3$}
\end{axis}
\end{tikzpicture}
```

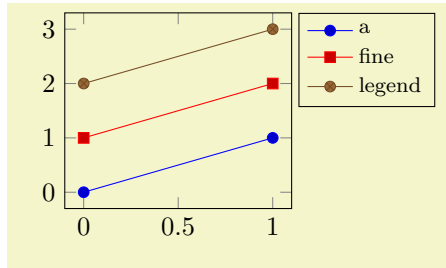


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=outer north east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{$l_1$, $l_2$, $l_3$}
\end{axis}
\end{tikzpicture}
```

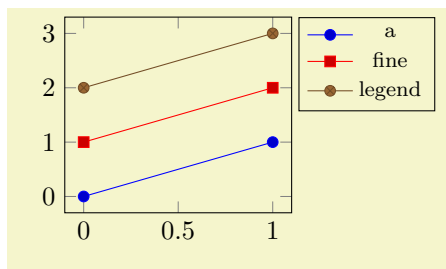
`/pgfplots/legend cell align=left|right|center`

(initially center)

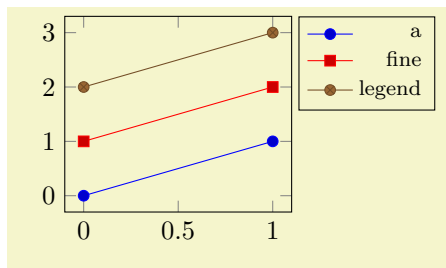
These keys provide horizontal alignment of legend cells.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend cell align=left,
legend pos=outer north east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{a, fine, legend}
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend cell align=center,
legend pos=outer north east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{a, fine, legend}
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend cell align=right,
legend pos=outer north east]
\addplot coordinates {(0,0) (1,1)};
\addplot coordinates {(0,1) (1,2)};
\addplot coordinates {(0,2) (1,3)};
\legend{a, fine, legend}
\end{axis}
\end{tikzpicture}
```

They are actually just styles for commonly used alignment choices: the choice `left` is equivalent to `legend style={cells={anchor=west}}`; the second choice `right` is equivalent to `legend style={cells={anchor=east}}`, and `center` to `legend style={cells={anchor=center}}`. Using different values allows more control over cell alignment.

`/pgfplots/legend columns={number}`

(default 1)

Allows to configure the maximum number of adjacent legend entries. The default value 1 places legend entries vertically below each other.

Use `legend columns=-1` to draw all entries horizontally.

`/pgfplots/legend plot pos=left|right|none`

(initially left)

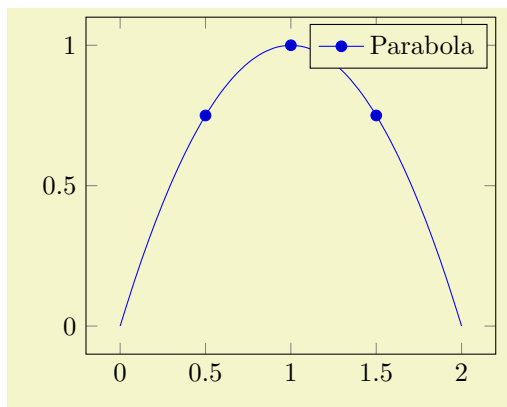
Configures where the small line specifications will be drawn: left of the description, right of the description or not at all.

`/pgfplots/every legend image post`

(style, no value)

A style which can be used to provide drawing options to every small legend image. These options apply after `current plot style` has been set, allowing users different line styles for legends than for plots.

For example, suppose you have a line plot and you plot selected markers on top of it (in the same color). Then, you may want to draw just a *single* legend entry (which should contain both the line *and* the markers). The following example shows a solution:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend image post style={mark=*}]
    \addplot+[only marks,forget plot]
      coordinates {(0.5,0.75) (1,1) (1.5,0.75)};
    \addplot+[mark=none,smooth,domain=0:2]
      {-x*(x-2)};
    \addlegendentry{Parabola}
  \end{axis}
\end{tikzpicture}
```

The example has two `\addplot` commands, one for the line and one for markers. Due to the `forget plot` option, the marker plot (the first one) doesn't advance the `cycle list`. The axis has only one legend entry, and since `legend image post style={mark=*}` has been used, the legend has a plot mark as well. Due to the `forget plot` option, the marker plot will not get a separate legend label.

`/pgfplots/legend image post style={⟨key-value-list⟩}`

An abbreviation for `every legend image post/.append style={⟨key-value-list⟩}`.

It appends options to the already existing style `every legend image post`.

`/pgfplots/legend image code/.code={⟨...⟩}`

Allows to replace the default images which are drawn inside of legends. When this key is evaluated, the current plot specification has already been activated (using `\begin{scope}[current plot style]`)⁴⁰, so any drawing operations use the same styles as the `\addplot` command.

The default is the style `line legend`.

Technical note: At the time when legend images are drawn, the style `every axis legend` is in effect – which have unwanted side-effects due to changed parameters (especially those concerning node placement, alignment, and shifting). It might be necessary to reset these parameters manually (PGFLOTS also attempts to reset the fill color).

`/pgfplots/line legend`

(style, no value)

A style which sets `legend image code` (back) to its initial value.

Its initial value is

```
\pgfplotsset{
  /pgfplots/line legend/.style={
    legend image code/.code={
      \draw[mark repeat=2,mark phase=2,##1]
        plot coordinates {
          (0cm,0cm)
          (0.3cm,0cm)
          (0.6cm,0cm)
        };
    }
  }
}
```

The style `line legend` can also be used to apply a different legend style to one particular plot (see the documentation on `area legend` for an example).

`/pgfplots/empty legend`

(style, no value)

A style which clears `legend image code`, thereby omitting the legend image.

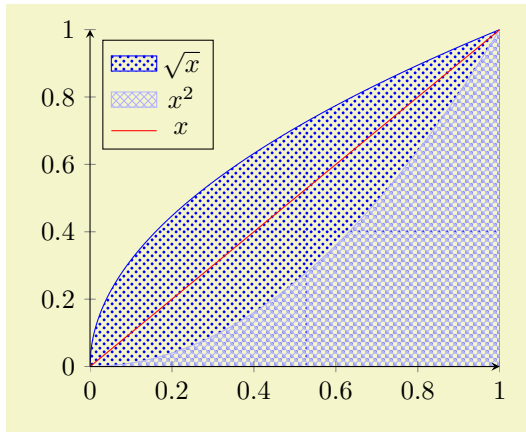
⁴⁰This was different in versions before 1.3. The new scope features allow plot styles to change `legend image code`.

/pgfplots/area legend

(style, no value)

A style which sets `legend image code` to

```
\pgfplotsset{
  legend image code/.code={%
    \draw[#1] (0cm,-0.1cm) rectangle (0.6cm,0.1cm);
  }
}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usetikzlibrary{patterns}
\begin{tikzpicture}
\begin{axis}[area legend,
  axis x line=bottom,
  axis y line=left,
  domain=0:1,
  legend style={at={(0.03,0.97)},
    anchor=north west},
  axis on top,xmin=0]
\addplot[pattern=crosshatch dots,
  pattern color=blue,draw=blue,
  samples=500]
{sqrt(x)} \closedcycle;

\addplot[pattern=crosshatch,
  pattern color=blue!30!white,
  draw=blue!30!white]
{x^2} \closedcycle;

\addplot[red,line legend] coordinates {(0,0) (1,1)};
\legend{$\sqrt{x}$,$x^2$,$x$}
\end{axis}
\end{tikzpicture}
```

/pgfplots/xbar legend

(no value)

/pgfplots/ybar legend

(no value)

/pgfplots/zbar legend

(no value)

/pgfplots/xbar interval legend

(no value)

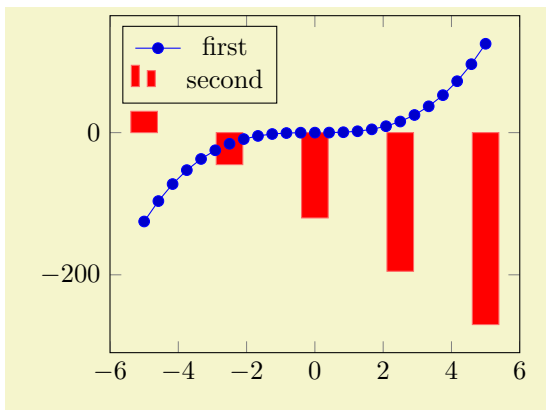
/pgfplots/ybar interval legend

(no value)

/pgfplots/zbar interval legend

(no value)

These style keys redefine `legend image code` such that legends use `xbar`, `ybar` or the `xbar interval` and `ybar interval` handlers.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[legend pos=north west]
\addplot {x^3};
\addplot[ybar,fill=red,draw=red!60,
  ybar legend,mark=none,samples=5]
{-30*(x +4)};
\legend{first,second}
\end{axis}
\end{tikzpicture}
```

The initial values for these styles might be interesting if someone wants to modify them. Here they are:

```

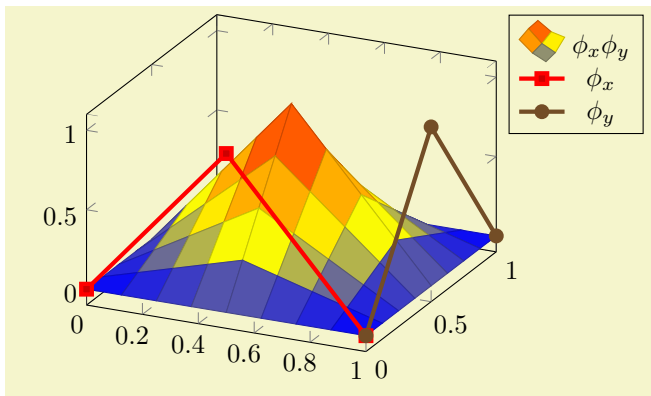
\pgfplotsset{
  /pgfplots/xbar legend/.style={
    /pgfplots/legend image code/.code={%
      \draw[##1,/tikz/.cd,bar width=3pt,yshift=-0.2em,bar shift=0pt]
        plot coordinates {(0cm,0.8em) (2*\pgfplotbarwidth,0.6em)};},
  },
  /pgfplots/ybar legend/.style={
    /pgfplots/legend image code/.code={%
      \draw[##1,/tikz/.cd,bar width=3pt,yshift=-0.2em,bar shift=0pt]
        plot coordinates {(0cm,0.8em) (2*\pgfplotbarwidth,0.6em)};},
  },
  /pgfplots/xbar interval legend/.style={%
    /pgfplots/legend image code/.code={%
      \draw[##1,/tikz/.cd,yshift=-0.2em,bar interval width=0.7,bar interval shift=0.5]
        plot coordinates {(0cm,0.8em) (5pt,0.6em) (10pt,0.6em)};},
  },
  /pgfplots/ybar interval legend/.style={
    /pgfplots/legend image code/.code={%
      \draw[##1,/tikz/.cd,yshift=-0.2em,bar interval width=0.7,bar interval shift=0.5]
        plot coordinates {(0cm,0.8em) (5pt,0.6em) (10pt,0.6em)};},
  },
}

```

/pgfplots/**mesh legend**

(no value)

Redefines **legend image code** such that it is compatible with **mesh** and **surf** plot handlers (for three dimensional visualization mainly).



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend pos=outer north east]
    \addplot3[surf,samples=9,domain=0:1]
      {(1-abs(2*(x-0.5))) * (1-abs(2*(y-0.5)))};
    \addlegendentry{\phi_x \phi_y}

    \addplot3+[ultra thick] coordinates {(0,0,0) (0.5,0,1) (1,0,0)};
    \addlegendentry{\phi_x}

    \addplot3+[ultra thick] coordinates {(1,0,0) (1,0.5,1) (1,1,0)};
    \addlegendentry{\phi_y}
  \end{axis}
\end{tikzpicture}

```

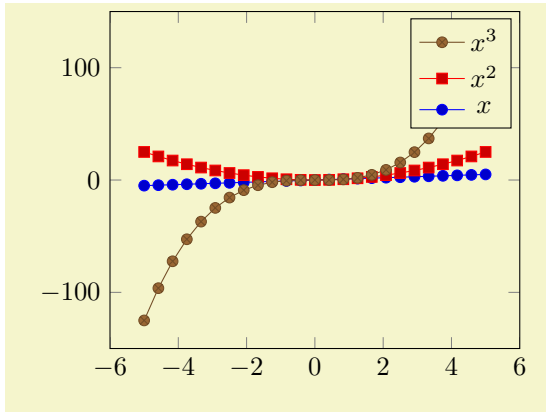
/pgfplots/**reverse legend**=true|false

(initially false)

/pgfplots/**legend reversed**=true|false

(initially false)

Allows to reverse the order in which the pairs (legend entry, plot style) are drawn.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[reverse legend]
    \addplot {x};
    \addlegendentry{$x$}
    \addplot {x^2};
    \addlegendentry{$x^2$}
    \addplot {x^3};
    \addlegendentry{$x^3$}
  \end{axis}
\end{tikzpicture}
```

/pgfplots/**transpose legend**=true|false

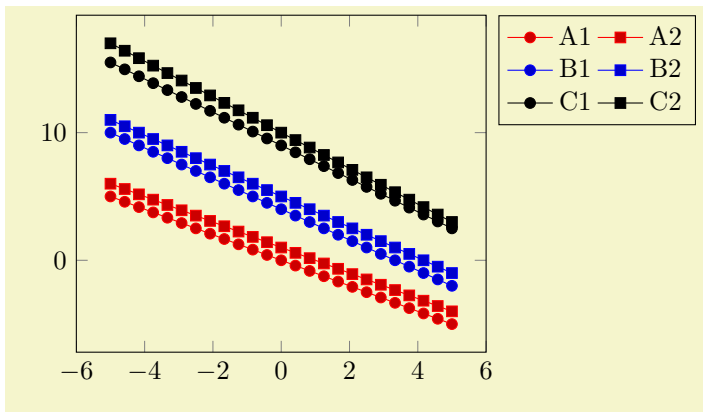
(initially false)

/pgfplots/**legend transposed**=true|false

(initially false)

Allows to transpose the order in which the pairs (legend entry, plot style) are drawn.

Consider a set of 3 experiments, each consisting of 2 parameters. We might want to draw them together as in the following example:

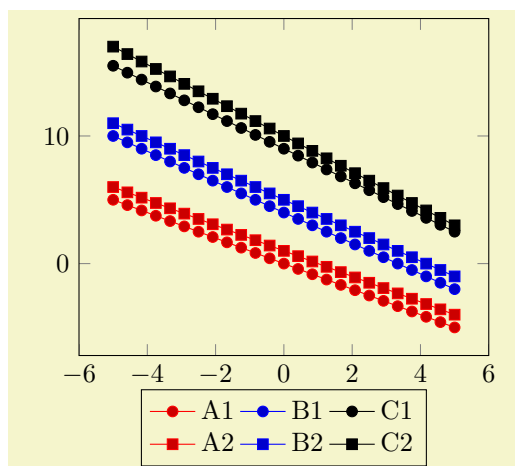


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    legend columns=2,
    legend pos=outer north east,
    cycle multi list={%
      color list\nextlist
      [2 of]mark list
    }
  ]
    \addplot {-x}; \addlegendentry{A1}
    \addplot {-x+1}; \addlegendentry{A2}

    \addplot {-1.2*x + 4}; \addlegendentry{B1}
    \addplot {-1.2*x + 5}; \addlegendentry{B2}

    \addplot {-1.3*x + 9}; \addlegendentry{C1}
    \addplot {-1.4*x + 10}; \addlegendentry{C2}
  \end{axis}
\end{tikzpicture}
```

An alternative might be to draw them horizontally – then, we'd like to use **transpose legend** to get a flat legend:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    transpose legend,
    legend columns=2,
    legend style={at={(0.5,-0.1)},anchor=north},
    cycle multi list={%
      color list\nextlist
      [2 of]mark list
    }
  ]
  \addplot {-x}; \addlegendentry{A1}
  \addplot {-x+1}; \addlegendentry{A2}

  \addplot {-1.2*x + 4}; \addlegendentry{B1}
  \addplot {-1.2*x + 5}; \addlegendentry{B2}

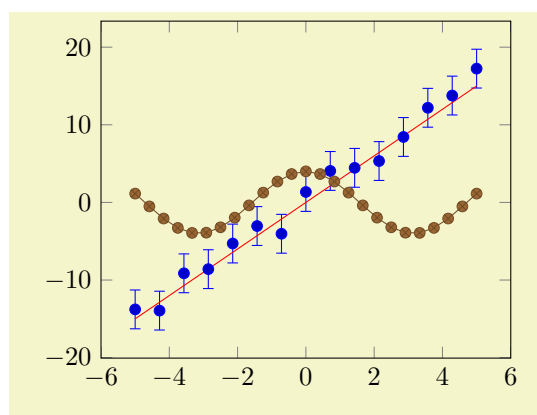
  \addplot {-1.3*x + 9}; \addlegendentry{C1}
  \addplot {-1.4*x + 10}; \addlegendentry{C2}
  \end{axis}
\end{tikzpicture}
```

Thus, `legend columns` defines the *input* columns, before the transposition (in other words, `legend columns` indicates the *rows* of the resulting legend).

Transposing legends has only an effect if `legend columns` > 1. Note that `reverse legend` has higher precedence: it is applied first.

4.8.6 Legends with `\label` and `\ref`

PGFLOTS offers a `\label` and `\ref` feature for L^AT_EX to assemble a legend manually, for example as part of the figure caption. These references work as usual L^AT_EX references: a `\label` remembers where and what needs to be referenced and a `\ref` expands to proper text. In context of plots, a `\label` remembers the plot specification of one plot and a `\ref` expands to the small image which would also be used inside of legends.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}[baseline]
\begin{axis}
  \addplot+[only marks,
    samples=15,
    error bars/y dir=both,
    error bars/y fixed=2.5]
    {3*x+2.5*rand};
  \label{pgfplots:label1}

  \addplot+[mark=none] {3*x};
  \label{pgfplots:label2}

  \addplot {4*cos(deg(x))};
  \label{pgfplots:label3}
\end{axis}
\end{tikzpicture}
```

The picture shows the estimations `\ref{pgfplots:label1}` which are subjected to noise. It appears the model `\ref{pgfplots:label2}` fits the data appropriately. Finally, `\ref{pgfplots:label3}` is only here to get three examples.

The picture shows the estimations $\bullet$ which are subjected to noise. It appears the model --- fits the data appropriately. Finally, $\text{---}\bullet\text{---}$ is only here to get three examples.

`\label{<label name>}`

`\label[<reference>]{<label name>}`

When used after `\addplot`, this command creates a L^AT_EX label named `<label name>`⁴¹. If this label is cross-referenced with `\ref{<label name>}` somewhere, the associated plot specification will be inserted.

Label3 = $\text{---}\bullet\text{---}$; Label2 = ---

Label3 = `\ref{pgfplots:label3}`;
Label2 = `\ref{pgfplots:label2}`

⁴¹This feature is *only* available in L^AT_EX, sorry.

The label is assembled using `legend image code` and the plot style of the last plot. Any PGFPLOTS option is expanded until only TikZ (or PGF) options remain; these options are used to get an independent label.

More precisely, the small image generated by `\ref{\label name}` is

```
\tikz[/pgfplots/every crossref picture] {...}
```

where the contents is determined by `legend image code` and the plot style.

The second syntax, `\label[reference]{\label name}` allows to label particular pieces of an `\addplot` command. It is (currently) only interesting for `scatter/classes`: there, it allows to reference particular classes of the scatter plot. See page 81 for more details.

Note that `\label` information, even the small TikZ pictures here, can be combined with the `external` library for image externalization, see Section 7.1 for details (in particular, the `external/mode` key). In other words, references remain valid even if the defining axis has been externalized.

`\ref{\label name}`

Can be used to reference a labeled, single plot. See the example above.

This will also work together with `hyperref` links and `\pageref`⁴².

`/pgfplots/refstyle={\label name}`

Can be used to set the *styles* of a labeled, single plot. This allows to write

```
\addplot[/pgfplots/refstyle={pgfplots:label2}]
```

somewhere. Please note that it may be easier to define a style with `.style`.

`/pgfplots/every crossref picture`

(style, no value)

A style which will be used by the cross-referencing feature for plots. The default is

```
\pgfplotsset{every crossref picture/.style={baseline,yshift=0.3em}}
```

`/pgfplots/invoke before crossref tikzpicture={\TeX code}`

`/pgfplots/invoke after crossref tikzpicture={\TeX code}`

Code which is invoked just before or just after every cross reference picture. This applies to legend images generated with `\ref`, `legend to name` and `colorbar to name` images.

The initial configuration checks if the `external` library is in effect. If so, it modifies the generated figure names by means of `\tikzappendtofigurename{\_crossref}`.

4.8.7 Legends Outside Of an Axis

Occasionally, one has multiple adjacent plots, each with the same legend – and just *one* legend suffices. But where shall it be placed? And how? One solution is to use the `overlay` key to exclude the legend from bounding box computations, and place it absolutely such that it fits. Another is the `legend to name` feature:

`/pgfplots/legend to name={\name}`

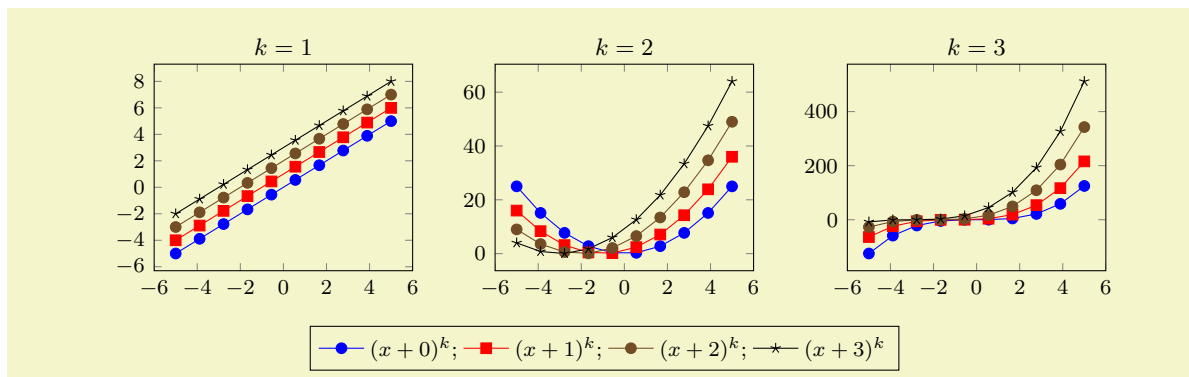
(initially empty)

Enables a legend export mode: instead of drawing the legend, a self-contained, independent set of drawing commands will be stored using the label `\name`. The definition is done using `\label{\name}`, just like any other L^AT_EX label. The name can be referenced using

`\ref{\name}`.

Thus, typing `\ref{\name}` somewhere outside of the axis, maybe even outside of any picture, will cause the legend to be drawn.

⁴²Older versions of PGFPLOTS required the use of `\protect\ref` when used inside of captions or section headings. This is no longer necessary.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{footnotesize,samples=10}
\begin{center}% note that \centering uses less vspace...
\begin{tikzpicture}
  \begin{axis}[
    legend columns=-1,
    legend entries={$(x+0)^k$,$(x+1)^k$,$(x+2)^k$,$(x+3)^k$},
    legend to name=named,
    title={k=1}]
    \addplot {x};
    \addplot {(x+1)};
    \addplot {(x+2)};
    \addplot {(x+3)};
  \end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
  \begin{axis}[title={k=2}]
    \addplot {x^2};
    \addplot {(x+1)^2};
    \addplot {(x+2)^2};
    \addplot {(x+3)^2};
  \end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
  \begin{axis}[title={k=3}]
    \addplot {x^3};
    \addplot {(x+1)^3};
    \addplot {(x+2)^3};
    \addplot {(x+3)^3};
  \end{axis}
\end{tikzpicture}
\\
\ref{named}
\end{center}

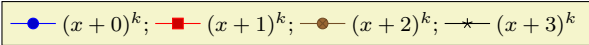
```

Note that only the *first* plot has `legend entries`. Thus, its legend will be created as usual, and stored under the name ‘named’, but it won’t be drawn. The stored legend can then be drawn with `\ref{named}` below the three plots. Since there is no picture in this context, a `\tikz` picture is created and a `\matrix[/pgfplots/every axis legend]` path is drawn inside of it, resulting in the legend as if it had been placed inside of the axis.

The stored legend will contain the currently active values of legend- and plot style related options. This includes `legend image code`, `every axis legend`, and any plot style options (and some more). The algorithm works in the same way as for `\label` and `\ref`, i.e. it keeps any options with `/tikz/` prefix and expands those with `/pgfplots/` prefix.

Note that the legend is drawn with `every axis legend`, even though the placement options might be chosen to fit into an axis. You may want to adjust the style in the same axis in which the stored legend has been defined (the value will be copied and restored as well).

About `\ref{<name>}` The `\ref{<name>}` command retrieves a stored legend (one defined by `legend to name`) and draws it.

`\ref{named}`: 

If you want the legend to be exported *and* drawn inside of the current axis, consider using `extra description/.append code={\ref{<name>}}`.

Note that `\ref` can be combined with the `external` library for image externalization. In other words, the legend will work even if the defining axis has been externalized, see Section 7.1 for details (in particular the `external/mode` key).

Note furthermore that this `.aux` file related stuff is (currently) only supported, if PGFPLOTS is run by means of L^AT_EX, sorry.

`\pgfplotslegendfromname{<name>}`

This command poses an equivalent alternative for `\ref{<name>}`: it has essentially the same effect, but it does not create links when used with the `hyperref` package⁴³.

`/pgfplots/every legend to name picture` (style, no value)

A style which is installed when `\ref` is used outside of a picture: a new picture will be created with `\tikz[/pgfplots/every legend to name picture]`.

Thus, you can redefine this style to set alignment options (such as `baseline`). For example, the initialization

```
\pgfplotsset{
  legend style={matrix anchor=west,at={(0pt,0pt)}},
  every legend to name picture/.style={baseline},
}
```

will cause the legend to be positioned such that its `west` anchor is at `y=0pt`. The `baseline` option will align this point of the legend with the text baseline (please refer to the documentation for `baseline` in Section 4.18 for details).

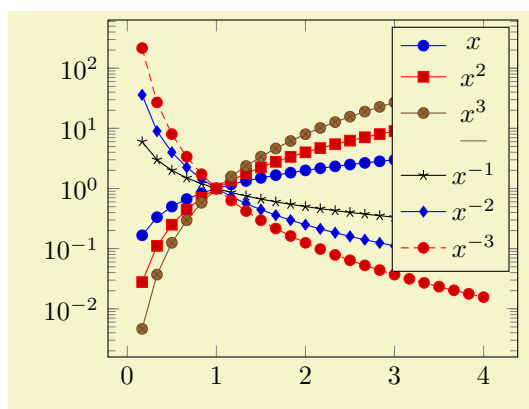
4.8.8 Legends with Customized Texts or Multiple Lines

`\addlegendimage{<options>}`

Adds a further legend image for legend creation.

Each `\addplot` command appends its plot style options to a list, and `\addlegendimage` adds `<options>` to the very same list.

Thus, the effect is as if you had provided `\addplot[<options>]`, but `\addlegendimage` bypasses all the logic usually associated with a plot. In other words: except for the legend, the state of the axis remains as if the command would not have been issued. Not even the current plot's index is advanced.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  domain=0:4,
]
  \addplot {x}; \addlegendentry{$x$}
  \addplot {x^2}; \addlegendentry{$x^2$}
  \addplot {x^3}; \addlegendentry{$x^3$}
  \addlegendimage{empty legend}
  \addlegendentry{---}
  \addplot {x^(-1)}; \addlegendentry{$x^{-1}$}
  \addplot {x^(-2)}; \addlegendentry{$x^{-2}$}
  \addplot {x^(-3)}; \addlegendentry{$x^{-3}$}
\end{semilogyaxis}
\end{tikzpicture}
```

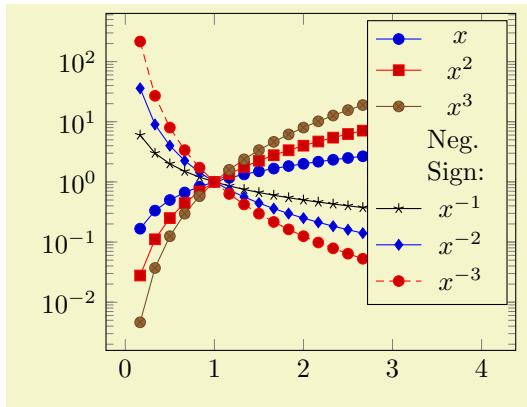
The example above has six plots, each with its legend entry. Furthermore, it has an `\addlegendimage` command and its separate legend entry. We see that `\addlegendimage` needs its own legend entry, but it is detached from the processing of plots as such. In our case, we chose `empty legend` as style for the separator.

⁴³Since this manual uses colored links, the text in `\ref` would usually be blue. Using `\pgfplotslegendfromname` avoids link text colors in the legend (this has been applied to the manual styles here).

Use `\addlegendimage` to provide custom styles into legends, for example to document custom `\draw` commands inside of an axis.

You can call `\label` after `\addlegendimage` just as for a normal style.

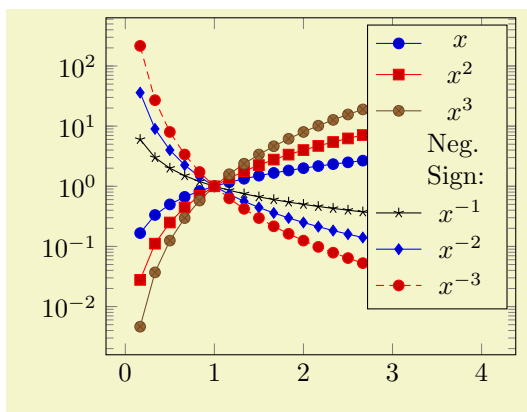
Occasionally, one may want multiple lines for legend entries. That is possible as well using a fixed `text width`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  domain=0:4,
]
  \addplot {x}; \addlegendentry{$x$}
  \addplot {x^2}; \addlegendentry{$x^2$}
  \addplot {x^3}; \addlegendentry{$x^3$}
  \addlegendimage{empty legend}
  \addlegendentry[text width=25pt,text depth=]
    {Neg. Sign:}
  \addplot {x^(-1)}; \addlegendentry{$x^{-1}$}
  \addplot {x^(-2)}; \addlegendentry{$x^{-2}$}
  \addplot {x^(-3)}; \addlegendentry{$x^{-3}$}
\end{semilogyaxis}
\end{tikzpicture}
```

The example provides options for the single multiline element. Note that the initial configuration of `legend style` employs `text depth=0.15em`, which needs to be reset manually to `text depth={}`⁴⁴.

There are two approaches with the same effect which are subject of the following example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  domain=0:4,
  legend entries={%
    $x$, $x^2$, $x^3$, %
    {[text width=25pt,text depth=]Neg. Sign:}, %
    $x^{-1}$, $x^{-2}$, $x^{-3}$}, %
  % same effect:
  % legend style={
  %   nodes={text width=25pt,text depth=}
  % }
]
  \addplot {x};
  \addplot {x^2};
  \addplot {x^3};
  \addlegendimage{empty legend}
  \addplot {x^(-1)};
  \addplot {x^(-2)};
  \addplot {x^(-3)};
\end{semilogyaxis}
\end{tikzpicture}
```

Here, the `legend entries` are provided using the single key syntax. Note that the special options are provided as part of the legend entry, using square brackets right before the text as such. The comments indicate that you could also add the `text width` stuff to `legend style`, in which case it would hold for every node.

Note that legend texts are realized using `\node[options] {text}`; so anything which produces a valid TikZ node is permitted (this includes `minipage` or `tabular` environments inside of `text`).

4.8.9 Axis Lines

An extension by Pascal Wolkotte

By default the axis lines are drawn as a box, but it is possible to change the appearance of the x and y axis lines.

```
/pgfplots/axis x line=box|top|middle|center|bottom|none (initially box)
/pgfplots/axis x line*=box|top|middle|center|bottom|none (initially box)
/pgfplots/axis y line=box|left|middle|center|right|none (initially box)
```

⁴⁴Perhaps I can reset `text depth` automatically in the future.

```
/pgfplots/axis y line*=box|left|middle|center|right|none (initially box)
/pgfplots/axis lines=box|left|middle|center|right|none
/pgfplots/axis lines*=box|left|middle|center|right|none
```

These keys allow to choose the locations of the axis lines. The last one, `axis lines` sets the same value for every axis.

Ticks and tick labels are placed according to the chosen value as well. The choice `bottom` will draw the x line at $y = y_{\min}$, `middle` will draw the x line at $y = 0$, and `top` will draw it at $y = y_{\max}$. Finally, `box` is a combination of options `top` and `bottom`. The choice `axis x line=none` is an alias for `hide x axis`. The y - and z variants work in a similar way.

The case `center` is a synonym for `middle`, both draw the line through the respective coordinate 0. If this coordinate is not part of the axis limit, the lower axis limit is chosen instead.

The starred versions `...line*` *only* affect the axis lines, without correcting the positions of axis labels, tick lines or other keys which are (possibly) affected by a changed axis line. The non-starred versions are actually styles which set the starred key *and* some other keys which also affect the figure layout:

- In case `axis x line=box`, the style `every boxed x axis` will be installed immediately.
- In case `axis x line≠box`, the style `every non boxed x axis` will be installed immediately. Furthermore, some of these choices will modify axis label positions.

The handling of `axis y line` and `axis z line` is similar. The default styles are defined as

```
\pgfplotsset{
  every non boxed x axis/.style={
    xtick align=center,
    enlarge x limits=false,
    x axis line style={-stealth}
  },
  every boxed x axis/.style={}
}
```

In addition, conditional modifications of axis label styles will be taken. For example, `axis x line=middle` will set

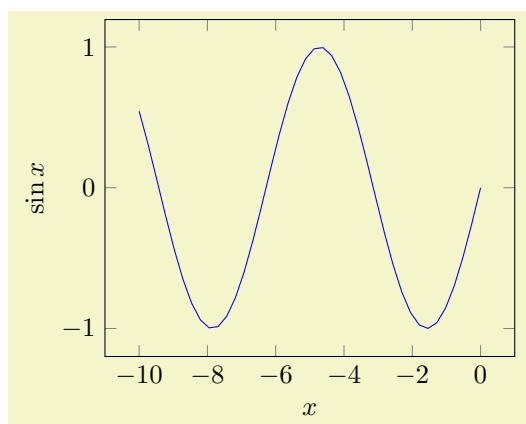
```
\pgfplotsset{every axis x label/.style={at={(current axis.left of origin)},anchor=south west}}
```

if the matching y style has value `axis y line=right` and

```
\pgfplotsset{every axis x label/.style={at={(current axis.right of origin)},anchor=south east}}
```

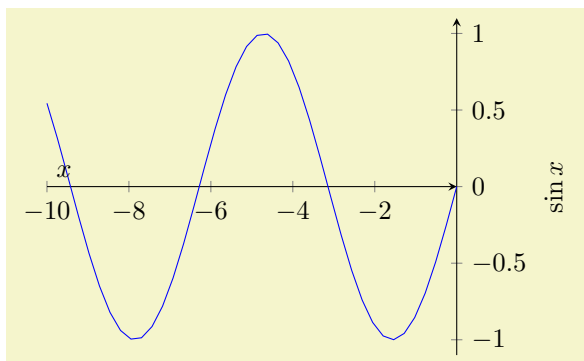
if `axis y line≠right`.

Feel free to overwrite these styles if the default doesn't fit your needs or taste. Again, these styles will *not* be used for `axis line*`.

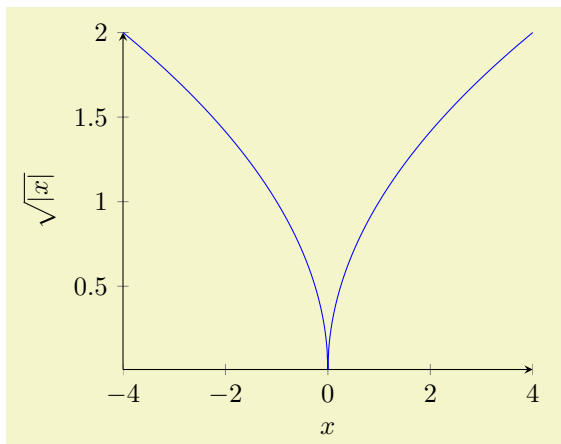


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xlabel=$x$,ylabel=$\sin x$

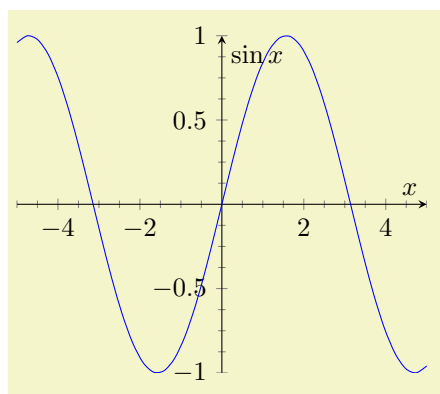
  \addplot[blue,mark=none,
    domain=-10:0,samples=40]
    {sin(deg(x))};
\end{axis}
\end{tikzpicture}
```



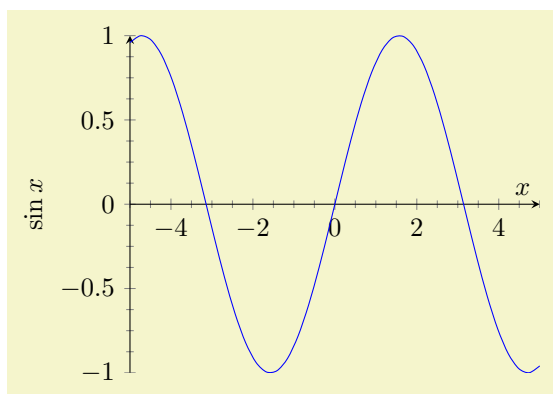
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  axis x line=middle,
  axis y line=right,
  ymax=1.1, ymin=-1.1,
  xlabel=$x$,ylabel=$\sin x$
]
  \addplot[blue,mark=none,
    domain=-10:0,samples=40]
    {sin(deg(x))};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  axis x line=bottom,
  axis y line=left,
  xlabel=$x$,ylabel=$\sqrt{|x|}$
]
  \addplot[blue,mark=none,
    domain=-4:4,samples=501]
    {sqrt(abs(x))};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  minor tick num=3,
  axis y line=center,
  axis x line=middle,
  xlabel=$x$,ylabel=$\sin x$
]
  \addplot[smooth,blue,mark=none,
    domain=-5:5,samples=40]
    {sin(deg(x))};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  minor tick num=3,
  axis y line=left,
  axis x line=middle,
  xlabel=$x$,ylabel=$\sin x$
]
  \addplot[smooth,blue,mark=none,
    domain=-5:5,samples=40]
    {sin(deg(x))};
\end{axis}
\end{tikzpicture}
```

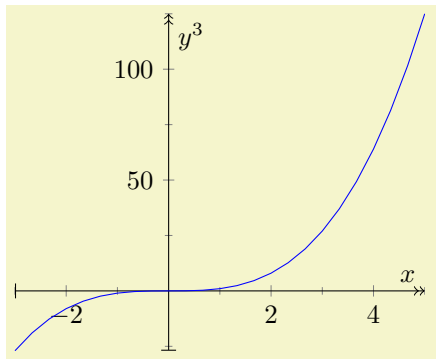
In case `middle`, the style `every inner axis x line` allows to adjust the appearance.

Note that three dimensional axes only support to use the same value for every axis, i.e. three dimensional axes support only the `axis lines` key (or, preferably for 3D axes, the `axis lines*` key – check what looks best). See Section 4.10.4 for examples of three dimensional axis line variations.

`/pgfplots/every inner x axis line` (no value)
`/pgfplots/every inner y axis line` (no value)
`/pgfplots/every inner z axis line` (no value)

A style key which can be redefined to customize the appearance of *inner* axis lines. Inner axis lines are those drawn by the `middle` (or `center`) choice of `axis x line`, see above.

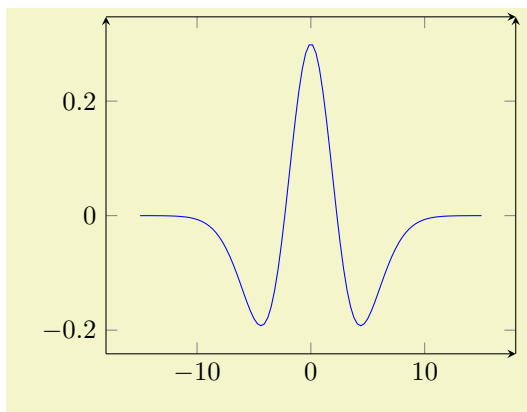
This style affects *only* the line as such.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  minor tick num=1,
  axis x line=middle,
  axis y line=middle,
  every inner x axis line/.append style=
    {|->>},
  every inner y axis line/.append style=
    {|->>},
  xlabel=$x$,ylabel=$y^3$
]
\addplot[blue,domain=-3:5] {x^3};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/every outer x axis line` (no value)
`/pgfplots/every outer y axis line` (no value)
`/pgfplots/every outer z axis line` (no value)

Similar to `every inner x axis line`, this style configures the appearance of all axis lines which are part of the outer box.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  separate axis lines, % important !
  every outer x axis line/.append style=
    {-stealth},
  every outer y axis line/.append style=
    {-stealth},
]
\addplot[blue,id=DoG,
  samples=100,
  domain=-15:15]
  gnuplot{1.3*exp(-x**2/10) - exp(-x**2/20)};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/axis line style={⟨key-value-list⟩}`

A command which appends `⟨key-value-list⟩` to *all* axis line appearance styles.

`/pgfplots/inner axis line style={⟨key-value-list⟩}`

A command which appends `⟨key-value-list⟩` to both, `every inner x axis line` and the `y` variant.

`/pgfplots/outer axis line style={⟨key-value-list⟩}`

A command which appends `⟨key-value-list⟩` to both, `every outer x axis line` and the `y` variant.

`/pgfplots/x axis line style={⟨key-value-list⟩}`

`/pgfplots/y axis line style={⟨key-value-list⟩}`

`/pgfplots/z axis line style={⟨key-value-list⟩}`

A command which appends `⟨key-value-list⟩` to all axis lines styles for either `x` or `y` axis.

`/pgfplots/every boxed x axis` (no value)
`/pgfplots/every boxed y axis` (no value)

`/pgfplots/every boxed z axis`

(no value)

A style which will be installed as soon as `axis x line=box (y)` is set.

The default is simply empty.

`/pgfplots/every non boxed x axis`

(no value)

`/pgfplots/every non boxed y axis`

(no value)

`/pgfplots/every non boxed z axis`

(no value)

A style which will be installed as soon as `axis x line (y)` will be set to something different than `box`.

The default is

```
\pgfplotsset{
  every non boxed x axis/.style={
    xtick align=center,
    enlarge x limits=false,
    x axis line style={-stealth}}}
```

with similar values for the `y`-variant. Feel free to redefine this style to your needs and taste.

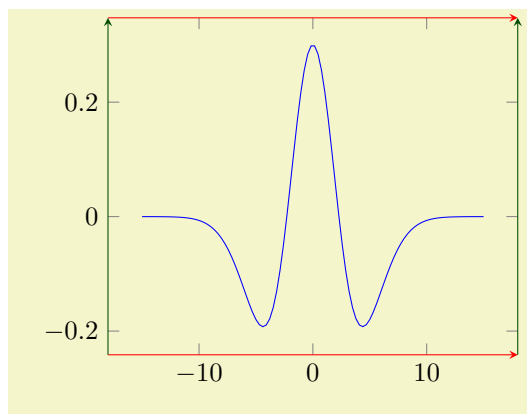
`/pgfplots/separate axis lines={\langle true,false \rangle}`

(default `true`)

Enables or disables separate path commands for every axis line. This option affects *only* the case if axis lines are drawn as a *box*.

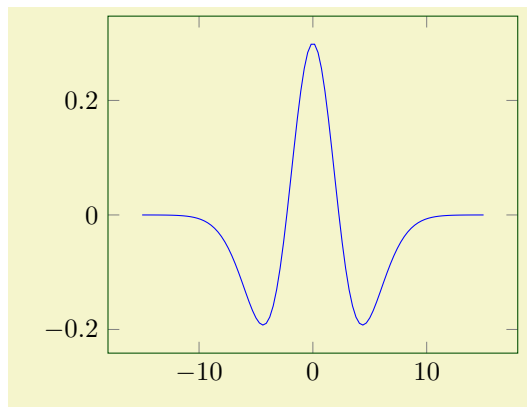
Both cases have their advantages and disadvantages, I fear there is no reasonable default (suggestions are welcome).

The case `separate axis lines=true` allows to draw arrow heads on each single axis line, but it can't close edges very well – in case of thick lines, unsatisfactory edges occur.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  separate axis lines,
  every outer x axis line/.append style={-stealth,red},
  every outer y axis line/.append style={-stealth,green!30!black},
]
\addplot[blue,
  samples=100,
  domain=-15:15]
{1.3*exp(0-x^2/10) - exp(0-x^2/20)};
% Unfortunately, there is a bug in PGF 2.00
% something like exp(-10^2)
% must be written as exp(0-10^2) :-(
\end{axis}
\end{tikzpicture}
```

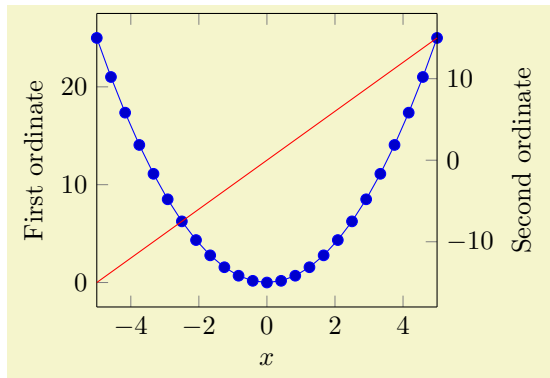
The case `separate axis lines=false` issues just *one* path for all axis lines. It draws a kind of rectangle, where some parts of the rectangle may be skipped over if they are not wanted. The advantage is that edges are closed properly. The disadvantage is that at most one arrow head is added to the path (and yes, only one drawing color is possible).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  separate axis lines=false,
  every outer x axis line/.append style={-stealth,red},
  every outer y axis line/.append style={-stealth,green!30!black},
]
\addplot[blue,id=DoG,
  samples=100,
  domain=-15:15]
{gnuplot{1.3*exp(-x**2/10) - exp(-x**2/20)}};
\end{axis}
\end{tikzpicture}
```

4.8.10 Two Ordinates (y axis) or Multiple Axes

In some applications, more than one y axis is used if the x range is the same. This section demonstrates how to create them. The idea in PGFPLOTS is to draw two axes on top of each other, one with descriptions only on the left and the second with descriptions only on the right:



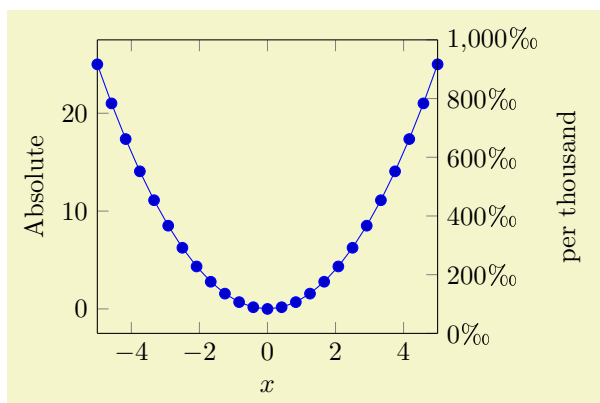
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
% let both axes use the same layers
\pgfplotsset{set layers}
\begin{axis}[
  scale only axis,
  xmin=-5,xmax=5,
  axis y line*=left,% the '*' avoids arrow heads
  xlabel=$x$,
  ylabel=First ordinate]
\addplot {x^2};
\end{axis}

\begin{axis}[
  scale only axis,
  xmin=-5,xmax=5,
  axis y line*=right,
  axis x line=none,
  ylabel=Second ordinate]
\addplot[red] {3*x};
\end{axis}
\end{tikzpicture}
```

Thus, the two axes are drawn “on top” of each other – one, which contains the x axis and the left y axis, and one which has *only* the right y axis. Since PGFPLOTS does not really know what it’s doing here, user attention in the following possibly non-obvious aspects is required:

1. Scaling. You should set `scale only axis` because this forces equal dimensions for both axis, without respecting any labels.
2. Same x limits. You should set those limits explicitly.
3. You need to tell PGFPLOTS that it should share the same graphics layers for both axes. In this case, PGFPLOTS will draw plots of the first axis and of the second axis onto the same layer. It will also draw background(s) into the background layer and descriptions into the foreground layer. Use the key `\pgfplotsset{set layers}` in front of the first axis to prepare the complete picture for layered graphics.

You may want to consider different legend styles. It is also possible to use only the axis, without any plots:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepackage{textcomp}
\begin{tikzpicture}
% let both axes use the same layers
\pgfplotsset{set layers}
\begin{axis}[
  scale only axis,
  xmin=-5,xmax=5,
  axis y line*=left,% '*' avoids arrow heads
  xlabel=$x$,
  ylabel=Absolute]
\addplot {x^2};
\end{axis}

\begin{axis}[
  scale only axis,
  xmin=-5,xmax=5,
  ymin=0,ymax=1000,
  yticklabel={
    {\pgfmathprintnumber{\tick}}\textperthousand},
  axis y line*=right,
  axis x line=none,
  ylabel=per thousand]
\end{axis}
\end{tikzpicture}
```

4.8.11 Axis Discontinuities

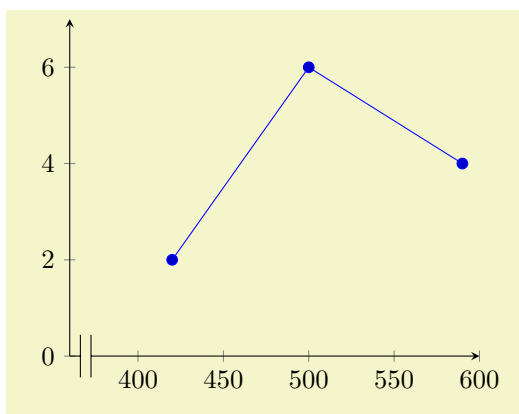
An extension by Pascal Wolkotte

In case the range of either of the axis do not include the zero value, it is possible to visualize this with a discontinuity decoration on the corresponding axis line.

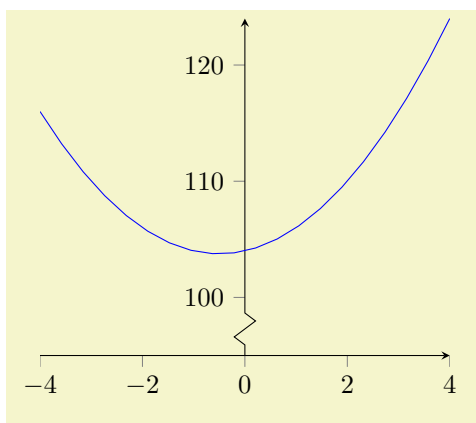
```
/pgfplots/axis x discontinuity=crunch|parallel|none (initially none)
/pgfplots/axis y discontinuity=crunch|parallel|none (initially none)
/pgfplots/axis z discontinuity=crunch|parallel|none (initially none)
```

Insert a discontinuity decoration on the x (or y , respectively) axis. This is to visualize that the y axis does cross the x axis at its 0 value, because the minimum x axis value is positive or the maximum value is negative.

The description applies to `axis y discontinuity` and `axis z discontinuity` as well, simply substitute x by y or z , respectively.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  axis x line=bottom,
  axis x discontinuity=parallel,
  axis y line=left,
  xmin=360, xmax=600,
  ymin=0, ymax=7,
  enlargelimits=false
]
\addplot coordinates {
  (420,2)
  (500,6)
  (590,4)
};
\end{axis}
\end{tikzpicture}
```

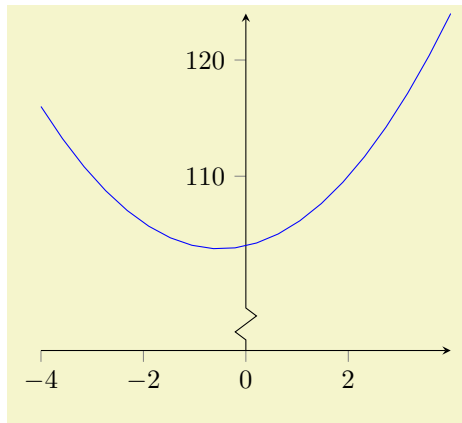


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  axis x line=bottom,
  axis y line=center,
  tick align=outside,
  axis y discontinuity=crunch,
  ymin=95, enlargelimits=false
]
\addplot[blue,mark=none,
  domain=-4:4,samples=20]
{x*x+x+104};
\end{axis}
\end{tikzpicture}
```

A problem might occur with the placement of the ticks on the axis. This can be solved by specifying the minimum or maximum axis value for which a tick will be placed.

```
/pgfplots/xtickmin={⟨coord⟩} (default axis limits)
/pgfplots/ytickmin={⟨coord⟩} (default axis limits)
/pgfplots/ztickmin={⟨coord⟩} (default axis limits)
/pgfplots/xtickmax={⟨coord⟩} (default axis limits)
/pgfplots/ytickmax={⟨coord⟩} (default axis limits)
/pgfplots/ztickmax={⟨coord⟩} (default axis limits)
```

The options `xtickmin`, `xtickmax` and `ytickmin`, `ytickmax` allow to define the axis tick limits, i.e. the axis values before respectively after no ticks will be placed. Everything outside of the axis tick limits will be not drawn. Their default values are equal to the axis limits.

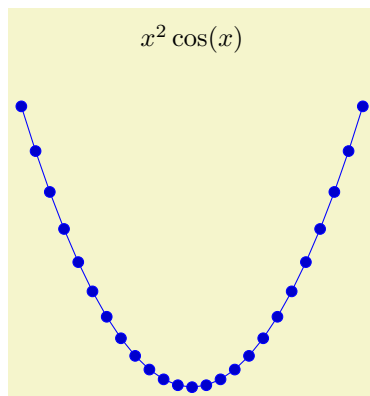


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  axis x line=bottom,
  axis y line=center,
  tick align=outside,
  axis y discontinuity=crunch,
  xtickmax=3,
  ytickmin=110,
  ymin=95, enlargelimits=false
]
  \addplot[blue,mark=none,
    domain=-4:4,samples=20]
    {x*x+x+104};
\end{axis}
\end{tikzpicture}
```

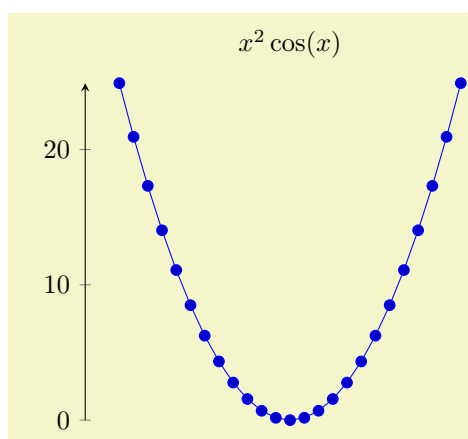
```
/pgfplots/hide x axis=true|false (initially false)
/pgfplots/hide y axis=true|false (initially false)
/pgfplots/hide z axis=true|false (initially false)
/pgfplots/hide axis=true|false (initially false)
```

Allows to hide either a selected axis or all of them. No outer rectangle, no tick marks and no labels will be drawn. Only titles and legends will be processed as usual.

Axis scaling and clipping (!) will be done as if you did not use `hide axis`.



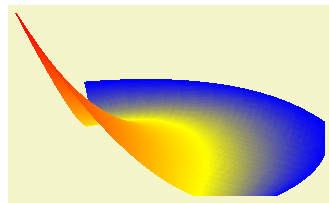
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  hide x axis,
  hide y axis,
  title={$x^2\cos(x)$}
  \addplot {\cos(x)*x^2};
\end{axis}
\end{tikzpicture}% <- eliminate space.
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  hide x axis,
  axis y line=left,
  title={$x^2\cos(x)$}
  \addplot {\cos(x)*x^2};
\end{axis}
\end{tikzpicture}% <- eliminate space
```

Note that a hidden axis contributes nothing to the resulting picture's bounding box⁴⁵, see `clip has bounding box`.

⁴⁵Since version 1.8.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepgfplotslibrary{patchplots}
% and compat=1.8 or newer
\begin{tikzpicture}
\begin{axis}[
  hide axis,
]
\addplot3[patch,
  patch type=biquadratic,
  shader=interp]
coordinates {
  (0,0,1) (6,1,0) (5,5,0) (-1,5,0)
  (3,1,0) (6,3,0) (2,6,0) (0,3,0)
  (3,3.75,0)
};
\end{axis}
\end{tikzpicture}% <- eliminate space
```

This can be used to embed a PGFLOTS path which needs an axis to a standard TikZ picture. See also Section 4.26 for details how to synchronize the alignment between a PGFLOTS figure (which typically rescales its coordinates) to that of a standard `tikzpicture`.

You may want to disable the `clip` path using the option `clip=false`.

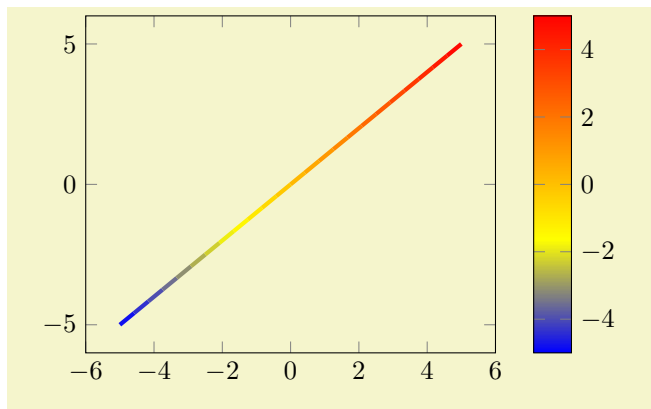
4.8.12 Color Bars

PGFLOTS supports mesh, surface and scatter plots which can use color maps. While color maps can be chosen as described in Section 4.6.6, they can be visualized using color bars.

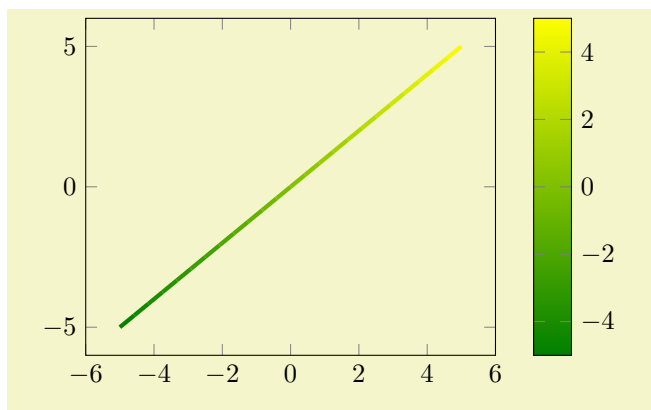
`/pgfplots/colorbar=true|false`

(initially false)

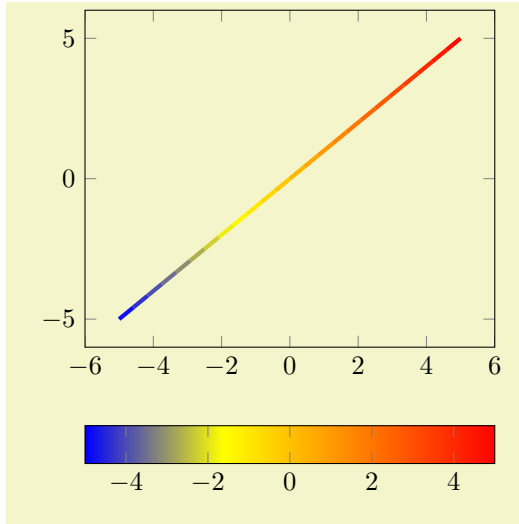
Activates or deactivates color bars.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[colorbar]
\addplot[mesh,ultra thick] {x};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar,colormap/greenyellow]
    \addplot[mesh,ultra thick] {x};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar horizontal]
    \addplot[mesh,ultra thick] {x};
  \end{axis}
\end{tikzpicture}
```

A color bar is only useful for plots with non-zero color data range, more precisely, for which minimum and maximum `point meta` data is available. Usually, this is the case for `scatter`, `mesh` or `surf` (or similar) plots, but you can also set `point meta min` and `point meta max` manually in order to draw a `colorbar`.

Color bars are just normal axes which are placed right besides their parent axes. The only difference is that they inherit several styles such as line width and fonts and they contain a bar shaded with the color map of the current axis.

Color bars are drawn internally with

```
\axis[every colorbar,colorbar shift,colorbar=false]
  \addplot graphics {};
\endaxis
```

where the placement, alignment, appearance and other options are done by the two styles `every colorbar` and `colorbar shift`. These styles and the possible placement and alignment options are described below.

Remarks for special cases:

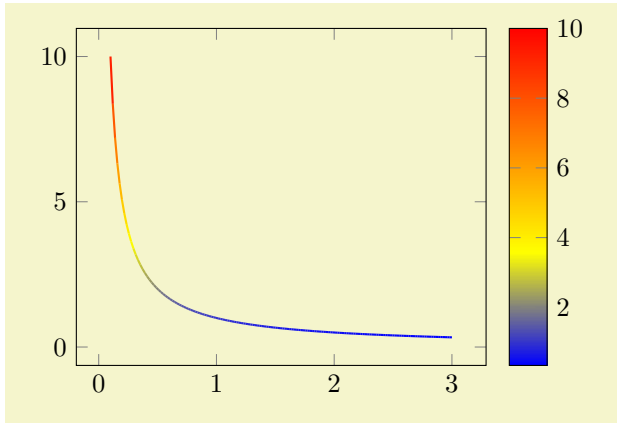
- Since there is always only one color bar per plot, this color bar uses the axis wide configurations of color map and color data. Consider using `colorbar source` to select color data limits of a particular `\addplot` command instead.
- If someone needs more than one color bar, the draw command above needs to be updated. See the key `colorbar/draw/.code` for this special case.

`/pgfplots/colorbar right`

(style, no value)

A style which redefines `every colorbar` and `colorbar shift` such that color bars are placed right of their parent axis.

This is the initial configuration.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar right]
    \addplot[mesh,thick,samples=150,domain=0.1:3]
      {1/x};
  \end{axis}
\end{tikzpicture}
```

The style `colorbar right` is defined as

```
\pgfplotsset{
  colorbar right/.style={
    /pgfplots/colorbar=true,
    /pgfplots/colorbar shift/.style={xshift=0.3cm},
    /pgfplots/every colorbar/.style={
      title=,
      xlabel=,
      ylabel=,
      zlabel=,
      legend entries=,
      axis on top,
      at={(parent axis.right of north east)},
      anchor=north west,
      xmin=0,
      xmax=1,
      ymin=\pgfkeysvalueof{/pgfplots/point meta min},
      ymax=\pgfkeysvalueof{/pgfplots/point meta max},
      plot graphics/xmin=0,
      plot graphics/xmax=1,
      plot graphics/ymin=\pgfkeysvalueof{/pgfplots/point meta min},
      plot graphics/ymax=\pgfkeysvalueof{/pgfplots/point meta max},
      enlargelimits=false,
      scale only axis,
      height=\pgfkeysvalueof{/pgfplots/parent axis height},
      x=\pgfkeysvalueof{/pgfplots/colorbar/width},
      yticklabel pos=right,
      xtick=\empty,
      colorbar vertical/lowlevel,
    }
  },
  /pgfplots/colorbar vertical/lowlevel/.style={
    plot graphics/lowlevel draw/.code 2 args={%
      \pgfuses shading{...} % some advanced basic level shading operations
    }
  },
}
}
```

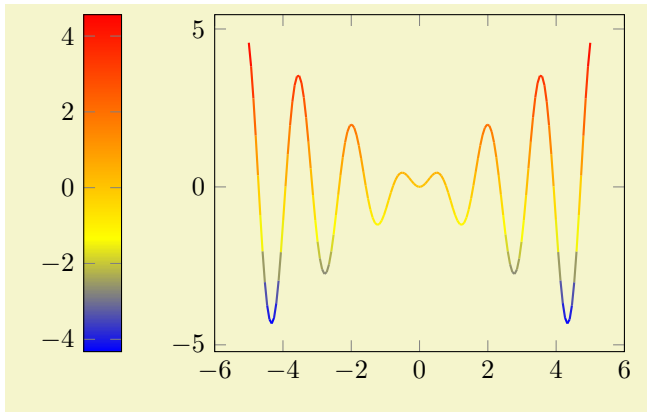
Attention: `colorbar right` redefines `every colorbar`. That means any user customization must take place *after* `colorbar right`:

```
% correct:
\begin{axis}[colorbar right, colorbar style={<some customization>}]
% wrong, colorbar right resets the customization:
\begin{axis}[colorbar style={<some customization>}, colorbar right]
```


/pgfplots/**colorbar left**

(style, no value)

A style which re-defines **every colorbar** and **colorbar shift** such that color bars are placed left of their parent axis.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar left]
    \addplot[mesh,thick,samples=150]
      {x*sin(deg(4*x))};
  \end{axis}
\end{tikzpicture}
```

The style **colorbar left** is defined as

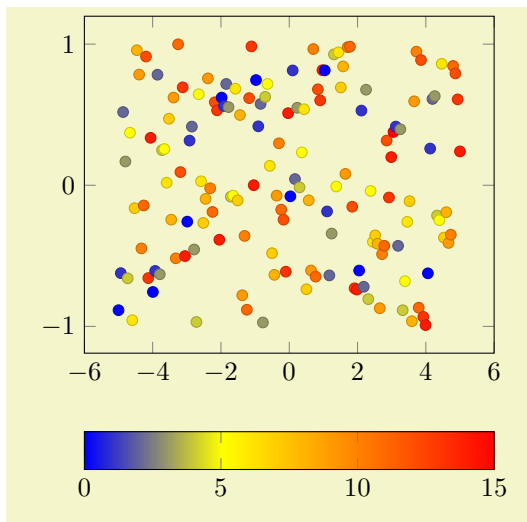
```
\pgfplotsset{
  colorbar left/.style={
    /pgfplots/colorbar right,
    /pgfplots/colorbar shift/.style={xshift=-0.3cm},
    /pgfplots/every colorbar/.append style={
      at={(parent axis.left of north west)},
      anchor=north east,
      yticklabel pos=left,
    }
  }
}
```

Attention: **colorbar left** redefines **every colorbar**. That means any user customization must take place *after* **colorbar left** (see also the documentation for **colorbar right**).

/pgfplots/**colorbar horizontal**

(style, no value)

A style which re-defines **every colorbar** and **colorbar shift** such that color bars are placed below their parent axis, with a horizontal bar.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar horizontal]
    \addplot[only marks,scatter,
      scatter src={mod(\coordindex,15)},samples=150]
      {rand};
  \end{axis}
\end{tikzpicture}
```

This style is defined as

```
\pgfplotsset{
  colorbar horizontal/.style={
    /pgfplots/colorbar=true,
    /pgfplots/colorbar shift/.style={yshift=-0.3cm},
    /pgfplots/every colorbar/.style={
      title=,
      xlabel=,
      ylabel=,
      zlabel=,
      legend entries=,
      axis on top,
      at={{(parent axis.below south west)}},
      anchor=north west,
      ymin=0,
      ymax=1,
      xmin=\pgfkeysvalueof{/pgfplots/point meta min},
      xmax=\pgfkeysvalueof{/pgfplots/point meta max},
      plot graphics/ymin=0,
      plot graphics/ymax=1,
      plot graphics/xmin=\pgfkeysvalueof{/pgfplots/point meta min},
      plot graphics/xmax=\pgfkeysvalueof{/pgfplots/point meta max},
      enlargelimits=false,
      scale only axis,
      width=\pgfkeysvalueof{/pgfplots/parent axis width},
      y=\pgfkeysvalueof{/pgfplots/colorbar/width},
      xticklabel pos=left,
      ytick=\empty,
      colorbar horizontal/lowlevel,
    }%
  },%
  /pgfplots/colorbar horizontal/lowlevel/.style={%
    plot graphics/lowlevel draw/.code 2 args={%
      \pgfuses shading{...} % some advanced basic level shading operations
    },%
  },%
}
}
```

Attention: `colorbar horizontal` re-defines `every colorbar`. That means any user customization must take place *after* `colorbar horizontal`:

```
% correct:
\begin{axis}[colorbar horizontal, colorbar style={<some customization>}]
% wrong, colorbar horizontal resets the customization:
\begin{axis}[colorbar style={<some customization>}, colorbar horizontal]
```

`/pgfplots/every colorbar` (style, no value)

This style governs the placement, alignment and appearance of color bars. Any desired detail changes for color bars can be put into this style. Additionally, there is a style `colorbar shift` which is set after `every colorbar`. The latter style is intended to contain only shift transformations like `xshift` or `yshift` (making it easier to overwrite or deactivate them).

While a color bar is drawn, the predefined node `parent axis` can be used to align at the parent axis.

Predefined node `parent axis`

A node for the parent axis of a color bar. It is only valid for color bars.

Thus,

```
\pgfplotsset{
  colorbar style={
    at={{(parent axis.right of north east)}},
    anchor=north west,
  },
  colorbar shift/.style={xshift=0.3cm}
}
```

places the colorbar in a way that its top left (north west) corner is aligned right of the top right corner (**right of north east**) of its parent axis. Combining this with the `colorbar shift` is actually the same as the initial setting.

Since color bars depend on some of its parent's properties, these properties are available as values of the following keys:

`/pgfplots/point meta min` (no value)
`/pgfplots/point meta max` (no value)

The values of these keys contain the lower and upper bound of the color map, i.e. the lower and upper limit for the color bar.

The value is `\pgfkeysvalueof{/pgfplots/point meta min}` inside of `every colorbar`.

The value is usually determined using the axis wide point meta limits, i.e. they are computed as minimum and maximum value over all plots (unless the user provided limits manually). Consider the `colorbar source` key if you'd like to select point meta limits of one specific `\addplot` command.

`/pgfplots/colorbar source={\langle true,false \rangle}` (initially `false`)

Allows to select a specific `\addplot` command whose point meta limits are taken as upper and lower limit of a `colorbar`'s data range. This affects the tick descriptions of the `colorbar`. It needs to be provided as argument to `\addplot`, i.e. using

```
\addplot[... ,colorbar source] ...
% or
\addplot+[colorbar source] ...
```

or as key inside of a `cycle list`.

Using `colorbar source` automatically implies `point meta rel=per plot` for that specific plot.

If there are more than one `\addplot` commands with `colorbar source`, the last one is selected.

`/pgfplots/parent axis width` (no value)
`/pgfplots/parent axis height` (no value)

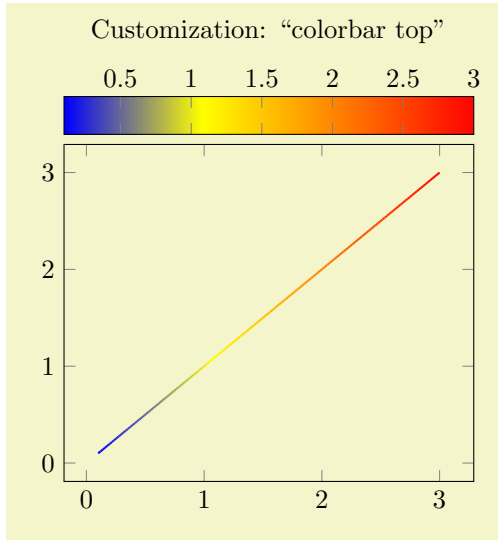
The values of these keys contain the size of the parent axis. They can be used as `width` and/or `height` arguments for `every colorbar` with `\pgfkeysvalueof{/pgfplots/parent axis width}`.

These values are only valid inside of color bars.

Besides these values, each color bar inherits a list of styles of its parent axis, namely

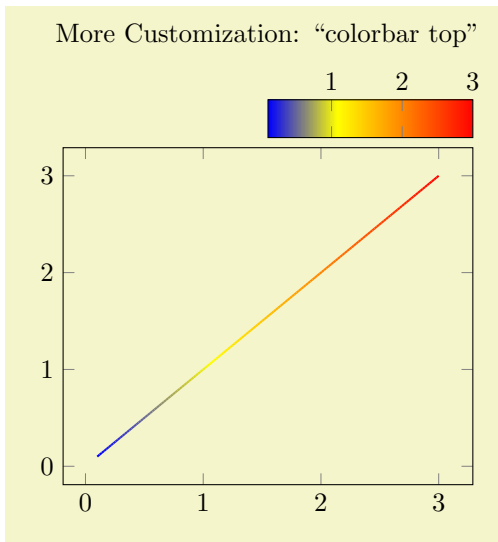
- `every tick`,
- `every minor tick`,
- `every major tick`,
- `every axis grid`,
- `every minor grid`,
- `every major grid`,
- `every tick label`.

This can be used to inherit line width and/or fonts.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  colorbar horizontal,
  colorbar style={
    at={(0.5,1.03)},anchor=south,
    xticklabel pos=upper
  },
  title style={yshift=1cm},
  title=Customization: ‘‘colorbar top’’]

  \addplot[mesh,thick,samples=150,domain=0.1:3]
    {x};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  colorbar horizontal,
  colorbar style={
    at={(1,1.03)},anchor=south east,
    width=0.5*
      \pgfkeysvalueof{/pgfplots/parent axis width},
    xticklabel pos=upper,
  },
  title style={yshift=1cm},
  title=More Customization: ‘‘colorbar top’’]

  \addplot[mesh,thick,samples=150,domain=0.1:3]
    {x};
\end{axis}
\end{tikzpicture}
```

Please take a look at the predefined styles `colorbar right`, `colorbar left` and `colorbar horizontal` for more details about configuration possibilities for every colorbar.

Remark: A color bar is just a normal axis. That means every colorbar can contain specifications where to place tick labels, extra ticks, scalings and most other features of a normal axis as well (except nested color bars).

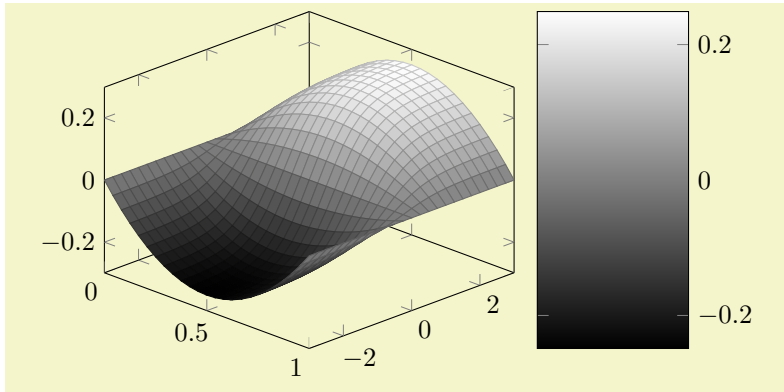
`/pgfplots/colorbar style={⟨key-value list⟩}`

A shortcut for every colorbar/.append style={⟨key-value list⟩}. It appends options to the colorbar style.

`/pgfplots/colorbar/width={⟨dimension⟩}`

(initially 0.5cm)

Sets the width of a color bar.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    view/az=45,
    colorbar,
    colorbar/width=2cm,
    colormap/blackwhite]

    \addplot3[surf,domain=0:1,y domain=-3:3] {x*(1-x)*tanh(y)};
  \end{axis}
\end{tikzpicture}
```

For horizontal color bars, this sets the height.

/pgfplots/**colorbar shift** (style, no value)

This style is installed after **every colorbar**. It is intended to contain only shift transformations like **xshift** and/or **yshift**. The reason to provide two separate styles is to allow easier deactivation of shift transformations.

```
\pgfplotsset{
  colorbar shift/.style={xshift=1cm}
}
```

Predefined node **current colorbar axis**

A predefined node for the color bar of an axis. After **\end{axis}**, this node can be used to align further graphical elements at the color bar. Note that **current axis** refers to the axis as such while **current colorbar axis** refers to the color bar (which is an axis itself).

/pgfplots/**colorbar/draw/.code**={ $\langle \dots \rangle$ }

This code key belongs to the low level interface of color bars. It is invoked whenever a color bar needs to be drawn. Usually, it won't be necessary to use or modify this key explicitly.

When this key is invoked, the styles inherited from the parent axis are already set and the required variables (see the documentation of **every colorbar**) are initialized.

This code key can be replaced if one needs more than one color bar (or other wrinkles).

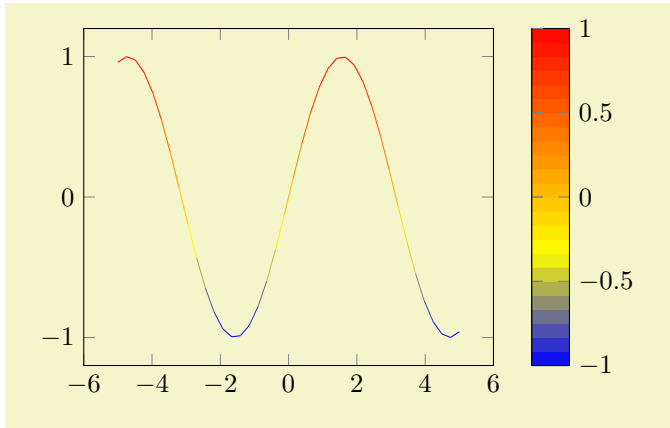
The initial configuration is

```
\pgfplotsset{colorbar/draw/.code={%
  \axis[every colorbar,colorbar shift,colorbar=false]
  \addplot graphics {};
  \endaxis
}}
}
```

Please note that a color bar axis is nothing special as such – it is just a normal axis with one **plot graphics** command and it is invoked with a special set of options. The only special thing is that a set of styles and some variables are inherited from its parent axis.

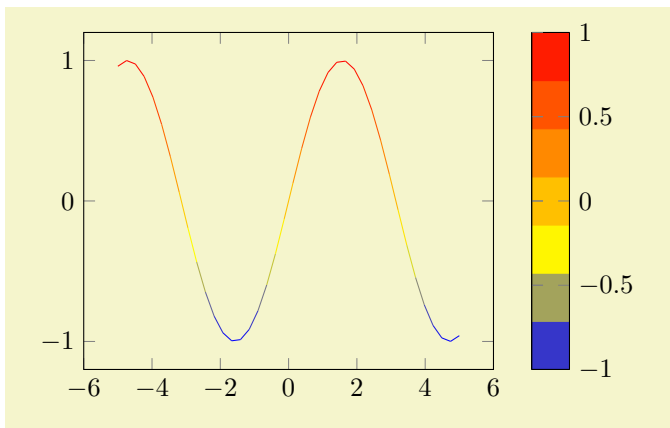
/pgfplots/**colorbar sampled**={ $\langle \text{optional options} \rangle$ } (style, default **surf,mark=none,shader=flat**)

A style which installs a discretely sampled color bar.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar sampled]
    \addplot[mesh,samples=40] {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

The style uses `\addplot3[options]` to draw the `colorbar`, with `domain` set to the color range and the current value of the `samples` key to determine the number of samples. In other words: it uses `plot expression` and a surface plot to visualize the `colorbar`. Use `colorbar style={samples=10}` to change the number of samples.

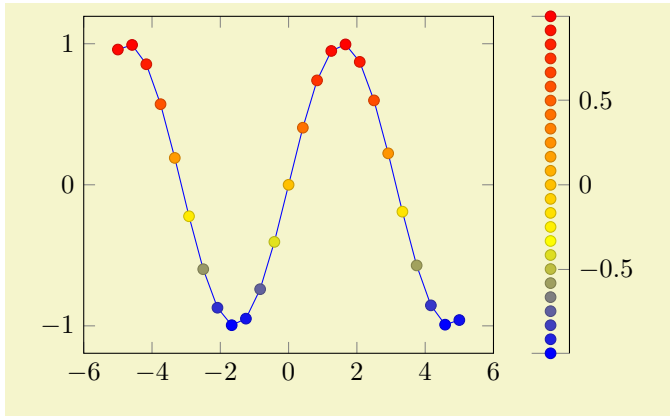


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar sampled,colorbar style={samples=8}]
    \addplot[mesh,samples=40] {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

The *options* can be used to change the `\addplot3` options used for the colorbar visualization. For example, `colorbar sampled={surf,shader=interp}` will use Gouraud shading which has visually the same effect as the standard color bar.

`/pgfplots/colorbar sampled line={optional options}` (style, default `scatter`, only marks)

A style which draws a discrete colorbar. In contrast to `colorbar sampled`, it visualizes the `colorbar` using a line plot, not a `surf` plot.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[colorbar sampled line]
    \addplot+[scatter] {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

The initial configuration uses a `scatter` plot to visualize the `colorbar`, it can be changed by specifying `<options>`.

Furthermore, the axis appearance is changed using `axis y line*=left|right`, depending on the position of the color bar (or `axis x line*=bottom` for `colorbar horizontal`).

Consider the `tick align=outside` feature if you prefer tick lines outside of the colorbar instead of inside.

`/pgfplots/every colorbar sampled line` (style, no value)

A style which is used by `colorbar sampled line` to change the color of the line without ticks.

It is initially set to `help lines`.

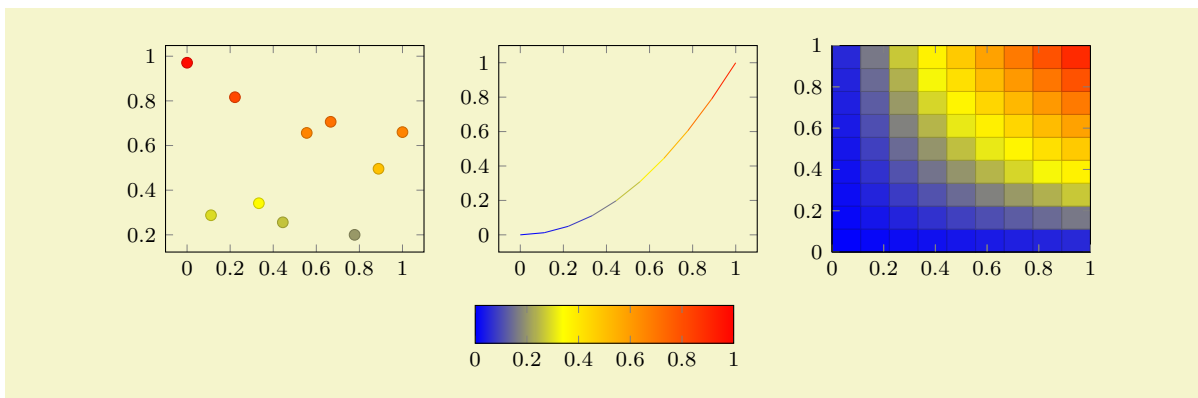
4.8.13 Color Bars Outside Of an Axis

Occasionally, one has multiple adjacent plots, each with the same `colormap` and the same `point meta min` and `point meta max` values and we'd like to show a *single* `colorbar`. PGFPLOTS supports the `colorbar to name` feature which is similar to the related method for legends, `legend to name`:

`/pgfplots/colorbar to name={<name>}` (initially `empty`)

Enables to detach a `colorbar` from its parent axis: instead of drawing the `colorbar`, a self-contained, independent set of drawing commands will be stored using the label `<name>`. The label is defined using `\label{<name>}`, just as for any other L^AT_EX label. The name can be referenced using `\ref{<name>}`.

Thus, typing `\ref{<name>}` somewhere outside of the axis, maybe even outside of any picture, will cause the `colorbar` to be drawn.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{footnotesize,samples=10, domain=0:1,point meta min=0, point meta max=1}
\begin{center}% note that \centering uses less vspace...
\begin{tikzpicture}
  \begin{axis}[colorbar,colorbar horizontal,colorbar to name={storedcolorbar}]
    \addplot[scatter,only marks,mark=*] {rnd};
  \end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
  \begin{axis}
    \addplot+[domain=0:1,mark=none,mesh] {x^2};
  \end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
  \begin{axis}[view={0}{90}]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
\\
\ref{storedcolorbar}
\end{center}

```

The feature works in the same way as described for `legend to name`, please refer to its description on page 187 for the details. We only summarize the differences here.

`\pgfplotscolorbarfromname{<name>}`

This command poses an equivalent alternative for `\ref{<name>}`: it has essentially the same effect, but it does not create links when used with the `hyperref` package.

`/pgfplots/every colorbar to name picture` (style, no value)

A style which is installed when `\ref` is used outside of a picture: a new picture will be created with `\tikz[/pgfplots/every colorbar to name picture]`.

See also the `every legend to name picture` style.

4.9 Scaling Options

There are a various options which control or change the scaling of an axis. Here, “scaling” typically means two aspects: first, the unit vector size in each direction and second, the displayed limits in each direction. Both together control the size of the axis box. In addition, axis descriptions change the size. However, PGFPLOTS scales only units and determines limits. It does not scale axis descriptions by default in order to keep consistent font sizes between the text and the figure.

Often, one wishes to provide the target size only and let PGFPLOTS do the rest. This is the default; it scales the axis to `width` and `height`. If you provide one of these options, the axis will be rescaled while keeping the aspect ratio.

Occasionally, one wants to provide unit vectors explicitly to ensure that one unit takes a prescribed amount of space. This is possible by means of the `x`, `y`, and `z` keys. In such a case, the `width` and `height` options will be ignored (or only applied for the unspecified unit vectors).

Another common approach is to enforce specific `unit vector ratios`: for example by specifying that each unit should take the same amount of space using `axis equal`. Here, PGFPLOTS scales the lengths of all vectors uniformly, i.e. it applies the same scale to each vector. This is done by means of the `scale mode` configuration which is basically one of `scale mode=stretch to fill` or `scale mode=scale uniformly`: PGFPLOTS tries to satisfy the prescribed `width` and `height` arguments by finding a *common* scaling factor. In addition, it attempts to enlarge the limits individually to fit into the prescribed dimensions.

In addition, you can use the option `/pgfplots/scale` to simply scale all final units up by some prescribed factor. This does not change text labels.

If needed, you can also supply `/tikz/scale` to an axis. This will scale the complete resulting image, including all text labels.

4.9.1 Common Scaling Options

All common options mentioned in the previous paragraphs are described here.

`/pgfplots/width={<dimen>}`

(initially empty)

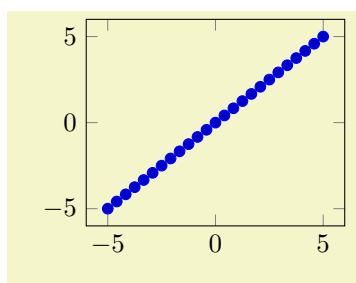
Sets the width of the final picture to {<dimen>}.

Any non-empty dimension like `width=5cm` sets the desired target width. Any TeX unit is accepted (like 200pt or 5in).

An empty value `width={}` means “use default width or rescale proportionally to `height`”. In this case, PGFLOTS uses the value of `\axisdefaultwidth` as target quantity. However, if the `height` key has been set, PGFLOTS will rescale the `\axisdefaultwidth` in a way which keeps the ratio between `\axisdefaultwidth` and `\axisdefaultheight`. This allows to specify just one of `width` and `height` and keep aspect ratios.

Consequently, if you specify just `width=5cm` but leave the default `height={}`, the scaling will respect the initial aspect ratio.

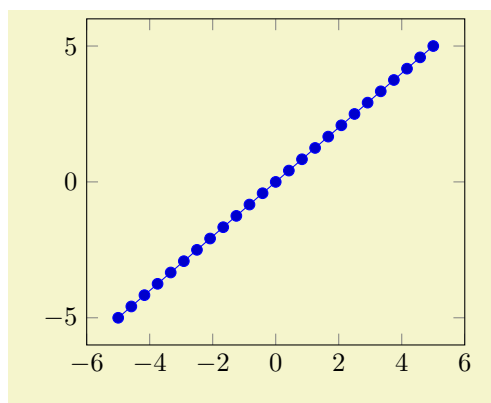
The scaling affects the unit vectors for x , y , and z . It does not change the size of text labels or axis descriptions.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[width=5cm]
\addplot {x};
\end{axis}
\end{tikzpicture}
```

Please note that PGFLOTS only estimates the size needed for axis- and tick labels. The estimate assumes a fixed amount of space for anything which is outside of the axis box. This has the effect that the final images may be slightly larger or slightly smaller than the prescribed dimensions. However, the fixed amount is always the same; it is set to 45pt. That means that multiple pictures with the same target dimensions will have the same size for their axis boxes – even if the size for descriptions varies.

It is also possible to scale the *axis box* to the prescribed width/height. In that case, the total width will be larger due to the axis descriptions. However, the axis box fills the desired dimensions exactly.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[width=5cm,scale only axis]
\addplot {x};
\end{axis}
\end{tikzpicture}
```

Note: changing `width` and/or `height` changes *only* the unit vector sizes. In particular, it does not change the font size for any axis description, nor does it change the default spacing between adjacent tick labels. It is best-practice to use `width/height` for “small” changes, i.e. changes for which the font size should remain the same. Consider using one of the styles `normalsize`, `small`, `footnotesize`, or `tiny` which are described in Section 4.9.2 on page 220, and *then* change to your desired dimensions if you need a different “quality” of scaling.

`\axisdefaultwidth`

This macro defines the default width. It is preset to 240pt.

This default width defines the aspect ratio which will be used whenever just one of `width` or `height` is specified: the aspect ratio is the ratio between `\axisdefaultwidth` and `\axisdefaultheight`.

You can change it using

```
\def\axisdefaultwidth{10cm}
```

`/pgfplots/height={⟨dimen⟩}` (initially empty)

Works in the same way as `width` except that an empty value `height={}` defaults to “use either `\axisdefaultheight` or scale proportionally if just `width` has been changed”.

`\axisdefaultheight`

This macro defines the default height. It is preset to 207pt.

See `\axisdefaultwidth`.

`/pgfplots/scale only axis=true|false` (initially false)

If `scale only axis` is enabled, `width` and `height` apply only to the axis rectangle. Consequently, the resulting figure is larger than `width` and `height` (because of any axis descriptions). However, the axis box has exactly the prescribed target dimensions.

If `scale only axis=false` (the default), PGFLOTS will try to produce the desired width *including* labels, titles and ticks.

`/pgfplots/x={⟨dimen⟩}` (initially empty)

`/pgfplots/y={⟨dimen⟩}` (initially empty)

`/pgfplots/z={⟨dimen⟩}` (initially empty)

`/pgfplots/x={⟨x⟩,⟨y⟩}`

`/pgfplots/y={⟨x⟩,⟨y⟩}`

`/pgfplots/z={⟨x⟩,⟨y⟩}`

Allows to provide zero, one, two, or three of the target unit vectors.

In this context, a “unit vector” is a two-dimensional vector which defines the projection onto the canvas: every logical plot coordinate (x, y) is drawn at the canvas position

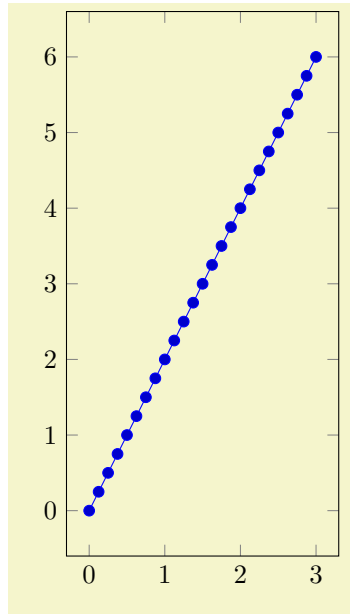
$$x \cdot \begin{bmatrix} e_{xx} \\ e_{xy} \end{bmatrix} + y \cdot \begin{bmatrix} e_{yx} \\ e_{yy} \end{bmatrix}.$$

The unit vectors e_x and e_y determine the paper position in the current (always two dimensional) image. For a standard three-dimensional axis, a plot coordinate (x, y, z) is drawn at

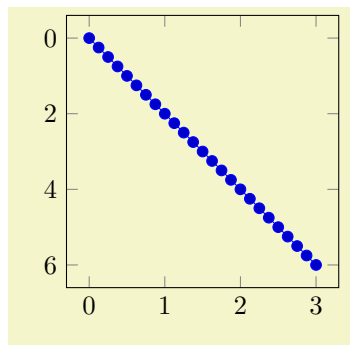
$$x \cdot \begin{bmatrix} e_{xx} \\ e_{xy} \end{bmatrix} + y \cdot \begin{bmatrix} e_{yx} \\ e_{yy} \end{bmatrix} + z \cdot \begin{bmatrix} e_{zx} \\ e_{zy} \end{bmatrix}.$$

The initial setting assigns empty values to each of these keys, i.e. `x={},y={},z={}`. In this case, PGFLOTS is free to choose these vectors as best. To this end, it uses `width`, `height`, `scale mode`, `plot box ratio`, `unit vector ratio`, `view`, and the axis limits.

The key `x={⟨dimen⟩}` simply sets $e_x = (\langle dimen \rangle, 0)^T$ while `y={⟨dimen⟩}` sets $e_y = (0, \langle dimen \rangle)^T$. Using `z={⟨dimen⟩}` results in $e_z = (\langle dimen \rangle, \langle dimen \rangle)^T$. In this context, $\langle dimen \rangle$ is any T_EX size like 1mm, 2cm or 5pt. Note that you should not use negative values for $\langle dimen \rangle$ (consider using `x dir` and its variants to reverse axis directions).

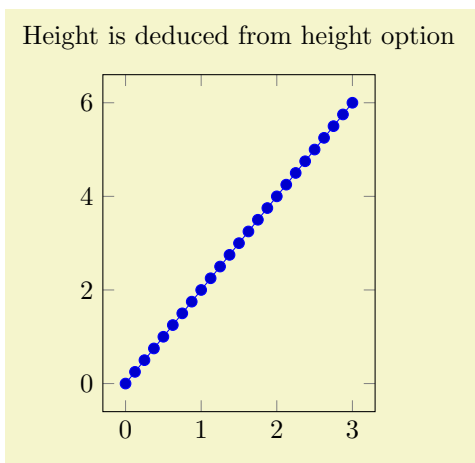


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x=1cm,y=1cm]
\addplot expression[domain=0:3] {2*x};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x=1cm,y=0.5cm,y dir=reverse]
\addplot expression[domain=0:3] {2*x};
\end{axis}
\end{tikzpicture}
```

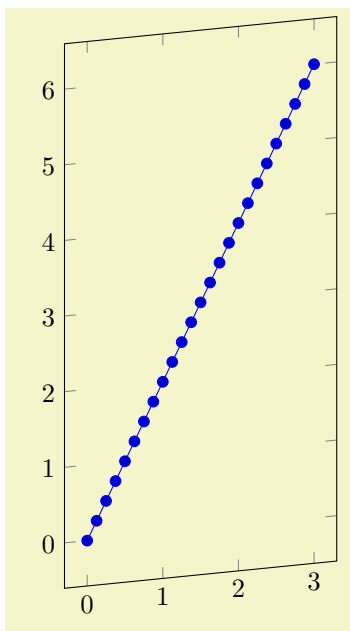
Note that if you change the unit vector for just one direction, the other vector(s) will be chosen by PGFPLOTS – and scaled in order to fill the prescribed `width` and `height` as best as PGFPLOTS can (but see remarks for three-dimensional plots at the end of this key).



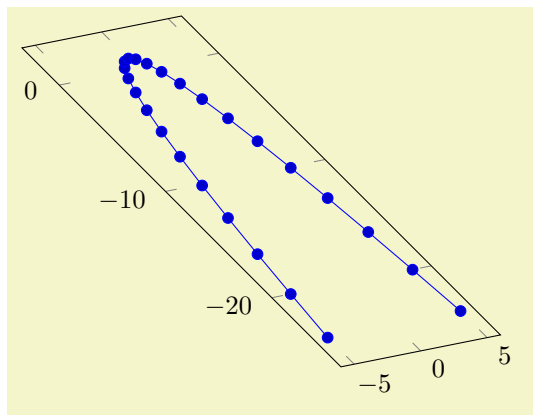
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x=1cm,
title=Height is deduced from height option]
\addplot expression[domain=0:3] {2*x};
\end{axis}
\end{tikzpicture}
```

The second syntax, `x={{\langle x \rangle, \langle y \rangle}}` sets $e_x = (\langle x \rangle, \langle y \rangle)^T$ explicitly⁴⁶. The corresponding keys for `y` and `z` work in a similar way. This allows to define skewed or rotated axes.

⁴⁶Please note that you need extra curly braces around the vector. Otherwise, the comma will be interpreted as separator for the next key-value pair.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x={(1cm,0.1cm)},y=1cm]
\addplot expression[domain=0:3] {2*x};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
x={(5pt,1pt)},
y={(-4pt,4pt)}]
\addplot {1-x^2};
\end{axis}
\end{tikzpicture}
```

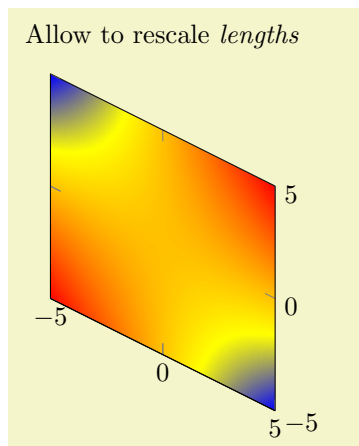
Setting `x` and/or `y` for logarithmic axis will set the dimension used for $1 \cdot e \approx 2.71828$ (or whatever has been set as `log basis x`).

Please note that it is *not* possible to specify `x` as argument to `tikzpicture`. The option

```
\begin{tikzpicture}[x=1.5cm]
\begin{axis}
...
\end{axis}
\end{tikzpicture}
```

does not have any effect because an axis rescales its coordinates (see the `width` option).

Note that providing unit vectors explicitly usually causes PGFPLOTS to ignore any other scaling options. In other words: if you say `y=0.1cm`, PGFPLOTS will use $(0\text{cm}, 0.1\text{cm})$ as y projection vector. However, if you add `scale mode=scale uniformly`, you allow PGFPLOTS to change the *lengths* of your vectors. Of course, it will keep their relative directions and relative sizes. In this case, PGFPLOTS will try to determine a good common scaling factor *and* it will try to change the axis limits in order to fill the prescribed `width` and `height` (see the documentation for `scale mode` for details).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title=Allow to rescale \emph{lengths},
  x={(0.1cm,-0.05cm)},
  y=0.1cm,
  z=0cm,
  axis on top,
  scale mode=scale uniformly,
]
\addplot3[surf,shader=interp] {x*y};
\end{axis}
\end{tikzpicture}
```

In the example above, PGFPLOTS decided that it should only rescale units – at the expensive of the `width` constraint.

Changes to font sizes: see also Section 4.9.2 if you want to change font sizes or the density of tick labels in a simple way.

Explicit units for 3D axes: As of version 1.5, it is also possible to supply unit vectors to three-dimensional axes. In this case, the following extra assumptions need to be satisfied:

1. If you want to control three-dimensional units, you need to provide *all* of `x`, `y`, and `z` keys. For two-dimensional axes, it is also supported to supply just one of `x` or `y`.
2. Any provided three-dimensional unit vectors are assumed to form a *right-handed coordinate system*. In other words: take your right hand, let the thumb point into the `x` direction, the index finger in `y` direction and the middle finger in `z` direction. If that is impossible, the PGFPLOTS output will be wrong. The reason for this assumption is that PGFPLOTS needs to compute the view direction out of the provided units (see below).

Consider using `x dir=reverse` or its variants in case you want to reverse directions.

3. For three-dimensional axes, PGFPLOTS computes a view direction out of the provided unit vectors. The view direction is required to allow the `z buffer` feature (i.e. to decide about depths)⁴⁷.

This feature is used to for the `\addplot3` graphics feature, compare the examples in Section 4.2.8 on page 45.

Limitations: Unfortunately, skewed axes are **not available for bar plots**.

```
/pgfplots/xmode=normal|linear|log (initially normal)
/pgfplots/ymode=normal|linear|log (initially normal)
/pgfplots/zmode=normal|linear|log (initially normal)
```

Allows to choose between linear (=normal) or logarithmic axis scaling or logplots for each `x`, `y`, `z`-combination.

Logarithmic plots use the current setting of `log basis x` and its variants to determine the basis (default is e).

```
/pgfplots/x dir=normal|reverse (initially normal)
/pgfplots/y dir=normal|reverse (initially normal)
/pgfplots/z dir=normal|reverse (initially normal)
```

Allows to reverse axis directions such that values are given in decreasing order.

This key is documented in all detail on page 241.

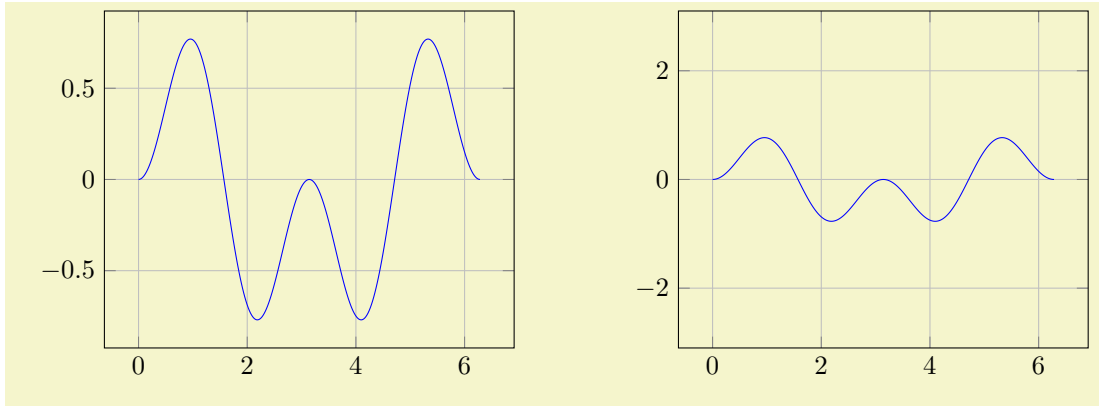
⁴⁷PGFPLOTS provides a debug option called `view dir={x}{y}{z}` to override the view direction, should that ever be interesting.

/pgfplots/`axis equal`={ $\langle true, false \rangle$ }

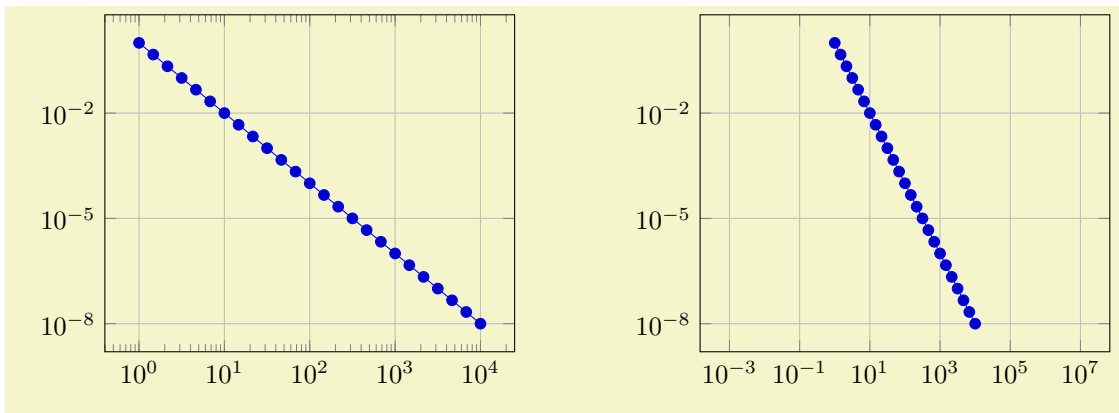
(initially `false`)

Each unit vector is set to the same length while the axis dimensions stay constant. Afterwards, the size ratios for each unit in x and y will be the same.

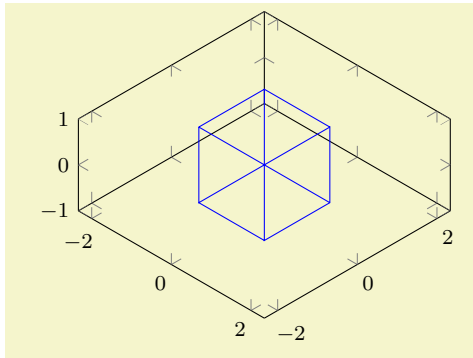
Axis limits will be enlarged to compensate for the scaling effect.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[axis equal=false,grid=major]
    \addplot[blue] expression[domain=0:2*pi,samples=300] {\sin(deg(x))*\sin(2*deg(x))};
  \end{axis}
\end{tikzpicture}
\hspace{1cm}
\begin{tikzpicture}
  \begin{axis}[axis equal=true,grid=major]
    \addplot[blue] expression[domain=0:2*pi,samples=300] {\sin(deg(x))*\sin(2*deg(x))};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[axis equal=false,grid=major]
    \addplot expression[domain=1:10000] {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
\hspace{1cm}
\begin{tikzpicture}
  \begin{loglogaxis}[axis equal=true,grid=major]
    \addplot expression[domain=1:10000] {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
```

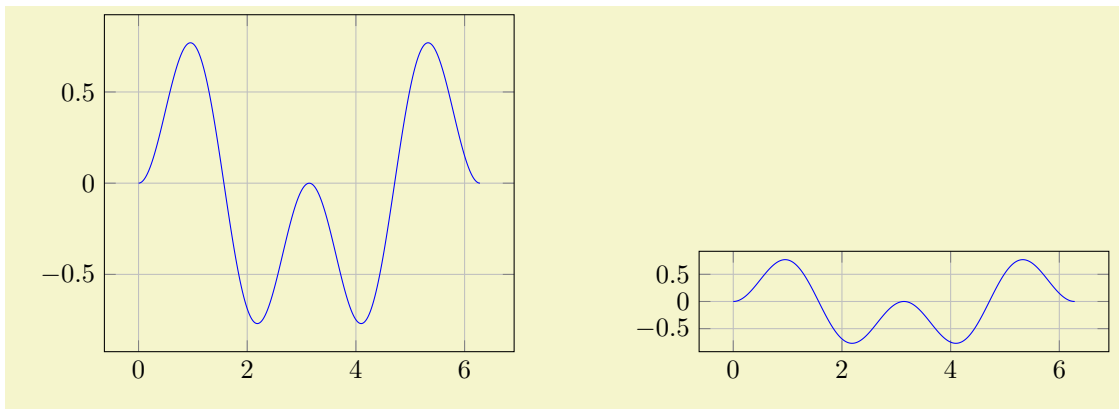


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis equal,small,view={45}{35.26}]
\addplot3[mark=cube, blue, mark size=1cm]
coordinates {(0,0,0)};
\end{axis}
\end{tikzpicture}
```

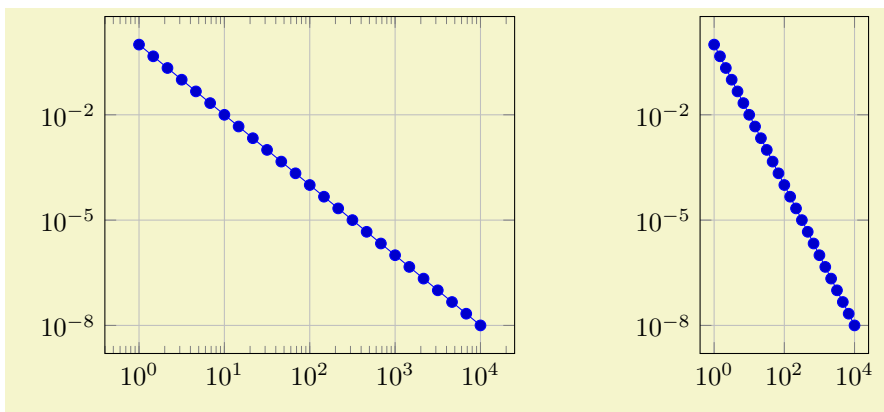
The configuration `axis equal=true` is actually just a style which sets `unit vector ratio=1 1 1`, `unit rescale keep size=true`.

`/pgfplots/axis equal image={\langle true,false\rangle}` (initially false)

Similar to `axis equal`, but the axis limits will stay constant as well (leading to smaller images).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[axis equal image=false,grid=major]
\addplot[blue] expression[domain=0:2*pi,samples=300] {sin(deg(x))*sin(2*deg(x))};
\end{axis}
\end{tikzpicture}
\hspace{1cm}
\begin{tikzpicture}
\begin{axis}[axis equal image=true,grid=major]
\addplot[blue] expression[domain=0:2*pi,samples=300] {sin(deg(x))*sin(2*deg(x))};
\end{axis}
\end{tikzpicture}
```



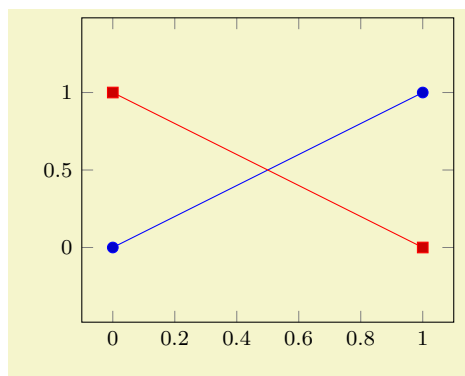
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[axis equal image=false,grid=minor]
    \addplot expression[domain=1:10000] {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
\hspace{1cm}
\begin{tikzpicture}
  \begin{loglogaxis}[axis equal image=true,grid=minor]
    \addplot expression[domain=1:10000] {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
```

The configuration `axis equal image=true` is actually just a style which sets `unit vector ratio=1 1 1`, `unit rescale keep size=false`.

`/pgfplots/unit vector ratio={⟨rx ry rz⟩}` (initially empty)

Allows to provide custom unit vector ratios.

The key allows to tell PGFLOTS that, for example, one unit in x direction should be twice as long as one unit in y direction:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[unit vector ratio=2 1,small]
    \addplot coordinates {(0,0) (1,1)};
    \addplot table[row sep=\\,col sep=&] {
      x & y \\
      0 & 1 \\
      1 & 0 \\
    };
  \end{axis}
\end{tikzpicture}
```

Providing `unit vector ratio=2 1` means that $\frac{e_x}{e_y} = 2$ where each coordinate (x, y) is placed at $xe_x + ye_y \in \mathbb{R}^2$ (see the documentation for `x` and `y` options). Note that `axis equal` is nothing but `unit vector ratio=1 1 1`.

The arguments $\langle rx \rangle$, $\langle ry \rangle$, and $\langle rz \rangle$ are ratios for x , y and z vectors, respectively. For two-dimensional axes, only $\langle rx \rangle$ and $\langle ry \rangle$ are considered; they are provided relative to the y axis. In other words: the x unit vector will be $\langle rx \rangle / \langle ry \rangle$ times longer than the y unit vector. For three-dimensional axes, all three arguments can be provided; they are interpreted relative to the z unit vector. Thus, a three dimensional axis with `unit vector ratio=1 2 4` will have an x unit which is $1/4$ the length of the z unit, and a y unit which is $2/4$ the length of the z unit.

Trailing values of 1 can be omitted, i.e. `unit vector ratio=2 1` is the same as `unit vector ratio=2;` and `unit vector ratio=3 2 1` is the same as `unit vector ratio=3 2`. An empty value `unit vector ratio={}` disables unit vector rescaling.

Note that an active `unit vector ratio` will implicitly set `scale mode=scale uniformly`⁴⁸.

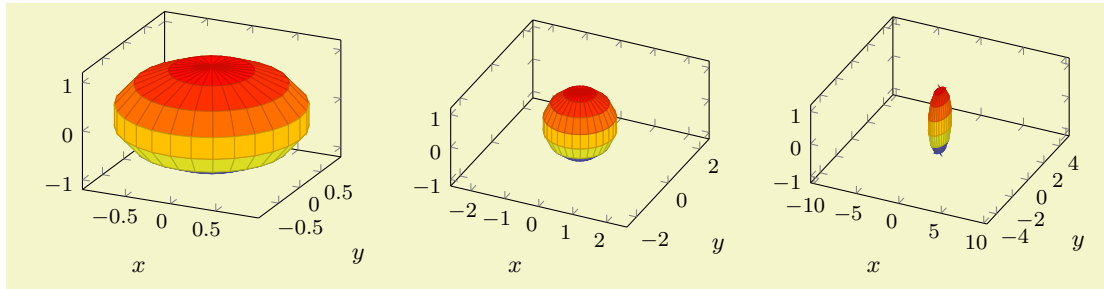
`/pgfplots/unit vector ratio*={⟨rx ry rz⟩}`

`/pgfplots/unit rescale keep size=true|false|unless limits declared` (initially unless limits declared)

In the default configuration, PGFLOTS maintains the original axis dimensions even though `unit vector ratio` involves different scalings.

It does so by enlarging the limits.

⁴⁸This has been introduced in version 1.6. For older versions, the `axis equal` feature produced wrong results for three-dimensional axes.

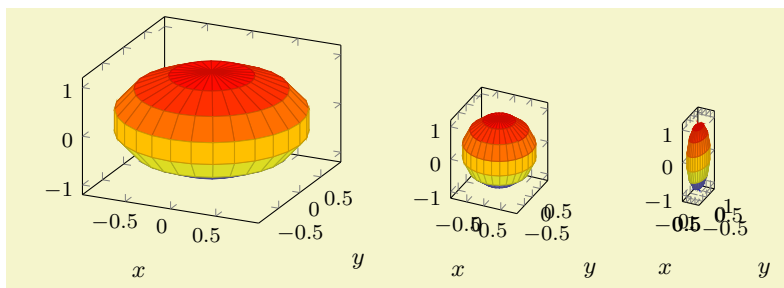


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,unit vector ratio=]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,unit vector ratio=1 1 1]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,unit vector ratio=0.25 0.5]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}
```

The example above has the same plot, with three different unit ratios. The first has no limitations (it is the default configuration). The second uses the same length for each unit vector and enlarges the limits in order to maintain the same dimensions. The third example has an x unit which is $1/4$ the length of a z unit, and an y unit which is $1/2$ the length of a z unit.

PGFplots does its best to respect the involved scaling options (the prescribed `width` and `height`, the `unit vector ratio`, and any specified axis limits). In the case above, it enlarged the horizontal limits and kept the z limit as-is. See `scale mode` and its documentation for details about the involved algorithm and its parameters.

The `unit rescale keep size=false` key, or, equivalently, `unit vector ratio*=...`, does not enlarge limits:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,unit vector ratio=]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,
    unit rescale keep size=false,
    unit vector ratio=1 1 1]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[footnotesize,xlabel=$x$,ylabel=$y$,
    unit vector ratio*=0.25 0.5, % the '*' implies 'unit rescale keep size=false'
  ]
    \addplot3[surf,z buffer=sort,samples=15,
      variable=\u, variable y=\v,
      domain=0:180, y domain=0:360]
      ({cos(u)*sin(v)}, {sin(u)*sin(v)}, {cos(v)});
  \end{axis}
\end{tikzpicture}

```

The key `unit rescale keep size` also affects `scale mode=scale` uniformly (which is closely related to `axis equal`).

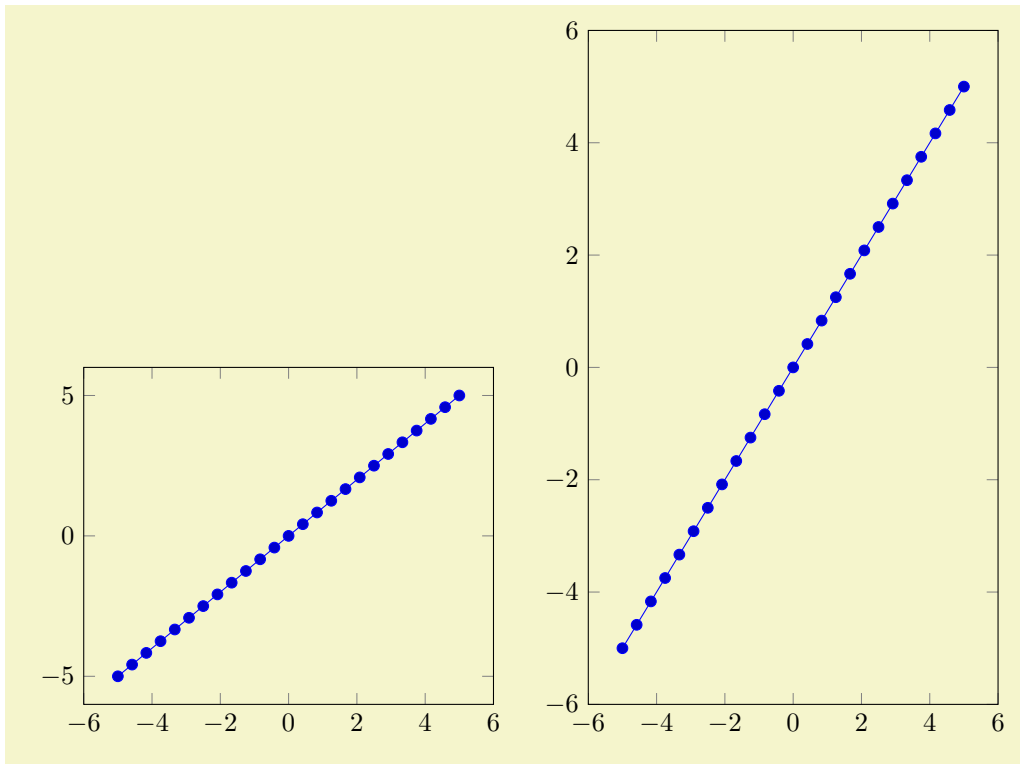
Here is the reference of the value of `unit rescale keep size`: the value `true` means that PGF-PLOTS will enlarge limits in order to keep the size. It will try to respect user provided limits, but if the user provided *all* limits, it will *override* the user-provided limits and will rescale them. Thus, `true` gives higher priority to the axis size than to user-provided limits. The choice `false` will never rescale axis limits. The choice `unless limits declared` is a mixture: it will enlarge limits unless the user provided them. If the user provides all limits explicitly, this choice is the same as `false`.

<code>/pgfplots/x post scale={\langle scale \rangle}</code>	(initially empty)
<code>/pgfplots/y post scale={\langle scale \rangle}</code>	(initially empty)
<code>/pgfplots/z post scale={\langle scale \rangle}</code>	(initially empty)
<code>/pgfplots/scale={\langle scale \rangle}</code>	(initially empty)

Lets PGFPLOTS compute the axis scaling based on `width`, `height`, `view`, `plot box ratio`, `axis equal` or explicit unit vectors with `x`, `y`, `z` and *rescales* the resulting vector(s) according to `\langle scale \rangle`.

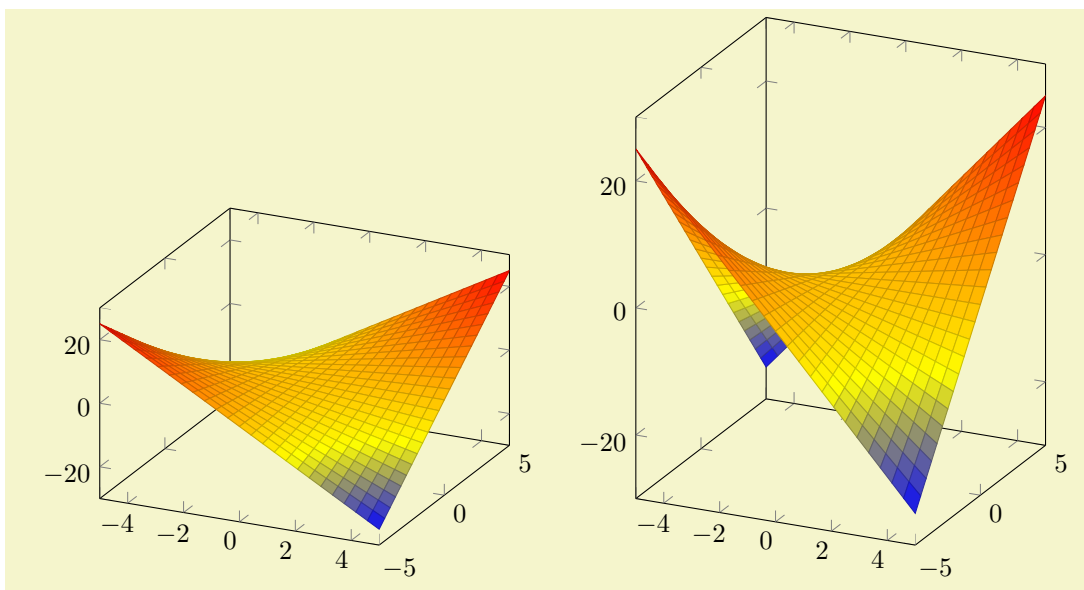
The `scale` key sets all three keys to the same `\langle uniform scale \rangle` value. This is effectively the same as if you rescale the complete axis (without changing sizes of descriptions).

The other keys allow individually rescaled axes.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[y post scale=1]
    \addplot {x};
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[y post scale=2]
    \addplot {x};
  \end{axis}
\end{tikzpicture}
```

Thus, the axis becomes *larger*. This overrules any previous scaling.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[z post scale=1]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
\begin{tikzpicture}
  \begin{axis}[z post scale=2]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```

4.9.2 Scaling Descriptions: Predefined Styles

It is reasonable to change font sizes, marker sizes etc. together with the overall plot size: Large plots should also have larger fonts and small plots should have small fonts and a smaller distance between ticks.

```
/tikz/font=\normalfont|\small|\tiny|...
/pgfplots/max space between ticks={\langle integer \rangle}
/pgfplots/try min ticks={\langle integer \rangle}
/tikz/mark size={\langle integer \rangle}
```

These keys should be adjusted to the figure's dimensions. Use

```
\pgfplotsset{tick label style={font=\footnotesize},
  label style={font=\small},
  legend style={font=\small}
}
```

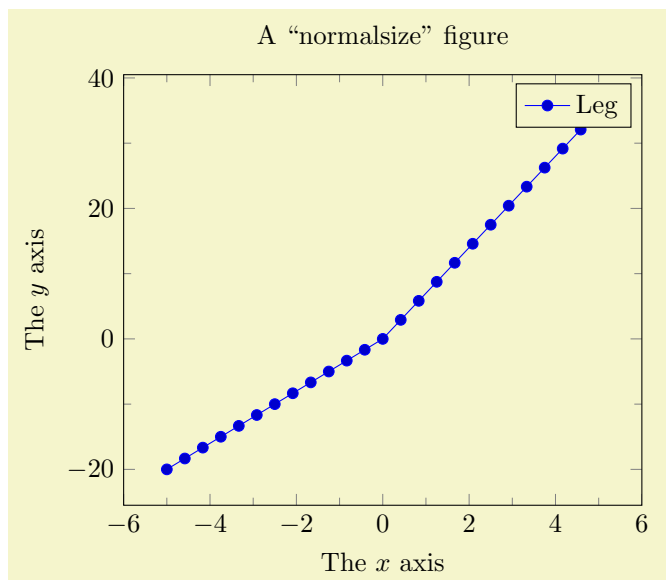
to provide different fonts for different descriptions.

The keys `max space between ticks` and `try min ticks` are described on page 261 and configure the approximate distance and number of successive tick labels (in pt). Please omit the `pt` suffix here.

There are a couple of predefined scaling styles which set some of these options:

```
/pgfplots/normalsize (style, no value)
```

Re-initialises the standard scaling options of PGFLOTS.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[normalsize,
    title=A ‘‘normalsize’’ figure,
    xlabel=The  $x$  axis,
    ylabel=The  $y$  axis,
    minor tick num=1,
    legend entries={Leg}]
    \addplot {max(4*x,7*x)};
  \end{axis}
\end{tikzpicture}
```

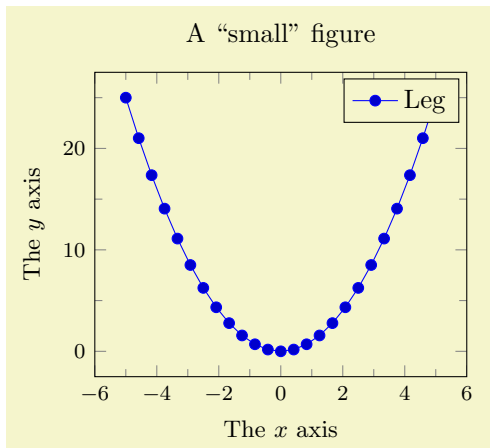
The initial setting is

```
\pgfplotsset{
  normalsize/.style={
    /pgfplots/width=240pt,
    /pgfplots/height=207pt,
    /pgfplots/max space between ticks=35
  }
}
```

/pgfplots/**small**

(style, no value)

Redefines several keys such that the axis is “smaller”.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small,
    title=A ‘‘small’’ figure,
    xlabel=The  $x$  axis,
    ylabel=The  $y$  axis,
    minor tick num=1,
    legend entries={Leg}]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

The initial setting is

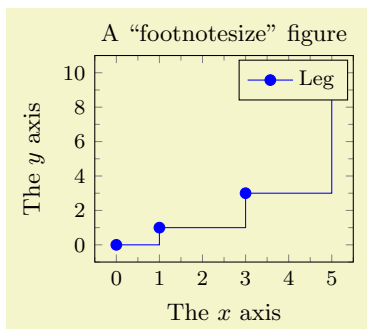
```
\pgfplotsset{
  small/.style={
    width=6.5cm,
    height=,
    tick label style={font=\footnotesize},
    label style={font=\small},
    max space between ticks=25,
  }
}
```

Feel free to redefine the scaling – the option may still be useful to get more ticks without typing too much. You could, for example, set **small,width=6cm**.

/pgfplots/**footnotesize**

(style, no value)

Redefines several keys such that the axis is even smaller. The tick labels will have `\footnotesize`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[footnotesize,
    title=A "footnotesize" figure,
    xlabel=The  $x$  axis,
    ylabel=The  $y$  axis,
    minor tick num=1,
    legend entries={Leg}]
    \addplot+[const plot]
      coordinates {
        (0,0) (1,1) (3,3) (5,10)
      };
  \end{axis}
\end{tikzpicture}
```

The initial setting is

```
\pgfplotsset{
  footnotesize/.style={
    width=5cm,
    height=,
    legend style={font=\footnotesize},
    tick label style={font=\footnotesize},
    label style={font=\small},
    title style={font=\small},
    every axis title shift=0pt,
    max space between ticks=15,
    every mark/.append style={mark size=8},
    major tick length=0.1cm,
    minor tick length=0.066cm,
  },
}
```

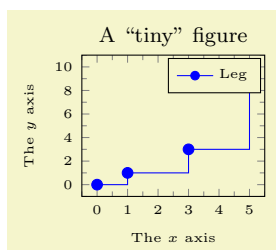
As for `small`, it can be convenient to set `footnotesize` and set `width` afterwards.

You will need `compat=1.3` or newer for this to work.

`/pgfplots/tiny`

(style, no value)

Redefines several keys such that the axis is very small. Most descriptions will have `\tiny` as fontsize.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[tiny,
    title=A "tiny" figure,
    xlabel=The  $x$  axis,
    ylabel=The  $y$  axis,
    minor tick num=1,
    legend entries={Leg}]
    \addplot+[const plot]
      coordinates {
        (0,0) (1,1) (3,3) (5,10)
      };
  \end{axis}
\end{tikzpicture}
```

The initial setting is

```
\pgfplotsset{
  tiny/.style={
    width=4cm,
    height=,
    legend style={font=\tiny},
    tick label style={font=\tiny},
    label style={font=\tiny},
    title style={font=\footnotesize},
    every axis title shift=0pt,
    max space between ticks=12,
    every mark/.append style={mark size=6},
    major tick length=0.1cm,
    minor tick length=0.066cm,
    every legend image post/.append style={scale=0.8},
  },
}
```

As for `small`, it can be convenient to use `tiny,width=4.5cm` to adjust the width.

You will need `compat=1.3` or newer for this to work.

4.9.3 Scaling Strategies

The content of this section is quite involved – and its knowledge is typically unnecessary because by default, PGFPLOTS controls the involved stuff automatically. You may want to skip this section.

`/pgfplots/scale mode=auto|none|stretch to fill|scale uniformly` (initially auto)

Specifies how to choose the (individual) unit vector scaling factors, their length ratios, and perhaps the axis limits in order to fill the prescribed `width` and `height`.

The `scale mode` implementation expects some “initial” set of unit vectors. This initial set of unit vectors is determined as follows: for standard two-dimensional axes, it is simply the unit cube $e_x = (1\text{pt}, 0\text{pt})^T$, $e_y = (0\text{pt}, 1\text{pt})^T$, $e_z = 0$. For three-dimensional axes, it is the outcome of the two keys `view` and `plot box ratio`. If you provided units explicitly by means of one of `x`, `y`, or `z`, this value is the initial unit vector.

In addition, it expects “initial” axis limits (i.e. values of `xmin`, `xmax`, etc.). The initial axis limits are those limits which have been deduced from your data or which have been provided explicitly. Furthermore, the initial axis limits already include changes of the `enlargelimits` key.

Given the initial set of unit vectors and the initial axis limits, the `scale mode` implementation is a kind of “post-processor” which creates modified unit vectors and modified axis limits in order to satisfy all specified constraints. These constraints are `width`, `height`, and `unit vector ratio`.

The initial choice `auto` tells PGFPLOTS to take full control over this key. It chooses one of the other possible choices depending on the actual context. The choice `auto` evaluates to `scale uniformly` if `unit vector ratio` is set. Otherwise it evaluates to `stretch to fill`.

The choice `none` does not apply any rescaling at all. Use this if prescribed lengths of `x`, `y` (and perhaps `z`) should be used. In other words: it ignores `width` and `height`. In this case, you may want to set `x post scale` and its variants to rescale units manually. See also `disabledatascaling`.

The choice `stretch to fill` takes the initial unit vectors and rescales the unit vectors with two *separate* scales: one which results in the proper `width` and one which results in the proper `height`. As a consequence, the unit vectors are modified and distorted such that the final image fits into the prescribed dimensions. This is usually what one expects unless one provides unit directions explicitly. This mode does not change axis limits. Note that if one of the unit vectors has been provided explicitly, PGFPLOTS will not change it. It will only change the remaining axis limits. This mode contradicts `axis equal` or `unit vector ratio`.

The choice `scale uniformly` takes the initial unit vectors and applies only *one* scaling factor to all units. In this case, there is just *one* common scaling factor for both `width` and `height`. Naturally, this will result in unsatisfactory results because either the final width or the final height will not be met. Therefore, this choice will adjust axis limits to get the desired dimensions. Thus, the unit vectors have exactly the same size *relations and angles* as they had before the scaling; only their magnitude is changed uniformly. In addition, axis limits may be changed (with individual scaling factors for each axis limit). Note that if unit vectors have been provided explicitly, PGFPLOTS can still rescale it with this choice – it will keep the relative directions and size ratios. The choice `scale uniformly` tries its best to modify the degrees of freedom in a “useful” way. The precise meaning of “useful” is the `scale uniformly strategy` key.

`/pgfplots/scale uniformly strategy=auto|units only|change vertical limits|change horizontal limits` (initially auto)

The `scale uniformly` method requires to determine one *common* scaling factor which rescales every axis *unit*. In addition, it allows one scaling factor *for each axis limit*, i.e. up to three.

The constraints for this search are that we want to satisfy the `width/height` constraint, have as few rescaling as possible and that we do not want to reduce limits (as this could possibly hide data points).

The choice `auto` chooses one of the other possibilities automatically. Depending on whether we have two dimensions or three dimensions, it compares the available methods and chooses the one which does not reduce limits and which involves the fewest rescaling (i.e. it may compare the outcome of

the other strategies). This is the default. If you keep the choice `auto`, you do not have to worry about the remaining choices. Note that manually provided axis limits will not be modified.

The choice `units only` will not enlarge axis limits. It will only rescale the units. To this end, it chooses the scaling factor such that the *smaller* target dimension is filled as desired. In other words: if `width < height`, it will scale to satisfy the `width` constraint. The `height` constraint will be ignored. The case `>` will be done the other way round. The choice `units only` typically results in a square axis as it takes the initial set of unit vectors (which are typically the unit box) and scales them with a common scaling factor. Consequently, you can choose `units only` if you want a boxed axis. You can still change axis limits manually, however.

The choice `change vertical limits` chooses a common scaling factor for the unit vectors on order to satisfy the `width` (!) constraint. This common scaling factor is similar to `units only` – but `units only` can also decide to satisfy the `height` constraint whereas `change vertical limits` will scale unit vectors to satisfy `width`. In order to satisfy the `height` constraint, `change vertical limits` modifies just the vertical limits. For two-dimensional axes, this is `ymin` and `ymax`. For three-dimensional axes, this is `zmin` and `zmax`. Clearly, there is a chance that it will *decrease* the displayed range – in this case, parts of the image will be clipped away. This method *assumes* that the vertical axis has not been rotated (i.e. that $e_{yx} = 0$ or $e_{zx} = 0$, respectively). It refuses to work and falls back to `units only` for rotated axes. Choose `change vertical limits` if you want the image (i.e. the actual content) as wide as possible. You can modify `width` and `height` to improve its outcome. Note that manually specified axis limits will not be changed, see below for details.

The choice `change horizontal limits` attempts a similar approach, but for the horizontal limits: it determines one suitable scaling factor which is applied to all unit vectors and modifies horizontal axis limits to satisfy the remaining constraints. For two-dimensional axes, this is quite simple because we typically have $e_{xy} = 0$ (i.e. the x unit vector has vanishing y component) and $e_{yx} = 0$ such that PGFPLOTS can change axis limits easily. If a two-dimensional axis has an x unit with $e_{xy} \neq 0$, the method is not applicable and falls back to `units only`. For three-dimensional axes, it assumes that the z vector is not rotated, i.e. $e_{zy} = 0$ and tries to change limits for both x and y . This choice is much more involved because here, x and y components are coupled. Consequently, the common unit scaling factor and the two involved axis limit compensation factors for x and y are tightly coupled as well. PGFPLOTS solves a system of non-linear equations iteratively to arrive at a suitable solution for all three scalings. Use this method if `change vertical limits` would clip away parts of the image (because it reduced the displayed range) and you do not want to change `width` and `height`. The choice `change horizontal limits` will typically result in more empty space in the resulting figure. But it will not clip away content. Manually specified axis limits will not be changed, see below for details.

Manually provided axis limits: Any manually provided arguments for `xmin` and its variants are considered to be immutable; PGFPLOTS will not change them. If you assign `xmin`, PGFPLOTS will only change `xmax` and vice-versa. If you assign both `xmin` and `xmax`, PGFPLOTS will not change x limits at all. Note that if you assign both `xmin` and `xmax`, PGFPLOTS will simply skip the scaling and will give up on the constraints. It will not try to compensate the lack of scaling opportunities by changing y limits, for example. This has the positive effect that assigning limits does not change the complete appearance of your axis. The allowed set of changes to axis limits can be configured with the following key.

Interaction with `enlargelimits`: Note that `enlargelimits` and `scale mode` are independent of another: the outcome of `enlargelimits` is used as “initial axis limits” and these limits may be changed by `scale mode` (even if you said `enlargelimits=false`). See the documentation of `enlargelimits` for details on this interaction.

`/pgfplots/unit rescale keep size=true|false|unless limits declared` (initially unless limits declared)

In the default configuration `unless limits declared`, unit rescaling may cause changes to the axis limits in order to keep the figure’s size intact. However, only those limits which have not been declared manually are subject to rescaling: if you say `xmin=1`, only `xmax` and the limits for y and z are free to change.

Setting `unit rescale keep size=false` will *disable* the modification of axis limits altogether, i.e. axis limits will not be rescaled to compensate scalings on unit vectors.

Setting `unit rescale keep size=true` will always rescale limits, even if they have been declared manually.

This key mainly affects `scale mode=scale` uniformly. This, in turn, is used for `axis equal` and `\addplot3` graphics.

See also the addition documentation for this key and related examples on page 216.

The `scale uniformly` choice is implicitly used for `axis equal` and for the `\addplot3` graphics feature, see the documentation in Section 4.2.8 on page 45 for its examples. Note that the common case is that the initial unit vectors form the unit cube (i.e. those before scaling, see above). In this case, `scale uniformly` is the same as `axis equal`.

4.10 3D Axis Configuration

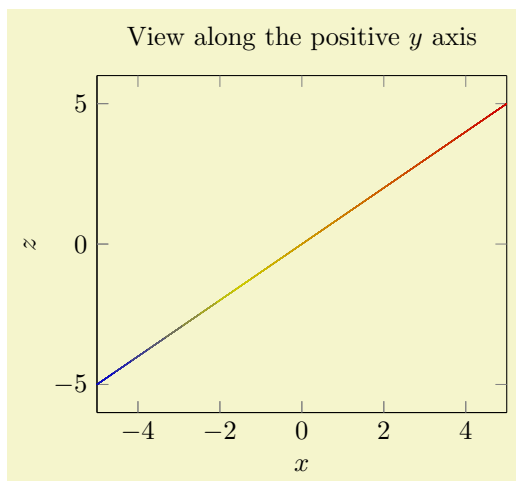
This section describes keys which are used to configure the appearance of three dimensional figures. Some of them apply for two-dimensional plots as special case as well, and they will also be discussed in the respective sections of this manual.

4.10.1 View Configuration

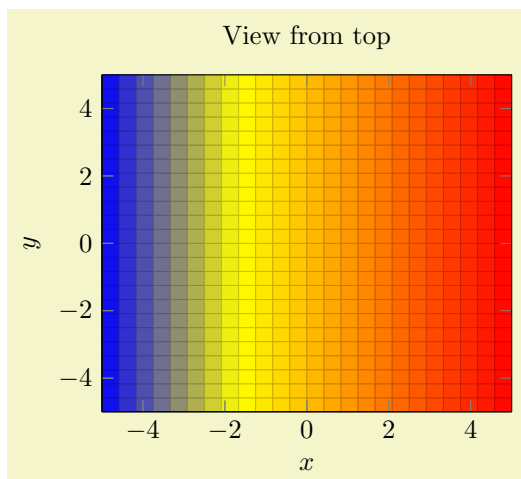
`/pgfplots/view={⟨azimuth⟩}{⟨elevation⟩}`

(initially {25}{30})

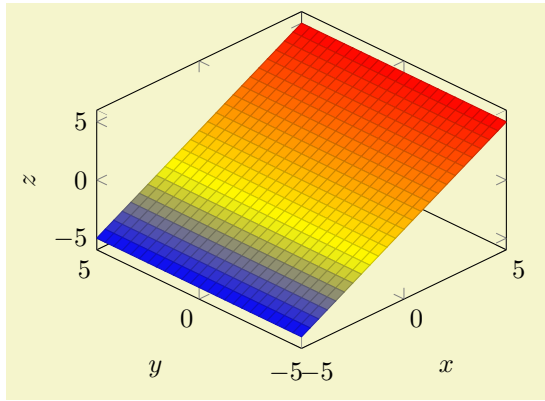
Changes both view angles of a 3D axis. The azimuth (first argument) is the horizontal angle which is rotated around the z axis. For a 3D plot, the z axis always points to the top. The elevation (second argument) is the vertical rotation around the (rotated) x axis. Positive elevation values indicate a view from above, negative a view from below. All values are measured in degree.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={0}{0},
    xlabel=$x$,
    ylabel=$y$,
    title=View along the positive $y$ axis]
    \addplot3[surf] {x};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={0}{90},
    xlabel=$x$,
    ylabel=$y$,
    title=View from top]
    \addplot3[surf] {x};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view={-45}{45},
    xlabel=$x$,ylabel=$y$,zlabel=$z$]
    \addplot3[surf] {x};
  \end{axis}
\end{tikzpicture}
```

The `view` is computed as follows. The view is defined by two rotations: the first rotation uses the $\langle azimuth \rangle$ angle to rotate around the z axis. Afterwards, the view is rotated $\langle elevation \rangle$ degrees around the *rotated* x axis (more precisely, it is rotated $-\langle elevation \rangle$ degrees). The resulting transformed x - z -plane is the viewport, i.e. the view direction is always the transformed positive y axis.

The `view` argument is compatible with the argument of the Matlab ($\text{\textcircled{R}}$) `view` command, i.e. you can use

```
[h,v] = view
```

in matlab and pack the resulting arguments into PGFPLOTS⁴⁹.

If you work with `gnuplot`, you can convert the view arguments as follows: the `gnuplot` command

```
set view v,h
```

is equivalent to `view={h}{90-v}`. For example, the default `gnuplot` configuration `set view 60,60` is equivalent to `view={60}{30}` in PGFPLOTS.

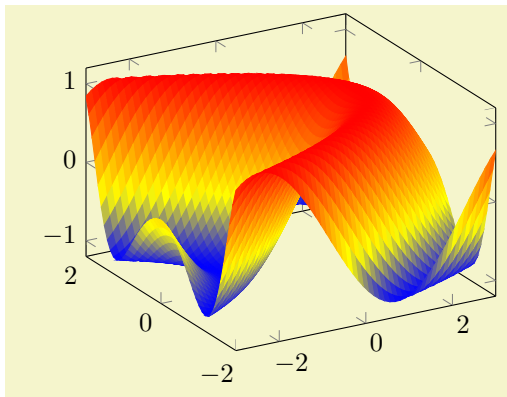
The `view` is (currently) always an orthogonal projection, no perspective is possible, yet. You can, however, specify projection unit vectors for x , y , and z explicitly to get a skewed three-dimensional axis.

```
/pgfplots/view/az={\langle azimuth \rangle}
```

```
/pgfplots/view/h={\langle azimuth \rangle}
```

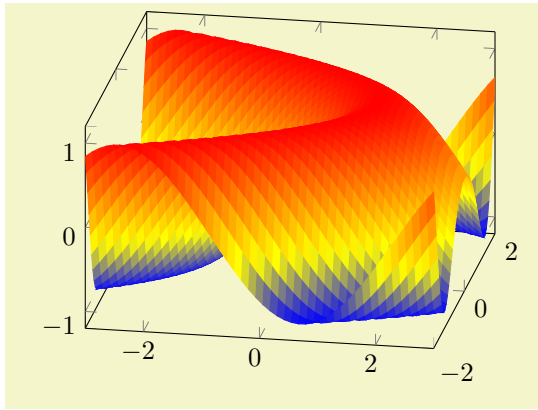
(initially 25)

Changes only the azimuth view angle, i.e. the horizontal (first) view angle which is rotated around the z axis.

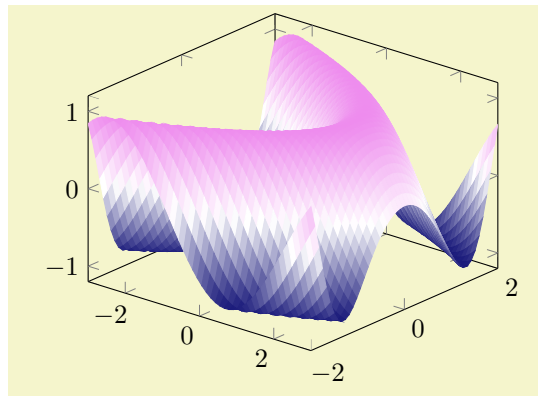


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view/h=-30]
    \addplot3[
      surf,
      % shader=interp,
      shader=flat,
      samples=50,
      domain=-3:3,y domain=-2:2]
      {sin(deg(x+y^2))};
  \end{axis}
\end{tikzpicture}
```

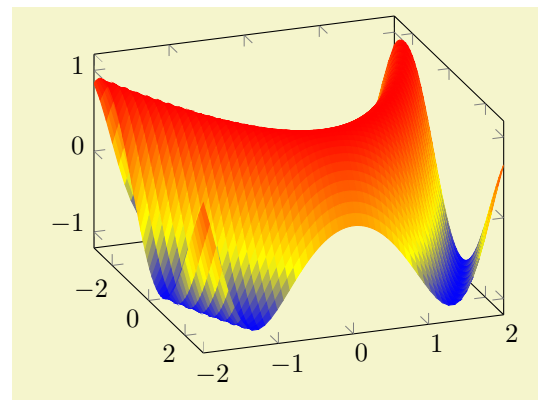
⁴⁹In case it does not work, try `h` and `-v` in PGFPLOTS.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view/h=10]
    \addplot3[
      surf,
      % shader=interp,
      shader=flat,
      samples=50,
      domain=-3:3,y domain=-2:2]
      {sin(deg(x+y^2))};
    \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view/h=40,colormap/violet]
    \addplot3[
      surf,
      % shader=interp,
      shader=flat,
      samples=50,
      domain=-3:3,y domain=-2:2]
      {sin(deg(x+y^2))};
    \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[view/h=70]
    \addplot3[
      surf,
      % shader=interp,
      shader=flat,
      samples=50,
      domain=-3:3,y domain=-2:2]
      {sin(deg(x+y^2))};
    \end{axis}
\end{tikzpicture}
```

`/pgfplots/view/el={elevation}`

`/pgfplots/view/v={elevation}`

(initially 30)

Changes only the vertical elevation, i.e. the second argument to `view`. Positive values view from above, negative values from below.

4.10.2 Styles Used Only For 3D Axes

`/pgfplots/every 3d description`

(style, no value)

This style allows to change the appearance of *descriptions* for three dimensional axes. Naturally, a three dimensional axis will display axis labels for x and y differently than a two dimensional axis (for example, the y axis label won't be rotated by 90 degrees). The `every 3d description` style installs the necessary display options for three dimensional axis descriptions.

The initial value is:

```

\pgfkeys{
  /pgfplots/every 3d description/.style={
    % Only these description styles can be changed here:
    every axis x label/.style={at={(\ticklabel cs:0.5)},
      anchor=near ticklabel},
    every axis y label/.style={at={(\ticklabel cs:0.5)},
      anchor=near ticklabel},
    every x tick scale label/.style={
      at={(\xticklabel cs:0.95,5pt)},
      anchor=near xticklabel,inner sep=0pt},
    every y tick scale label/.style={
      at={(\yticklabel cs:0.95,5pt)},
      anchor=near yticklabel,inner sep=0pt},
    try min ticks=3,
  }%
}

```

As the name suggests, `every 3d description` can only be used to set styles for axis labels, tick labels and titles. It has *not* been designed to reset other styles, you will need to change these options either for each axis separately or by means of user defined styles. The reason for this limitation is: other options can (and, in many cases, needs to) be set before the axis is processed. However, the decision whether we have a two dimensional or a three dimensional axis has to be postponed until the processing is more or less complete – so only some remaining keys can be set.

`/pgfplots/every 3d view {<h>}{<v>}` (style, no value)

A style which can be used for fine-tuning of the output for specific views.

This style will be installed right after `every 3d description`, but before other axis description related keys are set (in other words: it has higher precedence than `every 3d description`, but lower precedence than keys provided to the axis directly).

One example is preconfigured for `view={0}{90}` (from top):

```

\pgfplotsset{
  /pgfplots/every 3d view {0}{90}/.style={
    xlabel near ticks,
    ylabel near ticks,
    axis on top=true
  }
}

```

4.10.3 Appearance Of The 3D Box

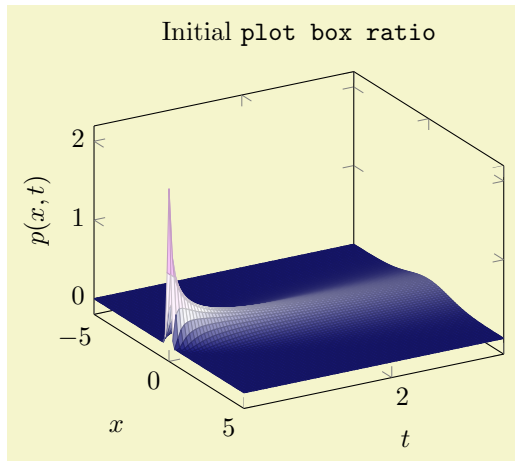
`/pgfplots/plot box ratio={<x stretch> <y stretch> <z stretch>}` (initially 1 1 1)

Allows to customize the aspect ratio between the three different axes in a three dimensional plot.

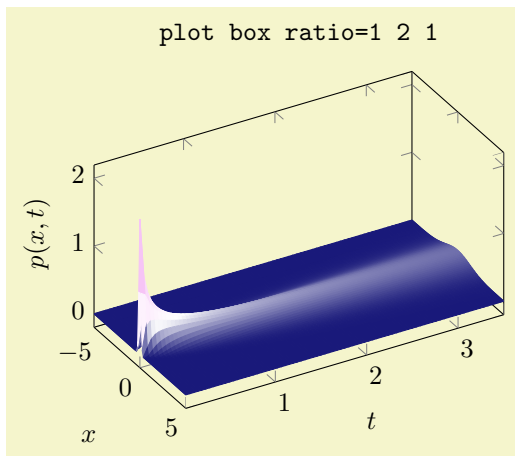
Note that this key is different from the related `unit vector ratio`: the plot box is only useful for three dimensional axes, and it will usually distort the unit vector ratios. If you want equal unit ratios, consider using `unit vector ratio`.

The `plot box ratio` is applied before any rotations and stretch-to-fill routines have been invoked. Thus, the initial setting⁵⁰ `1 1 1` makes all axes equally long before the stretch-to-fill routine is applied.

⁵⁰Note that you can also use the syntax `{1}{1}{1}` instead of space-separation.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    view/h=60,
    plot box ratio=1 1 1,
    colormap={violet}{[1cm] rgb255(0cm)=(25,25,122)
        color(1cm)=(white) rgb255(5cm)=(238,140,238)},
    xlabel=$x$,
    ylabel=$t$,
    zlabel={$p(x,t)$},
    shader=faceted,
    title=Initial \texttt{plot box ratio},
]
\addplot3[surf,y domain=0.02:3.5,samples=81]
    {1/(2*sqrt(pi*y)) * exp(0-x^2/y)};
    % the '0' is a work-around for a bug in PGF 2.00
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    view/h=60,
    plot box ratio=1 2 1,
    colormap={violet}{[1cm] rgb255(0cm)=(25,25,122)
        color(1cm)=(white) rgb255(5cm)=(238,140,238)},
    xlabel=$x$,
    ylabel=$t$,
    zlabel={$p(x,t)$},
    shader=flat,
    title=\texttt{plot box ratio=1 2 1},
]
\addplot3[surf,y domain=0.02:3.5,samples=81]
    {1/(2*sqrt(pi*y)) * exp(0-x^2/y)};
    % the '0' is a work-around for a bug in PGF 2.00
\end{axis}
\end{tikzpicture}
```

This key applies only to three dimensional axes. After the scaling, the axes will be stretched to fill the `width` and `height` for this plot. Thus, the effects of `plot box ratio` might be undone by this stretching for particular views.

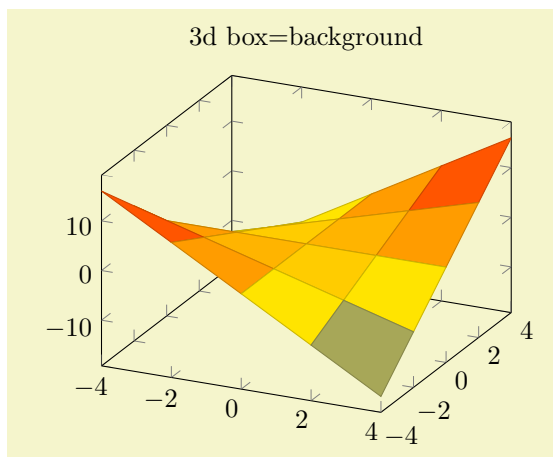
`/pgfplots/3d box=background|complete|complete*`

(initially `background`)

Allows to configure the appearance of boxed three dimensional axes.

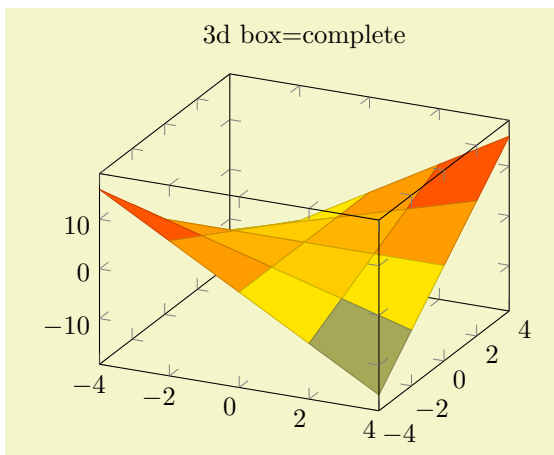
Type only `3d box` (without value) as alias for `3d box=complete`.

The choice `background` is the initial setting, it does not draw axis lines (and grid lines) which are in the foreground.



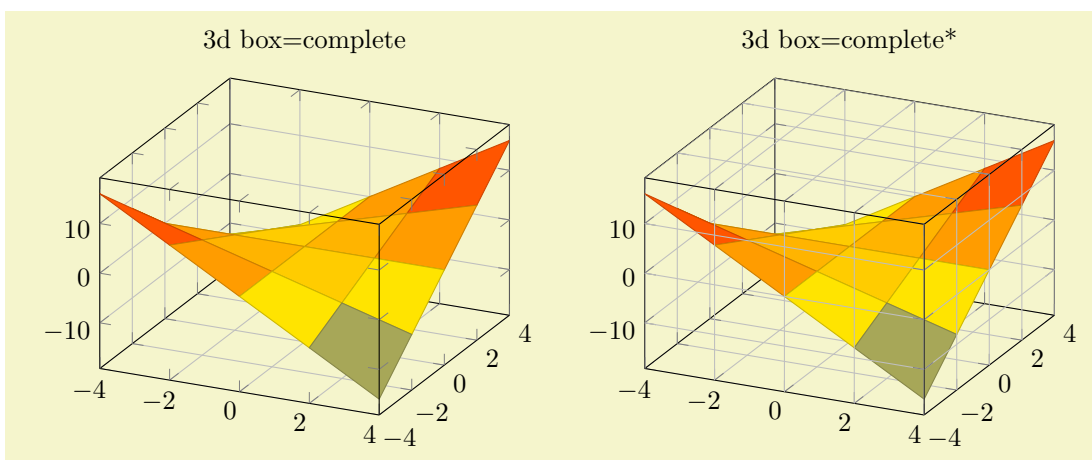
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    3d box=background,
    % pretty printing, but irrelevant:
    title={3d box=background},
    samples=5,
    domain=-4:4,
    xtick=data,
    ytick=data,
]
\addplot3[surf] {x*y};
\end{axis}
\end{tikzpicture}
```

The choice `complete` also draws axis lines and tick lines in the foreground, but it doesn't draw grid lines in the foreground. The result yields a complete box:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    3d box,% same as 3d box=complete
    % pretty printing, but irrelevant:
    title={3d box=complete},
    samples=5,
    domain=-4:4,
    xtick=data,
    ytick=data,
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```

Finally, the choice **complete*** is the same as **complete**, but it also draws grid lines in the foreground.



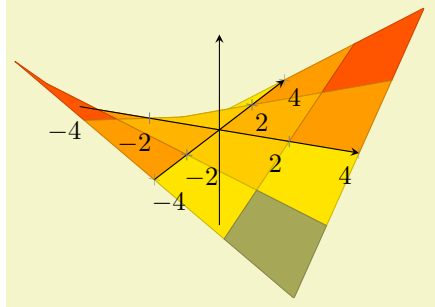
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    3d box=complete,
    grid=major,
    title={3d box=complete},
    samples=5, domain=-4:4,
    xtick=data, ytick=data,
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}%
~
\begin{tikzpicture}
  \begin{axis}[
    3d box=complete*,
    grid=major,
    title={3d box=complete*},
    samples=5, domain=-4:4,
    xtick=data, ytick=data,
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```

Before any foreground parts are actually processed, the style **every 3d box foreground** will be installed. This allows to change the appearance of foreground axis components like **tick style** or **axis line style** separately from the background components.

Note that **3d box=complete** is *only* available for boxed axes, i.e. together with **axis lines=box**. It is an error to use a different combination.

4.10.4 Axis Line Variants

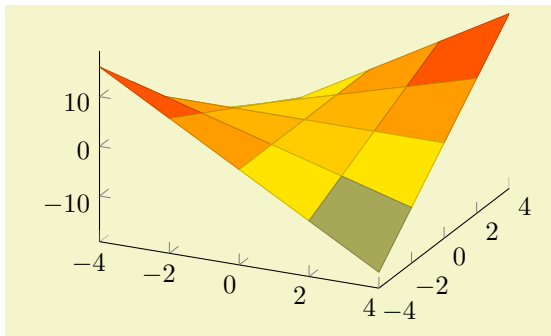
Three dimensional axes also benefit from the `axis lines=box` or `axis lines=center` styles discussed in Section 4.8.9. The choice `axis lines=box` is standard, it draws a box (probably affected by the `3d box=complete` key). The choice `axis lines=center` draws all three axes such that they pass through the origin. It might be necessary to combine this key with `axis on top` as there is no depth information.



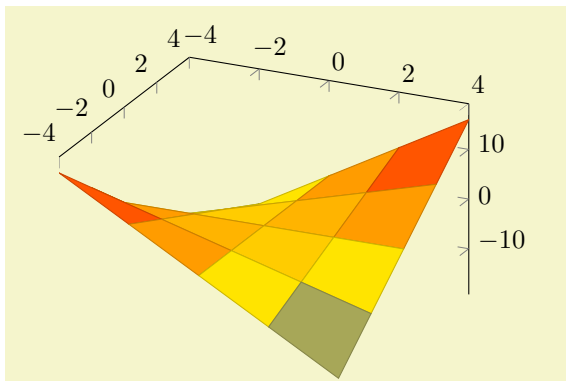
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% Attention: use compat=1.8 or higher
% to repair label positions
\begin{tikzpicture}
  \begin{axis}[
    axis lines=center,
    axis on top,
    samples=5, domain=-4:4,
    xtick=data, ytick=data,
    ztick=\empty, % no z ticks here
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```

The remaining choices `axis lines*=left` and `axis lines*=right` select different sets of axes in a way such that tick labels and axis label won't disturb the plot's content. The '*' suppresses the use of special styles which are mainly adequate for two-dimensional axes, see the documentation of `axis lines`. Such a set of axes is always on the boundary of the two-dimensional projection.

The choice `axis lines*=left` chooses a set of axes which are on the left (or bottom, respectively) whereas the choice `axis lines*=right` chooses a set of axes which are on the right (or top, respectively):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    axis lines*=left,
    samples=5, domain=-4:4,
    xtick=data, ytick=data,
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    axis lines*=right,
    samples=5, domain=-4:4,
    xtick=data, ytick=data,
  ]
    \addplot3[surf] {x*y};
  \end{axis}
\end{tikzpicture}
```

It is not possible to mix different styles like `axis x line=center, axis z line=top`.

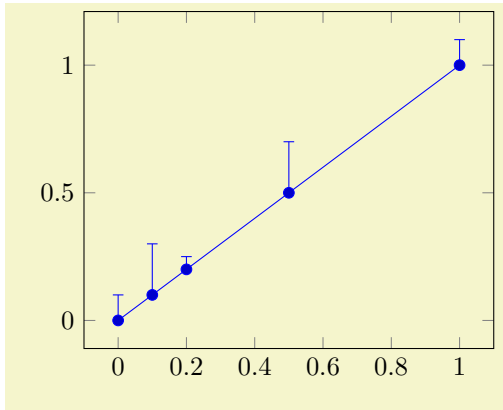
4.11 Error Bars

PGFplots supports error bars for normal and logarithmic plots.

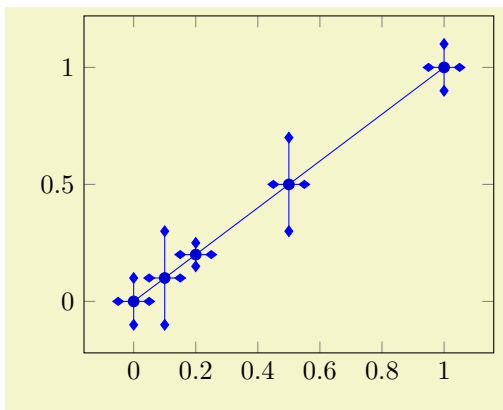
Error bars are enabled for each plot separately, using `<options>` after `\addplot`:

```
\addplot+[error bars/.cd,x dir=both,y dir=both] ...
```

Error bars inherit all drawing options of the associated plot, but they use their own marker and style arguments additionally.

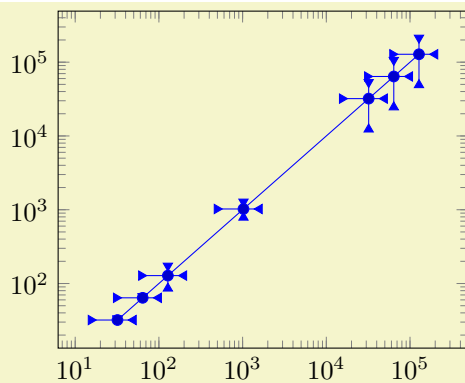


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[error bars/.cd,
  y dir=plus,y explicit]
coordinates {
  (0,0)      +- (0.5,0.1)
  (0.1,0.1) +- (0.05,0.2)
  (0.2,0.2) +- (0,0.05)
  (0.5,0.5) +- (0.1,0.2)
  (1,1)      +- (0.3,0.1)};
\end{axis}
\end{tikzpicture}
```



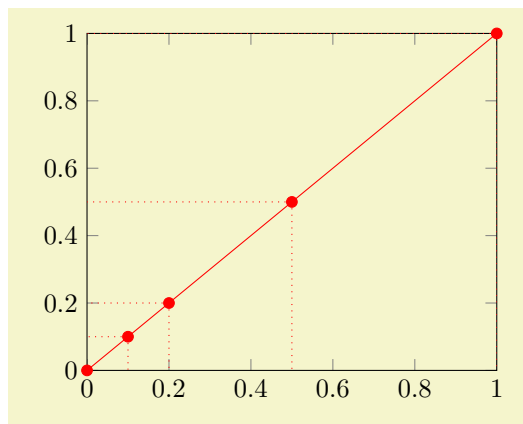
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[error bars/.cd,
  y dir=both,y explicit,
  x dir=both,x fixed=0.05,
  error mark=diamond*]
coordinates {
  (0,0)      +- (0.5,0.1)
  (0.1,0.1) +- (0.05,0.2)
  (0.2,0.2) +- (0,0.05)
  (0.5,0.5) +- (0.1,0.2)
  (1,1)      +- (0.3,0.1)};
\end{axis}
\end{tikzpicture}
```

x	y	errorx	errory
32	32	0	0
64	64	0	0
128	128	0	0.3
1,024	1,024	0	0.2
32,068	32,068	0	0.6
64,000	64,000	0	0.6
$1.28 \cdot 10^5$	$1.28 \cdot 10^5$	0	0.6



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotstabletypeset{pgfplots.testtable2.dat}

\begin{tikzpicture}
\begin{loglogaxis}
\addplot+[error bars/.cd,
  x dir=both,x fixed relative=0.5,
  y dir=both,y explicit relative,
  error mark=triangle*]
table[x=x,y=y,y error=error]
{pgfplots.testtable2.dat};
\end{loglogaxis}
\end{tikzpicture}
```

```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[enlargelimits=false]
\addplot[red,mark=*]
plot[error bars/.cd,
y dir=minus,y fixed relative=1,
x dir=minus,x fixed relative=1,
error mark=none,
error bar style={dotted}]
coordinates
{(0,0) (0.1,0.1) (0.2,0.2)
(0.5,0.5) (1,1)};
\end{axis}
\end{tikzpicture}
```

```
/pgfplots/error bars/x dir=none|plus|minus|both (initially none)
/pgfplots/error bars/y dir=none|plus|minus|both (initially none)
/pgfplots/error bars/z dir=none|plus|minus|both (initially none)
```

Draws either no error bars at all, only marks at $x + \epsilon_x$, only marks at $x - \epsilon_x$ or marks at both, $x + \epsilon_x$ and $x - \epsilon_x$. The x -error ϵ_x is acquired using one of the following options.

The same holds for the `y dir` option.

```
/pgfplots/error bars/x fixed={⟨value⟩} (initially 0)
/pgfplots/error bars/y fixed={⟨value⟩} (initially 0)
/pgfplots/error bars/z fixed={⟨value⟩} (initially 0)
```

Provides a common, absolute error $\epsilon_x = \langle value \rangle$ for all input coordinates.

For linear x axes, the error mark is drawn at $x \pm \epsilon_x$ while for logarithmic x axes, it is drawn at $\log(x \pm \epsilon_x)$. Computations are performed in PGF's floating point arithmetics.

```
/pgfplots/error bars/x fixed relative={⟨percent⟩} (initially 0)
/pgfplots/error bars/y fixed relative={⟨percent⟩} (initially 0)
/pgfplots/error bars/z fixed relative={⟨percent⟩} (initially 0)
```

Provides a common, relative error $\epsilon_x = \langle percent \rangle \cdot x$ for all input coordinates. The argument $\langle percent \rangle$ is thus given relatively to input x coordinates such that $\langle percent \rangle = 1$ means 100%.

Error marks are thus placed at $x \cdot (1 \pm \epsilon_x)$ for linear axes and at $\log(x \cdot (1 \pm \epsilon_x))$ for logarithmic axes. Computations are performed in floating point for linear axis and using the identity $\log(x \cdot (1 \pm \epsilon_x)) = \log(x) + \log(1 \pm \epsilon_x)$ for logarithmic scales.

```
/pgfplots/error bars/x explicit (no value)
/pgfplots/error bars/y explicit (no value)
/pgfplots/error bars/z explicit (no value)
```

Configures the error bar algorithm to draw x -error bars at any input coordinate for which user-specified errors are available. Each error is interpreted as absolute error, see `x fixed` for details.

The different input formats of errors are described in Section 4.11.1.

```
/pgfplots/error bars/x explicit relative (no value)
/pgfplots/error bars/y explicit relative (no value)
/pgfplots/error bars/z explicit relative (no value)
```

Configures the error bar algorithm to draw x -error bars at any input coordinate for which user-specified errors are available. Each error is interpreted as relative error, that means error marks are placed at $x(1 \pm \langle value \rangle(x))$ (works as for `error bars/x fixed relative`).

```
/pgfplots/error bars/error mark=⟨marker⟩
```

Sets an error marker for any error bar. $\{\langle marker \rangle\}$ is expected to be a valid plot mark, see Section 4.6.

```
/pgfplots/error bars/error mark options={⟨key-value-list⟩}
```

Sets a key-value list of options for any error mark. This option works similarly to the TikZ '`mark options`' key.

`/pgfplots/error bars/error bar style={⟨key-value-list⟩}`

Appends the argument to `/pgfplots/every error bar` which is installed at the beginning of every error bar.

`/pgfplots/error bars/draw error bar/.code 2 args={⟨...⟩}`

Allows to change the default drawing commands for error bars. The two arguments are

- the source point, (x, y) and
- the target point, $(\tilde{x}, \tilde{y})$.

Both are determined by PGFPLOTS according to the options described above. The default code is

```
\pgfplotsset{
  /pgfplots/error bars/draw error bar/.code 2 args={%
    \pgfkeysgetvalue{/pgfplots/error bars/error mark}%
    {\pgfploterrorbarsmark}%
    \pgfkeysgetvalue{/pgfplots/error bars/error mark options}%
    {\pgfploterrorbarsmarkopts}%
    \draw #1 -- #2 node[pos=1,sloped,allow upside down] {%
      \expandafter\tikz\expandafter[\pgfploterrorbarsmarkopts]{%
        \expandafter\pgfuseplotmark\expandafter{\pgfploterrorbarsmark}%
        \pgfusepath{stroke}}}%
    };
}
```

4.11.1 Input Formats of Error Coordinates

Error bars with explicit error estimations for single data points require some sort of input format. This applies to `'error bars/⟨[xy]⟩ explicit'` and `'error bars/⟨[xy]⟩ explicit relative'`.

Error bar coordinates can be read from `'plot coordinates'` or from `'plot table'`. The inline plot coordinates format is

```
\addplot coordinates {
  (1,2) +- (0.4,0.2)
  (2,4) +- (1,0)
  (3,5)
  (4,6) +- (0.3,0.001)
}
```

where $(1, 2) \pm (0.4, 0.2)$ is the first coordinate, $(2, 4) \pm (1, 0)$ the second and so forth. The point $(3, 5)$ has no error coordinate.

The `'plot table'` format is

```
\addplot table[x error=COLNAME,y error=COLNAME]
```

or

```
\addplot table[x error index=COLINDEX,y error index=COLINDEX]
```

These options are used like the `'x'` and `'x index'` options.

You can supply error coordinates even if they are not used at all; they will be ignored silently in this case.

4.12 Number Formatting Options

PGFPLOTS typesets tick labels rounded to given precision and in configurable number formats. The command to do so is `\pgfmathprintnumber`; it uses the current set of number formatting options. In addition, PGFPLOTS might prepare tick numbers before they are handed over to `\pgfmathprintnumber`.

The options related to number printing as such are described in all detail in the manual for `PGFPLOT-STABLE`, which comes with PGFPLOTS. This section contains the reference for everything which is specific to an axis, and only a brief survey over the number formatting options as such.

4.12.1 Frequently Used Number Printing Settings

This section provides a brief survey about the most frequently used aspects of number formatting in PGFPLOTS.

1. PGFPLOTS computes common tick scaling factors like $\cdot 10^2$ and produces only integers as tick labels.
In order to get numbers like 0.001 as tick labels instead of 1 with a separate label $\cdot 10^{-3}$, you can use `scaled ticks=false` in your axis. See the description of `scaled ticks` for details.
2. In order to customize the way numbers are rounded and/or displayed, use something like `xticklabel style={/pgf/number format/.cd,fixed,precision=5}`.
Here is a short list of possibilities:

123.46	<code>\pgfmathprintnumber{123.456789}</code>
12,345.68	<code>\pgfmathprintnumber{12345.6789}</code>
12,345.6789	<code>\pgfmathprintnumber [fixed,precision=5]{12345.6789}</code>
12,345.67890	<code>\pgfmathprintnumber [fixed,fixed zerofill,precision=5]{12345.6789}</code>
12.345,67890	<code>\pgfmathprintnumber [fixed,fixed zerofill,precision=5,use comma] {12345.6789}</code>
$1.23 \cdot 10^4$	<code>\pgfmathprintnumber [sci]{12345.6789}</code>
$1.23457 \cdot 10^4$	<code>\pgfmathprintnumber [sci,sci zerofill,precision=5]{12345.6789}</code>
1.23×10^1	<code>\pgfmathprintnumber [sci,sci generic= {mantissa sep=\times,exponent={10^{#1}}}] {12.345}</code>
$\frac{1}{3}; \frac{1}{2}$	<code>\pgfmathprintnumber[frac]{0.333333333333333}; \pgfmathprintnumber[frac]{0.5}</code>
+2	<code>\pgfmathprintnumber[print sign]{2}</code>
1 000 000.123456	<code>\pgfmathprintnumber [1000 sep={\,},fixed,precision=6]{1000000.123456}</code>
1 000 000.123 456	<code>\pgfmathprintnumber[1000 sep={\,},fixed,precision=6, 1000 sep in fractionals] {1000000.123456}</code>

Each of these keys requires the prefix ‘`/pgf/number format/`’ when used inside of a PGFPLOTS style (try `/pgf/number format/.cd,<number formatting keys>` to use the same prefix for many *<number formatting keys>*).

The number formatting uses `\pgfmathprintnumber`, a PGF command to typeset numbers. A full reference of all supported options is shipped with PGFPLOTS: it is documented in the reference manual for `PGFPLOTS`TABLE, Section ‘Number Formatting Options’. The same reference can be found in the documentation for PGF.

Note that the number printer knows *nothing* about PGFPLOTS. In particular, it is not responsible for logs and their representation.

3. For a logarithmic axis, one may want to modify the number formatting style for the *exponent only*. In this case, redefine the style `log plot exponent style` (its documentation contains a couple of examples).
4. In order to get *fixed point* tick labels on a logarithmic axis, you can use `log ticks with fixed point` (see below).

4.12.2 PGFPlots-specific Number Formatting

This section contains fine-tuning options to change number formatting aspects – but only things which are specific to PGFPLOTS like peculiarities of tick labels on logarithmic axes. Consider browsing Section 4.12.1 first to see if you need this section.

`\pgfmathprintnumber{⟨x⟩}`

Generates pretty-printed output for the (real) number $\langle x \rangle$. The input number $\langle x \rangle$ is parsed using `\pgfmathfloatparsenumber` which allows arbitrary precision.

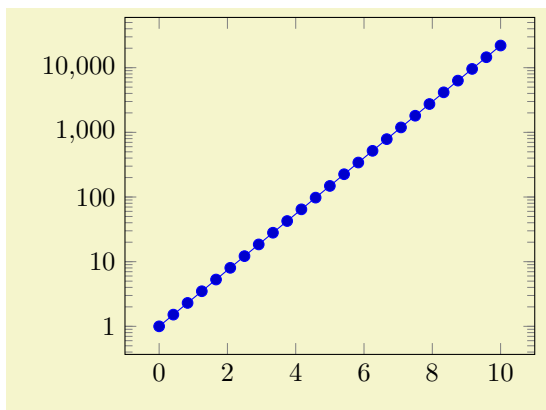
Numbers are typeset in math mode using the current set of number printing options, see below. Optional arguments can also be provided using `\pgfmathprintnumber[⟨options⟩]{⟨x⟩}`.

Please refer to the manual of `PGFPLOTS`TABLE (shipped with this package) for details about options related to number-printing.

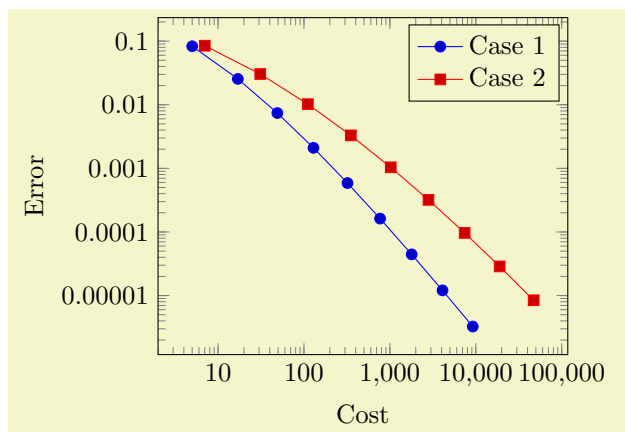
`/pgfplots/log ticks with fixed point`

(style, no value)

Reconfigures PGFPLOTS to display tick labels of logarithmic axes using *fixed point* numbers instead of the exponential style.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[log ticks with fixed point]
\addplot+[domain=0:10] {exp(x)};
\end{semilogyaxis}
\end{tikzpicture}
```

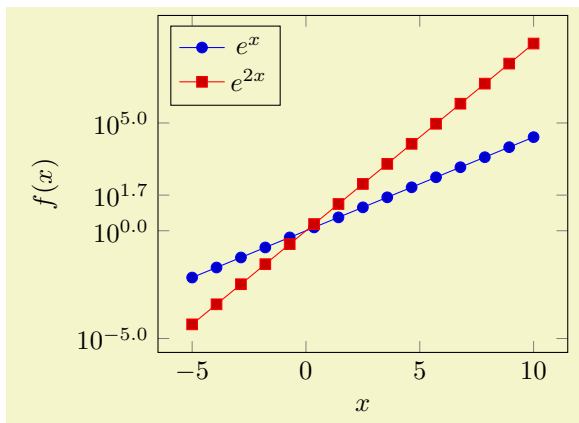


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
    log ticks with fixed point,
    xlabel=Cost,ylabel=Error]
\addplot coordinates {
    (5,      8.31160034e-02)
    (17,     2.54685628e-02)
    (49,     7.40715288e-03)
    (129,    2.10192154e-03)
    (321,    5.87352989e-04)
    (769,    1.62269942e-04)
    (1793,   4.44248889e-05)
    (4097,   1.20714122e-05)
    (9217,   3.26101452e-06)
};
\addplot coordinates {
    (7,      8.47178381e-02)
    (31,     3.04409349e-02)
    (111,    1.02214539e-02)
    (351,    3.30346265e-03)
    (1023,   1.03886535e-03)
    (2815,   3.19646457e-04)
    (7423,   9.65789766e-05)
    (18943,  2.87339125e-05)
    (47103,  8.43749881e-06)
};
\legend{Case 1,Case 2}
\end{loglogaxis}
\end{tikzpicture}
```

The style replaces `log number format basis`.

`/pgfplots/log plot exponent style={⟨key-value-list⟩}`

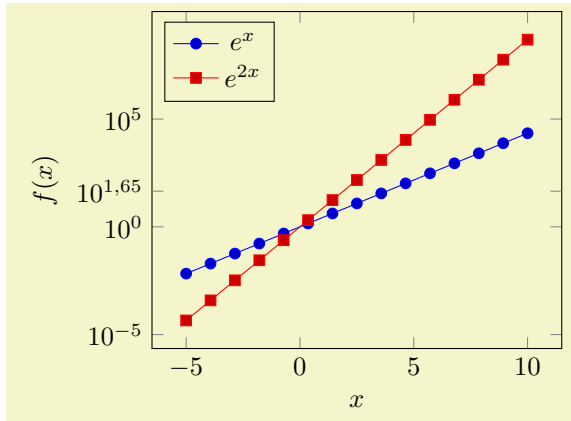
Allows to configure the number format of log plot exponents. This style is installed just before ‘`log number format basis`’ will be invoked. Please note that this style will be installed within the default code for ‘`log number format code`’.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{
    samples=15,
    width=7cm,
    xlabel=$x$,
    ylabel=$f(x)$,
    extra y ticks={45},
    legend style={at={(0.03,0.97)},
        anchor=north west}}
\begin{tikzpicture}
\begin{semilogyaxis}[
    log plot exponent style/.style={
        /pgf/number format/fixed zerofill,
        /pgf/number format/precision=1,
        domain=-5:10]

    \addplot {exp(x)};
    \addplot {exp(2*x)};

    \legend{$e^x$, $e^{2x}$}
\end{semilogyaxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{
  samples=15,
  width=7cm,
  xlabel=$x$,
  ylabel=$f(x)$,
  extra y ticks={45},
  legend style={at={(0.03,0.97)},
    anchor=north west}},
\begin{tikzpicture}
\begin{semilogyaxis}[
  log plot exponent style/.style={
    /pgf/number format/fixed,
    /pgf/number format/use comma,
    /pgf/number format/precision=2},
  domain=-5:10]

  \addplot {exp(x)};
  \addplot {exp(2*x)};

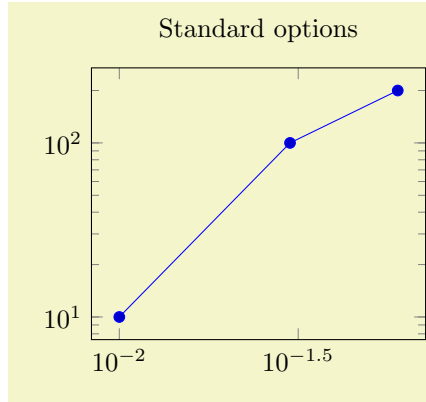
  \legend{$e^x$, $e^{2x}$}
\end{semilogyaxis}
\end{tikzpicture}
```

`/pgfplots/log identify minor tick positions=true|false` (initially false)

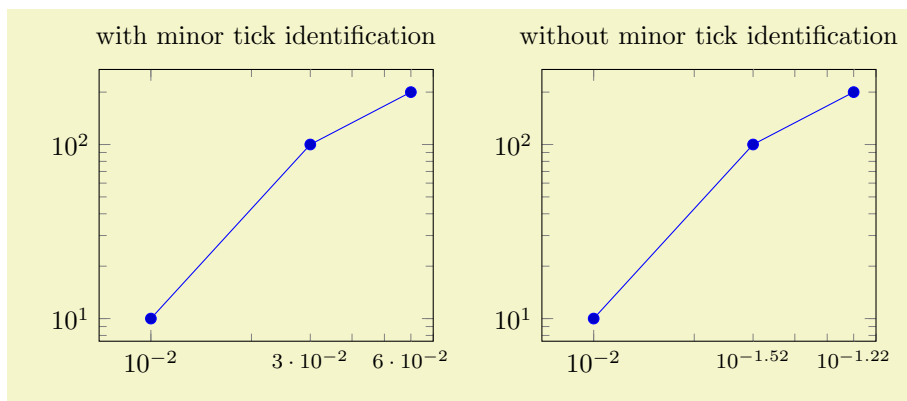
Set this to `true` if you want to identify log-plot tick labels at positions

$$i \cdot 10^j$$

with $i \in \{2, 3, 4, 5, 6, 7, 8, 9\}$, $j \in \mathbb{Z}$. This may be valuable in conjunction with the ‘`extra x ticks`’ and ‘`extra y ticks`’ options.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}%
\begin{loglogaxis}
  [title=Standard options,
  width=6cm]
\addplot coordinates {
  (1e-2,10)
  (3e-2,100)
  (6e-2,200)
};
\end{loglogaxis}
\end{tikzpicture}%
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={%
    width=6cm,
    xmin=7e-3,xmax=7e-2,
    extra x ticks={3e-2,6e-2},
    extra x tick style={major tick length=0pt,font=\footnotesize}
}}%

\begin{tikzpicture}%
  \begin{loglogaxis}[
    xtick={1e-2},
    title=with minor tick identification,
    extra x tick style={
      log identify minor tick positions=true}]
    \addplot coordinates {
      (1e-2,10)
      (3e-2,100)
      (6e-2,200)
    };
  \end{loglogaxis}
\end{tikzpicture}%

\begin{tikzpicture}%
  \begin{loglogaxis}[
    xtick={1e-2},
    title=without minor tick identification,
    extra x tick style={
      log identify minor tick positions=false}]
    \addplot coordinates {
      (1e-2,10)
      (3e-2,100)
      (6e-2,200)
    };
  \end{loglogaxis}%
\end{tikzpicture}%
```

This key is set by the default styles for extra ticks.

`/pgfplots/log number format code/.code={\langle... \rangle}`

Provides T_EX-code to generate log plot tick labels. Argument ‘#1’ is the (natural) logarithm of the tick position. The default implementation invokes `log base 10 number format code` after it changed the log basis to 10. It also checks the other log plot options.

This key will have a different meaning when the log basis has been chosen explicitly, see the `log basis x` key.

`/pgfplots/log base 10 number format code/.code={\langle... \rangle}`

Allows to change the overall appearance of base 10 log plot tick labels. The default implementation invokes `log number format basis={10}{#1}`.

Use `log plot exponent style` if you only want to change number formatting options for the exponent.

`/pgfplots/log number format basis/.code={\langle... \rangle}`

Typesets a logarithmic tick. The first supplied argument is the log basis, the second the exponent. The initial configuration is

```
\pgfplotsset{
  /pgfplots/log number format basis/.code 2 args={\pgfmathprintnumber{#2}}{#1}}
}
```

Use `log plot exponent style` if you only want to change number formatting options for the exponent.

4.13 Specifying the Plotted Range

```
/pgfplots/xmin={\langle coord \rangle}
/pgfplots/ymin={\langle coord \rangle}
/pgfplots/zmin={\langle coord \rangle}
/pgfplots/xmax={\langle coord \rangle}
```

```

/pgfplots/ymax={\coord}
/pgfplots/zmax={\coord}
/pgfplots/min={\coord}
/pgfplots/max={\coord}

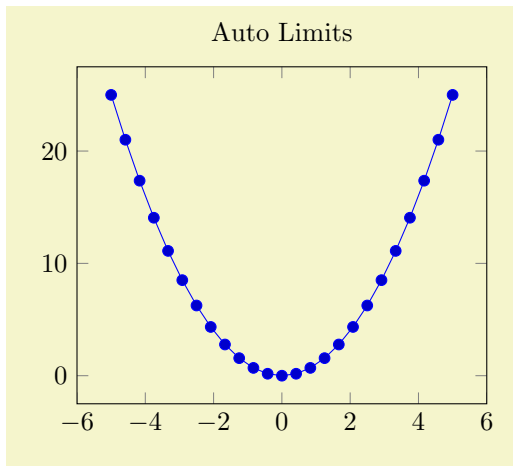
```

These options allow to define the axis limits, i.e. the lower left and the upper right corner. Everything outside of the axis limits will be clipped away.

Each of these keys is optional, and missing limits will be determined automatically from input data. Here, the `min` and `max` keys set limits for x , y and z to the same $\langle coord \rangle$.

If x -limits have been specified explicitly and y -limits are computed automatically, the automatic computation of y -limits will only considers points which fall into the specified x -range (and vice-versa). The same holds true if, for example, only `xmin` has been provided explicitly: in that case, `xmax` will be updated only for points for which $x \geq \text{xmin}$ holds. This feature can be disabled using `clip limits=false`.

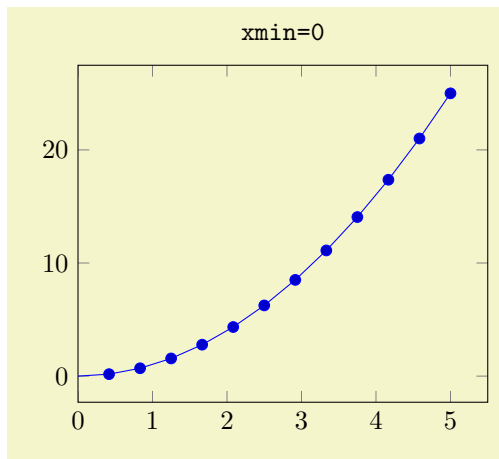
Axis limits can be increased automatically using the `enlargetimits` option.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[title=Auto Limits]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}

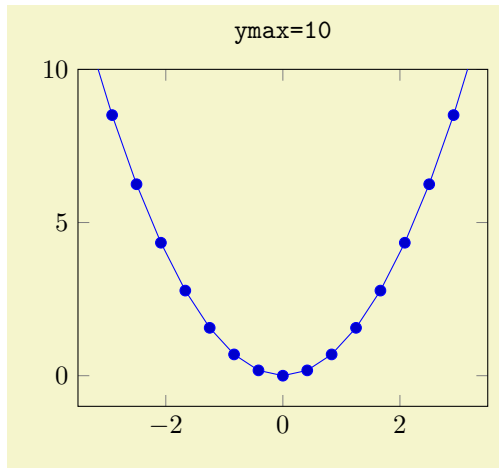
```



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[title={\texttt{xmin=0}},xmin=0]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}

```

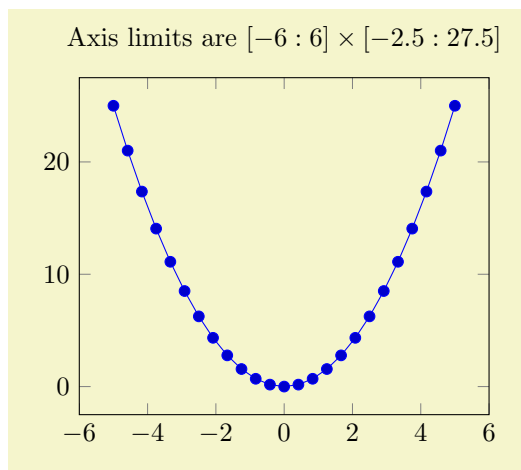



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[title={\texttt{ymax=10}},ymax=10]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

Note that even if you provide `ymax=10`, data points with $y > 10$ will still be visualized – producing a line which leaves the plotted range.

See also the `restrict x to domain` and `restrict x to domain*` keys – they allow to discard or clip input coordinates which are outside of some domain, respectively.

During the visualization phase, i.e. during `\end{axis}`, these keys will be set to the final axis limits. You can access the values by means of `\pgfkeysvalueof{/pgfplots/xmin}`, for example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    % Show (automatically) computed limits:
    title={
      Axis limits are
      $
      [\pgfmathprintnumber{\pgfkeysvalueof{/pgfplots/xmin}}
      : \pgfmathprintnumber{\pgfkeysvalueof{/pgfplots/xmax}}
      ] \times
      [\pgfmathprintnumber{\pgfkeysvalueof{/pgfplots/ymin}}
      : \pgfmathprintnumber{\pgfkeysvalueof{/pgfplots/ymax}}
      ] $ ,
    ]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

This access is possible inside of any axis description (like `xlabel`, `title`, `legend entries` etc.) or any annotation (i.e. inside of `\node`, `\draw` or `\path` and coordinates in `(axis cs:<x>,<y>)`), but not inside of `\addplot` (limits may not be complete at this stage).

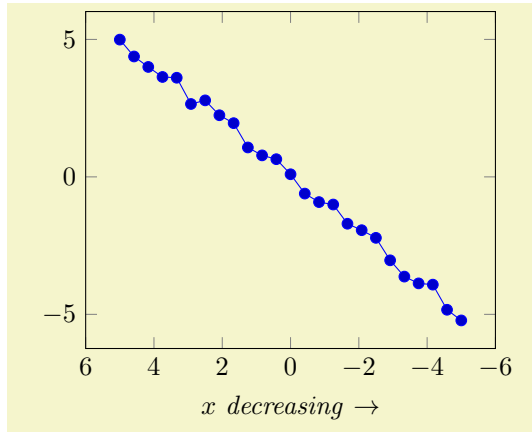
```
/pgfplots/xmode=normal|linear|log (initially normal)
/pgfplots/ymode=normal|linear|log (initially normal)
/pgfplots/zmode=normal|linear|log (initially normal)
```

Allows to choose between linear (=normal) or logarithmic axis scaling or logplots for each x, y, z -combination.

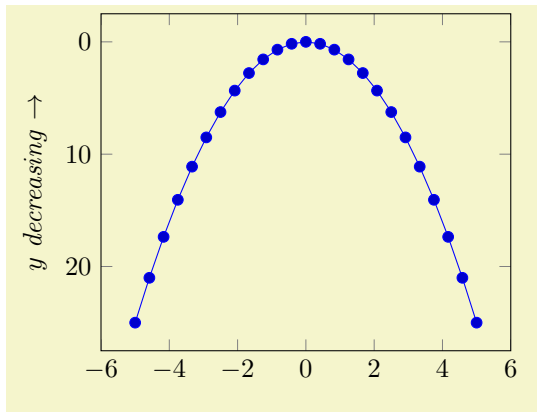
Logarithmic plots use the current setting of `log basis x` and its variants to determine the basis (default is e).

```
/pgfplots/x dir=normal|reverse (initially normal)
/pgfplots/y dir=normal|reverse (initially normal)
/pgfplots/z dir=normal|reverse (initially normal)
```

Allows to reverse axis directions such that values are given in decreasing order.

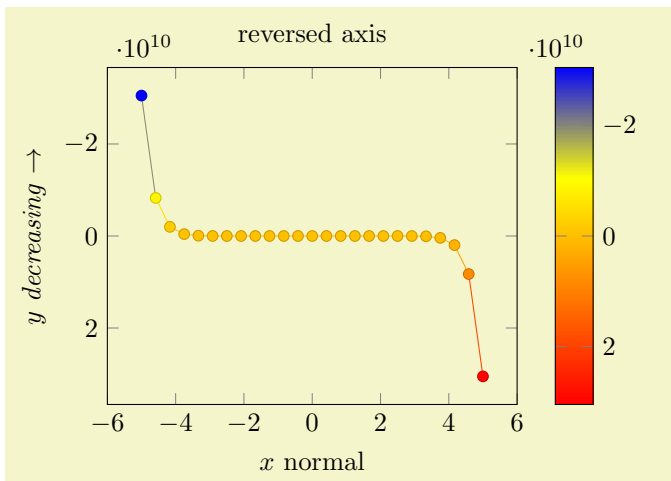


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xlabel=$x$ \emph{decreasing} $\to$,
  x dir=reverse]
\addplot {x+rand*0.3};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ylabel=$y$ \emph{decreasing} $\uparrow$,
  y dir=reverse]
\addplot {x^2};
\end{axis}
\end{tikzpicture}
```

Note that axis descriptions and relative positioning macros will stay at the same place as they would for non-reversed axes.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ylabel=$y$ \emph{decreasing} $\uparrow$,
  xlabel=$x$ normal,
  title=reversed axis,
  y dir=reverse,
  colorbar,
  colorbar style={y dir=reverse}]
\addplot+[mesh,scatter] {x^15};
\end{axis}
\end{tikzpicture}
```

Note that `colorbars` won't be reversed automatically, you will have to reverse the sequence of color bars manually in case this is required as in the preceding example.

`/pgfplots/clip limits=true|false` (initially true)

Configures what to do if some, but not all axis limits have been specified explicitly. In case `clip limits=true`, the automatic limit computation will *only* consider points which do not contradict the explicitly set limits.

This option has nothing to do with path clipping, it only affects how the axis limits are computed.

`/pgfplots/enlarge x limits=auto|true|false|upper|lower|<val>|value=<val>|abs value=<val>|abs=<val>|rel=<val>` (initially auto)
`/pgfplots/enlarge y limits=auto|true|false|upper|lower|<val>|value=<val>|abs value=<val>|abs=<val>|rel=<val>` (initially auto)
`/pgfplots/enlarge z limits=auto|true|false|upper|lower|<val>|value=<val>|abs value=<val>|abs=<val>|rel=<val>` (initially auto)
`/pgfplots/enlargelimits=<common value>`

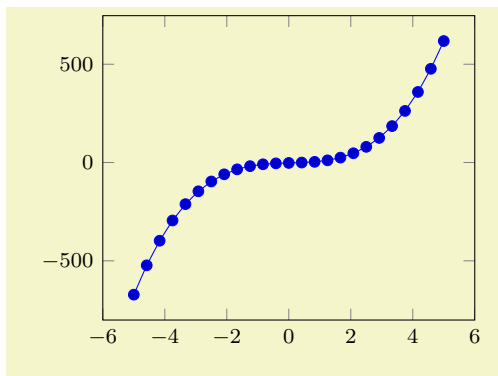
Enlarges the axis size for one axis (or all of them for `enlargelimits`) somewhat if enabled.

You can set `xmin`, `xmax` and `ymin`, `ymax` to the minimum/maximum values of your data and `enlarge x limits` will enlarge the canvas such that the axis doesn't touch the plots.

The value `true` enlarges the lower and upper limit.

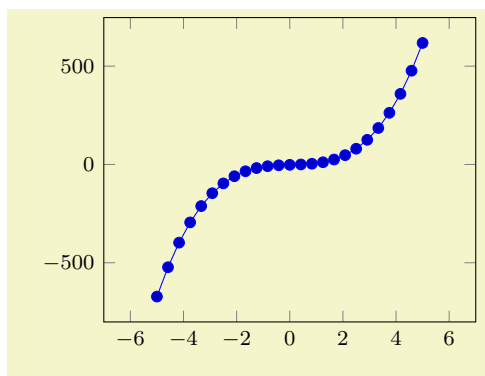
The value `false` uses tight axis limits as specified by the user (or read from input coordinates).

The value `auto` will enlarge limits only for axis for which axis limits have been determined automatically. For three-dimensional figures, the `auto` mechanism applies only for the *z* axis. The *x* and *y* axis won't be enlarged.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small]
    \addplot {5 * x^3 - x^2 + 4*x - 2};
  \end{axis}
\end{tikzpicture}
```

Specifying a number `value` like '`enlarge x limits=0.2`' will enlarge lower and upper axis limit relatively. The following example adds 20% of the axis limits on both sides:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small,enlarge x limits=0.2]
    \addplot {5 * x^3 - x^2 + 4*x - 2};
  \end{axis}
\end{tikzpicture}
```

The choice `rel={<value>}` is the same as `true,value={<value>}`, i.e. it activates relative enlargement for both upper and lower limit.

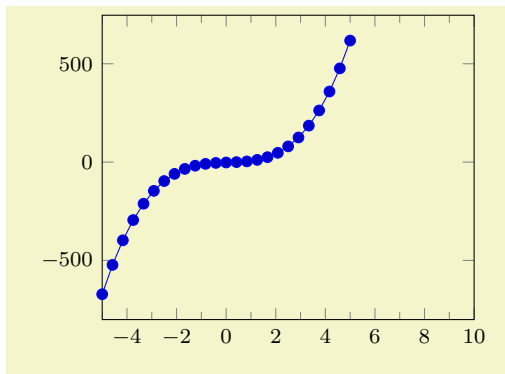
The value `upper` enlarges only the upper axis limit while `lower` enlarges only the lower axis limit. In this case, the amount added to the respective limit can be specified using the `value={<val>}` key. It can be combined with any of the other possible values. For example,

```
\pgfplotsset{enlarge x limits={value=0.2,upper}}
```

will enlarge (only) the upper axis limit by 20% of the axis range. Another example is

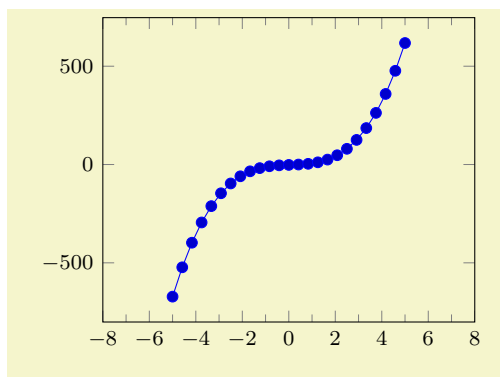
`\pgfplotsset{enlarge x limits={value=0.2,auto}}`

which changes the default threshold of the `auto` value to 20%.



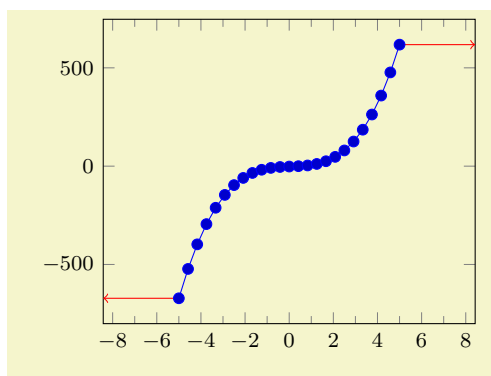
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small,minor x tick num=1,
    enlarge x limits={rel=0.5,upper}
  ]
    \addplot {5 * x^3 - x^2 + 4*x -2};
  \end{axis}
\end{tikzpicture}
```

While `value` uses relative thresholds, `abs value` accepts absolute values: it adds an absolute value to the selected axis. The choice `abs={⟨value⟩}` is the same as `true,abs value={⟨value⟩}`, i.e. it adds an absolute value to both upper and lower limit:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small,minor x tick num=1,
    enlarge x limits={abs=3}
  ]
    \addplot {5 * x^3 - x^2 + 4*x -2};
  \end{axis}
\end{tikzpicture}
```

Here, we enlarged by 3 units of the x axis. Note that you can also specify *dimensions* like 1cm:

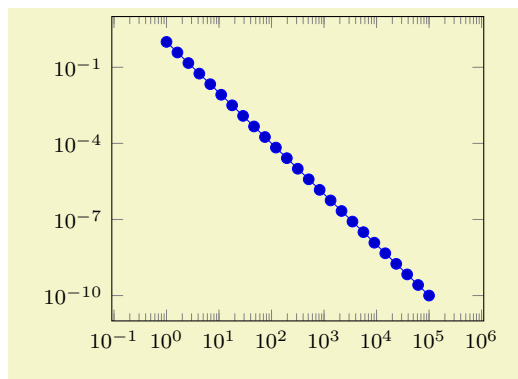


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[small,minor x tick num=1,
    enlarge x limits={abs=1cm}
  ]
    \addplot {5 * x^3 - x^2 + 4*x -2}
      coordinate[pos=0] (first)
      coordinate[pos=1] (last);

    \draw[red,->] (first) -- ++(-1cm,0pt);
    \draw[red,->] (last) -- ++(1cm,0pt);
  \end{axis}
\end{tikzpicture}
```

Technically, the use of absolute dimensions is a little bit different. For example, it allows to enlarge by more than `width` which is impossible for all other choices. PGFLOTS will try to fulfill both the provided `width/height` and the absolute axis enlargements. If it fails to do so, it will give up on `width/height` constraints and print a warning message to your log file. See also the key `enlargelimits respects figure size`.

Attention: `abs value` is applied *multiplicatively* for logarithmic axes! That means `abs value=10` for a logarithmic axis adds $\log 10$ to upper and/or lower axis limits.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    small,
    enlarge x limits={abs=11},
  ]
    \addplot+[domain=1:100000] {x^-2};
  \end{loglogaxis}
\end{tikzpicture}
```

Note that `enlargelimits` is applied before any changes to axis limits are considered as part of `scale mode`: `enlargelimits` will always be applied. Afterwards, the choice `scale mode=scale` uniformly will enlarge limits once more in order to satisfy all scaling constraints. The two limit enlargements are independent of each other, i.e. even if you say `enlargelimits=false`, `scale mode` will still increase axis limits if this seems to be necessary. An exception for this rule is `enlarge-by-dimension`, i.e. something like `abs=1cm` (see `enlargelimits respects figure size` for this case). See `scale mode` (especially `scale mode=units` only) and `unit rescale keep size` for detail on how to disable limit enlargement caused by `scale mode`.

`/pgfplots/enlargelimits respects figure size=true|false` (initially true)

A key which is *only* used for something like `enlarge x limits={abs=1cm}`, i.e. for `enlarge-by-dimension`. It controls if PGFPLOTS will try to respect `width/height`. You should probably always leave it as its default unless you run into problems.

If PGFPLOTS fails to respect the figure size, it will print a warning message of sorts “`enlargelimits respects figure size=true: could not respect the prescribed width/height`” to your log file.

`/pgfplots/log origin x=0|infty` (initially `infty`)

`/pgfplots/log origin y=0|infty` (initially `infty`)

`/pgfplots/log origin z=0|infty` (initially `infty`)

`/pgfplots/log origin=0|infty` (initially `infty`)

Allows to choose which coordinate is the logical “origin” of a logarithmic plot (either for a particular axis or for all of them).

The choice `log origin=infty` is probably useful for stacked plots: it defines the “origin” in log-coordinates to be $-\infty$. To be compatibly with older versions, this is the default.

The choice `log origin=0` defines the logarithmic origin to be the natural choice $\log(1) = 0$. This is particularly useful for `ycomb` plots.

`/pgfplots/update limits=true|false` (initially true)

Can be used to interrupt updates of the data limits (for example, for single `\addplot` commands).

This has the same effect as `\pgfplotsinterruptdatabb ... \endpgfplotsinterruptdatabb`.

`\begin{pgfplotsinterruptdatabb}`

<environment contents>

`\end{pgfplotsinterruptdatabb}`

Everything in *<environment contents>* will not contribute to the data bounding box.

The same effect can be achieved with `update limits=false` inside curly braces.

4.14 Tick Options

4.14.1 Tick Coordinates and Label Texts

`/pgfplots/xtick=\empty|data|{<coordinate list>}` (initially `{<>}`)

`/pgfplots/ytick=\empty|data|{<coordinate list>}` (initially `{<>}`)

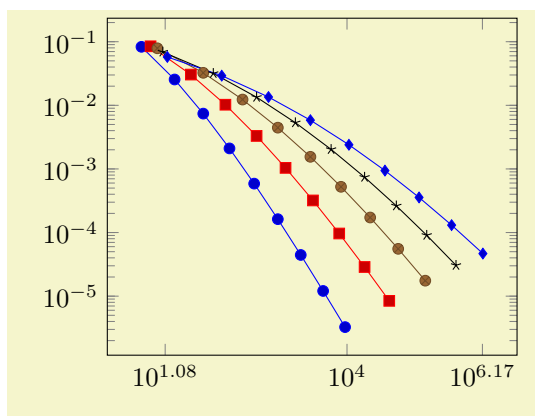
`/pgfplots/ztick=\empty|data|\{coordinate list\}` (initially `\{<\}`)

These options assign a list of *positions* where ticks shall be placed. The argument is either the empty string (which is the initial value), the command `\empty`, the special string ‘`data`’ or a list of coordinates. The initial configuration of an empty string means to generate these positions automatically. The choice `\empty` will result in no tick at all. The special value ‘`data`’ will produce tick marks at every coordinate of the first plot. Otherwise, tick marks will be placed at every coordinate in *coordinate list*.

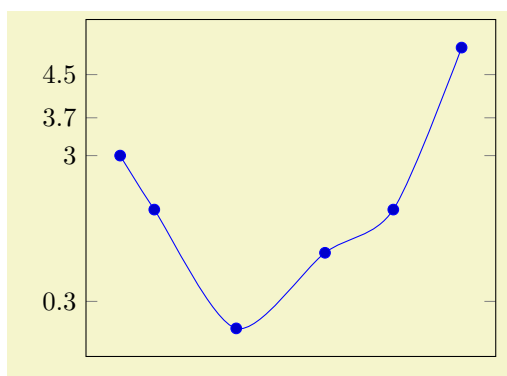
The *coordinate list* will be used inside of a `\foreach \x in {\i>coordinate list\}` statement. The format is as follows:

- `\{0,1,2,5,8,1e1,1.5e1\}` (a series of coordinates),
- `\{0,...,5\}` (the same as `\{0,1,2,3,4,5\}`),
- `\{0,2,...,10\}` (the same as `\{0,2,4,6,8,10\}`),
- `\{9,...,3.5\}` (the same as `\{9, 8, 7, 6, 5, 4\}`),
- See [5, Section 34] for a more detailed definition of the options.
- Please be careful with white spaces inside of *coordinate list* (at least around the dots).

For logplots, PGFLOTS will apply `log(\cdot)` to each element in ‘*coordinate list*’ (similarly, any custom transformations are applied to the argument list).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[xtick={12,9897,1468864}]
    % see above for this macro:
    \plotcoords
  \end{loglogaxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xtick=\empty,
    ytick={-2,0.3,3,3.7,4.5}
  ]
    \addplot+[smooth] coordinates {
      (-2,3) (-1.5,2) (-0.3,-0.2)
      (1,1.2) (2,2) (3,5);
    }
  \end{axis}
\end{tikzpicture}
```

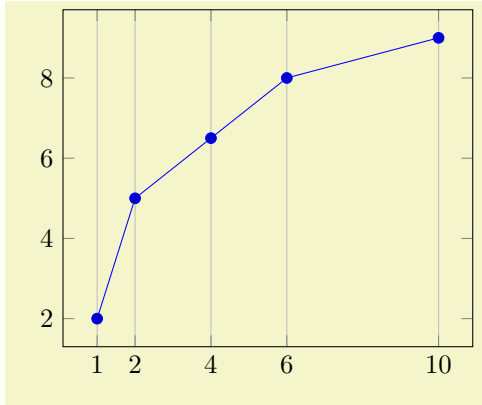
Attention: You can’t use the ‘`...`’ syntax if the elements are too large for $\text{\TeX}$! For example, ‘`xtick=1.5e5,2e7,3e8`’ will work (because the elements are interpreted as strings, not as numbers), but ‘`xtick=1.5,3e5,...,1e10`’ will fail because it involves real number arithmetics beyond $\text{\TeX}$ ’s capacities.

The default choice for tick *positions* in normal plots is to place a tick at each coordinate $i \cdot h$. The step size h depends on the axis scaling and the axis limits. It is chosen from a list of “feasible” step sizes such that neither too much nor too few ticks will be generated. The default for logplots is to place ticks at positions 10^i in the axis’ range. The positions depend on the axis scaling and the dimensions of the picture. If log plots contain just one (or two) positions 10^i in their limits, ticks will be placed at positions $10^{i \cdot h}$ with “feasible” step sizes h as in the case of linear axis.

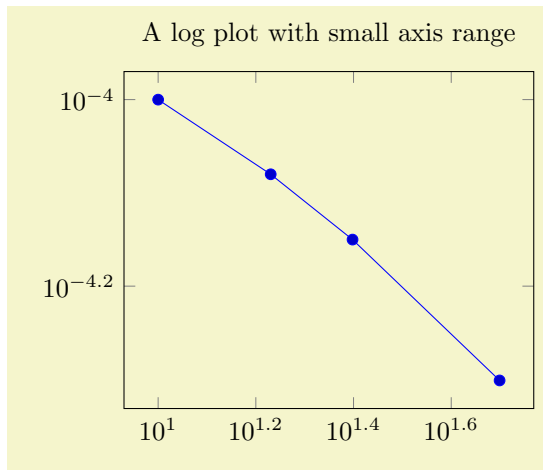
The tick *appearance* can be (re)configured with

```
\pgfplotsset{tick style={very thin,gray}}% modifies the style 'every tick'
\pgfplotsset{minor tick style={black}} % modifies the style 'every minor tick'
```

These style commands can be used at any time. The tick line width can be configured with ‘`major tick length`’ and ‘`minor tick length`’.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[xtick=data,xmajorgrids]
\addplot coordinates {
(1,2)
(2,5)
(4,6.5)
(6,8)
(10,9)
};
\end{axis}
\end{tikzpicture}
```



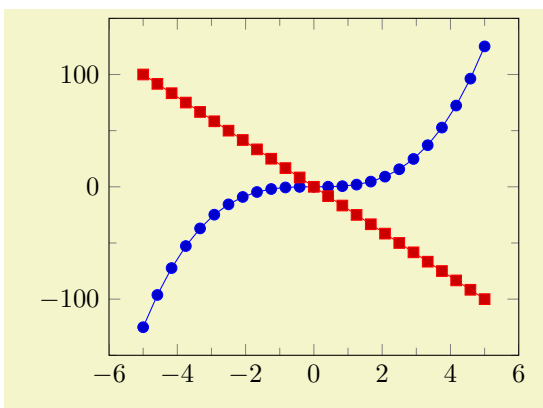
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
title=A log plot with small axis range]

\addplot coordinates {
(10,1e-4)
(17,8.3176e-05)
(25,7.0794e-05)
(50,5e-5)
};
\end{loglogaxis}
\end{tikzpicture}
```

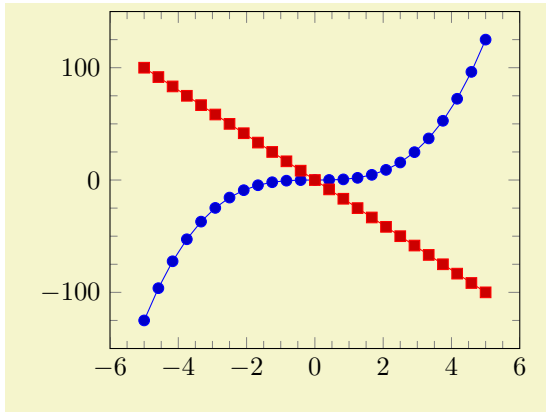
```
/pgfplots/minor x tick num={\number}} (initially 0)
/pgfplots/minor y tick num={\number}} (initially 0)
/pgfplots/minor z tick num={\number}} (initially 0)
/pgfplots/minor tick num={\number}}
```

Sets the number of minor tick lines used either for single axes or for all of them.

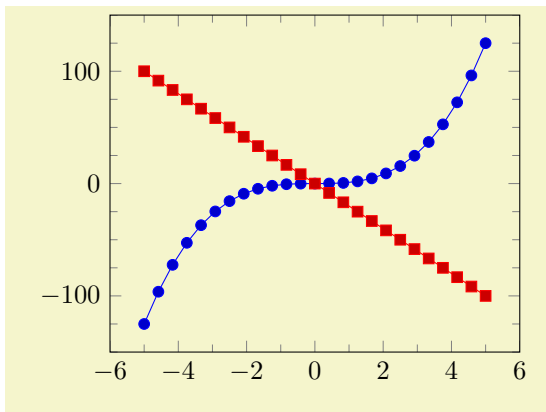
Minor ticks will be disabled if the major ticks don't have the same distance and they are currently only available for linear axes (not for logarithmic ones).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[minor tick num=1]
\addplot {x^3};
\addplot {-20*x};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[minor tick num=3]
    \addplot {x^3};
    \addplot {-20*x};
  \end{axis}
\end{tikzpicture}
```

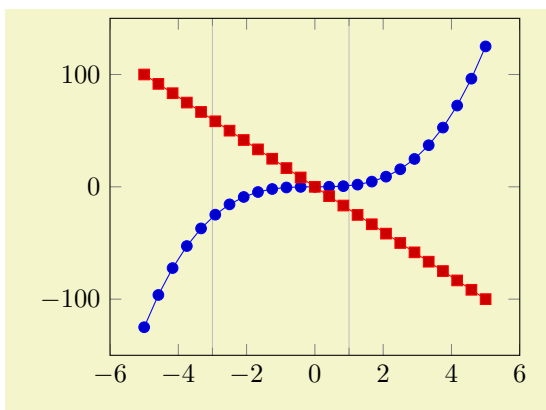


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[minor x tick num=1,
               minor y tick num=3]
    \addplot {x^3};
    \addplot {-20*x};
  \end{axis}
\end{tikzpicture}
```

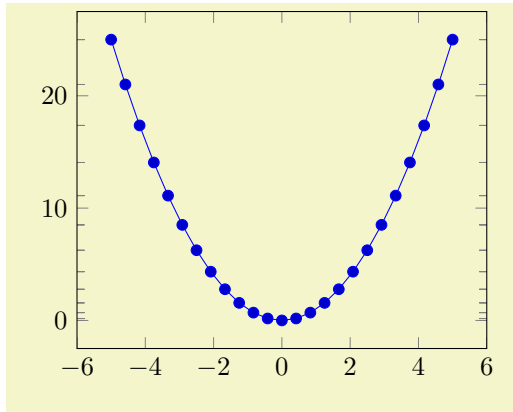
`/pgfplots/minor xtick=data|{<coordinate list>}` (initially empty)
`/pgfplots/minor ytick=data|{<coordinate list>}` (initially empty)
`/pgfplots/minor ztick=data|{<coordinate list>}` (initially empty)
`/pgfplots/minor tick=data|{<coordinate list>}`

Allows to provide a list of minor tick positions manually. The syntax is almost the same as for `xtick` or `ytick`: simply provide either a comma-separated list of tick positions or the special value `data`. An empty argument disables the `minor tick` feature (in contrast to `xtick` where the special value `\empty` clears the list and an empty argument causes PGFPlots to compute a default tick list).

In contrast to `minor x tick num`, this key allows to provide *non-uniform* minor tick positions.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[minor xtick={-3,1},grid=minor]
    \addplot {x^3};
    \addplot {-20*x};
  \end{axis}
\end{tikzpicture}
```

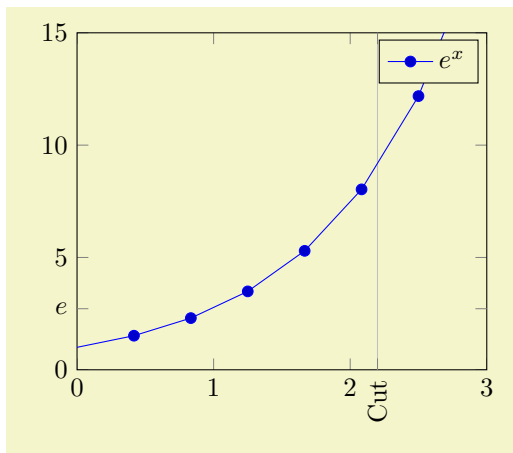



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[minor ytick=data]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

This key has precedence over `minor x tick num` and its variants; if both of them are given, `minor x tick` is preferred and `minor x tick num` is ignored.

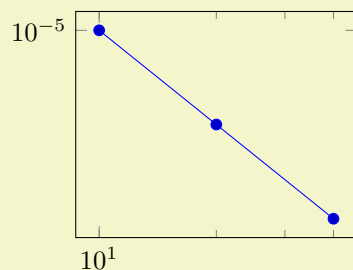
```
/pgfplots/extra x ticks={\langle coordinate list \rangle}
/pgfplots/extra y ticks={\langle coordinate list \rangle}
/pgfplots/extra z ticks={\langle coordinate list \rangle}
```

Adds *additional* tick positions and tick labels to the x or y axis. ‘Additional’ tick positions do not affect the normal tick placement algorithms, they are drawn after the normal ticks. This has two benefits: first, you can add single, important tick positions without disabling the default tick label generation and second, you can draw tick labels ‘on top’ of others, possibly using different style flags.

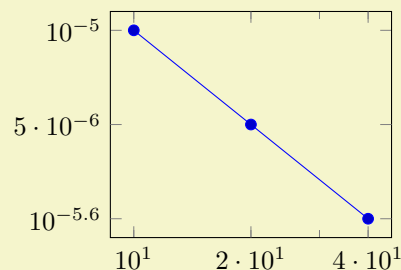
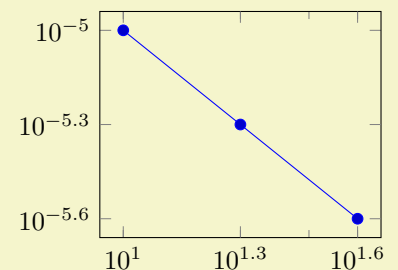


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xmin=0,xmax=3,ymin=0,ymax=15,
  extra y ticks={2.71828},
  extra y tick labels={\mathit{e}},
  extra x ticks={2.2},
  extra x tick style={grid=major,
    tick label style={
      rotate=90,anchor=east}},
  extra x tick labels={Cut},
]
  \addplot {exp(x)};
  \addlegendentry{\mathit{e}^x}
\end{axis}
\end{tikzpicture}
```

Explicitly Provided Limits



With Extra Ticks

With Extra Ticks; 10^e format

```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={width=5.3cm}}
\begin{tikzpicture}
\begin{loglogaxis}[
    title=Explicitly Provided Limits,
    xtickten={1,2},
    ytickten={-5,-6}]
\addplot coordinates
    {(10,1e-5) (20,5e-6) (40,2.5e-6)};
\end{loglogaxis}
\end{tikzpicture}

\begin{tikzpicture}
\begin{loglogaxis}[
    title=With Extra Ticks,
    xtickten={1,2},
    ytickten={-5,-6},
    extra x ticks={20,40},
    extra y ticks={5e-6,2.5e-6}]
\addplot coordinates
    {(10,1e-5) (20,5e-6) (40,2.5e-6)};
\end{loglogaxis}
\end{tikzpicture}

\begin{tikzpicture}
\begin{loglogaxis}[
    title=With Extra Ticks; $10^e$ format,
    extra tick style={log identify minor tick positions=false},
    xtickten={1,2},
    ytickten={-5,-6},
    extra x ticks={20,40},
    extra y ticks={5e-6,2.5e-6}]
\addplot coordinates
    {(10,1e-5) (20,5e-6) (40,2.5e-6)};
\end{loglogaxis}
\end{tikzpicture}

```

Remarks:

- Use `extra x ticks` to highlight special tick positions. The use of `extra x ticks` does not affect minor tick/grid line generation, so you can place extra ticks at positions $j \cdot 10^i$ in log-plots.
- Extra ticks are always typeset as major ticks.
They are affected by `major tick length` or options like `grid=major`.
- Use the style `every extra x tick` (`every extra y tick`) to configure the appearance.
- You can also use ‘`extra x tick style={\langle...\rangle}`’ which has the same effect.

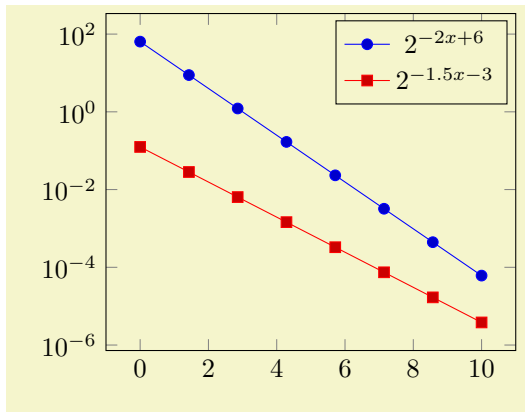
```

/pgfplots/xtickten={\langle exponent base 10 list \rangle}
/pgfplots/ytickten={\langle exponent base 10 list \rangle}
/pgfplots/ztickten={\langle exponent base 10 list \rangle}

```

These options allow to place ticks at selected positions 10^k , $k \in \{\langle \text{exponent base 10 list} \rangle\}$. They are only used for logplots. The syntax for $\{\langle \text{exponent base 10 list} \rangle\}$ is the same as above for `xtick={\langle list \rangle}` or `ytick={\langle list \rangle}`.

Using ‘`xtickten={1,2,3,4}`’ is equivalent to ‘`xtick={1e1,1e2,1e3,1e4}`’, but it requires fewer computational time and it allows to use the short syntax ‘`xtickten={1,\dots,4}`’.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  samples=8,
  ytickten={-6,-4,...,4},
  domain=0:10]

\addplot {2^(-2*x + 6)};
\addlegendentry{$2^{-2x + 6}$}

% or invoke gnuplot to generate coordinates:
\addplot gnuplot[id=pow2]
  {2**(-1.5*x -3)};
\addlegendentry{$2^{-1.5x -3}$}
\end{semilogyaxis}
\end{tikzpicture}
```

In case `log basis x` $\neq 10$, the meaning of `xtickten` changes. In such a case, `xtickten` will still assign the exponent, but for the chosen `log basis x` instead of base 10.

```
/pgfplots/xticklabels={\langle label list \rangle}
/pgfplots/yticklabels={\langle label list \rangle}
/pgfplots/zticklabels={\langle label list \rangle}
```

Assigns a *list of tick labels* to each tick position. Tick *positions* are assigned using the `xtick` and `ytick`-options.

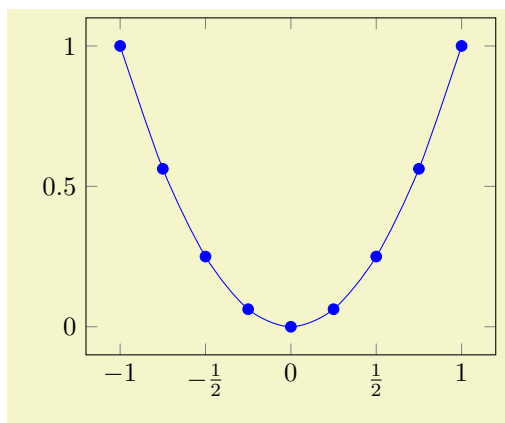
This is one of two options to assign tick labels directly. The other option is `xticklabel={\langle command \rangle}` (or `yticklabel={\langle command \rangle}`). The option ‘`xticklabel`’ offers higher flexibility while ‘`xticklabels`’ is easier to use. See also the variant `xticklabels from table`.

The argument `\langle label list \rangle` has the same format as for ticks, that means

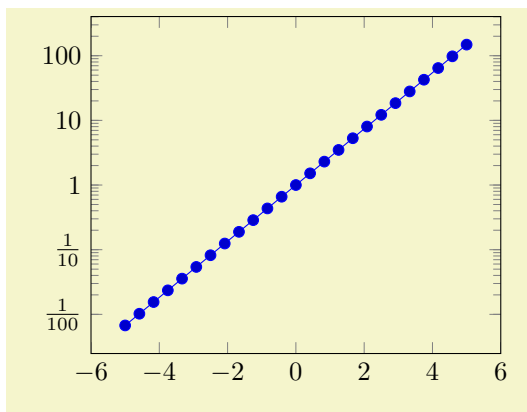
```
xticklabels={\frac{1}{2}$, $e$}
```

denotes the two-element-list $\{\frac{1}{2}, e\}$. The list indices match the indices of the tick positions. If you need commas inside of list elements, use

```
xticklabels={{0,5}, $e$}.
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xtick={-1.5,-1,...,1.5},
  xticklabels={%
    $-1\frac{1}{2}$,
    $-1$,
    $-\frac{1}{2}$,
    $0$,
    $\frac{1}{2}$,
    $1$,
    % note: \frac can be done automatically:
    % xticklabel style={/pgf/number format/frac},
  ]
\addplot[smooth,blue,mark=*]
coordinates {
  (-1, 1)
  (-0.75, 0.5625)
  (-0.5, 0.25)
  (-0.25, 0.0625)
  (0, 0)
  (0.25, 0.0625)
  (0.5, 0.25)
  (0.75, 0.5625)
  (1, 1)
};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{semilogyaxis}[
  ytickten={-2,-1,0,1,2},
  yticklabels={\frac{1}{100}$,%
    \frac{1}{10}$,%
    1,10,100},
]
  \addplot {exp(x)};
\end{semilogyaxis}
\end{tikzpicture}
```

Note that it is also possible to terminate list entries with two backslashes, `\\`. In that case, the last entry needs to be terminated by `\\` as well (it is the same alternative syntax which is also accepted for `\legend` and `cycle list`).

Please keep in mind that the arguments *always* refer to a list of tick positions, although it does not alter or define the list of positions. Consequently, you should also provide the list of positions. Note that a list of positions might be longer than what is actually displayed (in case the axis limits clip some of the value away), but the index mapping into `<label list>` still includes the clipped values.

```
/pgfplots/xticklabel=<command>
/pgfplots/yticklabel=<command>
/pgfplots/zticklabel=<command>
```

These keys change the $\text{\TeX}$ -command which creates the tick *labels* assigned to each tick position (see options `xtick` and `ytick`).

This is one of the two options to assign tick labels directly. The other option is `'xticklabels={<label list>}'` (or `yticklabels={<label list>}`). The option `'xticklabel'` offers higher flexibility while `'xticklabels'` is easier to use.

The argument `<command>` can be any $\text{\TeX}$ -text. The following commands are valid inside of `<command>`:

- `\tick` The current element of option `xtick` (or `ytick`).
- `\ticknum` The current tick number, starting with 0 (it is a macro containing a number).
- `\nexttick` This command is only valid if the `x tick label as interval` option is set (or the corresponding variable for *y*). It will contain the position of the next tick position, that means the right boundary of the tick interval.

The default argument is

- `\axisdefaultticklabel` for normal plots:

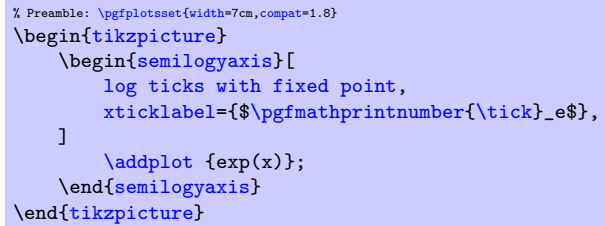
```
\def\axisdefaultticklabel{\pgfmathprintnumber{\tick}}
```

- `\axisdefaultticklabellog` for logplots:

```
\def\axisdefaultticklabellog{%
  \pgfkeysgetvalue{/pgfplots/log number format code/.@cmd}\pgfplots@log@label@style
  \expandafter\pgfplots@log@label@style\tick\pgfeov
}
```

That means you can configure the appearance of linear axis with the number formatting options described in Section 4.12 and logarithmic axis with `log number format code`, see below.

The key `yticklabel` is a code fragment which is supposed to handle any incoming `\tick` value. Consequently, it can be used to append custom suffixes or even units:



A special Prewavelet

The graph shows a piecewise linear function on the interval $[0, 1]$. The x-axis has tick marks at 0 , $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$, and 1 . The y-axis has tick marks at $11/10$, $1/10$, $-6/10$, and $-12/10$. The function is defined by the following points:

x	y
0	$-12/10$
$\frac{1}{4}$	$11/10$
$\frac{1}{2}$	$-6/10$
$\frac{3}{4}$	$1/10$
1	0

```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% \usepackage{nicefrac}% required
\begin{tikzpicture}
\begin{axis}[
% x ticks explicitly formatted:
xtick={0,1,0.5,0.25,0.75},
xticklabels={0$,1$, $\frac{1}{2}$,$\frac{1}{4}$,$\frac{3}{4}$},
% y ticks automatically by some code fragment:
ytick=data,
yticklabel={%
\scriptsize
\ifdim\tick pt<0pt % a TeX \if -- see TeX Book
\pgfmathparse{-10*\tick}%
$\- \nicefrac{\pgfmathprintnumber{\pgfmathresult}}{10}$%
\else
\ifdim\tick pt=0pt
\else
\pgfmathparse{10*\tick}%
$\nicefrac{\pgfmathprintnumber{\pgfmathresult}}{10}$%
\fi
\fi
},
% NOTE: this here does the same:
% yticklabel style={pgf/number format/.cd,frac,
% frac TeX=\nicefrac,frac whole=false,frac denom=10},
ymajorgrids,
title=A special Prewavelet,
axis x line=center,
axis y line=left,
]
\addplot coordinates {(0,-1.2) (0.25,1.1)
(0.5,-0.6) (0.75,0.1) (1,0)};
\end{axis}
\end{tikzpicture}
```

The $\text{\TeX}$ script takes the `\tick` macro as input and applies some logic. The `\ifdim\tick pt<0pt` means “if dimension `\tick pt < 0pt`”. The `\ifdim` is $\text{\TeX}$ ’s only way to compare real fixed point numbers and the author did not want to invoke `\pgfmath` for this simple task. Since `\ifdim` expects a dimension, we have to use the `pt` suffix which is compatible with `\pgfmath`. The result is that negative numbers, zero and positive numbers are typeset differently.

You can change the appearance of tick labels with

```
\pgfplotsset{tick label style={
  font=\tiny,
  /pgf/number format/sci}}% this modifies the ‘every tick label’ style
```

and/or

```
\pgfplotsset{x tick label style={
  above,
  /pgf/number format/fixed zerofill}}% this modifies the ‘every x tick label’ style
```

and

```
\pgfplotsset{y tick label style={font=\bfseries}}% modifies ‘every y tick label’
```

```
/pgfplots/xticklabels from table={\table or filename}\{colname\}
/pgfplots/yticklabels from table={\table or filename}\{colname\}
/pgfplots/zticklabels from table={\table or filename}\{colname\}
```

A variant of `xticklabels={\list}` which uses each entry in the column named `\colname` from a table as tick labels.

The first argument `\table or filename` can be either a loaded table macro (i.e. the result of `\pgfplotstableread{\file name}\{\table\}`) or just a file name.

The second argument can be a column name, a column alias or a **create on use** specification (see `PGFPLOTSTABLE` for the latter two). Furthermore, it can be `[index]\integer` in which case `\integer` is a column index.

The behavior of `xticklabels from table` is the same as if the column `\colname` would have been provided as comma separated list to `xticklabels`. This means the column can contain text, $\text{\TeX}$ macros or even math mode.

If you have white spaces in your cells, enclose the complete cell in curly braces, `{example cell}`. The detailed input format for tables is discussed in `\addplot table` and in the documentation for `PGFPLOTSTABLE`.

```
/pgfplots/extra x tick label={\TeX code\}
/pgfplots/extra y tick label={\TeX code\}
/pgfplots/extra z tick label={\TeX code\}
```

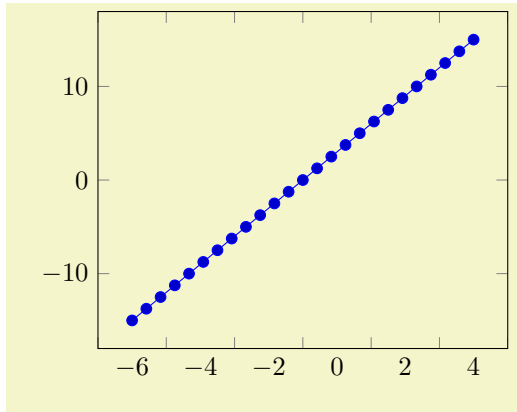
As `xticklabel` provides code to generate tick labels for each `xtick`, the key `extra x tick label` provides code to generate tick labels for every element in `extra x ticks`.

```
/pgfplots/extra x tick labels={\label list\}
/pgfplots/extra y tick labels={\label list\}
/pgfplots/extra z tick labels={\label list\}
```

As `xticklabels` provides explicit tick labels for each `xtick`, the key `extra x tick labels` provides explicit tick labels for every element in `extra x ticks`.

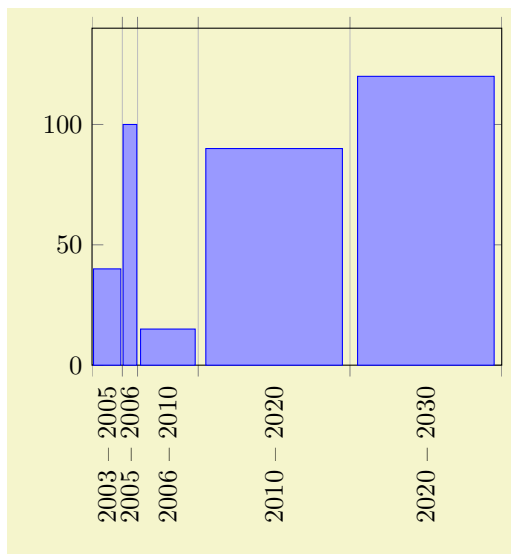
```
/pgfplots/x tick label as interval=true|false (initially false)
/pgfplots/y tick label as interval=true|false (initially false)
/pgfplots/z tick label as interval=true|false (initially false)
```

Allows to treat tick labels as intervals; that means the tick positions denote the interval boundaries. If there are n positions, $(n - 1)$ tick labels will be generated, one for each interval.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x tick label as interval]
\addplot {3*x};
\end{axis}
\end{tikzpicture}
```

This mode enables the use of `\nexttick` inside of `xticklabel` (or `yticklabel`). A common application might be a bar plot.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
ybar interval=0.9,
x tick label as interval,
xmin=2003,xmax=2030,
ymin=0,ymax=140,
xticklabel={
$\pgfmathprintnumber{\tick}$
-- $\pgfmathprintnumber{\nexttick}$,
xtick=data,
x tick label style={
rotate=90,anchor=east,
/pgf/number format/1000 sep=}
}
]
\addplot[draw=blue,fill=blue!40!white]
coordinates
{(2003,40) (2005,100) (2006,15)
(2010,90) (2020,120) (2030,130)};
\end{axis}
\end{tikzpicture}
```

<code>/pgfplots/xminorticks=true false</code>	(initially true)
<code>/pgfplots/yminorticks=true false</code>	(initially true)
<code>/pgfplots/zminorticks=true false</code>	(initially true)
<code>/pgfplots/xmajorticks=true false</code>	(initially true)
<code>/pgfplots/ymajorticks=true false</code>	(initially true)
<code>/pgfplots/zmajorticks=true false</code>	(initially true)
<code>/pgfplots/ticks=minor major both none</code>	(initially both)

Enables/disables the small tick lines either for single axis or for all of them. Major ticks are those placed at the tick positions and minor ticks are between tick positions. Please note that minor ticks are automatically disabled if `xtick` is not a uniform range⁵¹.

The key `minor tick length={⟨dimen⟩}` configures the tick length for minor ticks while the `major` variant applies to major ticks. You can configure the appearance using the following styles:

```
\pgfplotsset{every tick/.append style={color=black}} % applies to major and minor ticks,
\pgfplotsset{every minor tick/.append style={thin}} % applies only to minor ticks,
\pgfplotsset{every major tick/.append style={thick}} % applies only to major ticks.
```

There is also the style “`every tick`” which applies to both, major and minor ticks.

```
/pgfplots/xtickmin={⟨coord⟩}
/pgfplots/ytickmin={⟨coord⟩}
/pgfplots/ztickmin={⟨coord⟩}
```

⁵¹A uniform list means the difference between all elements is the same for linear axis or, for logarithmic axes, $\log(10)$.

```
/pgfplots/xtickmax={\coord}}
/pgfplots/ytickmax={\coord}}
/pgfplots/ztickmax={\coord}}
```

These keys can be used to modify minimum/maximum values before ticks are drawn. Because this applies to axis discontinuities, it is described on page 196 in Section 4.8.11, “Axis Discontinuities”.

4.14.2 Tick Alignment: Positions and Shifts

```
/pgfplots/xtick pos=left|right|both (initially both)
/pgfplots/ytick pos=left|right|both (initially both)
/pgfplots/ztick pos=left|right|both (initially both)
/pgfplots/tick pos=left|right|both
```

Allows to choose where to place the small tick lines. In the default configuration, this does also affect tick *labels*, see below. The **tick pos** style sets all of them to the same value (aliased by **tickpos**). This option is only useful for boxed axes.

For x , the additional choices **bottom** and **top** can be used which are equivalent to **left** and **right**, respectively. Both are accepted for y .

Changing **tick pos** will also affect the placement of tick labels.

Note that it can also affect the **axis lines** key, although not all combinations make sense. Make sure the settings are consistent.

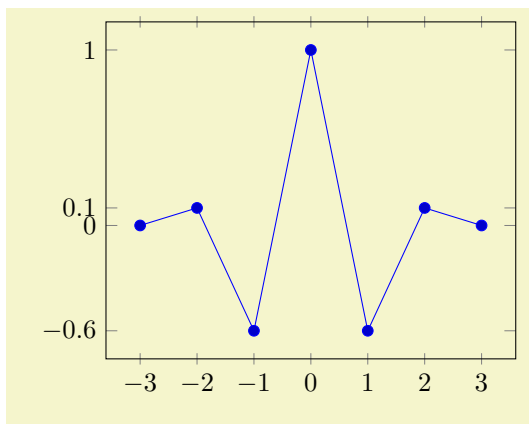
```
/pgfplots/xticklabel pos=left|right|default (initially default)
/pgfplots/yticklabel pos=left|right|default (initially default)
/pgfplots/zticklabel pos=left|right|default (initially default)
/pgfplots/ticklabel pos=left|right|default (initially default)
```

Allows to choose where to place tick *labels*. The choices **left** and **right** place tick labels either at the left or at the right side of the complete axis. The choice **default** uses the same setting as **xtick pos** (or **ytick pos**). This option is only useful for boxed axes – keep it to **default** for non-boxed figures. The **ticklabel pos** style sets all three of them to the same value.

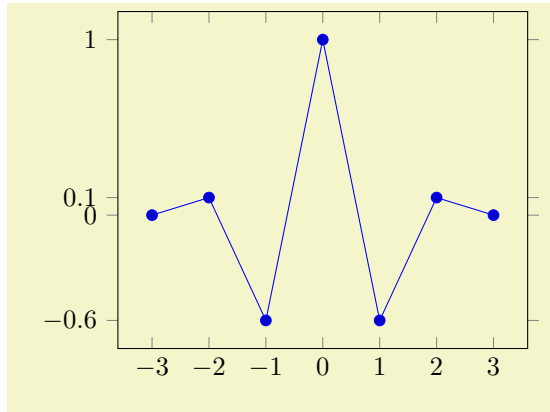
For x , the additional choices **bottom** and **top** can be used which are equivalent to **left** and **right**, respectively. Both are accepted for x .

```
/pgfplots/xtick align=inside|center|outside (initially inside)
/pgfplots/ytick align=inside|center|outside (initially inside)
/pgfplots/ztick align=inside|center|outside (initially inside)
/pgfplots/tick align=inside|center|outside (initially inside)
```

Allows to change the location of the ticks relative to the axis lines. The **tick align** sets all of them to the same value. Default is “inside”.

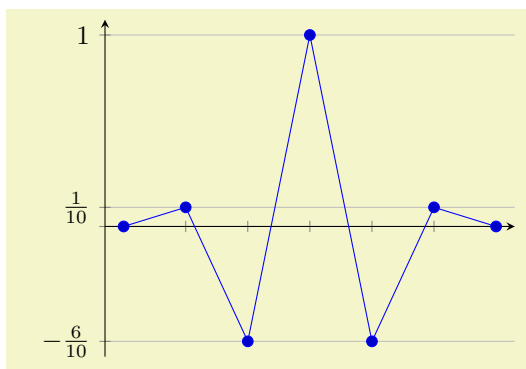


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xtick=data,ytick=data,
  xtick align=center]
\addplot coordinates
{(-3,0) (-2,0.1) (-1,-0.6)
(0,1)
(1,-0.6) (2,0.1) (3,0)};
\end{axis}
\end{tikzpicture}
```

```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xtick=data,ytick=data,
  ytick align=outside]
\addplot coordinates
  {(-3,0) (-2,0.1) (-1,-0.6)
   (0,1)
   (1,-0.6) (2,0.1) (3,0)};
\end{axis}
\end{tikzpicture}
```

These tick alignment options are set automatically by the `axis x line` and `axis y line` methods (unless one appends an asterisk ‘*’):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xtick=data,
  axis x line=center,
  xticklabels={,,},
  ytick={-0.6,0,0.1,1},
  yticklabels={
    $-\frac{6}{10}$,,
    $\frac{1}{10}$,$1$},
  ymajorgrids,
  axis y line=left,
  enlargelimits=0.05]
\addplot coordinates
  {(-3,0) (-2,0.1) (-1,-0.6)
   (0,1)
   (1,-0.6) (2,0.1) (3,0)};
\end{axis}
\end{tikzpicture}
```

```
/pgfplots/xticklabel shift=<{dimension}> (initially empty)
/pgfplots/yticklabel shift=<{dimension}> (initially empty)
/pgfplots/zticklabel shift=<{dimension}> (initially empty)
/pgfplots/ticklabel shift=<{dimension}> (initially empty)
```

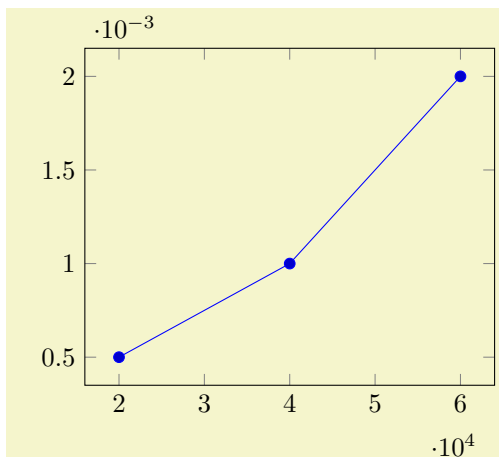
Shifts tick labels in direction of the outer unit normal of the axis by an amount of $\langle dimension \rangle$. The `ticklabel shift` sets the same value for all axes.

This is usually unnecessary as the `anchor` of a tick label already yields enough spacing in most cases.

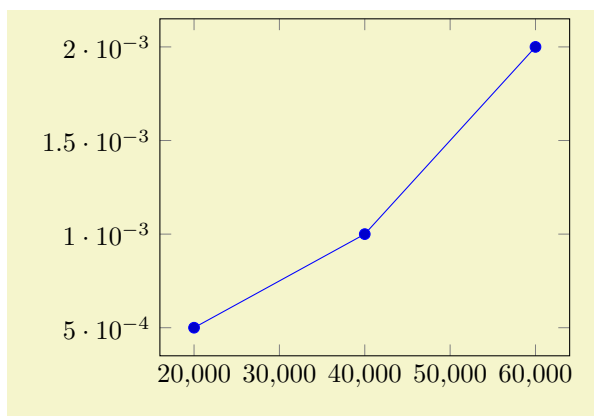
4.14.3 Tick Scaling - Common Factors In Ticks

```
/pgfplots/scaled ticks=true|false|base 10:<e>|real:<num>|manual:{<label>}{<code>} (initially true)
/pgfplots/scaled x ticks=<one of the values> (initially true)
/pgfplots/scaled y ticks=<one of the values> (initially true)
/pgfplots/scaled z ticks=<one of the values> (initially true)
```

Allows to factor out common exponents in tick labels for *linear axes*. For example, if you have tick labels 20000, 40000 and 60000, you may want to save some space and write 2, 4, 6 with a separate factor ‘ 10^4 ’. Use ‘`scaled ticks=true`’ to enable this feature. In case of `true`, tick scaling will be triggered if the data range is either too large or too small (see below).



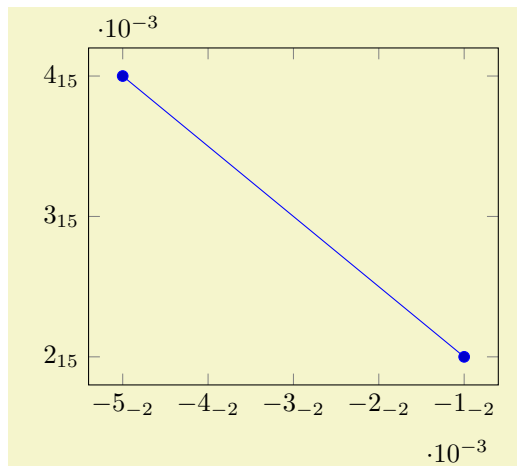
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[scaled ticks=true]
\addplot coordinates {
(20000,0.0005)
(40000,0.0010)
(60000,0.0020)
};
\end{axis}
\end{tikzpicture}%
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[scaled ticks=false]
\addplot coordinates {
(20000,0.0005)
(40000,0.0010)
(60000,0.0020)
};
\end{axis}
\end{tikzpicture}
```

The `scaled ticks` key is a style which simply sets scaled ticks for both, x and y .

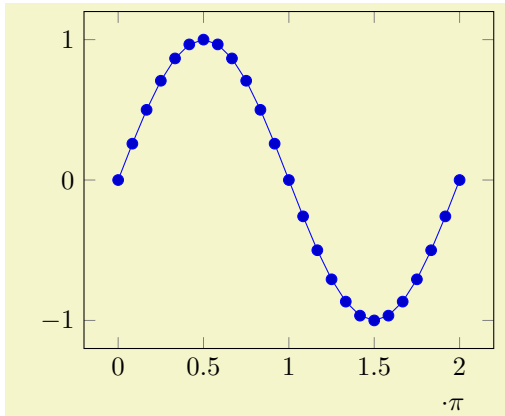
The value `base 10:⟨e⟩` allows to adjust the algorithm manually. For example, `base 10:3` will divide every tick label by 10^3 :



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[scaled ticks=base 10:3,
/pgf/number format/sci subscript]
\addplot coordinates
{(-0.00001,2e12) (-0.00005,4e12) };
\end{axis}
\end{tikzpicture}
```

Here, the `sci subscript` option simply saves space. In general, `base 10:⟨e⟩` will divide every tick by 10^e . The effect is not limited by the “too large or too small” decisions mentioned above.

The value `real:⟨num⟩` allows to divide every tick by a fixed $\langle num \rangle$. For example, the following plot is physically ranged from 0 to 2π , but the tick scaling algorithm is configured to divide every tick label by π .



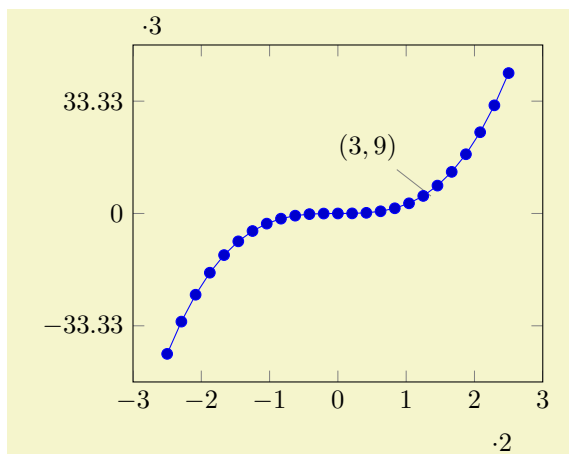
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    xtick={0,1.5708,...,10},
    domain=0:2*pi,
    scaled x ticks={real:3.1415},
    xtick scale label code/.code={\cdot \pi$}
  ]
  \addplot {sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

Setting `scaled ticks=real:⟨num⟩` also changes the `tick scale label code` to

```
\pgfkeys{/pgfplots/xtick scale label code/.code=
  {\pgfkeysvalueof{/pgfplots/tick scale binop} \pgfmathprintnumber{#1}$}}.
```

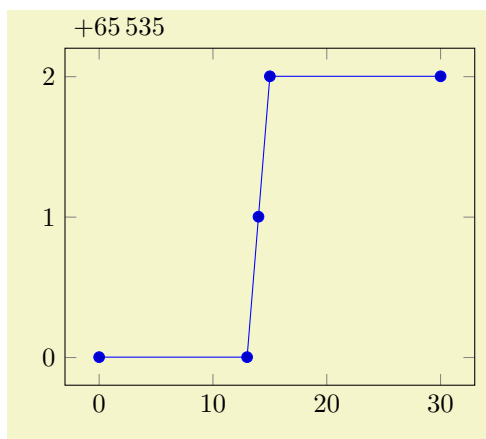
The key `tick scale binop` is described below, it is set initially to `\cdot`.

A further – not very useful – example is shown below. Every x tick label has been divided by 2, every y tick label by 3.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    scaled x ticks=real:2,
    scaled y ticks=real:3
  ]
  \addplot {x^3};
  \node[pin=135:{$(3,9)$}] at (axis cs:3,9) {};
  \end{axis}
\end{tikzpicture}
```

The last option, `scaled ticks>manual:⟨label⟩⟨code⟩` allows even more customization. It allows *full control* over the displayed scaling label *and* the scaling code: $\langle label \rangle$ is used as-is inside of the tick scaling label while $\langle code \rangle$ is supposed to be a one-argument-macro which scales each tick. Example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    % warning: the '%' signs are necessary (?)
    scaled y ticks>manual:{$+65\,535$}{%
      \pgfmathparse{#1-65535}%
    },
    yticklabel style={
      /pgf/number format/fixed,
      /pgf/number format/precision=1,
    }
  ]
  \addplot coordinates {
    (0, 65535)
    (13, 65535)
    (14, 65536)
    (15, 65537)
    (30, 65537)
  };
  \end{axis}
\end{tikzpicture}
```

The example uses `+$+65\,535$` as tick scale label content. Furthermore, it defines the customized tick

label formula $y - (+6.5535 \cdot 10^4) = y - 65535$ to generate y tick labels.

The $\langle label \rangle$ can be arbitrary. It is completely in user control. The second argument, $\langle code \rangle$ is supposed to be a one-argument-macro in which $\#1$ is the current tick position in floating point representation. The macro is expected to assign `\pgfmathresult` (as a number). The PGF manual [5] contains detailed documentation about its math engine.

This feature may also be used to transform coordinates in case they can't be processed with PGFPLOTS: transform them and supply a proper tick scaling method such that tick labels represent the original range.

If $\langle label \rangle$ is empty, the tick scale label won't be drawn (and no space will be occupied).

Tick scaling does *not* work for logarithmic axes.

```
/pgfplots/xtick scale label code/.code={\dots}
/pgfplots/ytick scale label code/.code={\dots}
/pgfplots/ztick scale label code/.code={\dots}
```

Allows to change the default code for scaled tick labels. The default is

```
\pgfplotsset{
  xtick scale label code/.code={\cdot 10^{#1}$}
}
```

More precisely, it is

```
\pgfplotsset{
  xtick scale label code/.code={\pgfkeysvalueof{/pgfplots/tick scale binop} 10^{#1}$}
}
```

and the initial value of `tick scale binop` is `\cdot`, but it can be changed to `\times` if desired.

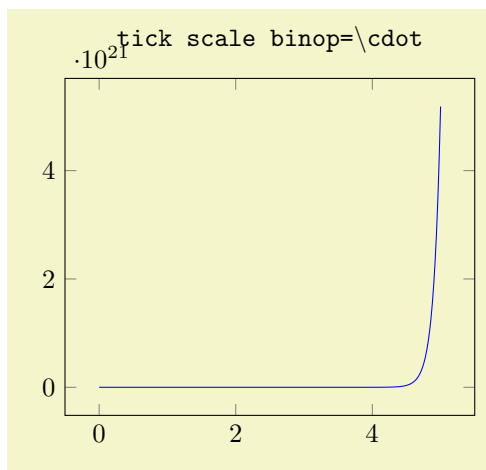
If the code is empty, no tick scale label will be drawn (and no space is consumed).

```
/pgfplots/tick scale label code/.code={\dots}
```

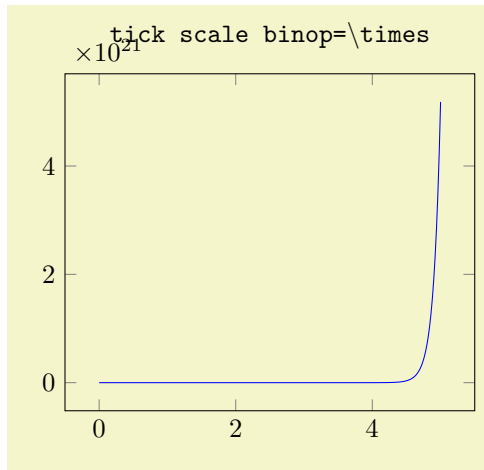
A style which sets `xtick scale label code` and those for y and z .

```
/pgfplots/tick scale binop={\TeX math operator}} (initially \cdot)
```

Sets the binary operator used to display tick scale labels.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  title=\texttt{tick scale
    binop=\textbackslash cdot}]
\addplot
  [mark=none,blue,samples=250,
  domain=0:5]
  {exp(10*x)};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title=\texttt{tick scale
        binop=\textbackslash times},
    tick scale binop=\times]
\addplot
    [mark=none,blue,samples=250,
    domain=0:5]
    {exp(10*x)};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/scale ticks below exponent={⟨exponent⟩}` (initially -1)

Allows fine tuning of the ‘`scaled ticks`’ algorithm: if the axis limits are of magnitude 10^e and $e < \langle \text{exponent} \rangle$, the common prefactor 10^e will be factored out. The default is

`/pgfplots/scale ticks above exponent={⟨exponent⟩}` (initially 3)

Allows fine tuning of the ‘`scaled ticks`’ algorithm: if the axis limits are of magnitude 10^e and $e > \langle \text{exponent} \rangle$, the common prefactor 10^e will be factored out.

4.14.4 Tick Fine-Tuning

The tick placement algorithm depends on a number of parameters which can be tuned to get better results.

`/pgfplots/max space between ticks={⟨number⟩}` (initially 35)

Configures the maximum space between adjacent ticks in full points. The suffix “pt” has to be omitted and fractional numbers are not supported.

`/pgfplots/try min ticks={⟨number⟩}` (initially 4)

Configures a loose lower bound on the number of ticks. It should be considered as a suggestion, not a tight limit. This number will increase the number of ticks if ‘`max space between ticks`’ produces too few of them.

The total number of ticks may still vary because not all fractional numbers in the axis’ range are valid tick positions.

`/pgfplots/try min ticks log={⟨number⟩}` (initially 3)

The same as `try min ticks`, but for logarithmic axis.

`/pgfplots/tickwidth={⟨dimension⟩}` (initially 0.15cm)

`/pgfplots/major tick length={⟨dimension⟩}` (initially 0.15cm)

Sets the length of major tick lines.

It can be accessed using `\pgfkeysvalueof{/pgfplots/major tick length}`.

`/pgfplots/subtickwidth={⟨dimension⟩}` (initially 0.1cm)

`/pgfplots/minor tick length={⟨dimension⟩}` (initially 0.1cm)

Sets the length of minor tick lines.

It can be accessed using `\pgfkeysvalueof{/pgfplots/minor tick length}`.

`/pgfplots/xtick placement tolerance` (initially 0.05pt)

`/pgfplots/ytick placement tolerance` (initially 0.05pt)

`/pgfplots/ztick placement tolerance` (initially 0.05pt)

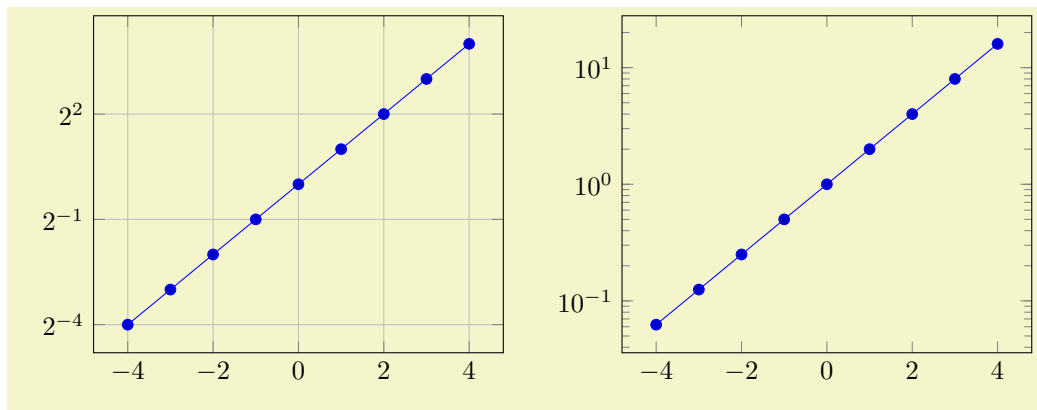
Tick lines and labels will be placed if they are no more than this tolerance beyond the axis limits. This threshold should be chosen such that it does not produce visible differences while still providing fault tolerance.

The threshold is given in paper units of the final figure.

`/pgfplots/log basis x={⟨number⟩}` (initially empty)
`/pgfplots/log basis y={⟨number⟩}` (initially empty)
`/pgfplots/log basis z={⟨number⟩}` (initially empty)

Allows to change the logarithms used for logarithmic axes.

Changing to a different log basis is nothing but a scale. However, it also changes the way tick labels are displayed: they will also be shown in the new basis.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{semilogyaxis}[log basis y=2,grid=minor,samples at={-4,...,4}]
    \addplot {2^x};
  \end{semilogyaxis}
\end{tikzpicture}
~
\begin{tikzpicture}
  \begin{semilogyaxis}[log basis y=10,samples at={-4,...,4}]
    \addplot {2^x};
  \end{semilogyaxis}
\end{tikzpicture}

```

The initial setting is ‘`log basis x=`’ which defaults to: the natural logarithm for any coordinates (basis $\exp(1)$), and the logarithm base 10 for the display of tick labels.

If the log basis is changed to something different than the empty string, the chosen logarithm will be applied to any input coordinate (if the axis scale is log as well) and tick labels will be displayed in this basis.

In other words: usually, you see log axes base 10 and that’s it. It is only interesting for coordinate filters: the initial setting (with empty $\langle number \rangle$) uses coordinate lists basis e although the display will use basis 10 (i.e. it is rescaled). Any non-empty value $\langle number \rangle$ causes both, coordinate lists *and* display to use $\langle number \rangle$ as basis for the logarithm. The javascript code of the `clickable` library will always use the *display* basis (which is usually 10) when it computes slopes.

Technical remarks. When `log basis x` is used, the style `log basis ticks={⟨axis char⟩}` will be installed (in this case `log basis ticks=x`). This style in turn will change `log number format code`.

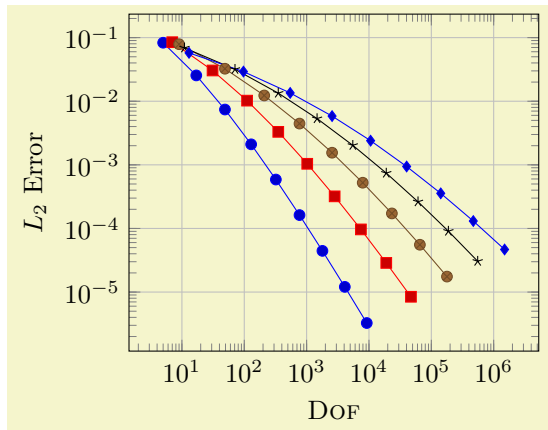
Please note that `xtickten` will be used differently now: it will provide the desired ticks in the new basis! Despite the misleading name “ten”, `xtickten={1,2,3,4}` will yield ticks at $2^1, 2^2, 2^3, 2^4$ if `log basis x=2` has been set.

4.15 Grid Options

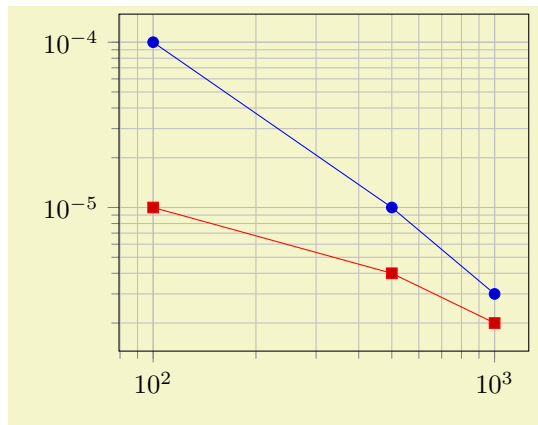
`/pgfplots/xminorgrids=true|false` (initially false)
`/pgfplots/yminorgrids=true|false` (initially false)
`/pgfplots/zminorgrids=true|false` (initially false)
`/pgfplots/xmajorgrids=true|false` (initially false)
`/pgfplots/ymajorgrids=true|false` (initially false)
`/pgfplots/zmajorgrids=true|false` (initially false)
`/pgfplots/grid=minor|major|both|none` (initially false)

Enables/disables different grid lines. Major grid lines are placed at the normal tick positions (see `xmajorticks`) while minor grid lines are placed at minor ticks (see `xminorticks`).

This example employs the coordinates defined on page 19.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
  xlabel={\textsc{Dof}},
  ylabel={$L_2$ Error},
  grid=major
]
% see above for this macro:
\plotcoords
\end{loglogaxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[
  grid=both,
  tick align=outside,
  tickpos=left
]
\addplot coordinates
  {(100,1e-4) (500,1e-5) (1000,3e-6)};
\addplot coordinates
  {(100,1e-5) (500,4e-6) (1000,2e-6)};
\end{loglogaxis}
\end{tikzpicture}
```

Grid lines will be drawn before tick lines are processed, so ticks will be drawn on top of grid lines. You can configure the appearance of grid lines with the styles

```
\pgfplotsset{grid style={help lines}} % modifies the style 'every axis grid'
\pgfplotsset{minor grid style={color=blue}} % modifies the style 'every minor grid'
\pgfplotsset{major grid style={thick}} % modifies the style 'every major grid'
```

4.16 Custom Annotations

Often, one may want to add custom drawing elements or descriptive texts to an axis. These graphical elements should be associated to some logical coordinate, grid point, or perhaps they should just be placed somewhere into the axis.

PGFPLOTS assists with the following ways when it comes to annotations:

1. You can explicitly provide any TikZ instruction like `\draw ...` ; into the axis. Here, the `axis cs` allows to provide coordinates of PGFPLOTS.
Furthermore, `rel axis cs` allows to position TikZ elements relatively (like “50% of the axis’ width”).
2. PGFPLOTS can automatically generate nodes at every coordinate using its `nodes near coords` feature.
3. PGFPLOTS allows you to place nodes on a plot, using the `\addplot ... node[pos={fraction}] {};` feature.

This section explains all of the approaches, except for the `nodes near coords` feature which is documented in its own section.

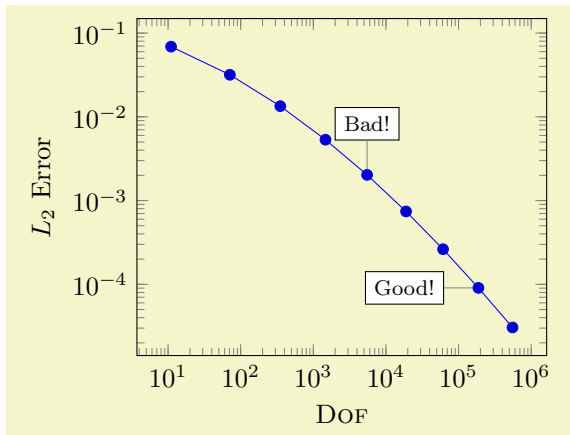
4.16.1 Accessing Axis Coordinates in Graphical Elements

Coordinate system **axis cs**

PGFPLOTS provides a new coordinate system for use inside of an axis, the “axis coordinate system”, **axis cs**.

It can be used to draw any TikZ-graphics at axis coordinates. It is used like

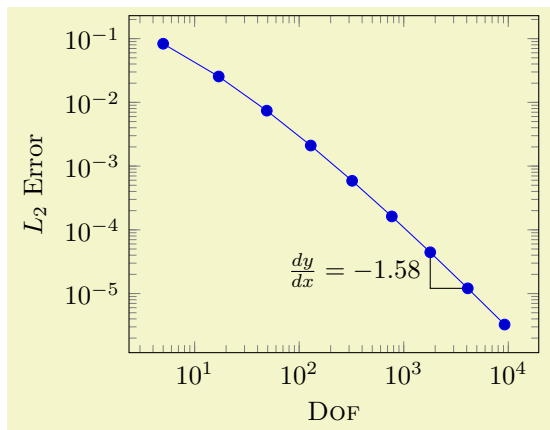
```
\draw
  (axis cs:18943,2.873391e-05)
  |- (axis cs:47103,8.437499e-06);
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\tikzstyle{every pin}=[fill=white,
  draw=black,
  font=\footnotesize]
\begin{tikzpicture}
  \begin{loglogaxis}[
    xlabel={\textsc{Dof}},
    ylabel={\$L_2\$ Error}]

    \addplot coordinates {
      (11,      6.887e-02)
      (71,     3.177e-02)
      (351,    1.341e-02)
      (1471,   5.334e-03)
      (5503,   2.027e-03)
      (18943,  7.415e-04)
      (61183,  2.628e-04)
      (187903, 9.063e-05)
      (553983, 3.053e-05)
    };

    \node[coordinate,pin=above:{Bad!}]
      at (axis cs:5503,2.027e-03) {};
    \node[coordinate,pin=left:{Good!}]
      at (axis cs:187903,9.063e-05) {};
  \end{loglogaxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    xlabel={\textsc{Dof}},
    ylabel={\$L_2\$ Error}
  ]
  \draw
    (axis cs:1793,4.442e-05)
    |- (axis cs:4097,1.207e-05)
    node[near start,left]
    {\$\frac{dy}{dx} = -1.58\$};

  \addplot coordinates {
    (5,      8.312e-02)
    (17,     2.547e-02)
    (49,     7.407e-03)
    (129,    2.102e-03)
    (321,    5.874e-04)
    (769,    1.623e-04)
    (1793,   4.442e-05)
    (4097,   1.207e-05)
    (9217,   3.261e-06)
  };
  \end{loglogaxis}
\end{tikzpicture}
```

Whenever you draw additional graphics, consider using **axis cs**! It applies any custom transformations (including **symbolic x coords**), logarithms, data scaling transformations or whatever PGFPLOTS usually does and provides a low level PGF coordinate as result.

In case you need only one component (say, the y component) of such a vector, you can use the **\pgfplotstransformcoordinatey** command, see Section 8.4 for details about basic level access.

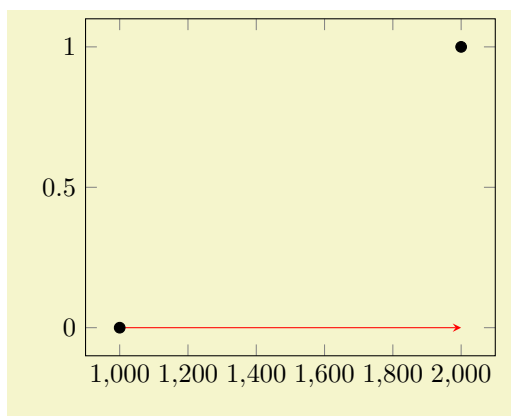
The result of `axis cs` is always an absolute position inside of an axis. This means, in particular, that *adding* two points has unexpected effects: the expression `(axis cs:0,0) ++ (axis cs:1,0)` is not necessarily the same as `(axis cs:1,0)`. The background for such unexpected effects is that PGFLOTS applies a *shifted* linear transformation which moves the origin in order to support its high accuracy and high data range (compare the documentation of `disabledatascaling`).

In order to express *relative* positions (or lengths), you need to use `axis direction cs`.

Coordinate system `axis direction cs`

While `axis cs` allows to supply *absolute positions*, `axis direction cs` supplies *directions*. It allows to express *relative* positions, including lengths and dimensions, by means of axis coordinates.

As noted in the documentation for `axis cs`, adding two coordinates by means of the TikZ `++` operator may have unexpected effects. The correct way for `++` operations is `axis direction cs`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\draw[red,-stealth]
(axis cs:1000,0)
-- % = line-to
++ % = calculate a vector sum
(axis direction cs:1000,0);

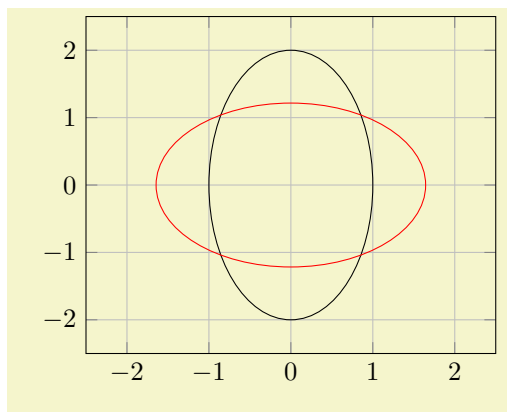
\addplot [only marks,mark=*]
coordinates { (1000,0) (2000,1) };
\end{axis}
\end{tikzpicture}
```

Here, the target of the red arrow is the position `(axis cs:2000,0)` as expected.

Using relative positions is mainly useful for linear axes. Applying this command to log-axes might still work, but it requires more care.

One use-case is to supply lengths – for example in order to support `circle` or `ellipse` paths. The correct way to draw an ellipse in PGFLOTS would be to specify the two involved radii by means of two `(axis direction cs:<x>,<y>)` expressions. In general, this is possible if you use the basic level macros `\pgfpathellipse` and `\pgfplotspointaxisdirectionxy`. Please refer to the documentation of `\pgfplotspointaxisdirectionxy` for two examples of drawing arbitrary ellipses by means of this method.

Since drawing circles and ellipses inside of an axis is a common use-case, PGFLOTS automatically communicates its coordinate system transformations to TikZ: whenever you write `\draw ellipse[x radius=<x>,y radius=<y>]`, the arguments `<x>` and `<y>` are considered to be PGFLOTS direction vectors and are handed over to `axis direction cs`. Consequently, ellipses with axis parallel radii are straightforward and use the normal TikZ syntax:



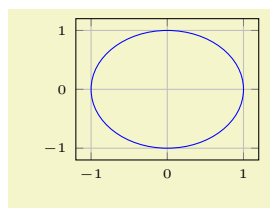
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \pgfplotsset{compat=1.5.1} !
\begin{tikzpicture}
\begin{axis}[
  xmin=-2.5,   xmax=2.5,
  ymin=-2.5,   ymax=2.5,
  xtick={-2,-1,0,1,2},
  ytick={-2,-1,0,1,2},
  grid=major,
]
% standard tikz syntax:
\draw[black] (axis cs:0,0)
  ellipse [
    x radius=1, y radius=2];

\draw[red] (axis cs:0,0)
  ellipse [rotate=90,
    x radius=1, y radius=2];
% see \pgfplotspointaxisdirectionxy
% for arbitrary ellipses
\end{axis}
\end{tikzpicture}
```

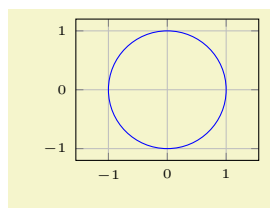
Here, the two ellipses are specified as usual in *TikZ*. PGFplots ensures that all necessary transformations are applied to the two radii. Note that PGFplots usually has different axis scales for x and y . As a consequence, the rotated red ellipse does not fit into the axis lines; we would need to use `axis equal` to allow properly rotated ellipses.

Attention: this modification to circles and ellipses requires `\pgfplotsset{compat=1.5.1}`.

The same applies to circles: in the standard view, a circle with `radius=r` will appear as an ellipse due to the different axis scales. Supplying `axis equal` results in true circles:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \pgfplotsset{compat=1.5.1} !
\begin{tikzpicture}
\begin{axis}[tiny,enlargelimits,
  xmin=-1,xmax=1,
  ymin=-1,ymax=1,
  xtick={-1,0,1},
  ytick={-1,0,1},
  grid=major,
]
\draw[blue] (axis cs:0,0) circle[radius=1];
\end{axis}
\end{tikzpicture}
```

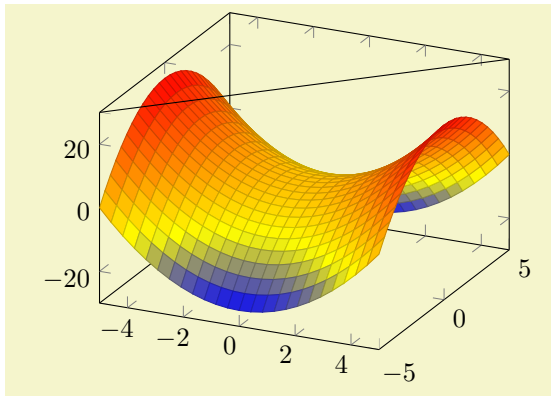


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny,enlargelimits,
  axis equal,
  xmin=-1,xmax=1,
  ymin=-1,ymax=1,
  xtick={-1,0,1},
  ytick={-1,0,1},
  grid=major,
]
\draw[blue] (axis cs:0,0) circle[radius=1];
\end{axis}
\end{tikzpicture}
```

In case you need access to `axis direction cs` inside of math expressions, you can employ the additional math function `transformdirectionx`. It does the same as `axis direction cs`, but only in x direction. The result of `transformdirectionx` is a dimensionless unit which can be interpreted relative to the current PGF x unit vector e_x (see the documentation of `\pgfplotstransformdirectionx` for details). There are the math commands `transformdirectionx`, `transformdirectiony`, and (if the axis is three-dimensional) `transformdirectionz`. Each of them defines `\pgfmathresult` to contain the result of `\pgfplotstransformdirectionx` (or its variants for y and z , respectively).

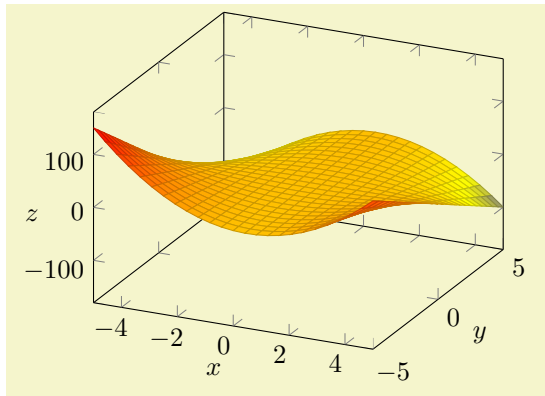
Coordinate system `rel axis cs`

The “relative axis coordinate system”, `rel axis cs`, uses the complete axis vectors as units. That means ‘ $x = 0$ ’ denotes the point on the lower x axis range and ‘ $x = 1$ ’ the point on the upper x axis range (see the remark below for `x dir=reverse`).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}

\addplot3[surf] {x^2 - y^2};
\draw (rel axis cs:0,0,1)
-- (rel axis cs:1,1,1);
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
xlabel=$x$,
ylabel=$y$,
zlabel=$z$,
every axis x label/.style={
at={(rel axis cs:0.5,-0.15,-0.15)}},
every axis y label/.style={
at={(rel axis cs:1.15,0.5,-0.15)}},
every axis z label/.style={
at={(rel axis cs:-0.15,-0.15,0.5)}},
]
\addplot3[surf] {x*(1-x)*y};
\end{axis}
\end{tikzpicture}
```

Points identified by `rel axis cs` use the syntax

`(rel axis cs:< $x$ >,< $y$ >)` or

`(rel axis cs:< $x$ >,< $y$ >,< $z$ >)`

where $\langle x \rangle$, $\langle y \rangle$ and $\langle z \rangle$ are coordinates or constant mathematical expressions. The second syntax is only available in three dimensional axes.

There is one specialty: if you reverse an axis (with `x dir=reverse`), points provided by `rel axis cs` will be *unaffected* by the axis reversal. This is intended to provide consistent placement even for reversed axes. Use `allow reversal of rel axis cs=false` to disable this feature.

There is also a low-level interface to access the transformations and coordinates, see Section 8 on page 417.

Predefined node `current plot begin`

This coordinate will be defined for every plot and can be used as *trailing path commands* or after a plot. It is the first coordinate of the current plot.

Predefined node `current plot end`

This coordinate will be defined for every plot. It is the last coordinate of the current plot.

`/pgfplots/allow reversal of rel axis cs=true|false` (initially true)

A fine-tuning key which specifies how to deal with `x dir=reverse` and `rel axis cs` and `ticklabel cs`.

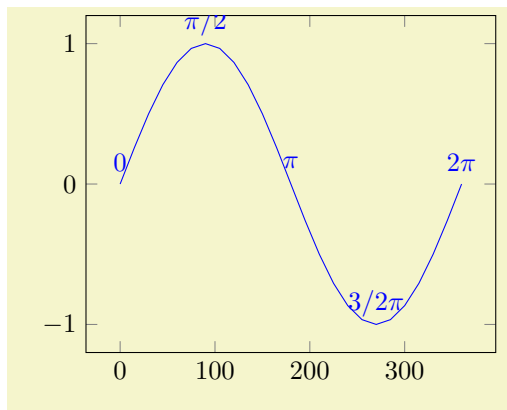
The initial configuration `true` means that points placed with `rel axis cs` and/or `ticklabel cs` will be at the same position inside of the axes even if its ordering has been reversed. The choice `false` will disable the special treatment of `x dir=reverse`.

4.16.2 Placing Nodes on Coordinates of a Plot

The `\addplot` command is not only used for PGFPLOTS, it can also carry additional drawing instructions which are handed over to TikZ after the plot's path is complete. Among others, this can be used to add further nodes on the path.

`/tikz/pos={⟨fraction⟩}`

The `⟨fraction⟩` identifies a part of the recently completed plot if it is used before the trailing semicolon:



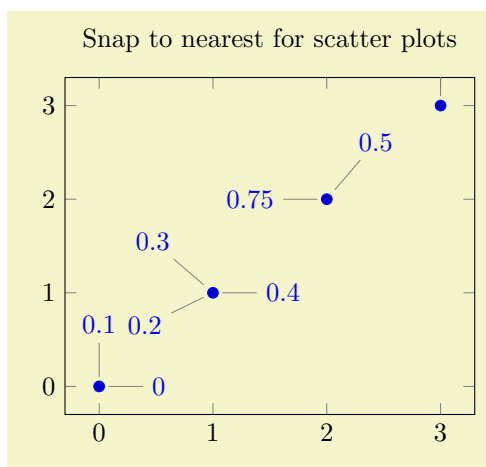
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[blue,domain=0:360] {sin(x)}
    [yshift=8pt]
    node[pos=0] {$0$}
    node[pos=0.25] {$\pi/2$}
    node[pos=0.5] {$\pi$}
    node[pos=0.75] {$3/2\pi$}
    node[pos=1] {$2\pi$}
  ;
  \end{axis}
\end{tikzpicture}
```

Here, the `[yshift=8pt]` tells TikZ to shift all following nodes upwards. The `node[pos=0] {$0$}` instruction tells TikZ to add a text node at 0% of the recently completed plot. The relative position 0% (`pos=0`) refers to the first coordinate which has been seen by PGFPLOTS, and 100% (`pos=1`) refers to the last coordinate. Any value between 0 and 1 is interpolated in-between. Note that all these nodes belong to the plot's visualization (which is terminated by the semicolon). Consequently, all these nodes inherit the same graphic settings (like color choices).

The position on the plot is computed by PGFPLOTS using *logical* coordinates. That means: it computes the overall length of the curve before the curve is projected to screen coordinates and identifies the desired position⁵². Afterwards, it projects the final position to screen coordinates. Thus, the position identifies a location on the plot which is always the same, even in case of a rotated three-dimensional axis. PGFPLOTS will linearly interpolate the fraction between successive coordinates.

Valid choices for `⟨fraction⟩` are any numbers in the range [0, 1].

Note that the precise meaning of `pos` depends on the current plot handler: for most plot handlers, it defaults to linear interpolation (as in the examples above). For `only marks`, `scatter`, `ybar`, `xbar`, `ybar interval`, and `xbar interval`, it snaps to the nearest encountered coordinate. In this context, “snap to nearest” means that `pos=p` refers to the coordinate with index $i = \text{round}(p \cdot N)$ where N is the total number of points:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[title=Snap to nearest for scatter plots]
    \addplot+[only marks]
      coordinates {(0,0) (1,1) (2,2) (3,3)}
    node[pos=0, pin=0 :0 ] {}
    node[pos=0.1, pin=90 :0.1 ] {}
    node[pos=0.2, pin=200:0.2 ] {}
    node[pos=0.3, pin=135:0.3 ] {}
    node[pos=0.4, pin=0 :0.4 ] {}
    node[pos=0.5, pin=60 :0.5 ] {}
    node[pos=0.75,pin=180:0.75] {}
    node[pos=1, pin=90 :1 ] {}
  ;
  \end{axis}
\end{tikzpicture}
```

the previous example shows that `pos=p` maps to one of the four available coordinates, namely the one

⁵²This can be a time-consuming process. Consider using the external library if you have lots of such figures.

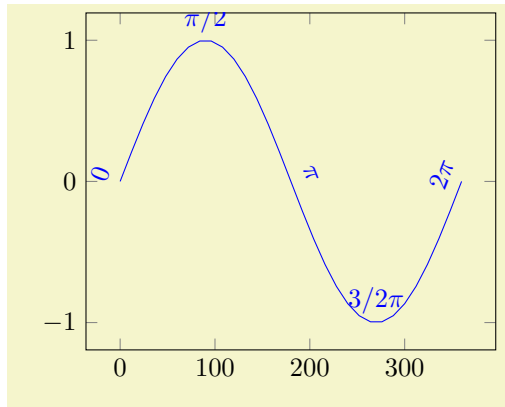
whose index is closest to $p \cdot N$. Note that in such a case, the distance between coordinates is irrelevant – only the coordinate index counts.

Note that the fact that PGFPLOTS uses *logical* coordinates to compute the target positions can produce unexpected effects if x and y axis operate on a different scales. Suppose, for example, that x is always of order 10^3 whereas y is of order 10^{-3} . In such a scenario, the y coordinate have no significant contribution to the curve’s length – although the rescaled axes clearly show “significant” y dynamics. Consider using `axis equal` together with `pos` to produce comparable effects.

`/tikz/sloped`

(initially false)

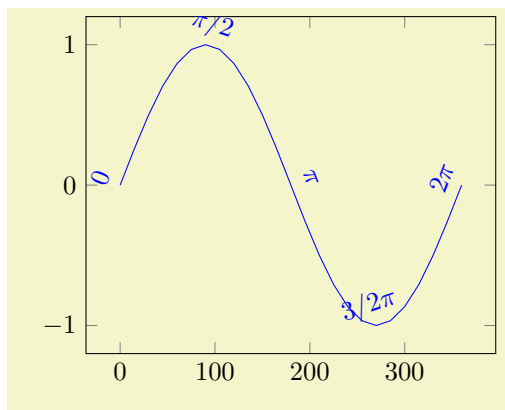
Providing the TikZ key `sloped` to a node identified by `pos` causes it to be rotated such that it adapts to the plot’s gradient.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[blue,domain=0:360,samples=31] {sin(x)}
    [every node/.style={yshift=8pt},sloped]
    node[pos=0] {$0$}
    node[pos=0.25] {$\pi/2$}
    node[pos=0.5] {$\pi$}
    node[pos=0.75] {$3/2\pi$}
    node[pos=1] {$2\pi$}
  ;
  \end{axis}
\end{tikzpicture}
```

Note that the sequence in which `sloped` and shift transformations are applied is important: if shifts are applied first (as would be the case without the `every node/.style` construction), the shifts do not respect the rotation. If `sloped` is applied first, any subsequent shifts will be applied in the *rotated* coordinates. Thus, the case `every node/.style={yshift=8pt}` shifts every node by 8pt in direction of its normal vector.

The `sloped` transformation is based on the gradient between two points (the two points adjacent to `pos`). Consequently, it inherits any sampling weaknesses. To see this, consider the example above with a different number of samples:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% same as above with different number of samples
\begin{tikzpicture}
  \begin{axis}
    \addplot[blue,domain=0:360,samples=25] {sin(x)}
    [every node/.style={yshift=8pt},sloped]
    node[pos=0] {$0$}
    node[pos=0.25] {$\pi/2$}
    node[pos=0.5] {$\pi$}
    node[pos=0.75] {$3/2\pi$}
    node[pos=1] {$2\pi$}
  ;
  \end{axis}
\end{tikzpicture}
```

Here, the two extreme points have small slopes due to the sampling. While this does not seriously affect the quality of the plot, it has a huge impact on the transformation matrices. Keep this in mind when you work with `sloped` (perhaps it even helps to add a further `rotate` argument).

`/tikz/allow upside down=true|false`

(initially false)

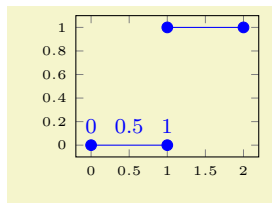
If `/tikz/sloped` is enabled and one has some difficult line plot, the transformation may cause nodes to be drawn upside down. The default configuration `allow upside down=false` will switch the rotation matrix, whereas `allow upside down` allows this case.

`/tikz/pos segment={\segment index}`

(initially empty)

Occasionally, one has a single plot which consists of multiple segments (like those generated by `empty line=jump` or `contour prepared`). The individual segments will typically have different lengths, so it is tedious to identify a position on one of these segments.

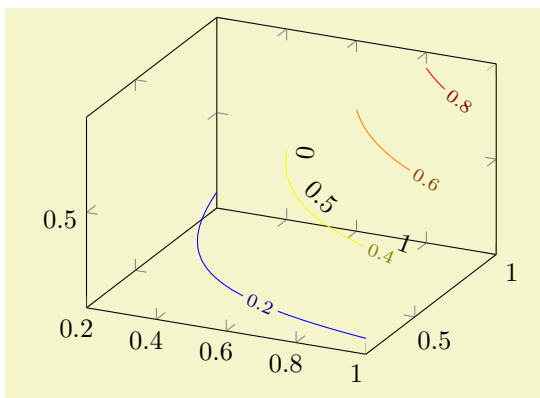
If `pos segment=<segment index>` is non-empty, the key `pos=<fraction>` is interpreted relatively to the provided segment rather than the whole plot. The argument `<segment index>` is an integer, where 0 denotes the first segment.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[tiny]
\addplot coordinates {
(0,0) (1,0)

(1,1) (2,1)}
[pos segment=0,yshift=7pt,font=\footnotesize]
node[pos=0] {0}
node[pos=0.5] {0.5}
node[pos=1] {1};
\end{axis}
\end{tikzpicture}
```

Here, the plot has two segments. However, all three annotation nodes are placed with `pos segment=0`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot3[contour gnuplot,domain=0:1] {x*y}
[sloped,
allow upside down,
pos segment=2,
every node/.style={yshift=7pt}]
node[pos=0] {0}
node[pos=0.5] {0.5}
node[pos=1] {1}
;
\end{axis}
\end{tikzpicture}
```

This plot has four segments (which are generated automatically by the plot handler). The annotation nodes are placed on the third segment, where `sloped` causes them to be rotated, `allow upside down` improves the rendering of the '0', and `every node/.style` install a shift in direction of the normal vector (see the documentation of `sloped` for details).

Occasionally, one wants to place a node using `pos` and one wants to typeset the coordinates of that point inside of the node. This can be accomplished using `\pgfplotspointplotattime`:

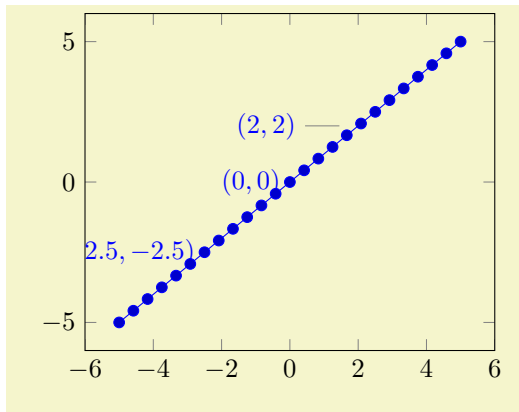
```
\pgfplotspointplotattime
\pgfplotspointplotattime{<fraction>}
```

This command is part of the `pos={<fraction>}` implementation: it defines the current point of PGF to `<fraction>` of the current plot. Without an argument in curly braces, `\pgfplotspointplotattime` will take the current argument of the `pos` key.

Thus, the command computes the basic PGF coordinates – but it also returns the *logical* coordinates of the resulting point into the following keys:

```
/data point/x (no value)
/data point/y (no value)
/data point/z (no value)
```

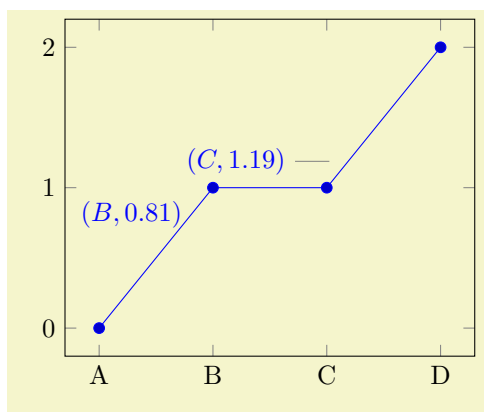
After `\pgfplotspointplotattime` returns, these macros contain the x , y , and z coordinates of the resulting point. They can be used by means of `\pgfkeysvalueof{/data point/x}`, for example.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot {x}
    [left,/pgf/number format/relative=0]
    node[pos=0.5] {%
      \pgfplotspointplotattime
      $(\pgfmathprintnumber
        {\pgfkeysvalueof{/data point/x}}),
        \pgfmathprintnumber
        {\pgfkeysvalueof{/data point/y}})$
    }
    node[pos=0.25] {%
      \pgfplotspointplotattime
      $(\pgfmathprintnumber
        {\pgfkeysvalueof{/data point/x}}),
        \pgfmathprintnumber
        {\pgfkeysvalueof{/data point/y}})$
    }
    node[pos=0.7,pin=180: {%
      \pgfplotspointplotattime{0.7}
      $(\pgfmathprintnumber
        {\pgfkeysvalueof{/data point/x}}),
        \pgfmathprintnumber
        {\pgfkeysvalueof{/data point/y}})$
    }] {}
    ;
  \end{axis}
\end{tikzpicture}
```

In the example above, three nodes have been placed using different `pos=` arguments. Invoking `\pgfplotspointplotattime` inside of the associated node's body checks if `pos` already has a value and uses that value. The third node displays the coordinates inside of a `pin`. Due to internals of TikZ, the `pin` knows nothing about the `pos=0.7` argument of its enclosing `node`, so we need to replicate the '0.7' argument for `\pgfplotspointplotattime{0.7}`. The `/pgf/number format/relative=0` style causes the number printer to round relative to 10^0 (compare against the same example without this style).

In case you have `symbolic x coords` (or any other `x coord inv tafo` which produces non-numeric results), the output stored in `/data point/x` will be the symbolic expression:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[symbolic x coords={A,B,C,D}]
    \addplot coordinates {(A,0) (B,1) (C,1) (D,2)}
    [left]
    node[pos=0.3] {%
      \pgfplotspointplotattime
      $(\pgfkeysvalueof{/data point/x},
        \pgfmathprintnumber
        {\pgfkeysvalueof{/data point/y}})$
    }
    node[pos=0.7,pin=180: {%
      \pgfplotspointplotattime{0.7}
      $(\pgfkeysvalueof{/data point/x},
        \pgfmathprintnumber
        {\pgfkeysvalueof{/data point/y}})$
    }] {}
    ;
  \end{axis}
\end{tikzpicture}
```

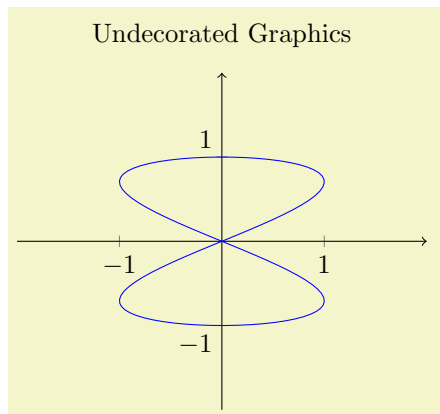
In that specific case, you have to avoid `\pgfmathprintnumber` since the argument is *no* number. Note that `symbolic x coords` cannot return fractions between, say, *A* and *B* as you would expect. However, the point will still be placed at the fractional position (unless you have a `scatter` or `bar plot`).

The computation of coordinates for the `pos` feature is computationally expensive for plots with many points. To reduce time, PGFLOTS will cache computed values: invoking the command `\pgfplotspointplotattime` multiple times with the same argument will reuse the computed value.

4.16.3 Placing Decorations on Top of a Plot

TikZ comes with the powerful `decorations` library (or better: set of libraries). Decorations allow to replace or extend an existing path by means of fancy additional graphics. An introduction into the decorations functionality of TikZ is beyond the scope of this manual and the interested reader should read the associated section in [5].

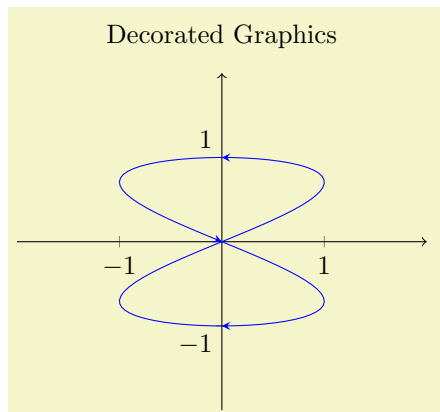
This section shows how to use decorations to enhance plots in PGFPLOTS. Suppose you have some graphics for which you would like to add “direction pointers”:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}[]
% An undecorated graphics with a lot of
% pretty-printing styles:
\begin{axis}[
  axis lines=middle,
  title=Undecorated Graphics,
  xmin=-2, xmax=2, ymin=-2, ymax=2,
  xtick={-1,1}, ytick={-1,1},
  % this disables the standard
  % tick label *text* (but not the line)
  yticklabel=\ ,
  extra description/.code={
    % this generates custom y labels to implement
    % individual styles for every tick:
    \node[below left] at (axis cs:0,-1) {$-1$};
    \node[above left] at (axis cs:0,1) {$1$};
  },
  axis line style={->},
]
\addplot[blue,samples=100,domain=0:2*pi]
  ({sin(deg(2*x))}, {sin(deg(x))});
\end{axis}
\end{tikzpicture}
```

Our aim is to add short pointers indicating the direction of the parameterization.

The solution is to use `\usetikzlibrary{decorations.markings}` and a decoration inside of `\addplot`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usetikzlibrary{decorations.markings}
\begin{tikzpicture}[]
% Same as in previous example, but with decorations:
\begin{axis}[axis lines=middle,
  title=Decorated Graphics,
  xmin=-2, xmax=2, ymin=-2, ymax=2,
  xtick={-1,1}, ytick={-1,1},
  % this disables the standard
  % tick label *text* (but not the line)
  yticklabel=\ ,
  extra description/.code={
    % this generates custom y labels to implement
    % individual styles for every tick:
    \node[below left] at (axis cs:0,-1) {$-1$};
    \node[above left] at (axis cs:0,1) {$1$};
  },
  axis line style={->},
]
\addplot[blue,samples=100,domain=0:2*pi,
  postaction={decorate},% -----
  decoration={markings,% -----
    mark=at position 0.25 with {\arrow{stealth}},
    mark=at position 0.5 with {\arrow{stealth}},
    mark=at position 0.75 with {\arrow{stealth}}}
  ]
  ({sin(deg(2*x))}, {sin(deg(x))});
\end{axis}
\end{tikzpicture}
```

The only changes are in the option list for `\addplot`: it contains a `postaction={decorate}` which activates the decoration (without replacing the original path) and some specification `decoration` containing details about how to decorate the path.

A discussion of details of the `decorations` libraries is beyond the scope of this manual (see [5] for details), but the main point is to add the required decorations to `\addplot` and its option list.

4.17 Style Options

4.17.1 All Supported Styles

PGFPLOTS provides many styles to customize its appearance and behavior. They can be defined and changed in any place where keys are allowed. Furthermore, own styles are defined easily.

Key handler `<key>/.style={<key-value-list>}`

Defines or redefines a style `<key>`. A style is a normal key which will set all options in `<key-value-list>` when it is set.

Use `\pgfplotsset{<key>/.style={<key-value-list>}}` to (re)define a style `<key>` in the namespace `/pgfplots`.

Key handler `<key>/.append style={<key-value-list>}`

Appends `<key-value-list>` to an already existing style `<key>`. This is the preferred method to change the predefined styles: if you only append, you maintain compatibility with future versions.

Use `\pgfplotsset{<key>/.append style={<key-value-list>}}` to append `<key-value-list>` to the style `<key>`. This will assume the prefix `/pgfplots`.

Styles installed for linear/logarithmic axis

`/pgfplots/every axis` (style, initially **empty**)

Installed at the beginning of every axis. TikZ options inside of it will be used for anything inside of the axis rectangle and any axis descriptions.

`/pgfplots/every axis post` (style, initially **empty**)

A style which is applied right *after* arguments provided to an axis are processed.

In the following example, such a style is used to override the `xmin` and `xmax` options provided as arguments to `\begin{axis}[...]`:

```
\begin{tikzpicture}
  \pgfplotsset{
    every axis post/.style={
      xmin=0,xmax=1,
    },
  }
  \begin{axis}[
    xmin=-1,xmax=2,
    ymin=0,ymax=1]
    ...
  \end{axis}
\end{tikzpicture}
```

It is processed right after the arguments of `\begin{axis}`, but before styles like `yticklabel style` etc. are evaluated.

`/pgfplots/every semilogx axis` (style, initially **empty**)

Installed at the beginning of every plot with linear x axis and logarithmic y axis, but after ‘`every axis`’.

`/pgfplots/every semilogy axis` (style, initially **empty**)

Likewise, but with interchanged roles for x and y .

`/pgfplots/every loglog axis` (style, initially **empty**)

Installed at the beginning of every log–log plot.

`/pgfplots/every linear axis` (style, initially **empty**)

Installed at the beginning of every plot with normal axis scaling.

Styles installed for single plots

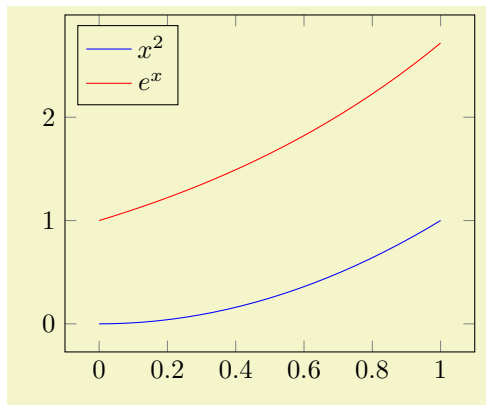
`/pgfplots/every axis plot` (style, initially **empty**)

Installed for each plot. This style may contain options like `samples`, `gnuplot parameters`, `error bars` and it may contain options which affect the final drawing commands.

`/pgfplots/every axis plot post`

(style, initially empty)

This style is similar to `every axis plot` in that it applies to any drawing command in `\addplot`. However, it is set *after* any user defined styles or `cycle list` options.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\pgfplotsset{
  every axis plot post/.append style={
    {mark=none}}
}

\begin{axis}[
  legend style={
    at={(0.03,0.97)},anchor=north west,
    domain=0:1]
  \addplot {x^2};
  \addplot {exp(x)};
  \legend{$x^2$, $e^x$}
\end{axis}
\end{tikzpicture}
```

`/pgfplots/every axis plot no #`

(style, initially empty)

Used for every $\#$ th plot where $\# = 0, 1, 2, 3, 4, \dots$

`/pgfplots/every forget plot`

(style, initially empty)

Used for every plot which has `forget plot` activated.

`/pgfplots/forget plot style={\langle key-value-list \rangle}`

An abbreviation for `every forget plot/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every forget plot`.

Styles for axis descriptions

`/pgfplots/every axis label`

(style, initially empty)

Used for all axis label (like `xlabel` and `ylabel`).

`/pgfplots/label style={\langle key-value-list \rangle}`

An abbreviation for `every axis label/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every axis label`.

`/pgfplots/every axis x label`

(style, no value)

`/pgfplots/every axis y label`

(style, no value)

`/pgfplots/every axis z label`

(style, no value)

Used only for x , y , or z labels, respectively and installed after ‘`every axis label`’.

The initial settings are set by `xlabel absolute` and its variants (if the initial configuration `compat=pre 1.3` is active) or `xlabel near ticks` which provides the better spacing as it incorporates the tick label sizes to compute the position.

Attention: These styles will be overwritten by `axis x line` and/or `axis y line`. Please remember to place your modifications after the axis line variations.

`/pgfplots/x label style={\langle key-value-list \rangle}`

`/pgfplots/y label style={\langle key-value-list \rangle}`

`/pgfplots/z label style={\langle key-value-list \rangle}`

`/pgfplots/xlabel style={\langle key-value-list \rangle}`

`/pgfplots/ylabel style={\langle key-value-list \rangle}`

`/pgfplots/zlabel style={\langle key-value-list \rangle}`

Different abbreviations for `every axis x label/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every axis y label/.append style={\langle key-value-list \rangle}`, and z , `every axis z label/.append style={\langle key-value-list \rangle}`).

`/pgfplots/every axis title` (style, no value)

Used for any axis title. The `at=<(x,y)>` syntax will place the title using `axis description cs`.

The initial setting is

```
\pgfplotsset{every axis title/.style={at={{(0.5,1)}},above,yshift=6pt}}
```

To be more precise, the `yshift` doesn't use the hardcoded `6pt`: it uses the value of

`/pgfplots/every axis title shift={{<default shift>}}` (initially `6pt`)

which can be reset if needed.

`/pgfplots/title style={{<key-value-list>}}`

An abbreviation for `every axis title/.append style={{<key-value-list>}}`.

It appends options to the already existing style `every axis title`.

`/pgfplots/every axis legend` (style, no value)

Installed for each legend. As described for `axis description cs`, the legend's position can be placed using coordinates between 0 and 1 (it employs `axis description cs` automatically).

The initial setting is

```
\pgfplotsset{every axis legend/.style={
  cells={anchor=center},
  inner xsep=3pt,inner ysep=2pt,nodes={inner sep=2pt,text depth=0.15em},
  anchor=north east,
  shape=rectangle,
  fill=white,draw=black,
  at={{(0.98,0.98)}}}}
```

`/pgfplots/legend style={{<key-value-list>}}`

An abbreviation for `every axis legend/.append style={{<key-value-list>}}`.

It appends options to the already existing style `every axis legend`.

`/pgfplots/every legend image post` (style, no value)

Allows to change the appearance of the small legend images *after* the options of the plot style have been applied. Thus, legend formatting can be changed independently of the plot style using `every legend image post`.

This key is also documented on page 181.

`/pgfplots/legend image post style={{<key-value-list>}}`

An abbreviation for `every legend image post/.append style={{<key-value-list>}}`.

It appends options to the already existing style `every legend image post`.

`/pgfplots/every legend to name picture` (style, no value)

A style for use with `legend to image`, see the documentation therein.

`/pgfplots/every colorbar` (style, no value)

A style to change the `colorbar`. See page 202 for the reference documentation of `every colorbar`.

`/pgfplots/colorbar style={{<key-value-list>}}`

An abbreviation for `every colorbar/.append style={{<key-value-list>}}`.

It appends options to the already existing style `every colorbar`.

Styles for axis lines

`/pgfplots/every outer x axis line` (style, initially empty)

`/pgfplots/every outer y axis line` (style, initially empty)

`/pgfplots/every outer z axis line` (style, initially `empty`)

Installed for every axis line which lies on the outer box.

If you want arrow heads, you may also need to check the `separate axis lines` boolean key.

`/pgfplots/every inner x axis line` (style, initially `empty`)

`/pgfplots/every inner y axis line` (style, initially `empty`)

`/pgfplots/every inner z axis line` (style, initially `empty`)

Installed for every axis line which is drawn using the `center` or `middle` options.

`/pgfplots/axis line style={\langle key-value-list \rangle}`

`/pgfplots/inner axis line style={\langle key-value-list \rangle}`

`/pgfplots/outer axis line style={\langle key-value-list \rangle}`

`/pgfplots/x axis line style={\langle key-value-list \rangle}`

`/pgfplots/y axis line style={\langle key-value-list \rangle}`

`/pgfplots/z axis line style={\langle key-value-list \rangle}`

These options modify parts of the axis line styles. They append options to `every inner x axis line` and `every outer x axis line` and the respective y/z variants.

Please refer to Section 4.8.9 on page 193 for details about styles for axis lines.

`/pgfplots/every 3d box foreground` (style, no value)

Installed for the parts drawn by `3d box=complete`. This affects axis lines, tick lines and grid lines drawn in the *foreground*. The background drawing operations have already been done when this style is evaluated.

`/pgfplots/3d box foreground style={\langle key-value-list \rangle}`

An abbreviation for `every 3d box foreground/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every 3d box foreground`.

`/pgfplots/every colorbar sampled line` (style, no value)

To be used in conjunction with `colorbar sampled line`, see the documentation therein.

`/pgfplots/colorbar sampled line style={\langle key-value-list \rangle}`

An abbreviation for `every colorbar sampled line/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every colorbar sampled line`.

Styles for ticks

`/pgfplots/every tick` (style, initially `very thin,gray`)

Installed for each of the small tick *lines*.

`/pgfplots/tick style={\langle key-value-list \rangle}`

An abbreviation for `every tick/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every tick`.

`/pgfplots/every minor tick` (style, initially `empty`)

Used for each minor tick line, installed after ‘`every tick`’.

`/pgfplots/minor tick style={\langle key-value-list \rangle}`

An abbreviation for `every minor tick/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every minor tick`.

`/pgfplots/every major tick` (style, initially `empty`)

Used for each major tick line, installed after ‘`every tick`’.

`/pgfplots/major tick style={\langle key-value-list \rangle}`

An abbreviation for `every major tick/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every major tick`.

`/pgfplots/every tick label` (style, initially empty)

Used for each x and y tick labels.

`/pgfplots/tick label style={\langle key-value-list \rangle}`

`/pgfplots/ticklabel style={\langle key-value-list \rangle}`

Different abbreviations for `every tick label/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every tick label/.append style={\langle key-value-list \rangle}`, and z , `every tick label/.append style={\langle key-value-list \rangle}`).

`/pgfplots/every x tick label` (style, initially empty)

`/pgfplots/every y tick label` (style, initially empty)

`/pgfplots/every z tick label` (style, initially empty)

Used for each x (or y or z , respectively) tick label, installed after ‘`every tick label`’.

`/pgfplots/x tick label style={\langle key-value-list \rangle}`

`/pgfplots/y tick label style={\langle key-value-list \rangle}`

`/pgfplots/z tick label style={\langle key-value-list \rangle}`

`/pgfplots/xticklabel style={\langle key-value-list \rangle}`

`/pgfplots/yticklabel style={\langle key-value-list \rangle}`

`/pgfplots/zticklabel style={\langle key-value-list \rangle}`

Different abbreviations for `every x tick label/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every y tick label/.append style={\langle key-value-list \rangle}`, and z , `every z tick label/.append style={\langle key-value-list \rangle}`).

`/pgfplots/every x tick scale label` (style, no value)

`/pgfplots/every y tick scale label` (style, no value)

`/pgfplots/every z tick scale label` (style, no value)

Configures placement and display of the nodes containing the order of magnitude of tick labels, see Section 4.14.3 for more information about `scaled ticks`.

The initial settings with `compat=1.8` or higher are

```
\pgfplotsset{
  every x tick scale label/.style={
    at={(\xticklabel cs:0.9,5pt)},
    anchor=near xticklabel,
    inner sep=0pt},
  every y tick scale label/.style={
    at={
      (\yticklabel* cs:1.03,-0.3em)},
    /pgfplots/near ticklabel align=outside,
    anchor=near yticklabel opposite,
    inner sep=0pt},
  every z tick scale label/.style={
    at={(\zticklabel* cs:1.2,-0.3em)},
    anchor=near zticklabel,
    inner sep=0pt},
```

`/pgfplots/x tick scale label style={\langle key-value-list \rangle}`

`/pgfplots/y tick scale label style={\langle key-value-list \rangle}`

`/pgfplots/z tick scale label style={\langle key-value-list \rangle}`

An abbreviation for `every x tick scale label/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every y tick scale label/.append style={\langle key-value-list \rangle}`, and the z -axis, `every z tick scale label/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every x tick scale label`.

`/pgfplots/every x tick` (style, initially empty)

`/pgfplots/every y tick` (style, initially empty)

`/pgfplots/every z tick` (style, initially empty)

Installed for tick *lines* on either x or y axis.

`/pgfplots/xtick style={\langle key-value-list \rangle}`

```
/pgfplots/ytick style={\langle key-value-list \rangle}
/pgfplots/ztick style={\langle key-value-list \rangle}
```

An abbreviation for `every x tick/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every y tick/.append style={\langle key-value-list \rangle}`, and the z -axis, `every z tick/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every x tick`.

```
/pgfplots/every minor x tick (style, initially empty)
/pgfplots/every minor y tick (style, initially empty)
/pgfplots/every minor z tick (style, initially empty)
```

Installed for minor tick lines on either x or y axis.

```
/pgfplots/minor x tick style={\langle key-value-list \rangle}
/pgfplots/minor y tick style={\langle key-value-list \rangle}
/pgfplots/minor z tick style={\langle key-value-list \rangle}
```

An abbreviation for `every minor x tick/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every minor y tick/.append style={\langle key-value-list \rangle}`, and the z -axis, `every minor z tick/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every minor x tick`.

```
/pgfplots/every major x tick (style, initially empty)
/pgfplots/every major y tick (style, initially empty)
/pgfplots/every major z tick (style, initially empty)
```

Installed for major tick lines on either x or y axis.

```
/pgfplots/major x tick style={\langle key-value-list \rangle}
/pgfplots/major y tick style={\langle key-value-list \rangle}
/pgfplots/major z tick style={\langle key-value-list \rangle}
```

An abbreviation for `every major x tick/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every major y tick/.append style={\langle key-value-list \rangle}`, and the z -axis, `every major z tick/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every major x tick`.

```
/pgfplots/every extra x tick (style, no value)
/pgfplots/every extra y tick (style, no value)
/pgfplots/every extra z tick (style, no value)
```

Allows to configure the appearance of ‘`extra x ticks`’. This style is installed before touching the first extra x tick. It is possible to set any option which affects tick or grid line generation.

The initial setting is

```
\pgfplotsset{
  every extra x tick/.style={/pgfplots/log identify minor tick positions=true},
  every extra y tick/.style={/pgfplots/log identify minor tick positions=true}}
```

Useful examples are shown below.

```
\pgfplotsset{every extra x tick/.append style={grid=major}}
\pgfplotsset{every extra x tick/.append style={major tick length=0pt}}
\pgfplotsset{every extra x tick/.append style={/pgf/number format=sci subscript}}
\pgfplotsset{extra x tick style={
  color=red,
  tickwidth=3mm,
  % the initial 'every tick style' defines a 'line width'.
  % this here redefines it:
  tick style={
    line width=2mm,
  },
}
}
```

```
/pgfplots/extra x tick style={\langle key-value-list \rangle}
```



```
/pgfplots/extra y tick style={\langle key-value-list \rangle}
/pgfplots/extra z tick style={\langle key-value-list \rangle}
```

An abbreviation for `every extra x tick/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every extra y tick/.append style={\langle key-value-list \rangle}`, and the z -axis, `every extra z tick/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every extra x tick`.

```
/pgfplots/extra tick style={\langle key-value-list \rangle}
```

An abbreviation which appends `\langle key-value-list \rangle` to `every extra x tick`, `every extra y tick` and `every extra z tick`.

Styles for grid lines

```
/pgfplots/every axis grid (style, initially thin,black!25)
```

Used for each grid line.

```
/pgfplots/grid style={\langle key-value-list \rangle}
```

An abbreviation for `every axis grid/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every axis grid`.

```
/pgfplots/every minor grid (style, initially empty)
```

Used for each minor grid line, installed after ‘`every axis grid`’.

```
/pgfplots/minor grid style={\langle key-value-list \rangle}
```

An abbreviation for `every minor grid/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every minor grid`.

```
/pgfplots/every major grid (style, initially empty)
```

Likewise, for major grid lines.

```
/pgfplots/major grid style={\langle key-value-list \rangle}
```

An abbreviation for `every major grid/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every major grid`.

```
/pgfplots/every axis x grid (style, initially empty)
```

```
/pgfplots/every axis y grid (style, initially empty)
```

```
/pgfplots/every axis z grid (style, initially empty)
```

Used for each grid line in either x or y direction.

```
/pgfplots/x grid style={\langle key-value-list \rangle}
```

```
/pgfplots/y grid style={\langle key-value-list \rangle}
```

```
/pgfplots/z grid style={\langle key-value-list \rangle}
```

An abbreviation for `every axis x grid/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every axis y grid/.append style={\langle key-value-list \rangle}`, and the z -axis, `every axis z grid/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every axis x grid`.

```
/pgfplots/every minor x grid (style, initially empty)
```

```
/pgfplots/every minor y grid (style, initially empty)
```

```
/pgfplots/every minor z grid (style, initially empty)
```

Used for each minor grid line in either x or y direction.

```
/pgfplots/minor x grid style={\langle key-value-list \rangle}
```

```
/pgfplots/minor y grid style={\langle key-value-list \rangle}
```

```
/pgfplots/minor z grid style={\langle key-value-list \rangle}
```

An abbreviation for `every minor x grid/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every minor y grid/.append style={\langle key-value-list \rangle}`, and the z -axis, `every minor z grid/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every minor x grid`.

`/pgfplots/every major x grid` (style, initially empty)
`/pgfplots/every major y grid` (style, initially empty)
`/pgfplots/every major z grid` (style, initially empty)

Used for each major grid line in either x or y direction.

`/pgfplots/major x grid style={\langle key-value-list \rangle}`
`/pgfplots/major y grid style={\langle key-value-list \rangle}`
`/pgfplots/major z grid style={\langle key-value-list \rangle}`

An abbreviation for `every major x grid/.append style={\langle key-value-list \rangle}` (or the respective styles for y , `every major y grid/.append style={\langle key-value-list \rangle}`, and the z -axis, `every major z grid/.append style={\langle key-value-list \rangle}`).

It appends options to the already existing style `every major x grid`.

Styles for error bars

`/pgfplots/every error bar` (style, initially thin)

Installed for every error bar.

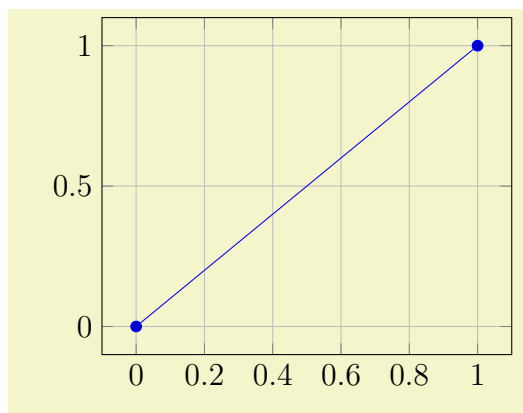
`/pgfplots/error bars/error bar style={\langle key-value-list \rangle}`

An abbreviation for `every error bar/.append style={\langle key-value-list \rangle}`.

It appends options to the already existing style `every error bar`.

4.17.2 (Re)Defining Own Styles

Use `\pgfplotsset{\langle style name \rangle/.style={\langle key-value-list \rangle}}` to create own styles. If $\langle style name \rangle$ exists already, it will be replaced. Please note that it is *not* possible to use the TikZ-command `\tikzstyle{\langle style name \rangle}=[]` in this context⁵³.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{my personal style/.style=
  {grid=major,font=\large}}

\begin{tikzpicture}
\begin{axis}[my personal style]
  \addplot coordinates {(0,0) (1,1)};
\end{axis}
\end{tikzpicture}
```

4.18 Alignment Options

4.18.1 Basic Alignment

Alignment works with two main methods: a coordinate where the axis shall be drawn and an “anchor” inside of the axis which shall be drawn at this particular coordinate. This methodology is common for each TikZ node – and an axis is nothing but a (special) TikZ node. The coordinate can be specified using the `at` key, while the anchor can be specified with the `anchor` key. In most cases, it is sufficient to provide only an anchor – unless one needs more than one axis in the same picture environment.

`/pgfplots/at={\langle coordinate expression \rangle}`

Assigns a position for the complete axis image. This option works similarly to the `at`-option of `\node[at={\langle coordinate expression \rangle}]`, see [5]. The common syntax is `at={(\langle x,y \rangle)}`.

⁵³This was possible in a previous version and is still supported for backwards compatibility. But in some cases, it may not work as expected.

The idea is to provide an $\langle \text{coordinate expression} \rangle$ where the axis will be placed. The axis' anchor will be placed at $\langle \text{coordinate expression} \rangle$.

`/pgfplots/anchor={\langle name \rangle}` (initially **south west**)

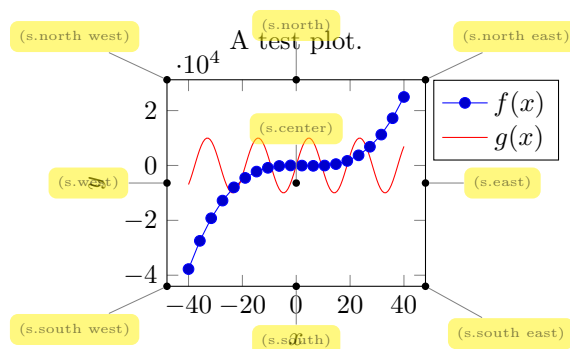
Chooses one of the different possible positions inside of an axis which is placed with `at`. The `at` key defines the position where to place the axis inside of the embedding picture, the `anchor` key defines which point of the axis shall be positioned by '`at`'. The initial configuration assumes `at={0,0}`. Thus, `anchor=center` will place the axis' center at the logical picture position (0,0). Similarly, `anchor=south west` will position the lower left corner of the axis at (0,0).

For users who are familiar with TikZ: an axis is actually a very special node, so anchors work as in [5].

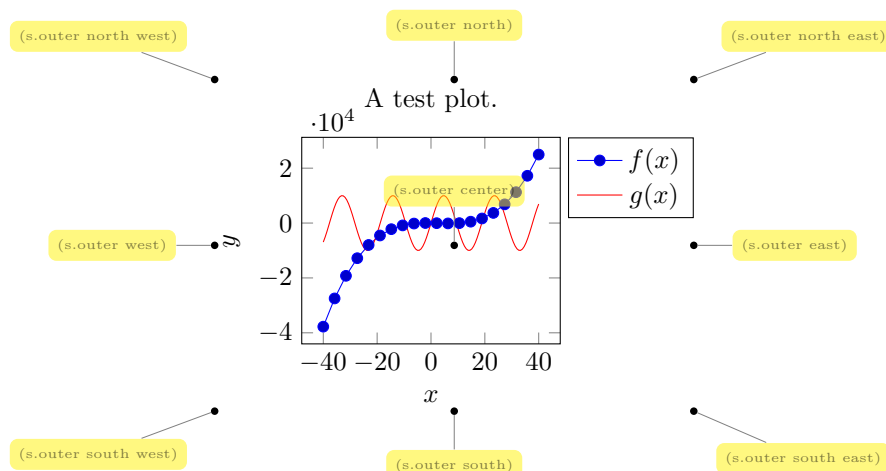
Anchors are useful in conjunction with horizontal or vertical alignment of plots, see the examples below.

There are four sets of anchors available: anchors positioned on the axis bounding box, anchors on the outer bounding box and anchors which have one coordinate on the outer bounding box and the other one at a position of the axis rectangle. Finally, one can place anchors near the origin.

In more detail, we have anchors on the axis rectangle (the bounding box around the axis)⁵⁴,

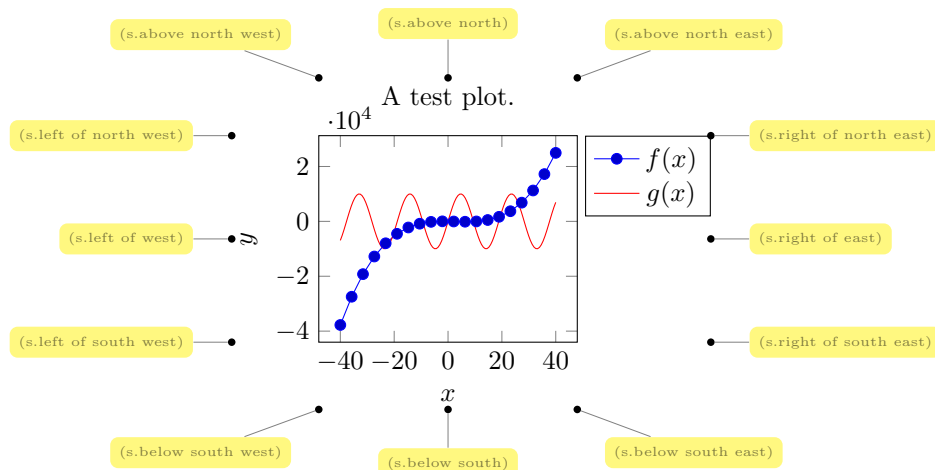


Anchors on the outer bounding box,

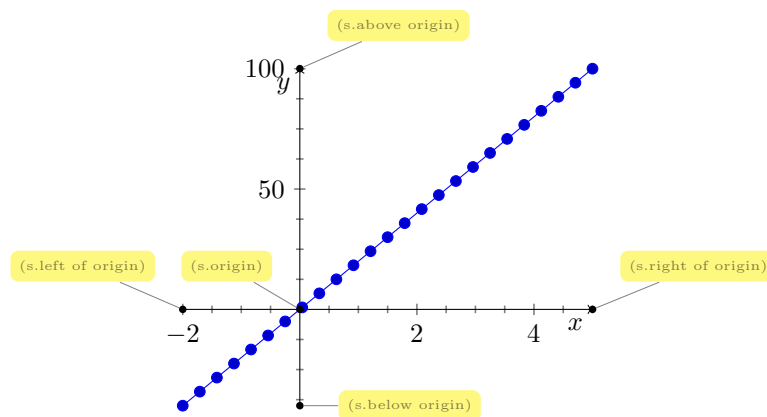


There are anchors which have one coordinate on the outer bounding box, and one on the axis rectangle,

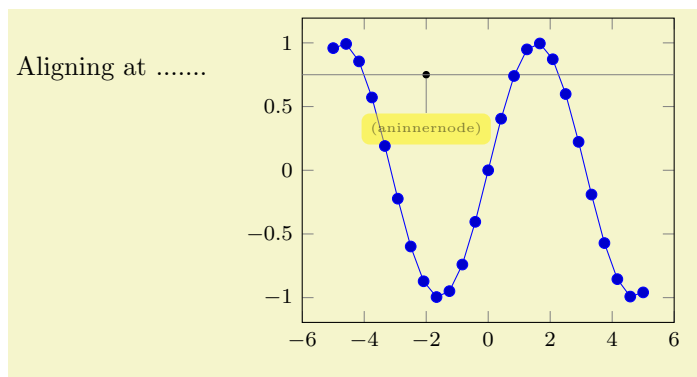
⁵⁴Versions prior to PGFPLOTS v.1.3 did *not* use the bounding box of the axis, they used axis coordinates to orient these anchors. This has been fixed. If you *really* want to undo the bugfix, see `compat/anchors`.



And finally, we have origin anchors which are especially useful when axis lines pass through the origin,

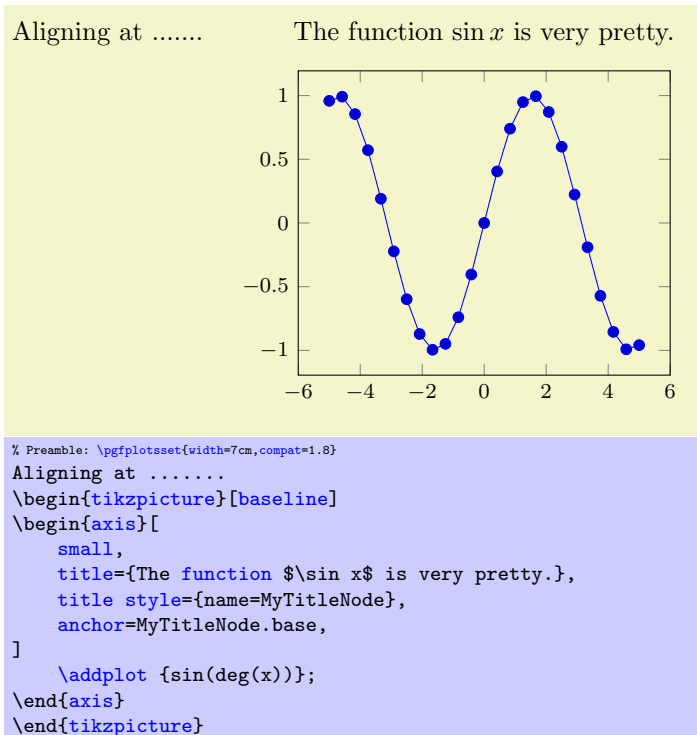


There is a fifth anchor which is not directly related to the axis: you can provide the anchor of a *named inner node*. Thus, you can define your own anchor, by writing `\node (<name>) at (<point coordinate>){}`; as follows (using the `baseline` option described below):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
Aligning at .....
\begin{tikzpicture}[baseline]
\begin{axis}[small,anchor=aninnernode.center]
\addplot {sin(deg(x))};
\node
[pin=-90:(aninnernode),fill=black,circle,scale=0.3]
(aninnernode) at (axis cs:-2,0.75) {};
\draw[help lines] (axis cs:-6,0.75) -- (axis cs:6,0.75);
\end{axis}
\end{tikzpicture}
```

What happens is that a node is placed at `(axis cs:-2,0.75)`. Note that the options `[pin=...]` are merely to show the `\node` (the pin style has been defined by the PGFPLLOTS manual). Since a name can also be assigned using `name=<node's name>` and since any PGFPLLOTS description is also a `\node`, you can align your plot at selected axis descriptions:



The default value is `anchor=south west`. You can use anchors in conjunction with the TikZ `baseline` option and/or `\begin{pgfinterruptboundingbox}` to perform alignment.

Remarks: Each of the anchors on the axis rectangle has an equivalent to a coordinate in the `axis description cs` described in Section 4.8.1. That means the first set of anchors actually lives on the *tight bounding box around the axis* (without any ticks or descriptions). The `south west` anchor will always be the lower left corner of this bounding box, even in case of a rotated or skewed coordinate system⁵⁵. Similar statements hold for the other anchors.

4.18.2 Vertical Alignment with baseline

`/tikz/baseline` (no value)

The `baseline` option should be provided as argument to a `tikzpicture`. It configures TikZ to shift the picture position $y = 0$ to the embedding text's baseline:

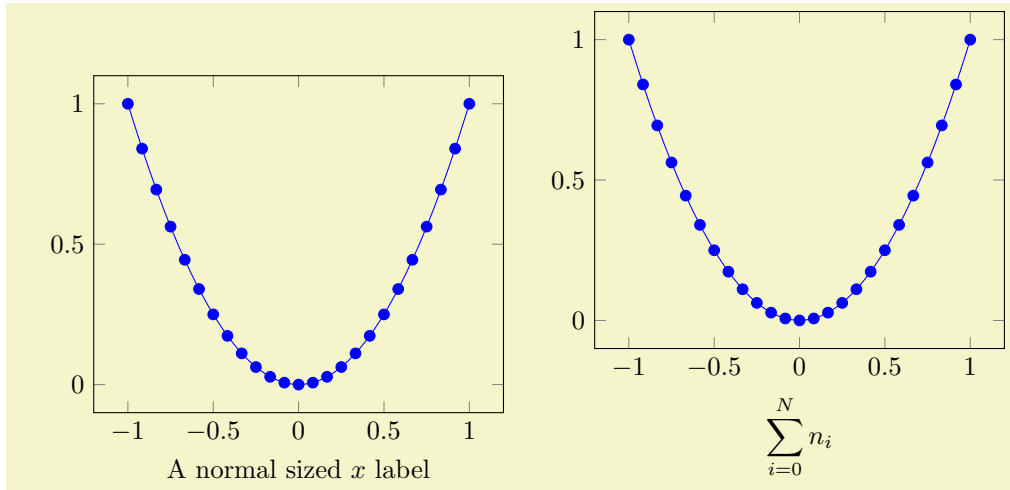
This is  a picture, here  another one.

```
This is \tikz[baseline]\fill[red] (0,0) circle(3pt); a picture,
here \tikz[baseline]\fill[red] (0,10pt) circle(3pt); another one.
```

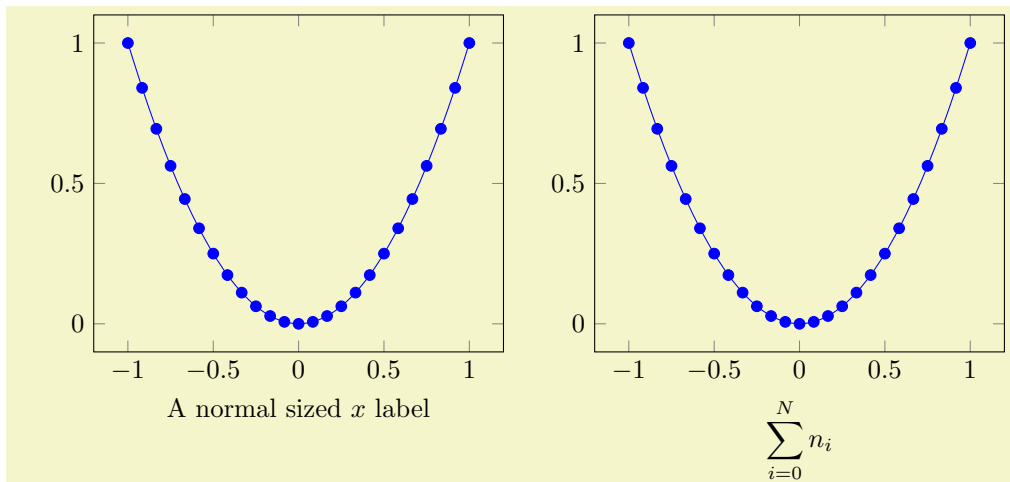
Consequently, the `baseline` option allows to align different `tikzpictures`. An axis is, by default, placed with `at={(0,0)}`, and the `anchor` key specifies which part of the axis is placed at $(0,0)$. Consequently, the `baseline` option, together with `anchor`, allows to align different axes with the embedding text.

The default axis anchor is `south west`, which means that the picture coordinate $(0,0)$ is the lower left corner of the axis. As a consequence, the TikZ option “`baseline`” allows vertical alignment of adjacent plots:

⁵⁵Note that this is only true for versions since 1.3.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% 1. Unaligned:
\pgfplotsset{domain=-1:1}
\begin{tikzpicture}
  \begin{axis}[xlabel=A normal sized  $x$  label]
    \addplot[smooth,blue,mark=*] {x^2};
  \end{axis}
\end{tikzpicture}%
\hspace{0.15cm}
\begin{tikzpicture}
  \begin{axis}[xlabel={\displaystyle \sum_{i=0}^N n_i}]
    \addplot[smooth,blue,mark=*] {x^2};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% 2. Aligned:
\pgfplotsset{domain=-1:1}
\begin{tikzpicture}[baseline]
  \begin{axis}[xlabel=A normal sized  $x$  label]
    \addplot[smooth,blue,mark=*] {x^2};
  \end{axis}
\end{tikzpicture}%
\hspace{0.15cm}
\begin{tikzpicture}[baseline]
  \begin{axis}[xlabel={\displaystyle \sum_{i=0}^N n_i}]
    \addplot[smooth,blue,mark=*] {x^2};
  \end{axis}
\end{tikzpicture}
```

Note that it is also possible to write `baseline=5cm` in which case the image offset at $y = 5\text{cm}$ will be used as baseline.

The `baseline` key is related to `\begin{minipage}[\langle alignment \rangle]` or `\begin{tabular}[\langle alignment \rangle]`:

the $\langle alignment \rangle$ tells L^AT_EX which part of the `minipage` or `tabular` shall be positioned on the baseline. Thus, `baseline` does the same for pictures (with more freedom for $\langle alignment \rangle$).

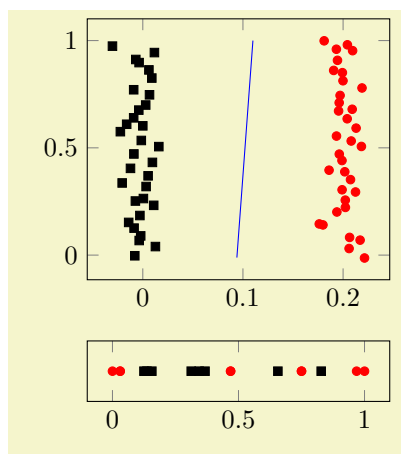
4.18.3 Horizontal Alignment

Horizontal alignment can be done in two ways:

1. Using separate `tikzpicture` environments which have reduced bounding boxes or
2. A single `tikzpicture` environment in which the complete alignment is done.

The first approach requires the use of reduced bounding boxes and is discussed in Section 4.19.1.

The second approach, a single `tikzpicture` environment, employs the `at` and `anchor` keys to align parts of the images. For example, if you place multiple `axes` into a single `tikzpicture` and use the ‘`anchor`’-option, you can control horizontal alignment:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\pgfplotsset{every axis/.append style={
cycle list={
{red,only marks,mark options={
fill=red,scale=0.8,mark=*},
{black,only marks,mark options={
fill=black,scale=0.8,mark=square*}}}}}
\begin{axis}[width=4cm,scale only axis,
name=main plot]
\addplot file
{plotdata/pgfplots_scatterdata1.dat};
\addplot file
{plotdata/pgfplots_scatterdata2.dat};
\addplot[blue] coordinates {
(0.093947, -0.011481)
(0.101957, 0.494273)
(0.109967, 1.000027)};
\end{axis}

\begin{axis}[
at={(main plot.below south west)},yshift=-0.1cm,
anchor=north west,
width=4cm,scale only axis,height=0.8cm,
ytick=\empty]
\addplot file
{plotdata/pgfplots_scatterdata1_latent.dat};
\addplot file
{plotdata/pgfplots_scatterdata2_latent.dat};
\end{axis}
\end{tikzpicture}
```

Here, the second axis uses `at={(main plot.below south west)}` to be placed below the first one. Furthermore, it has `yshift=-0.1cm` in order to leave additional space, and it uses `anchor=north west` to place the upper left corner at the specified position. Instead of the `at={}` construction, we could also have used `yshift` with larger negative shift.

4.18.4 Alignment In Array Form (Subplots)

Sometimes multiple alignment axes in array form are desired. PGFPLOTS supports this task in several ways which are described in the following. There are basically three related, yet different, approaches:

1. Simply place `\begin{tikzpicture}...\end{tikzpicture}` into a L^AT_EX table. This is straightforward; you would do the very same thing with `\includegraphics`.
In addition to `\includegraphics`, the `baseline` feature allows simple yet effective vertical alignment. In addition, the `trim left` and `trim right` features allow simple yet effective horizontal alignment (see below).
2. Use a *single* picture which contains an array of axes, i.e. a pattern like

`\begin{tikzpicture} \matrix{ <multiple axes>; \end{tikzpicture}`.

This allows considerably simpler alignment! Alas, it needs special handling for `legend entries` due to a weakness of `\matrix`. If you use the `external` library (which is recommended), it takes more time since the picture gets larger.

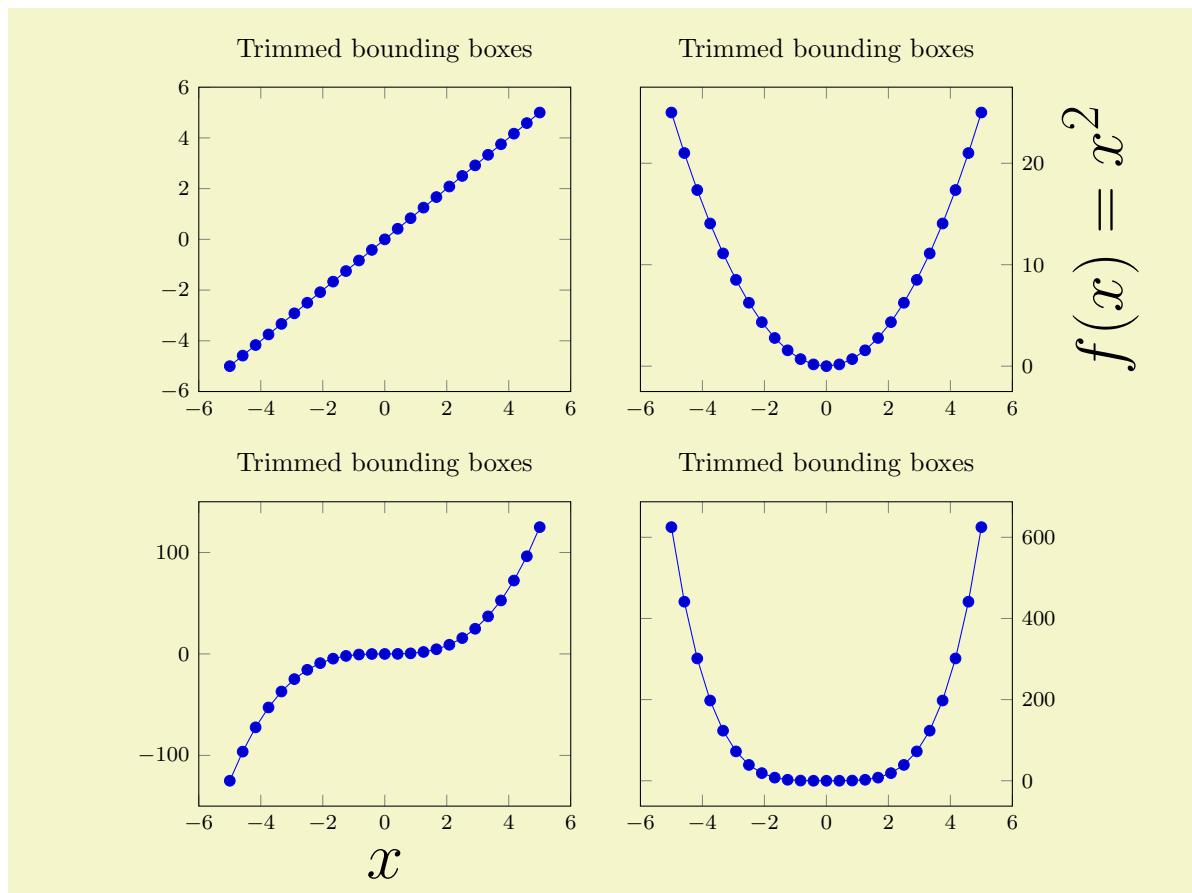
3. Use the `groupplots` library shipped with PGFPLOTS. It is specialized on axes in array form with particular strength if the axes are closely related (for example if they share axis descriptions like `xlabel` or even tick labels). Note, however, that the other approaches are better when it comes to automatic handling of bounding boxes.

The `groupplots` library is discussed in all detail in Section 5.5. This section discusses the other two approaches.

Array Alignment using L^AT_EX Tables The idea is simple: use a L^AT_EX table and provide one `tikzpicture` for every cell. You are probably familiar with this sort of alignment, perhaps together with `\includegraphics`. It works in the very same way for PGFPLOTS. The approach is the simplest one since it doesn't need special knowledge. Its disadvantage, however, is more difficulty to control positions *inside* of the image (like differently sized axis descriptions).

It is strongly recommended to employ the `baseline` option for each cell picture, which simplifies vertical alignment considerably. If you want a simple solution to place separate axes in array form, and you prefer to use one `tikzpicture` for every axis, the probably most simple and most effective way to get horizontal alignment are the `trim left` and `trim right` features – or styles based on them:

The `trim axis left` feature can be used to exclude axis descriptions on the left from the bounding box, and the `trim axis right` can exclude axis descriptions on the right from the bounding box. Thus, alignment is done using the vertical axis lines. Since both keys effectively modify the bounding box, they are documented in Section 4.19.1 “Bounding Box Restrictions”. Here is just a small example for array alignment by means of `tabular`, `baseline` and the `trim left`/`trim right` features:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{
  small,
  title=Trimmed bounding boxes
}
\begin{center}
\begin{tabular}{rl}
\begin{tikzpicture}[baseline,trim axis left]
\begin{axis}
\addplot {x};
\end{axis}
\end{tikzpicture}
&
\begin{tikzpicture}[baseline,trim axis right]
\begin{axis}[
  ylabel={f(x)=x^2},
  yticklabel pos=right,
  ylabel style={font=\Huge}]
\addplot {x^2};
\end{axis}
\end{tikzpicture}
\\
%
\begin{tikzpicture}[baseline,trim axis left]
\begin{axis}[xlabel=$x$,xlabel style={font=\Huge}]
\addplot {x^3};
\end{axis}
\end{tikzpicture}%
&
\begin{tikzpicture}[baseline,trim axis right]
\begin{axis}[yticklabel pos=right]
\addplot {x^4};
\end{axis}
\end{tikzpicture}%
\\
\end{tabular}%
\end{center}

```

The example has 2×2 axes. The `baseline` feature controls the vertical alignment: the lower axis lines are always on the same height. The `trim axis left` key is a style which tells *TikZ* to trim everything which is left of the left axis line. Similarly, the `trim axis right` key does not include picture parts right of the right axis line. Together with `\begin{center}` and the `yticklabel pos=right` key, we get correct horizontal and vertical alignment together with centering at the left- and right axis lines (without descriptions).

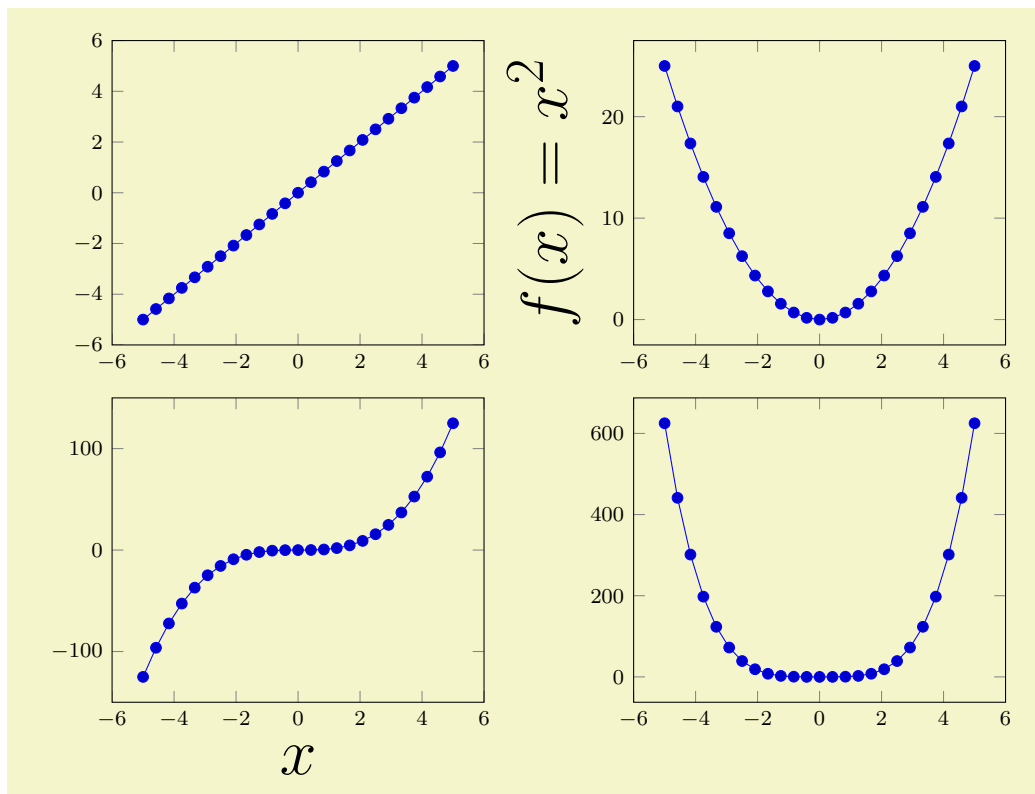
A strong advantage is that this type of alignment requires almost no changes to your pictures. Thus, you can copy-paste existing images (T_EX code) relatively simple.

Note that the approach is fully compatible with the image `externalization` library: each picture is exported separately, and the bounding box restrictions (and the `baseline` offset) are stored in separate `.dpth` files. The `trim left/trim right` approach for horizontal alignment is the *only* supported way for reduced bounding boxes and image externalization.

Array Alignment using *TikZ* Matrices While it is possible to use (for example) `tabular` combined with the vertical and horizontal alignment methods discussed above, it might be better to use a *TikZ matrix* since it automatically handles the size of axis descriptions.

A *TikZ* matrix is some sort of “graphical” table. It knows everything about picture alignment and it has more flexibility than `tabular` when it comes to graphics. The idea is to pack the complete array into a *single* picture.

The complete documentation of a *TikZ* matrix is beyond the scope of this manual, please refer to [5] for details. But we provide an example here:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \pgfplotsset{small}
  \matrix {
    \begin{axis}
      \addplot {x};
    \end{axis}
    &
    % differently large labels are aligned automatically:
    \begin{axis}[ylabel={f(x)=x^2},ylabel style={font=\Huge}]
      \addplot {x^2};
    \end{axis}
    \\
    %
    \begin{axis}[xlabel={x},xlabel style={font=\Huge}]
      \addplot {x^3};
    \end{axis}
    &
    \begin{axis}
      \addplot {x^4};
    \end{axis}
    \\
  };
\end{tikzpicture}
```

So, a matrix is a picture element inside of `tikzpicture`. Its cells are separated by ‘&’ as in tabular (or, if ‘&’ causes problems, with `\pgfmatrixnextcell`). Its rows are separated by ‘\\’. Each cell is aligned using the cells’ anchor. Since, by default, the anchor of an axis is placed at the lower left corner, the example above is completely aligned, without the need for any bounding box modifications – even the labels are aligned correctly. If another anchor shall be used, simply place

```
\pgfplotsset{anchor=...}
\matrix {
  ...
};
```

in front of the matrix. This will use the same configuration for every sub-plot.

Attention: Unfortunately, the array alignment with `\matrix` needs special *attention with legends*. A legend is also a `\matrix` and TikZ matrices can't be nested. You will need to use the `legend to name` feature (or to assemble a legend by means of `\label` and `\ref`) to overcome this weakness (see Section 4.8.6 for details).

4.18.5 Miscellaneous for Alignment

Predefined node `current axis`

A node which refers to the current axis or the last typeset axis.

You can use this node in axis descriptions, for example to place axis labels or titles.

Remark: If you use `current axis` inside of axis descriptions, the “current axis” is not yet finished. That means you *can't use any outer anchor* inside of axis descriptions.

It is also possible to use `current axis` in any drawing or plotting commands inside of an axis (but no outer anchor as these are not defined when drawing commands are processed). This usage is similar to the `axis description cs`.

4.19 The Picture's Size: Bounding Box and Clipping

This section explains how a picture receives its final dimensions. The picture's dimension is the bounding box. It is possible to restrict the bounding box, but display graphical elements outside of the bounding box. This is called subject of Section 4.19.1. Another use-case is to restrict both the bounding box and the clip the graphical elements to some outer path which is subject of Section 4.19.2.

4.19.1 Bounding Box Restrictions

Bounding box restrictions are a useful and often necessary tool if multiple pictures need to be aligned properly. Consequently, it is often applied together with the Alignments methods of Section 4.18.

Bounding box restrictions can be achieved with several methods of PGF:

1. The `overlay` option,
2. The `pgfinterruptboundingbox` environment,
3. The `\pgfresetboundingbox` command,
4. The `\useasboundingbox` path,
5. The `trim left` and `trim right` feature (which is the *only* supported way of restricted bounding boxes and image externalization; at least for PDF output).

An additional item is a specific use-case of PGFPLOTS:

6. The `hide axis` feature will exclude any axis-specific stuff from the bounding box. See the reference for `hide axis` for details.

Note that image externalization (the `external` library) is more or less incompatible with methods (1.)–(4.). The problem is that `pdflatex` crops everything outside of the bounding box away. There are only two safe ways to “restrict” bounding boxes of external `.pdf` images: the first is the mentioned `trim left/trim right` feature and the second is to use negative `\hspace` or `\vspace` commands (or options to `\includegraphics`).

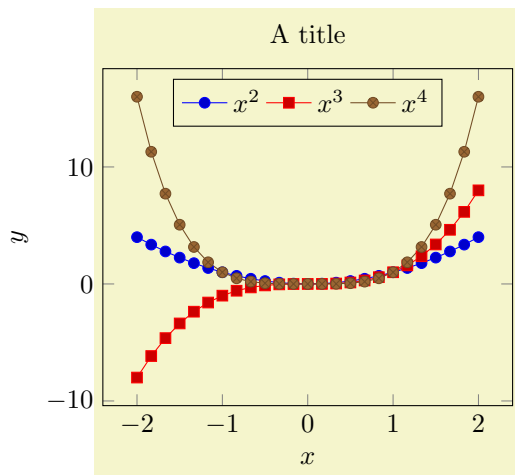
`/tikz/overlay`

(no value)

A special key of PGF which disables bounding box updates for (parts of) the image. The effect is that those parts are an “overlay” over the document.

For PGFPLOTS, `overlay` can be useful to position legends or other axis descriptions outside of the axis – without affecting its size (and without affecting alignment).

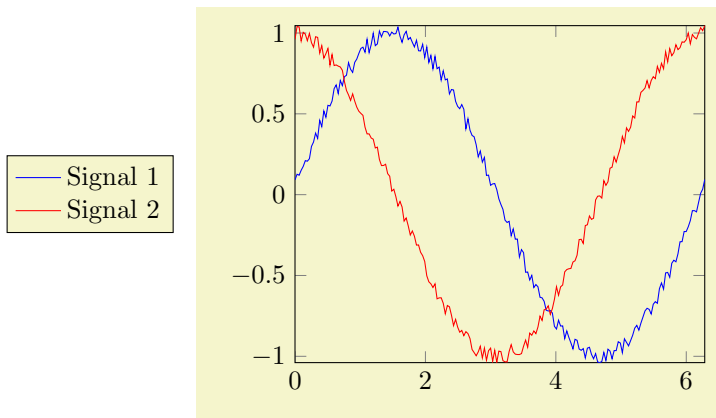
For example, one may want to include only certain parts of the axis into the final bounding box. This would allow horizontal alignment (centering):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}%
  \begin{axis}[
    title=A title,
    ylabel style={overlay},
    yticklabel style={overlay},
    xlabel={\$x\$},
    ylabel={\$y\$},
    legend style={at={(0.5,0.97)},
      anchor=north,legend columns=-1},
    domain=-2:2
  ]
  \addplot {x^2};
  \addplot {x^3};
  \addplot {x^4};
  \legend{\$x^2\$, \$x^3\$, \$x^4\$}
  \end{axis}
\end{tikzpicture}%
```

Now, the left axis descriptions (y label and y ticks) stick out of the bounding box.

The following example places a legend somewhere without affecting the bounding box.

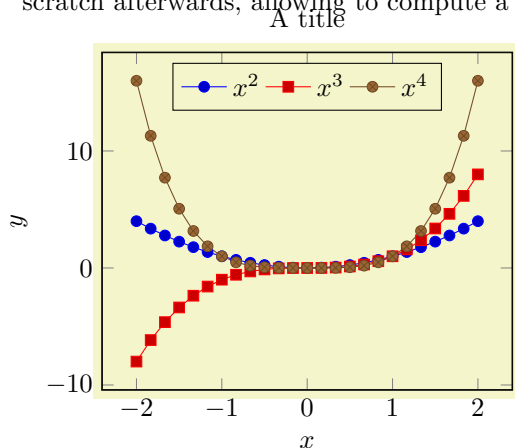


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    domain=0:6.2832,samples=200,
    legend style={
      overlay,
      at={(-0.5,0.5)},
      anchor=center},
    every axis plot post/.append style={mark=none},
    enlargelimits=false]
  \addplot {sin(deg(x))+rand*0.05};
  \addplot {cos(deg(x))+rand*0.05};
  \legend{Signal 1,Signal 2}
  \end{axis}
\end{tikzpicture}
```

More information about the `overlay` option can be found in the PGF manual [5].

`\pgfresetboundingbox`

This command of PGF resets the bounding box of the current picture. The computation starts from scratch afterwards, allowing to compute a user-defined bounding box.



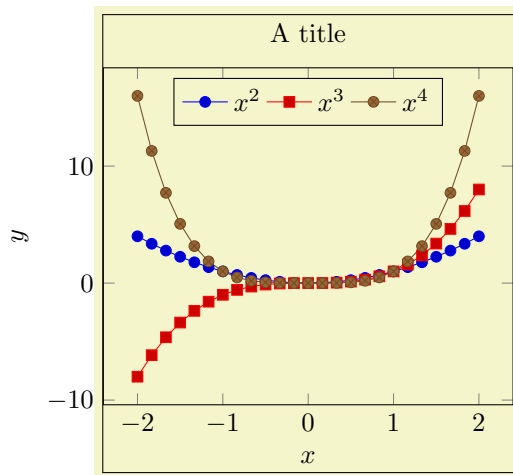
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\setlength{\fboxsep}{0pt}%
\fbbox{%
\begin{tikzpicture}%
  \begin{axis}[
    title=A title,
    xlabel={\$x\$},
    ylabel={\$y\$},
    legend style={at={(0.5,0.97)},
      anchor=north,legend columns=-1},
    domain=-2:2
  ]
  \addplot {x^2};
  \addplot {x^3};
  \addplot {x^4};
  \legend{\$x^2\$, \$x^3\$, \$x^4\$}
  \end{axis}

  \pgfresetboundingbox
  \path
    (current axis.south west)
    rectangle (current axis.north east);
\end{tikzpicture}%
}%
```

The example draws a normal picture, containing an axis. Afterwards, it throws the bounding box away and creates a new one based on the `current axis` node and its anchors.

```
\begin{pgfinterruptboundingbox}
  <environment contents>
\end{pgfinterruptboundingbox}
```

Yet another approach with the same effect is shown below: the bounding box is interrupted manually, and resumed afterwards.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\setlength{\fboxsep}{0pt}%
\fbbox{%
\begin{tikzpicture}%
  \begin{pgfinterruptboundingbox}
    \begin{axis}[
      title=A title,
      xlabel={x},
      ylabel={y},
      legend style={at={(0.5,0.97)},
        anchor=north,legend columns=-1},
      domain=-2:2
    ]
      \addplot {x^2};
      \addplot {x^3};
      \addplot {x^4};
      \legend{$x^2$, $x^3$, $x^4$}
    \end{axis}
  \end{pgfinterruptboundingbox}

  \useasboundingbox
    (current axis.below south west)
    rectangle (current axis.above north east);
\end{tikzpicture}%
}%
```

The `pgfinterruptboundingbox` environment does not include its content into the image's bounding box, and `\useasboundingbox` sets the picture's bounding box to the following argument (see [5]).

```
/tikz/trim left={<x coordinate or point>} (default 0pt)
/tikz/trim right={<x coordinate or point>}
```

These two keys allow to reduce the size of the bounding box.

The `trim left` key expects either a single x coordinate like `1cm` or a point like `(current axis.west)`. If a point is provided, it uses only the x coordinate of that point. Then, the left end of the bounding box is set to the resulting x coordinate and everything left of it is outside of the bounding box.

The `trim right` key has the same effect, only for the right end of the bounding box.

More detailed documentation can be found in the TikZ manual.

```
/tikz/trim axis left (style, no value)
```

A style with value `trim left=(current axis.south west)`.

The style needs to be provided as argument to `\begin{tikzpicture}[trim axis left]`. It expects (at least) one PGFLOTS environment in the picture. The effect is to trim everything which is left of the last axis' anchor `south west` (i.e. everything left of the left axis boundary).

```
/tikz/trim axis right (style, no value)
```

A style with value `trim right=(current axis.south east)`.

It works similarly to `trim axis left`: the effect is that everything right of the right axis line of the last axis environment is truncated from the bounding box.

```
/tikz/trim axis group left (style, no value)
```

A style which has the same effect as `trim axis left`, but is tailored for the `groupplots` library.

It has the value `trim left=(group c1r1.south west)`.

The style needs to be provided as argument to `\begin{tikzpicture}[trim axis group left]`. It expects (at least) one `groupplot` environment in the picture. The effect is to trim everything which is left of the first group axis' anchor `south west` (i.e. everything left of the left axis boundary).

`/tikz/trim axis group right` (style, no value)

A style which has the same effect as `trim axis right`, but is tailored for the `groupplots` library. It works similarly to `trim axis group left`: the effect is that everything right of the rightmost axis in a group plot (the last element of the `groupplot` environment) is truncated from the bounding box.

4.19.2 Clipping

Clipping influences both the picture size and the visible output in contrast to bounding box restrictions which reduce the picture's final size while keeping the same graphical output.

Typically, PGFLOTS uses the path for a boxed axis as clip path. However, clipping has some special features and fine-tuning keys which are explained in this section.

`/pgfplots/clip=true|false` (initially true)

Controls whether any paths inside of an axis shall be clipped.

This is in effect even if `hide axis=true`.

Note that a clip path can contribute to the picture's bounding box. Starting with `compat=1.8`, PGFLOTS applies intelligence to separate the responsibilities clipping and bounding box control, see `clip bounding box` and its choices. As of `compat=1.8`, a clip path can be in effect although the bounding box is considerably smaller than the clip path. This is typically what one expects if the clip path is invisible.

The clip path is generated using `\pgfplotspathaxisoutline`, i.e. it is the path induced by boxed axis lines. For a three-dimensional plot, only the outer axis lines are used. A plot with centered axis lines uses the outer axis lines as well.

`/pgfplots/clip marker paths=true|false` (initially false)

The initial choice `clip marker paths=false` causes markers to be drawn *after* the clipped region. Only their positions will be clipped.

As a consequence, markers will be drawn completely, or not at all. The value `clip marker paths=true` is here for backwards compatibility: it does not introduce special marker treatment, so markers may be drawn partially if they are close to the clipping boundary⁵⁶.

This key has no effect if `clip=false`.

Note that clip marker paths also affects the sequence in which plots and their markers are drawn on top of each other. See also the related key `clip mode`.

`/pgfplots/clip bounding box=default tikz|upper bound` (initially controlled by compat key)

Controls how the path generated by `clip=true` contributes to the bounding box. This has a consequence for `axis lines=box`, in particular, for `hide axis`: if the value is `default tikz`, hiding (parts of) the axis will not reduce the bounding box because the clip path is as large as before. The value `upper bound` allows to reduce the bounding box also in case of `hide axis`.

More precisely, the choice `default tikz` installs the clip path induced by the axis as ordinary TikZ path (see `\pgfplotspathaxisoutline`). That means its bounding box essentially contributes to the picture's bounding box, irrespective of the size of contained paths.

The choice `upper bound` allows to *reduce* the picture's bounding box to what is actually shown: if the picture only contains graphical elements which are completely within the bounding box of `\pgfplotspathaxisoutline`, the bounding box is made up of those contained elements. If the contained elements are actually larger than the bounding box of `\pgfplotspathaxisoutline`, they are clipped to the outline's path ("upper bound"). The latter case ensures that parts of the graphics which are excluded by `clip` are not counted for the bounding box.

Keep in mind that `hide axis` is independent of `clip=true`: the clip path might still be in effect even though the axis outline is invisible.

⁵⁶Please note that clipped marker paths may be slightly faster during TeX compilation.

This key is irrelevant if `clip=false`. In addition, it has no effect for `axis lines=box` since the `box` path is made up from `\pgfplotspathaxisoutline`. It has an effect for `hide axis=true` or for choices of `axis lines` in which parts of the axis are empty.

This key is controlled by the `compat` level. Its default is `default tikz`. Since `compat=1.8`, it is set to `upper bound`.

The key has no effect if `clip=false`.

`/pgfplots/clip mode=global|individual` (initially `global`)

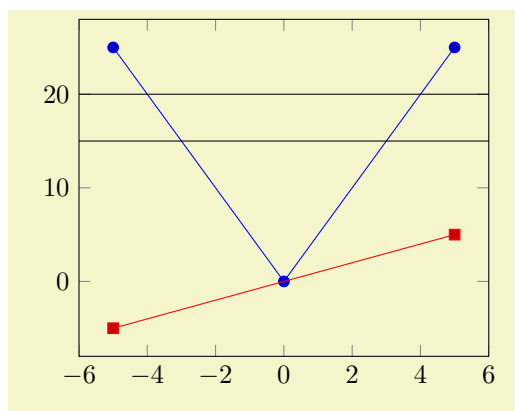
This key controls how PGFPLOTS implements the `clip=true` feature (which is on by default). Its primary motivation is control where markers are placed: are markers on top of everything else (choice `global`) or are they overdrawn by following plots (choice `individual`)?

The choice `global` tells PGFPLOTS to install one single clip path for the complete picture⁵⁷. In order to avoid clipped marker paths, any markers are processed after the clip path has been closed, i.e. on a separate layer (see `clip marker paths`). An unexpected side-effect is that marks are on top of plots, even if the plots have been added after the markers.

The choice `individual` instructs PGFPLOTS to install a separate clip path for *every* `\addplot` command. Consequently, the plot will be clipped. But most importantly, its markers will be drawn immediately after the clip path has been deactivated.

An unexpected side-effect of `clip mode=individual` is that

1. the resulting pdf will be slightly larger due to the repeated paths,
2. custom drawing instruction like `\node` or `\draw` need to be clipped *manually*: use



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    clip mode=individual,
  ]
    \addplot+[samples=3] {x^2};

    \begin{scope}
      \clip \pgfextra{\pgfplotspathaxisoutline};

      \draw (axis cs:-20,15) -- (axis cs:20,15);

      \draw (axis cs:-20,20) -- (axis cs:20,20);
    \end{scope}

    \addplot+[samples=2] {x};
  \end{axis}
\end{tikzpicture}
```

to install a custom clip path around your `\draw` instructions for such a use-case. Here, the path instruction `\pgfplotspathaxisoutline` results in a path of the axis outline, i.e. the path which is used for the background paths or for clipping. Since it is a basic level macro, it needs to be encapsulated by `\pgfextra`.

Note that `clip marker paths` can lead to the same result as `clip mode=individual` if the plot does not reach the boundaries.

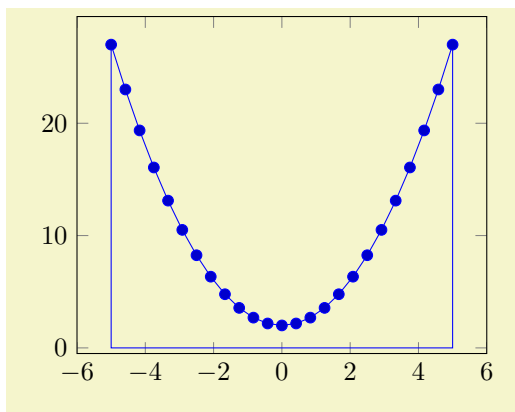
4.20 Closing Plots (Filling the Area Under Plots)

`\closedcycle`

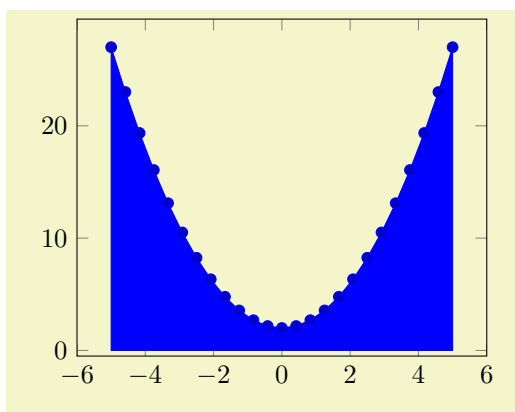
Provide `\closedcycle` as *⟨trailing path commands⟩* after `\addplot` to draw a closed line from the last plot coordinate to the first one.

Use `\closedcycle` whenever you intend to fill the area under a plot.

⁵⁷The choice `clip mode=global` was the only supported clipping mechanism up to and including version 1.5.

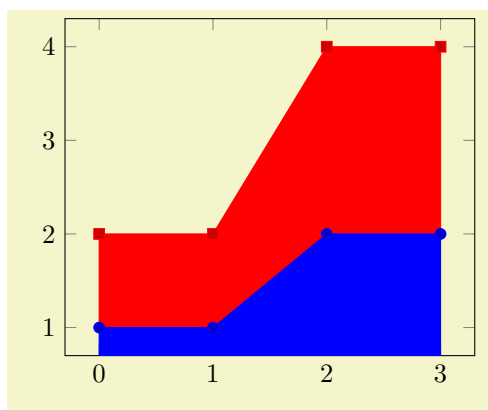


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot {x^2+2} \closedcycle;
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot+[fill] {x^2+2} \closedcycle;
  \end{axis}
\end{tikzpicture}
```

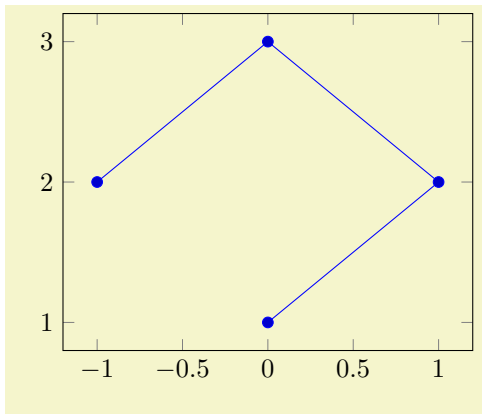
In case of stacked plots, `\closedcycle` connects the current plot with the previous plot instead of connecting with the x axis⁵⁸.



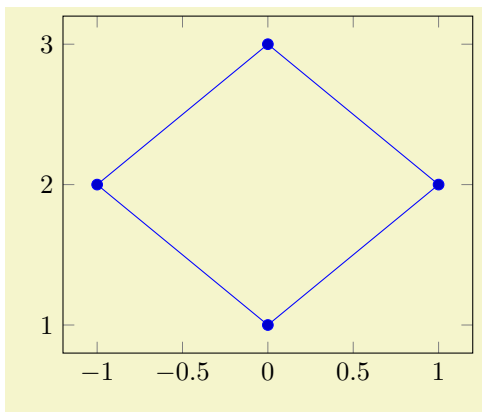
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[stack plots=y]
    \addplot+[fill] coordinates
      {(0,1) (1,1) (2,2) (3,2)} \closedcycle;
    \addplot+[fill] coordinates
      {(0,1) (1,1) (2,2) (3,2)} \closedcycle;
  \end{axis}
\end{tikzpicture}
```

Note that `\closedcycle` has been designed for functions (i.e. for a plot where every x has at most one y value). For arbitrary curves, you can safely use the TikZ path `--cycle` instead which simply connects the last and the first path element:

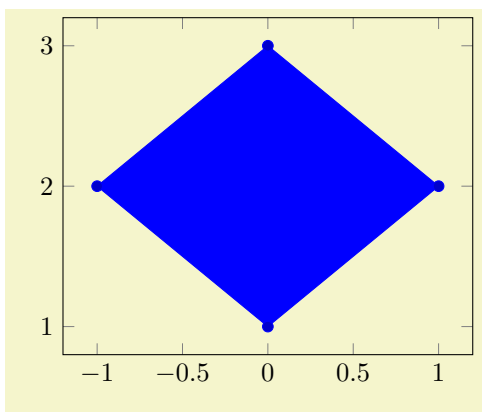
⁵⁸The implementation for stacked plots requires some additional logic to determine the filled area: `\closedcycle` will produce a `plot coordinates` command with *reversed* coordinates of the previous plot. This is usually irrelevant for end users, but it assumes that the plot's type is symmetric. Since constant plots are inherently asymmetric, `\closedcycle` will use `const plot mark right` as reversed sequence for `const plot mark left`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot coordinates
      {(0,1) (1,2) (0,3) (-1,2)};
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot coordinates
      {(0,1) (1,2) (0,3) (-1,2)} --cycle;
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot+[fill] coordinates
      {(0,1) (1,2) (0,3) (-1,2)} --cycle;
  \end{axis}
\end{tikzpicture}
```

The `--cycle` is actually a path instruction of [5]; it connects the first and the last coordinate of one path. Note that this is automatically done for `filled` paths.

4.21 Symbolic Coordinates and User Transformations

PGFPLOTS supports user transformations which can be applied to input and output coordinates. Suppose the plot shall display days versus account statements over time. Then, one wants to visualize date versus credit balance. But: dates need to be transformed to numbers before doing so! Furthermore, tick labels shall be displayed as dates as well. This, and more general transformations, can be implemented using the `x coord trafo` and `y coord trafo` keys.

Remark: This section applies to users who want to have non-standard input *coordinates*. If you have normal numbers which don't need to be transformed and you like to have special symbols as tick labels, you should consider using the `xticklabels` (`yticklabels`) key described on page 251.

```
/pgfplots/x coord trafo/.code={\langle...\rangle}
```

```

/pgfplots/y coord trafo/.code={\...}
/pgfplots/z coord trafo/.code={\...}
/pgfplots/x coord inv trafo/.code={\...}
/pgfplots/y coord inv trafo/.code={\...}
/pgfplots/z coord inv trafo/.code={\...}

```

These code keys allow arbitrary coordinate transformations which are applied to input coordinates and output tick labels.

The `x coord trafo` and `y coord trafo` command keys take one argument which is the input coordinate. They are expected to set `\pgfmathresult` to the final value.

At this level, the input coordinate is provided as it is found in the `\addplot` statement. For example, if x coordinates are actually of the form $\langle year \rangle - \langle month \rangle - \langle day \rangle$, for example 2008-01-05, then a useful coordinate transformation would transform this string into a number (see below for a predefined realization).

In short, *no* numerics has been applied to input coordinates when this transformation is applied⁵⁹.

The input coordinate transformation is applied to

- any input coordinates (specified with `\addplot` or `axis cs`),
- any user-specified `xtick` or `ytick` options,
- any user-specified `extra x ticks` and `extra y ticks` options,
- any user-specified axis limits like `xmin` and `xmax`.

The output coordinate transformation `x coord inv trafo` is applied to tick positions just before evaluating the `xticklabel` and `yticklabel` keys. The argument to `x coord inv trafo` is a fixed point number (which may have trailing zeros after the period). The tick label code may use additional macros defined by the inverse transformation.

Remark: PGFLOTS will continue to produce tick positions as usual, no extra magic is applied. It may be necessary to provide tick positions explicitly if the default doesn't respect the coordinate space properly.

The initial value of these keys is

```

\pgfplotsset{
  x coord trafo/.code={},
  x coord inv trafo/.code={}}

```

which simply disables the transformation (the same for y , of course).

Remark: It might be necessary to set

```

\pgfplotsset{
  xticklabel={\tick},
  scaled x ticks=false,
  plot coordinates/math parser=false,
}

```

in order to avoid number formatting routines on `\tick` or numerics for tick scale methods. This is done automatically by the predefined symbolic coordinate styles (see below).

4.21.1 String Symbols as Input Coordinates

It is possible to provide a string dictionary to PGFLOTS. An input coordinate can then use any symbol provided in that dictionary.

```

/pgfplots/symbolic x coords={\dictionary}
/pgfplots/symbolic y coords={\dictionary}
/pgfplots/symbolic z coords={\dictionary}

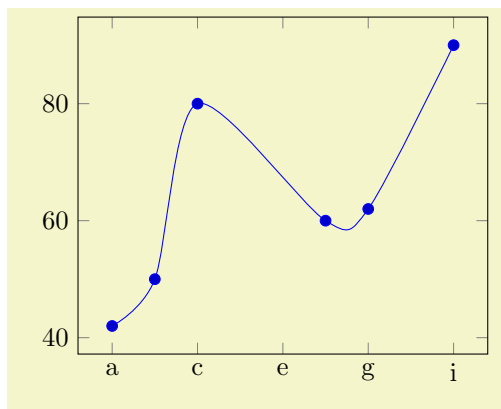
```

A style which sets `x coord trafo` and `x coord inv trafo` (or the respective y or z variants) such that any element in $\langle dictionary \rangle$ is a valid input coordinate. The $\langle dictionary \rangle$ can be a comma separated list

⁵⁹Of course, if coordinates have been generated by gnuplot or PGF, this does no longer hold.

or a list terminated with ‘\’. In both cases, white space is considered to be part of the names (use ‘%’ at end of lines).

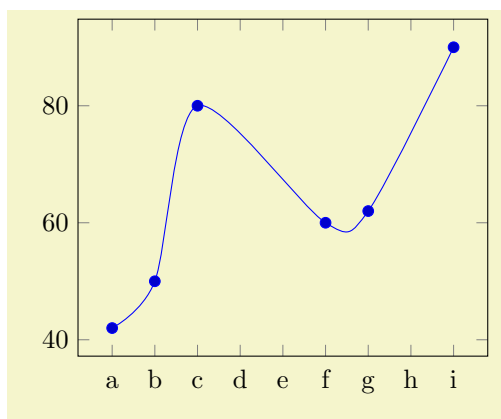
The dictionary will assign integer numbers to every element. These integers are used internally for arithmetics. Finally, the inverse transformation takes a fixed point number and maps it to the nearest integer, and that integer is mapped into the dictionary.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[symbolic x coords={a,b,c,d,e,f,g,h,i}]
\addplot+[smooth] coordinates {
(a,42)
(b,50)
(c,80)
(f,60)
(g,62)
(i,90)};
\end{axis}
\end{tikzpicture}
```

The effect of the transformation is simply that input coordinates can be elements of the dictionary and tick labels will be chosen out of this dictionary as well.

Note that `symbolic x coords` is more-or-less equivalent to explicitly provided `xtick` positions and `xticklabels`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
xtick={0,1,2,...,20},
xticklabels={a,b,c,d,e,f,g,h,i},
xticklabel style={
anchor=base,
yshift=-\baselineskip
},
]
\addplot+[smooth] coordinates {
(0,42)
(1,50)
(2,80)
(5,60)
(6,62)
(8,90)};
\end{axis}
\end{tikzpicture}
```

The difference is that the approach with `symbolic x coords` is simpler to read whereas the `xtick` approach is simpler with respect to coordinate arithmetics (for example to increase limits using `enlargelimits`). The `xticklabel style` here is an attempt to align all tick labels at their base line (which would be useful for `symbolic x coords` as well as soon as labels have characters which exceed the baseline).

See also the option to add tick and/or grid lines at every encountered coordinate using `xtick=data` (or `minor xtick=data`).

4.21.2 Dates as Input Coordinates

The already mentioned application of using dates as input coordinates has been predefined, together with support for hours and minutes. It relies on the PGF calendar library which converts dates to numbers in the Julian calendar. Then, one coordinate unit is one day.

```
\usepgfplotslibrary{dateplot} % LATEX and plain TEX
\usepgfplotslibrary[dateplot] % ConTEXt
\usetikzlibrary{pgfplots.dateplot} % LATEX and plain TEX
```

```
\usetikzlibrary[pgfplots.dateplot] % ConTeXt
Loads the coordinate transformation code.
```

```
/pgfplots/date coordinates in=<coordinate>
```

Installs `x coord trafo` and `x coord inv trafo` (or the respective variant for `<coordinate>`) such that ISO dates of the form `<year>-<month>-<day>` are accepted. Here, `<coordinate>` is usually one of `x`, `y`, or `z`, but it can also contain stuff like `hist/data`.

After installing this style, input values like 2006-02-28 will be converted to an “appropriate” integer using the Julian calendar. Input coordinates may be of the form

`<year>-<month>-<day>`

or they may contain times as

`<year>-<month>-<day> <hour>:<minute>`.

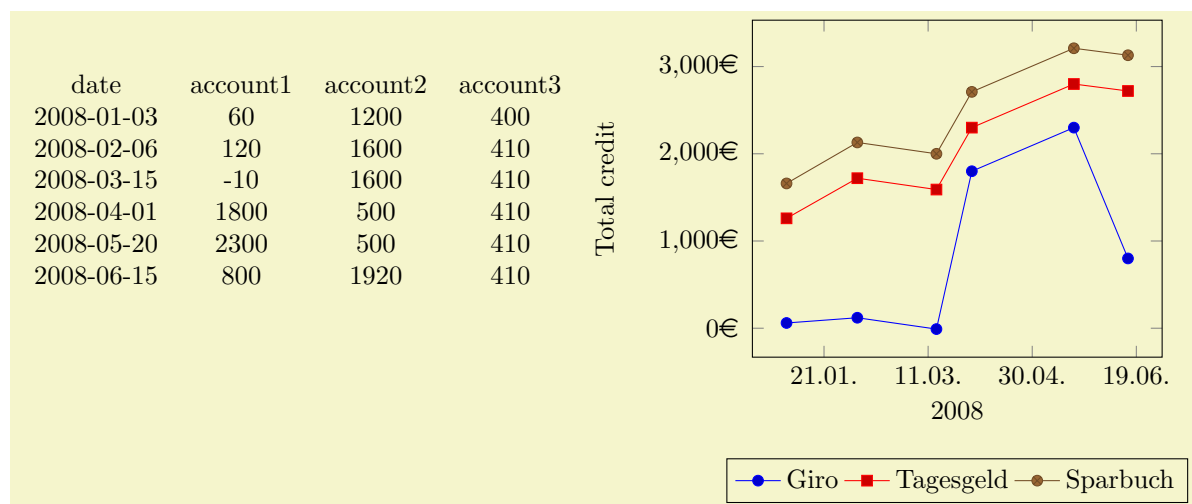
The result of the transformation are numbers where one unit is one day and times are fractional numbers.

The transformation is implemented using the PGF-calendar module, see [5, Calendar Library]. This reference also contains more information about extended syntax options for dates.

The inverse transformation provides the following macros which are available during tick label evaluation (i.e. when used inside of `xticklabel` or `yticklabel`):

- `\year` expands to the year component,
- `\month` expands to the month component,
- `\day` expands to the day component,
- `\hour` expands to the hour component (using two digits),
- `\Hour` expands to the hour component (but omits leading zeros),
- `\minute` expands to the minute component (two digits),
- `\Minute` expands to the minute component (omits leading zeros),
- `\lowlevel` expands to the low level number representing the tick,
- `\second` will always be 00.

This allows to use `\day.\month.\year` or `\day.\hour:\minute` inside of `xticklabel`, for example. A complete example (with fictional data) is shown below.

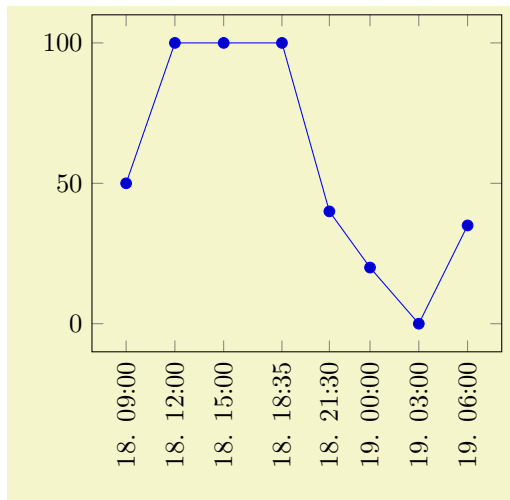


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepgfplotslibrary{dateplot} !

\pgfplotstabletypeset[string type]{plotdata/accounts.dat}

\begin{tikzpicture}
  \begin{axis}[
    date coordinates in=x,
    xticklabel={\day.\month.},
    xlabel={2008},
    stack plots=y,
    yticklabel={\pgfmathprintnumber{\tick}\EUR{}}, % <- requires \usepackage{eurosym}
    ylabel=Total credit,
    ylabel style={yshift=10pt},
    legend style={
      at={(0.5,-0.3)},anchor=north,legend columns=-1}}

  \addplot table[x=date,y=account1]{plotdata/accounts.dat};
  \addplot table[x=date,y=account2]{plotdata/accounts.dat};
  \addplot table[x=date,y=account3]{plotdata/accounts.dat};
  \legend{Giro,Tagesgeld,Sparbuch}
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepgfplotslibrary{dateplot} !
\begin{tikzpicture}
  \begin{axis}[
    date coordinates in=x,
    xtick=data,
    xticklabel style={
      {rotate=90,anchor=near xticklabel},
    },
    xticklabel={\day.\hour:\minute},
    date ZERO=2009-08-18,% <- improves precision!
  ]
  \addplot coordinates {
    (2009-08-18 09:00, 050)
    (2009-08-18 12:00, 100)
    (2009-08-18 15:00, 100)
    (2009-08-18 18:35, 100)
    (2009-08-18 21:30, 040)
    (2009-08-19, 020)
    (2009-08-19 3:00, 000)
    (2009-08-19 6:0, 035)
  };
  \end{axis}
\end{tikzpicture}
```

Attention: If you intend to use hours and minutes, you should *always* provide the `date ZERO` to maintain adequate precision!

`/pgfplots/date ZERO=<year>-<month>-<day>` (initially 2006-01-01)

A technical key which defines the 0 coordinate of `date coordinates in`. Users will never see the resulting numbers, so one probably never needs to change it. However, the resulting numbers may become very large and a mantisse of 6 significant digits may not be enough to get accurate results. In this case, `date ZERO` should be set to a number which falls into the input date range.

4.22 Skipping Or Changing Coordinates – Filters

```
/pgfplots/x filter/.code={\...}
/pgfplots/y filter/.code={\...}
/pgfplots/z filter/.code={\...}
/pgfplots/filter point/.code={\...}
```

The code keys `x filter` and `y filter` allow coordinate filtering which are based on a *single* coordinate. A coordinate filter gets an input coordinate as #1 (on input, the same value is stored in `\pgfmathresult`), applies some operation and writes the result into the macro `\pgfmathresult`. If `\pgfmathresult` is

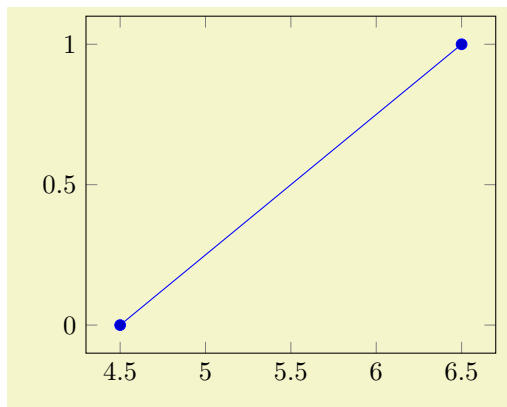
empty afterwards, the coordinate is discarded. You can also set `\pgfmathresult` to `nan` or `inf` in which case the coordinate can be either discarded (if `unbounded coords=discard` is set) or the plot can be interrupted (the case `unbounded coords=jump`).

The `filter point/.code` filter allows filtering depending on all components forming a complete point (x , y and z); it is described below.

It is allowed that filters do not change `\pgfmathresult`. In this case, the unfiltered coordinate will be used.

Coordinate filters are useful in automatic processing system, where PGFPLOTS is used to display automatically generated plots. You may not want to filter your coordinates by hand, so these options provide a tool to do this automatically.

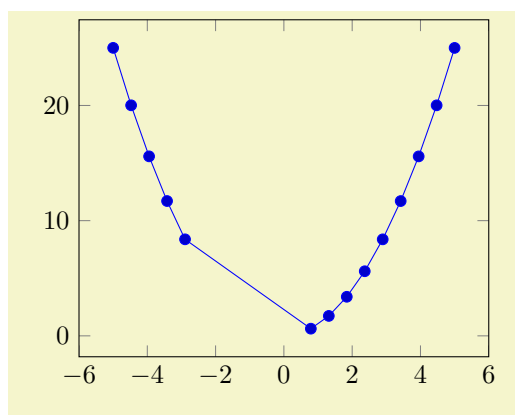
The following filter adds 0.5 to every x coordinate.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[x filter/.code=
  {\pgfmathadd{#1}{0.5}}]
\addplot coordinates {
  (4,0)
  (6,1)
};
\end{axis}
\end{tikzpicture}
```

Please refer to [5, pgfmath manual] for details about the math engine of PGF. Please keep in mind that the math engine works with limited T_EX precision.

During evaluation of the filter, the macro `\coordindex` contains the number of the current coordinate (starting with 0). Thus, the following filter discards all coordinates after the 5th and before the 10th.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  samples=20,
  x filter/.code={
    \ifnum\coordindex>4
      \ifnum\coordindex<11
        \def\pgfmathresult{}
      \fi
    \fi
  }
]
\addplot {x^2};
\end{axis}
\end{tikzpicture}
```

There is also a style key which simplifies selection by index, see below.

PGFPLOTS invokes the filter with argument `#1` set to the input coordinate. For x -filters, this is the x -coordinate as it is specified to `\addplot`, for y -filters it is the y -coordinate.

If the corresponding axis is logarithmic, `#1` is the *logarithm* (see `log basis x` and its variants) of the coordinate as a real number, for example `#1=4.2341`. In case the logarithm was undefined, the argument will be empty.

The arguments to coordinate filters are minimally preprocessed: first, for logarithmic axes, the *log* of the argument is supplied. Second, any high level coordinate maps like `x coord trafo` (which may be used to map dates to numbers or string to numbers or so) are applied. In consequence, the `#1` argument is supposed to be a number. No further transformation has been applied.

Occasionally, it might be handy to get the “raw”, completely unprocessed input coordinate as it has been reported by the coordinate input routine. This unprocessed data is available in the three math

parser constants `rawx`, `rawy` and `rawz` (use `\pgfmathrawx`, `\pgfmathrawy` and `\pgfmathrawz` as a way to assign the value of interest to `\pgfmathresult`). All these values are ready for use in filters (and some other methods influence plots as well).

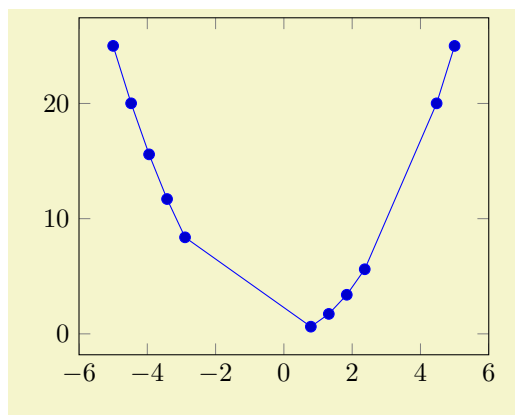
If key filters are invoked for `plot table`, access to the current row’s data can be achieved using `\thisrow{<column name>}` (and its variants). This includes all columns of the table.

The `filter point` key is more technical. It doesn’t take an argument: its arguments are given in terms of the `pgfkeys` variables `/data point x`, `/data point y` and `/data point z`. It may change its coordinates using `\pgfkeyssetvalue{/data point x}{<new value>}`; access to variables can be accessed with `\pgfkeysvalueof{/data point/x}` or, if the argument shall be written into a macro, with `\pgfkeysgetvalue`. This filter is evaluated after the other ones.

Note that you can provide different `x filter/y filter` arguments to each `\addplot` command. It seems there are only problems with the ‘#1’ argument, and I haven’t yet found out why. Please use `\pgfmathresult` in place of #1 if you provide `\addplot[x filter/.code={...}]`.

`/pgfplots/skip coords between index={<begin>}{<end>}`

A style which appends an `x filter` which discards selected coordinates. The selection is done by index where indexing starts with 0, see `\coordindex`. Every coordinate with index $\langle begin \rangle \leq i < \langle end \rangle$ will be skipped.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  samples=20,
  skip coords between index={5}{11},
  skip coords between index={15}{18}]

\addplot {x^2};
\end{axis}
\end{tikzpicture}
```

Technical note : this style usually applies to x coordinates (i.e. it counts x coordinates). In case you want to apply it to something like `hist/data` or `quiver/u`, you can

1. append an asterisk ‘*’ to the style’s name and
2. provide the target coordinate’s name as first argument.

For example, `skip coords between index*={hist/data}{2}` applies to `hist/data`.

`/pgfplots/each nth point={<integer>}`

A style which appends an `x filter` which discards all but each n th input coordinate.

This downsampling works fairly well. It can be used to reduce a huge amount of coordinates from an input file. In this case, you should also set `filter discard warning=false` to avoid repeated notifications about skipped coordinates and `unbounded coords=discard` such that PGFLOTS should silently forget any discarded points (rather than generated interrupted plots).

Note that there is also a `mark repeat` style which applies the same operation to plot marks only.

Technical note : this style usually applies to x coordinates (i.e. it counts x coordinates). In case you want to apply it to something like `hist/data` or `quiver/u`, you can

1. append an asterisk ‘*’ to the style’s name and
2. provide the target coordinate’s name as first argument.

For example, `each nth point*={hist/data}{2}` applies to `hist/data`.

`/pgfplots/restrict x to domain={<min>}:{<max>}`

```

/pgfplots/restrict y to domain=<min>:<max>
/pgfplots/restrict z to domain=<min>:<max>
/pgfplots/restrict x to domain*=<min>:<max>
/pgfplots/restrict y to domain*=<min>:<max>
/pgfplots/restrict z to domain*=<min>:<max>

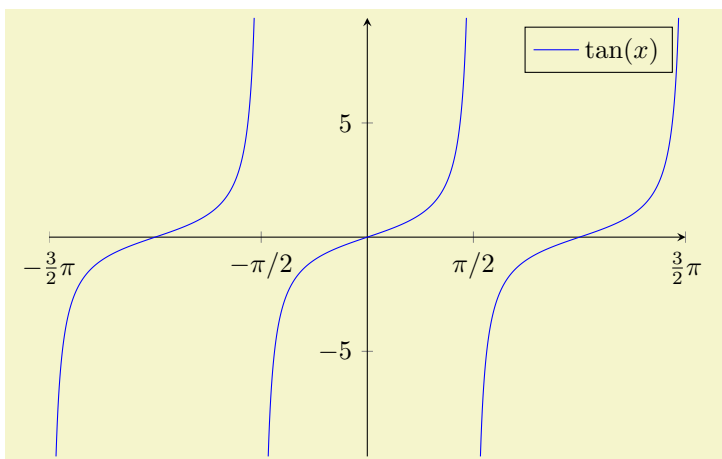
```

These keys append x (or y or z) coordinate filters to restrict the respective coordinate to a domain.

The versions without star (like `restrict x to domain`) will assign the value `-inf` if the coordinate is below $\langle min \rangle$ and `+inf` if the coordinate is above $\langle max \rangle$. The starred versions (like `restrict x to domain*`) will truncate coordinates to $[\langle min \rangle, \langle max \rangle]$, i.e. they assign the value $\langle min \rangle$ if the coordinate falls outside of the lower limit and $\langle max \rangle$ if the value falls outside of the upper limit.

For logarithmic axes, $\langle min \rangle$ and $\langle max \rangle$ are *logs* of the respective values. A variant which uses the non-logarithmic number might be to use `restrict expr to domain={\pgfmathrawx}{\langle min \rangle}{\langle max \rangle}`.

The non-starred versions also set `unbounded coords=jump` which leads to interrupted plots.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  restrict y to domain=-10:10,
  samples=1000,
  % some fine-tuning for the display:
  width=10cm, height=210pt,
  xmin=-4.7124, xmax=4.7124,
  xtick={-4.7124,-1.5708,...,10},
  xticklabels={\frac{32}{\pi},-\frac{\pi}{2},\frac{\pi}{2},\frac{32}{\pi}},
  axis x line=center,
  axis y line=center]

\addplot[blue] gnuplot[id=tangens,domain=-1.5*pi:1.5*pi] {tan(x)};
\legend{\tan(x)}
\end{axis}
\end{tikzpicture}

```

```

/pgfplots/restrict expr to domain={\langle expression \rangle}{\langle min \rangle}{\langle max \rangle}
/pgfplots/restrict expr to domain*={\langle expression \rangle}{\langle min \rangle}{\langle max \rangle}

```

Appends an x coordinate filter which sets the x coordinate to `-inf` if the $\langle expression \rangle$ evaluates to something less than $\langle min \rangle$ and to `inf` if $\langle expression \rangle$ evaluates to something larger than $\langle max \rangle$.

The starred variant, `restrict to domain*` assigns $\langle min \rangle$ if $\langle expression \rangle$ is less than the lower limit and $\langle max \rangle$ if it is larger than the upper limit.

The non-starred version also sets `unbounded coords=jump` which leads to interrupted plots.

In contrast to `restrict x to domain`, $\langle expression \rangle$ can depend on anything which is valid during `\addplot`, in particular `\coordindex` or table columns (`\thisrow{column name}`) and friends). The expression doesn't need to depend on x at all.

```

/pgfplots/@restrict to domain={\langle filter name \rangle}{\langle expression \rangle}{\langle min \rangle}{\langle max \rangle}|0|1

```

A low-level (technical) key which allows to apply the `restrict * to ...` features also to something like `hist/data`.

For example, `@restrict to domain={hist/data}{0:1}{0}` applies the domain-restriction to the histogram-input `hist/data`. The final ‘0’ means that it works in a similar way as the key `restrict x to domain=0:1`, i.e. it skips everything which is outside of $[0, 1]$. In a similar way, `@restrict to domain={hist/data}{0:1}{1}` applies the functionality of `restrict x to domain*=0:1` to `hist/data`: it truncates values outside of $[0, 1]$ to the domain’s end-points.

The `<filter name>` is expected to be a coordinate name like `x`, `y`, `z` (or `hist/data`).

The `<expression>` configures an expression which will be used rather than the value of `<filter name>`. It can be empty.

The `<min>:<max>` are as described above.

If the last argument is 1, any coordinate outside of the allowed domain will take the domain boundary as value. If it is 0, such a coordinate will get either `inf` or `-inf`.

`/pgfplots/filter discard warning=true|false` (initially true)

Issues a notification in your logfile whenever coordinate filters discard coordinates.

You can find somewhat more on coordinate filtering in Section 4.4.12: “Interrupted Plots”.

4.23 Transforming Coordinate Systems

Usually, PGFPLOTS works with cartesian coordinates. However, one may want to provide coordinates in a different coordinate system.

In this case, the `data cs` key can be used to identify the input coordinate system:

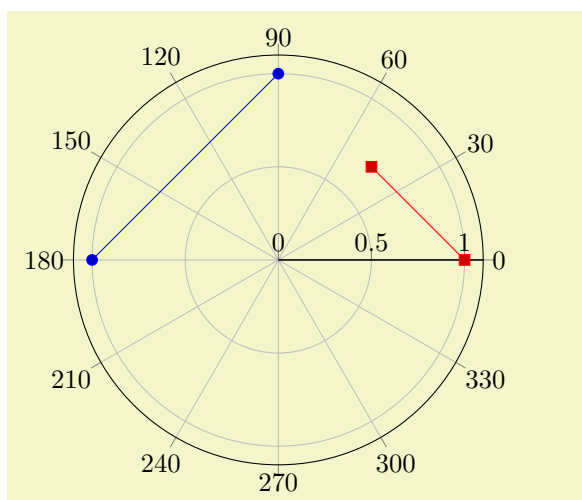
`/pgfplots/data cs=cart|polar|polarrad` (initially cart)

Defines the coordinate system (‘cs’) of the input coordinates. PGFPLOTS will apply transformations if the argument does not match the expected coordinate system.

Use `data cs` if your input has a different coordinate system than the axis. More precisely, every axis type has its own coordinate system. For example, a normal `axis` has the `cart` coordinate system, whereas a `polaraxis` has a `polar` coordinate system. The use of `data cs` with a different argument than the default of your axis instructs PGFPLOTS to apply transformations.

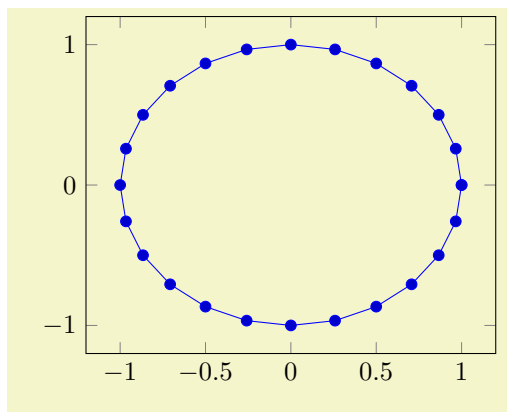
At the time of this writing, PGFPLOTS supports the following values for `data cs`:

The `data cs=cart` denotes the cartesian coordinate system. It is the coordinate system of the usual `axis` (or its logarithmic variants). It can have three components, x , y , and z . Specifying it is only necessary if you have a non-cartesian axis:



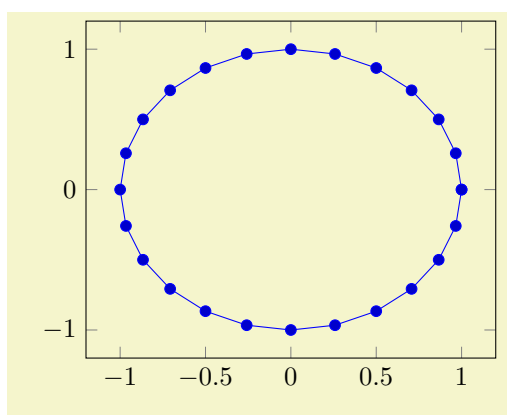
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepgfplotslibrary{polar}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot coordinates {(90,1) (180,1)};
    \addplot+[data cs=cart]
      coordinates {(1,0) (0.5,0.5)};
  \end{polaraxis}
\end{tikzpicture}
```

The `data cs=polar` is the (two-dimensional) coordinate system with (angle, radius), i.e. the first component “ x ” is the angle and the second component “ y ” is the radius. The angle is a number in the periodic range $[0, 360)$; the radius is any number. If a `polar` coordinate has a z component, it is taken as-is (the transformations ignore it).



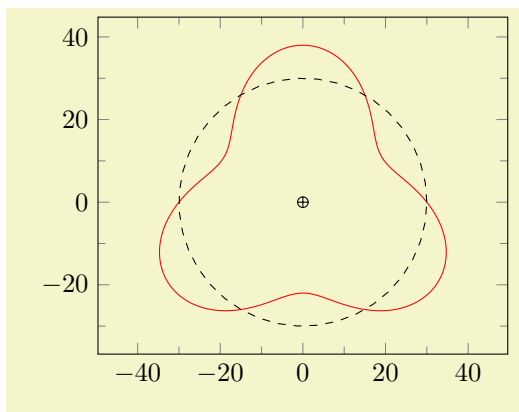
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[data cs=polar,domain=0:360] (\x,1);
\end{axis}
\end{tikzpicture}
```

The `data cs=polarrrad` is similar to `polar`, but it expects the angle in radians, i.e. in the periodic range $[0, 2\pi)$.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[data cs=polarrrad,domain=0:2*pi] (\x,1);
\end{axis}
\end{tikzpicture}
```

Note that the math function `deg(\langle rad \rangle)` transforms $\langle rad \rangle$ into degrees and `rad(\langle degree \rangle)` transforms $\langle degree \rangle$ into radians. Consequently, `polar` and `polarrrad` are more-or-less equivalent for plot expression.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
axis equal,
minor tick num=1,
]
\def\FREQUENCY{3}
\addplot[red,domain=0:360,samples=200,
smooth,data cs=polar]
(x,{30-8*sin(\FREQUENCY*x)});
\addplot[samples=40,domain=0:2*pi,dashed,
data cs=polar] (deg(x),30);
\addplot[mark=oplus,only marks] coordinates {(0,0)};
\end{axis}
\end{tikzpicture}
```

At the point of this writing, the `data cs` method will work for most plot handlers. But for complicated plot handlers, further logic may be needed which is not yet available (for example, the `quiver` plot handler might not be able to convert its direction vectors correctly)⁶⁰.

\pgfplotsaxistransformcs $\{\langle fromname \rangle\}\{\langle toname \rangle\}$

Expects the current point in a set of keys, provided in the coordinate system $\langle fromname \rangle$ and replaces them by the same coordinates represented in $\langle toname \rangle$.

On input, the coordinates are stored in `/data point/x`, `/data point/y`, and `/data point/z` (the latter may be empty). The macro will test if there is a declared coordinate transformation from $\langle fromname \rangle$

⁶⁰In case you run into problems, consider writing a bug report or ask others in T_EX online discussion forums.

to $\langle toname \rangle$ and invoke it. If there is none, it will attempt to convert to `cart` first and then from `cart` to $\langle toname \rangle$. If that does not exist either, the operation fails.

`\pgfplotsdefinecstransform{ $\langle fromname \rangle$ }{ $\langle toname \rangle$ }{ $\langle code \rangle$ }`

Defines a new coordinate system transformation. The $\langle code \rangle$ is expected to get input and write output as described for `\pgfplotsaxistransformcs`.

Implementing a new coordinate system immediately raises the question in which math mode the operations shall be applied. PGFPLOTS supports different so-called “coordinate math systems” for generic operations, and for each individual coordinate as well. These coordinate math systems can either use basic PGF math arithmetics, the `fpu`, or perhaps there will come a LuaTeX library.

The documentation of this system is beyond the scope of this manual⁶¹. Please consider reading the source-code comments and the source of existing transformations if you intend to write own transformations.

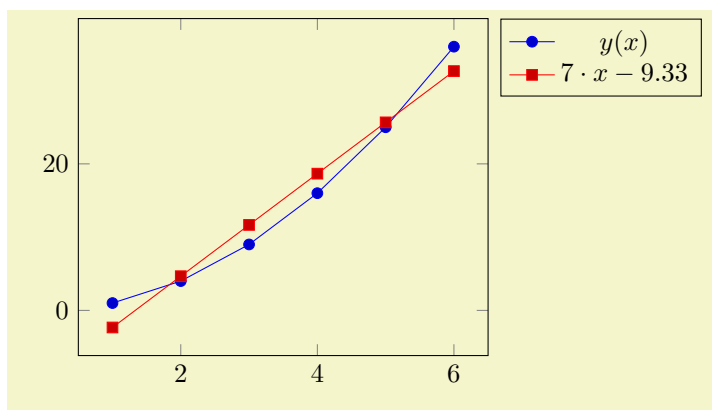
4.24 Fitting Lines – Regression

This section documents the attempts of PGFPLOTS to fit lines to input coordinates. PGFPLOTS currently supports `create col/linear regression` applied to columns of input tables. The feature relies on `PGFPLOTSTABLE`, it is actually implemented as a table postprocessing method.

`/pgfplots/table/create col/linear regression={ $\langle key-value-config \rangle$ }`

A style for use in `\addplot table` which computes a linear (least squares) regression $y(x) = a \cdot x + b$ using the sample data (x_i, y_i) which has to be specified inside of $\langle key-value-config \rangle$ (see below).

It creates a new column on-the-fly which contains the values $y(x_i) = a \cdot x_i + b$. The values a and b will be stored (globally) into `\pgfplotstableregressiona` and `\pgfplotstableregressionb`.



⁶¹Which is quite comprehensive even without API documentation, as you will certainly agree...

```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[legend pos=outer north east]
    \addplot table {% plot X versus Y. This is original data.
      X Y
      1 1
      2 4
      3 9
      4 16
      5 25
      6 36
    };
    \addplot table[
      y={create col/linear regression={y=Y}}] % compute a linear regression from the input table
    {
      X Y
      1 1
      2 4
      3 9
      4 16
      5 25
      6 36
    };
    % \xdef\slope{\pgfplotstableregressiona} %<-- might be handy occasionally
    \addlegendentry{$y(x)$}
    \addlegendentry{%
      $\pgfmathprintnumber{\pgfplotstableregressiona}$ \cdot x
      \pgfmathprintnumber[print sign]{\pgfplotstableregressionb}$}
  \end{axis}
\end{tikzpicture}

```

The example above has two plots: one showing the data and one containing the linear regression line. We use `y={create col/linear regression={}}` here, which means to create a new column⁶² containing the regression values automatically. As arguments, we need to provide the y column name explicitly⁶³. The x value is determined from context: `linear regression` is evaluated inside of `\addplot table`, so it uses the same x as `\addplot table` (i.e. if you write `\addplot table[x={⟨col name⟩}]`, the regression will also use `⟨col name⟩` as its x input). Furthermore, it shows the line parameters a and b in the legend.

Note that the uncommented line with `\xdef\slope{\pgfplotstableregressiona}` is useful if you have *more than one* regression line: it copies the value of `\pgfplotstableregressiona` (in this case) into a new global variable called ‘`\slope`’. This allows to use ‘`\slope`’ instead of `\pgfplotstableregressiona` – even after `\pgfplotstableregressiona` has been overwritten.

The following *⟨key-value-config⟩* keys are accepted as comma-separated list:

`/pgfplots/table/create col/linear regression/table={⟨\macro or file name⟩}` (initially empty)

Provides the table from where to load the x and y columns. It defaults to the currently processed one, i.e. to the value of `\pgfplotstablename`.

`/pgfplots/table/create col/linear regression/x={⟨column⟩}` (initially empty)

`/pgfplots/table/create col/linear regression/y={⟨column⟩}` (initially empty)

Provides the source of x_i and y_i data, respectively. The argument `⟨column⟩` is usually a column name of the input table, yet it can also contain `[index]⟨integer⟩` to designate column indices (starting with 0), `create on use` specifications or `aliases` (see the `PGFLOTS&TABLE` manual for details on `create on use` and `alias`).

The initial configuration (an empty value) checks the context where the linear regression is evaluated. If it is evaluated inside of `\pgfplotstabletypeset`, it uses the first and second table columns. If it is evaluated inside of `\addplot table`, it uses the same x input as the `\addplot table` statement. The `y` key needs to be provided explicitly (unless the table has only two columns).

`/pgfplots/table/create col/linear regression/xmode=auto|linear|log` (initially auto)

⁶²The `y={create col/}` feature is available for any other `PGFLOTS&TABLE` postprocessing style, see the `create on use` documentation in the `PGFLOTS&TABLE` manual.

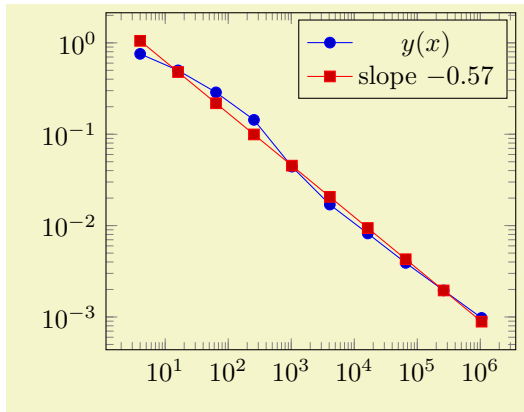
⁶³In fact, `PGFLOTS` sees that there are only two columns and uses the second by default. But you need to provide it if there are at least 3 columns.

`/pgfplots/table/create col/linear regression/ymode=auto|linear|log` (initially auto)

Enables or disables processing of logarithmic coordinates. Logarithmic processing means to apply $\ln$ before computing the regression line and $\exp$ afterwards.

The choice `auto` checks if the column is evaluated inside of a PGFLOTS axis. If so, it uses the axis scaling of the embedding axis. Otherwise, it uses `linear`.

In case of logarithmic coordinates, the `log basis x` and `log basis y` keys determine the basis.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}
\addplot table[x=dof,y=error2]
{pgfplotstable.example1.dat};
\addlegendentry{$y(x)$}

\addplot table[
x=dof,
y={create col/linear regression={y=error2}}]
{pgfplotstable.example1.dat};

% might be handy occasionally:
% \xdef\slope{\pgfplotstableregressiona}
\addlegendentry{slope
$\pgfmathprintnumber{\pgfplotstableregressiona}$}
\end{loglogaxis}
\end{tikzpicture}
```

The (commented) line containing `\slope` is explained above; it allows to remember different regression slopes in our example.

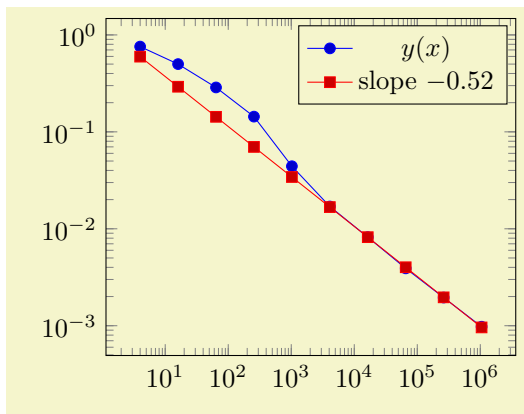
`/pgfplots/table/create col/linear regression/variance list={\langle list \rangle}` (initially empty)

`/pgfplots/table/create col/linear regression/variance={\langle column name \rangle}` (initially empty)

Both keys allow to provide uncertainties (variances) to single data points. A high (relative) variance indicates an unreliable data point, a value of 1 is standard.

The `variance list` key allows to provide variances directly as comma-separated list, for example `variance list={1000,1000,500,200,1,1}`.

The `variance` key allows to load values from a table `\langle column name \rangle`. Such a column name is (initially, see below) loaded from the same table where data points have been found. The `\langle column name \rangle` may also be a `create on use` name.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}
\addplot table[x=dof,y=error2]
{pgfplotstable.example1.dat};
\addlegendentry{$y(x)$}

\addplot table[
x=dof,
y={create col/linear regression={
y=error2,
variance list={1000,800,600,500,400}}
}
]
{pgfplotstable.example1.dat};

\addlegendentry{slope
$\pgfmathprintnumber{\pgfplotstableregressiona}$}
\end{loglogaxis}
\end{tikzpicture}
```

If both, `variance list` and `variance` are given, the first one will be preferred. Note that it is not necessary to provide variances for every data point.

`/pgfplots/table/create col/linear regression/variance src={\langle table or file name \rangle}` (initially empty)

Allows to load the `variance` from another table. The initial setting is empty. It is acceptable if the `variance` column in the external table has fewer entries than expected, in this case, only the first ones will be used.

Limitations: Currently, PGFPLOTS supports only linear regression, and it only supports regression together with `\addplot table`. Furthermore, long input tables might need quite some time.

4.25 Miscellaneous Options

`/pgfplots/disablelogfilter=true|false` (initially false, default true)

Disables numerical evaluation of $\log(x)$ in T_EX. If you specify this option, any plot coordinates and tick positions must be provided as $\log(x)$ instead of x . This may be faster and – possibly – more accurate than the numerical log. The current implementation of $\log(x)$ normalizes x to $m \cdot 10^e$ and computes

$$\log(x) = \log(m) + e \log(10)$$

where $y = \log(m)$ is computed with a Newton method applied to $\exp(y) - m$. The normalization involves string parsing without T_EX-registers. You can safely evaluate $\log(1 \cdot 10^{-7})$ although T_EX-registers would produce an underflow for such small numbers.

`/pgfplots/disabledatascaling=true|false` (initially false, default true)

Disables internal re-scaling of input data. Normally, every input data like plot coordinates, tick positions or whatever, are parsed without using T_EX’s limited number precision. Then, a transformation like

$$T(x) = 10^{q-m} \cdot x - a$$

is applied to every input coordinate/position where m is “the order of x ” base 10. Example: $x = 1234 = 1.234 \cdot 10^3$ has order $m = 4$ while $x = 0.001234 = 1.234 \cdot 10^{-3}$ has order $m = -2$. The parameter q is the order of the axis’ width/height.

The **effect** of the transformation is that your plot coordinates can be of *arbitrary magnitude* like 0.0000001 and 0.0000004. For these two coordinates, PGFPLOTS will use 100pt and 400pt internally. The transformation is quite fast since it relies only on period shifts. This scaling allows precision beyond T_EX’s capabilities.

The option “`disabledatascaling`” disables this data transformation. This has two consequences: first, coordinate expressions like $\langle \text{axis cs}:x,y \rangle$ have the same effect as $\langle x,y \rangle$, no re-scaling is applied. Second, coordinates are restricted to what T_EX can handle⁶⁴.

So far, the data scale transformation applies only to normal axes (logarithmic scales do not need it).

`/pgfplots/execute at begin plot={\langle commands \rangle}`

This axis option allows to invoke $\langle \text{commands} \rangle$ at the beginning of each `\addplot` command. The argument $\langle \text{commands} \rangle$ can be any T_EX content.

You may use this in conjunction with `x filter=...` to reset any counters or whatever. An example would be to change every 4th coordinate.

`/pgfplots/execute at end plot={\langle commands \rangle}`

This axis option allows to invoke $\langle \text{commands} \rangle$ after each `\addplot` command. The argument $\langle \text{commands} \rangle$ can be any T_EX content.

`/pgfplots/execute at begin axis={\langle commands \rangle}`

Allows to invoke $\langle \text{commands} \rangle$ at the end of `\begin{axis}` (or the other “begin axis” statements).

The statement is executed as (almost) last statement before the preparation has been completed.

`/pgfplots/execute at end axis={\langle commands \rangle}`

The counterpart for `execute at begin axis`. The hook is actually superfluous, it is executed immediately after `before end axis`. It is executed in the same T_EX group as `execute at begin axis`.

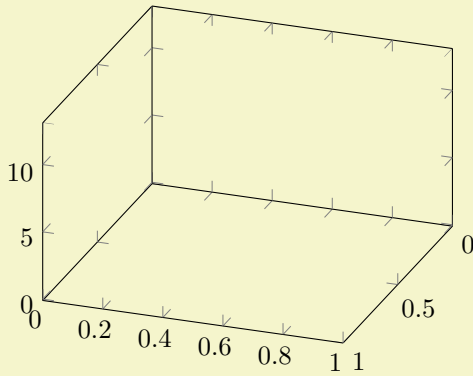
⁶⁴Please note that the axis’ scaling requires to compute $1/(x_{\max} - x_{\min})$. The option `disabledatascaling` may lead to overflow or underflow in this context, so use it with care! Normally, the data scale transformation avoids this problem.

`/pgfplots/execute at begin plot visualization={\<commands>}`

Allows to add customized code which is executed at the beginning of each plot visualization. In contrast to `execute at begin plot`, this happens not immediately during `\addplot`, but late during the postprocessing of `\end{axis}` when actual drawing commands are generated.

One possible application is shown below: suppose you want to use `\usepackage{ocg}` in order to switch layers dynamically, for example in a beamer package. This can be implemented as follows:

Dynamic PDF Layer Support (see Acrobat Layers)



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepackage[pdftex]{ocg}
\begin{tikzpicture}
\begin{axis}[
  title=Dynamic PDF Layer Support (see Acrobat Layers),
  view={110}{35}]
\addplot3+[
  execute at begin plot visualization=\begin{ocg}{First Layer}{FirstLayer}{0},
  execute at end plot visualization=\end{ocg},
]
  coordinates {(0,0,12) (0,1,2) (1,0,6) (0,0,12)};

\addplot3+[
  execute at begin plot visualization=\begin{ocg}{Second Layer}{SecondLayer}{0},
  execute at end plot visualization=\end{ocg},
]
  coordinates {(0,0,9) (0,1,8) (1,0,4) (0,0,9)};

\addplot3+[
  execute at begin plot visualization=\begin{ocg}{Third Layer}{ThirdLayer}{0},
  execute at end plot visualization=\end{ocg},
]
  coordinates {(0,0,1) (0,1,7) (1,0,3) (0,0,1)};
\end{axis}
\end{tikzpicture}
```

The `execute *` hooks insert the OCG-statements at the correct positions, and the single plot commands are added to different dynamic layers. Use the Acrobat Reader and its “Layers” Tab to switch each of them on or off. Note that it would not be enough to add the `\begin{ocg}...` statements right into the text since PGFPLOTS postpones drawing commands until `\end{axis}` (splitting of survey and visualization phase).

See <http://www.texample.net/weblog/2008/nov/02/creating-pdf-layers> for more details on OCG and how to obtain it.

Technical note: these hooks are also inserted for `\pgfplotsextra` commands.

`/pgfplots/execute at end plot visualization={\<commands>}`

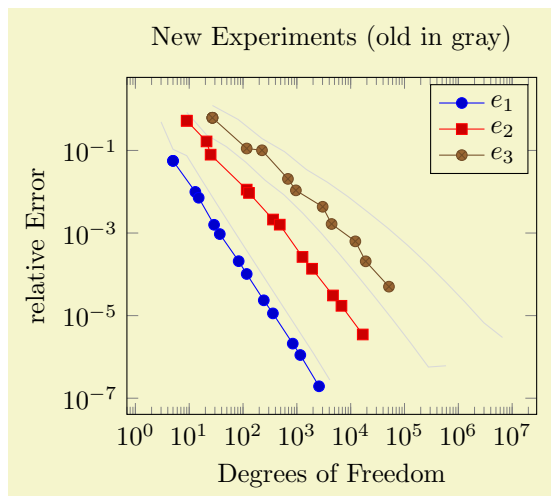
This is the counter-part of `execute at begin plot visualization`.

`/pgfplots/forget plot={\<true,false>}`

(initially **false**)

Allows to include plots which are not remembered for legend entries, which do not increase the number of plots and which are not considered for cycle lists.

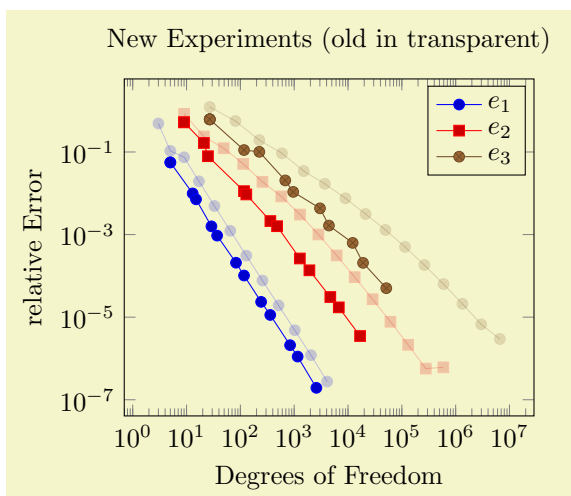
A forgotten plot can be some sort of decoration which has a separate style and does not influence the axis state, although it is processed as any other plot. Provide this option to `\addplot` as in the following example.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    % some descriptions:
    table/x=Basis,
    table/y={L2/r},
    xlabel=Degrees of Freedom,
    ylabel=relative Error,
    title=New Experiments (old in gray),
    legend entries={$e_1$, $e_2$, $e_3$}
  ]
    \addplot[black!15,forget plot]
      table {plotdata/oldexperiment1.dat};
    \addplot[black!15,forget plot]
      table {plotdata/oldexperiment2.dat};
    \addplot[black!15,forget plot]
      table {plotdata/oldexperiment3.dat};
    \addplot table {plotdata/newexperiment1.dat};
    \addplot table {plotdata/newexperiment2.dat};
    \addplot table {plotdata/newexperiment3.dat};
  \end{loglogaxis}
\end{tikzpicture}
```

Since forgotten plots won't increase the plot index, they will use the same `cycle list` entry as following plots.

The style `every forget plot` can be used to configure styles for each such plot:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[
    forget plot style={opacity=0.2},
    % same as above:
    table/x=Basis,
    table/y={L2/r},
    xlabel=Degrees of Freedom,
    ylabel=relative Error,
    title=New Experiments (old in transparent),
    legend entries={$e_1$, $e_2$, $e_3$},
  ]
    \foreach \exp in {1,2,3} {
      \addplot+[forget plot]
        table {plotdata/oldexperiment\exp.dat};
      \addplot table {plotdata/newexperiment\exp.dat};
    }
  \end{loglogaxis}
\end{tikzpicture}
```

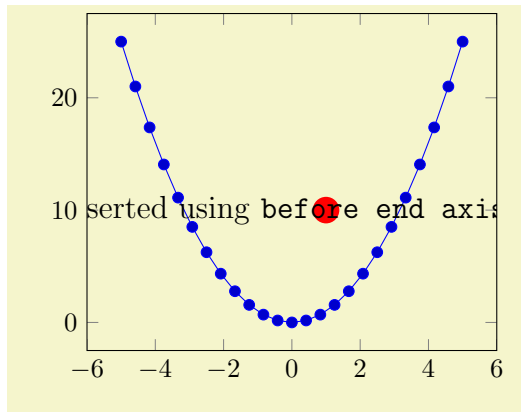
Here, the `\addplot+` command means we are using the same `cycle list` as the following plot and `forget plot style` modifies `every forget plot` style and yields transparency of the “old experiments”.

Please note that `every plot no <index>` styles are not applicable here.

A forgotten plot will be stacked normally if `stack plots` is enabled!

`/pgfplots/before end axis/.code={<...>}`

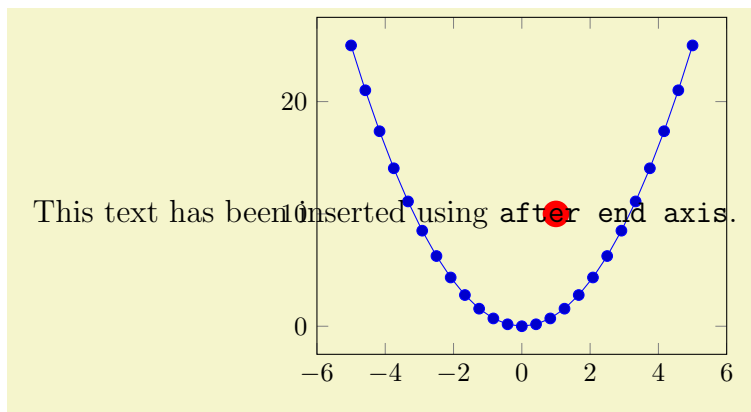
Allows to insert `<commands>` just before the axis is ended (see also `execute at end axis`). This option takes effect inside of the clipped area.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={
  before end axis/.code={
    \fill[red] (axis cs:1,10) circle(5pt);
    \node at (axis cs:-4,10)
      {\large This text has been inserted
        using \texttt{before end axis}.};
  }}}
\begin{tikzpicture}
  \begin{axis}
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

`/pgfplots/after end axis/.code={\langle...\rangle}`

Allows to insert $\langle commands \rangle$ right after the end of the clipped drawing commands. While `before end axis` has the same effect as if $\langle commands \rangle$ had been placed inside of your axis, `after end axis` allows to access axis coordinates without being clipped.

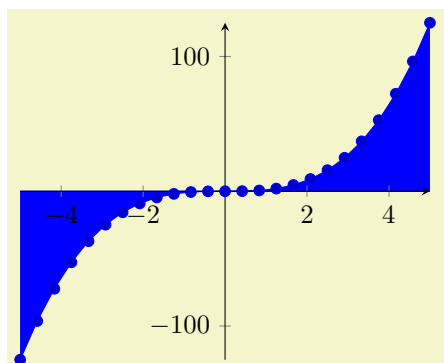


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\pgfplotsset{every axis/.append style={
  after end axis/.code={
    \fill[red] (axis cs:1,10) circle(5pt);
    \node at (axis cs:-4,10)
      {\large This text has been inserted using \texttt{after end axis}.};
  }}}
\begin{tikzpicture}
  \begin{axis}
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}
```

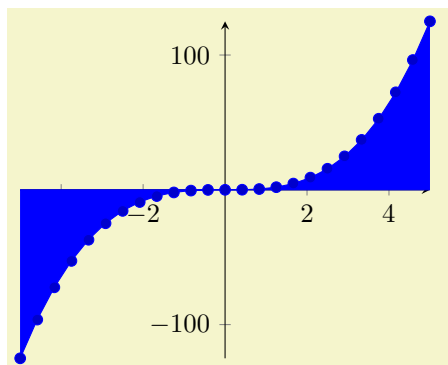
`/pgfplots/axis on top=true|false`

(initially false)

If set to `true`, axis lines, ticks, tick labels and grid lines will be drawn on top of plot graphics.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \begin{axis}
      axis on top=true,
      axis x line=middle,
      axis y line=middle
    \end{axis}
    \addplot+[fill] {x^3} \closedcycle;
  \end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[
    axis on top=false,
    axis x line=middle,
    axis y line=middle]
    \addplot+[fill] {x^3} \closedcycle;
  \end{axis}
\end{tikzpicture}
```

Please note that this feature does not affect plot marks. I think it looks unfamiliar if plot marks are crossed by axis descriptions.

```
/pgfplots/visualization depends on=<\macro> (initially empty)
/pgfplots/visualization depends on=<expression>\as<\macro> (initially empty)
/pgfplots/visualization depends on=value <content>\as<\macro> (initially empty)
```

Allows to communicate data to PGFPLOTS which is essential to perform the visualization although PGFPLOTS isn't aware of it.

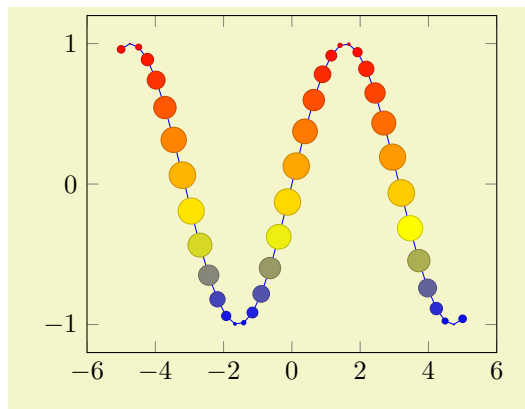
Suppose you want a scatter plot, which depends on the (x, y) coordinates, the `point meta` data to draw individual colors and furthermore data which influences the `mark size`. Thus, you need a total of 4 coordinates for every data point, although PGFPLOTS supports only 3 in its initial configuration.

Before we actually come to the main point of the problem, we'll talk about how to get a scatter plot which has individual colors *and* individual sizes. It is not sufficient to set `mark size` alone, since `mark size` is evaluated only once, before markers are processed (the same holds for `every mark`). Thus, we can use `scatter` combined with

`scatter/@pre marker code/.append style={/tikz/mark size=\perpointmarksize}`.

The `@pre marker code` is installed for every marker of a scatter plot individually. Now, we come to the problem as such: where can we get the value for `mark size`, in our case called `\perpointmarksize`?

A solution is `visualization depends on` (using the second input syntax at this point):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot+[
      scatter,
      scatter src=y,
      samples=40,
      visualization depends on=
        {5*cos(deg(x))} \as \perpointmarksize,
      scatter/@pre marker code/.append style=
        {/tikz/mark size=\perpointmarksize}
    ]
    {\sin(deg(x))};
  \end{axis}
\end{tikzpicture}
```

Here, we define `\perpointmarksize` as `5*cos(deg(x))`. The expression will be evaluated together with all other coordinates. Thus, everything which is available during the survey phase can be used here. This includes the final coordinates `x`, `y`, `z`; the constant `meta` expands to the current per point meta data. Furthermore, `\thisrow{<colname>}` expands to the value of a table column.

The command `visualization depends on` evaluates and remembers every value in internal data structures. The remembered value is then available as `<\macro>` during the visualization phase. In our example, the `@pre marker code` is evaluated during the visualization phase and applies `mark size=5*cos(deg(x))`.

The first syntax, `visualization depends on=<\macro>`, tells PGFPLOTS to use an already defined `<\macro>`. The second syntax with `<content>\as<\macro>` provides also the value.

There can be more than one `visualization depends on` phrase.

In case the stored value is not of numerical type⁶⁵, you can use the prefix ‘value’ before the argument, i.e.

```
visualization depends on=value <\macro> or
visualization depends on=value <content>\as <\macro>.
```

Such a value will be expanded and stored, but not parsed as number (at least not by PGFPLOTS).

`/pgf/fpu={true,false}` (initially true)

This key activates or deactivates the floating point unit. If it is disabled (`false`), the core PGF math engine written by Mark Wibrow and Till Tantau will be used for `plot expression`. However, this engine has been written to produce graphics and is not suitable for scientific computing. It is limited to fixed point numbers in the range ± 16384.00000 .

If the `fpu` is enabled (`true`, the initial configuration) the high-precision floating point library of PGF written by Christian Feuersänger will be used. It offers the full range of IEEE double precision computing in $\text{\TeX}$. This FPU is also part of `PGFPLOTS`TABLE, and it is activated by default for `create col/expr` and all other predefined mathematical methods.

Use

```
\pgfkeys{/pgf/fpu=false}
```

in order to de-activate the extended precision. If you prefer using the `fp` (fixed point) package, possibly combined with Mark Wibrows corresponding PGF library, the `fpu` will be deactivated automatically. Please note, however, that `fp` has a smaller data range (about $\pm 10^{17}$) and may be slower.

4.26 TikZ Interoperability

PGFPLOTS is built on top of TikZ/PGF, and it inherits most of power to visualize plots. However, their coordinate systems do not match up – for good reason: PGFPLOTS operates on logical (data) coordinates whereas TikZ operates on image coordinates.

Occasionally, one may want to synchronize both in order to generate a graphic – and the question arises how to match the coordinates from TikZ to PGFPLOTS and vice-versa. This section explains how to match coordinates and it discusses the necessary restrictions.

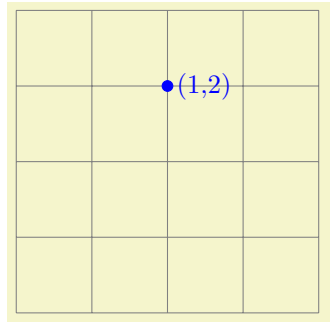
There are a couple of keys in PGFPLOTS which control the mapping of coordinates. The purpose of these keys is to implement visualization techniques, but they do things different than TikZ (and they should). To match coordinates with TikZ, one needs the following aspects:

1. Restrict your visualization type: a logarithmic axis simply may not fit into TikZ (to be more precise: it may fit, but a TikZ unit will correspond to a log-unit in PGFPLOTS).
2. Configure matching unit vectors by means of the `x` and `y` keys. The default configuration of TikZ is to use `x=1cm,y=1cm,z={(0,0)}`. Note that these settings are usually overridden by PGFPLOTS in order to respect `width` and `height` (and `view` for three-dimensional axes).
3. Disable the data scaling by means of `disabledatascaling`: PGFPLOTS will internally apply linear coordinate transformations in order to provide the data range required for floating point arithmetics (using approximately floating point precision). Disabling the data scaling means to restrict yourself to the (small) data range supported by TikZ—but that’s probably what you want in that case.
4. Define `anchor` and position of the `axis`, probably using `anchor=origin,at={(0,0)}`. The `at={(0,0)}` configures PGFPLOTS to place the axis at the TikZ position (0,0) whereas `anchor=origin` means that PGFPLOTS will place its data origin (0,0,0) at the place designated by `at` (see Section 4.18 for details).
5. Make sure that the PGFPLOTS axis contains the data origin (0,0,0) in the displayed data range (i.e. configure `xmin`, `xmax`, `ymin`, and `ymax` appropriately).

Without this, the `anchor=origin` key required in the previous item will be truncated to the next coordinate which is part of the displayed range.

⁶⁵Or if it is just a constant and you’d like to improve speed.

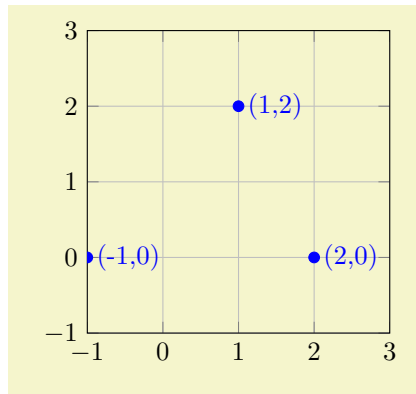
Here is a simple example, first with TikZ:



```
\begin{tikzpicture}
\coordinate (Point) at (1,2);

\draw [gray] (-1,-1) grid (3,3);
\draw [blue,fill] (Point) circle (2pt)
node [right] {(1,2)};
\end{tikzpicture}
```

it displays a grid with $x, y \in [-1, 3]$ and shows a node inside of it. Now, we apply the keys discussed above to match this setting in PGFPLOTS:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\coordinate (Point) at (1,2);

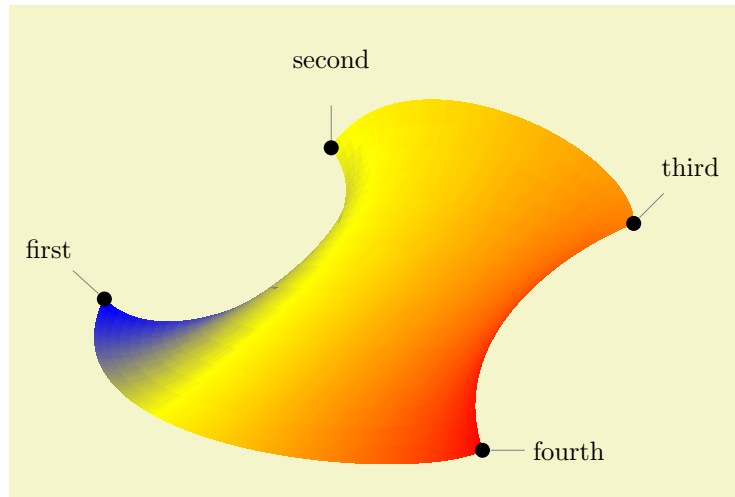
\begin{axis}[
% tell pgfplots to "grab" the axis at its
% internal (0,0) coord:
anchor=origin,
% tell pgfplots to place its anchor at (0,0):
% (This is actually the default and can
% be omitted)
at={{(0pt,0pt)}},
% tell pgfplots to use the "natural" dimensions:
disabledatascaling,
% tell pgfplots to use the same unit vectors
% as tikz:
x=1cm,y=1cm,
%
% this is just as usual in pgfplots. I guess
% it is only useful if (0,0) is part of the
% range... try it out.
xmin=-1,xmax=3, ymin=-1,ymax=3,grid=both]
% this uses the point defined OUTSIDE of the axis
\draw [blue,fill] (Point) circle (2pt)
node [right] {(1,2)};

% this uses a TIKZ coordinate (2,0) in the axis:
\draw [blue,fill] (2,0) circle (2pt)
node [right] {(2,0)};

% this here will always work inside of an axis:
\draw [blue,fill] (axis cs:-1,0) circle (2pt)
node [right] {(-1,0)};
\end{axis}
\end{tikzpicture}
```

The example demonstrates several things: first, it defines a coordinate in the enclosing `tikzpicture` and uses it inside of the `axis` (at the correct position). Second, it uses the standard TikZ coordinate (2,0) inside of the `axis`, and it is placed at the expected position. Third, it uses the approach provided by PGFPLOTS by using the `axis cs` to designate a coordinate (this last approach does also work without the coordinate matching).

Here is an example which inserts a PGFPLOTS graphics correctly into a `tikzpicture`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% requires \usepgfplotslibrary{patchplots}
\begin{tikzpicture}
\begin{axis}[
  % tell pgfplots to "grab" the axis at its internal (0,0) coord:
  anchor=origin,
  % tell pgfplots to place its anchor at (0,0):
  % (This is actually the default and can be omitted)
  at={(0pt,0pt)},
  % tell pgfplots to use the "natural" dimensions:
  disabledatascaling,
  % tell pgfplots to use the same unit vectors as tikz:
  x=1cm,y=1cm,
  %
  hide axis,
]
\addplot[patch,patch type=coons,
  shader=interp,point meta=explicit]
coordinates {
  (0,0) [0] % first corner
  (1,-1) [0] % bezier control point between (0) and (3)
  (4,0.7) [0] % bezier control point between (0) and (3)
  %
  (3,2) [1] % second corner
  (4,3.5) [1] % bezier control point between (3) and (6)
  (7,2) [1] % bezier control point between (3) and (6)
  %
  (7,1) [2] % third corner
  (6,0.6) [2] % bezier control point between (6) and (9)
  (4.5,-0.5) [2] % bezier control point between (6) and (9)
  %
  (5,-2) [3] % fourth corner
  (4,-2.5) [3] % bezier control point between (9) and (0)
  (-1,-2) [3] % bezier control point between (9) and (0)
};
\end{axis}

% this requires pgf 2.10
\begin{scope}[every node/.style={circle,inner sep=2pt,fill=black}]
\node[pin=140:first] at (0,0) {};
\node[pin=second] at (3,2) {};
\node[pin=45:third] at (7,1) {};
\node[pin=0:fourth] at (5,-2) {};
\end{scope}
\end{tikzpicture}
```

The example employs one of the `patch` plots of the `patchplots` library. Since these graphical elements typically require depth information (`z buffering`) and color data (`point meta`), they are only available inside of PGFPLOTS. However, the configuration above ensures that coordinates match one-to-one between PGFPLOTS and TikZ. The `hide axis` flag disables anything of PGFPLOTS, so only the visualized `patch` plot remains⁶⁶.

⁶⁶Note that the (0,0,0) coordinate of PGFPLOTS is part of the data range here.

4.27 Layers

It is important that several parts of an axis are drawn “on top” of others. Usually, PGFPLOTS ensures this by drawing them in a suitable sequence (usually background followed by grid lines, followed by tick lines and tick labels, followed by plots and finally axis descriptions). While this works reasonable in most cases, there are cases where more control is desired. One common use-case is if multiple axes shall be drawn into the same picture: here, the sequence from above should be applied to all involved axes simultaneously.

4.27.1 Summary

This section is the technical reference for using and customizing layered graphics in PGFPLOTS. As such, it is hard reading.

For most purposes, the following is *completely* sufficient for you: If you want to enable layered graphics, put the following statement into the `tikzpicture` which is supposed to have layered graphics:

```
\begin{tikzpicture}
  \pgfplotsset{set layers}
  \begin{axis}
    ...
  \end{axis}

  % perhaps a second axis which should use the same layers?
  \begin{axis}
    ...
  \end{axis}
\end{tikzpicture}
```

This enables layered graphics for that specific `tikzpicture`.

You may want layered graphics if you have multiple axes in the same picture, or if you have specific needs for your plot.

Consider reading [on layer](#) if you want to move particular elements of your axis to a different layer.

4.27.2 Using Predefined Layers

The main key to control layered graphics with PGFPLOTS is `set layers`:

`/pgfplots/set layers=none` | *<layer configuration name>* (initially `none`)

This key enables layered graphics for either the current axis or for all following axes.

Enabling layered graphics has the effect that the order in which graphical elements are given is unrelated to the ordering in which they will be drawn. The main benefit is if you have multiple axes in the same figure: the axes can share the same layers.

The invocation `set layers=none` disables layered graphics.

The invocation `set layers` (without equal sign and without arguments) is the same as if you would write `set layers=default`.

In all other cases, `set layers` expects a *<layer configuration name>*. There are two predefined configurations available (the prefix `/pgfplots/layers/` is optional):

`/pgfplots/layers/standard` (no value)

A layer configuration which defines the layers `axis background`, `axis grid`, `axis ticks`, `axis lines`, `axis tick labels`, `main`, `axis descriptions`, `axis foreground`. They are drawn in the order of appearance.

`/pgfplots/layers/axis on top` (no value)

A layer configuration which uses the same layer names as `layers/standard`, but with a different sequence: `axis background`, `main`, `axis grid`, `axis ticks`, `axis lines`, `axis tick labels`, `axis descriptions`, `axis foreground`.

This layer is automatically used if the key `axis on top` is used together with `set layers=<any layer configuration name>`.

As soon as the key `set layers=<layer configuration name>` is encountered, PGFPLOTS starts the PGF command `\pgfsetlayers{<layer names>}` with the layer names of the respective configuration. Usually,

this *replaces* the current layer configuration of the embedding `tikzpicture`. Furthermore, `set layers` stores the name of $\langle layer\ configuration\ name \rangle$ such that every following `axis` knows how to map graphical elements to layer names.

There is one huge difference to any other key which tunes PGFPLOTS: layer configurations are properties of a complete `tikzpicture` whereas any other option affects only axis objects and their contents. Layers, however, affect every graphical element of the embedding picture. Due to this property, layer configurations need to be given at one of several supported positions:

1. Directly within the picture:

```
\begin{tikzpicture}
  \pgfplotsset{set layers=default}
  \begin{axis}
    ...
  \end{axis}
\end{tikzpicture}
```

This option explicitly tells the reader of your source code that a significant portion of your picture has been changed: the complete picture has and uses a $\langle layer\ configuration\ name \rangle$ (in this case `default`).

2. As option for one or more axes which is/are directly within the picture:

```
\begin{tikzpicture}
  \begin{axis}[set layers]
    ...
  \end{axis}
\end{tikzpicture}
```

Here, PGFPLOTS implicitly communicates its layer configuration to the enclosing `tikzpicture`. Thus, the effect of `set layers` is *not local to an axis*; it survives until `\end{tikzpicture}`. Any other option only survives until `\end{axis}`.

In this case, only the *last* activated layer configuration will apply to the picture.

Limitation: no environments or local $\text{\TeX}$ groups allowed. Standard usages as within the examples of this manual will always work. But since the layer name configuration is essentially part of a PGF picture (at a low level), one cannot arbitrarily set them; PGF will complain if they are changed within some nested $\text{\TeX}$ groups or $\text{\LaTeX}$ environments. Typically, you will never need to worry about this.

In short, the following examples are *forbidden* because the axis is within locally nested groups.

```
\begin{tikzpicture}
  {% FORBIDDEN! Consider using case (1) above!
    \begin{axis}[set layers]
      ...
    \end{axis}
  }
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \begin{scope} % FORBIDDEN! Consider using case (1) above!
    \begin{axis}[set layers]
      ...
    \end{axis}
  \end{scope}
\end{tikzpicture}
```

These examples are forbidden because the layer configuration will be cleared by the ‘}’ of the first forbidden example and by the ‘`\end{scope}`’ of the second example. A solution would be one of the different placement options (i.e. choice (1.) or (3.)).

3. outside of any picture:

```
\pgfplotsset{set layers=default}
\begin{tikzpicture}
  \begin{axis}
    ...
  \end{axis}
\end{tikzpicture}
```

This choice configures the layer configuration for *every* following `tikzpicture`.

Limitation: axis alignment restricted to inner anchors. This applies only if you changed the default value of `anchor` (which is `anchor=south west`). Any axis which uses layered graphics should use one of the following values of `anchor`: `north`, `north west`, `west`, `south west`, `south`, `south east`, `east`, `north east`, `north`, `center`, `origin`, `above origin`, `left of origin`, `right of origin`, `below origin`. In case you really need another anchor, PGFLOTS requires the use of `cell picture=true`, causing the layers to be local for that specific axis.

The technical background for this limitation is a hen-and-egg problem: outer anchors (like `outer south west`) are only available *after* the complete axis has been generated – and layers can only be drawn after each drawing instruction has been issued. The technical keys for further reading are `cell picture=false` or `cell picture=if necessary` (one of them is active for layered graphics).

`\pgfplotssetlayers`

An alias for `\pgfplotsset{set layers}`. It activates the `layers/default` layer configuration.

`\pgfplotssetlayers{<layer configuration name>}`

An alias for `\pgfplotsset{set layers={<layer configuration name>}}`.

Key handler `<key>/\define layer set={<ordered layer names>}{<style definitions>}`

Allows to define a new layer set configuration named `<key>`. Afterwards, `<key>` is a valid argument for `set layers=<key>`.

The first argument `<ordered layer names>` is a comma-separated list of layer names. The names are arbitrary, and `\pgfdeclarelayer` will be called for every encountered argument⁶⁷. There is just one “magic” name: the layer `main` should be part of every `<ordered layer names>` as it will contain every graphical element which is not associated with a specific layer.

The second argument `<style definitions>` contains options – just as if you would have written `<key>/\style={<style definitions>}`. The `<style definitions>` are supposed to contain PGFLOTS style redefinitions which make use of each encountered element of `<ordered layer names>`. This is probably best explained by an example: the `layers/standard` layer configuration is defined by

```
\pgfplotsset{
  layers/standard/.define layer set=
    {axis background,axis grid,axis ticks,axis lines,axis tick labels,main,%
      axis descriptions,axis foreground}
  {
    grid style=      {/pgfplots/on layer=axis grid},
    tick style=      {/pgfplots/on layer=axis ticks},
    axis line style= {/pgfplots/on layer=axis lines},
    label style=     {/pgfplots/on layer=axis descriptions},
    legend style=    {/pgfplots/on layer=axis descriptions},
    title style=     {/pgfplots/on layer=axis descriptions},
    colorbar style=  {/pgfplots/on layer=axis descriptions},
    ticklabel style= {/pgfplots/on layer=axis tick labels},
    axis background@ style={/pgfplots/on layer=axis background},
    3d box foreground style={/pgfplots/on layer=axis foreground},
  },
}
```

This definition declares a couple of layers, and it adjusts PGFLOTS styles by adding `on layer` commands. The arguments for `on layer` are the elements of `<ordered layer names>`.

Note that if you have an element in `<ordered layer names>` which is never referenced inside of `<style definitions>`, this layer will always be empty. In other words: the *only* reference to the names in `<ordered layer names>` is `<style definitions>`, PGFLOTS has no hard-coded magic layer names (except for `main` as explained above).

Since the second argument `<style definitions>` defines `<key>` to be a normal style key, one can simply use `<key>` in order to set `<style definitions>`. This allows to inherit them. For example, the `layers/axis on top` layer configuration is defined by means of

⁶⁷To be more precise: `set layers` calls `\pgfdeclarelayer` when it uses `<ordered layer names>`.

```
\pgfplotsset{
  /pgfplots/layers/axis on top/.define layer set=
    {axis background,main,axis grid,axis ticks,axis lines,axis tick labels,%
     axis descriptions,axis foreground}
  {\pgfplots/layers/standard}
}
```

i.e. it only redefines the *sequence* of the layers and re-uses the style definitions of `layers/standard`. Any number of layer configurations can be defined.

4.27.3 Changing the Layer of Graphical Elements

There are a couple of keys which change the layer of a graphical element.

```
/pgfplots/on layer={⟨layer name⟩}
```

Providing this key somewhere in a PGFPLOTS style or inside of a PGFPLOTS axis will change the layer for all graphical elements for which the style applies.

For example,

```
...
\begin{axis}[set layers,grid style={/pgfplots/on layer=axis foreground}]
...
```

will change the layer for any grid lines to `axis foreground`.

The argument `⟨layer name⟩` is expected to be part of the current layer configuration, i.e. the argument of `set layers` should contain it.

Note that if you have two *plots* with different values of `on layer`, you may also want to enable `clip mode=clip individual` or to deactivate clipping altogether using `clip=false`. Clipping options need to be provided as option to the axis, not to the plot. The technical background is that clip paths needs to be replicated for the layer on which the drawing is supposed to happen – otherwise they will be applied to the wrong layer.

```
/pgfplots/mark layer=auto|like plot|⟨layer name⟩ (initially auto)
```

An advanced key which defines the layer for plot `marks`. It is typically the best choice to leave it at `auto`.

If you write `\addplot[on layer=⟨layer name⟩]`, the layer will be used for the complete plot. Plot marks are treated with special care, so you can define an own layer for plot marks.

The initial choice `auto` will automatically define a “suitable” choice, leaving the responsibility with PGFPLOTS. Here, “suitable” means to respect `clip mode` and `clip marker paths` in a way such that plot marks will not be clipped even though the default layer for your plot will be clipped.

The choice `like plot` will pack the marks onto the same layer as the plot they belong to. This might cause clipped markers, i.e. markers which are only displayed partially if they are close to the boundary of the axis.

Finally, one can provide any `⟨layer name⟩`, just as for `on layer` – but the layer can be different from the layer used for the plot.

4.28 Technical Internals

This section describes keys which are usually set by internal routines – it is typically unnecessary to use them. However, they may impose limitations or influence performance. Such cases are documented clearly in other sections of this manual. This here is the reference on the involved internals.

```
/pgfplots/cell picture=true|false|if necessary (initially true)
```

This key is set automatically by PGFPLOTS if necessary (for example by `set layers`).

Typically, PGFPLOTS creates a so-called “cell picture”. A cell picture is a separate picture which is typeset into a node. Finally, the node is shifted to fulfill special `anchor` requirements. The necessity for a cell picture is given if the `anchor` of an axis is only known after the complete axis has been drawn.

The initial choice `true` means that PGFPLOTS will create a cell picture for every axis. This allows all `anchors`, but it is unsuited if multiple graphics layers are desired or if one wants SVG export. In order

to create a cell picture, PGFPLOTS interrupts the embedding `tikzpicture`, draws a new `tikzpicture`, and finally typesets the result into a node.

The choice `false` tells PGFPLOTS to draw its paths directly into the embedding `tikzpicture`. Such an approach is necessary if the axis shall use layers of the embedding `tikzpicture`. This is possible if and only if the `anchor` can be determined without actually drawing the complete axis. If so, PGFPLOTS will modify the transformation matrix in advance. Note that axes with `cell picture=false` will *contain* all the usual anchors – the only difference is that the axis itself can only use one of the following anchors for its alignment: `north`, `north west`, `west`, `south west`, `south`, `south east`, `east`, `north east`, `north`, `center`, `origin`, `above origin`, `left of origin`, `right of origin`, `below origin`.

The choice `if necessary` will check if the chosen anchor is one of the list above. If so, it will use `cell picture=false`. Otherwise, it will use `cell picture=true`.

`/pgfplots/compat/show suggested version=true|false` (initially `true`)

If enabled, PGFPLOTS will show you which value for `compat=<version>` results in the largest active feature set and highest quality.

This key will generate a warning if the current version is so old that the quality degrades seriously.

The notification will be printed to your `.log` file (during `\end{document}`).

Chapter 5

Related Libraries

This section describes some libraries which come with PGFPLOTS, but they are more or less special and need to be activated separately.

5.1 Clickable Plots

```
\usepgfplotslibrary{clickable} %  $\LaTeX$  and plain  $\TeX$ 
\usepgfplotslibrary[clickable] % Con $\TeX$ t
\usetikzlibrary{pgfplots.clickable} %  $\LaTeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.clickable] % Con $\TeX$ t
```

A library which generates small popups whenever one clicks into a plot. The popup displays the coordinate under the mouse pointer, supporting the optional snap-to-nearest `clickable coords` feature with customizable displayed information. Furthermore, the library allows to display slopes if one holds the mouse pressed and drags it to another point in the plot.

The library has two purposes: to compute slopes in a simple way¹ and to provide related, optional information to single data points which are not important enough to be listed in the main text (like prototype parameters or other technical things).

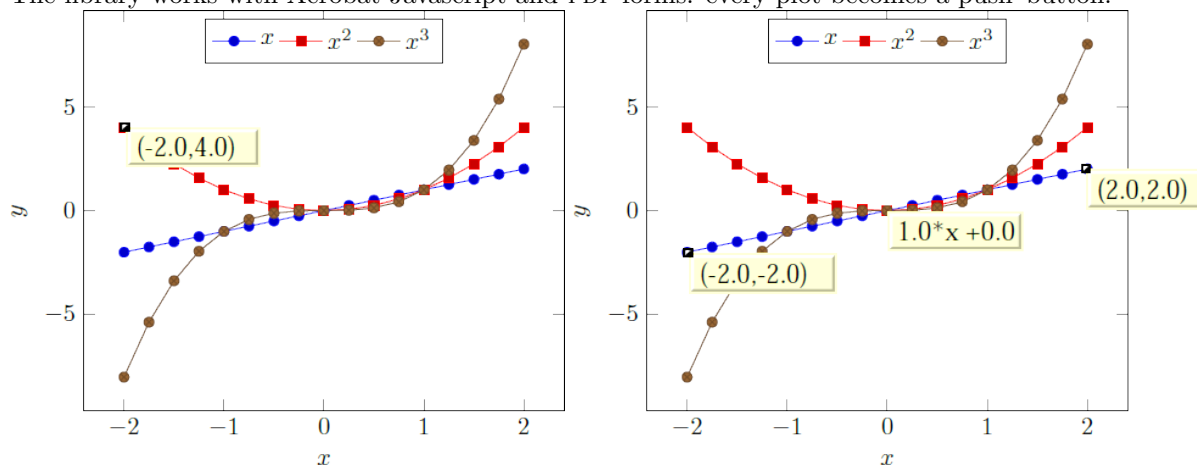
5.1.1 Overview

It is completely sufficient to write

```
\usepgfplotslibrary{clickable}
```

in the document preamble. This will automatically prepare every plot.

The library works with Acrobat Javascript and PDF forms: every plot becomes a push-button.



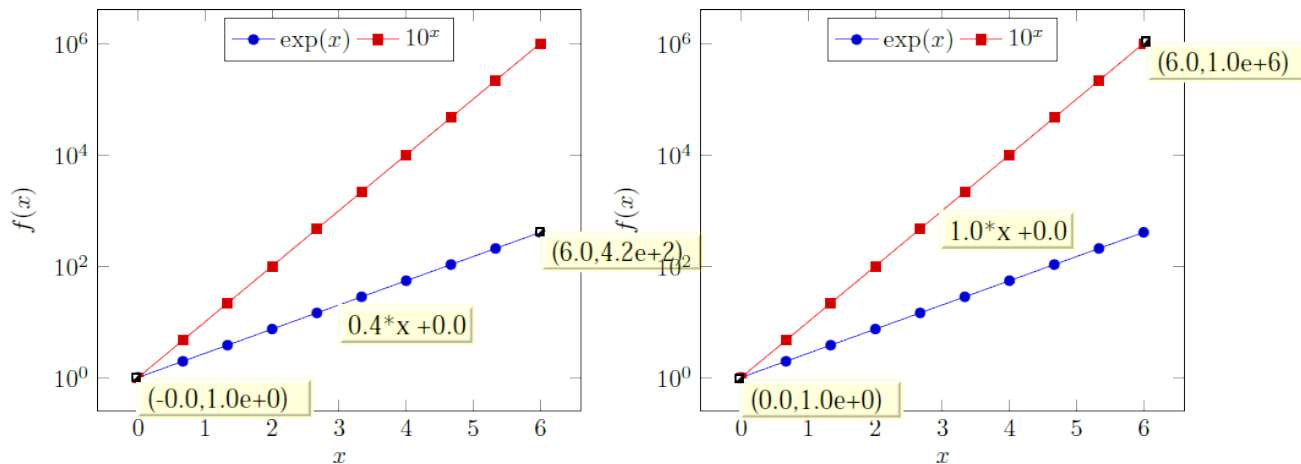
These screenshots show the result of clicking into the axis range (left column) and of dragging from one point to another (right column). The second case shows the result of Drag-and-Drop: it displays start- and

¹The author is applied mathematician...

end points and the equation for the line segment between the first point of the drag- and drop and the second point where the mouse has been released. The line segment is

$$l(x; x_0, y_0, x_1, y_1) = m \cdot x + n$$

where $m = (y_1 - y_0)/(x_1 - x_0)$ is the slope and n the offset chosen such that $l(x_0; \dots) = y_0$. For logarithmic plots, logarithms will be applied before computing slopes.



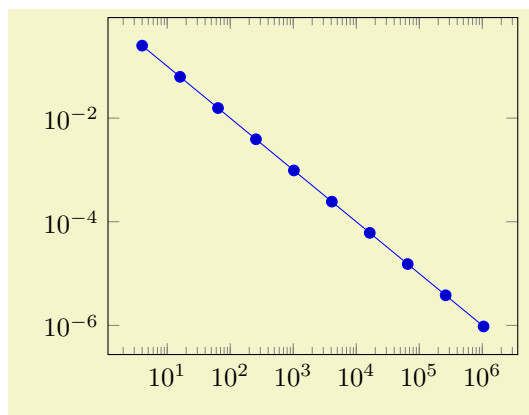
These screen shots show the result of drag- and drop for *logarithmic* axes: the end points show, again, the coordinates (without logs) and the form field in the middle shows the slope and offset of the linear equation in log coordinates.

The log basis for any logarithmic axes is usually 10, but it respects the current setting of `log basis x` and `log basis y`. The applied log will always use the same logarithm which is also used for the axis descriptions (this is not necessarily the same as used by `PGFPLOTS`TABLE!).

This document has been produced with the `clickable` library, so it is possible to load it into Acrobat Reader and simply click into a plot.

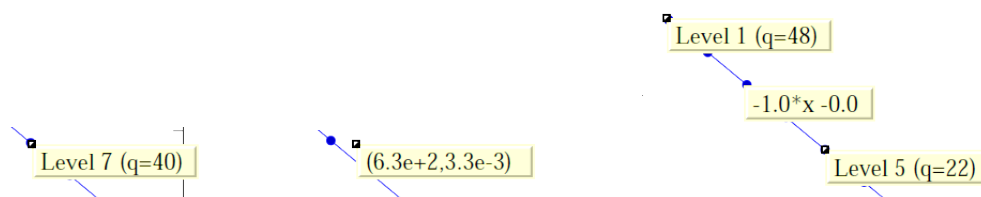
`/pgfplots/clickable coords={⟨displayed text⟩}`

Activates a snap-to-nearest feature when clicking onto plot coordinates. The `⟨displayed text⟩` is the coordinate's x and y value by default (i.e. you write just `clickable coords` without an equal sign).



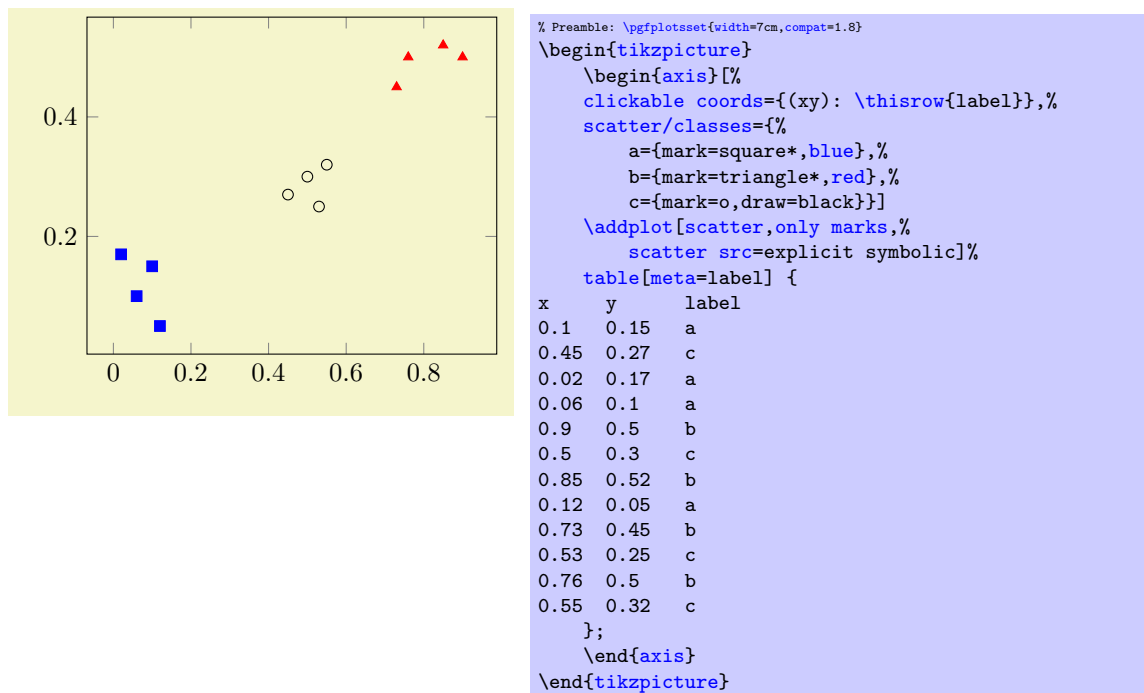
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{loglogaxis}[clickable coords=
    {Level \thisrow{level} (q=\thisrow{q})}]
    \addplot table[x=dof,y=error] {
      level  dof    error      q
      1      4      2.50000000e-01  48
      2     16      6.25000000e-02  25
      3     64      1.56250000e-02  41
      4    256      3.90625000e-03   8
      5   1024      9.76562500e-04  22
      6   4096      2.44140625e-04  46
      7  16384      6.10351562e-05  40
      8  65536      1.52587891e-05   3
      9 262144      3.81469727e-06   1
     10 1048576      9.53674316e-07   9
    };
  \end{loglogaxis}
\end{tikzpicture}
```

Now, clicking onto a data point yields 'Level 7 (q=40)' whereas clicking besides a data point results in the click coordinates as before,

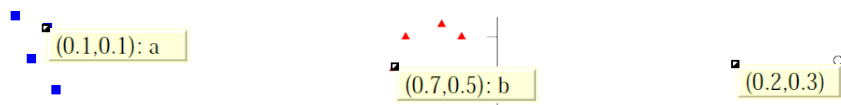


Note that logarithmic slopes work as before.

If you want the (x, y) values to be displayed, use the special placeholder string ‘ (xy) ’ inside of $\langle displayed text \rangle$. As an example, we consider again the `scatter/classes` example of page 81:



Here, we find popups like



The $\langle displayed text \rangle$ is a richtext string displayed with *Javascript*. For most purposes, it is used like an unformatted C string: it contains characters, perhaps line breaks with ‘`\n`’ or tabulators with ‘`\t`’, but it should not contain $\text{\TeX}$ formatting instructions, especially no math mode (the ‘ (xy) ’ replacement text is formatted with `sprintf`, see below). Consider `clickable coords` code in case you’d like to preprocess data before displaying it. If you experience problems with special characters, try prepending a backslash to them. If that doesn’t work either, try to prefix the word with ‘`\\`’ and/or with ‘`\string`’. Consider using `clickable coords size` if you intend to work with multiline fields and the size allocation needs improvements.

In fact, $\langle displayed text \rangle$ can even contain richtext (=XHTML) formatting instructions like ‘`<br/>`’ (note the final slash) or ‘`<span style="color:#7E0000;">text</span>`’ (note the backslash before ‘`#`’) which changes the color for text. The `<span style="">` arguments are CSS fields, consider an HTML reference for a list of CSS attributes.

It is possible to use `clickable coords` together with three dimensional axes. Note that dynamic (clickable) features of a three dimensional axis without `clickable coords` will be disabled (they appear to be useless). Furthermore, three dimensional axes do not support slope calculations; only the snap-to-nearest feature is available.

Consider using `annot/snap dist=6` to increase the snap-to-nearest distance.

The `clickable coords` can be specified for all plots in an axis (as in the examples above), but also once for every single `\addplot` commands for which the snap-to-nearest feature is desired (with different $\langle displayed text \rangle$).

If multiple `clickable coords` are on the same position, each click chooses the next one (in the order of appearance).

`/pgfplots/clickable coords code={ $\langle \text{\TeX code which defines } \backslash\text{pgfplotsretval} \rangle$ }`

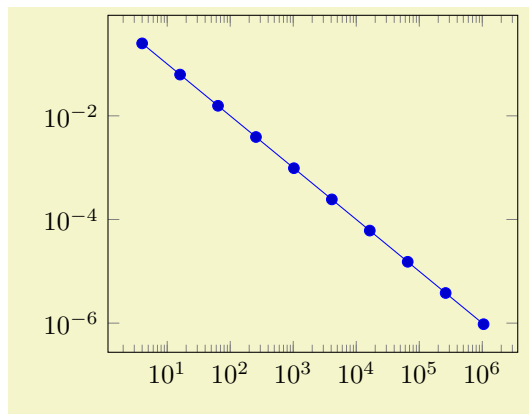
A variant of `clickable coords` which allows to prepare the displayed information before it is handed over to Javascript.

The value should be $\text{\TeX}$ code which defines `\pgfplotsretval` somehow. The result is used as simple, unformatted string which is associated to coordinates.

Consider using

```
\pgfmathprintnumber to[verbatim]{\langle number \rangle}\macroname
\edef\pgfplotsretval{Number=\macroname}
```

to provide number printing. The `\pgfmathprintnumber to[verbatim]` doesn't use math mode to format a number², and it writes its result into `\macroname`. The name '`\macroname`' is arbitrary, use anything like '`\eps`' or '`\info`'. The `\edef` means "expanded definition" and has the effect of expanding all macros to determine the value, in our case "`Number= \langle the value \rangle`". The following example uses it twice to pretty-print the data:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{loglogaxis}[clickable coords code={%
  \pgfmathprintnumber to[verbatim,precision=1]{%
    {\thisrow{error}}}%
  \error%
  \pgfmathprintnumber to[verbatim,frac]{%
    {\thisrow{frac}}}%
  \fraccomp%
  \edef\pgfplotsretval{error \error, R=\fraccomp}%
}]%
\addplot table[x=dof,y=error] {
level  dof    error          frac
1      4      2.50000000e-01  0.5
2     16      6.25000000e-02  0.75
3     64      1.56250000e-02  0.1
4    256      3.90625000e-03  0.2
5   1024      9.76562500e-04  0.5
6   4096      2.44140625e-04  0.8
7  16384      6.10351562e-05  0.125
8  65536      1.52587891e-05  0.725
9 262144      3.81469727e-06  0.625
10 1048576 9.53674316e-07  1
};

\end{loglogaxis}
\end{tikzpicture}
```

resulting in

log error 3.9e-3, R=1/5

log error 2.4e-4, R=4/5

The $\langle \text{\TeX code} \rangle$ is evaluated inside of a local scope, all locally declared variables are freed afterwards (that's why you can use any names you want).

`/pgfplots/clickable coords size=auto` or `{\langle max chars \rangle}` or `{\langle max chars x, max chars y \rangle}` (initially auto)

This is actually just another name for `annot/popup size snap`, see its documentation below.

5.1.2 Requirements for the Library

- The library relies on the $\text{\LaTeX}$ packages `insdljs` ("Insert document level Javascript") and `eforms` which are both part of the freely available `AcroTeX` education bundle [4]³. The `insdljs` package creates a temporary file with extension `.djs`.
- At the time of this writing, only Adobe Acrobat Reader interpretes Javascript and Forms properly. The library doesn't have any effect if the resulting document is used in other viewers (as far as I know).

Note that although this library has been written for PGFPLOTS, it can be used independently of a PGFPLOTS environment.

²See the `PGFPLOTS` manual for details about number printing.

³These packages rely on $\text{\LaTeX}$, so the library is only available for $\text{\LaTeX}$, not for plain $\text{\TeX}$ or `ConTeXt`.

Compatibility issues: There are several restrictions when using this library. Most of them will vanish in future versions – but up to now, I can’t do magic.

- The library does not yet support rotated axes. Use `clickable=false` for those axes.
- The library works only with `pdflatex`; `dvips` or `dvipdfm` are not supported⁴.
- Up to now, it is *not* possible to use this library together with the `external` library and other image externalization methods of Section 7.

To be more precise, you can (with two extra preamble lines, see below) get correctly annotated, exported PDF documents, but the `\includegraphics` command does not import the dynamic features.

In case you decide to use this work-around, you need to insert

```
% \maxdeadcycles=10000 % in case you get the error 'Output loop---<N> consecutive dead cycles.'
\usepackage[pdftex]{forms}
```

before loading PGF, TikZ or PGFPLOTS. The `\maxdeadcycles` appears to be necessary for large documents, try it out.

As long as you are working on a draft version of your document, you might want to use

```
\pgfkeys{/pgf/images/include external/.code={\href{file:#1}{\pgfimage{#1}}}}
```

in your preamble. This will generate hyperlinks around the graphics files which link to the exported figures. Clicking on the hyperlinks opens the exported figure which, in turn, has been generated with the `clickable` library and allows dynamic features⁵.

- The library automatically calls `\begin{Form}` at `\begin{document}` and `\end{Form}` at the end of the document. This environment of `hyperref` is necessary for dynamic user interaction and should be kept in mind if the document contains other form elements.

Acknowledgements:

- I have used a Javascript `sprintf` implementation of Kevin van Zonneveld [6] (the Javascript API has only a limited set of conversions).

5.1.3 Customization

It is possible to customize the library with several options.

`/pgfplots/clickable=true|false` (initially `true`)
Allows to disable the library for single plots.

`/pgfplots/annot/js fillColor={\langle Javascript color \rangle}` (initially `["RGB",1,1,.855]`)
Sets the background (fill) color of the short popup annotations.
Possible choices are `transparent`, `gray`, `RGB` or `CMYK` color specified as four-element-arrays of the form `["RGB", \langle red \rangle, \langle green \rangle, \langle blue \rangle]`. Each color component is between 0 and 1.
Again: this option is for Javascript. It is *not* possible to use colors as in PGF.

`/pgfplots/annot/point format={\langle sprintf-format \rangle}` (initially `(%.1f,%.1f)`)
`/pgfplots/annot/point format 3d={\langle sprintf-format \rangle}` (initially `(%.1f,%.1f,%.1f)`)
Allows to provide an `sprintf` format string which is used to fill the annotations with text. The first argument to `sprintf` is the *x*-coordinate and the second argument is the *y*-coordinate.
The `point format 3d` variant is used for any three-dimensional axis whereas the `point format` is used (only) for two-dimensional ones.
The `every semilogx axis`, `every semilogy axis` and `every loglog axis` styles have been updated to

⁴In fact, they should be. I don’t really know why they don’t . . . any hint is welcome.

⁵This special treatment needs the external files in the same base directory as the main document, so this approach is most certainly *not* suitable for a final document.

```
\pgfplotsset{
  every semilogy axis/.append style={/pgfplots/annot/point format={(\% .1f,\%.1e)}},
  every semilogx axis/.append style={/pgfplots/annot/point format={(\% .1e,\%.1f)}},
  every loglog axis/.append style={/pgfplots/annot/point format={(\% .1e,\%.1e)}}
}
```

such that every logarithmic coordinate is displayed in scientific format.

`/pgfplots/annot/slope format={\sprintf-format}` (initially `%.1f*x %+.1f`)

Allows to provide an `sprintf` format string which is used to fill the slope-annotation with text. The first argument is the slope and the second the line offset.

`/pgfplots/annot/printable=true|false` (initially `false`)

Allows to configure whether the small annotations will be printed. Otherwise, they are only available on screen.

`/pgfplots/annot/font={\Javascript font name}` (initially `font.Times`)

Allows to choose a Javascript font for the annotations. Possible choices are limited to what Javascript accepts (which is *not* the same as L^AT_EX). The default fonts and its names are shown below.

Font Name	Name in Javascript
Times-Roman	font.Times
Times-Bold	font.TimesB
Times-Italic	font.TimesI
Times-BoldItalic	font.TimesBI
Helvetica	font.Helv
Helvetica-Bold	font.HelvB
Helvetica-Oblique	font.HelvI
Helvetica-BoldOblique	font.HelvBI
Courier	font.Cour
Courier-Bold	font.CourB
Courier-Oblique	font.CourI
Courier-BoldOblique	font.CourBI
Symbol	font.Symbol
ZapfDingbats	font.ZapfD

`/pgfplots/annot/textSize={\Size in Point}` (initially 11)

Sets the text size of annotations in points.

`/pgfplots/annot/popup size generic=auto or {\langle x \rangle} or {\langle x,y \rangle}` (initially `auto`)

`/pgfplots/annot/popup size snap=auto or {\langle x \rangle} or {\langle x,y \rangle}` (initially `auto`)

`/pgfplots/annot/popup size={\value}`

The first key defines the size of popups if you just click into an axis. The second key defines the size of popups for the snap-to-nearest feature (i.e. those prepared by `clickable coords`). The third key sets both to the same `\value`.

The argument can be `auto` in which case PGFLOTS tries to be smart and counts characters. This may fail for multiline texts. The choice `\langle x \rangle` provides the *horizontal* size only, in units of `annot/textSize`. Thus, `annot/popup size generic=6` makes the popup $6 \cdot 11$ points wide. In this case, only one line will be allocated. Finally, `\langle x,y \rangle` allows to provide horizontal and vertical size, both in units of `annot/textSize`.

See also `clickable coords size` which is an alias for `annot/popup size snap`.

`/pgfplots/annot/snap dist={\Size in Point}` (initially 4)

Defines the size within two mouse clicks are considered to be equivalent, measured in points (Euclidean distance).

`/pgfplots/annot/richtext=true|false` (initially `true`)

Enables or disables richtext formatting in `clickable coords` arguments. Richtext is kind of XHTML and allows CSS styles like colors, font changes and other CSS attributes, see the documentation for `clickable coords` for details.

The case `annot/richtext=false` is probably more robust.

5.1.4 Using the Clickable Library in Other Contexts

This library provides essentially one command, `\pgfplotsclickablecreate` which creates a clickable area of predefined size, combined with Javascript interaction code. It can be used independently of PGFPLOTS.

`\pgfplotsclickablecreate` [*required key-value-options*]

Creates an area which is clickable. A click produces a popup which contains information about the point under the cursor.

The complete (!) context needs to be provided using key-value-pairs, either set before calling this method or inside of [*required key-value-options*].

This command actually creates an AcroForm which invokes Javascript whenever it is clicked. A Javascript Object is created which represents the context (axis limits and options). This Javascript object is available at runtime.

This method is public and it is *not* restricted to PGFPLOTS. The PGFPLOTS hook simply initializes the required key-value-pairs.

This method does not draw anything. It initializes only a clickable area and Javascript code.

The required key-value-pairs are documented below.

Attention: Complete key-value validation is *not* performed here. It can happen that invalid options will produce Javascript bugs when opened with Acrobat Reader. Use the Javascript console to find them.

All options described in the following are only interesting for users who intend to use this library without PGFPLOTS.

`/pgfplots/annot/width={⟨dimension⟩}` (initially -)

This required key communicates the area's width to `\pgfplotsclickablecreate`. It must be a TeX dimension like 5cm.

`/pgfplots/annot/height={⟨dimension⟩}` (initially -)

This required key communicates the area's height to `\pgfplotsclickablecreate`. It must be a TeX dimension like 5cm.

`/pgfplots/annot/jsname={⟨string⟩}` (initially -)

This required key communicates a unique identifier to `\pgfplotsclickablecreate`. This identifier is used to identify the object in Javascript, so there can't be more than one of them. If it is empty, a default identifier will be generated.

`/pgfplots/annot/xmin={⟨number⟩}`
`/pgfplots/annot/xmax={⟨number⟩}`
`/pgfplots/annot/ymin={⟨number⟩}`
`/pgfplots/annot/ymax={⟨number⟩}` (initially empty)

These required keys communicate the axis limits to `\pgfplotsclickablecreate`. They should be set to numbers which can be assigned to a Javascript floating point number (standard IEEE double precision).

`/pgfplots/annot/collected plots={⟨nested arrays⟩}` (initially empty)

The low level interface to implement a snap-to-nearest feature. The value is an array of plots, where each plot is again an array of coordinates and each coordinate is an array of three elements, x , y and text. Please consult the code comments for details and examples.

5.2 Colormaps

An extension by Patrick Häcker

```
\usepgfplotslibrary{colormaps} %  $\LaTeX$  and plain TeX
\usepgfplotslibrary[colormaps] % ConTeXt
\usetikzlibrary{pgfplots.colormaps} %  $\LaTeX$  and plain TeX
```

`\usetikzlibrary[pgfplots.colormaps]` % ConTeXt

A small library providing a number of additional `colormaps`. Many of these `colormaps` originate from the free Matlab package “SC — powerful image rendering” of Oliver Woodford.

The purpose of this library is to provide further `colormaps` to all users and to provide some of them which are similar to those used by Matlab (®).

`/pgfplots/colormap/autumn` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={autumn}{rgb255=(255,0,0) rgb255=(255,255,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/bled` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={bled}{rgb255=(0,0,0) rgb255=(43,43,0) rgb255=(0,85,0)
    rgb255=(0,128,128) rgb255=(0,0,170) rgb255=(213,0,213) rgb255=(255,0,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/bright` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={bright}{rgb255=(0,0,0) rgb255=(78,3,100) rgb255=(2,74,255)
    rgb255=(255,21,181) rgb255=(255,113,26) rgb255=(147,213,114) rgb255=(230,255,0)
    rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/bone` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={bone}{[1cm]rgb255(0cm)=(0,0,0) rgb255(3cm)=(84,84,116)
    rgb255(6cm)=(167,199,199) rgb255(8cm)=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/cold` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={cold}{rgb255=(0,0,0) rgb255=(0,0,255) rgb255=(0,255,255)
    rgb255=(255,255,255)}
}
```





This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/copper** (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={copper}{[1cm]rgb255(0cm)=(0,0,0) rgb255(4cm)=(255,159,101)
    rgb255(5cm)=(255,199,127)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/copper2** (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={copper2}{rgb255=(0,0,0) rgb255=(68,62,63) rgb255=(170,112,95)
    rgb255=(207,194,138) rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/earth** (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={earth}{rgb255=(0,0,0) rgb255=(0,28,15) rgb255=(42,39,6)
    rgb255=(28,73,33) rgb255=(67,85,24) rgb255=(68,112,46) rgb255=(81,129,83)
    rgb255=(124,137,87) rgb255=(153,147,122) rgb255=(145,173,164) rgb255=(144,202,180)
    rgb255=(171,220,177) rgb255=(218,229,168) rgb255=(255,235,199) rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/gray** (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={gray}{rgb255=(0,0,0) rgb255=(255,255,255)}
}
```



This **colormap** is an alias for the standard **colormap/blackwhite**.

This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/**colormap/hot2** (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={hot2}{[1cm]rgb255(0cm)=(0,0,0) rgb255(3cm)=(255,0,0)
    rgb255(6cm)=(255,255,0) rgb255(8cm)=(255,255,255)}
}
```



Note that this particular choice ships directly with PGFPLOTS, you do not need to load the `colormaps` library for this value.

This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/hsv` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={hsv}{rgb255=(255,0,0) rgb255=(255,255,0) rgb255=(0,255,0)
    rgb255=(0,255,255) rgb255=(0,0,255) rgb255=(255,0,255) rgb255=(255,0,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/hsv2` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={hsv2}{rgb255=(0,0,0) rgb255=(128,0,128) rgb255=(0,0,230)
    rgb255=(0,255,255) rgb255=(0,255,0) rgb255=(255,255,0) rgb255=(255,0,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/jet` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={jet}{rgb255(0cm)=(0,0,128) rgb255(1cm)=(0,0,255)
    rgb255(3cm)=(0,255,255) rgb255(5cm)=(255,255,0) rgb255(7cm)=(255,0,0)
    rgb255(8cm)=(128,0,0)}
}
```



Note that this particular choice ships directly with PGFPLOTS, you do not need to load the `colormaps` library for this value.

This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/pastel` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={pastel}{rgb255=(0,0,0) rgb255=(120,0,5) rgb255=(0,91,172)
    rgb255=(215,35,217) rgb255=(120,172,78) rgb255=(255,176,24) rgb255=(230,255,0)
    rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/colormap/pink

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={pink}{rgb255=(0,0,0) rgb255=(12,16,46) rgb255=(62,22,43)
    rgb255=(53,53,65) rgb255=(79,72,58) rgb255=(122,80,67) rgb255=(147,91,102)
    rgb255=(147,115,140) rgb255=(144,145,154) rgb255=(173,163,146) rgb255=(216,171,149)
    rgb255=(250,179,179) rgb255=(255,198,227) rgb255=(246,229,255) rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/colormap/sepia

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={sepia}{rgb255(0cm)=(0,0,0) rgb255(1cm)=(26,13,0)
    rgb255(18cm)=(255,230,204) rgb255(20cm)=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/colormap/spring

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={spring}{rgb255=(255,0,255) rgb255=(255,255,0)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/colormap/summer

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={summer}{rgb255=(0,128,102) rgb255=(255,255,102)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

/pgfplots/colormap/temp

(style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={temp}{rgb255=(36,0,217) rgb255=(25,29,247) rgb255=(41,87,255)
    rgb255=(61,135,255) rgb255=(87,176,255) rgb255=(117,211,255) rgb255=(153,235,255)
    rgb255=(189,249,255) rgb255=(235,255,255) rgb255=(255,255,235) rgb255=(255,242,189)
    rgb255=(255,214,153) rgb255=(255,172,117) rgb255=(255,120,87) rgb255=(255,61,61)
    rgb255=(247,40,54) rgb255=(217,22,48) rgb255=(166,0,33)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/thermal` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={thermal}{rgb255=(0,0,0) rgb255=(77,0,179) rgb255=(255,51,0)
    rgb255=(255,255,0) rgb255=(255,255,255)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

`/pgfplots/colormap/winter` (style, no value)

A style which is equivalent to

```
\pgfplotsset{
  /pgfplots/colormap={winter}{rgb255=(0,0,255) rgb255=(0,255,128)}
}
```



This colormap is similar to one shipped with Matlab (®) under a similar name.

5.3 Dates as Input Coordinates

```
\usepgfplotslibrary{dateplot} % LATEX and plain TEX
\usepgfplotslibrary[dateplot] % ConTEXt
\usetikzlibrary{pgfplots.dateplot} % LATEX and plain TEX
\usetikzlibrary[pgfplots.dateplot] % ConTEXt
```

A library which allows to use dates like 2008-01-01 or dates with time like 2008-01-01 11:35 as input coordinates in plots. The library converts dates to numbers and tick labels will be pretty-printed dates (or times).

This library is documented in Section 4.21.2 on page 297.

5.4 Image Externalization

```
\usepgfplotslibrary{external} % LATEX and plain TEX
\usepgfplotslibrary[external] % ConTEXt
\usetikzlibrary{pgfplots.external} % LATEX and plain TEX
\usetikzlibrary[pgfplots.external] % ConTEXt
```

The `external` library offers a convenient method to export every single `tikzpicture` into a separate .pdf (or .eps). Later runs of L^AT_EX will simply include these graphics, thereby reducing typesetting time considerably.

This library is documented in more detail in Section 7.1 “Export to PDF/EPS”.

The `external` library has been written by Christian Feuersänger (author of PGFPLOTS). It has been contributed to TikZ as general purpose library, so the reference documentation along with all tweaks can be found in [5, Section “Externalization Library”]. The command `\usepgfplotslibrary{external}` is actually just a wrapper which loads `\usetikzlibrary{external}` or, if this library does not yet exist because the installed PGF has at most version 2.00, it will load a copy which is shipped with PGFPLOTS.

5.5 Grouping plots

by Nick Papior Andersen

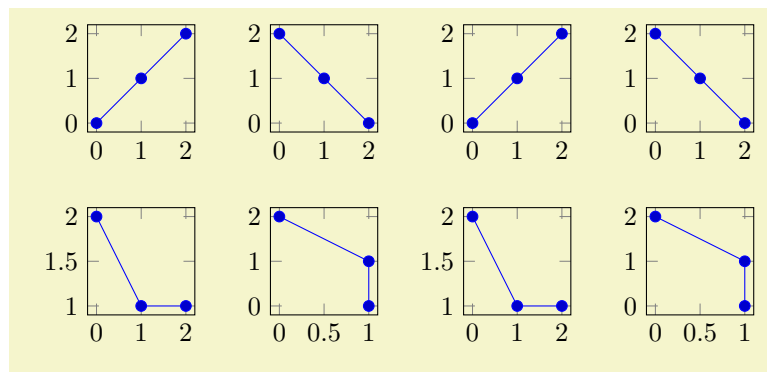
```
\usepgfplotslibrary{groupplots} %  $\LaTeX$  and plain  $\TeX$ 
\usepgfplotslibrary[groupplots] % Con $\TeX$ t
\usetikzlibrary{pgfplots.groupplots} %  $\LaTeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.groupplots] % Con $\TeX$ t
```

A library which allows the user to typeset several plots in a matrix like structure. Often one has to compare two plots to one another, or you simply need to display two plots in conjunction with each other. Either way the following section describes this library which makes matrix structure easier than alternative methods discussed in Section 4.18.4.

```
\begin{groupplot}[\langle options \rangle]
  \langle environment contents \rangle
\end{groupplot}
```

Once you have loaded the `groupplots` library you will gain access to this environment. This environment is limited to the same restrictions as the `axis` environment. It actually utilizes this environment so consider it as an extension of this. What is important to note is that $[\langle options \rangle]$ are applied to all plots in the entire environment. This can be really handy when you need the same `xmin`, `xmax`, `ymin` and `ymax`.

With such an environment one can typeset plots in matrix like styles

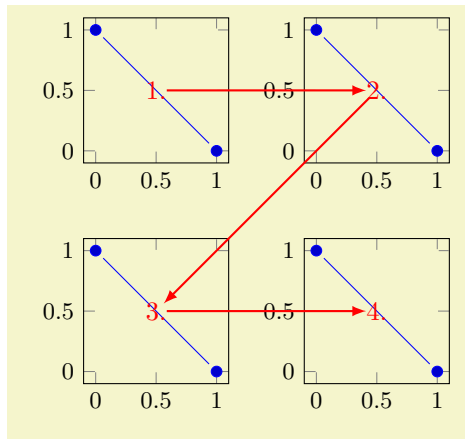


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% Example using groupplots library
\begin{tikzpicture}
  \begin{groupplot}[group style={group size=2 by 2,height=3cm,width=3cm}]
    \nextgroupplot
    \addplot coordinates {(0,0) (1,1) (2,2)};
    \nextgroupplot
    \addplot coordinates {(0,2) (1,1) (2,0)};
    \nextgroupplot
    \addplot coordinates {(0,2) (1,1) (2,1)};
    \nextgroupplot
    \addplot coordinates {(0,2) (1,1) (1,0)};
  \end{groupplot}
\end{tikzpicture}
% Same example created as done without the library
\begin{tikzpicture}
  \begin{axis}[name=plot1,height=3cm,width=3cm]
    \addplot coordinates {(0,0) (1,1) (2,2)};
  \end{axis}
  \begin{axis}[name=plot2,at={($ (plot1.east)+(1cm,0)$)},anchor=west,height=3cm,width=3cm]
    \addplot coordinates {(0,2) (1,1) (2,0)};
  \end{axis}
  \begin{axis}[name=plot3,at={($ (plot1.south)-(0,1cm)$)},anchor=north,height=3cm,width=3cm]
    \addplot coordinates {(0,2) (1,1) (2,1)};
  \end{axis}
  \begin{axis}[name=plot4,at={($ (plot2.south)-(0,1cm)$)},anchor=north,height=3cm,width=3cm]
    \addplot coordinates {(0,2) (1,1) (1,0)};
  \end{axis}
\end{tikzpicture}
```

The equivalent code is seen as the second example and it is clear that you have to type a lot less. So how do you use it? First of all you need to utilize the new environment `groupplot`. Within this environment the following command works.

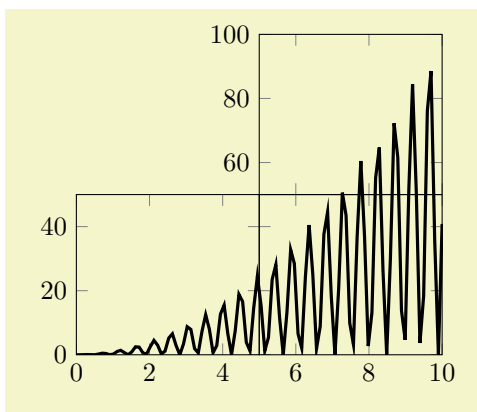
`\nextgroupplot` [*axis options*] *<normal plot commands>*

This command shifts the placement of the plot. Therefore one should always start the environment `groupplot` with the command `\nextgroupplot` in order to create the first plot. The [*axis options*] are the options that are supplied to the following axes until the next `\nextgroupplot` command is seen by $\text{\TeX}$. The order in which figures are typeset are as seen in the next example.



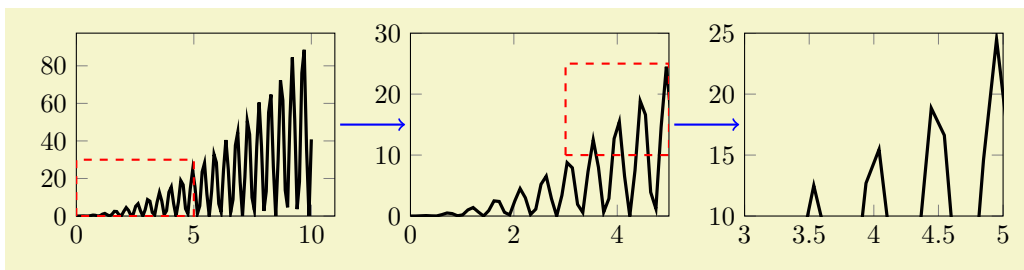
```
\begin{tikzpicture}[shorten >=4pt,shorten <=4pt]
\begin{groupplot}[group style={group size=2 by 2},
height=3.5cm,width=3.5cm,/tikz/font=\small]
\nextgroupplot% 1
\addplot coordinates {(0,1) (1,0)};
\nextgroupplot% 2
\addplot coordinates {(0,1) (1,0)};
\nextgroupplot% 3
\addplot coordinates {(0,1) (1,0)};
\nextgroupplot% 4
\addplot coordinates {(0,1) (1,0)};
\end{groupplot}
\draw[thick,>=latex,->,red]
(group c1r1.center) node {1.} --
(group c2r1.center) node {2.};
\draw[thick,>=latex,->,red]
(group c2r1.center) --
(group c1r2.center) node {3.};
\draw[thick,>=latex,->,red]
(group c1r2.center) --
(group c2r2.center) node {4.};
\end{tikzpicture}
```

The plot first fills the first row, then the next row and so on. Just like a table, thus the names `group c<column>r<row>`. The power of the `groupplot` is to quickly create an aligned structure of plots. But you can also utilize it to structure data more creatively. Consider the next example.



```
\begin{tikzpicture}
\begin{groupplot}[group style={group size=2 by 2,
horizontal sep=0pt,vertical sep=0pt,
xticklabels at=edge bottom},
xmin=0,ymin=0,
height=3.7cm,width=4cm,no markers]
\nextgroupplot[group/empty plot]
\nextgroupplot[xmin=5,xmax=10,ymin=50,ymax=100]
\addplot[very thick] file {plotdata/group-1.dat};
\nextgroupplot[xmax=5,ymax=50]
\addplot[very thick] file {plotdata/group-1.dat};
\nextgroupplot[xmin=5,xmax=10,ymax=50,
yticklabels={}]
\addplot[very thick] file {plotdata/group-1.dat};
\end{groupplot}
\end{tikzpicture}
```

Or for instance zooming in on data as in the next example.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{groupplot}[group style={group size=3 by 1},xmin=0,ymin=0,height=4cm,width=5cm,no markers]
    \nextgroupplot
    \addplot[very thick] file {plotdata/group-1.dat};
    \draw[red,dashed,thick] (axis cs:0,0) rectangle (axis cs:5,30);
    \nextgroupplot[xmax=5,ymax=30]
    \addplot[very thick] file {plotdata/group-1.dat};
    \draw[red,dashed,thick] (axis cs:3,10) rectangle (axis cs:5,25);
    \nextgroupplot[xmin=3,xmax=5,ymin=10,ymax=25]
    \addplot[very thick] file {plotdata/group-1.dat};
  \end{groupplot}
  \draw[thick,blue,->,shorten >=2pt,shorten <=2pt]
    (group c1r1.east) -- (group c2r1.west);
  \draw[thick,blue,->,shorten >=2pt,shorten <=2pt]
    (group c2r1.east) -- (group c3r1.west);
\end{tikzpicture}
```

5.5.1 Grouping options

`/pgfplots/group style= $\{\langle options with group/ prefix \rangle\}$`

This key sets all $\langle options \rangle$ using the `/pgfplots/group/` prefix.

Note that the distinction between `group/` and normal options is important as some of them are quite similar.

For example, the following statements are all equivalent:

```
\pgfplotsset{group style={a=2,b=3}}
\pgfplotsset{group/a=2,group/b=3}
\pgfplotsset{group/.cd,a=2,b=3}
```

All the following keys are in the subdirectory `group`.

`/pgfplots/group/group size= $\langle columns \rangle$  by  $\langle rows \rangle$` (initially 1 by 1)
`/pgfplots/group/columns= $\langle columns \rangle$` (initially 1)
`/pgfplots/group/rows= $\langle rows \rangle$` (initially 1)

These keys determine the total number of plots that can be in one environment `groupplot`. It is thus important not to add more `\nextgroupplot` in the environment than $\langle columns \rangle \times \langle rows \rangle$. This is critical to set if one uses more than 1 more plot. As the key `group size` uses `columns` and `rows` you should stick to either `group size` or both `columns` and `rows`.

`/pgfplots/group/horizontal sep= $\langle dimension \rangle$` (initially 1cm)
`/pgfplots/group/vertical sep= $\langle dimension \rangle$` (initially 1cm)

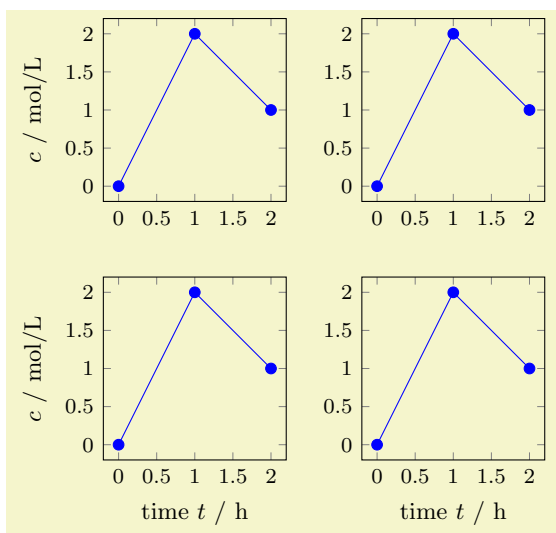
The spacing between the plots in the horizontal and vertical direction, respectively. If you thus want them to be *glued* together you should set them both to a length of `0pt`.

`/pgfplots/group/every plot/.style= $\{\langle style \rangle\}$` (initially empty)

This style is used on every plot as the first style. It is thus equivalent as $\langle options \rangle$ in the `groupplot` environment.

`/pgfplots/group/xlabels at=all|edge bottom|edge top` (initially all)
`/pgfplots/group/ylabels at=all|edge left|edge right` (initially all)

In order to determine which plots get labels typeset one can use these keys. By default all axes get typeset normally and thus have both x and y axis labels.



```
\begin{tikzpicture}
  \begin{groupplot}[
    group style={
      group name=my plots,
      group size=2 by 2,
      xlabel at=edge bottom,
      ylabel at=edge left,
    },
    footnotesize,
    width=4cm,
    height=4cm,
    %
    xlabel=time $t$ / h,
    ylabel=$c$ / mol/L,
  ]
  \nextgroupplot
  \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
  \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
  \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
  \addplot coordinates{(0,0) (1,2) (2,1)};
  \end{groupplot}
\end{tikzpicture}
```

In the example above, only the bottom row gets the label defined in the beginning `groupplot`-environment on the x axis and only the first column of plots gets labels on the y axis on their left side. These keys are especially handy when using *glued* plots.

`/pgfplots/group/xticklabels at=all|edge top|edge bottom` (initially all)
`/pgfplots/group/yticklabels at=all|edge left|edge right` (initially all)

In order to determine which plots get tick labels typeset one can use these keys. By default all axes gets typeset normally and thus have both x and y axis tick labels. If one sets

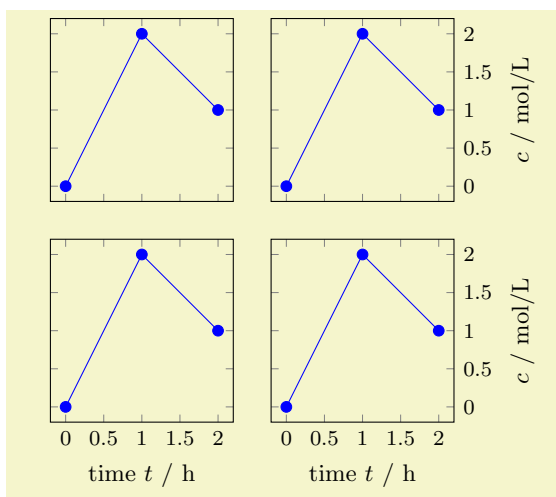
```
\pgfplotsset{group/xticklabels at=edge bottom,group/yticklabels at=edge right}
```

only the bottom row gets tick labels on the x axis and only the last column gets tick labels on the y axis on their right side. These keys are specially handy when using *glued* plots.

Keep in mind that this implies the same ticks for all plots.

`/pgfplots/group/x descriptions at=all|edge top|edge bottom` (initially all)
`/pgfplots/group/y descriptions at=all|edge left|edge right` (initially all)

These are simply a short hand for using both `xticklabels at` and `xlabels at` simultaneously:



```
\begin{tikzpicture}
  \begin{groupplot}[
    group style={
      group name=my plots,
      group size=2 by 2,
      %
      x descriptions at=edge bottom,
      y descriptions at=edge right,
      horizontal sep=0.5cm,
      vertical sep=0.5cm,
    },
    footnotesize,
    width=4cm,
    height=4cm,
    %
    xlabel=time $t$ / h,
    ylabel=$c$ / mol/L,
  ]
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \end{groupplot}
\end{tikzpicture}
```

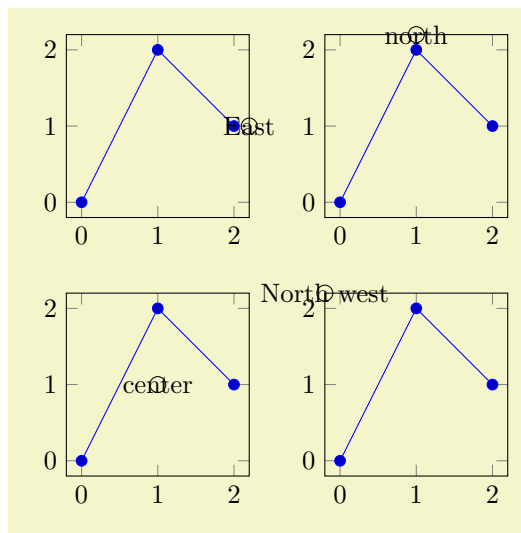
Here, `x descriptions at=edge bottom` yields that x descriptions (`xlabel` and `xticklabel`) are only used for the lowest row. Furthermore, `y descriptions at=edge right` places y descriptions only for the rightmost column. Consider modifying the `horizontal sep` and `vertical sep` for your needs.

As for `xticklabels at`, usage of this key implies the same ticks for all plots.

This might require `compat=1.3` (or newer).

`/pgfplots/group/group name={\name}` (initially `group`)

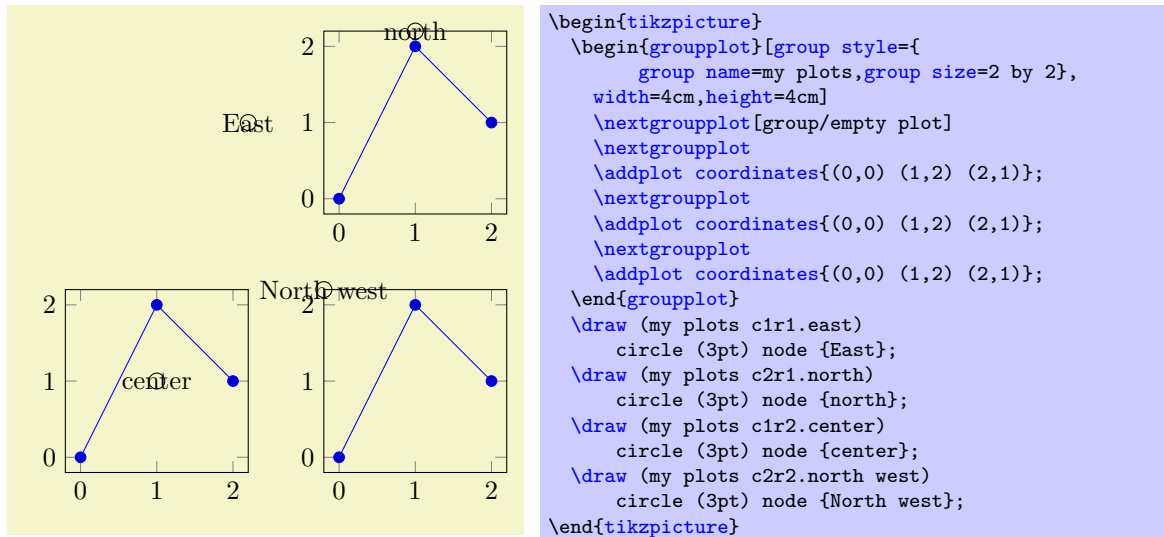
This sets what you can refer the plots to after typesetting. Thus you can use their anchors later. See the following example



```
\begin{tikzpicture}
  \begin{groupplot}[group style={
    group name=my plots,group size=2 by 2},
    width=4cm,height=4cm]
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \nextgroupplot
    \addplot coordinates{(0,0) (1,2) (2,1)};
  \end{groupplot}
  \draw (my plots c1r1.east)
    circle (3pt) node {East};
  \draw (my plots c2r1.north)
    circle (3pt) node {North};
  \draw (my plots c1r2.center)
    circle (3pt) node {center};
  \draw (my plots c2r2.north west)
    circle (3pt) node {North west};
\end{tikzpicture}
```

`/pgfplots/group/empty plot/.style={\style}` (initially `/pgfplots/hide axis`)

This key can be used as an option to the command `\nextgroupplot`. This makes the next plot invisible (only the axes) but maintains its anchors and name. If you want it to behave in another style then you can redefine it. Consider the same example as before.



Notice that you need to call a `\nextgroupplot` againwards to jump to the next plot.

5.6 Patchplots Library

```

\usepgfplotslibrary{patchplots} %  $\TeX$  and plain  $\TeX$ 
\usepgfplotslibrary{patchplots} % Con $\TeX$ t
\usetikzlibrary{pgfplots.patchplots} %  $\TeX$  and plain  $\TeX$ 
\usetikzlibrary{pgfplots.patchplots} % Con $\TeX$ t

```

A library for advanced `patch` plots. Its strength is the creation of patches with smooth boundaries and smoothly shaded colors.

A `patch` plot is a plot in which each individual patch is available. Here, “available” means that the user provided each individual patch manually. This can be achieved by means of a long series of patches which have been concatenated in a suitable way (compare the description of `patch` plots in section 4.5.12) or by means of a mathematical expression which is sampled (compare the key `patch type sampling`). Most `patch types` expect a series of point evaluations in a specific sequence.

Note that even though each individual patch might have a smooth boundary, the `patchplots` library *does not interpolate smoothly between adjacent patches*. Consequently, it is task of the one who creates the patches (which means: evaluated some function at its vertices) to ensure that patches can be glued together in an adequate way. This allows a lot of freedom, including both jumps and smoothly concatenated edges.

The `patchplots` library comes with a couple of inherently two-dimensional `patch types` (including second order triangles/rectangular patches and cubic tensor product patches known for finite elements). Typically, these patches live in a three-dimensional axis. Often, they are used to visualize the surface of function values $f(x, y)$. The `patchplots` library ensures that such patches are drawn in a way which respects the current view. In particular, if a patch folds over itself (which is possible), it is drawn such that foreground areas are in the foreground and background areas are in the background.

The `patchplots` library comes with smoothly shaded patches. More precisely, both the boundary of patches and their color shading are smooth. Note, however, that the patch boundary typically has much more smoothness than the color shading.

The `patchplots` library also allows automatic conversion from a higher-order patch to triangles (triangulation) by means of the key `patch to triangles`. Furthermore, it features automatic `patch refines`.

Use the `patchplots` library if you want to have smooth boundaries for your patches, or if you need advanced shadings, or if you want polygon plots, or if you want more freedom in one-dimensional patches.

5.6.1 Additional Patch Types

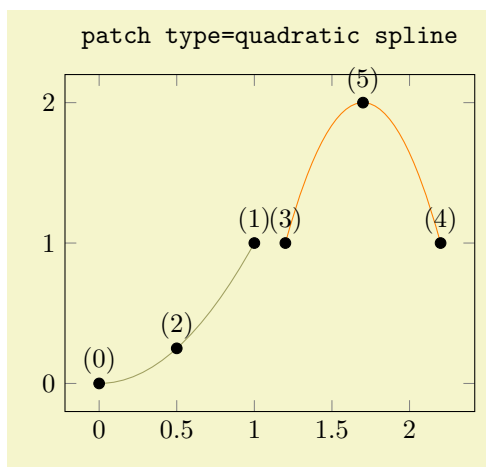
/pgfplots/**patch type**=default|rectangle|triangle|line|quadratic spline|cubic spline|bilinear|triangle quadr|biquadratic|bicubic|polygon|coons|tensor bezier (initially default)

The **patchplots** library supports several new **patch types** in addition to the initially available choices (which are **rectangle**, **triangle** and **line**). The documentation of the two-dimensional choices from page 133 is repeated here.

The new **patch types** are discussed in detail on the following pages.

One-Dimensional Patch Types

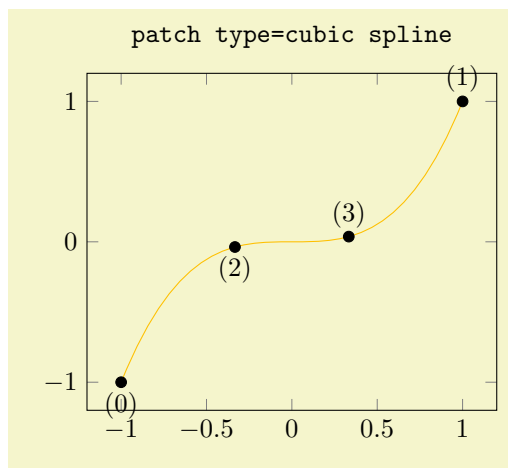
There are two new one-dimensional patch types, namely **quadratic spline** and **cubic spline**. Here, **patch type=quadratic spline** consists of quadratic patches of $n = 3$ vertices each. The vertices are interpolated exactly:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title={\texttt{patch type=quadratic spline}}]
\addplot[
mark=*,
patch,
patch type=quadratic spline]
coordinates {
% left, right, middle-> first segment
(0,0) (1,1) (0.5,0.5^2)
% left, right, middle-> second segment
(1.2,1) (2.2,1) (1.7,2)
};
\end{axis}
\end{tikzpicture}
```

In our example, the first segment interpolates $f(x) = x^2$ at the points $\{0, 1/2, 1\}$. The **quadratic spline** is actually nothing but piecewise Lagrangian interpolation with quadratic polynomials: it expects three points in the sequence ‘(left end), (right end), (middle)’ and interpolates these three points with a quadratic polynomial. Unlike the default 1d **mesh** visualization (which uses **patch type=line** implicitly), you have to use the special syntax above (or the equivalent approach by means of **patch table**). Note that **patch type=quadratic spline** results in correct shapes, but uses *just constant color* for each segment; high-order color shading is only supported approximately using **patch refines**.

The **patch type=cubic spline** is very similar: it expects patches of $n = 4$ vertices and interpolates them with a cubic polynomial:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title={\texttt{patch type=cubic spline}}]
\addplot[
mark=*,
patch,
patch type=cubic spline]
coordinates {
% left, right, left middle, right middle
(-1,-1)
(1,1)
(-1/3,{(-1/3)^3})
(1/3,{(1/3)^3})
};
\end{axis}
\end{tikzpicture}
```

Here, we interpolated $f(x) = x^3$ at the four equidistant points $\{-1, -1/3, 1/3, 1\}$ with a cubic polynomial (which is x^3). The **cubic spline** expects a sequence of patches, each with four coordinates, given in

the sequence ‘(left end), (right end), (interpolation point at $1/3$), (interpolation point at $2/3$)’. It has limitations and features like `quadratic spline`, see above.

Providing Patches by means of Mathematical Expressions

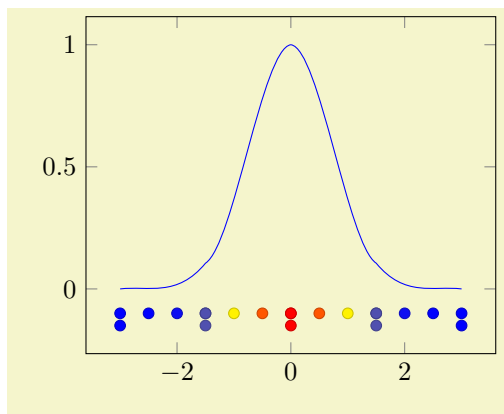
Most `patch types` expect a specific number of vertices in a specific sequence. This is part of what the `patchplots` library is. But it is still tedious to provide this sort of data.

For simple `patch types` like `line`, `rectangle` and `bilinear`, you can provide the input coordinates with any of the input methods which are available for all other plot handlers. In particular, `line` is just a `sharp plot` (with individually colored segments) and `rectangle` is nothing but a `surf` plot. Note that both `rectangle` and `bilinear` also accept the standard matrix input (with scanlines, see `mesh/ordering` and its documentation). In summary: *simple patch types accept a simple input format.*

`/pgfplots/patch type sampling=true|false` (initially false)

There are some complicated `patch types`. In particular, all `patch types` of higher order (i.e. `quadratic spline`, `cubic spline`, `triangle quadr`, `biquadratic`, `bicubic`) need more points than just their corners. For such patch types, you need to resort to `mesh input=patches`. That means you need to provide extra vertices and their function evaluation values in a specific sequence.

The `patch type sampling` method allows to simplify the procedure for such complicated `patch types`⁶: it works together with `\addplot expression` and evaluates the mathematical expression at each of the required vertices (in the correct sequence):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot[
      samples=5,domain=-3:3,
      mesh,patch type=cubic spline,
      patch type sampling,
      % avoid individual colors per segment:
      blue,point meta=none,
    ]
      {exp(-x^2)};

    % a second plot which shows the
    % generated x positions:
    \addplot[
      mark=*,only marks,scatter,
      samples=5,domain=-3:3,
      patch type=cubic spline,
      patch type sampling,
      point meta={exp(-x^2)},
    ]
      {-0.1};

    % a third plot which shows the marks
    % without patch type sampling:
    \addplot[
      mark=*,only marks,scatter,
      samples=5,domain=-3:3,
      point meta={exp(-x^2)},
    ]
      {-0.15};

  \end{axis}
\end{tikzpicture}
```

The first plot above is almost a normal plot by expression. The `samples` and `domain` key controls the sampling procedure, and `blue,point meta=none` defines the global color to use. Note that the special choice `point meta=none` simply disables individual colors per mesh segment (which is the default for `mesh` plots). However, the `patch type sampling` key here makes a huge difference: it tells PGFLOTS to check the current value of `patch type` and to sample a coordinate sequence which is suitable as input for that `patch type`. We see that the outcome is a partially smooth function (more about that below).

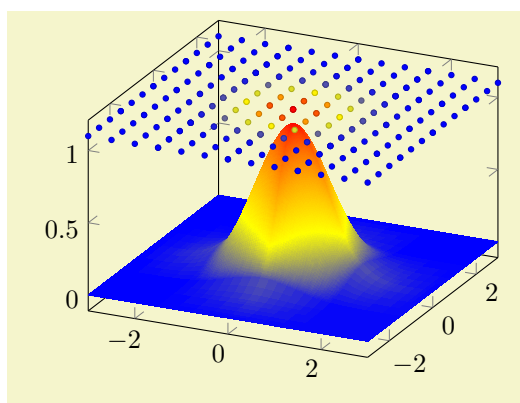
⁶Note that `patch type sampling` is more or less useless for simple patch types.

The method `patch type sampling` samples x just as usual. The result is a sequence $[x_0, x_1, \dots, x_k]$. For each interval $[x_i, x_{i+1}]$, a `patch type` is sampled inside of the interval. To this end, the current `patch type` is used to generate a standardized vertex pattern in the unit cube. For `patch type=cubic spline`, this generates four points $0, 1/3, 2/3, 1$. These standardized numbers are mapped into $[x_i, x_{i+1}]$. Then, any mathematical expressions (in our case $\exp(-x^2)$) are evaluated at the resulting positions.

The second plot in our example above shows the `markers` resulting from `patch type sampling`. Note that we see 13 markers even though we have said `samples=5`. These 5 samples are shown in the third plot. This is because `patch type=cubic spline` needs 4 points for each patch (i.e. 4 points in each sampled interval).

Note that even though the result in our example above is *partially* smooth, it is *not* globally smooth. In other words: each resulting mesh segment is a polynomial of third order. But: the five cubic polynomials are determined independently; and they are simply glued together without any intelligence. In particular, they are *unsmooth* at the five initial sampling points! This key cannot apply global smoothing. It is really just a convenient method which simplifies sampling of such patch types.

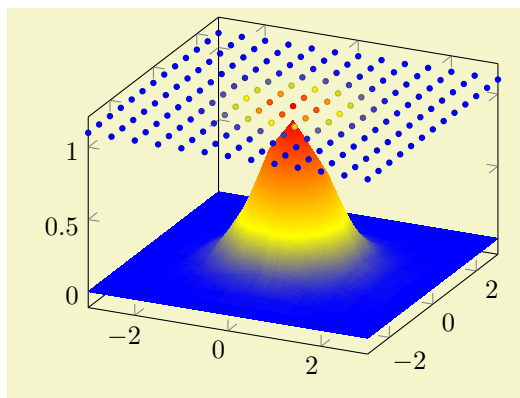
The method `patch type sampling` can also be used for `surf` plots, i.e. for matrix sampling. It works in the same way:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot3[surf,shader=interp,
    patch type=bicubic,
    patch type sampling,
    samples=5,domain=-3:3]
    {exp(-x^2-y^2)};

% show the generated grid on top:
\addplot3[
    mark=*,mark size=1pt,only marks,scatter,
    samples=5,domain=-3:3,
    patch type=bicubic,
    patch type sampling,
    point meta={exp(-x^2-y^2)},
]
{1.1};
\end{axis}
\end{tikzpicture}
```

The example is similar to our one-dimensional example above: it uses the same 1d function as product. We see that it has 13^2 samples instead of just 5^2 , and we see that the geometry is partially smooth (see above for “partially”). Note, however, that the color interpolation is only applied once per patch. The following example shows a `bilinear` patch with unsmooth geometry, but higher resolution for the color data, on a 13×13 mesh:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot3[surf,shader=interp,
    patch type=bilinear,
    samples=13,domain=-3:3]
    {exp(-x^2-y^2)};

% show the generated grid on top:
\addplot3[
    mark=*,mark size=1pt,only marks,scatter,
    samples=13,domain=-3:3,
    point meta={exp(-x^2-y^2)},
]
{1.1};
\end{axis}
\end{tikzpicture}
```

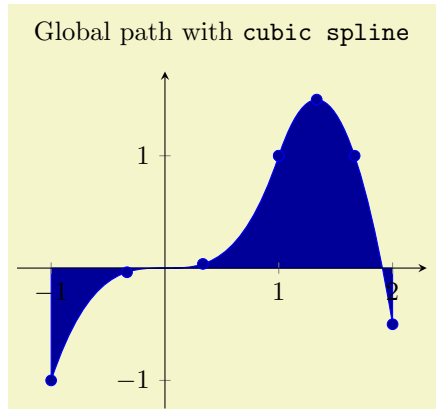
Note that you may want to view the preceding examples in Acrobat Reader. Many free pdf viewers cannot display these shadings properly.

Global One-Dimensional Curves with Smooth Splines

Typically, PGFPLOTS assumes that you want individually colored patch segments whenever you use one of the plot handlers `mesh`, `surf`, or `patch`. The individual colors are determined by the current `colormap` and the value of `point meta` (compare section 4.7).

Technically, individually colored path segments are one unit. If you `fill` them, you fill only one segment. You cannot fill them against the axis. In particular, you cannot use `\closedcycle` for individually colored `mesh` or `patch` plots.

The `patchplots` library comes with one-dimensional `patch` types like `quadratic spline` or `cubic spline`. It would be useful to draw a global path, that is: one which has a single color such that `\closedcycle` works. This is supported if you write `point meta=none`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    axis lines=middle,
    axis on top,
    enlargelimits,
    title={Global path with
        \texttt{cubic spline}}]
\addplot[
    mark=*,
    patch,
    patch type=cubic spline,
    point meta=none,% allow \closedcycle
    blue,
    fill=blue!60!black,
]
table {
    % left, right, left middle, right middle
    -1      -1
    1       1
    -0.33333  -0.037037
    0.33333  +0.037037

    1       1
    2       -0.5
    1.33333  1.5
    1.66666  1
}
\closedcycle;
\end{axis}
\end{tikzpicture}
```

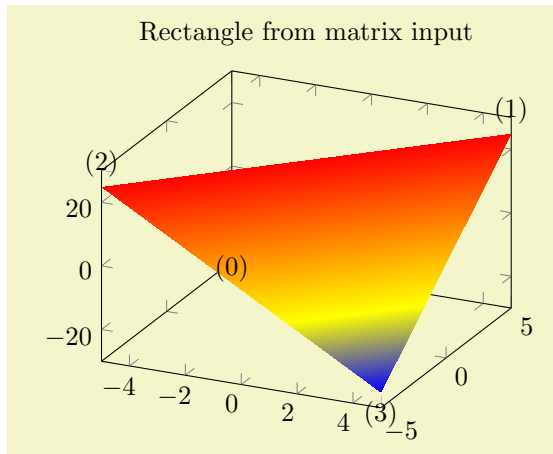
Two-Dimensional Patch Types

The `patchplots` library is especially strong for `shader=interp`, so this is our main focus in the remaining documentation here.

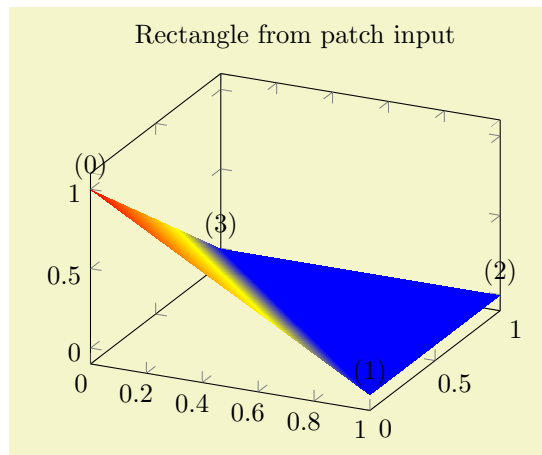
Attention: At the time of this writing, many free pdf viewers do not fully support the following shadings⁷. The preferred viewer is Adobe Acrobat Reader.

The choice `rectangle` expects one or more rectangular patches with $n = 4$ vertices each. These vertices are either encoded as a matrix or as individual patches (using `mesh input=patches`), in the sequence in which you would connect the vertices:

⁷The author of this package has submitted bugfixes to Linux viewers based on xpdf/libpoppler, so the problem will (hopefully) vanish in future versions.



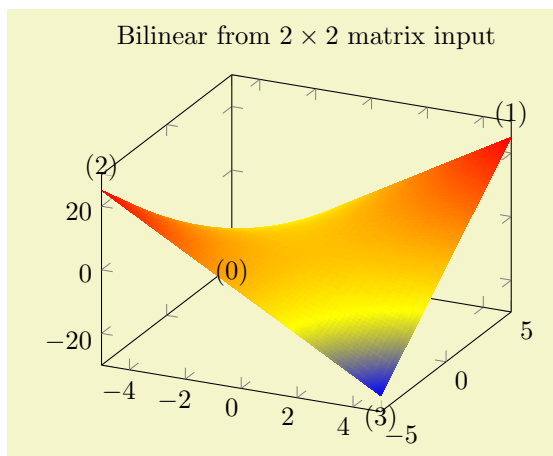
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={{\coordindex}},
title=Rectangle from matrix input]
% note that surf implies 'patch type=rectangle'
\addplot3[surf,shader=interp,samples=2,
patch type=rectangle]
{x*y};
\end{axis}
\end{tikzpicture}
```



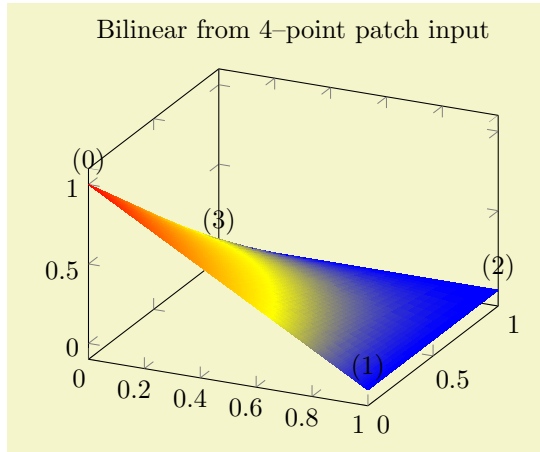
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={{\coordindex}},
title=Rectangle from patch input]
\addplot3[patch,shader=interp,patch
type=rectangle] coordinates {
(0,0,1) (1,0,0) (1,1,0) (0,1,0)
};
\end{axis}
\end{tikzpicture}
```

As already documented on page 133, the `shader=interp` implementation for `rectangle` uses two triangles and interpolates them linearly. The differences between the two examples above arise due to z buffering approaches: the matrix input reorders the matrix in linear time, whereas the second example would sort complete rectangles. In our case, this yields to the different corner sequence.

The choice `bilinear` is essentially the same as `rectangular` with respect to its input formats and stroke paths, but it uses correct bilinear shading for `shader=interp`. Moreover, the geometry is also interpolated bilinearly instead of just two triangles. The two examples from above now become



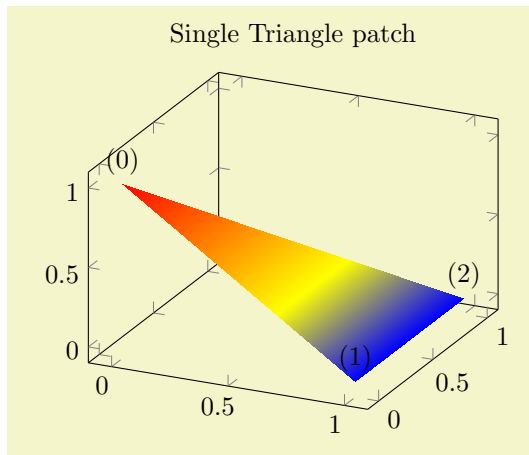
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={{\coordindex}},
title=Bilinear from  $2 \times 2$  matrix input]
% note that surf implies 'patch type=rectangle'
\addplot3[surf,shader=interp,samples=2,
patch type=bilinear]
{x*y};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title=Bilinear from $4$-point patch input]
\addplot3[patch,shader=interp,patch type=bilinear]
coordinates {
(0,0,1) (1,0,0) (1,1,0) (0,1,0)
};
\end{axis}
\end{tikzpicture}
```

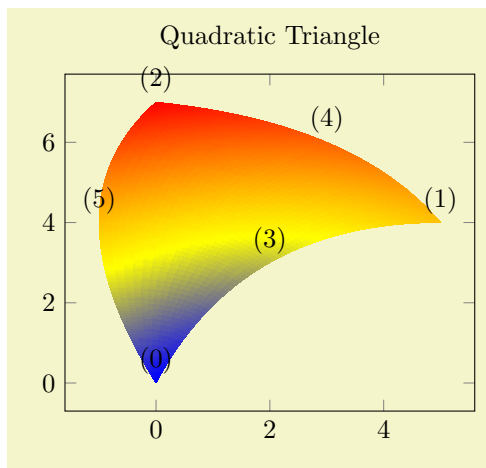
Use `patch type=bilinear` if you want to improve the shape of individual patches and the quality of the color interpolation. In contrast to the simpler `patch type=rectangle`, it might result in a huger output document.

The choice `triangle` expects a sequence of linear triangles, each encoded using $n = 3$ vertices:

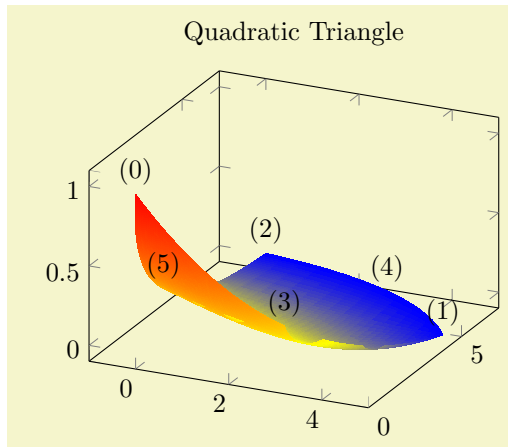


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[enlargelimits,
nodes near coords={(\coordindex)},
title=Single Triangle patch]
\addplot3[patch,shader=interp] coordinates {
(0,0,1)
(1,0,0)
(1,1,0)
};
\end{axis}
\end{tikzpicture}
```

The choice `triangle quadr` expects a sequence of isoparametric quadratic triangles, each defined by $n = 6$ vertices:



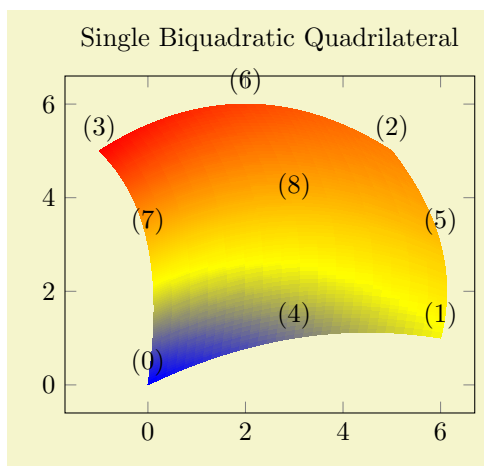
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title=Quadratic Triangle]
\addplot[patch,patch type=triangle quadr,
shader=interp,point meta=explicit]
coordinates {
(0,0) [1] (5,4) [2] (0,7) [3]
(2,3) [1] (3,6) [2] (-1,4) [3]
};
\end{axis}
\end{tikzpicture}
```

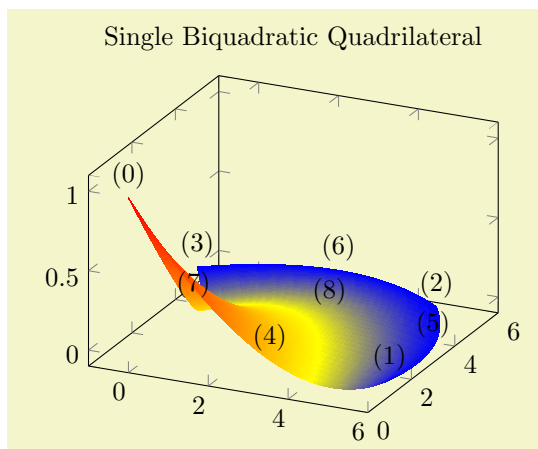
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title=Quadratic Triangle]
\addplot3[patch,patch type=triangle quadr,
shader=interp]
coordinates {
(0,0,1) (5,4,0) (0,7,0)
(2,3,0) (3,6,0) (-1,4,0)
};
\end{axis}
\end{tikzpicture}
```

Here, the edges have the correct quadratic shape. However, the color interpolation is just *bilinear*; using the color values of the corners and ignoring the rest (consider using `patch refines` to improve the color interpolation). For three dimensions, PGFplots checks the depth of corners to determine foreground/background. For two dimensions, strongly distorted elements may fold over each other in unexpected ways.

The choice `biquadratic` expects a sequence of isoparametric biquadratic quadrilaterals each defined by $n = 9$ vertices. Their main use is to get “rectangles” with smooth boundaries:



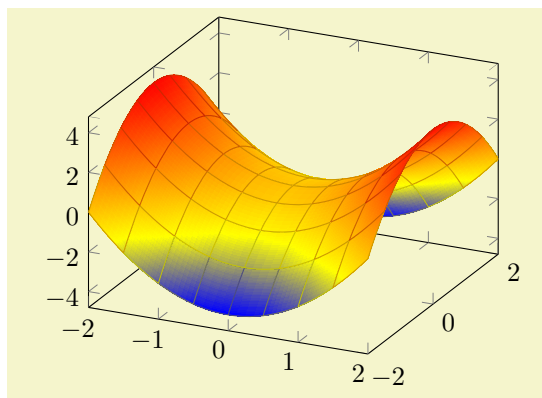
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title=Single Biquadratic Quadrilateral]
\addplot[patch,patch type=biquadratic,
shader=interp,point meta=explicit]
coordinates {
(0,0) [1] (6,1) [2] (5,5) [3] (-1,5) [4]
(3,1) [1] (6,3) [2] (2,6) [3] (0,3) [4]
(3,3.75) [4]
};
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
title=Single Biquadratic Quadrilateral]
\addplot3[patch,patch type=biquadratic,shader=interp]
coordinates {
(0,0,1) (6,1,0) (5,5,0) (-1,5,0)
(3,1,0) (6,3,0) (2,6,0) (0,3,0)
(3,3.75,0)
};
\end{axis}
\end{tikzpicture}
```

Similar to `triangle quadr`, the edges have the correct quadratic shape – but the color interpolation is just *bilinear*; using the color values of the corners and ignoring the rest. Again, ensure that the mesh width so small enough in order to improve the quality of the color interpolation (see also `patch refines`).

Note that a function of (x, y) is biquadratic if it is quadratic w.r.t. x if $y = \text{const}$ and also quadratic w.r.t. y if $x = \text{const}$ (note that this is not an “if and only if”). For example, $f(x, y) = x^2 - y^2$ is biquadratic. Consequently, we can represent a surface plot of f with just one biquadratic patch – only the color interpolation is just bilinear. We do so using `\addplot table[z expr=<expression>]`:



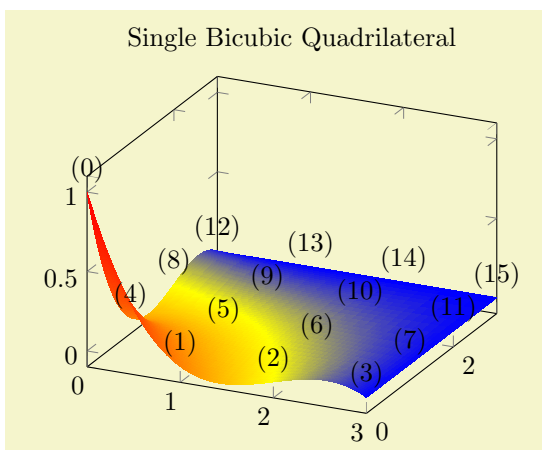
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}
    \addplot3[patch,patch refines=3,
      shader=faceted interp,
      patch type=biquadratic]
      table[z expr=x^2-y^2]
      {
        x y
        -2 -2
        2 -2
        2 2
        -2 2
        0 -2
        2 0
        0 2
        -2 0
        0 0
      };
  \end{axis}
\end{tikzpicture}
```

We see that the shape’s boundary is reconstructed exactly using the `biquadratic` patch. In addition, `patch refines` improves the (first order) color interpolation. Details for `patch refines` are discussed in Section 5.6.2 and details and limitations regarding superimposed grid lines are discussed in Section 5.6.4.

Note that `biquadratic` can easily be combined with `patch type sampling` in order to sample an arbitrary surface plot with smooth boundaries.

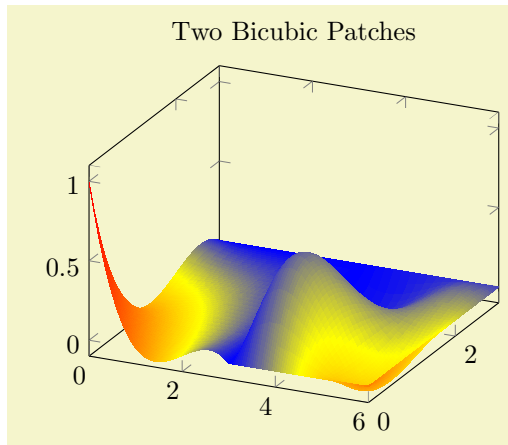
A patch with type `biquadratic` and `shader=interp` has a bounding box which is determined from the input vertices. Due to the high order of the patch, parts of the patch can be outside of that bounding box. This holds for all advanced patch types.

The choice `bicubic` is similar to `biquadratic`: it allows to defines two-dimensional patches whose boundary is defined by four cubic polynomials. Consequently, it allows very smooth boundaries – especially since the viewer constructs these boundaries at every zoom level. A `bicubic` patch is constructed from 16 points which are arranged in a 4×4 matrix. Each consecutive 16 points make up a single `bicubic` patch. The 17th point starts the next `bicubic` patch (just as for any other `patch type`).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[nodes near coords={(\coordindex)},
    title=Single Bicubic Quadrilateral]
    \addplot3[patch,patch type=bicubic,shader=interp]
      coordinates {
        (0,0,1) (1,0,0) (2,0,0) (3,0,0)
        (0,1,0) (1,1,0) (2,1,0) (3,1,0)
        (0,2,0) (1,2,0) (2,2,0) (3,2,0)
        (0,3,0) (1,3,0) (2,3,0) (3,3,0)
      };
  \end{axis}
\end{tikzpicture}
```

Just as for `biquadratic`, the color interpolation of `bicubic` is (just) bilinear, even though the geometry is of higher order. The color interpolation uses the `point meta` values determined at the four corners of each patch; all other values of `point meta` are ignored by the shader (although their values are used to compute `point meta min` and `point meta max`).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title=Two Bicubic Patches
\addplot3[patch,patch type=bicubic,
    shader=interp,point meta=explicit]
coordinates {
    (0,0,1) [1] (1,0,0) [0] (2,0,0) [0] (3,0,0) [0]
    (0,1,0) [0] (1,1,0) [0] (2,1,0) [0] (3,1,0) [0]
    (0,2,0) [0] (1,2,0) [0] (2,2,0) [0] (3,2,0) [0]
    (0,3,0) [0] (1,3,0) [0] (2,3,0) [0] (3,3,0) [0]

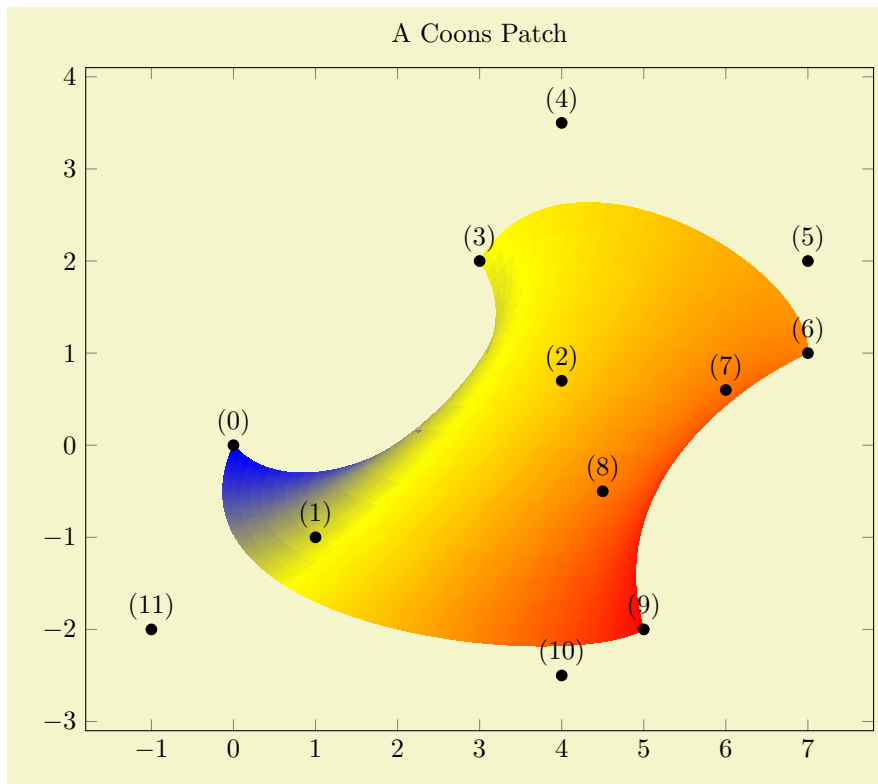
    (3,0,0) [0] (4,0,0) [0] (5,0,0) [0] (6,0,0) [0.7]
    (3,1,0) [0] (4,1,.5) [1] (5,1,0) [0] (6,1,0) [0]
    (3,2,0) [0] (4,2,0) [0] (5,2,0) [0] (6,2,0) [0]
    (3,3,0) [0] (4,3,0) [0] (5,3,0) [0] (6,3,0) [0.1]
};
\end{axis}
\end{tikzpicture}
```

The previous example uses two patches of type `bicubic`. Note that the color data (`point meta`) has been provided explicitly – and its values are only used at the corners (the `[1]` value after the point `(4,1,.5)` is ignored). Color interpolation of `bicubic` patches uses only the color data at the patch's corners. The remaining color data values are ignored. Note that if you leave the default (which is `point meta=f(x)` instead of `point meta=explicit`), the second patch will be blue. This is because the four corner vertices of the second patch define the color shading – and their z value is 0.

Note that `bicubic` can easily be combined with `patch type sampling` in order to sample an arbitrary surface plot with smooth boundaries.

Just as described for `biquadratic`, a patch with type `bicubic` and `shader=interp` can have a bounding box which is slightly smaller than the region which is actually drawn (because the bounding box is computed from the input points).

The choice `coons` expects a sequence of one or more Coons patches, made up of $n = 12$ points each. A Coons patch is delimited by four cubic Bézier curves, with the end points attached to each other – and the n points provide the required control points for these curves in a specific ordering which is illustrated in the following example:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[nodes near coords={(\coordindex)},
width=12cm,
title=A Coons Patch]
\addplot[mark=*,patch,patch type=coons,
shader=interp,point meta=explicit]
coordinates {
(0,0) [0] % first corner
(1,-1) [0] % Bezier control point between (0) and (3)
(4,0.7) [0] % Bezier control point between (0) and (3)
%
(3,2) [1] % second corner
(4,3.5) [1] % Bezier control point between (3) and (6)
(7,2) [1] % Bezier control point between (3) and (6)
%
(7,1) [2] % third corner
(6,0.6) [2] % Bezier control point between (6) and (9)
(4.5,-0.5) [2] % Bezier control point between (6) and (9)
%
(5,-2) [3] % fourth corner
(4,-2.5) [3] % Bezier control point between (9) and (0)
(-1,-2) [3] % Bezier control point between (9) and (0)
};
\end{axis}
\end{tikzpicture}

```

The four cubic Bézier curves are *equivalent* to `curveto` paths of PGF, i.e. to a sequence of the form $\langle\langle\textit{corner 1}\rangle\rangle.. \textit{controls}(\langle\textit{control point A}\rangle) \text{ and } (\langle\textit{control point B}\rangle) .. \langle\langle\textit{corner 2}\rangle\rangle$. The interpolated shading is bilinear. More precisely, a bilinear shading in the unit cube $[0, 1]^2$ is initialised which is then mapped into the Coons patch such that the corners match. The color interpolation uses only the color data of the four corners, color values of intermediate control points are ignored for the shading (although their value will be respected for the upper and lower limit of color data). In contrast to the finite element patches, a Coons patch is inherently two-dimensional. While you can still use three-dimensional coordinates, PGFLOTS will draw the shading as you provide it, without checking for the depth information (as it does for the other `patch types`). In other words: depending on the current `view` angle, the shading might fold over itself in unexpected ways.

Even for two dimensions, Coons patches may fold over themselves. To determine which part is foreground and which part is background, the following rule applies: the four corner points (0), (3), (6), (9) are associated to the unit cube points $(u, v) = (0, 0), (0, 1), (1, 1)$ and $(1, 0)$, respectively. The edge between corner (3) and (6) (i.e. the one with $v = 1$) is foreground, the edge between (1) and (9) is background. Thus, large values of v are drawn on top of small values of v . If v is constant, large values of u are drawn on top of small values of u . Thus, reordering the patch vertices (choosing a different first vertex and/or reversing the sequence) allows to get different foreground/background configurations⁸.

Note that `patch type sampling` is unavailable for `patch type=coons` because the control points are no point evaluation of the same function.

The choice `tensor bezier` is similar to `patch type=coons`: it allows to define a bezier patch. However, it allows more freedom: it has 16 control points instead of the 12 of a `coons` patch. The four additional control points are situated in the center of each patch. This `patch type` generates `.pdf` shadings of type 7 (whereas `coons` patches are shadings of type 6). It has been added for reasons of completeness, although it has not been tested properly. Please refer to the specification of the `.pdf` format for details⁹. The choice `tensor bezier` is actually *the same* as `patch type=bicubic` – except that `bicubic` automatically respects the view depth (foreground/background) and is given in a different by means of function evaluations rather than control points.

Note that `patch type sampling` is unavailable for `patch type=tensor bezier` because the control points are no point evaluation of the same function.

The choice `polygon` expects polygons with a fixed number of vertices. This `patch type` requires the number of vertices as argument:

```
/pgfplots/vertex count=<count>
```

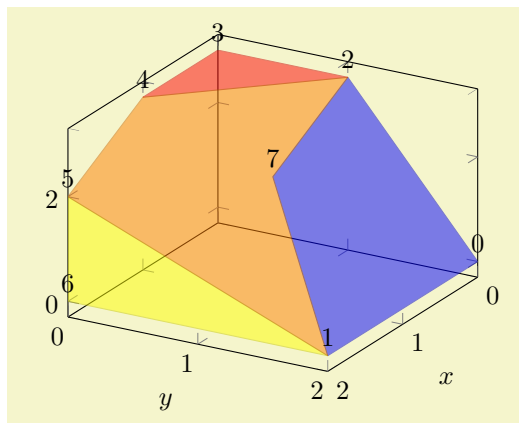
⁸Internally, PGFLOTS employs such mechanisms to map the higher order isoparametric patch types to Coons patches, sorting according their corner's depth information.

⁹If someone is willing to test it and document it, feel free to email me!

The number of vertices to be used for `patch type=polygon`. The number can be arbitrary. All input patches are expected to have this many vertices – but it is acceptable if a patch uses the same vertex multiple times. This means that `patch type=polygon` accepts polygons with different numbers of vertices, but you need to apply some sort of “manual padding”.

This parameter is (currently) mandatory.

A `patch` plot with `patch type=polygon` simply connects the $n=\text{vertex count}$ vertices in their order of appearance and closes the resulting path:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[view/h=120,xlabel=$x$,ylabel=$y$]
\addplot3[
opacity=0.5,
table/row sep=\\,
patch,
patch type=polygon,
vertex count=5,
patch table with point meta={%
% pt1 pt2 pt3 pt4 pt5 cdata
0 1 7 2 2 0\\
1 6 5 5 5 1\\
1 5 4 2 7 2\\
2 4 3 3 3 3\\
}]
table {
x y z\\
0 2 0\\% 0
2 2 0\\% 1
0 1 3\\% 2
0 0 3\\% 3
1 0 3\\% 4
2 0 2\\% 5
2 0 0\\% 6
1 1 2\\% 7
};
% replicate the vertex list to show \coordindex:
\addplot3[only marks,nodes near coords=\coordindex]
table[row sep=\\] {
0 2 0\\ 2 2 0\\ 0 1 3\\ 0 0 3\\
1 0 3\\ 2 0 2\\ 2 0 0\\ 1 1 2\\
};
\end{axis}
\end{tikzpicture}
```

The example above defines the `patch` by means of a connectivity table (`patch table with point meta`) and a vertex list (the normal input coordinates of the plot): there are 8 vertices and 4 polygons. Note that 2 of these polygons are triangles, one has 4 corners and only of them actually has all 5 allocated corners. This effect can be achieved by replicating one of the corners. The connectivity table in our example defines a unique color for each polygon: 0 for the first patch, 1 for the second, 2 for the third, and 3 for the last. These numbers map into the current `colormap`.

The `patch type=polygon` supports *neither* triangulation *nor* shading *nor* refinement. The order of appearance of the input points is supposed to be the order in which the line-to operations of the resulting path are generated.

5.6.2 Automatic Patch Refinement and Triangulation

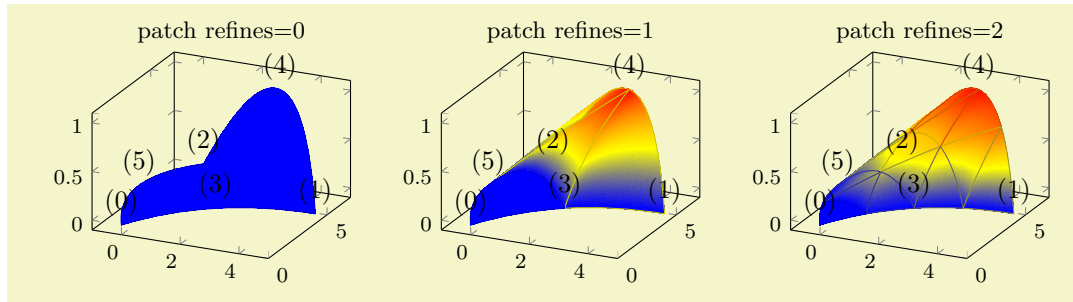
PGFPLOTS supports automatic patch refinement for most of its `patch types`. There are mainly two purposes for patch refinement: to increase the quality of `z buffer=sort` and/or to improve color interpolation for high-order patches.

`/pgfplots/patch refines={⟨levels⟩}` (initially 0)

This key controls patch refinement. The initial choice `patch refines=0` disables refinement and visualizes elements as they have been found in input files.

A positive `⟨levels⟩` enables (recursive) patch refinement: each patch is refined individually.

The following example illustrates the `patch refines` feature for a `triangle quadr` shape function on an edge. Note that since PGFLOTS uses only first order shading which is based on the corner points (0), (1) and (2), the specified shape function of `patch refines=0` has constant color. Higher `levels` approximate the patch with increasing quality:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\foreach \level in {0,1,2} {%
  \begin{tikzpicture}
    \begin{axis}[
      nodes near coords={(\coordindex)},
      footnotesize,
      title={patch refines=\level}]

      \addplot3[patch,patch type=triangle quadr,
        shader=faceted interp,patch refines=\level]
        coordinates {
          (0,0,0) (5,4,0) (0,7,0)
          (2,3,0) (3,6,1) (-1,4,0)
        };
    \end{axis}
  \end{tikzpicture}
}
```

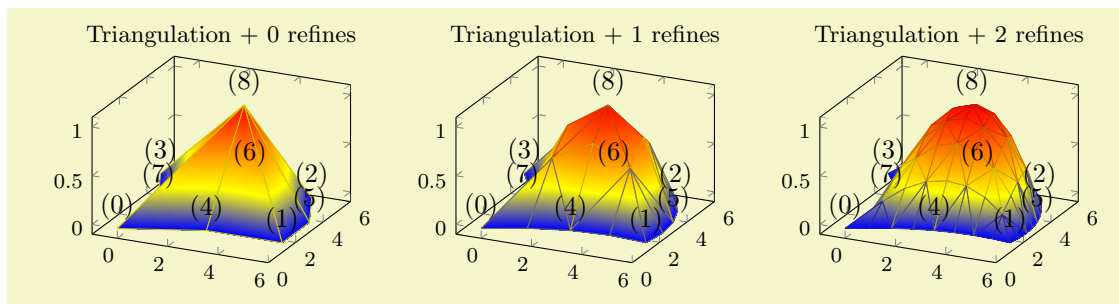
In this example, patch refinement makes a huge difference since it is just one element with huge displacements. For practical examples, you probably won't need many refinement levels.

The refined patches reproduce the geometry's shape exactly. In addition, they improve color interpolation. Note that its purpose is just visualization, therefore hanging nodes are allowed (and will be generated by `patch refine` for most `patch types`).

Patch refinement is implemented for all supported patches except for `patch type=coons`, `tensor bezier`, `bicubic` (might follow eventually) and `polygon`.

`/pgfplots/patch to triangles=true|false` (initially false)

Occasionally, one has a complicated `patch type` on input and would like to visualize it as a `triangle` mesh. PGFLOTS supports automatic triangulation of patches. Triangulation means to replace each individual input patch by one or more triangles. Triangulation can be combined with `patch refines` in which case `patch refines` is applied first and the resulting refined patches are then triangulated.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\foreach \level in {0,1,2} {%
  \begin{tikzpicture}
    \begin{axis}[
      nodes near coords={(\coordindex)},
      footnotesize,
      title={Triangulation + \level\ refines}]

      \addplot3[patch,patch type=biquadratic,shader=faceted interp,
        patch to triangles,patch refines=\level]
        coordinates {
          (0,0,0) (6,1,0) (5,5,0) (-1,5,0)
          (3,1,0) (6,3,0) (2,6,0) (0,3,0)
          (3,3.75,1)
        };
    \end{axis}
  \end{tikzpicture}%
}
```

For one-dimensional `patch` types like `quadratic spline`, `patch to triangles` results in approximation by means of `patch type=line` instead of `triangle`.

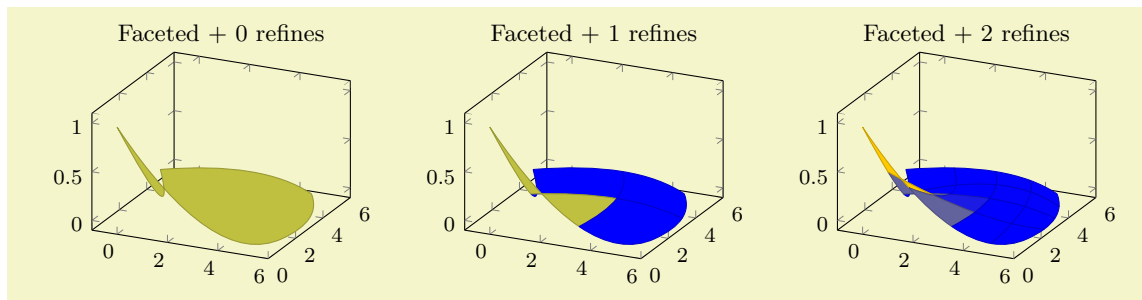
The `patch to triangles` feature is implemented for all supported patches except for `patch type=coons`, `tensor bezier`, and `polygon`.

5.6.3 Peculiarities of Flat Shading and High Order Patches

The `patchplots` library has been optimized for use with interpolated shadings, i.e. for `shader=interp`: it allows the filled area to fold over itself or to be outside of the patch boundaries.

PGFPLOTS also supports `shader=flat` and `shader=faceted` by simply stroking and/or filling the patch boundaries. Naturally, such an approach works only if the enclosed patch boundary and the filled area are essentially the same! Consider using `shader=flat` or `shader=faceted` only if the *mesh width is small enough* such that patches do not fold over themselves.

The following example illustrates the effect: the coarse single element on the left folds over itself, resulting in strange fill patterns. Refining the mesh reduces the effect.

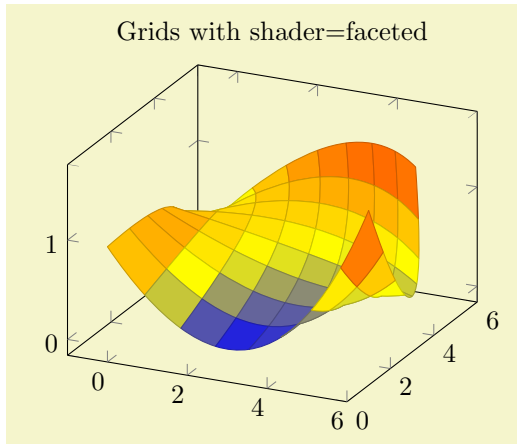


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\foreach \level in {0,1,2} {%
  \begin{tikzpicture}
    \begin{axis}[
      footnotesize,
      title={Faceted + \level\ refines}]

      \addplot3[patch,patch type=biquadratic,shader=faceted,
        patch refines=\level]
        coordinates {
          (0,0,1) (6,1,0) (5,5,0) (-1,5,0)
          (3,1,0) (6,3,0) (2,6,0) (0,3,0)
          (3,3.75,0)
        };
    \end{axis}
  \end{tikzpicture}
}
```


5.6.4 Drawing Grids

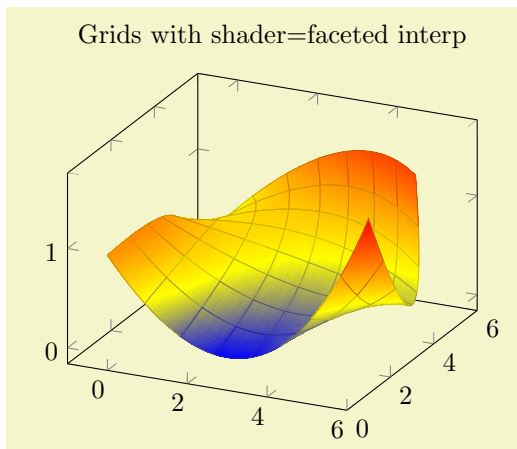
The `patchplots` library supports grid (`mesh`) visualization in the same way as for two/three-dimensional `mesh`- and `surf` plots. This includes four different approaches: the first is `shader=faceted`, which uses constant fill color and `faceted color` for stroke paths (as we already saw in Section 5.6.3). The second approach is to use `shader=faceted interp` which uses interpolated shadings for filling and issues stroke paths on top of each interpolated element. The third approach is to issue two `\addplot` commands, one with the filled `patch` plot, and one with a `patch,mesh` style which only draws (colored) grid lines on top of the previous plot. The three approaches are shown below.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title={Grids with shader=faceted}]

\addplot3[patch,patch type=biquadratic,
    shader=faceted,patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\end{axis}
\end{tikzpicture}
```

As already discussed in Section 5.6.3, the approach with `shader=faceted` works well if the mesh width is small enough (such that single patches do not overlap and their fill area is within the patch boundaries).

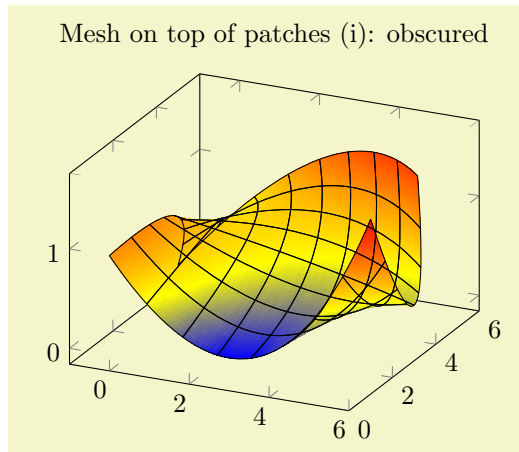


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title={Grids with shader=faceted interp}]

\addplot3[patch,patch type=biquadratic,
    shader=faceted interp,patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\end{axis}
\end{tikzpicture}
```

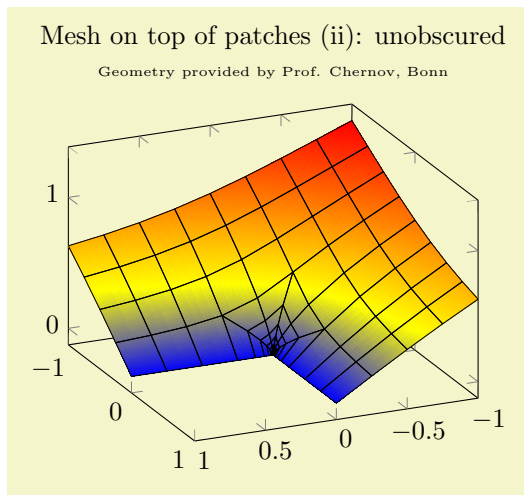
Here, grid lines are defined to be the patch boundary, so it may occasionally happen for coarse patches that grid lines cross the filled area. If you experience problems, consider using the `patch refines` key. The `shader=faceted interp` supports `z buffer` – at the cost of generating one shading for *each* patch element (the stroke path is drawn immediately after the patch element is shaded). This can become quite expensive¹⁰ at display time and may lead to huge pdf files. However, `shader=faceted interp` provides smooth shadings and, at the same time, good grid lines which are drawn in the correct order.

¹⁰I would really like to hear any well-founded ideas how to improve this issue. In case you have an idea– let me know!



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title={Mesh on top of patches (i): obscured}]

\addplot3[patch,patch type=biquadratic,shader=interp,
    patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\addplot3[patch,patch type=biquadratic,mesh,black,
    patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\end{axis}
\end{tikzpicture}
```

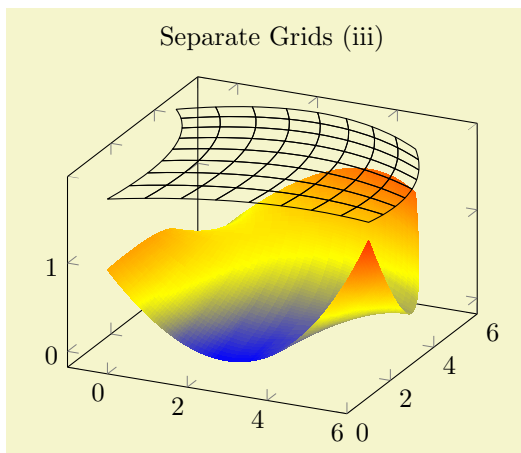


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title={Mesh on top of patches (ii): unobscured\\
    \tiny Geometry provided by Prof. Chernov, Bonn},
    title style={align=center},
    view={156}{28}]

\addplot3[patch,patch type=bilinear,
    shader=interp,
    patch table=plotdata/patchexample_conn.dat]
file {plotdata/patchexample_verts.dat};

\addplot3[patch,patch type=bilinear,
    mesh,black,
    patch table=plotdata/patchexample_conn.dat]
file {plotdata/patchexample_verts.dat};
\end{axis}
\end{tikzpicture}
```

The approach to draw grids separately is done by means of two `\addplot` statements; the first using `patch` as before, the second using `patch,mesh`. This configures PGFLOTS to visualize just the mesh. Make sure you provide ‘`mesh`’ after ‘`patch`’ since the latter activates filled `surf` visualization. The approach of meshes on top of patches implies to draw grid lines simply over any previous drawing operations. Thus, depth information is lost (as displayed in the first example above). Overlaying grid lines on top of the surface works in special cases (see bottom picture). An approach which always works is to provide the mesh at a fixed z position as displayed in the following example:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
    title={Separate Grids (iii)}

\addplot3[patch,patch type=biquadratic,shader=interp,
    patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\addplot3[patch,patch type=biquadratic,
    mesh,black,
    z filter/.code={\def\pgfmathresult{1.8}},
    patch refines=3]
coordinates {
    (0,0,1) (6,1,1.6) (5,5,1.3) (-1,5,0)
    (3,1,0) (6,3,0.4) (2,6,1.1) (0,3,0.9)
    (3,3.75,0.5)
};
\end{axis}
\end{tikzpicture}
```

Here, the first `\addplot3` command is the same as above, just with `shader=interp`. The second reproduces the same geometry, but uses a `z filter` to fix the z coordinate (in this case to $z = 1.8$). This effectively overrules all z coordinates.

Thus, grid lines can be drawn either by means of flat fill color with `shader=faceted` (efficient), by means of interpolated fill colors with `shader=faceted interp` (inefficient, see above) or, for special applications, using a separate `patch,mesh` plot which is drawn on top of the patches (efficient). In any case, the mesh visualization considers the `faceted color` which can depend on mapped color.

5.7 Polar Axes

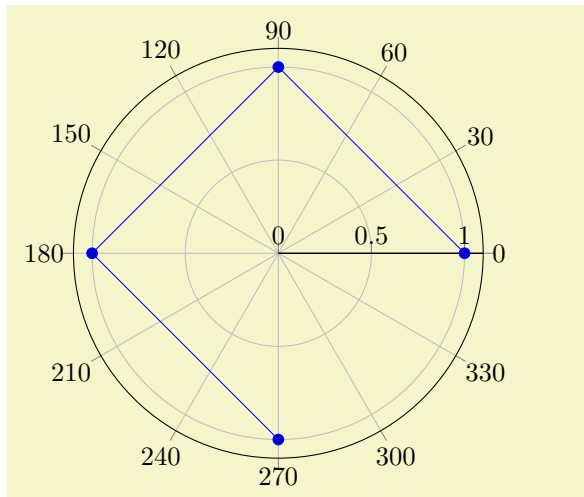
```
\usepgfplotslibrary{polar} %  $\TeX$  and plain  $\TeX$ 
\usepgfplotslibrary[polar] % Con $\TeX$ t
\usetikzlibrary{pgfplots.polar} %  $\TeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.polar] % Con $\TeX$ t
```

A library to draw polar axes and plot types relying on polar coordinates, represented by angle (in degrees or, optionally, in radians) and radius.

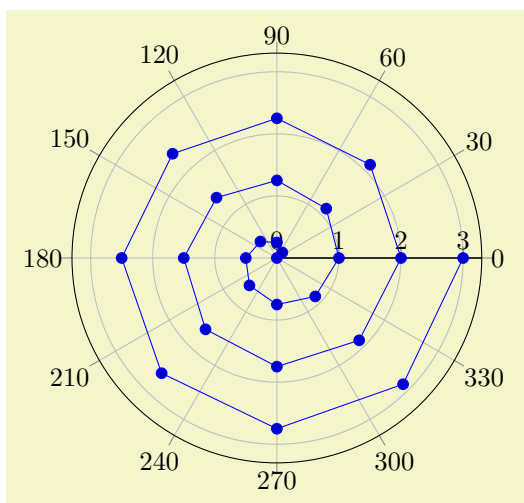
5.7.1 Polar Axes

```
\begin{polaraxis}
    <environment contents>
\end{polaraxis}
```

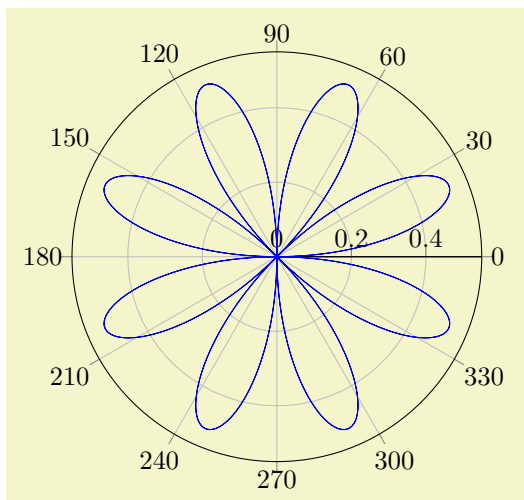
The `polar` library provides the `polaraxis` environment. Inside of such an environment, all coordinates are expected to be given in polar representation of the form $(\langle angle \rangle, \langle radius \rangle)$, i.e. the x coordinate is always the angle and the y coordinate the radius:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot coordinates {(0,1) (90,1)
      (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}
```

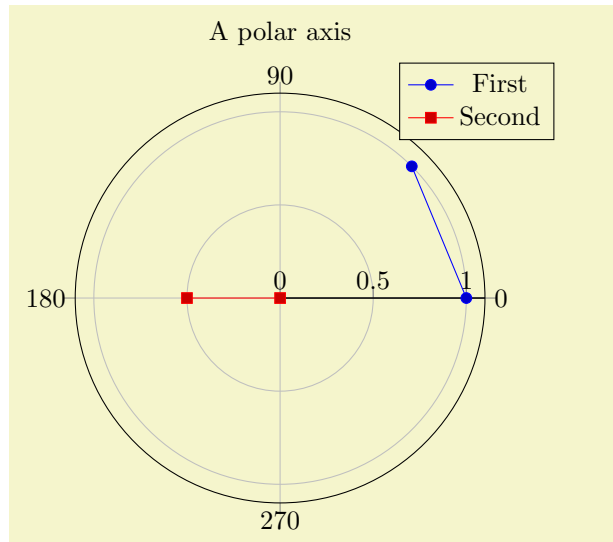


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot+[domain=0:3] (360*x,x); % (angle,radius)
  \end{polaraxis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot+[mark=none,domain=0:720,samples=600]
      {sin(2*x)*cos(2*x)};
    % equivalent to (x,{sin(..)cos(..)}), i.e.
    % the expression is the RADIUS
  \end{polaraxis}
\end{tikzpicture}
```

Polar axes support most of the PGFPLOTS user interface, i.e. [legend entries](#), any axis descriptions, [xtick/ytick](#) and so on:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[
    xtick={0,90,180,270},
    title=A polar axis]

    \addplot coordinates {(0,1) (45,1)};
    \addlegendentry{First}

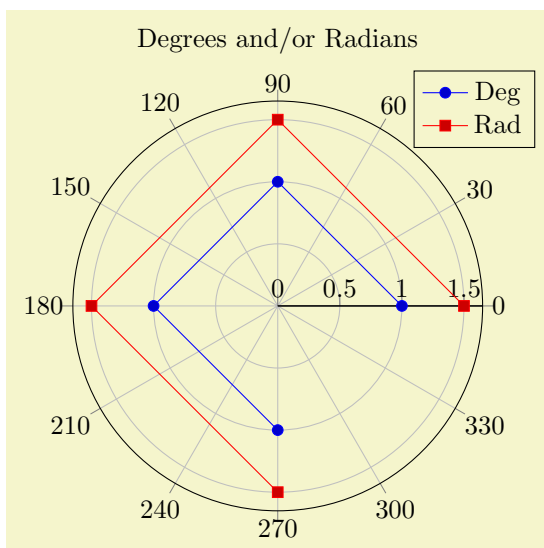
    \addplot coordinates {(180,0.5) (0,0)};
    \addlegendentry{Second}
  \end{polaraxis}
\end{tikzpicture}
```

Furthermore, you can use all of the supported input coordinate methods (like `\addplot coordinates`, `\addplot table`, `\addplot expression`). The only difference is that polar axes interpret the (first two) input coordinates as polar coordinates of the form $(\langle \text{angle in degrees} \rangle, \langle \text{radius} \rangle)$.

It is also possible to provide `\addplot3`; in this case, the third coordinate will be ignored (although it can be used as color data using `point meta=z`). An example can be found below in Section 5.7.3.

5.7.2 Using Radians instead of Degrees

The initial configuration uses degrees for the angle (x component of every input coordinate). PGFLOTS also supports to provide the angle in radians using the `data cs=polarrrad` switch:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[title={Degrees and/or Radians}]
    \addplot
      coordinates {(0,1) (90,1) (180,1) (270,1)};
    \addlegendentry{Deg}

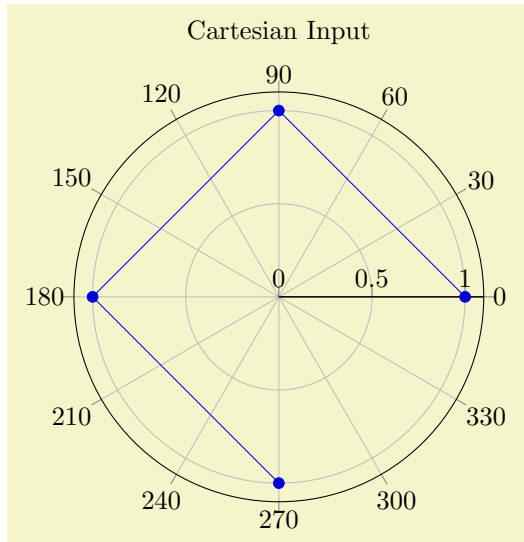
    \addplot+[data cs=polarrrad]
      coordinates {(0,1.5) (pi/2,1.5)
        (pi,1.5) (pi*3/2,1.5)};
    \addlegendentry{Rad}
  \end{polaraxis}
\end{tikzpicture}
```

The `data cs` key is described in all detail on page 303; it tells PGFLOTS the coordinate system of input data. PGFLOTS will then take steps to automatically transform each coordinate into the required coordinate

system (in our case, this is `data cs=polar`).

5.7.3 Mixing With Cartesian Coordinates

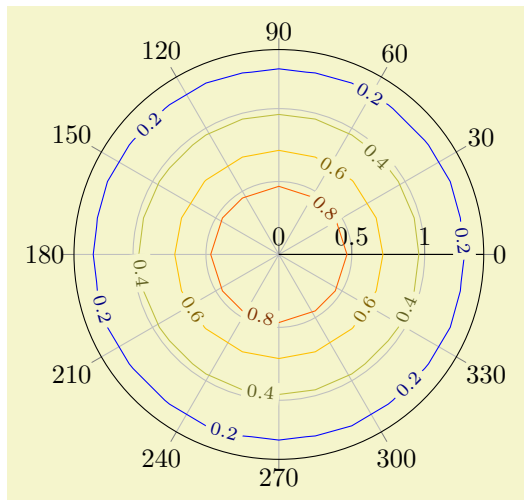
Similarly to the procedure described above, you can also provide Cartesian coordinates inside of a polar axis: simply tell PGFPLOTS that it should automatically transform them to polar representation by means of `data cs=cart`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[title=Cartesian Input]
    \addplot+[data cs=cart]
      coordinates {(1,0) (0,1) (-1,0) (0,-1)};
  \end{polaraxis}
\end{tikzpicture}
```

More details about the `data cs` key can be found on page 303.

This does also allow more involved visualization techniques which may operate on Cartesian coordinates. The following example uses `\addplot3` to sample a function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$, computes contour lines (with the help of `gnuplot`) and displays the result in a `polaraxis`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot3[contour gnuplot,domain=-3:3,
      data cs=cart]
      {exp(-x^2-y^2)};
  \end{polaraxis}
\end{tikzpicture}
```

What happens is that $z = \exp(-x^2 - y^2)$ is sampled for $x, y \in [-3, 3]$, then contour lines are computed on (x, y, z) , then the resulting triples (x, y, z) are transformed to polar coordinates (α, r, z) (leaving z intact). Finally, the z coordinate is used as `point meta` to determine the color.

Note that `\addplot3` allows to process three-dimensional input types, but the result will always be two-dimensional (the z coordinate is ignored for point placement in `polaraxis`). However, the z coordinate can be used to determine point colors (using `point meta=z`).

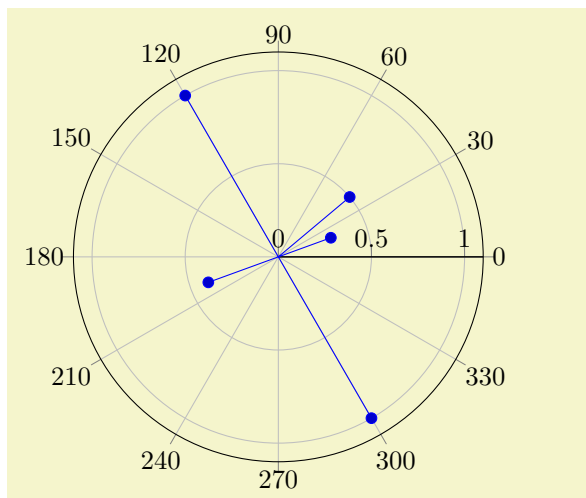
5.7.4 Special Polar Plot Types

`/tikz/polar comb`

(no value)

`\addplot+[polar comb]`

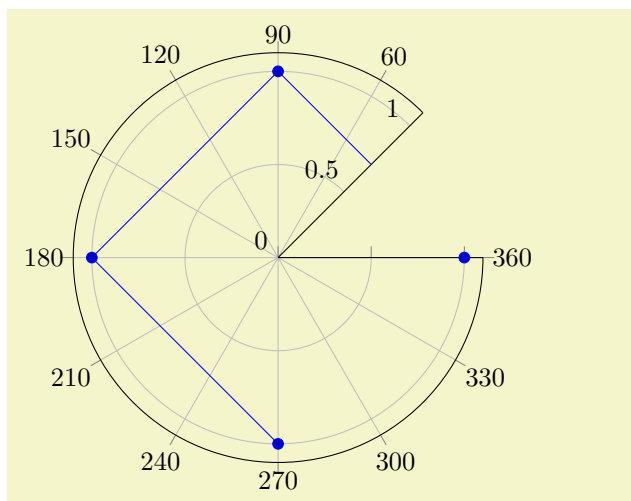
The `polar comb` plot handler is provided by TikZ; it draws paths from the origin to the designated position and places `marks` at the positions (similar to the `comb` plot handler). Since the paths always start at the origin, it is particularly suited for `polaraxis`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}
    \addplot+[polar comb]
      coordinates {(300,1) (20,0.3) (40,0.5)
        (120,1) (200,0.4)};
  \end{polaraxis}
\end{tikzpicture}
```

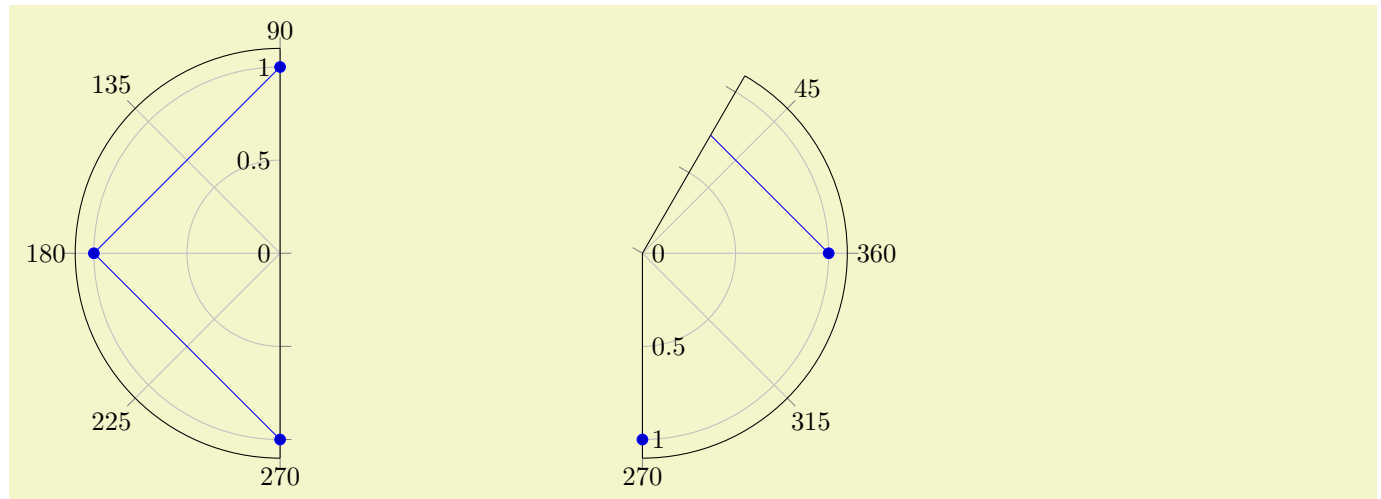
5.7.5 Partial Polar Axes

The `polar` library also supports partial axes. If you provide `xmin/xmax`, you can restrict the angles used for the axis:



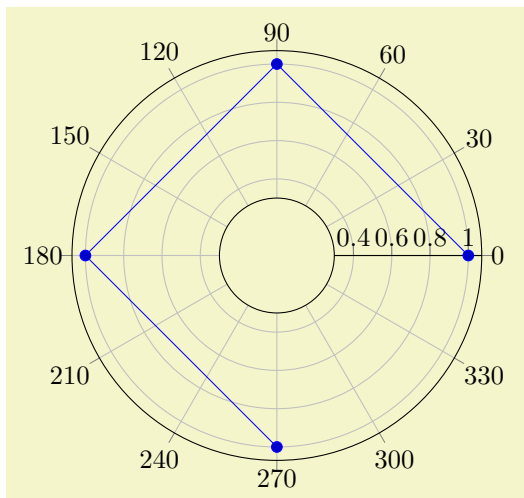
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[xmin=45,xmax=360]
    \addplot coordinates {(0,1) (90,1) (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}
```

Currently, the first angle must be lower than the second one. But you can employ the periodicity to get pies as follows:



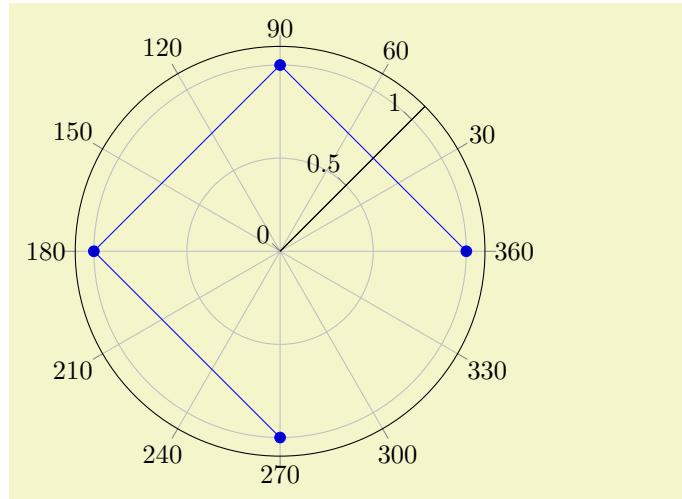
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[xmin=90,xmax=270]
    \addplot coordinates {(0,1) (90,1) (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}~%
\begin{tikzpicture}
  \begin{polaraxis}[xmin=270,xmax=420]
    \addplot coordinates {(0,1) (90,1) (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}
```

Similarly, an explicitly provided value for `ymin` allows to reduce the displayed range away from 0:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[ymin=0.3]
    \addplot coordinates {(0,1) (90,1)
      (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}
```

Modifying `xmin` and `xmax` manually can also be used to move the y axis line (the line with `ytick` and `yticklabels`):



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{polaraxis}[xmin=45,xmax=405]
    \addplot coordinates {(0,1) (90,1) (180,1) (270,1)};
  \end{polaraxis}
\end{tikzpicture}
```

5.8 Smith Charts

```
\usepgfplotslibrary{smithchart} %  $\LaTeX$  and plain  $\TeX$ 
\usepgfplotslibrary[smithchart] % Con $\TeX$ t
\usetikzlibrary{pgfplots.smithchart} %  $\LaTeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.smithchart] % Con $\TeX$ t
```

A library to draw Smith Charts.

A Smith Chart maps the complex half plane with positive real parts to the unit circle. The `smithchart` library allows PGFLOTS to visualize Smith Charts: it visualizes two-dimensional input coordinates $z \in \mathbb{C}$ of the form $z = x + jy \in \mathbb{C}$ (j being the imaginary unit, $j^2 = -1$) with $x \geq 0$ using the map

$$r: [0, \infty] \times [-\infty, \infty] \rightarrow \{a + jb \mid a^2 + b^2 = 1\}, \quad r(z) = \frac{z - 1}{z + 1}$$

using complex number division. The result is always in the unit circle.

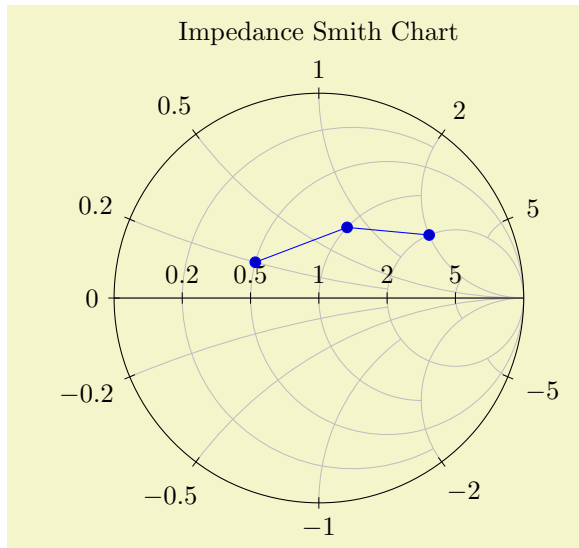
The main application for Smith Charts is in the area of electrical and electronics engineers specializing in radio frequency: to show the reflection coefficient $r(z)$ for normalised impedance z . It is beyond the scope of this manual to delve into the radio frequency techniques; for us, it is important to note that the `smithchart` library supports

- the data map $r(z)$ shown above,
- an axis class which interprets x as the real components and y as the imaginary components,
- a visualization of grid lines as arcs,
- the possibility to stop grid lines to allow uniform spacing in Smith Charts,
- a large set of the PGFLOTS axis fine tuning parameters,
- input of already mapped coordinates $r(z)$ (i.e. Cartesian coordinates in the unit circle),
- many of the PGFLOTS plot handlers.

5.8.1 Smith Chart Axes

```
\begin{smithchart}[\langle options \rangle]
  \langle environment contents \rangle
\end{smithchart}
```

The `\begin{smithchart}` environment draws Smith Charts. It accepts the same $\langle options \rangle$ as `\begin{axis}`. In fact, it is equivalent to `\begin{axis}[\langle options \rangle, axis type=smithchart]`.



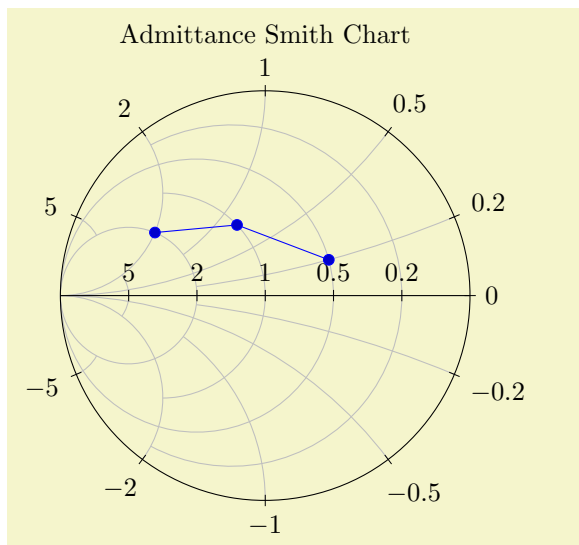
```
\begin{tikzpicture}
  \begin{smithchart}[title=Impedance Smith Chart]
    \addplot coordinates {(0.5,0.2) (1,0.8) (2,2)};
  \end{smithchart}
\end{tikzpicture}
```

The example above visualizes three data points using the initial configuration of Smith Charts; the data points are interpreted as complex numbers $z = x + jy$ and are mapped using $r(z)$.

Here, the x coordinate refers to the cycles described by the horizontal line whereas the y coordinate refers to the cycles described by the tick labels on the outside.

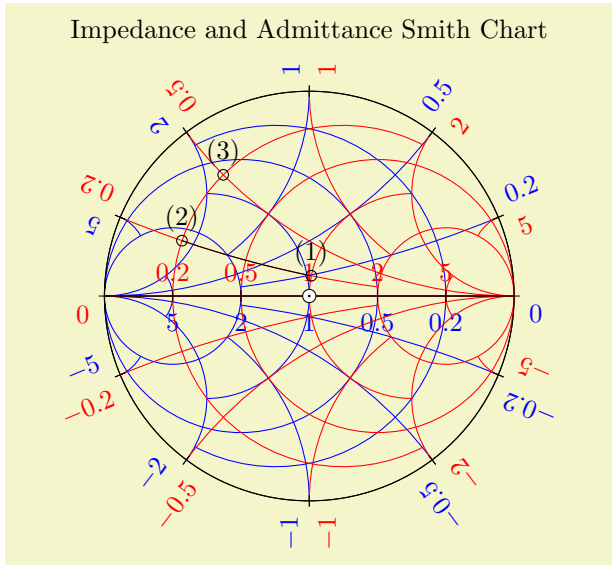
`/pgfplots/smithchart mirrored=true|false` (initially **false**)

PGFplots also supports Admittance Smith Charts. Here, the origin is on the right side of the circle:



```
\begin{tikzpicture}
  \begin{smithchart}[
    smithchart mirrored,
    title=Admittance Smith Chart]
    \addplot coordinates {(0.5,0.2) (1,0.8) (2,2)};
  \end{smithchart}
\end{tikzpicture}
```

Since PGFplots can draw two axes on top of each other, a combined Impedance/Admittance Smith Chart is also possible.



```
\begin{tikzpicture}[]
% First, the Admittance chart:
\begin{smithchart}[
  title=Impedance and Admittance Smith Chart,
  smithchart mirrored,
  xticklabel shift=-19pt,
  grid style={blue},
  ticklabel style={blue},
  yticklabel around circle,
]
\end{smithchart}

% Second, overlay the impedance chart:
\begin{smithchart}[
  show origin,
  grid style={red},
  ticklabel style={red},
  yticklabel around circle*,
]
\addplot+[black,mark=o,only marks,point meta=explicit symbolic,nodes near coords]
coordinates {
  (0.2,0.2) [(2)]
  (1,0.2) [(1)]
};

\addplot+[black,no marks,domain=0.2:1] {0.2};

\addplot+[black,mark=o,only marks,point meta=explicit symbolic,nodes near coords]
coordinates{
  (0.2,0.5) [(3)]
};
\end{smithchart}
\end{tikzpicture}
```

Since such a chart easily becomes crowded, it should be tuned manually by means of “suitable” appearance options (if you feel that there is a “suitable default”, let me know).

Details for `show origin`, `yticklabel around circle`, and `yticklabel around circle*` can be found later in this section.

5.8.2 Size Control

A Smith Chart can be resized by providing either `width` or `height` as argument to the axis. If you provide both, the Chart is drawn as an ellipsis.

The tick and grid positions for `smithchart` axes are realized by means of three manually tuned sets of grid lines: one for small-sized plots, one for medium-sized plots and one for huge plots. The actual parameters for `width` or `height` are considered to select one of the following sets:

`/pgfplots/few smithchart ticks`

(style, no value)

This produces the output of the example above – it constitutes the initial configuration for Smith Chart which has a width of less than 14cm.

The `few smithchart ticks` style is defined by:

```
\pgfplotsset{
  few smithchart ticks/.style={
    default smithchart xtick/.style={
      xtick={0.2,0.5,1,2,5},
    },
    default smithchart ytick/.style={
      ytick={%
        0,%
        0.2, 0.5, 1, 2, 5,%
        -0.2,-0.5,-1,-2,-5},
    },
    default smithchart xytick/.style={
      xgrid each nth passes y={2},
      ygrid each nth passes x={2},
    },
  },
}
```

Note that `few smithchart ticks` contains syntactical overhead to distinguish between “default ticks” and final tick positions: it does not assign `xtick` and `ytick` directly. Instead, it provides them as separate `default xtick` style arguments. The purpose of this distinction is to mark them as “default” arguments – the underlying styles `smithchart/every default xtick` is used if and only if there is no `xtick` value given.

In case you want to override this default, you can either

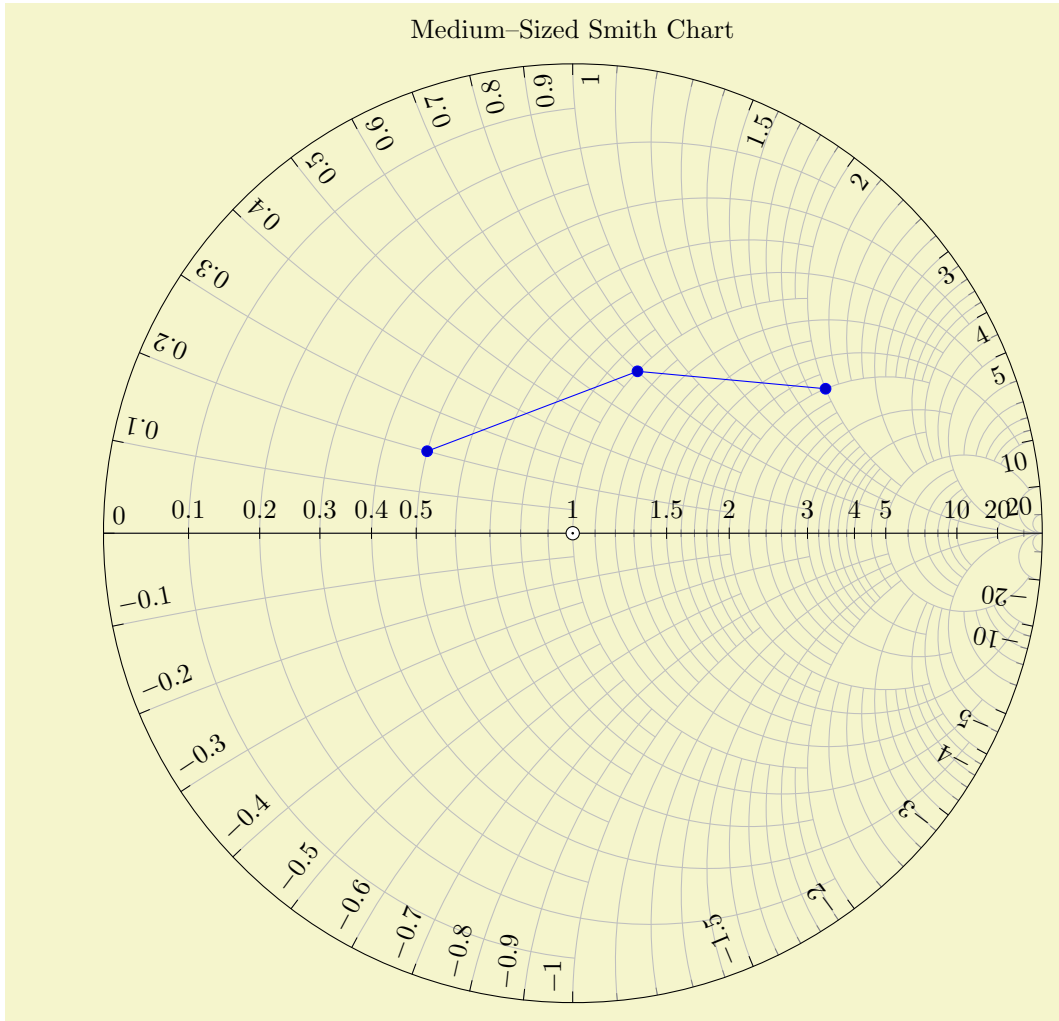
- copy–paste the definition above and adjust it or
- omit all the `default smithchart xtick/.style` stuff and write `xtick={⟨your list⟩}` directly.

As mentioned, the only purpose of the `default smithchart xtick/.style` overhead is to distinguish between `\begin{smithchart}[xtick={⟨user defined⟩}]` and default arguments (see the documentation of `default smithchart xtick/.style` for more about this technical detail).

For fine tuning of the scaling decisions, see the `smith chart ticks by size` key.

`/pgfplots/many smithchart ticks` (style, no value)

The `many smithchart ticks` style is used for every Smith Chart whose width exceeds 14cm although it is less than 20cm:



```
\begin{tikzpicture}
  \begin{smithchart}[
    title=Medium--Sized Smith Chart,
    width=14cm]
    \addplot coordinates {(0.5,0.2) (1,0.8) (2,2)};
  \end{smithchart}
\end{tikzpicture}
```

We see that `many smithchart ticks` has different placement and alignment options than `few smithchart ticks`: it uses sloped tick labels inside of the unit circle for the y descriptions (imaginary axis).

The initial configuration is realized by means of *two* separate styles: one which defines only the tick positions (the `many smithchart ticks*` style) and one which also changes placement and alignment options. The initial configuration can be changed individually (see the end of this section for examples). The initial configuration is:

```

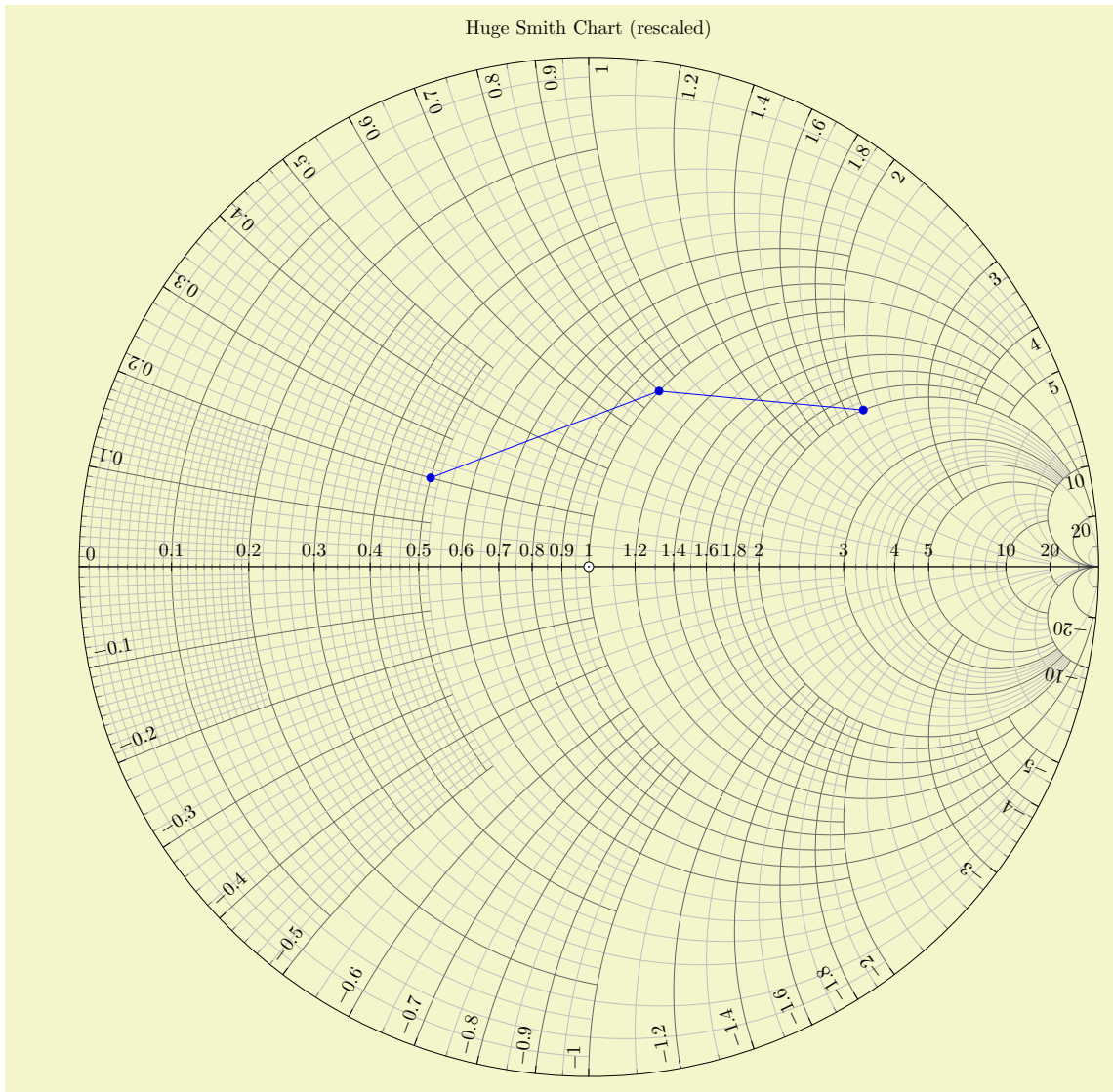
\pgfplotsset{
  many smithchart ticks*/.style={
    default smithchart xtick/.style={
      xtick={
        0.1,0.2,0.3,0.4,0.5,1,1.5,2,3,4,5,10,20%
      },
      minor xtick={0.6,0.7,0.8,0.9,1.1,1.2,1.3,1.4,1.6,1.7,1.8,1.9,
        2.2,2.4,2.6,2.8,3.2,3.4,3.6,3.8,4.5,6,7,8,9,50},
    },
    default smithchart ytick/.style={
      ytick={%
        0,%
        0.1,0.2,...,1,1.5,2,3,4,5,10,20,%
        -0.1,-0.2,...,-1,-1.5,-2,-3,-4,-5,-10,-20%
      },
      minor ytick={%
        1.1,1.2,1.3,1.4,1.6,1.7,1.8,1.9,2.2,2.4,2.6,2.8,3.2,3.4,3.6,3.8,
        4.5,6,7,8,9,50,%
        -1.1,-1.2,-1.3,-1.4,-1.6,-1.7,-1.8,-1.9,-2.2,-2.4,-2.6,-2.8,
        -3.2,-3.4,-3.6,-3.8,-4.5,-6,-7,-8,-9,-50%
      },
    },
    default smithchart xytick/.style={
      xgrid each nth passes y={1,2,4,5,10,20},
      ygrid each nth passes x={1,2,3,5,10:3,20:3},
    },
  },
  /pgfplots/many smithchart ticks/.style={
    many smithchart ticks*,
    yticklabel in circle,
    show origin=true,
  },
}

```

See the documentation for `few smithchart ticks` for an explanation of the `default smithchart xtick/.style` overhead.

`/pgfplots/dense smithchart ticks` (style, no value)

The `dense smithchart ticks` style assigns the set of tick positions for every Smith Chart whose width is at least 20cm:



```
\begin{tikzpicture}[scale=0.75]
  \begin{smithchart}[
    title=Huge Smith Chart (rescaled),
    width=20cm]
    \addplot coordinates {(0.5,0.2) (1,0.8) (2,2)};
  \end{smithchart}
\end{tikzpicture}
```

Attention: This style might change in future versions!

Similarly to `many smithchart ticks` (see above), the initial configuration is realized by means of *two* separate styles: one which defines only the tick positions (the `many smithchart ticks*` style) and one which also changes placement- and alignment options:

```

\pgfplotsset{
  dense smithchart ticks*/.style={
    default smithchart xtick/.style={
      xtick={
        0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1,1.2,1.4,1.6,1.8,2,3,4,5,10,20%
      },
      minor xtick={%
        0.01,0.02,0.03,0.04,0.05,0.06,0.07,0.08,0.09,0.11,0.12,0.13,0.14,0.15,0.16,0.17,
        0.18,0.19,0.22,0.24,0.26,0.28,0.32,0.34,0.36,0.38,0.42,0.44,0.46,0.48,%
        0.52,% This is sub-optimal and will (hopefully) be improved in the future.
        0.55,0.65,0.75,0.85,0.95,%
        % 0.6,0.7,0.8,0.9,%
        1.1,1.3,1.5,1.7,1.9,%
        2.2,2.4,2.6,2.8,3.2,3.4,3.6,3.8,4.5,6,7,8,9,50},
      },
    default smithchart ytick/.style={
      ytick={%
        0,%
        0.1,0.2,...,1,1.2,1.4,1.6,1.8,2,3,4,5,10,20,%
        -0.1,-0.2,...,-1,-1.2,-1.4,-1.6,-1.8,-2,-3,-4,-5,-10,-20%
      },
      minor ytick={%
        0.01,0.02,0.03,0.04,0.05,0.06,0.07,0.08,0.09,0.11,0.12,0.13,0.14,0.15,0.16,0.17,
        0.18,0.19,0.22,0.24,0.26,0.28,0.32,0.34,0.36,0.38,0.42,0.44,0.46,0.48,%
        0.55,0.65,0.75,0.85,0.95,%
        1.1,1.3,1.5,1.7,1.9,2.2,2.4,2.6,2.8,3.2,3.4,3.6,3.8,4.5,6,7,8,9,50,%
        -0.01,-0.02,-0.03,-0.04,-0.05,-0.06,-0.07,-0.08,-0.09,-0.11,-0.12,-0.13,-0.14,
        -0.15,-0.16,-0.17,-0.18,-0.19,-0.22,-0.24,-0.26,-0.28,-0.32,-0.34,-0.36,-0.38,
        -0.42,-0.44,-0.46,-0.48,-0.55,-0.65,-0.75,-0.85,-0.95,%
        -1.1,-1.3,-1.5,-1.7,-1.9,-2.2,-2.4,-2.6,-2.8,-3.2,-3.4,-3.6,-3.8,-4.5,-6,-7,-8,
        -9,-50%
      },
    },
    default smithchart xytick/.style={
      xgrid each nth passes y={0.2 if < 0.2001,0.5 if < 0.5001,1 if < 1.001,2,4,5,10,20},
      ygrid each nth passes x={0.2 if < 0.2001,0.52 if < 0.52001,1 if < 1.001,2,3,5,10:3,20:3},
    },
  },
  dense smithchart ticks/.style={
    yticklabel in circle,
    dense smithchart ticks*,
    show origin=true,
    every major grid/.style={black!60},
  },
}

```

See the documentation for `few smithchart ticks` for an explanation of the `default smithchart xtick/.style` overhead.

```

/pgfplots/default smithchart xtick (no value)
/pgfplots/default smithchart ytick (no value)
/pgfplots/default smithchart xytick (no value)

```

The `default smithchart xtick` style is installed if and only if you do not provide `xtick` manually.

Similarly, the `default smithchart ytick` style is installed if and only if you do not provide `ytick` manually.

Finally, the `default smithchart xytick` style is installed if and only if you provide neither `xtick` nor `ytick`.

These styles are usually defined in `few smithchart ticks` and its variants, see above.

5.8.3 Working with Prepared Data

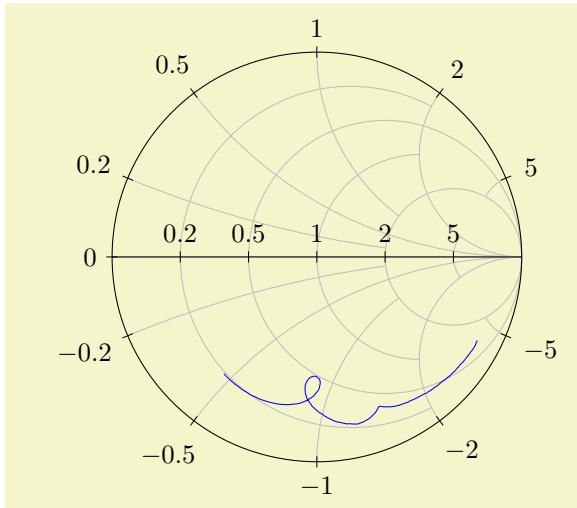
```

/pgfplots/is smithchart cs=true|false (initially false)

```

Occasionally, you may already have input data transformed into unit-circle Cartesian coordinate $r(z) = (x, y)$.

You can provide them to PGFLOTS with the `is smithchart cs` key:



```
\begin{tikzpicture}
  \begin{smithchart}
    % smithchart_data.dat contains
    % 0.78395 -0.40845
    % 0.78165 -0.41147
    % 0.77934 -0.41466
    % 0.77774 -0.41869
    % ...
    \addplot[blue,is smithchart cs]
      file {plotdata/smithchart_data.dat};
    \end{smithchart}
  \end{tikzpicture}
```

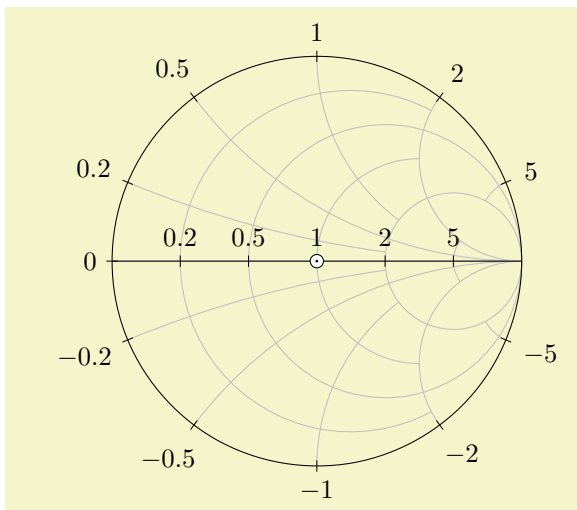
Using `is smithchart cs` tells PGFPLOTS to skip the transformation $r(z)$.

5.8.4 Appearance Control and Styles

`/pgfplots/show origin=true|false`

(initially false)

Allows to place an extra description at the point (0,0) to mark the origin.



```
\begin{tikzpicture}
  \begin{smithchart}[show origin]
  \end{smithchart}
\end{tikzpicture}
```

`/pgfplots/show origin code/.code={\langle...\rangle}`

Allows to redefine the code to draw the origin marker. The initial configuration is

```
\pgfplotsset{
  show origin code/.code={%
    \path[draw=black,fill=white] (0pt,0pt) circle (2.5pt);
    \path[fill=black] (0pt,0pt) circle (0.5pt);
  }
}
```

`/pgfplots/yticklabel in circle`

(style, no value)

This style draws Smith Chart tick labels for imaginary components (the `ytick` arguments) inside of the circle.

It installs transformations to rotate and shift tick labels. See the `many smithchart ticks` style for an example.

The initial configuration for this style is


```

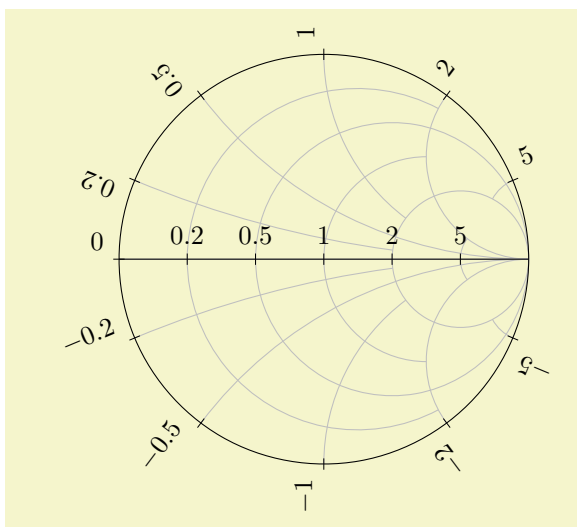
\pgfplotsset{
  yticklabel in circle/.style={
    ytick align=inside,
    yticklabel style={
      rotate=90,
      sloped like y axis={%
        execute for upside down={\tikzset{anchor=north east}},
        % allow upside down,
        reset nontranslations=false},
      anchor=south west,
      % font=\tiny,
    }
  }
}

```

/pgfplots/**yticklabel around circle**

(style, no value)

This style draws Smith Chart tick labels for imaginary components (the **ytick** arguments) outside of the circle, but rotated to fit the slope of the circle.



```

\begin{tikzpicture}
  \begin{smithchart}[yticklabel around circle]
    \end{smithchart}
\end{tikzpicture}

```

The initial configuration for this style is

```

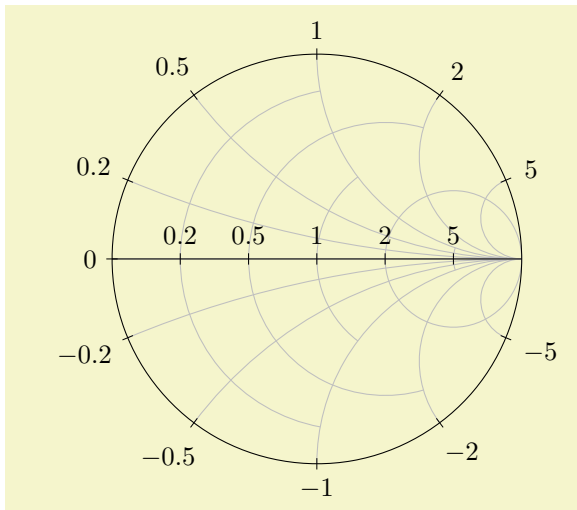
\pgfplotsset{
  yticklabel around circle/.style={
    ytick align=center,
    yticklabel style={
      rotate=90,
      sloped like y axis={%
        execute for upside down={\tikzset{anchor=south west}},
        % allow upside down,
        reset nontranslations=false},
      anchor=south east,
    }
  },
}

```

/pgfplots/**yticklabel around circle***

(style, no value)

A variant of **yticklabel around circle** which exchanges the anchors:



```
\begin{tikzpicture}
  \begin{smithchart}[
    xtick={0.2,0.5,1,2,5},
    ytick={
      0,
      0.2, 0.5, 1, 2, 5,
      -0.2,-0.5,-1,-2,-5},
    xgrid each nth passes y={1,2},
  ]
  \end{smithchart}
\end{tikzpicture}
```

The example overwrites the default `smithchart` ticks to define a new layout: now, every `ytick` uses the complete arc, but some of the grid lines for `xtick` stop at $y = 1$ and, if they pass, they may stop at $y = 2$.

The argument *list of stop entries* is a comma-separated list of entries. Each entry is, in the simplest case, a y coordinate (it should be a coordinate which appears in the `ytick` list). This simplest case means “only each second x grid line may pass the grid line for this y ”. The second syntax allows to provide a natural number, using $\langle y \text{ coord} \rangle : \langle number \rangle$. This means to let only each $\langle number \rangle$ ’s x grid line pass the designated y grid line. The third syntax also allows to write `if <  $\langle x \text{ value} \rangle$` . It means the entry is considered only for x grid lines which are less than $\langle x \text{ value} \rangle$. To summarize: there are the three possible forms of entries

1. single y coordinates, for example `xgrid each nth passes y={1,2}` or
2. the same as above, followed by an integer, for example `xgrid each nth passes y={1:3,2:2}` or
3. an additional restriction clause like `xgrid each nth passes y={0.2 if <0.3}`.

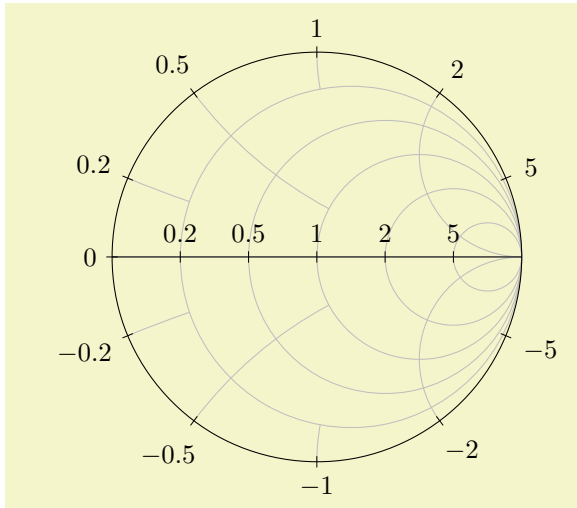
In this case, the all x grid lines which fulfill $x \leq 0.3$ will be checked if they are allowed to pass $y = 0.2$. All x grid lines with $x > 0.3$ are not affected by the constraint. See the `dense smithchart ticks` style for an application example.

Note that `xgrid each nth passes y` always employs symmetry; you do not need to provide y and $-y$ (if you want to, you may use the `xgrid stop at y` key to overrule the “each nth”-strategy).

In order to check if a given `xtick` argument is the “nth” grid line, PGFLOTS collects all `xtick` and `minor xtick` arguments into *one* large array and sorts it. Then, it uses the resulting sequence to assign the indices. Consequently, you can freely intermix minor and major ticks; it will still work. The only way to affect the counting is the `xgrid each nth passes y start` key, see below.

/pgfplots/`ygrid each nth passes x={list of stop entries}` (initially empty)

As you may already have guessed, this is the y counterpart of `xgrid each nth passes y`. It restricts the arcs for y grid lines by provided x ticks:



```
\begin{tikzpicture}
  \begin{smithchart}[
    xtick={0.2,0.5,1,2,5},
    ytick={
      0,
      0.2, 0.5, 1, 2, 5,
      -0.2,-0.5,-1,-2,-5},
    ygrid each nth passes x={0.2,1:2},
  ]
  \end{smithchart}
\end{tikzpicture}
```

The syntax is exactly the same as explained for `xgrid each nth passes y`. The only difference is that the `if <` syntax uses absolute values `y` (to maintain symmetry).

Now, we know how to use `xgrid each nth passes y` and the corresponding `ygrid each nth passes x` separately. Can we use both keys at the same time? Yes – but it may happen that lines end in white space! PGFPLOTS applies some logic to avoid arcs ending in white space by extending them to the next feasible stopping point. The result of mixing both of these keys is thus corrected automatically.

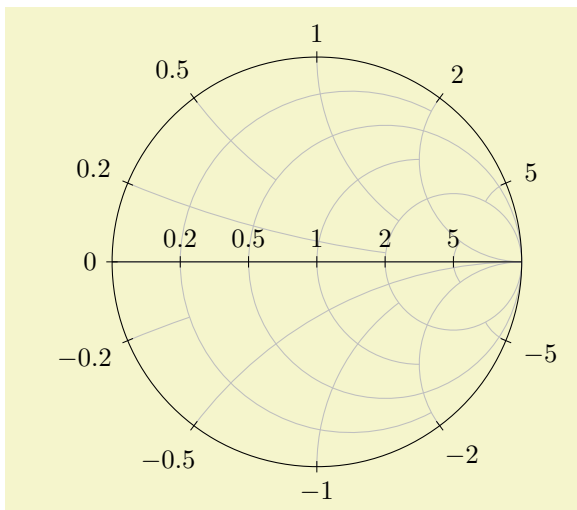
```
/pgfplots/xgrid each nth passes y start={⟨integer⟩} (initially 0)
/pgfplots/ygrid each nth passes x start={⟨integer⟩} (initially 0)
```

Allows to modify where the “each nth” counting starts. The argument can be considered as a shift. I consider this key to be more or less experimental – in the hope it may be useful. Try it out.

```
/pgfplots/xgrid stop at y={⟨list⟩} (initially empty)
/pgfplots/ygrid stop at x={⟨list⟩} (initially empty)
```

These keys allow to provide *individual* stop points for explicitly chosen tick positions. These explicit stop points have higher precedence over the each nth features described above.

The `ygrid stop at x` key accepts a comma-separated list of entries $\langle y \text{ coord} \rangle : \langle x \text{ stop point} \rangle$:



```
\begin{tikzpicture}
  \begin{smithchart}[
    ygrid stop at x={0.5:0.5,-0.2:0.2}
  ]
  \end{smithchart}
\end{tikzpicture}
```

In this example, the $y = 0.5$ arc stops at the $x = 0.5$ arc whereas the $y = -0.2$ arc stops at $x = 0.2$.

The `ygrid stop at x` key allows unsymmetric layouts (different stop points for y and $-y$).

5.9 Statistics

```
\usepgfplotslibrary{statistics} %  $\LaTeX$  and plain  $\TeX$ 
```

```
\usepgfplotslibrary[statistics] % ConTeXt
\usetikzlibrary{pgfplots.statistics} % LATEX and plain TEX
\usetikzlibrary[pgfplots.statistics] % ConTeXt
```

A library which provides plot handlers for statistics.

5.9.1 Box Plots

Box plots are visualizations for one-dimensional distributions. They provide a fast overview over characteristics of the distribution. Box plots are inherently one-dimensional; they only use a second axis to place multiple box plots next to each other.

PGFplots supports two related plot handlers: `boxplot` and `boxplot prepared`. The `boxplot` handler takes a one-dimensional sample as input, computes the `median`, the `lower quartile`, the `upper quartile`, the `lower whisker` and the `upper whisker`, and visualizes the result using the `boxplot prepared` handler. The `boxplot prepared` handler expects all required values on input and visualizes them.

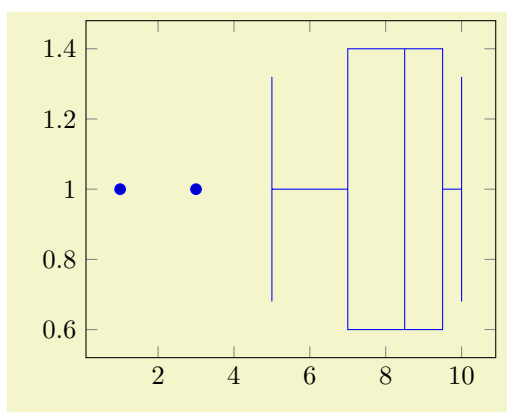
Prepared Box Plots and Common Options

The `boxplot prepared` handler is discussed first; all its customizations apply to `boxplot` as well.

```
/pgfplots/boxplot prepared={\options with boxplot/ prefix}
```

```
\addplot+[boxplot prepared={\options with boxplot/ prefix}]
```

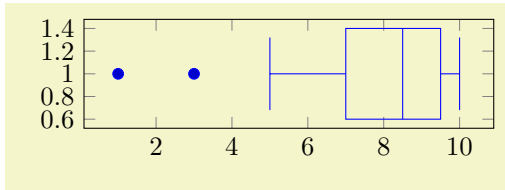
A `boxplot prepared` takes a couple of key-value pairs which describe the required statistics and a coordinate stream of outliers on input and draws a box plot.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[
    boxplot prepared={
        lower whisker=5,
        lower quartile=7,
        median=8.5,
        upper quartile=9.5,
        upper whisker=10,
    },
]
    table[row sep=\\,y index=0] {
        data\\ 1\\ 3\\
    };
\end{axis}
\end{tikzpicture}
```

The previous example shows the main idea of `boxplot prepared`: each required quantity has to be provided explicitly using a key-value syntax. The following coordinate stream can be empty; all coordinates inside of it are considered to be outliers. They are drawn as scatter plot.

A box plot produces two coordinates: one which belongs to the input data (for example `median`) and one which is only used for drawing purposes. Limits will be updated for both of them. While this is clear for the axis which shows the input data (x in this example, see also `draw direction`), it should be noted that limits and scaling parameters for the other axis will be chosen just as for any other plot. The box's extend is a little bit less than one unit by default (compare `box extend`). As a consequence, would might need to adjust either the limits or the scaling parameters for the remaining axis:



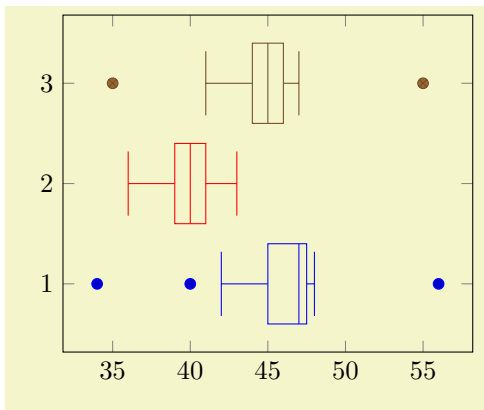
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
  y=1.5cm,
]
\addplot+[
  boxplot prepared={
    lower whisker=5,
    lower quartile=7,
    median=8.5,
    upper quartile=9.5,
    upper whisker=10,
  },
]
  table[row sep=\\,y index=0] {
    data\\ 1\\ 3\\
  };
\end{axis}
\end{tikzpicture}
```

If you place multiple blots with handler `boxplot prepared` into the same axis, they will automatically be placed next to each other by means of the default value of `draw position`:

`/pgfplots/boxplot/draw position={\langle axis unit to place box \rangle}` (initially `1+\plotnumofactualtype`)

The `draw position` key determines how to choose the position of the box plot, i.e. the axis unit which is free to choose.

The initial configuration places the first box plot at $y = 1$ and all following ones at the next integer numbers.



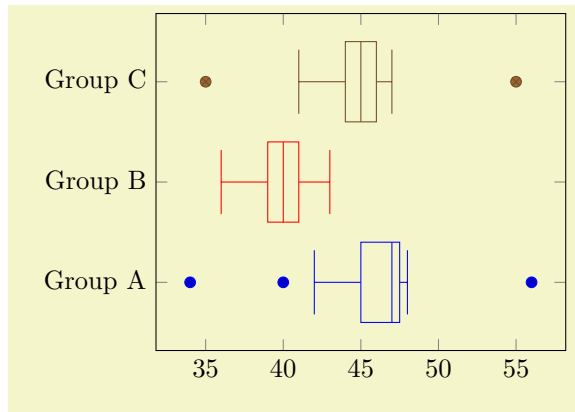
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
\addplot+[
  boxplot prepared={
    lower whisker=42, lower quartile=45,
    median=47,
    upper quartile=47.5, upper whisker=48,
  },
]
  table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };

\addplot+[
  boxplot prepared={
    lower whisker=36, lower quartile=39,
    median=40,
    upper quartile=41, upper whisker=43,
  },
]
  % no outliers:
  coordinates {};

\addplot+[
  boxplot prepared={
    lower whisker=41, lower quartile=44,
    median=45,
    upper quartile=46, upper whisker=47,
  },
]
  coordinates {(0,35) (0,55)};
\end{axis}
\end{tikzpicture}
```

The preceding example shows three box plots in the same axis. Note that they have been aligned using the default setting.

The fact that the first N `boxplot` (or the equivalent `boxplot prepared`) are placed at the coordinates $1, 2, 3, \dots, N$ makes it simple to assign tick labels: either use `ytick=data` or `ytick=1,2,3` combined with `yticklabels`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}
  ytick={1,2,3},
  yticklabels={Group A, Group B, Group C},
]
  \addplot+
    boxplot prepared={
      lower whisker=42, lower quartile=45,
      median=47,
      upper quartile=47.5, upper whisker=48,
    },
  ]
  table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };

  \addplot+
    boxplot prepared={
      lower whisker=36, lower quartile=39,
      median=40,
      upper quartile=41, upper whisker=43,
    },
  ]
  coordinates {};

  \addplot+
    boxplot prepared={
      lower whisker=41, lower quartile=44,
      median=45,
      upper quartile=46, upper whisker=47,
    },
  ]
  coordinates {(0,35) (0,55)};
\end{axis}
\end{tikzpicture}
```

The `draw position` may be read from some input table using `draw position=\thisrow{colname}`. In this case, the last encountered data row will be used (this remark is, of course, only useful if a data stream is present).

The preceeding examples read their outlier data streams from the y coordinate of the input streams: for `\addplot table`, we have explicitly said `y index=0` and for `\addplot coordinates`, we have used `(0,35)` `(0,55)` where the x components are ignored. This default can be changed using the `boxplot/data` key.

`/pgfplots/boxplot/data={\expression}` (initially `y`)

Tells `boxplot` how to get its data. The common idea is to provide a mathematical $\langle expression \rangle$ which depends on data supplied by the `\addplot` statement. For example, if you have `\addplot expression`, the $\langle expression \rangle$ may depend upon `x`, `y` or `z`. In case of an `\addplot table` input routine, the $\langle expression \rangle$ can employ `\thisrow{colname}` to access the currently active table row in the designated column.

It is also possible to avoid invocations of the math parser. Use `boxplot/data value={\value}` instead to do so. Here, $\langle value \rangle$ should be of a numeric constant.

The initial configuration employs what would usually become the final `y` coordinate as input (to be more precise, the initial value is `data value=\pgfkeysvalueof{/data point/y}`).

`/pgfplots/boxplot/lower whisker={\value}` (initially `auto`)
`/pgfplots/boxplot/lower quartile={\value}` (initially `auto`)
`/pgfplots/boxplot/median={\value}` (initially `auto`)
`/pgfplots/boxplot/upper quartile={\value}` (initially `auto`)
`/pgfplots/boxplot/upper whisker={\value}` (initially `auto`)
`/pgfplots/boxplot/average={\value}` (initially `empty`)

These keys constitute the supported statistics. Typically, a box plot uses each of them except for `average`.

Any numeric value for $\langle value \rangle$ will be used as-is. This holds for both `boxplot prepared` and `boxplot`. An empty $\langle value \rangle$ disables the respective key: its associated visualization will be omitted. This is the default for `average`.

The value `auto` tells PGFPLOTS to include the statistics in the automatic computation applied by `boxplot`. It is irrelevant for `boxplot prepared` (where it is essentially the same as an empty $\langle value \rangle$). The definition of the values is as follows. Assume that we have a given sample of a distribution, say $x_1, \dots, x_N$, and assume that the values are sorted, $x_1 < \dots < x_N$ (which is not a requirement for `boxplot`, by the way). For any real number p with $0 \leq p \leq 1$, the “ p -quantile” (or p -percentage) is defined as

$$x_p := \begin{cases} x_{N \cdot p} & \text{if } N \cdot p \text{ is an integer number} \\ \frac{1}{2}(x_{\lfloor N \cdot p \rfloor} + x_{\lceil N \cdot p \rceil}) & \text{if } N \cdot p \text{ is not an integer.} \end{cases}$$

`median` is the 0.5-quantile of the input data: half of the points are less and half of the points are larger than the median.

`lower quartile` is the 0.25-quantile of the input data.

`upper quartile` is the 0.75-quantile of the input data.

`lower whisker` is the smallest data value which is larger than `lower quartile`– $1.5 \cdot \text{IQR}$ where IQR is the “inter-quartile-range”, i.e. the difference between `upper quartile` and `lower quartile`.

`upper whisker` is the largest data value which is smaller than `upper quartile`+ $1.5 \cdot \text{IQR}$.

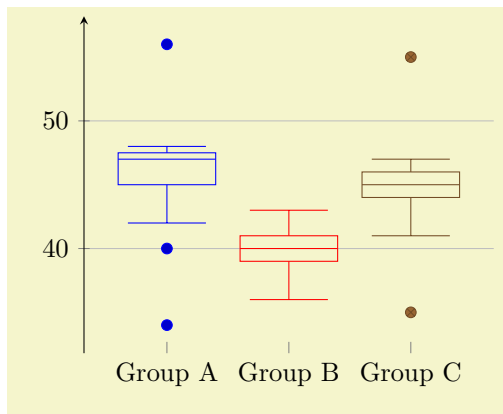
`average` is the sample average. It is omitted by `boxplot` in its default configuration. Set it to `auto` to enable its auto-computation.

`/pgfplots/boxplot/draw direction=x|y` (initially `x`)

Since `boxplot` is inherently one-dimensional, it can be visualized along the x or the y axis.

The default configuration uses the x direction as seen above.

The alternative choice y lets the boxes and their whiskers extend along the y axis and stacks multiple box plots along the x axis:



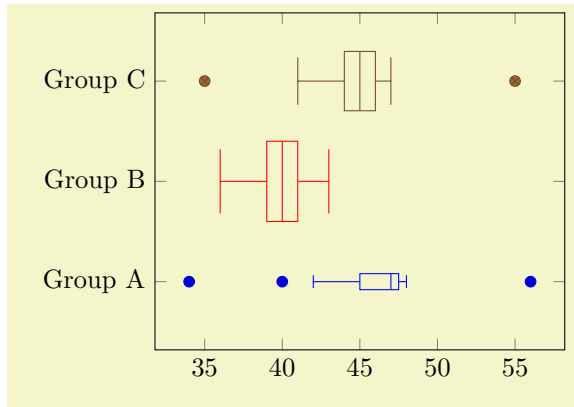
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  boxplot/draw direction=y,
  x axis line style={opacity=0},
  axis x line*=bottom,
  axis y line=left,
  enlarge y limits,
  ymajorgrids,
  xtick={1,2,3},
  xticklabels={Group A, Group B, Group C},
]
\addplot+[
  boxplot prepared={
    lower whisker=42, lower quartile=45,
    median=47,
    upper quartile=47.5, upper whisker=48,
  },
]
table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };
\addplot+[
  boxplot prepared={
    lower whisker=36, lower quartile=39,
    median=40,
    upper quartile=41, upper whisker=43,
  },
]
coordinates {};
\addplot+[
  boxplot prepared={
    lower whisker=41, lower quartile=44,
    median=45,
    upper quartile=46, upper whisker=47,
  },
]
coordinates {(0,35) (0,55)};
\end{axis}
\end{tikzpicture}
```


/pgfplots/boxplot/**variable width**=true|false

(initially false)

If enabled, the **box extend** will be scaled according to the **sample size** relative to all other **boxplot** or **boxplot prepared** within the same axis.

The key **variable width** only has an effect if **sample size** is available. For **boxplot prepared**, one needs to provide **sample size** explicitly.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ytick={1,2,3},
  yticklabels={Group A, Group B, Group C},
  boxplot/variable width,
]
\addplot+[% Group A:
  boxplot prepared={
    lower whisker=42, lower quartile=45,
    median=47,
    upper quartile=47.5, upper whisker=48,
    sample size=1000,
  },
]
table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };

\addplot+[% Group B:
  boxplot prepared={
    lower whisker=36, lower quartile=39,
    median=40,
    upper quartile=41, upper whisker=43,
    sample size=100000,
  },
]
coordinates {};

\addplot+[% Group C:
  boxplot prepared={
    lower whisker=41, lower quartile=44,
    median=45,
    upper quartile=46, upper whisker=47,
    sample size=50000,
  },
]
coordinates {(0,35) (0,55)};
\end{axis}
\end{tikzpicture}
```

The **variable width** computation computes the largest and smallest value of **sample size**, chosen among all box plots in the same axis. If a single plot has no **sample size**, it will be omitted from the computation (and it will not be scaled). If a single box plot has **variable width=false**, its **sample size** will not contribute either. The box plot with largest value of **sample size** will be drawn with 100% of **box extend**. The box plot with smallest value will be drawn with **variable width min target** times **box extend** as size (i.e. it will receive the smallest configured size). All box plots in-between are scaled linearly.

Note that the previous paragraph is not entirely true: **sample size** is only indirectly related to the scaling factor. Instead, the **variable width expr** is evaluated with the **sample size** as argument (see below for details).

/pgfplots/boxplot/**sample size**=<number>

(initially auto)

The number of samples used to derive the statistics. This number is used if **variable width=true**.

The value **auto** means to “use it whenever it can be acquired somewhere”. For a **boxplot**, it means that the size of the input sample is taken as-is. For a **boxplot prepared**, it means that the data is unavailable.

The empty string means that the value is unavailable.

Otherwise, a number is expected.

/pgfplots/boxplot/**variable width expr**=<math expression>

(initially $\sqrt{\#1}$)

A math expression which is used to evaluate the scaling factors of `variable width`. The argument is the current value of `sample size`. This key is used to implement common (nonlinear) transformations which are to be applied to the `sample size` before the result is used to scale down box sizes.

Typically, the argument should be a monotonically increasing function.

```
/pgfplots/boxplot/sample size min={\langle min sample size of group \rangle} (initially empty)
/pgfplots/boxplot/sample size max={\langle max sample size of group \rangle} (initially empty)
```

This is part of the `variable width` scaling: it is used to determine the `box extend` relative to all other box plots of the same group. It fixes the range.

```
/pgfplots/boxplot/variable width min target={\langle factor for the box width minimal size \rangle} (initially 0.2)
```

Used for the `variable width` feature to determine the size for the box plot with smallest value of `sample size`. The argument is interpreted to be a scaling factor in the range $[0, 1]$.

It is to be understood as percentage of `box extend`: a value of 1 means 100% of `box extend`. The initial configuration is 0.2, meaning 20% of `box extend`.

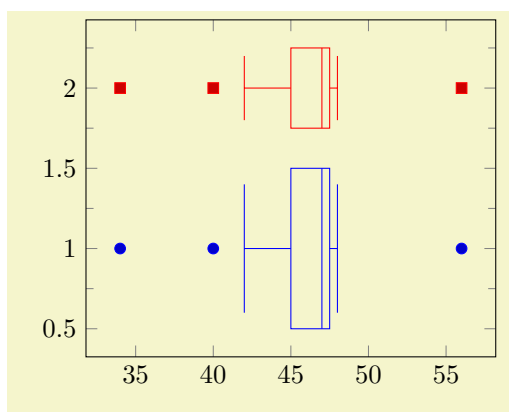
The box plot with largest value of `sample size` has 100% of `box extend`.

```
/pgfplots/boxplot/box extend={\langle axis unit for box extension \rangle} (initially 0.8)
```

A parameter which controls the size of the boxes with respect to the axis orthogonal to the data axis.

It is supposed to be the low-level size of a box plot, although it can be interpreted and used as a “low-level-variant” of `variable width`.

It is interpreted as coordinate in the axis, and it affects the `ymin` and `ymax` values for a `boxplot`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[minor y tick num=1]
\addplot+
boxplot prepared={
lower whisker=42, lower quartile=45,
median=47,
upper quartile=47.5, upper whisker=48,
box extend=1, }, ]
table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };

\addplot+
boxplot prepared={
lower whisker=42, lower quartile=45,
median=47,
upper quartile=47.5, upper whisker=48,
box extend=0.5, }, ]
table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };
\end{axis}
\end{tikzpicture}
```

The `box extend` controls the size of the box and the length of the median line. It also controls the size of whiskers, although they have a separate parameter `whisker extend`.

```
/pgfplots/boxplot/whisker extend={\langle axis unit for whisker extension \rangle} (initially \pgfkeysvalueof{/pgfplots/boxplot/box extend}*0.8)
```

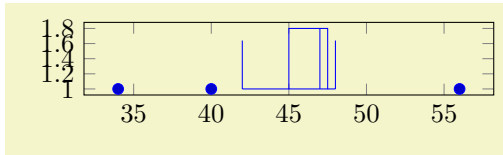
A parameter which configures how large whisker lines are with respect to the non-data axis.

It is used in the same way as `box extend`, and it also affects axis limits.

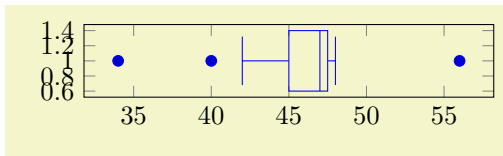
The initial configuration couples its value to `box extend` (it is 80% of `box extend`, to be more precise).

```
/pgfplots/boxplot/draw relative anchor={\langle number between 0 and 1 \rangle} (initially 0.5)
```

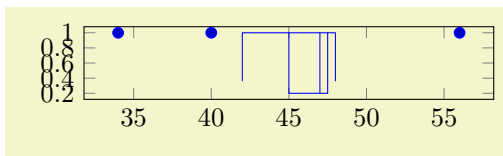
A key which customizes the anchor inside of the box where the whiskers are attached.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1cm]
\addplot+
    boxplot prepared={
        lower whisker=42, lower quartile=45,
        median=47,
        upper quartile=47.5, upper whisker=48,
        draw relative anchor=0, }, ]
    table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1cm]
\addplot+
    boxplot prepared={
        lower whisker=42, lower quartile=45,
        median=47,
        upper quartile=47.5, upper whisker=48,
        draw relative anchor=0.5, }, ]
    table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1cm]
\addplot+
    boxplot prepared={
        lower whisker=42, lower quartile=45,
        median=47,
        upper quartile=47.5, upper whisker=48,
        draw relative anchor=1, }, ]
    table[row sep=\\,y index=0] { 40\\ 34\\ 56\\ };
\end{axis}
\end{tikzpicture}
```

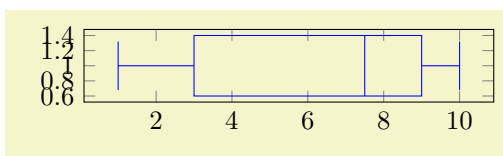
The value 0 means that whisker lines are attached to the bottom of the box (0% of `box extend`). The value 1 means that whisker lines are attached to the top edge of the box. Any value in-between is scaled linearly. The initial configuration is 0.5 which means that whiskers are attached to the middle of the box.

Analyzing Samples Automatically

`/pgfplots/boxplot={\langle options with boxplot/ prefix \rangle}`

`\addplot+[boxplot={\langle options with boxplot/ prefix \rangle}]`

The `boxplot` handler takes a one-dimensional sample as input, computes the `median`, the `lower quartile`, the `upper quartile`, the `lower whisker` and the `upper whisker`, and visualizes the result using the `boxplot prepared` handler.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1cm]
\addplot+[boxplot]
    table[row sep=\\,y index=0] {
        data\\
        1\\ 2\\ 1\\ 5\\ 4\\ 10\\
        7\\ 10\\ 9\\ 8\\ 9\\ 9\\
    };
\end{axis}
\end{tikzpicture}
```

The values do not need to be sorted. However, if they are sorted in ascending order, PGFLOTS might need less time to analyze them.

Data points can be given by means of any supported input stream, although the most useful ones are probably `\addplot table` and `\addplot coordinates`. In any case, `boxplot` acquires only one-dimensional data. To this end, it uses the current value of the `boxplot/data` key to see which input coordinate is to be used. In the default configuration, this is the y coordinate of the input stream. All other input items are ignored (except for `point meta`, which is handed down to the outlier stream).

`/pgfplots/boxplot/whisker range={\langle number \rangle}` (initially 1.5)

Defines how to determine `lower whisker` and `upper whisker`. In the default configuration, the lower whisker is placed at the smallest data point which is larger than `lower quartile` $-1.5 \cdot \text{IQR}$. The upper whisker is placed at the largest data point which is smaller than `upper quartile` $+1.5 \cdot \text{IQR}$. Here, IQR is the inter-quartile-range, defined as

$\text{IQR} := \text{upper quartile} - \text{lower quartile}$.

Everything outside of the whisker range is supposed to be an outlier.

Styles

`/pgfplots/boxplot/every boxplot` (style, no value)

A style which is immediately installed whenever `boxplot` or `boxplot prepared` are set.

The initial value is empty.

`/pgfplots/boxplot/every whisker` (style, no value)

A style which is installed whenever a whisker is drawn. It is empty initially.

`/pgfplots/boxplot/every box` (style, no value)

A style which is installed whenever a box is drawn. It is empty initially. Note that this does not apply to the path for the `median`.

`/pgfplots/boxplot/every median` (style, no value)

A style which is installed whenever a median is drawn. It is empty initially.

`/pgfplots/boxplot/every average` (style, no value)

A style which is installed whenever an average is drawn. The initial configuration is

```
\pgfplotsset{
  boxplot/every average/.style={%
    /tikz/mark=diamond*,
  },
}
```

Placing Annotations

`\pgfplotsboxplotvalue{\langle key name \rangle}`

Same as

`\pgfkeysvalueof{/pgfplots/boxplot/\langle key name \rangle}`.

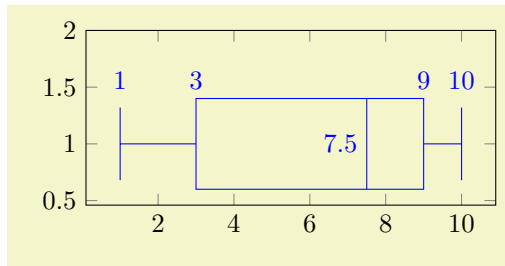
`\boxplotvalue{\langle key name \rangle}`

Same as `\pgfplotsboxplotvalue{\langle key name \rangle}` (just shorted).

Coordinate system `boxplot box`

The `boxplot box cs` accepts two arguments specified a tuple of the form `boxplot box cs=(\langle data coordinate, box-relative offset \rangle)` where the first is a value of the box plot's data (it is expressed in the same space as `median` or `upper whisker`).

The second argument is an offset expressed as signed multiple of `box extend`. An offset of 0 means to place the point exactly on the bottom line of the box. An offset of 1 places the point on the top line of the box. An offset of 0.5 places the point in the middle.



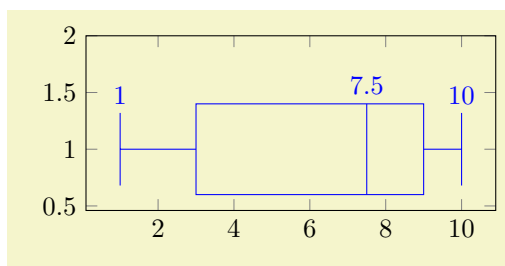
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1.5cm, ymax=2]
\addplot+[boxplot]
table[row sep=\\,y index=0] {
  data\\
  1\\ 2\\ 1\\ 5\\ 4\\ 10\\
  7\\ 10\\ 9\\ 8\\ 9\\ 9\\
}
[above]
node at
  (boxplot box cs: \boxplotvalue{lower whisker},1)
  {\pgfmathprintnumber{\boxplotvalue{lower whisker}}}
node at
  (boxplot box cs: \boxplotvalue{lower quartile},1)
  {\pgfmathprintnumber{\boxplotvalue{lower quartile}}}
node[left] at
  (boxplot box cs: \boxplotvalue{median},0.5)
  {\pgfmathprintnumber{\boxplotvalue{median}}}
node at
  (boxplot box cs: \boxplotvalue{upper quartile},1)
  {\pgfmathprintnumber{\boxplotvalue{upper quartile}}}
node at
  (boxplot box cs: \boxplotvalue{upper whisker},1)
  {\pgfmathprintnumber{\boxplotvalue{upper whisker}}}
;
\end{axis}
\end{tikzpicture}
```

Coordinate system `boxplot whisker`

A coordinate system which is almost the same as `boxplot box cs`, except that it aligns at `whisker extend` instead of `box extend`.

The `boxplot whisker cs` accepts two arguments of the form `boxplot whisker cs=(data coordinate, whisker-relative offset)` where the first is a value of the box plot's data (it is expressed in the same space as `median` or `upper whisker`).

The second argument is an offset expressed as signed multiple of `whisker extend`. An offset of 0 means to place the point exactly on the lower end of the whisker line. An offset of 1 places the point on the upper end of the whisker line. An offset of 0.5 places the point in the middle of the whisker line.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[y=1.5cm, ymax=2]
\addplot+[boxplot]
table[row sep=\\,y index=0] {
  data\\
  1\\ 2\\ 1\\ 5\\ 4\\ 10\\
  7\\ 10\\ 9\\ 8\\ 9\\ 9\\
}
[above]
node at
  (boxplot whisker cs:\boxplotvalue{lower whisker},1)
  {\pgfmathprintnumber{\boxplotvalue{lower whisker}}}
node at
  (boxplot box cs: \boxplotvalue{median},1)
  {\pgfmathprintnumber{\boxplotvalue{median}}}
node at
  (boxplot whisker cs:\boxplotvalue{upper whisker},1)
  {\pgfmathprintnumber{\boxplotvalue{upper whisker}}}
;
\end{axis}
\end{tikzpicture}
```

Customizing Visualization Paths

The following keys are of interest if you want to redefine the shape of a box, of a median, or of the whiskers.

Note that you should customize styles like `boxplot/every box` if you merely wish to change fill colors.

```
/pgfplots/boxplot/draw/lower whisker/.code={\dots}
```

```
/pgfplots/boxplot/draw/upper whisker/.code={\langle... \rangle}
/pgfplots/boxplot/draw/whisker/.code={\langle... \rangle}
```

A couple of code keys which customize the stroke paths for whiskers.

The initial configuration is

```
\pgfplotsset{
  boxplot/draw/lower whisker/.style={%
    /pgfplots/boxplot/draw/whisker=%
    {\pgfplotsboxplotvalue{lower quartile}}
    {\pgfplotsboxplotvalue{lower whisker}}
  },
  boxplot/draw/upper whisker/.style={%
    /pgfplots/boxplot/draw/whisker=%
    {\pgfplotsboxplotvalue{upper quartile}}
    {\pgfplotsboxplotvalue{upper whisker}}%
  },
  boxplot/draw/whisker/.code 2 args={%
    \draw[/pgfplots/boxplot/every whisker/.try]
      (boxplot cs:#1) -- (boxplot cs:#2)
      (boxplot whisker cs:#2,0)
      --
      (boxplot whisker cs:#2,1)
  },%
}
```

The key `draw/lower whisker` key is used if and only if `lower whisker` has a numeric value. The key `draw/upper whisker` is used if and only if `upper whisker` has a value.

If one of `lower quartile` or `upper quartile` is empty, both are replaced by the following values:

`lower quartile := upper whisker` and

`upper quartile := lower whisker`.

Thus, if the box cannot be drawn but you only have whiskers, the two whiskers will be connected with each other.

```
/pgfplots/boxplot/draw/box/.code={\langle... \rangle}
```

A path which is used for `every box`.

```
\pgfplotsset{
  boxplot/draw/box/.code={%
    \draw[/pgfplots/boxplot/every box/.try]
      (boxplot box cs:\pgfplotsboxplotvalue{lower quartile},0)
      rectangle
      (boxplot box cs:\pgfplotsboxplotvalue{upper quartile},1)
  },%
}
```

If either `lower quartile` or `upper quartile` is empty, this key will not be invoked.

Note that `draw/median` will be invoked after this key.

```
/pgfplots/boxplot/draw/median/.code={\langle... \rangle}
```

A path which is used for every `median`. Its initial configuration is

```
\pgfplotsset{
  boxplot/draw/median/.code={%
    \draw[/pgfplots/boxplot/every median/.try]
      (boxplot box cs:\pgfplotsboxplotvalue{median},0)
      --
      (boxplot box cs:\pgfplotsboxplotvalue{median},1)
  },%
}
```

This key will be omitted if `median` is empty.

```
/pgfplots/boxplot/draw/average/.code={\langle... \rangle}
```

The path which is used to visualize an `average`. The initial configuration is

```
\pgfplotsset{
  boxplot/draw/average/.code={%
    \draw[/pgfplots/boxplot/every average/.try]
      \pgfextra
      % do NOT use \draw[mark=] plot coordinates because
      % boxplots uses the same plot handler to draw its
      % outliers.
      \pgftransformshift{%
        % basic level access to 'boxplot box cs':
        \pgfplotsboxplotpointatbox
          {\pgfplotsboxplotvalue{average}}
          {0.5}%
      }%
      \pgfuseplotmark{\tikz@plot@mark}%
      \endpgfextra
  },
}
```

This key will be omitted if `average` has an empty value (the default).

The key `draw/average` will be evaluated after `draw/median` and after `draw/box`.

5.9.2 Histograms

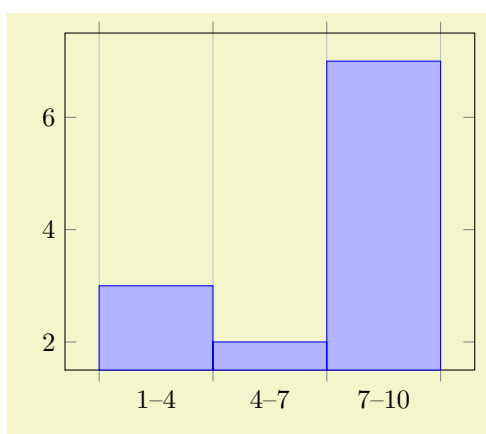
`/pgfplots/hist={⟨options with hist/ prefix⟩}`

`\addplot+[hist={⟨options with hist/ prefix⟩}]`

A histogram plot takes one-dimensional input data and counts the occurrence of values: it determines the data range $[\underline{m}, \overline{m}]$ and subdivides it into N equally sized bins with $(N + 1)$ end-points. Then, it counts the number of points falling into each bin. More precisely, it computes the $N + 1$ points $\underline{m} =: x_0 < x_1 < \dots < x_N := \overline{m}$ using $x_i := \underline{m} + i \cdot (\overline{m} - \underline{m})/N$. Then, it creates the $N + 1$ coordinates (x_i, y_i) , $i = 0, \dots, N - 1$ by means of

$$y_i := \begin{cases} \text{bincount}([x_i, x_{i+1})) & i < N \\ y_{N-1} & i = N, \end{cases}$$

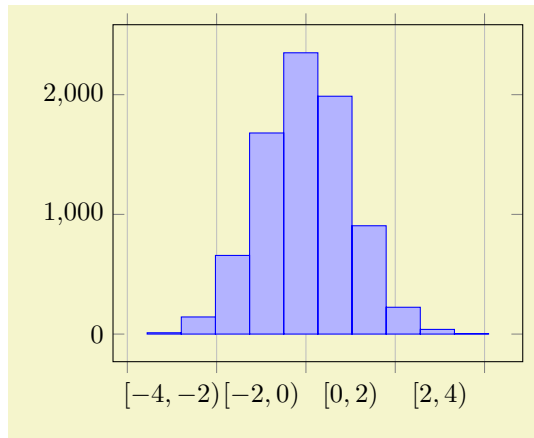
i.e. the value of the last coordinate is replicated. This set of $(N + 1)$ interval boundaries is then visualized by an `ybar interval` plot handler.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ybar interval,
  xticklabel=
\pgfmathprintnumber\tick--\pgfmathprintnumber\nexttick
]
  \addplot+[hist={bins=3}]
    table[row sep=\\,y index=0] {
      data\\
      1\\ 2\\ 1\\ 5\\ 4\\ 10\\
      7\\ 10\\ 9\\ 8\\ 9\\ 9\\
    };
\end{axis}
\end{tikzpicture}
```

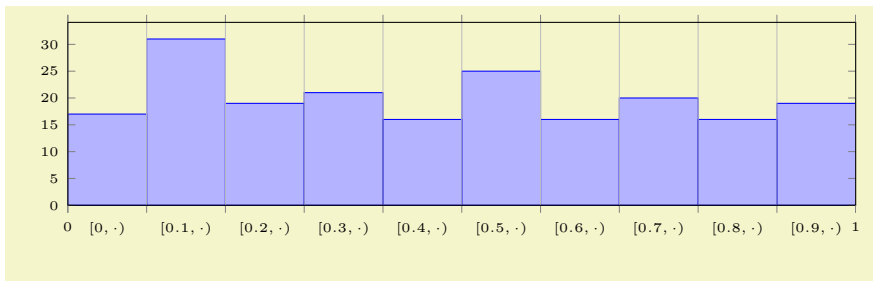
We see that `hist={bins=3}` takes a table with one column as input. The data values fall into the range $[1, 10]$ which is partitioned into 3 intervals (of equal lengths). Finally, the number of points falling into each of the three bins is plotted. The `xticklabel` key shows the range (note that it works only in conjunction with `x tick label as interval` which has been enabled by `ybar interval` before). We see that there are 3 elements in the range $[1, 4)$, 2 elements in the range $[4, 7)$ and finally 7 elements in the range $[7, 10]$.

The bins are half-open intervals, i.e. the end-point does not belong to the bin. Only the last bin contains its right end point.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ybar interval,
  xtick=,% reset from ybar interval
  xticklabel=
    {${\pgfmathprintnumber\tick,%
      \pgfmathprintnumber\nexttick}$}
]
% a data file containing 8000 normally distributed
% random numbers of mean 0 and variance 1
\addplot+[hist={data=x}]
  file {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}
```

The `hist` plot type can be combined with `plot expression` as well: provide the usual $\langle expression \rangle$ as you would for a line plot. Then, configure the value for `data= $\langle expression \rangle$` in dependence of `x`, `y`, or `z`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  tiny,
  height=4cm,width=12cm,
  ybar interval,
  ymin=0,
  xmin=0,xmax=1,
  axis on top,
  extra x ticks={0,1},
  extra x tick style={
    grid=none,
    x tick label as interval=false,
    xticklabel=${\pgfmathprintnumber\tick$}
  },
  xticklabel=${\pgfmathprintnumber[fixed]\tick,\cdot}$}
]
\addplot+[samples=200,hist] {rnd};
\end{axis}
\end{tikzpicture}
```

The example uses the `rnd` method of PGF which defines `y` to contain uniform random numbers in the range $[0, 1]$. Then, it configures `hist`. Note that `hist` has the default `data=y` such that it uses the `y` coordinate as input. Note furthermore that the `x` value is effectively ignored here. The options after `\begin{axis}[...]` are mainly to scale the graphics and to insert the right limits. The `extra x ticks` method is inserted to demonstrate how to add further tick marks without affecting the overall layout. Note that the `extra x tick style` sets `x tick label as interval=false` to disable the special tick handling which is active for the rest of the plot.

The following keys configure `hist`. If they are provided inside of $\langle options \rangle$, the common key prefix `hist/` can be omitted.

`/pgfplots/hist/data= $\langle expression \rangle$` (initially `y`)

Tells `hist` how to get its data. The common idea is to provide a mathematical $\langle expression \rangle$ which depends on data supplied by the `\addplot` statement. For example, if you have `\addplot`

expression, the $\langle expression \rangle$ may depend upon **x**, **y** or **z**. In case of an `\addplot table` input routine, the $\langle expression \rangle$ can employ `\thisrow{colname}` to access the currently active table row in the designated column.

It is also possible to avoid invocations of the math parser. Use `hist/data value={\langle value \rangle}` instead to do so. Here, $\langle value \rangle$ should be of a numeric constant.

The initial configuration employs what would usually become the final **y** coordinate as input (to be more precise, the initial value is `data value=\pgfkeysvalueof{/data point/y}`).

`/pgfplots/hist/data min={\langle min value \rangle}` (initially `/pgfplots/xmin`)
`/pgfplots/hist/data max={\langle max value \rangle}` (initially `/pgfplots/xmax`)

Allows to provide the min/max values (the $\underline{m}$ and $\overline{m}$) values manually.

If empty, these v (values will be deduced from the input data range.

The resulting interval will be splitted into `hist/bins` intervals.

The initial configuration uses any provided data limits, i.e. the (natural) choices `hist/data min=xmin` and `hist/data max=xmax`.

`/pgfplots/hist/bins={\langle number of intervals \rangle}` (initially 10)

Specifies the number of intervals to use.

`/pgfplots/hist/intervals={\langle true,false \rangle}` (initially `true`)

If `intervals=true` (the initial configuration), `hist` will generate $N + 1$ coordinates, with

$$\underline{m} = x_0 < x_1 < \dots < x_N = \overline{m}$$

where $[\underline{m}, \overline{m}]$ is the data range. In this case, the data points for x_{N-1} and x_N will get the same value, namely the number of elements in the last bin. This is (only) useful in conjunction with `const plot` or `ybar interval`.

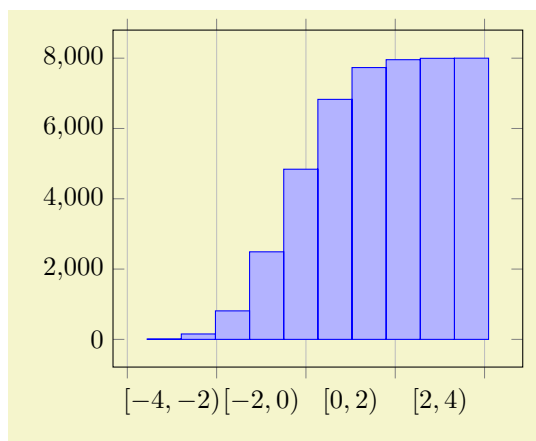
If `intervals=false`, the last data point will be omitted and exactly N coordinates will be generated. In this case, the right end point is not returned explicitly.

`/pgfplots/hist/cumulative={\langle true,false \rangle}` (initially `false`)

Allows to compute a cumulative histogram.

A cumulative histogram uses the sum of all previous bins and the current one as final value.

Here is the example from above, this time with `hist/cumulative`:

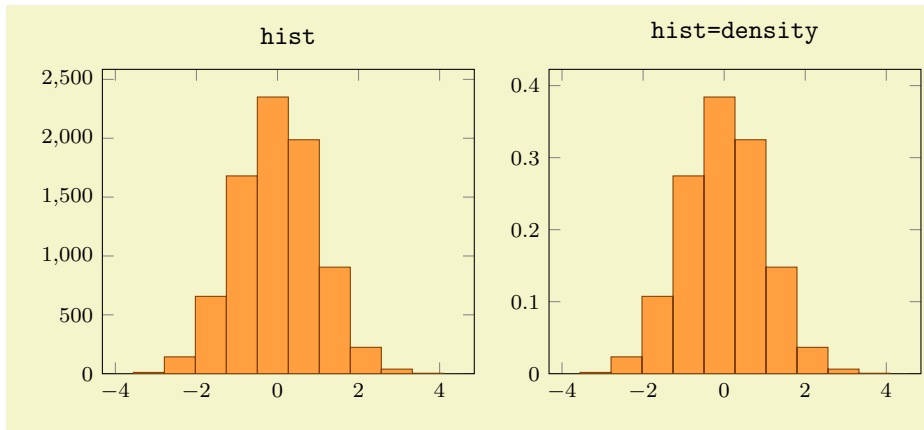


```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
ybar interval,
xtick=,% reset from ybar interval
xticklabel=
{${\pgfmathprintnumber\tick,
\pgfmathprintnumber\nexttick}$}
]]
% a data file containing 8000 normally distributed
% random numbers of mean 0 and variance 1
\addplot+[hist={
data=x,
cumulative}]
file {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}
```

`/pgfplots/hist/density={\langle true,false \rangle}` (initially `false`)

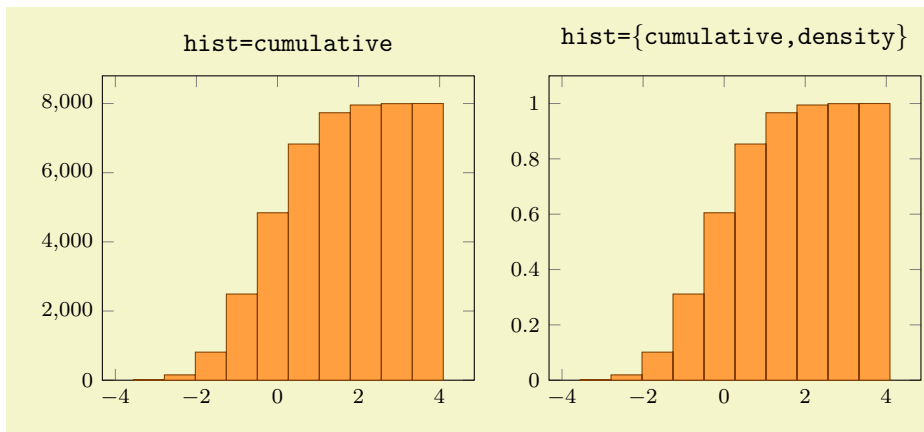
An extension by Jörnjakob Dugge

Enables density estimation mode. If `hist/density` is active, the resulting data points will be renormalized such that the overall “mass” equals 1.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[small,ymin=0,title=\texttt{hist}]
\addplot [
    hist,
    fill=orange!75,
    draw=orange!50!black
    table [y index=0] {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
\begin{axis}[small,ymin=0, title=\texttt{hist=density}]
\addplot [
    hist=density,
    fill=orange!75,
    draw=orange!50!black
    table [y index=0] {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}
```

The keys `hist/density` and `hist/cumulative` can be combined as well:



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[small,ymin=0, title=\texttt{hist=cumulative}]
\addplot [
    hist=cumulative,
    fill=orange!75,
    draw=orange!50!black]
    table [y index=0] {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}
%
\begin{tikzpicture}
\begin{axis}[small,ymin=0, title=\texttt{hist=\{cumulative,density\}}]
\addplot [
    hist=\{cumulative,density\},
    fill=orange!75,
    draw=orange!50!black]
    table [y index=0] {plotdata/pgfplots.randn.dat};
\end{axis}
\end{tikzpicture}

```

`/pgfplots/hist/handler` (style, initially `ybar interval`)

Allows to change the way the generated coordinates are visualized. The `hist/handler` key is a style, so use `hist/handler/.style={const plot}` to change it.

`/pgfplots/hist/data filter/.code={\langle...\rangle}`

Allows to define coordinate filters, similar to the coordinate filter key `x filter` described in Section 4.22. The argument `#1` is the coordinate as it has been found after processing `hist/data`. The code is supposed to assign `\pgfmathresult` to contain the result. If `\pgfmathresult` is empty afterwards, it will be skipped. Otherwise, it is supposed to contain a number.

This filter is applied *before* the histogram is computed. Note that `x filter` and `y filter` are applied *after* the histogram is computed.

Note that predefined styles like `each nth point` can also be applied to `hist/data` if

1. an asterisk ‘*’ is appended to the predefined style’s name and
2. the first argument to the style is `hist/data`.

For example, `each nth point*={hist/data}{2}` will skip each second input value of `hist/data` (try it out).

`/pgfplots/hist/data coord trafo/.code={\langle...\rangle}`

`/pgfplots/hist/data coord inv trafo/.code={\langle...\rangle}`

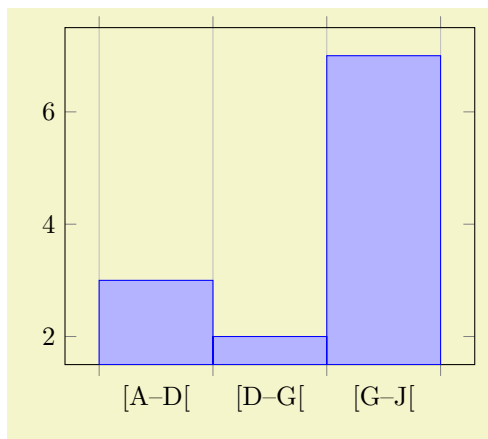
These keys work in the same way as for `x coord trafo` and `x coord inv trafo`. They are applied to the `hist/data` value before the histogram is evaluated and after the result value is assigned, respectively.

Note that `hist` will apply the `hist/data coord inv trafo` before it visualizes its results. Consequently, it may be necessary to assign a similar transformation to `x coord trafo` as well.

See the documentation of `x coord trafo` for more information about custom transformations.

`/pgfplots/hist/symbolic coords={\langle list\rangle}`

A style which enables `symbolic x coords` for an axis containing `hist` plots:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  ybar interval,
  hist/symbolic coords={A,B,C,D,E,F,G,H,I,J},
  xticklabel={[\tick--\nexttick[]],
]
  \addplot+[hist={bins=3}]
    table[row sep=\\,y index=0] {
      data\\
      A\\ B\\ A\\ D\\ F\\ J\\
      G\\ J\\ I\\ H\\ I\\ I\\
    };
\end{axis}
\end{tikzpicture}
```

The style does two things: first, it defines `hist/data coord trafo` and `hist/data coord inv trafo`, then, it calls `symbolic x coords` with the same argument.

Attention : do not use `hist/data=x` or other symbolic values as input when you have `symbolic coords`. Rather than symbolic values, you need to provide *expandable* values like `\pgfkeysvalueof{/data point/x}` (which has the same effect, but directly expands to the correct value).

Please refer to the documentation of `symbolic x coords` for further details about symbolic coordinates.

5.10 Ternary Diagrams

```
\usepgfplotslibrary{ternary} %  $\TeX$  and plain  $\TeX$ 
\usepgfplotslibrary[ternary] % Con $\TeX$ t
\usetikzlibrary{pgfplots.ternary} %  $\TeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.ternary] % Con $\TeX$ t
```

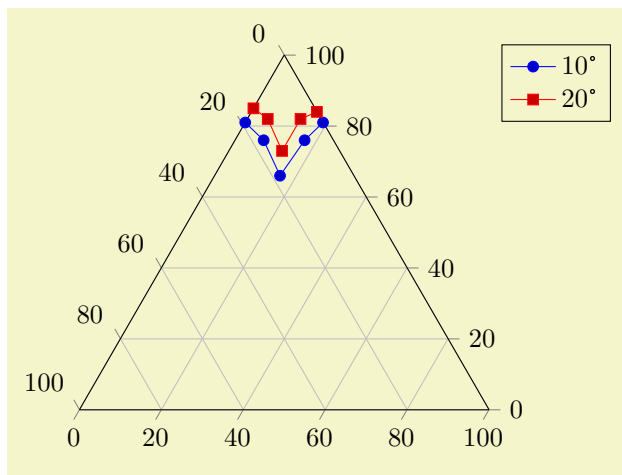
A library to draw ternary diagrams.

A ternary diagram visualizes three-component systems such that the sum of them yields 100%. Ternary diagrams are triangular axes.

5.10.1 Ternary Axis

```
\begin{ternaryaxis}[\langle options \rangle]
  \langle environment contents \rangle
\end{ternaryaxis}
```

The axis environment for ternary axes.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}
  \addplot3 coordinates {
    (0.81, 0.19, 0.00)
    (0.76, 0.17, 0.07)
    (0.66, 0.16, 0.16)
    (0.76, 0.07, 0.17)
    (0.81, 0.00, 0.19)
  };

  \addplot3 coordinates {
    (0.85, 0.15, 0.00)
    (0.82, 0.13, 0.05)
    (0.73, 0.14, 0.13)
    (0.82, 0.06, 0.13)
    (0.84, 0.00, 0.16)
  };

  \legend{$10^\circ$, $20^\circ$}
\end{ternaryaxis}
\end{tikzpicture}

```

A `ternaryaxis` works with *relative coordinates*: each data point consists of three components x, y, z . Their sum forms a compound entity which has 100% (of whatever). In the standard configuration, we have $x, y, z \in [0, 1]$. The unit interval is not necessary: you can as well choose *absolute data ranges* $x \in [x_{\min}, x_{\max}]$, $y \in [y_{\min}, y_{\max}]$ and $z \in [z_{\min}, z_{\max}]$. The important thing is that the relative values

$$\tilde{x} := \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad \tilde{y} := \frac{y - y_{\min}}{y_{\max} - y_{\min}}, \quad \tilde{z} := \frac{z - z_{\min}}{z_{\max} - z_{\min}}$$

sum up to 100%, i.e. $\tilde{x} + \tilde{y} + \tilde{z} = 1$. Thus, PGFPLOTS computes $\tilde{x}$, $\tilde{y}$ and $\tilde{z}$ and interpretes them as barycentric (triangular) coordinates.

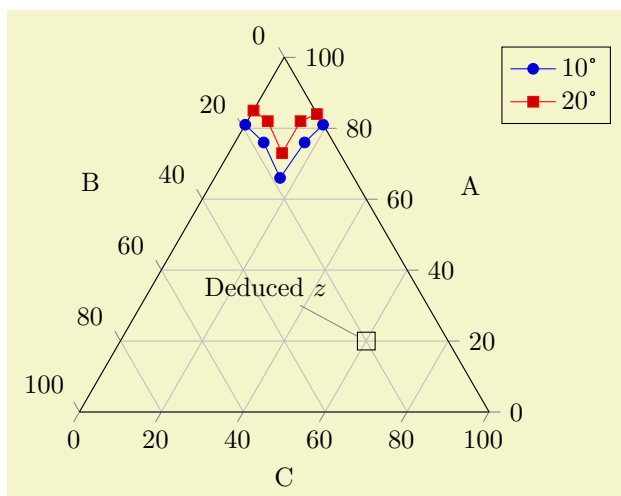
For this to work, it is **crucial to provide** `xmin`, `xmax`, `ymin`, `ymax` and `zmin`, `zmax` precisely! The initial configuration fixes them to the unit interval.

What happens behind the scenes is that a data point (x, y, z) is placed at X, Y determined by

$$\begin{bmatrix} X(x, y, z) \\ Y(x, y, z) \end{bmatrix} = \tilde{x}A + \tilde{y}B + \tilde{z}C = \begin{bmatrix} \frac{1}{2}\tilde{x} + 2\tilde{z} \\ \frac{\sqrt{3}}{2}\tilde{x} \end{bmatrix}$$

where $A = (1/2, \sqrt{3}/2)$ is top corner of the triangle, $B = (0, 0)$ the lower left and $C = (1, 0)$ the lower right one. The $\tilde{y}$ component is not really necessary due to the linear dependency $\tilde{x} + \tilde{y} + \tilde{z} = 1$.

The input coordinate (100%, 0%, 0%) is mapped to A , the input coordinate (0%, 100%, 0%) to B and (0%, 0%, 100%) to C (Acrobat Reader: click into the axis to verify it).



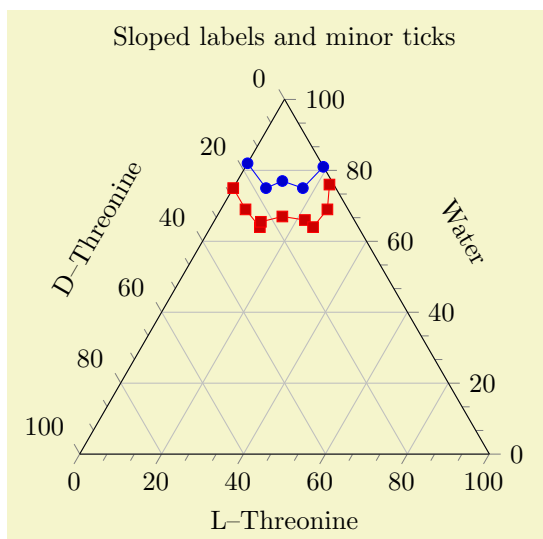
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[xlabel=A,ylabel=B,zlabel=C]
  \addplot3 coordinates {
    (0.81, 0.19, 0.00)
    (0.76, 0.17, 0.07)
    (0.66, 0.16, 0.16)
    (0.76, 0.07, 0.17)
    (0.81, 0.00, 0.19)
  };

  \addplot3 coordinates {
    (0.85, 0.15, 0.00)
    (0.82, 0.13, 0.05)
    (0.73, 0.14, 0.13)
    (0.82, 0.06, 0.13)
    (0.84, 0.00, 0.16)
  };

  \node[pin=130:Deduced  $z$ ,draw=black] at (axis cs:0.2,0.2) {};

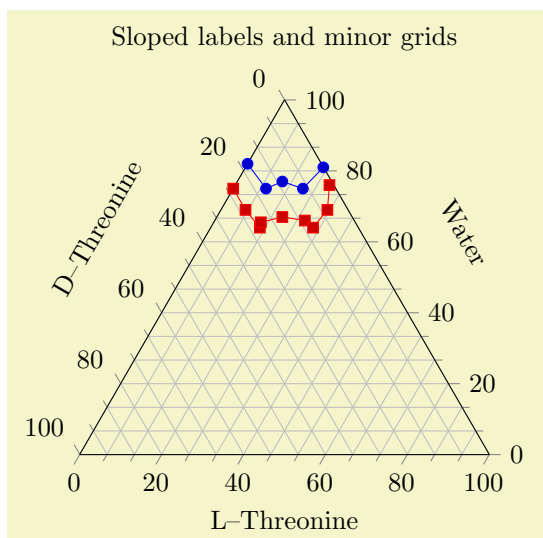
  \legend{$10$ \textdegree, $20$ \textdegree}
\end{ternaryaxis}
\end{tikzpicture}
```

A `ternaryaxis` can contain zero, one or more `\addplot3` commands, just as a usual `axis`. In case you provide only two-dimensional coordinates (for example using `\addplot` or `axis cs`), the third component is deduced automatically such that components sum to 100%. The `\addplot3` command can use any of the accepted input formats, for example using `coordinates`, `table`, `expression` or whatever – but the input is always interpreted as barycentric coordinates (three components summing up to 100%).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  title=Sloped labels and minor ticks,
  xlabel=Water,
  ylabel=D--Threonine,
  zlabel=L--Threonine,
  label style={sloped},
  minor tick num=2,
]
  \addplot3 coordinates {
    (0.82, 0.18, 0.00)
    (0.75, 0.17, 0.08)
    (0.77, 0.12, 0.11)
    (0.75, 0.08, 0.17)
    (0.81, 0.00, 0.19)
  };

  \addplot3 coordinates {
    (0.75, 0.25, 0.00)
    (0.69, 0.25, 0.06)
    (0.64, 0.24, 0.12)
    (0.655, 0.23, 0.115)
    (0.67, 0.17, 0.16)
    (0.66, 0.12, 0.22)
    (0.64, 0.11, 0.25)
    (0.69, 0.05, 0.26)
    (0.76, 0.01, 0.23)
  };
\end{ternaryaxis}
\end{tikzpicture}
```



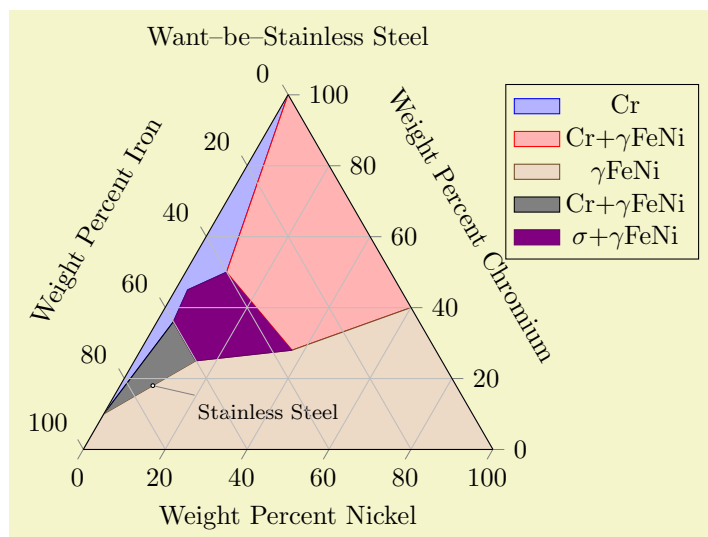
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  title=Sloped labels and minor grids,
  xlabel=Water,
  ylabel=D--Threonine,
  zlabel=L--Threonine,
  label style={sloped},
  minor tick num=2,
  grid=both,
]
  \addplot3 coordinates {
    (0.82, 0.18, 0.00)
    (0.75, 0.17, 0.08)
    (0.77, 0.12, 0.11)
    (0.75, 0.08, 0.17)
    (0.81, 0.00, 0.19)
  };
  \addplot3 coordinates {
    (0.75, 0.25, 0.00)
    (0.69, 0.25, 0.06)
    (0.64, 0.24, 0.12)
    (0.655, 0.23, 0.115)
    (0.67, 0.17, 0.16)
    (0.66, 0.12, 0.22)
    (0.64, 0.11, 0.25)
    (0.69, 0.05, 0.26)
    (0.76, 0.01, 0.23)
  };
\end{ternaryaxis}
\end{tikzpicture}
```

A `ternaryaxis` supports (most of) the PGFLOTS axis interface, among them the `grid` option, the `xtick={\langle positions \rangle}` way to provide ticks, including `extra x ticks` and its variants. Of course, it can also contain any of the `mark`, `color` and `cycle list` options of a normal axis.

The following example is a (crude) copy of an example of

http://www.sv.vt.edu/classes/MSE2094_NoteBook/96ClassProj/experimental/ternary2.html

and uses `area style` to change `cycle list` and the legend appearance.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  title=Want--be--Stainless Steel,
  xlabel=Weight Percent Chromium,
  ylabel=Weight Percent Iron,
  zlabel=Weight Percent Nickel,
  label style=sloped,
  area style,
]
  \addplot3 table {
    A B C
    1 0 0
    0.5 0.4 0.1
    0.45 0.52 0.03
    0.36 0.6 0.04
    0.1 0.9 0
  };
  \addlegendentry{Cr}

  \addplot3 table {
    A B C
    1 0 0
    0.5 0.4 0.1
    0.28 0.35 0.37
    0.4 0 0.6
  };
  \addlegendentry{Cr+$\gamma$FeNi}

  \addplot3 table {
    0.4 0 0.6
    0.28 0.35 0.37
    0.25 0.6 0.15
    0.1 0.9 0
    0 1 0
    0 0 1
  };
  \addlegendentry{$\gamma$FeNi}

  \addplot3 table {
    0.1 0.9 0
    0.36 0.6 0.04
    0.25 0.6 0.15
  };
  \addlegendentry{Cr+$\gamma$FeNi}

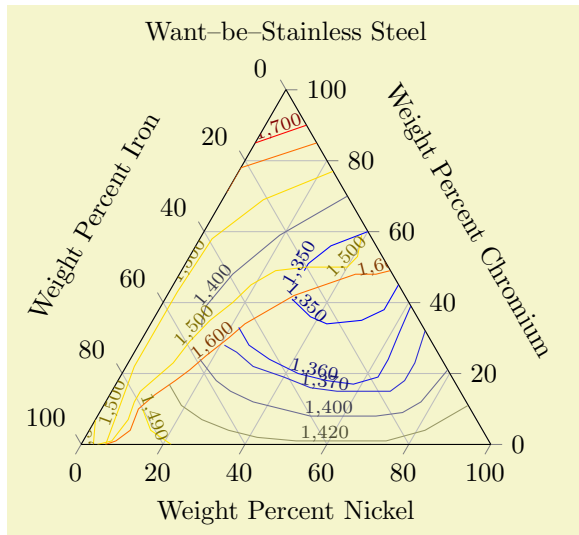
  \addplot3 table {
    0.5 0.4 0.1
    0.45 0.52 0.03
    0.36 0.6 0.04
    0.25 0.6 0.15
    0.28 0.35 0.37
  };
  \addlegendentry{$\sigma$+$\gamma$FeNi}

  \node[inner sep=0.5pt,circle,draw,fill=white,pin=-15:\footnotesize Stainless Steel]
    at (axis cs:0.18,0.74,0.08) {};
\end{ternaryaxis}
\end{tikzpicture}

```

Ternary plots can also use `contour prepared` to plot contour lines. The following example is a (crude) copy of an example of

http://www.sv.vt.edu/classes/MSE2094_NoteBook/96ClassProj/experimental/ternary2.html:



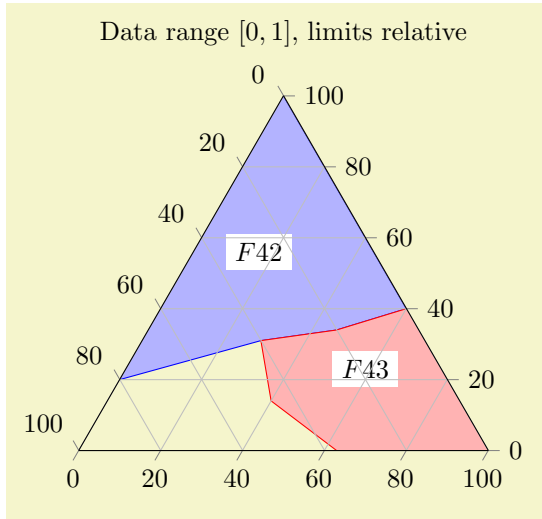
```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  title=Want-be-Stainless Steel,
  xlabel=Weight Percent Chromium,
  ylabel=Weight Percent Iron,
  zlabel=Weight Percent Nickel,
  label style=sloped,
]
% plotdata/pgfplotsternary.example1.dat:
%
% Chromium Iron Nickel Temperature
% 0.90 0.0 0.10 1700
% 0.85 0.14 0.00 1700
%
% 0.85 0.00 0.15 1600
% 0.78 0.22 0.00 1600
% 0.71 0.29 0.00 1600
% ....
\addplot3[contour prepared={labels over line},
  point meta=\thisrow{Temperature}]
  table[x=Chromium,y=Iron,z=Nickel]
  {plotdata/pgfplotsternary.example1.dat};
\end{ternaryaxis}
\end{tikzpicture}
```

The `contour prepared={labels over line}` installs the display style `contour/labels over line` and expects precomputed contour lines from the input stream. Here, the input stream is a table, consisting of the three relative components for Chromium, Iron and Nickel – and the `point meta` is set to be the Temperature column. The `contour prepared` style uses the (x, y, z) coordinate to plot the data point and the `point meta` to determine contour labels (the initial configuration of `contour prepared` is to use `point meta=z`). The output thus allows to use both barycentric coordinates (ternary components) *and* contour labels.

```
/pgfplots/ternary limits relative=true|false (initially true)
/pgfplots/ternary relative limits=true|false (initially true)
```

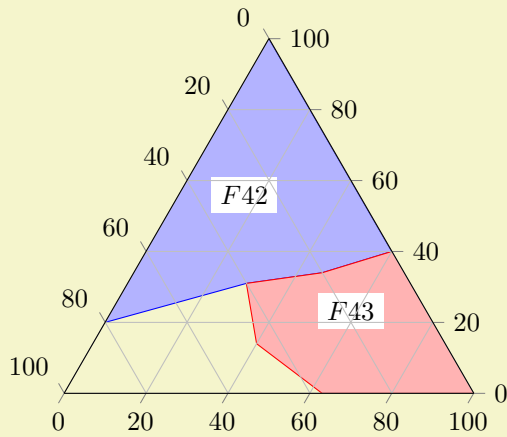
Allows to switch tick labels between relative numbers in the range $[0, 100]$ or absolute numbers.

The choice `ternary limits relative=true` accepts data in any input number range, for example $(x, y, z) \in [0, 1]^3$, or $(x, y, z) \in [0, 100]^3$ or in any absolute scala of the form $x_i \in [\underline{x}_i, \overline{x}_i]$ for $x_i \in \{x, y, z\}$ (remember that it is crucial to communicate these limits to PGFLOTS explicitly using `xmin`, `xmax`, `ymin`, `ymax` and `zmin`, `zmax` such that relative coordinates can be computed, see the description above for details). In every case, relative tick labels are drawn, i.e. tick labels in the range $[0, 100]$.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
    ternary limits relative,
    title={Data range  $[0,1]$ , limits relative},
    area style]
\addplot3 coordinates {
    (0.2,0.8,0)
    (0.31,0.4,0.29)
    (0.34,0.2,0.46)
    (0.4,0,0.6)
    (1,0,0)
};
\addplot3 coordinates {
    (0.4,0,0.6)
    (0.34,0.2,0.46)
    (0.31,0.4,0.29)
    (0.14,0.46,0.4)
    (0,0.37,0.63)
    (0,0,1)
};
\node[fill=white]
    at (axis cs:0.56,0.28,0.16) {$F_{42}$};
\node[fill=white]
    at (0.7,0.2) {$F_{43}$};
\end{ternaryaxis}
\end{tikzpicture}
```

Data range $x \in [0, 500]$, $y \in [1, 2]$, $z \in [0, 1]$ limits relative



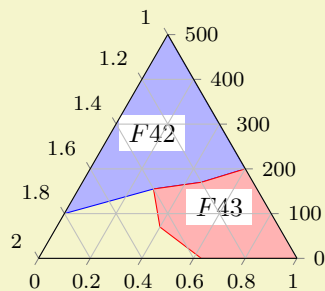
```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  xmax=500,ymin=1,ymax=2,
  ternary limits relative,
  title={Data range  $x\in[0,500]$ ,
     $y\in[1,2]$ ,  $z\in[0,1]$  limits relative},
  area style]
\addplot3 coordinates {
  (100,1.8,0)
  (155,1.4,0.29)
  (170,1.2,0.46)
  (200,1,0.6)
  (500,1,0)
};
\addplot3 coordinates {
  (200,1,0.6)
  (170,1.2,0.46)
  (155,1.4,0.29)
  (70,1.46,0.4)
  (0,1.37,0.63)
  (0,1,1)
};
\node[fill=white]
  at (axis cs:280,1.28,0.16) {$F_{42}$};
\node[fill=white]
  at (0.7,0.2) {$F_{43}$};
\end{ternaryaxis}
\end{tikzpicture}

```

The choice `ternary limits relative=false` accepts the same data ranges, but it draws tick labels in the very same data ranges.

Data range $x \in [0, 500]$, $y \in [1, 2]$, $z \in [0, 1]$ limits absolute



```

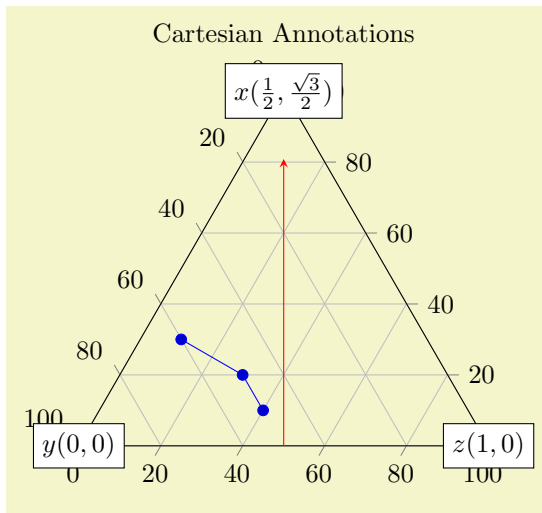
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
    ternary limits relative=false,
    xmax=500,ymin=1,ymax=2,
    title={Data range  $x\in[0,500]$ ,
         $y\in[1,2]$ ,  $z\in[0,1]$  limits absolute},
    footnotesize, % just for the sake of demonstration...
    area style]
\addplot3 coordinates {
    (100,1.8,0)
    (155,1.4,0.29)
    (170,1.2,0.46)
    (200,1,0.6)
    (500,1,0)
};
\addplot3 coordinates {
    (200,1,0.6)
    (170,1.2,0.46)
    (155,1.4,0.29)
    (70,1.46,0.4)
    (0,1.37,0.63)
    (0,1,1)
};
\node[fill=white]
    at (axis cs:280,1.28,0.16) {$F_{42}$};
\node[fill=white]
    at (0.7,0.2) {$F_{43}$};
\end{ternaryaxis}
\end{tikzpicture}

```

Coordinate system **cartesian cs**

A coordinate system which allows Cartesian coordinates. The lower left point has coordinate $(0,0)$, the lower right point has $(1,0)$ and the upper point of the triangle is at $(1/2, \sqrt{3}/2)$.

If you use the standard point syntax (x,y) in path commands inside of the axis, you'll get Cartesian coordinates. If you want to use it for axis descriptions (like **xlabel**), you'll have to write **cartesian cs:0,0** explicitly (axis labels have the default coordinate system **axis description cs**).



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
    title=Cartesian Annotations,
    clip=false]

\addplot3 coordinates {
    (0.1,0.5,0.4)
    (0.2,0.5,0.3)
    (0.3,0.6,0.1)
};

\node[fill=white,draw] at (0,0) {$y(0,0)$};
\node[fill=white,draw] at (1,0) {$z(1,0)$};
\node[fill=white,draw] at (0.5,{sqrt(3)/2})
    {$x(\frac{1}{2},\frac{\sqrt{3}}{2})$};

\draw[red,-stealth] (0.5,0) -- (0.5,0.7);
\end{ternaryaxis}
\end{tikzpicture}

```

/pgfplots/**every ternary axis**

(style, no value)

A style which is installed at the beginning of every ternary axis. It is used to adjust some of the PGFLOTS keys to fit the triangular shape.

The initial configuration is

```

\pgfplotsset{
  every ternary axis/.style={
    tick align=outside,
    grid=none,
    xticklabel style={anchor=west},
    every 3d description/.style={},
    every axis x label/.style={at={{ticklabel cs:0.5}},anchor=near ticklabel},
    every axis y label/.style={at={{ticklabel cs:0.5}},anchor=near ticklabel},
    every axis z label/.style={at={{ticklabel cs:0.5}},anchor=near ticklabel},
    every x tick scale label/.style={
      at={{xticklabel cs:0.95,5pt}},anchor=near xticklabel,inner sep=0pt},
    every y tick scale label/.style={
      at={{yticklabel cs:0.95,5pt}},anchor=near yticklabel,inner sep=0pt},
    every z tick scale label/.style={
      at={{yticklabel cs:0.95,5pt}},anchor=near yticklabel,inner sep=0pt},
    every axis title shift=15pt,
    every axis legend/.style={
      cells={anchor=center},
      inner xsep=3pt,inner ysep=2pt,nodes={inner sep=2pt,text depth=0.15em},
      shape=rectangle,
      fill=white,
      draw=black,
      at={{(1.03,1.03)},
      anchor=north west,
    },
    annot/point format 3d/.initial={{(\% .2f, \% .2f, \% .2f)}},
  },
}

```

5.10.2 Tieline Plots

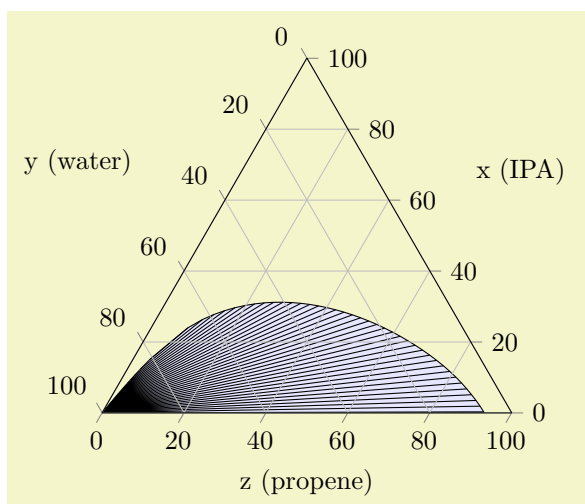
`/tikz/tieline={\options with tieline/ prefix}`

`\addplot+[tieline={\options with tieline/ prefix}]`

A plot handler for use in ternary diagrams which plots tie lines and binodal curves.

On input, it accepts *pairs* of coordinates, $A^{(i)} = (A_x^{(i)}, A_y^{(i)}, A_z^{(i)})$ and $B^{(i)} = (B_x^{(i)}, B_y^{(i)}, B_z^{(i)})$, for $i = 1, \dots, N$ (i.e. it requires a total of six coordinates, perhaps plus additional color data).

On output, it connects the pairs, i.e. for every fixed $i = 1, \dots, N$, it connects $A^{(i)} - B^{(i)}$ (the so-called “tie lines”). In addition, it also draws the binodal curve, which is made up by connecting all $A^{(i)}$ and then, in reverse ordering, all $B^{(i)}$: $A^{(1)} - A^{(2)} - \dots - A^{(N)} - B^{(N)} - B^{(N-1)} - \dots - B^{(1)}$.



```

% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  xlabel=x (IPA),
  ylabel=y (water),
  zlabel=z (propene),
  axis on top,
]
% plotdata/ternary_data.txt is a table of the form
% A_propene A_water A_IPA B_propene B_water B_IPA
% 0.0009 0.9990 0 0.9333 0.0667 0
% 0.0009 0.9988 0.0002 0.9303 0.0665 0.0032
% 0.0011 0.9975 0.0013 0.9135 0.0673 0.0191
% 0.0013 0.9962 0.0024 0.8956 0.0693 0.0351
% ...
\addplot3[tieline,fill=blue!10]
table [x=A_IPA,y=A_water,z=A_propene]
{plotdata/ternary_data.txt};
\end{ternaryaxis}
\end{tikzpicture}

```

We see that each input line has six columns, and each six columns are taken into account (this is different from other plot handlers!). The six columns make up the three components of the A and B points, respectively. In the example above, we used explicit column names and provided A_x using `x=A_IPA`, A_y using `y=A_water` and A_z using `z=A_propene`. Note that these keys are the common input method for `\addplot table`; they are nothing special (that means we could also use `x index` instead). The three columns for B can be provided manually (see below), or deduced automatically: in our case, the value

for B_x has been found in the third column after $x=A_IPA$ (which is B_IPA); the value for B_y has been found in the third column after $y=A_water$ and B_z is made up from the third column after $z=A_propene$. In other words, the B value is searched (by default) by adding 3 to the column index of the respective A coordinate.

You do not need to provide *any* column names; in this case, the first three columns make up A (in the order of appearance) and the following three make up B .

The only supported input type for `tieline` plots is table input. It is optimized to use `\addplot3 table` (as described above). To use the two-dimensional variant `\addplot table`, you need to tell PGFPLOTS explicitly which columns make up A_x, A_y, B_x, B_y ; the z coordinates are deduced automatically such that the result sums to 100%.

```
/pgfplots/table/tie end x={\colname}} (initially empty)
/pgfplots/table/tie end y={\colname}} (initially empty)
/pgfplots/table/tie end z={\colname}} (initially empty)
/pgfplots/table/tie end x index={\col index}} (initially empty)
/pgfplots/table/tie end y index={\col index}} (initially empty)
/pgfplots/table/tie end z index={\col index}} (initially empty)
```

These keys can be used to provide column names or column indices for B_x, B_y and B_z , respectively. They can be provided like

```
\addplot3[tieline] table[tie end y=B_water] ....
```

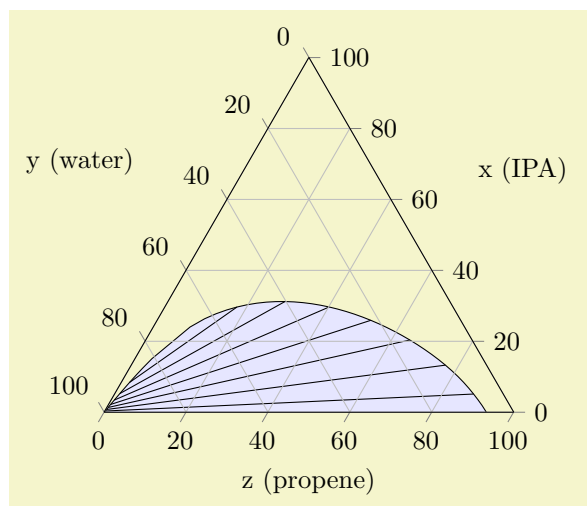
Note that the `tie end x` keys are *only* available if the `tieline` option has been used before.

The values for A are provided with `table/x`, `table/x index` and its variants as for any other plot type.

The `tieline` plot handler accepts several options to customize the appearance. You can provide them as argument after `tieline`, using `tieline={\options}`. In this case, the `tieline/` prefix can be omitted. The keys are described in the following:

```
/pgfplots/tieline/each nth tie={\number}} (initially empty)
```

Allows to draw only each n th tie line, even though the binodal curve uses all provided coordinates:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  xlabel=x (IPA),
  ylabel=y (water),
  zlabel=z (propene),
  axis on top,
]
] plotdata/ternary_data.txt is a table of the form
% A_propene A_water A_IPA B_propene B_water B_IPA
% 0.0009 0.9990 0 0.9333 0.0667 0
% 0.0009 0.9988 0.0002 0.9303 0.0665 0.0032
% 0.0011 0.9975 0.0013 0.9135 0.0673 0.0191
% 0.0013 0.9962 0.0024 0.8956 0.0693 0.0351
% ...
\addplot3[
  tieline={each nth tie=5},
  fill=blue!10,
]
table [x=A_IPA,y=A_water,z=A_propene]
{plotdata/ternary_data.txt};
\end{ternaryaxis}
\end{tikzpicture}
```

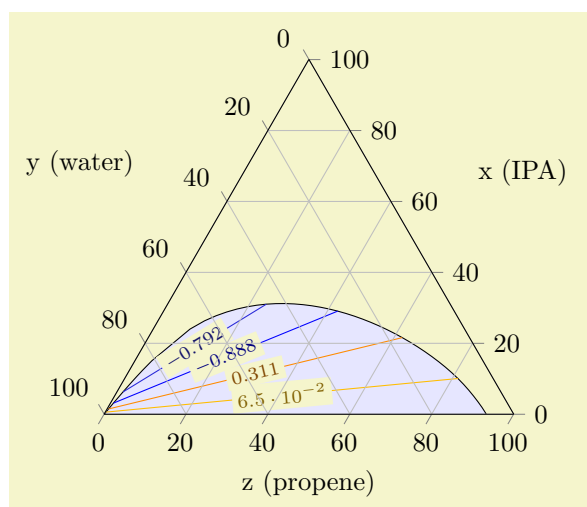
Note that plot `marks` (if any) are drawn on every input position, use the `mark repeat` option to change that.

```
/pgfplots/tieline/tieline style={\options}}
```

Appends `\options` to the style `tieline/every tieline`.

Useful `\options` are, for example, other plot handlers to adjust the appearance of tie lines. Suppose that you have additional color data for every tie line (which might have been provided as further

input column). In our case, we provide random color data using `point meta=rand`, and visualize the single tielines as with `contour prepared`:



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{ternaryaxis}[
  xlabel=x (IPA),
  ylabel=y (water),
  zlabel=z (propene),
  axis on top,
]
% plotdata/ternary_data.txt is a table of the form
% A_propene A_water A_IPA B_propene B_water B_IPA
% 0.0009 0.9990 0 0.9333 0.0667 0
% 0.0009 0.9988 0.0002 0.9303 0.0665 0.0032
% 0.0011 0.9975 0.0013 0.9135 0.0673 0.0191
% 0.0013 0.9962 0.0024 0.8956 0.0693 0.0351
% ...
\addplot3[
  point meta=rand,
  tieline={
    each nth tie=8,
    tieline style={contour prepared}
  },
  fill=blue!10,
]
table [x=A_IPA,y=A_water,z=A_propene]
{plotdata/ternary_data.txt};
\end{ternaryaxis}
\end{tikzpicture}
```

The effect here is that contour labels and line colors are chosen for every tie line, where the actual color is determined using `point meta` and `colormap`. Other choices for plot handlers in `tieline style` might be the `mesh`.

```
/pgfplots/tieline/curve style={\options}
```

Appends `\options` to the style `tieline/every curve`.

The `curve style` allows to customize the plot handler for the curve. A possible choice might be `curve style={smooth}` or a separate fill/draw color.

5.11 Units in Labels

by Nick Papior Andersen

```
\usepgfplotslibrary{units} % LATEX and plain TEX
\usepgfplotslibrary[units] % ConTEXt
\usetikzlibrary{pgfplots.units} % LATEX and plain TEX
\usetikzlibrary[pgfplots.units] % ConTEXt
```

A library which allows to use automatic typesetting of units in labels. The library utilizes different keys to typeset the final output in a consistent way. Calling one of the commands automatically sets the key `'use units=true'` so one does not have to worry about this.

PGFPLOTS has the capability of supporting units. This provides quick customization of the plot as well as the addition of units in labels.

Loading the library automatically enables the typesetting of units in labels. Currently it only supports predefined SI units but a per-user customization is also implemented such that it can be used in any way you like.

First the key which enables you to switch on/off the unit system.

```
/pgfplots/use units={\boolean} (initially true)
```

This key simply enables PGFPLOTS to use what is described next. This key will be set to true if you load the library. You can use this to temporarily determine whether the unit library should be used in plots.

```
/pgfplots/x unit={\unit}} (initially empty)
/pgfplots/y unit={\unit}} (initially empty)
/pgfplots/z unit={\unit}} (initially empty)
```

These keys set the unit in their respective axis. In SI units you could for instance set the `x unit` in Newton as `x unit=N`.

```
/pgfplots/x unit prefix={\prefix}} (initially empty)
/pgfplots/y unit prefix={\prefix}} (initially empty)
/pgfplots/z unit prefix={\prefix}} (initially empty)
```

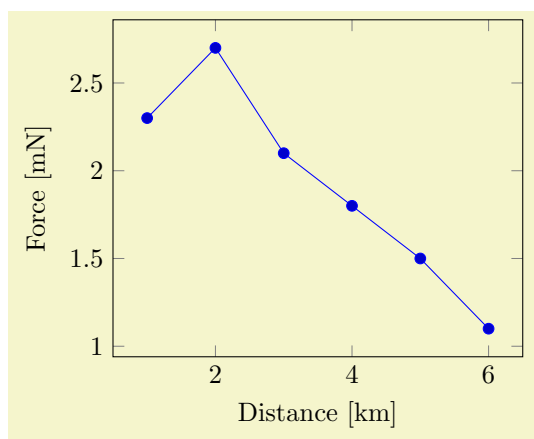
These keys set the prefix of the unit. If a value on the `y axis` is in kilo you would set the `y unit prefix=k`. Prefix will be typeset in front of the unit.

This command will not intervene with the basis of the axis system. I.e. a prefix as just mentioned will not divide every `y axis` number by 1000. In order to do this, see key `\axis SI prefix`, see Section 5.11.1.

Notice that if the `\axis unit` isn't set the entire unit will not be typeset.

Remarks: Remember that all typesetting of labels occur within math mode (i.e. within `$$` delimiters). Therefore one can use `\frac` and other mathematics commands.

Often one just has to utilize the above mentioned keys. It is the basis of the unit typesetting system provided by PGFLOTS.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[use units,
x unit=m,x unit prefix=k,
y unit=N,y unit prefix=m,
xlabel=Distance,ylabel=Force]
\addplot coordinates {
(1,2.3)
(2,2.7)
(3,2.1)
(4,1.8)
(5,1.5)
(6,1.1)
};
\end{axis}
\end{tikzpicture}
```

Below is an example of what would be obtained according to the styles

```
% x label becomes "Temperature [T]", y label becomes "Nothing"
\pgfplotsset{use units,x unit=T,xlabel=Temperature,ylabel=Nothing}
% x label becomes "Temperature", y label becomes "Nothing"
\pgfplotsset{use units,x unit prefix=m,xlabel=Temperature,ylabel=Nothing}
```

Notice the second example. Only setting the prefix will not activate the unit typesetting. Therefore one should ensure to use the `x unit` key if the typesetting of the labels should be done.

For typesetting the units one can also change the appearance. For instance one might not like the square brackets which surround the unit. These can luckily be changed using the below keys.

```
/pgfplots/unit marking pre={\pre}} (initially \left[)
/pgfplots/unit marking post={\post}} (initially \right])
/pgfplots/unit markings=parenthesis|square brackets|slash space (initially square brackets)
```

These keys set the surroundings of the unit. The initial yields $\left[\frac{1}{2}\right]$ such that you can typeset fractions in units. Be aware that you can only obtain large fractions if you use `\dfrac`. These can easily be set using the option key `unit markings` where the options typesets as the following

```
\pgfplotsset{x unit=T,unit markings=parenthesis} % x unit becomes "\left(T\right)"
\pgfplotsset{x unit=T,unit markings=square brackets} % x unit becomes "\left[T\right]"
\pgfplotsset{x unit=T,unit markings=slash space} % x unit becomes "/ T"
```

Notice that all typesetting of units first inserts a space and then the `unit marking pre` code.

Of course you can just manually set each of them with the `unit marking pre` and `unit marking post` keys. Just remember that they are typeset within a `$$`.

One will typically typeset the unit with a specific font. To do so an option of changing the typesetting command is supplied.

```
/pgfplots/unit code/.code 2 args={\...}
```

This can be utilized to great extent. By default, units are typeset as $\mathrm{\langle unit prefix \rangle \langle unit \rangle}$. But if one for instance wishes to utilize the package `siunitx`, which has great capabilities in typesetting both units, numbers and angles, one can just set the key as

```
\pgfplotsset{unit code/.code 2 args={\si{#1#2}}}
```

which would yield the unit as $\mathrm{\langle unit prefix \rangle \langle unit \rangle}$.

The first argument is typeset as $\langle unit prefix \rangle$ and the second argument is $\langle unit \rangle$.

The most important thing is that the command needs exactly two arguments. So if you would like a command that typesets the prefix in bold face and the unit in normal roman font you should call

```
\pgfplotsset{unit code/.code 2 args={\mathbf{#1}\mathrm{#2}}}
```

5.11.1 Preset SI prefixes

To support the SI system a number of preset keys are defined. This should yield a more intuitive way of supplying the prefix as well as add some more functionality. For instance it provides an easy scaling mechanism.

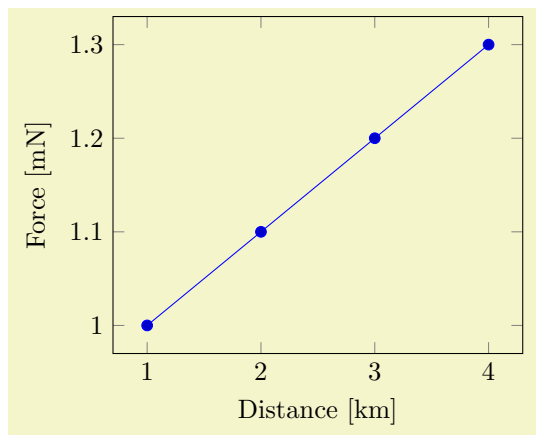
```
/pgfplots/x SI prefix=yocto|...|milli|centi|deci|deca|hecto|kilo|...|yotta (initially none)
/pgfplots/y SI prefix=yocto|...|milli|centi|deci|deca|hecto|kilo|...|yotta (initially none)
/pgfplots/z SI prefix=yocto|...|milli|centi|deci|deca|hecto|kilo|...|yotta (initially none)
/pgfplots/change x base=true|false (initially false)
/pgfplots/change y base=true|false (initially false)
/pgfplots/change z base=true|false (initially false)
```

These keys sets the prefix of the unit. The allowed prefixes are:

Prefix	Power	Prefix	Power
yocto	-24	deca	1
zepto	-21	hecto	2
atto	-18	kilo	3
femto	-15	mega	6
pico	-12	giga	9
nano	-9	tera	12
micro	-6	peta	15
milli	-3	exa	18
centi	-2	zetta	21
deci	-1	yotta	24

As well as resetting the base of the axis if the key `change  $\langle axis \rangle$  base=true`. Just **remember** to set the `change  $\langle axis \rangle$  base` before using the $\langle axis \rangle$ SI prefix key.

See the utilization as in the example below.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[change x base,
x SI prefix=kilo,x unit=m,
y SI prefix=milli,y unit=N,
xlabel=Distance,ylabel=Force]
\addplot coordinates {
(1000,1)
(2000,1.1)
(3000,1.2)
(4000,1.3)
};
\end{axis}
\end{tikzpicture}
```

Notice that the **x axis** has changed base without displaying the $\cdot 10^3$. This is done by using the key **change x base**. Even though you have used the key **y SI prefix=milli** the base isn't changed on the **y axis**. Try adding **change y base** just after **change x base** and see the result!

The above keys are the easy implementation of the base change. Below is a further customization of the base change. It makes it easy to implement a prefix with a custom base change.

`/pgfplots/axis base prefix=axis { $\langle axis \rangle$ } base { $\langle base \rangle$ } prefix { $\langle prefix \rangle$ }` (initially empty)

One can utilize this key to customize further of the base and setting the prefix.

```
\pgfplotsset{change x base,axis base prefix={axis x base -3 prefix k}}
\pgfplotsset{change x base,x SI prefix=kilo}
```

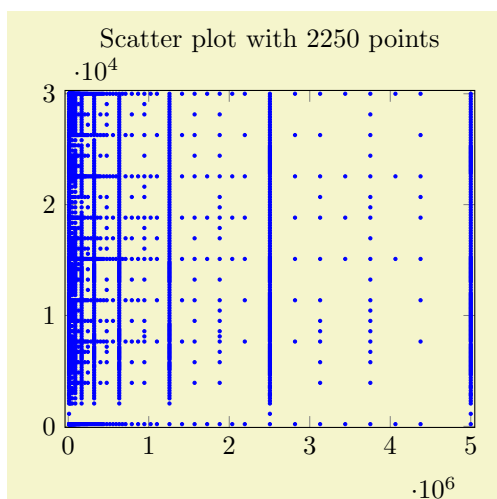
The above two commands are thus equivalent. Remember that the base should operate in opposite of prefix!

Chapter 6

Memory and Speed considerations

6.1 Memory Limits of T_EX

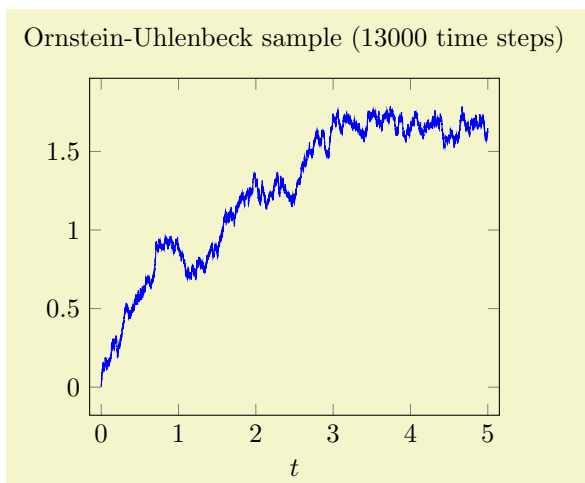
PGFPLOTS can typeset plots with several thousand points if memory limits of T_EX are configured properly. Its runtime is roughly proportional to the number of input points¹.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  enlarge limits=0.01,
  title style={yshift=5pt},
  title=Scatter plot with $2250$ points]

\addplot[blue,
  mark=*,only marks,mark options={scale=0.3}]
  file[skip first]
  {plotdata/pgfplots_scatterdata3.dat};

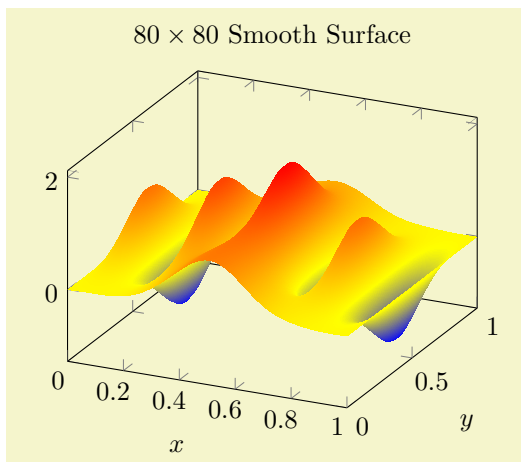
\end{axis}
\end{tikzpicture}
```



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  enlarge x limits=0.03,
  title=Ornstein-Uhlenbeck sample
    ($13000$ time steps),
  xlabel=$t$]

\addplot[blue] file {plotdata/ou.dat};
\end{axis}
\end{tikzpicture}
```

¹In fact, the runtime is pseudo-linear: starting with about 100,000 points, it will become quadratic. This limitation applies to the path length of PGF paths as well. Furthermore, the linear runtime is not possible yet for stacked plots.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
% huger graphs are possible; consider lualatex
\begin{tikzpicture}
\begin{axis}[
    title=$80 \times 80$ Smooth Surface,
    xlabel=$x$,
    ylabel=$y$]
\addplot3[surf,samples=80,shader=interp,domain=0:1]
    {sin(deg(8*pi*x))* exp(-20*(y-0.5)^2)
    + exp(-(x-0.5)^2*30
    - (y-0.25)^2 - (x-0.5)*(y-0.25))};
\end{axis}
\end{tikzpicture}
```

PGFPLOTS relies completely on T_EX to do all typesetting. It uses the front-end-layer and basic layer of PGF to perform all drawing operations. For complicated plots, this may take some time, and you may want to read Section 7 for how to write single figures to external graphics files. Externalization is the best way to reduce typesetting time.

However, for large scale plots with a lot of points, limitations of T_EX’s capacities are reached easily.

6.2 Memory Limitations

The default settings of most T_EX-distributions are quite restrictive, so it may be necessary to adjust them.

Usually, the log-file or the final error message contains a summary about the used resources, giving a hint which parameter needs to be increased.

6.2.1 LuaLaT_EX

One solution which works quite well is to switch the LaT_EX executable: if you have a decent T_EX distribution, you will have the `lualatex` executable as well. This, in turn, uses dynamic memory allocation such that it usually has enough memory for any PGFPLOTS axis.

The LuaLaT_EX executable `lualatex` is supposed to be almost compatible with `pdflatex`.

This approach works for any platform.

6.2.2 MikT_EX

If you are running MikT_EX and you do not want to (or cannot switch) to `lualatex`, you can proceed as follows.

For MikT_EX, memory limits can be increased in two ways. The first is to use command line switches:

```
pdflatex
--stack-size=n --save-size=n
--main-memory=n --extra-mem-top=n --extra-mem-bot=n
--pool-size=n --max-strings=n
```

Experiment with these settings if MikT_EX runs out of memory. Usually, one doesn’t invoke `pdflatex` manually: there is a development aid which does all the invocations, so this one needs to be adjusted.

Sometimes it might be better to adjust the MikT_EX configuration file permanently, for example to avoid reconfiguring the T_EX development program. This can be implemented using the command

```
initexmf --edit-config-file=pdflatex
```

which can be typed either on a command prompt in Windows or using Start » Execute. As a result, an editor will be opened with the correct config file. A sample config file could be

```
main_memory=90000000
save_size=80000
```

or any of the config file entries which are listed below can be entered. Thanks to “LeSpocky” for his documentation in

<http://blog.antiblaue.de/2009/04/21/speicherlimits-von-miktex-erhoehen>.

6.2.3 T_EXLive or similar installations

In addition to the option to switch to `lualatex`, you can proceed as follows to keep existing `dvips` or `pdflatex` workflows.

For Unix installations, one needs to adjust config files. This can be done as follows:

1. Locate `texmf.cnf` on your system. On my Ubuntu installation, it is in `/usr/share/texmf/web2c/texmf.cnf`.
2. Either change `texmf.cnf` directly, or copy it to some convenient place. If you copy it, here is how to proceed:
 - keep only the changed entries in your local copy to reduce conflicts. T_EX will always read *all* config files found in its search path.
 - Adjust the search path to find your local copy. This can be done using the environment variable `TEXMFCNF`. Assuming your local copy is in `~/texmf/mytexcnf/texmf.cnf`, you can write

```
export TEXMFCNF=~/texmf/mytexcnf:
```

to search first in your directory, then in all other system directories.

3. You should change the entries

```
main_memory = n
extra_mem_top = n
extra_mem_bot = n
max_strings = n
param_size = n
save_size = n
stack_size = n
```

The log-file usually contains information about the parameter which needs to be enlarged.

An example of this config file thing is shown below. It changes memory limits.

1. Create the file `~/texmf/mytexcnf/texmf.cnf` (and possibly the paths as well).

```
% newly created file ~/texmf/mytexcnf/texmf.cnf:
% If you want to change some of these sizes only for a certain TeX
% variant, the usual dot notation works, e.g.,
% main_memory.hugetex = 20000000
main_memory = 230000000 % words of inmemory available; also applies to inif&mp
extra_mem_top = 10000000 % extra high memory for chars, tokens, etc.
extra_mem_bot = 10000000 % extra low memory for boxes, glue, breakpoints, etc.
save_size = 150000 % for saving values outside current group
stack_size = 150000 % simultaneous input sources

% Max number of characters in all strings, including all error messages,
% help texts, font names, control sequences. These values apply to TeX and MP.
% pool_size = 1250000
% Minimum pool space after TeX/MP's own strings; must be at least
% 25000 less than pool_size, but doesn't need to be nearly that large.
% string_vacancies = 90000
% Maximum number of strings.
% max_strings = 100000
% min pool space left after loading .fmt
% pool_free = 47500
```

2. Run `texhash` such that T_EX updates its `~/texmf/ls-R` database.
3. Create the environment variable `TEXMFCNF` and assign the value `'~/texmf/mytexcnf:'` (including the trailing `:'`!). For my linux system, this can be done using by adding

```
export TEXMFCNF=~/texmf/mytexcnf:
```

to `~/bashrc`.

Unfortunately, T_EX does not allow arbitrary memory limits, there is an upper bound hard coded in the executables.

6.3 Reducing Typesetting Time

PGFPLOTS does a lot of computations ranging from abstract coordinate computations to low level `.pdf` drawing commands (implemented by PGF). For complex plots, this may take a considerable time – especially for 3D plots.

One possibility to reduce typesetting time is to tell PGF to generate single, temporary `.pdf` (or `.eps`) documents for a subset (or all) graphics in one run and re-use these temporary images in successive runs. For PGFPLOTS, this is the most effective way to reduce typesetting time. It can be accomplished using the [external](#) library described in Section 7.1.

Chapter 7

Import/Export From Other Formats

This section contains information of how to single pictures into separate PDF graphics files (or EPS graphics files). Furthermore, it explains a matlab (®) script which allows to convert from matlab to PGFPLOTS.

7.1 Export to PDF/EPS

It is possible to export images to single PDF-documents using routines of PGF and/or TikZ.

7.1.1 Using the Automatic Externalization Framework of TikZ

```
\usepgfplotslibrary{external} %  $\LaTeX$  and plain  $\TeX$ 
\usepgfplotslibrary[external] % Con $\TeX$ t
\usetikzlibrary{pgfplots.external} %  $\LaTeX$  and plain  $\TeX$ 
\usetikzlibrary[pgfplots.external] % Con $\TeX$ t
```

The `external` library offers a convenient method to export every single `tikzpicture` into a separate .pdf (or .eps). Later runs of $\LaTeX$ will simply include these graphics, thereby reducing typesetting time considerably.

The library can also be used to submit documents to authors who do not even have PGFPLOTS or TikZ installed.

Technical foreword: The `external` library has been written by Christian Feuersänger (author of PGFPLOTS). It has been contributed to TikZ as general purpose library, so the reference documentation along with all tweaks can be found in [5, Section “Externalization Library”]. The command `\usepgfplotslibrary{external}` is actually just a wrapper which loads `\usetikzlibrary{external}` or, if this library does not yet exist because the installed PGF has at most version 2.00, it will load a copy which is shipped with PGFPLOTS.

The `external` library has been designed such that *no changes* to the document as such are necessary. The idea is as follows:

1. Every `\begin{tikzpicture} ... \end{tikzpicture}` gets a file name. The file name can be assigned manually with `\tikzsetnextfilename{<output file name>}` or automatically, in which case `<tex file name>-figure<number>` is used with an increasing `<number>`.
2. The library writes the resulting images using system calls of the form `pdflatex --jobname <output file name>` automatically, using the `writeln` system call of $\TeX$. It is the same framework which can be used to call `gnuplot`.

The only steps which are necessary is to use

```
\usepgfplotslibrary{external}
\tikzexternalize
```

somewhere in your document’s preamble (see below for system-dependent configuration options). No further modification to the document is necessary. Suppose we have a file called `test.tex`:

```

\documentclass{article}

\usepackage{pgfplots}

\usepgfplotslibrary{external}
\tikzexternalize% activate externalization!

\begin{document}
  \begin{figure}
    \begin{tikzpicture}
      \begin{axis}
        \addplot {x^2};
      \end{axis}
    \end{tikzpicture}
    \caption{Our first external graphics example}
  \end{figure}

  \begin{figure}
    \begin{tikzpicture}
      \begin{axis}
        \addplot {x^3};
      \end{axis}
    \end{tikzpicture}
    \caption{A second graphics}
  \end{figure}
\end{document}

```

To enable the system calls, we type

```
pdflatex -shell-escape test
```

and L^AT_EX will now generate the required graphics files `test-figure0.pdf` and `test-figure1.pdf` automatically. Any further call to `pdflatex` will simply use `\includegraphics` and the `tikzpictures` as such are no longer considered (you need a different command line switch for MikT_EX, see the [shell escape](#) option).

If a figure shall be remade, one can simply delete all or selected graphics files and regenerate them. Alternatively, one can use the command `\tikzset{external/force remake}` somewhere in the document to remake every following picture automatically.

There are three ways to modify the file names of externalized figures:

- Changing the overall file name using a [prefix](#),
- Changing the file name for a single figure using `\tikzsetnextfilename`,
- Changing the file name for a restricted set of figures using [figure name](#).

`/tikz/external/prefix={\file name prefix}` (initially empty)

A shortcut for `\tikzsetexternalprefix{\file name prefix}`, see below.

`\tikzsetexternalprefix{\file name prefix}`

Assigns a common prefix used by all file names. For example,

```
\tikzsetexternalprefix{figures/}
```

will prepend `figures/` to every external graphics file name.

`\tikzsetnextfilename{\file name}`

Sets the file name for the *next* TikZ picture or `\tikz` short command. It will *only* be used for the next picture.

Pictures for which no explicit file name has been set will get automatically generated file names.

Please note that [prefix](#) will still be prepended to `\file name`.


```

\documentclass{article}
% main document, called main.tex
\usepackage{tikz}

\usepgfplotslibrary{external}
\tikzexternalize[prefix=figures/]% activate with a name prefix

\begin{document}

\tikzsetnextfilename{firstplot}
\begin{tikzpicture} % will be written to 'figures/firstplot.pdf'
\begin{axis}
\addplot {x};
\end{axis}
\end{tikzpicture}

\begin{tikzpicture} % will be written to 'figures/main-figure0.pdf'
\draw[help lines] (0,0) grid (5,5);
\end{tikzpicture}
\end{document}

```

```
pdflatex -shell-escape main
```

/tikz/external/figure name={\langle name \rangle}

Same as `\tikzsetfigurename{\langle name \rangle}`.

`\tikzsetfigurename{\langle name \rangle}`

Changes the names of *all* following figures. It is possible to change `figure name` during the document using `\tikzset{external/figure name={\langle name \rangle}}`. A unique counter¹ will be used for each different `\langle name \rangle`, and each counter will start at 0.

The value of `prefix` will be applied after `figure name` has been evaluated.

```

\documentclass{article}
% main document, called main.tex
\usepackage{tikz}

\usepgfplotslibrary{external}
\tikzexternalize% activate externalization!

\begin{document}

% will be written to 'main-figure0.pdf'
\begin{tikzpicture}
\begin{semilogyaxis}
\addplot {exp(x)};
\end{semilogyaxis}
\end{tikzpicture}

{
\tikzset{external/figure name={subset_}}
A simple image is \tikz \fill (0,0) circle(5pt);. % will be written to 'subset_0.pdf'

\begin{tikzpicture} % will be written to 'subset_1.pdf'
\begin{axis}
\addplot {x^2};
\end{axis}
\end{tikzpicture}
}% here, the old file name will be restored:

\begin{tikzpicture} % will be written to 'main-figure1.pdf'
\begin{axis}
\addplot[domain=1e-3:100] {1/x};
\end{axis}
\end{tikzpicture}
\end{document}

```

The scope of `figure name` ends with the next closing brace (as all values set by `\tikzset` do).

¹These counters are stored into different *macros*. In other words: no TeX register will be needed.

Remark: Use `\tikzset{external/figure name/.add={\langle prefix\rangle}{\langle suffix\rangle}}` to prepend a $\langle prefix \rangle$ and append a $\langle suffix \rangle$ to the actual value of `figure name`. Might be useful for something like

```
\tikzset{external/figure name=main}

% uses main_0.pdf, main_1.pdf, ...

\chapter{The first section}
{\tikzset{external/figure name/.add={}{_firstsection}}
  ...
  % uses main_firstsection_0.pdf, main_firstsection_1.pdf, ...
}

\chapter{The second section}
{\tikzset{external/figure name/.add={}{secondsection_}}
  ...
  % uses main_secondsection_0.pdf, main_secondsection_1.pdf, ...
  \section{Second subsection}
  {\tikzset{external/figure name/.add={}{sub_}}
    ...
    % uses main_secondsection_sub_0.pdf, main_secondsection_sub_1.pdf, ...
  }
  % uses main_secondsection_2.pdf, main_secondsection_3.pdf, ...
}
```

`\tikzappendtofigurename{\langle suffix\rangle}`

Appends $\langle suffix \rangle$ to the actual value of `figure name`.

It is a shortcut for `\tikzset{external/figure name/.add={}{\langle suffix\rangle}}` (a shortcut which is also supported if TikZ is not installed, see below).

Configuration option for eps output or MikTeX: Since the `external` lib works by means of system calls, it has to be modified to fit the local system. This is necessary for MikTeX since it uses a different option to enable these system calls. It is also necessary for EPS output since this involves a different set of utilities.

Note that the *most important part* is to enable system calls. This is typically done by typesetting your document with `pdflatex -shell-escape` or `pdflatex -enable-write18` (MikTeX). These options *need to be configured in your T_EX editor*. Besides this step, one may want to configure the system call:

`/tikz/external/system call={\langle template\rangle}`

A template string used to generate system calls. Inside of $\langle template \rangle$, the macro `\image` can be used as placeholder for the image which is about to be generated while `\texsource` contains the main file name (in truth, it contains `\input{\langle main file name\rangle}`, but that doesn't matter).

The default is

```
\tikzset{external/system call={pdflatex \tikzexternalcheckshellescape -halt-on-error
-interaction=batchmode -jobname "\image" "\texsource"}}
```

where `\tikzexternalcheckshellescape` inserts the value of the configuration key `shell escape` if and only if the current document has been typeset with `-shell-escape`².

For eps output, you can (and need to) use

```
\tikzset{external/system call={latex \tikzexternalcheckshellescape -halt-on-error
-interaction=batchmode -jobname "\image" "\texsource" &&
dvips -o "\image".ps "\image".dvi}}
```

The argument $\langle template \rangle$ will be expanded using `\edef`, so any control sequences will be expanded. During this evaluation, `'\'` will result in a normal backslash, `'\'`. Furthermore, double quotes `"`, single quotes `'`, semicolons and dashes `-` will be made to normal characters if any package uses them as macros. This ensures compatibility with the `german` package, for example.

²Note that this is always true for the default configuration. This security consideration applies mainly for `mode=list` and `make` which will also work *without* shell escapes.

`/tikz/external/shell escape={\command-line arg}` (initially `-shell-escape`)

Contains the command line option for `latex` which enables the `\write18` feature.

For `TEX-Live`, this is `-shell-escape`. For `MikTEX`, you should use `\tikzexternalize[shell escape=-enable-write18]`.

Support for Labels and References In External Files The `external` library comes with extra support for `\label` and `\ref` (and other commands which usually store information in the `.aux` file) inside of external files.

There are, however, some points which need your attention when you try to use

- a) `\ref` to something in the main document inside of an externalized graphics or
- b) `\label` in the externalized graphics which is referenced in the main document.

For point a), a `\ref` inside of an externalized graphics works *only* if you issue the required system call *manually* or by `make`. The initial configuration `mode=convert with system call` does *not* support `\ref`. But you can copy-paste the system call generated by `mode=convert with system call` and issue it manually. The reason is that `\ref` information is stored in the main `.aux` file – but this auxiliary file is not completely written when `mode=convert with system call` is invoked (there is a race condition). Note that `\pageref` is not supported (sorry). Thus: if you have `\ref` inside of external graphics, consider using `mode=list and make` or copy-paste the system call for the image(s) and issue it manually.

Point b) is realized automatically by the external library. In detail, a `\label` inside of an externalized graphics causes the external library to generate separate auxiliary files for every external image. These files are called `(imagename).dpth`. The extension `.dpth` indicates that the file also contains the image’s depth (the `baseline` key of `TikZ`). Furthermore, anything which would have been written to an `.aux` file will be redirected to the `.dpth` file – but only things which occur inside of the externalized `tikzpicture` environment. When the main document loads the image, it will copy the `.dpth` file into the main `.aux` file. Then, successive compilations of the main document contain the external `\label` information. In other words, a `\label` in an external graphics needs the following work flow:

1. The external graphics needs to be generated together with its `.dpth` (usually automatically by `TikZ`).
2. The main document includes the external graphics and copies the `.dpth` content into its main `.aux` file.
3. The main document needs to be translated once again to re-read its `.aux` file³.

There is just a special case if a `\label/\ref` drawn as a `tikzpicture`. This is, for example, the case for the legend `\ref` images or for the `\pgfplotslegendfromname` feature. In such cases, you need to proceed as for case a) since `mode=convert with system call` can’t handle that stuff on its own.

In other words: a `\label` in an external document works automatically, just translate the main document often enough. A `\ref` might need manual adjustments as described for case a) above.

Operation Modes

`/tikz/external/mode=convert with system call|list and make|...` (initially `convert with system call`)

This allows to change the default operation mode. There are a handful of choices possible, all of them are described in detail in [5, section “Externalization Library”]. The most useful ones are probably the initial configuration `convert with system call` and the specialized choice `list and make`.

The choice `list and make` configures the library to check if there are already external graphics and uses them. If there are no graphics, the library will *skip* the figure. However, it will also generate a `makefile` to generate the graphics, and a list of all required graphics files.

It is not required to use `make`: the library expects you to generate the images somehow and it doesn’t care about the “how”. Using `make -f <name-of-tex-file>.makefile -j 2` allows parallel execution

³Note that it is not possible to activate the content of an auxiliary file after `\begin{document}` in `LATEX`.

which might, indeed, be an option. Furthermore, the makefile also supports file dependencies: if one of your data tables has been updated, the external graphics will be remade automatically. PGFPLOTS tells the external library about any file dependencies (input files and tables).

The two modes have the following characteristics:

1. `convert` with `system call` is automatic and does everything on-the-fly. However, it *can't* work with `\ref` and/or `\label` information in external pictures.
2. `list` and `make` requires either manual (by issuing the system calls manually) or semi-automatic conversion (using the generated `\main\makefile`), and multiple runs of `pdflatex`. The generated Makefile can be processed in parallel. Furthermore, `list` and `make` provides *full support* for `\ref` and `\label`: any `\label` defined inside of an externalized graphics is still available for the main document.

If you have legends with `legend to name` or `\label/\ref`, you need to generate the graphics defining the `\label` (or `legend to name`), then run `pdflatex` twice on the main document. Afterwards, you can externalize the legend graphics.

The complete reference documentation and remaining options are documented in [5, “Externalization Library”]. This reference also contains information about

- how to use `\tikzset{external/force remake}` and `\tikzset{external/remake next}` to re-make selected figures,
- how to disable the externalization partially with `\tikzset{external/export=false}` or completely with `\tikzexternaldisable`,
- how to optimize the speed of the conversion process using `\tikzset{external/optimize command away=\myExpensiveMacro}`,
- how to add further remake-dependencies with `\tikzpicturedepends on file{<name>}` and/or `\tikzexternalfiledepends on file{<external file>}{<name>}`,
- examples how to enable `png` export,
- how to typeset such a document without `PGF` installed or
- how to provide work-arounds with `.pdf` images and bounding box restrictions.

Using the Library Without `PGF` or `PGFPLOTS` Installed There is a small replacement package `tikzexternal.sty` which can be used once every figure has been exported. The idea is to uncomment `\usepackage{tikz}` and `\usepackage{pgfplots}` and write `\usepackage{tikzexternal}` instead:

```
% \usepackage{tikz}
% \usepackage{pgfplots}
\usepackage{tikzexternal}
\tikzexternalize% activate externalization

\begin{document}
\begin{tikzpicture}
...
\end{tikzpicture}
...
\end{document}
```

You do not need `PGF`, `TikZ` or `PGFPLOTS` installed. What you need is `tikzexternal.sty` and all generated figures (consisting of the image files, `.pdf` and the `.dpth` files containing information of the `baseline` option). The file `tikzexternal.sty` is shipped with `PGF` in the directory

```
latex/pgf/utilities/tikzexternal.sty
```

and a copy is shipped with `PGFPLOTS` in

```
tex/generic/pgfplots/oldpgfcompatib/pgfplotsoldpgfsupp_tikzexternal.sty
```

Just copy the file into your directory and rename it to `tikzexternal.sty`.

Attention: The small replacement package doesn't support key-value interfaces. Thus, it is necessary to use `\tikzsetexternalprefix` instead of the `prefix` option and `\tikzsetfigurename` instead of the `figure name` option since `\tikzset` is not available in such a context. Also, you may want to define a dummy-macro `\pgfplotsset` if you have used `\pgfplotsset`.

7.1.2 Using the Externalization Framework of PGF “By Hand”

Another way to export T_EX-pictures to single graphics files is to use the externalization framework of PGF, which requires more work but works more generally than the `external` library. The basic idea is to encapsulate the desired parts with

```
\beginpgfgraphicnamed{<output file name>}
<picture contents>
\endpgfgraphicnamed.
```

Furthermore, one needs to tell PGF the name of the main document using

```
\pgfrealjobname{<the real job's name>}
```

in the preamble. This enables two different modes:

1. The first is the normal typesetting mode. L^AT_EX checks whether a file named *<output file name>* with one of the accepted file extensions exists – if that is the case, the graphics file is included with `\pgfimage` and the *<picture contents>* is skipped. If no such file exists, the *<picture contents>* is typeset normally. This mode is applied if `\jobname` equals *<the real job's name>*.
2. The second mode applies if `\jobname` equals *<output file name>*, it initiates the “conversion mode” which is used to write the graphics file *<output file name>*. In this case, *only* *<picture contents>* is written to `\jobname`, the complete rest of the L^AT_EX is processed as normal, but it is silently discarded.

This mode needs to be started manually with `pdflatex --jobname <output file name>` for every externalized graphics file.

A complete example may look as follows.

```
\documentclass{article}

\usepackage{pgfplots}

\pgfrealjobname{test}

\begin{document}
  \begin{figure}
    \beginpgfgraphicnamed{testfigure}
    \begin{tikzpicture}
      \begin{axis}
        \addplot {x^2};
      \end{axis}
    \end{tikzpicture}
    \endpgfgraphicnamed
    \caption{Our first external graphics example}
  \end{figure}

  \begin{figure}
    \beginpgfgraphicnamed{testfigure2}
    \begin{tikzpicture}
      \begin{axis}
        \addplot {x^3};
      \end{axis}
    \end{tikzpicture}
    \endpgfgraphicnamed
    \caption{A second graphics}
  \end{figure}
\end{document}
```

The file is named `test.tex`, and it is processed (for example) with

```
pdflatex test
```

Now, we type

```
pdflatex --jobname testfigure test
pdflatex --jobname testfigure2 test
```

to enter conversion mode. These last calls will *only* write the contents of our named graphics environments, one for `<testfigure>` and one for `<testfigure2>` into the respective output files `testfigure.pdf` and `testfigure2.pdf`.

In summary, one needs `\pgfrealjobname` and calls `pdflatex --jobname <graphics file>` for every externalized graphics environment. Please note that it is absolutely necessary to use the syntax above, *not* `\begin{pgfgraphicnamed}`.

These steps are explained in much more detail in Section “Externalizing Graphics” of [5].

Attention: Do not forget a correct `\pgfrealjobname` statement! If it is missing, externalization simply won’t work. If it is wrong, any call to L^AT_EX will produce empty output files.

It should be noted that this approach of image externalization is not limited to TikZ picture environments. In fact, it collects everything between the begin and end statements into the external file. It is implicitly assumed that the encapsulated stuff is one box, but you can also encapsulate complete paragraphs using something like the L^AT_EX minipage (or a `\vbox` which is not as powerful but does not affect the remaining document that much).

`/pgf/images/aux in dpth=true|false` (initially false)

If this boolean is set to `true`, any `\label` information generated inside of the external image is stored into the already mentioned `.dpth` file. The main document can thus reference label information of externalized parts of the document (although you may need to run `latex` several times).

Label support is provided for `\ref`, and probably `\cite`. The `\pageref` command is only partially supported.

Using the Library Without PGF Installed Simply uncomment the packages `\usepackage{tikz}` and `\usepackage{pgfplots}` and use

```
\long\def\beginpgfgraphicnamed#1#2\endpgfgraphicnamed{%
  \begingroup
  \setbox1=\hbox{\includegraphics{#1}}%
  \openin1=#1.dpth
  \ifeof1 \box1
  \else
    \read1 to\pgfincludeexternalgraphicsdp \closein1
    \dimen0=\pgfincludeexternalgraphicsdp\relax
    \hbox{\lower\dimen0 \box1 }%
  \fi
  \endgroup
}
```

instead. This will include the generated graphics files (and it will respect the `baseline` information stored in `.dpth` files). Consequently, you won’t need PGF or PGFLOTS installed. See Section “Externalizing Graphics” of [5] for details.

7.2 Importing From Matlab

7.2.1 Importing Mesh Data From Matlab To PGFPlots

While it is easy to write Matlab vectors to files (using `save P.dat data -ASCII`), it is more involved to export mesh data.

The main problem is to communicate the mesh structure to PGFLOTS.

Here is an example how to realize this task: in Matlab, we have mesh data `X`, `Y` and `Z` which are matrices of the same size. For example, suppose we have

```
[X,Y] = meshgrid( linspace(-1,1,5), linspace(4,5,10) );
Z = X + Y;
surf(X,Y,Z)
```

as data. Then, we can generate an $N \times 3$ table containing all single elements in column-wise ordering with

```
data = [ X(:) Y(:) Z(:) ]
save P.dat data -ASCII
```

where the second command stores the $N \times 3$ table into `P.dat`. Finally, we can use

```
\addplot3[surf,mesh/rows=10,mesh/ordering=colwise,shader=interp] file {P.dat};
```

in PGFPLOTS to read this data. We need to provide either the number of rows (10 here) or the number of columns – and the ordering (which is `colwise` for Matlab matrices).

An alternative which is faster in PGFPLOTS would be to transpose the matrices in Matlab and tell PGFPLOTS they are in `rowwise` ordering. So, the last step becomes

```
XX=X'; YY=Y'; ZZ=Z';
data = [ XX(:) YY(:) ZZ(:) ]
save P.dat data -ASCII
```

with PGFPLOTS command

```
\addplot3[surf,mesh/cols=10,mesh/ordering=rowwise,shader=interp] file {P.dat};
```

7.2.2 matlab2pgfplots.m

This is a Matlab (®) script which attempts to convert a Matlab figure to PGFPLOTS. It requires Matlab version 7.4 (or higher).

Attention: This script is largely outdated and supports only a very small subset of PGFPLOTS. You may want to look at `matlab2tikz`, a conversion script of Nico Schlömer available at

<http://www.mathworks.com/matlabcentral/fileexchange/22022-matlab2tikz>

which also uses PGFPLOTS for the L^AT_EX conversion.

The idea of `matlab2pgfplots.m` is to

- use a complete matlab figure as input,
- acquire axis labels, axis scaling (log or normal) and legend entries,
- acquire all plot coordinates

and write an equivalent `.pgf` file which typesets the plot with PGFPLOTS.

The intention is *not* to simulate matlab. It is a first step for a conversion. Type

```
> help matlab2pgfplots
```

on your matlab prompt for more information about its features and its limitations.

This script is experimental.

7.2.3 matlab2pgfplots.sh

A bash-script which simply starts matlab and runs

```
f=hgload( 'somefigure.fig' );
matlab2pgfplots( 'outputfile.pgf', 'fig', f );
```

See `matlab2pgfplots.m` above.

7.2.4 Importing Colormaps From Matlab

Occasionally, you may want to reuse your matlab `colormap` in PGFPLOTS. Here is a small Matlab script which converts it to PGFPLOTS:

```
C = colormap; % gets data of the current colormap.
% C = colormap(jet) % gets data of "jet"
eachnth = 1;
I = 1:eachnth:size(C,1); % this is nonsense for eachnth=1 -- but perhaps you don't want each color.
CC = C(I,:);
TeXstring = [ ...
    sprintf('\pgfplotsset{\n\tcolormap={matlab}{\n') ...
    sprintf('\t\ttrgb=(% f,%f,%f)\n',CC') ...
    sprintf('\t}\n\n') ]
```

7.3 SVG Output

It is possible to write every single TikZ picture into a scalable vector graphics (`.svg`) file. This has nothing to do with PGFPLOTS, it is a separate driver of PGF. Please refer to [5, Section “Producing HTML / SVG Output”].

7.4 Generate pgfplots Graphics Within Python

Mario Orne DÍAZ ANADÓN contributed a small python script `pgfplots.py` which provides a simple interface to generate PGFPLOTS figures from within python. It can be found in the PGFPLOTS installation directory, in `pgfplots/scripts/pgfplots/pgfplots.py`; documentation can be found in the file.

Chapter 8

Utilities and Basic Level Commands

This section documents commands which provide access to more basic elements of PGFPLOTS. Most of them are closely related to the basic level of PGF, especially various point commands which are specific to an axis. Some of them are general purpose utilities like loops.

However, most elements in this section are only interesting for advanced users – and perhaps only for special cases.

8.1 Utility Commands

`\foreach` $\langle variables \rangle$ in $\langle list \rangle$ { $\langle commands \rangle$ }

A powerful loop command provided by TikZ, see [5, Section Utilities].

Iterating 1. Iterating 2. Iterating 3. Iterating 4.

```
\foreach \x in {1,2,...,4} {Iterating \x. }%
```

A PGFPLOTS related example could be

```
\foreach \i in {1,2,...,10} {\addplot table {datafile\i}; }%
```

`\pgfplotsforeachungrouped` $\langle variable \rangle$ in $\langle list \rangle$ { $\langle command \rangle$ }

A specialised variant of `\foreach` which can do two things: it does not introduce extra groups while executing $\langle command \rangle$ and it allows to invoke the math parser for (simple!) $\langle x_0 \rangle, \langle x_1 \rangle, \dots, \langle x_n \rangle$ expressions.

Iterating 1. Iterating 2. Iterating 3. Iterating 4. All collected = , 1, 2, 3, 4.

```
\def\allcollected{}
\pgfplotsforeachungrouped \x in {1,2,...,4} {Iterating \x. \edef\allcollected{\allcollected, \x}}%
All collected = \allcollected.
```

A more useful example might be to work with tables. The following example is taken from [PGFPLOT-STABLE](#):

```
\pgfplotsforeachungrouped \i in {1,2,...,10} {%
  \pgfplotstablevertcat{\output}{datafile\i} % appends 'datafile\i' -> '\output'
}%
% since it was ungrouped, \output is still defined (would not work
% with \foreach)
```

Remark: The special syntax $\langle list \rangle = \langle x_0 \rangle, \langle x_1 \rangle, \dots, \langle x_n \rangle$, i.e. with two leading elements, followed by dots and a final element, invokes the math parser for the loop. Thus, it allows larger number ranges than any other syntax if `/pgf/fpu` is active. In all other cases, `\pgfplotsforeachungrouped` invokes `\foreach` and provides the results without $\text{\TeX}$ groups.

Keep in mind that inside of an axis environment, all loop constructions (including custom loops, `\foreach` and `\pgfplotsforeachungrouped`) need to be handled with care: loop arguments can only be used in places where they are immediately evaluated; but PGFPLOTS postpones the evaluation of

many macros. For example, to loop over something and to generate axis descriptions of the form `\node at (axis cs:\i,0.5)...`, the loop macro `\i` will be evaluated in `\end{axis}` – but at that time, the loop is over and its value is lost. The correct way to handle such an application is to *expand* the loop variable *explicitly*. For example:

```
\pgfplotsforeachungrouped \i/\j in {
  1 / a,
  2 / b,
  3 / c
}{
  \edef\temp{\noexpand\node at (axis cs: \i,0.5) {\j}};
  % \show\temp % lets TeX show you what \temp contains
  \temp
}
```

The example generates three loop iterations: `\i=1, \j=a`; then `\i=2, \j=b`; then `\i=3, \j=c`. Inside of the loop body, it expands them and assigns the result to a macro using an “expanded definition”, `\edef`. The result no longer contains either `\i` or `\j` (since these have been expanded). Then, it invokes the resulting macro. Details about the T_EX command `\edef` and expansion control can be found in the document [TeX-programming-notes.pdf](#) which comes with PGFPLOTS.

`\pgfplotsinvokeforeach`{*<list>*}{*<command>*}

A variant of `\pgfplotsforeachungrouped` (and such also of `\foreach`) which replaces any occurrence of `#1` inside of *<command>* once for every element in *<list>*. Thus, it actually assumes that *<command>* is like a `\newcommand` body.

In other words, *<command>* is invoked for every element of *<list>*. The actual element of *<list>* is available as `#1`.

As `\pgfplotsforeachungrouped`, this command does *not* introduce extra scopes (i.e. it is ungrouped as well).

The difference to `\foreach \x in <list>{\command}` is subtle: the `\x` would *not* be expanded whereas `#1` is.

Invoke them: [a] [b] [c] [d]

```
\pgfkeys{
  otherstyle a/.code={[a]},
  otherstyle b/.code={[b]},
  otherstyle c/.code={[c]},
  otherstyle d/.code={[d]}}
\pgfplotsinvokeforeach{a,b,c,d}
{ \pgfkeys{key #1/.style={otherstyle #1}}}
Invoke them:
\pgfkeys{key a} \pgfkeys{key b}
\pgfkeys{key c} \pgfkeys{key d}
```

The counter example would use a macro (here `\x`) as loop argument:

Invoke them: [d] [d] [d] [d]

```
\pgfkeys{
  otherstyle a/.code={[a]},
  otherstyle b/.code={[b]},
  otherstyle c/.code={[c]},
  otherstyle d/.code={[d]}}
\pgfplotsforeachungrouped \x in {a,b,c,d}
{ \pgfkeys{key \x/.style={otherstyle \x}}}
Invoke them:
\pgfkeys{key a} \pgfkeys{key b}
\pgfkeys{key c} \pgfkeys{key d}
```

Restrictions: you can’t nest this command yet (since it does not introduce protection by scopes).

`\pgfmathparse`{*<expression>*}

Invokes the PGF math parser for *<expression>* and defines `\pgfmathresult` to be the result.

The result is ‘42.0’.

```
\pgfmathparse{1+41}

The result is ‘\pgfmathresult’.
```

The math engine in PGF typically uses $\TeX$'s internal arithmetics. That means: it is well suited for numbers in the range $[-16384, 16384]$ and has a precision of 5 digits.

The number range is typically too small for plotting applications. PGFPLOTS improves the number range by means of `\pgfkeys{/pgf/fpu}\pgfmathparse{1+41}` to activate the “floating point unit” (fpu) and to apply all following operations in floating point.

In PGFPLOTS, the key `/pgfplots/use fpu` is typically on, which means that any coordinate arithmetics are carried out with the `fpu`. However, all PGF related drawing operations still use the standard math engine.

In case you ever need to process numbers of extended precision, you may want to use

The result is $1 \cdot 10^6$.

```
\pgfkeys{/pgf/fpu}%
\pgfmathparse{1000*1000}

The result is '\pgfmathprintnumber{\pgfmathresult}'.
```

Note that results of the `fpu` are typically not in human-readable format, so `\pgfmathprintnumber` is the preferred way to typeset such numbers.

Please refer to [5] for more details.

`\pgfplotstableread{<file>}`

Please refer to the manual of [PGFPLOTSTABLE](#), `pgfplotstable.pdf`, which is part of the PGFPLOTS-bundle.

`\pgfplotstabletypeset{<macro>}`

Please refer to the manual of [PGFPLOTSTABLE](#), `pgfplotstable.pdf`, which is part of the PGFPLOTS-bundle.

`\pgfplotsiffileexists{<filename>}{<true code>}{<false code>}`

Invokes `<true code>` if `<filename>` exists and `<false code>` if not. Can be used in looping macros, for example to plot every data file until there are no more of them.

`\pgfplotsutilifstringequal{<first>}{<second>}{<true code>}{<false code>}`

A simple “strcmp” tool which invokes `<true code>` if `<first> = <second>` and `<false code>` otherwise. This does not expand macros.

`\pgfkeys`

`\pgfeov`

`\pgfkeysvalueof`

`\pgfkeysgetvalue`

These commands are part of the TikZ way of specifying options, its sub-package `pgfkeys`. The `\pgfplotsset` command is actually nothing but a wrapper around `\pgfkeys`.

A short introduction into `\pgfkeys` can be found in [7] whereas the complete reference is, of course, the TikZ manual [5].

The key `\pgfkeysvalueof{<key name>}` expands to the value of a key; `\pgfkeysgetvalue{<key name>}{<macro>}` stores the value of `<key name>` into `<macro>`. The `\pgfeov` macro is used to delimit arguments for code keys in `\pgfkeys`, please refer to the references mentioned above.

8.2 Commands Inside Of PGFPLOTS Axes

`\autoplotspeclist`

This command should no longer be used, although it will be kept as technical implementation detail. Please use the ‘`cycle list`’ option, Section 4.6.7.

`\logten`

Expands to the constant $\log(10)$. Useful for logplots because $\log(10^i) = i \log(10)$. This command is only available inside of a TikZ-picture.

`\pgfmathprintnumber{⟨number⟩}`

Generates pretty-printed output¹ for $\langle number \rangle$. This method is used for every tick label.

The number is printed using the current number printing options, see the manual of `PGFPLOTS`TABLE which comes with this package for the different number styles, rounding precision and rounding methods.

`\numplots`

Inside of any of the axis environments, associated style, option or command, `\numplots` expands to the total number of plots.

`\numplotssofaractualtype`

Like `\numplots`, this macro returns the total number of plots which have the same plot handler. Thus, if you have `sharp plot` active, it returns the number of all `sharp plots`. If you have `ybar` active, it returns the number of `ybar` plots and so on.

`\plotnum`

Inside of `\addplot` or any associated style, option or command, `\plotnum` expands to the current plot's number, starting with 0.

`\plotnumsofaractualtype`

Like `\plotnum`, but it returns the number among all plots of the same type. The number of all such plots is available using `\numplotssofaractualtype`.

`\coordindex`

Inside of an `\addplot` command, this macro expands to the number of the actual coordinate (starting with 0).

It is useful together with `x filter` or `y filter` to (de)select coordinates.

8.3 Path Operations

`\path`

`\draw`

`\fill`

`\node`

`\matrix`

These commands are TikZ drawing commands all of which are documented in [5]. They are used to draw or fill paths, generate text nodes or aligned text matrices. They are equivalent to `\path[draw]`, `\path[fill]`, `\path[node]`, `\path[matrix]`, respectively.

`\path ... --⟨coordinate⟩ ...;`

A TikZ path operation which connects the current point (the last one before `--`) and $\langle coordinate \rangle$ with a straight line.

`\path ... |-⟨coordinate⟩ ...;`

A TikZ path operation which connects the current point and $\langle coordinate \rangle$ with *two* straight lines: first vertical, then horizontal.

`\path ... -|⟨coordinate⟩ ...;`

A TikZ path operation which connects the current point and $\langle coordinate \rangle$ with *two* straight lines: first horizontal, then vertical.

`/tikz/xshift=⟨dimension⟩`

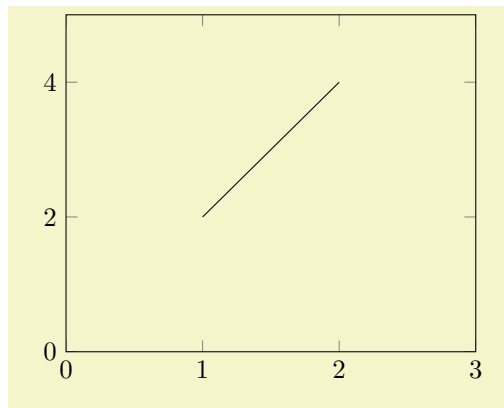
`/tikz/yshift=⟨dimension⟩`

These TikZ keys allow to shift something by $\langle dimension \rangle$ which is any TeX size (or expression).

¹This method was previously `\prettyprintnumber`. Its functionality has been included into PGF and the old command is now deprecated.

`\pgfplotsextra{⟨low-level path commands⟩}`

A command to execute *⟨low-level path commands⟩* in a PGFPLOTS axis. Since any drawing commands inside of an axis need to be postponed until the axis is complete and the scaling has been initialised, it is not possible to simply draw any paths. Instead, it is necessary to draw them as soon as the axis is finished. This is done automatically for every TikZ path – and it is also done manually if you write `\pgfplotsextra{⟨commands⟩}`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[xmin=0,xmax=3,ymin=0,ymax=5]
    \pgfplotsextra{%
      \pgfpathmoveto{\pgfplotspointaxisxy{1}{2}}%
      \pgfpathlineto{\pgfplotspointaxisxy{2}{4}}%
      \pgfusepath{stroke}%
    }
  \end{axis}
\end{tikzpicture}
```

The example above initializes an axis and executes the basic level path commands as soon as the axis is ready. The execution of multiple `\path`, `\addplot` and `\pgfplotsextra` commands is in the same sequence as they occur in the environment².

`\pgfplotspathaxisoutline`

Generates a path which resembles the outline of the current axis. This path is used for clip paths and the background paths (if any).

8.4 Specifying Basic Coordinates

`\pgfplotspointaxisxy{⟨x coordinate⟩}{⟨y coordinate⟩}`

`\pgfplotspointaxisxyz{⟨x coordinate⟩}{⟨y coordinate⟩}{⟨z coordinate⟩}`

Point commands like `\pgfpointxy` which take logical, absolute coordinates and return a low-level point. Every transformation from user transformations to logarithms is applied.

Since the transformations are initialized after the axis is complete, this command needs to be postponed (see `\pgfplotsextra`).

This command is the basic-level variant of `axis cs:⟨x coordinate⟩,⟨y coordinate⟩,⟨z coordinate⟩`.

`\pgfplotspointaxisdirectionxy{⟨x coordinate⟩}{⟨y coordinate⟩}`

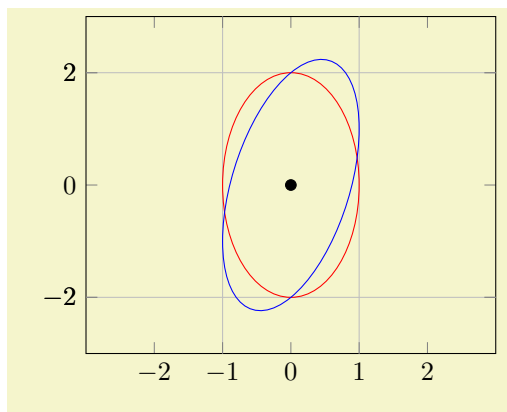
`\pgfplotspointaxisdirectionxyz{⟨x coordinate⟩}{⟨y coordinate⟩}{⟨z coordinate⟩}`

Point commands like `\pgfpointxy` which take logical, *relative* coordinates and return a low-level point. Every transformation from user transformations to logarithms is applied. The difference to `\pgfplotspointaxisxy` is that the shift of the linear transformation is skipped here (compare `disabledatascaling`).

This command is the basic-level variant of `axis direction cs:⟨x coordinate⟩,⟨y coordinate⟩,⟨z coordinate⟩`. Please refer to the documentation of `axis direction cs` for more details.

Use this command whenever something of *relative* character like directions or lengths need to be supplied. One use-case is to draw ellipses:

²Except for stacked plots where the sequence may be reverse, see the key `reverse stack plots`.



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
\begin{axis}[
  xmin=-3,   xmax=3,
  ymin=-3,   ymax=3,
  extra x ticks={-1,1},
  extra y ticks={-2,2},
  extra tick style={grid=major},
]
\draw[red] \pgfextra{
  \pgfpathellipse{\pgfplotspointaxisxy{0}{0}}
    {\pgfplotspointaxisdirectionxy{1}{0}}
    {\pgfplotspointaxisdirectionxy{0}{2}}
  % see also the documentation of
  % 'axis direction cs' which
  % allows a simpler way to draw this ellipse
};
\draw[blue] \pgfextra{
  \pgfpathellipse{\pgfplotspointaxisxy{0}{0}}
    {\pgfplotspointaxisdirectionxy{1}{1}}
    {\pgfplotspointaxisdirectionxy{0}{2}}
};
\addplot [only marks,mark=] coordinates
  { (0,0) };
\end{axis}
\end{tikzpicture}
```

Since the transformations are initialized after the axis is complete, this command needs to be provided either inside of a TikZ `\path` command (like `\draw` in the example above) or inside of `\pgfplotsextra`.

`\pgfplotspointrelaxisxy{<rel x coordinate>}{<rel y coordinate>}`

`\pgfplotspointrelaxisxyz{<rel x coordinate>}{<rel y coordinate>}{<rel z coordinate>}`

Point commands which take *relative* coordinates such that $x = 0$ is the *lower* x axis limit and $x = 1$ the *upper* x axis limit.

These commands are used for `rel axis cs`.

Please note that the transformations are only initialised if the axis is complete! This means you need to provide `\pgfplotsextra`.

`\pgfplotspointdescriptionxy{<x fraction>}{<y fraction>}`

`\pgfplotspointdescriptionxy{<x fraction>}{<y fraction>}`

Point commands such that `{0}{0}` is the lower left corner of the axis' bounding box and `{1}{1}` the upper right one; everything else is in between. The 'q' variant is quicker as it doesn't invoke the math parser on its arguments.

They are used for `axis description cs`, see Section 4.8.1.

`\pgfplotspointaxisorigin`

A point coordinate at the origin, $(0,0,0)$. If the origin is not part of the axis limits, the nearest point on the boundary is returned instead.

This is the same coordinate as returned by the `origin` anchor.

`\pgfplotstransformcoordinatex{<x coordinate of an axis>}`

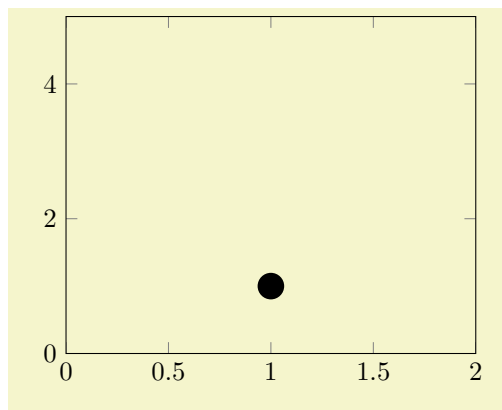
`\pgfplotstransformcoordinatex{<y coordinate of an axis>}`

`\pgfplotstransformcoordinatex{<z coordinate of an axis>}`

Defines `\pgfmathresult` to be the low-level PGF coordinate corresponding to the input argument.

The command applies any `[xyz] coord trafo` keys, data scalings and/or logarithms or whatever PGF-PLOTS does to map input coordinates to internal coordinates.

The result can be used inside of a `\pgfpointxy` statement (i.e. it still needs to be scaled with the respective PGF unit vector).



```
% Preamble: \pgfplotsset{width=7cm,compat=1.8}
\begin{tikzpicture}
  \begin{axis}[xmin=0,xmax=2,ymin=0,ymax=5]
    \pgfplotsextra{%
      \pgfplotstransformcoordinatex{1}%
      \let\xcoord=\pgfmathresult
      \pgfplotstransformcoordinatey{1}%
      \let\ycoord=\pgfmathresult
      \pgfpathcircle
        {\pgfqpointxy{\xcoord}{\ycoord}}
        {5pt}%
      \pgfusepath{fill}%
    }%
  \end{axis}
\end{tikzpicture}
```

The result of this command is also available as math method `transformcoordinatex` (see the documentation for `axis cs`).

Please note that the transformations are only initialised if the axis is complete. This means you need to provide `\pgfplotsextra` as is shown in the example above.

`\pgfplotstransformdirectionx`{ $\langle x \text{ direction of an axis} \rangle$ }
`\pgfplotstransformdirectiony`{ $\langle y \text{ direction of an axis} \rangle$ }
`\pgfplotstransformdirectionz`{ $\langle z \text{ direction of an axis} \rangle$ }

Defines `\pgfmathresult` to be a low-level PGF *direction vector component*.

A direction vector needs to be *added* to some coordinate in order to get a coordinate, compare the documentation for `\pgfplotspointaxisdirectionxy` and `axis direction cs`.

The argument $\langle x \text{ direction of an axis} \rangle$ is processed in (almost) the same way as for the macro which operates on absolute positions, `\pgfplotstransformcoordinatex`. The only difference is that *directions* need no shifting transformation.

The result of this command is also available as math method `transformdirectionx` (see the documentation for `axis direction cs`).

See `axis direction cs` for details and examples about this command.

`\pgfplotsconvertunittocoordinate`{ $\langle x, y \text{ or } z \rangle$ }{ $\langle dimension \rangle$ }

Converts a dimension (with unit!) to a corresponding x , y or z coordinate. The result will be written to `\pgfmathresult` (without units).

It is possible to use the result as arguments for the `\pgfqpointxyz` commands.

The effect is to multiply $\langle dimension \rangle$ with the inverse length of the unit vector for the specified axis. These lengths are precomputed in PGFLOTS so the operation is fast.

```
\pgfplotsconvertunittocoordinate{x}{5pt}
% now, the command uses exactly 5pt in x direction:
\pgfqpointxyz{\pgfmathresult}{4}{3}
```

`\pgfplotspointunitx`
`\pgfplotspointunity`
`\pgfplotspointunitz`

Low-level point commands which return the canvas x , y or z unit vectors.

The `\pgfplotspointunitx` is the PGF unit vector in x direction.

These vectors are essentially the same as `\pgfqpointxyz{1}{0}{0}`, `\pgfqpointxyz{0}{1}{0}`, and `\pgfqpointxyz{0}{0}{1}`, respectively.

The unit z vector is only defined for three dimensional axes.

`\pgfplotsunitxlength`
`\pgfplotsunitylength`
`\pgfplotsunitzlength`
`\pgfplotsunitxinlength`

`\pgfplotsunityinvlength`
`\pgfplotsunitzinvlength`

Macros which expand to the vector length $\|x_i\|$ of the respective unit vector x_i or the inverse vector length, $1/\|x_i\|$. These macros can be used inside of `\pgfmathparse`, for example.

The x_i are the `\pgfplotspointunitx` variants.

`\pgfplotsqpointoutsideofaxis`{ $\langle$ three-char-string $\rangle$ }{ $\langle$ coordinate $\rangle$ }{ $\langle$ normal distance $\rangle$ }

Provides a point coordinate on one of the available four axes in case of a two dimensional figure or on one of the available twelve axes in case of a three dimensional figure.

The desired axis is uniquely identified by a three character string, provided as first argument to the command. The first of the three characters is ‘0’ if the x coordinate of the specified axis passes through the lower axis limit. It is ‘1’, if the x coordinate of the specified axis passes through the upper axis limit. Furthermore, it is ‘2’ if it passes through the origin. The second character is also either 0, 1 or 2 and it characterizes the position on the y axis. The third character is for the third dimension, the z axis. It should be left at ‘0’ for two dimensional plots. However, *one* of the three characters should be ‘v’, meaning the axis varies. For example, v01 denotes $\{(x, y_{\min}, z_{\max}) | x \in \mathbb{R}\}$.

The second argument, $\langle$ coordinate $\rangle$ is the logical coordinate on that axis. Since two coordinates of the axis are fixed, $\langle$ coordinate $\rangle$ refers to the varying component of the axis. It must be a number without unit; no math expressions are supported here.

The third argument $\langle$ normal distance $\rangle$ is a dimension like 10pt. It shifts the coordinate away from the designated axis in direction of the outer normal vector. The outer normal vector always points away from the axis. It is computed using `\pgfplotspointouternormalvectorofaxis`.

There are several variants of this command which are documented in the source code. One of them is particularly useful:

`\pgfplotsqpointoutsideofaxisrel`{ $\langle$ three-char-string $\rangle$ }{ $\langle$ axis fraction $\rangle$ }{ $\langle$ normal distance $\rangle$ }

This point coordinate is a variant of `\pgfplotsqpointoutsideofaxis` which allows to provide an $\langle$ axis fraction $\rangle$ instead of an absolute coordinate. The fraction is a number between 0 (lower axis limit) and 1 (upper axis limit), i.e. it is given in percent of the total axis. It is possible to provide negative values or values larger than one.

The `\pgfplotsqpointoutsideofaxisrel` command is similar in spirit to `rel axis cs`.

There is one speciality in conjunction with reversed axes: if the axis has been reversed by `x dir=reverse` and, in addition, `allow reversal of rel axis cs` is true, the value 0 denotes the *upper* limit while 1 denotes the *lower* limit. The effect is that coordinates won’t change just because of axis reversal.

`\pgfplotspointouternormalvectorofaxis`{ $\langle$ three-char-string $\rangle$ }

A point command which yields the outer normal vector of the respective axis. The normal vector has length 1 (computed with `\pgfpointnormalised`). It is the same normal vector used inside of `\pgfplotsqpointoutsideofaxis` and its variants.

The output of this command will be cached and re-used during the lifetime of an axis.

`\pgfplotsticklabelaxispec`{ $\langle$ x , y or z $\rangle$ }

Expands to the three-character-identification for the axis containing tick labels for the chosen axis, either $\langle$ x $\rangle$, $\langle$ y $\rangle$ or $\langle$ z $\rangle$.

`\pgfplotsvalueoflargesttickdimen`{ $\langle$ x , y or z $\rangle$ }

Expands to the largest distance of a tick position to its tick label bounding box in direction of the outer unit normal vector. It does also include the value of the `ticklabel shift` key.

This value is used for `ticklabel cs`.

`\pgfplotsmathfloatviewdepthxyz`{ $\langle$ x $\rangle$ }{ $\langle$ y $\rangle$ }{ $\langle$ z $\rangle$ }

`\pgfplotsmathviewdepthxyz`{ $\langle$ x $\rangle$ }{ $\langle$ y $\rangle$ }{ $\langle$ z $\rangle$ }

Both macros define `\pgfmathresult` to be the “depth” of a three dimensional point $\bar{x} = (x, y, z)$. The depth is defined to be the scalar product of $\bar{x}$ with $\vec{d}$, the view direction of the current axis.

For `\pgfplotsmathfloatviewdepthxyz`, the arguments are parsed as floating point numbers and the result is encoded in floating point. A fixed point representation can be generated with `\pgfmathfloattofixed`{`\pgfmathresult`}.

For `\pgfplotsmathviewdepthxyz`, TeX arithmetics is employed for the inner product and the result is assigned in fixed point. This is slightly faster, but has considerably smaller data range.

Both commands can only be used *inside* of a three dimensional PGFPLOTS axis (as soon as the axis is initialised, see `\pgfplotsextra`).

`\ifpgfplotsthreedim` $\langle true\ code\rangle\else\langle else\ code\rangle\fi$

A TeX `\if` which evaluates the $\langle true\ code\rangle$ if the axis is three dimensional and the $\langle else\ code\rangle$ if not.

8.5 Accessing Axis Limits

It is also possible to access axis limits during the visualization phase, i.e. during `\end{axis}`. Please refer to the reference documentation for `xmin` on page 241.

8.6 Layer Access

`\pgfplotsonlayer` $\{\langle layer\ name\rangle\}$

A low-level command which will check if the current axis has layer support activated and, if so, calls `\pgfonlayer` $\{\langle layer\ name\rangle\}$.

There must be a `\endpgfplotsonlayer` to delimit the environment.

`\endpgfplotsonlayer`

The end of `\pgfplotsonlayer`.

`\pgfonlayer` $\{\langle layer\ name\rangle\}$

A low-level command of PGF which will collect everything until the matching `\endpgfonlayer` into layer $\langle layer\ name\rangle$.

The $\langle layer\ name\rangle$ must be active, i.e. it must be part of the layer names of `set layers`.

The only special case is if you call `\pgfdeclarelayer{discard}` somewhere: this special layer has a “magical name” which serves as `/dev/null` if it is enabled using `\pgfonlayer{discard}`: it does not need to be active and everything assigned to this layer will be thrown away if it is not part of the layer name configuration.

There must be a `\endpgfonlayer` to delimit the environment.

`\endpgfonlayer`

The end of `\pgfonlayer`.

`\pgfsetlayers` $\{\langle layer\ list\rangle\}$

This is a low-level command of PGF. At the time of this writing, it is the only way to tell PGF which layers it shall use for the current / next picture. It is used implicitly by `set layers`.

Index

— Symbols —

plot ($\langle x \text{ expression} \rangle, \langle y \text{ expression} \rangle$) 35
 ($\langle x \text{ expression} \rangle, \langle y \text{ expression} \rangle, \langle z \text{ expression} \rangle$) .. 94
 -- path operation 420
 .style key 335, 337
 plot { $\langle \text{math expression} \rangle$ } 33
 { $\langle \text{math expression} \rangle$ } 93
 3d box key 229
 3d box foreground style key 276

— —

unit rescale keep size key 224

— A —

Accuracy

Data Transformation 308
 Floating Point in PGFPLOTS 25
 High Precision for Plot Expression 34

\addlegendentry 175
 \addlegendentryexpanded 175
 \addlegendimage 189
 \addplot 24
 \addplot3 90
 after arrow key 72
 after end axis key 311

Alignment

Array 285
 Subplots 285

allow reversal of rel axis cs 424
 allow reversal of rel axis cs key 267
 allow upside down key 172, 269
 anchor key 281

Anchors

near ticklabel 167
 near ticklabel opposite 169
 near xticklabel 167
 near xticklabel opposite 169
 near yticklabel 167
 near yticklabel opposite 169
 near zticklabel 167
 near zticklabel opposite 169

annot/

collected plots 327
 font 326
 height 327
 js fillColor 325
 jsname 327
 point format 325
 point format 3d 325
 popup size 326
 popup size generic 326
 popup size snap 326
 printable 326
 richtext 326
 slope format 326
 snap dist 326
 textSize 326
 width 327
 xmax 327
 xmin 327
 ymax 327

ymax 327
 .append style handler 273
 area cycle list key 75
 area legend key 183
 area style key 75
 array
 Array Alignment 285
 at key 280
 \autoplotspeclist 419
 autumn key 328
 aux in dpth key 414
 average key 375, 382
 axis environment 22
 axis background key 156
 axis base prefix key 402
 axis cs coordinate system 264
 axis description cs coordinate system 161
 axis direction cs coordinate system 265
 axis equal key 214
 axis equal image key 215
 axis line style key 193, 276
 axis lines key 191
 axis lines* key 191
 axis on top key 311, 316
 axis x discontinuity key 196
 axis x line key 190
 axis x line* key 190
 axis y discontinuity key 196
 axis y line key 190
 axis y line* key 191
 axis z discontinuity key 196
 \axisdefaultheight 210
 \axisdefaultwidth 209
 az key 226

— B —

bar cycle list key 64
 bar direction key 65
 Bar Plots
 Skewed axes problems 213
 bar shift key 65
 bar width key 64
 baseline key 283
 before arrow key 71
 before end axis key 310
 Behavior Options 25
 bins key 385
 blackwhite key 146
 bled key 328
 bluered key 146
 bone key 328
 Bounding Box Control 291
 Disable *data* bounding box modifications .. 245
 Excluding Image Parts 289
 Image Externalization Problems 412
 pgfinterruptboundingbox 291
 box key 382
 box extend key 378
 boxplot/
 average 375

box extend	378
data	375
draw/	
average	382
box	382
lower whisker	381
median	382
upper whisker	382
whisker	382
draw direction	376
draw position	374
draw relative anchor	378
every average	380
every box	380
every boxplot	380
every median	380
every whisker	380
lower quartile	375
lower whisker	375
median	375
sample size	377
sample size max	378
sample size min	378
upper quartile	375
upper whisker	375
variable width	377
variable width expr	377
variable width min target	378
whisker extend	378
whisker range	380
boxplot key	379
boxplot box coordinate system	380
boxplot prepared key	373
boxplot whisker coordinate system	381
\boxplotvalue	380
bright key	328
— C —	
cartesian cs coordinate system	396
.cd handler	56
cell picture key	319
change x base key	401
change y base key	401
change z base key	401
check key	100
classes key	81
clickable key	325
clickable library	321
clickable coords key	322
clickable coords code key	323
clickable coords size key	324
clip key	292
clip bounding box key	292
clip limits key	243
clip marker paths key	292
clip mode key	293
closedcycle	
Mesh or patch plots	342
\closedcycle	293
cmd key	125
.code handler	55
.code 2 args handler	55
col sep key	32
cold key	328
collected plots key	327
color key	142
color burning	114
color input key	108
colorbar/	
draw	205
width	204
colorbar key	198
colorbar horizontal key	201
colorbar left key	201
colorbar right key	199
colorbar sampled key	205
colorbar sampled line key	206
colorbar sampled line style key	276
colorbar shift key	205
colorbar source key	203
colorbar style key	204, 275
colorbar to name key	207
colored key	71
colormap/	
autumn	328
blackwhite	146
bled	328
bluered	146
bone	328
bright	328
cold	328
cool	146
copper	329
copper2	329
earth	329
gray	329
greenyellow	146
hot	145
hot2	145, 329
hsv	330
hsv2	330
jet	145, 330
pastel	330
pink	331
redyellow	147
sepia	331
spring	331
summer	331
temp	331
thermal	332
violet	147
winter	332
colormap key	143
colormap access key	161
colormap default colorspace key	145
colormap name key	143
colormaps library	327
colorspace explicit color input key	112
colorspace explicit color output key	112
cols key	92
columns key	335
comment chars key	33
compat/	
empty line	9
general	9
labels	9
path replacement	9

plot3graphics	9	current colorbar axis node.....	205
scale mode	9	current plot begin node	267
scaling	9	current plot end node	267
show suggested version	320	curve style key	399
compat key	8, 173	cycle list key	147
const plot key	57	cycle list name key	147
const plot mark left key	58	cycle list shift key	155
const plot mark mid key	58	cycle multi list key	153
const plot mark right key	58	— D —	
contour/		dashdotdotted key	139
contour dir.....	118	dashdotted key	139
contour label style.....	124	dashed key	139
draw color	123	data key	375, 384
every contour label.....	123	data coord inv trafo key	387
handler	124	data coord trafo key	387
label distance	123	data cs key.....	303
label node code	125	data filter key	387
labels.....	123	data max key	385
labels over line	124	data min key	385
levels	118	/data point/	
number.....	118	x	270
contour dir key	118	y	270
contour external/		z	270
cmd	125	date coordinates in key	298
file	125	date ZERO key	299
output point meta	125	dateplot library	297, 332
scanline marks	125	debug key	52
script.....	125	default smithchart xtick key	367
script extension	125	default smithchart ytick key	367
contour external key	125	default smithchart ytick key	367
contour gnuplot key.....	115, 126	.define layer set handler	318
contour label style key	124	dense smithchart ticks key	365
contour prepared key	121	densely dashdotdotted key.....	139
contour prepared format key	121	densely dashdotted key	139
cool key	146	densely dashed key.....	139
Coordinate systems		densely dotted key.....	139
axis cs	264	density key.....	385
axis description cs.....	161	disabledatascaling key	308
axis direction cs	265	disablelogfilter key	308
boxplot box.....	380	domain key	35
boxplot whisker	381	domain y key	36
cartesian cs.....	396	dotted key	139
rel axis cs.....	267	Downsampling	301
ticklabel cs.....	163	\draw	420
ticklabel* cs	164	draw key	142, 205
xticklabel cs	163	draw color key	123
xticklabel* cs	163	draw direction key.....	376
yticklabel cs	163	draw error bar key.....	234
yticklabel* cs	163	draw position key	374
zticklabel cs	163	draw relative anchor key	378
zticklabel* cs	163	— E —	
plot coordinates	27	each nth point key.....	301
coordinates	91	each nth tie key	398
\coordindex	38, 420	earth key	329
copper key	329	el key	227
copper2 key.....	329	empty legend key	182
crossref file suffix	187	empty line key	9, 26
cube/		\endpgfonlayer.....	425
size x	136	\endpgfplotsonlayer	425
size y	136	enlarge x limits key	243
size z	136	enlarge y limits key	243
cumulative key	385	enlarge z limits key	243
current axis node	289		

enlargetimits key	243	every axis y label key	274
enlargetimits respects figure size key....	245	every axis z grid key	279
Environments		every axis z label key	274
axis	22	every box key	380
groupplot	333	every boxed x axis key	193
loglogaxis	23	every boxed y axis key	193
pgfinterruptboundingbox	291	every boxed z axis key	194
pgfplotsinterruptdatabb	245	every boxplot key	380
polaraxis	354	every colorbar key.....	202, 275
semilogxaxis.....	22	every colorbar sampled line key.....	207, 276
semilogyaxis.....	22	every colorbar to name picture key	208
smithchart	360	every contour label key	123
ternaryaxis.....	388	every crossref picture key	187
tikzpicture.....	22	every error bar key	280
error bar style key	234, 280	every extra x tick key	278
error bars/		every extra y tick key	278
draw error bar	234	every extra z tick key	278
error bar style	234, 280	every forget plot key	274
error mark	233	every inner x axis line key	193, 276
error mark options.....	233	every inner y axis line key	193, 276
x dir.....	233	every inner z axis line key	193, 276
x explicit	233	every legend image post key	181, 275
x explicit relative.....	233	every legend to name picture key.....	189, 275
x fixed	233	every linear axis key	273
x fixed relative	233	every loglog axis key	273
y dir.....	233	every major grid key	279
y explicit	233	every major tick key	276
y explicit relative.....	233	every major x grid key	280
y fixed	233	every major x tick key	278
y fixed relative	233	every major y grid key	280
z dir.....	233	every major y tick key	278
z explicit	233	every major z grid key	280
z explicit relative.....	233	every major z tick key	278
z fixed	233	every mark key	136
z fixed relative	233	every median key	380
error mark key	233	every minor grid key	279
error mark options key	233	every minor tick key	276
Error Messages		every minor x grid key	279
No room for a new dimen	12	every minor x tick key	278
Errors		every minor y grid key	279
dimension too large.....	34, 308	every minor y tick key	278
enlargetimits respects figure size=true: could not		every minor z grid key	279
respect the prescribed width/height ...	244f.	every minor z tick key	278
Patch Input and Memory Problems	131	every node near coord key.....	86
Skewed axes and bar plots.....	213	every non boxed x axis key	194
every 3d box foreground key	276	every non boxed y axis key	194
every 3d description key	227	every non boxed z axis key	194
every 3d view $\{\langle h \rangle\}\{\langle v \rangle\}$ key	228	every outer x axis line key	193, 275
every arrow key	71	every outer y axis line key	193, 275
every average key	380	every outer z axis line key	193, 276
every axis key	273	every patch key	134
every axis grid key	279	every semilogx axis key	273
every axis label key	274	every semilogy axis key	273
every axis legend key	177, 275	every smithchart axis key.....	370
every axis plot key	273	every ternary axis key	396
every axis plot no # key	274	every tick key	276
every axis plot post key	138, 274	every tick label key	277
every axis post key	273	every whisker key	380
every axis title key	275	every x tick key	277
every axis title shift key.....	275	every x tick label key	277
every axis x grid key	279	every x tick scale label key	277
every axis x label key	274	every y tick key	277
every axis y grid key	279	every y tick label key	277

every y tick scale label key	277
every z tick key	277
every z tick label key	277
every z tick scale label key	277
execute at begin axis key	308
execute at begin plot key	308
execute at begin plot visualization key	309
execute at end axis key	308
execute at end plot key	308
execute at end plot visualization key	309
execute for upside down key	172
plot expression	35
expression	94
external library	332, 407
External Graphics	
Bounding Box Issues	412
extra description key	174
extra tick style key	279
extra x tick label key	254
extra x tick labels key	254
extra x tick style key	278
extra x ticks key	249
extra y tick label key	254
extra y tick labels key	254
extra y tick style key	279
extra y ticks key	249
extra z tick label key	254
extra z tick labels key	254
extra z tick style key	279
extra z ticks key	249

— F —

faceted color key	106
few smithchart ticks key	362
figure name key	409
plot file	52
file	91
file key	125
\fill	420
fill key	142
filter discard warning key	303
filter point key	299
Floating Point Unit	34
font key	140, 220, 326
footnotesize key	221
\foreach	417
forget plot key	309
forget plot style key	274
fpu key	313
plot function	40

— G —

general key	9
plot gnuplot	38
plot graphics	41
gray key	329
greenyellow key	146
grid key	262
grid style key	279
group/	
columns	335
empty plot/	
.style	337
every plot/	

.style	335
group name	337
group size	335
horizontal sep	335
rows	335
vertical sep	335
x descriptions at	336
xlabels at	335
xticklabels at	336
y descriptions at	336
ylabels at	335
yticklabels at	336

Group library

Subplots	332
group name key	337
group size key	335
group style key	335
groupplot environment	333
groupplots library	333

— H —

h key	226
handler key	124, 387
header key	31
height key	210, 327
hide axis key	197
hide x axis key	197
hide y axis key	197
hide z axis key	197
hist/	
bins	385
cumulative	385
data	384
data coord inv trafo	387
data coord trafo	387
data filter	387
data max	385
data min	385
density	385
handler	387
intervals	385
symbolic coords	387
hist key	383
horizontal sep key	335
hot key	145
hot2 key	145, 329
hsv key	330
hsv2 key	330

— I —

id key	40f.
ifpgfplotsthreedim	425
ignore chars key	33
ignore first key	53
includegraphics key	45
includegraphics cmd key	45
inner axis line style key	193, 276
interior colormap key	106
interior colormap name key	106
interior colormap thresh key	108
Interrupted Plots	88
intervals key	385
invoke after crossref tikzpicture key	187
invoke before crossref tikzpicture key	187

is smithchart cs key 367

— J —

jet key 145, 330

js fillColor key 325

jsname key 327

jump mark left key 59

jump mark mid key 59

jump mark right key 59

— K —

Key handlers

 .append style 273

 .cd 56

 .code 55

 .code 2 args 55

 .define layer set 318

 .style 273

— L —

\label 186

label distance key 123

label node code key 125

label shift key 173

label style key 274

labels key 9, 123

labels over line key 124

layers/

 axis on top 316

 standard 316

\legend 176

legend cell align key 181

legend columns key 181

legend entries key 176

legend image code key 182

legend image post style key 182, 275

legend plot pos key 181

legend pos key 180

legend reversed key 184

legend style key 180, 275

legend to name key 187

legend transposed key 185

levels key 118

Libraries

 clickable 321

 colormaps 327

 dateplot 297, 332

 external 332, 407

 groupplots 333

 patchplots 338

 polar 354

 smithchart 360

 statistics 372

 ternary 388

 units 399

line cap key 140

line join key 140

line legend key 182

line width key 140

linear regression key 305

\lineno 38

log base 10 number format code key 239

log basis x key 262

log basis y key 262

log basis z key 262

log identify minor tick positions key 238

log number format basis key 239

log number format code key 239

log origin key 245

log origin x key 245

log origin y key 245

log origin z key 245

log plot exponent style key 237

log ticks with fixed point key 236

loglogaxis environment 23

\logten 419

loosely dashdotdotted key 139

loosely dashdotted key 139

loosely dashed key 139

loosely dotted key 139

lower quartile key 375

lower whisker key 375, 381

lowlevel draw key 45

— M —

major grid style key 279

major tick length key 261

major tick style key 276

major x grid style key 280

major x tick style key 278

major y grid style key 280

major y tick style key 278

major z grid style key 280

major z tick style key 278

many smithchart ticks key 363

mark color key 137

mark indices key 137

mark layer key 319

mark list fill key 151

mark options key 138

mark phase key 137

mark repeat key 137

mark size key 136, 220

math parser key 28

\matrix 420

max key 240

max space between ticks key 220, 261

median key 375, 382

mesh

 closedcycle 342

 patch type=line 120

 point meta=none and global paths 342

 scanlinewise 120

mesh/

 check 100

 color input 108

 colospace explicit color input 112

 colospace explicit color output 112

 cols 92

 interior colormap 106

 interior colormap name 106

 interior colormap thresh 108

 ordering 92

 rows 92

 scanline verbose 92

mesh key 87, 98

mesh input key 133

mesh legend key 184

meta key	32	pastel key	330
meta expr key	32	patch	
meta index key	32	closedcycle	342
min key	240	point meta=none and global paths	342
minor grid style key	279	patch key	127
minor tick key	248	Patch plots	
minor tick length key	261	Bounding Box	346
minor tick num key	247	patch refines key	349
minor tick style key	276	patch table key	129
minor x grid style key	279	patch table with individual point meta key	129
minor x tick num key	247	patch table with point meta key	129
minor x tick style key	278	patch to triangles key	350
minor xtick key	248	patch type key	133, 339
minor y grid style key	279	patch type sampling key	340
minor y tick num key	247	patchplots library	338
minor y tick style key	278	\path	420
minor ytick key	248	Path operations	
minor z grid style key	279	--	420
minor z tick num key	247	path replacement key	9
minor z tick style key	278	/pgf/	
minor ztick key	248	bar shift	65
miter limit key	140	bar width	64
mode key	411	fpu	313
		images/	
— N —		aux in dpth	414
near ticklabel anchor	167	mark color	137
near ticklabel align key	170	text mark	138
near ticklabel opposite anchor	169	text mark as node	138
near xticklabel anchor	167	text mark style	138
near xticklabel opposite anchor	169	\pgfdeclareplotmark	138
near yticklabel anchor	167	\pgfeov	419
near yticklabel opposite anchor	169	pgfinterruptboundingbox environment	291
near zticklabel anchor	167	\pgfkeys	419
near zticklabel opposite anchor	169	\pgfkeysgetvalue	419
\nextgroupplot	334	\pgfkeysvalueof	419
no markers key	137	\pgfmathparse	418
\node	420	\pgfmathprintnumber	236, 420
node key	45	\pgfonlayer	425
nodes near coords key	84	\pgfplotbarshift	65
nodes near coords align key	86	\pgfplotbarwidth	65
nodes near coords* key	84	\pgfplotsaxistransformcs	304
normalize size key	220	\pgfplotsboxplotvalue	380
number key	118	\pgfplotsclickablecreate	327
\numplots	420	\pgfplotscolorbarfromname	208
\numplots of actual type	420	\pgfplotscolormaptoshadingspec	147
— O —		\pgfplotsconvertunittocoordinate	423
on layer key	319	\pgfplotscreateplotcyclelist	152
only marks key	78	\pgfplotsdefinecstransform	305
Options		\pgfplotsextra	421
Distinction Behavior, Style Options	25	\pgfplotsforeachungrouped	417
ordering key	92	\pgfplotsiffileexists	419
outer axis line style key	193, 276	pgfplotsinterruptdatabb environment	245
output point meta key	125	\pgfplotsinvokeforeach	418
overlay key	289	\pgfplotslegendfromname	189
— P —		\pgfplotsmathfloatviewdepthxyz	424
parametric/		\pgfplotsmathviewdepthxyz	424
var 1d	40	\pgfplotsonlayer	425
var 2d	40	\pgfplotspathaxisoutline	421
parametric key	40	\pgfplotspointaxisdirectionxy	421
parent axis node	202	\pgfplotspointaxisdirectionxyz	421
parent axis height key	203	\pgfplotspointaxisorigin	422
parent axis width key	203	\pgfplotspointaxisxy	421
		\pgfplotspointaxisxyz	421

<code>\pgfplotspointdescriptionxy</code>	422	<code>plot function</code>	40
<code>\pgfplotspointouternormalvectorofaxis</code>	424	<code>plot gnuplot</code>	38
<code>\pgfplotspointplotatime</code>	270	<code>plot graphics</code>	41
<code>\pgfplotspointrelaxisxy</code>	422	<code>plot shell</code>	40
<code>\pgfplotspointrelaxisxyz</code>	422	<code>plot table</code>	29, 37
<code>\pgfplotspointunitx</code>	423	<code>plot $\{(\textit{math expression})\}$</code>	33
<code>\pgfplotspointunity</code>	423	<code>plot3graphics key</code>	9
<code>\pgfplotspointunitz</code>	423	<code>\plotnum</code>	420
<code>\pgfplotsqpointdescriptionxy</code>	422	<code>\plotnumofactualtype</code>	420
<code>\pgfplotsqpointoutsideofaxis</code>	424	<code>point format key</code>	325
<code>\pgfplotsqpointoutsideofaxisrel</code>	424	<code>point format 3d key</code>	325
<code>\pgfplotsset</code>	55	<code>point meta</code>	
<code>\pgfplotssetlayers</code>	318	<code>point meta=none for smooth patch plots</code> ..	342
<code>\pgfplotstableread</code>	419	<code>point meta key</code>	157
<code>\pgfplotstabletypeset</code>	419	<code>point meta max key</code>	160, 203
<code>\pgfplotsticklabelaxispec</code>	424	<code>point meta min key</code>	160, 203
<code>\pgfplotstransformcoordinatex</code>	422	<code>point meta rel key</code>	160
<code>\pgfplotstransformcoordinatexy</code>	422	<code>points key</code>	44
<code>\pgfplotstransformdirectionx</code>	423	<code>polar library</code>	354
<code>\pgfplotstransformdirectiony</code>	423	<code>polar comb key</code>	357
<code>\pgfplotsunitxinlength</code>	423	<code>polaraxis environment</code>	354
<code>\pgfplotsunitxlength</code>	423	<code>popup size key</code>	326
<code>\pgfplotsunityinlength</code>	424	<code>popup size generic key</code>	326
<code>\pgfplotsunitylength</code>	423	<code>popup size snap key</code>	326
<code>\pgfplotsunitzinlength</code>	424	<code>pos key</code>	268
<code>\pgfplotsunitzlength</code>	423	<code>pos segment key</code>	269
<code>\pgfplotsutilifstringequal</code>	419	<code>Precision</code>	34, 313
<code>\pgfplotsvalueoflargesttickdimen</code>	424	<code>precision key</code>	108
<code>\pgfresetboundingbox</code>	290	<code>Predefined node</code>	
<code>\pgfsetlayers</code>	425	<code>current axis</code>	289
<code>pink key</code>	331	<code>current colorbar axis</code>	205
<code>plot box ratio key</code>	228	<code>current plot begin</code>	267
<code>plot coordinates/</code>		<code>current plot end</code>	267
<code>math parser</code>	28	<code>parent axis</code>	202
<code>plot file/</code>		<code>prefix key</code>	40f., 408
<code>ignore first</code>	53	<code>prefixes</code>	
<code>skip first</code>	53	<code>unit</code>	401
<code>plot graphics/</code>		<code>printable key</code>	326
<code>debug</code>	52		
<code>includegraphics</code>	45	— Q —	
<code>includegraphics cmd</code>	45	<code>quiver/</code>	
<code>lowlevel draw</code>	45	<code>after arrow</code>	72
<code>node</code>	45	<code>before arrow</code>	71
<code>points</code>	44	<code>colored</code>	71
<code>xmax</code>	44	<code>every arrow</code>	71
<code>xmin</code>	44	<code>quiver legend</code>	72
<code>ymax</code>	44	<code>scale arrows</code>	71
<code>ymin</code>	44	<code>u</code>	70
<code>zmax</code>	44	<code>u value</code>	71
<code>zmin</code>	44	<code>update limits</code>	71
<code>plot graphics key</code>	45	<code>v</code>	70
<code>Plot operations</code>		<code>v value</code>	71
$(\langle x \textit{ expression} \rangle, \langle y \textit{ expression} \rangle, \langle z \textit{ expression} \rangle)$		<code>w</code>	70
94		<code>w value</code>	71
<code>coordinates</code>	91	<code>quiver key</code>	68
<code>expression</code>	94	<code>quiver legend key</code>	72
<code>file</code>	91		
<code>table</code>	92	— R —	
$\{(\textit{math expression})\}$	93	<code>raw gnuplot key</code>	40
<code>plot $(\langle x \textit{ expression} \rangle, \langle y \textit{ expression} \rangle)$</code>	35	<code>read completely key</code>	32
<code>plot coordinates</code>	27	<code>redyellow key</code>	147
<code>plot expression</code>	35	<code>\ref</code>	187
<code>plot file</code>	52	<code>refstyle key</code>	187
		<code>rel axis cs coordinate system</code>	267

reset nontranslations key	172
restrict expr to domain key	302
restrict expr to domain* key	302
restrict x to domain key	301
restrict x to domain* key	302
restrict y to domain key	302
restrict y to domain* key	302
restrict z to domain key	302
restrict z to domain* key	302
reverse legend key	184
reverse stacked plots key	74
richtext key	326
row sep key	32
rows key	92, 335

— S —

sample size key	377
sample size max key	378
sample size min key	378
samples key	36
samples at key	36
samples y key	36
scale key	218
scale arrows key	71
scale mode key	9, 223
scale only axis key	210
scale ticks above exponent key	261
scale ticks below exponent key	261
scale uniformly strategy key	223
scaled ticks key	257
scaled x ticks key	257
scaled y ticks key	257
scaled z ticks key	257
scaling key	9
scanline marks key	125
scanline verbose key	92

scatter/

classes	81
use mapped color	80
scatter key	79
scatter src key	79
script key	125
script extension key	125
semilogxaxis environment	22
semilogyaxis environment	22
semithick key	141
separate axis lines key	194
sepia key	331
set layers key	316
set point meta if empty key	160
shader key	103
sharp plot key	56
plot shell	40
shell escape key	411
show origin key	368
show origin code key	368
show suggested version key	320
size x key	136
size y key	136
size z key	136
skip coords between index key	301
skip first key	53
skip first n key	33
slope format key	326

sloped/

allow upside down	172
execute for upside down	172
reset nontranslations	172
sloped key	269
sloped like x axis key	171
sloped like y axis key	171
sloped like z axis key	171
small key	221
smithchart environment	360
smithchart library	360
smithchart mirrored key	361
smooth key	57
snap dist key	326
solid key	139
spring key	331
stack dir key	74
stack plots key	72
standard key	316
statistics library	372
.style handler	273
Subplots	285
subtickwidth key	261
summer key	331
surf	
point meta=none and global paths	342
surf key	101
surf shading/	
precision	108
symbolic coords key	387
symbolic x coords key	296
symbolic y coords key	296
symbolic z coords key	296
system call key	410

— T —

plot table	29, 37
Unbalanced Columns	31
table/	
col sep	32
comment chars	33
create col/	
linear regression	305
header	31
ignore chars	33
meta	32
meta expr	32
meta index	32
read completely	32
row sep	32
skip first n	33
tie end x	398
tie end x index	398
tie end y	398
tie end y index	398
tie end z	398
tie end z index	398
white space chars	33
x	31
x error	32
x error expr	32
x error index	32
x expr	31
x index	31

y	31	dashed	139
y error	32	densely dashdotdotted	139
y error expr	32	densely dashdotted	139
y error index	32	densely dashed	139
y expr	31	densely dotted	139
y index	31	dotted	139
z	31	draw	142
z error	32	every mark	136
z error expr	32	external/	
z error index	32	figure name	409
z expr	31	mode	411
z index	31	prefix	408
table	92	shell escape	411
table key	306	system call	410
temp key	331	fill	142
ternary library	388	font	140, 220
ternary limits relative key	393	id	40f.
ternary relative limits key	393	jump mark left	59
ternaryaxis environment	388	jump mark mid	59
text mark key	138	jump mark right	59
text mark as node key	138	line cap	140
text mark style key	138	line join	140
textSize key	326	line width	140
thermal key	332	loosely dashdotdotted	139
thick key	141	loosely dashdotted	139
thin key	141	loosely dashed	139
\thisrow	38	loosely dotted	139
\thisrowno	38	mark indices	137
tick align key	256	mark options	138
tick label style key	277	mark phase	137
tick pos key	256	mark repeat	137
tick scale binop key	260	mark size	136, 220
tick scale label code key	260	miter limit	140
tick style key	276	only marks	78
ticklabel cs coordinate system	163	overlay	289
ticklabel pos key	256	polar comb	357
ticklabel shift key	257	pos	268
ticklabel style key	277	pos segment	269
ticklabel* cs coordinate system	164	prefix	40f.
ticks key	255	raw gnuplot	40
tickwidth key	261	semithick	141
tie end x key	398	sharp plot	56
tie end x index key	398	sloped	269
tie end y key	398	sloped like x axis	171
tie end y index key	398	sloped like y axis	171
tie end z key	398	sloped like z axis	171
tie end z index key	398	smooth	57
tieline/		solid	139
curve style	399	thick	141
each nth tie	398	thin	141
tieline style	398	tieline	397
tieline key	397	trim axis group left	291
tieline style key	398	trim axis group right	292
/tikz/		trim axis left	291
allow upside down	269	trim axis right	291
baseline	283	trim left	291
color	142	trim right	291
const plot	57	ultra thick	141
const plot mark left	58	ultra thin	141
const plot mark mid	58	very thick	141
const plot mark right	58	very thin	141
dashdotdotted	139	xbar	59
dashdotted	139	xbar interval	67

xcomb	68
xshift	420
ybar	61
ybar interval	66
ycomb	68
yshift	420
\tikzappendtofigurename	410
tikzpicture environment	22
\tikzsetexternalprefix	408
\tikzsetfigurename	409
\tikzsetnextfilename	408
tiny key	222
title key	174
title style key	275
translate gnuplot key	40
transpose legend key	185
trim axis group left key	291
trim axis group right key	292
trim axis left key	291
trim axis right key	291
trim left key	291
trim right key	291
try min ticks key	220, 261
try min ticks log key	261

— U —

u key	70
u value key	71
ultra thick key	141
ultra thin key	141
Unbalanced Columns	31
unbounded coords key	88
unit code key	401
unit marking post key	400
unit marking pre key	400
unit markings key	400
unit rescale keep size key	216
unit vector ratio key	216
unit vector ratio* key	216
units library	399
update limits key	71, 245
upper quartile key	375
upper whisker key	375, 382
use mapped color key	80
use units key	399

— V —

v key	70, 227
v value key	71
var 1d key	40
var 2d key	40
variable key	36
variable width key	377
variable width expr key	377
variable width min target key	378
variable y key	36
variance key	307
variance list key	307
variance src key	307
vertex count key	348
vertical sep key	335
very thick key	141
very thin key	141
view/	

az	226
el	227
h	226
v	227
view key	225
violet key	147
visualization depends on key	312

— W —

w key	70
w value key	71
whisker key	382
whisker extend key	378
whisker range key	380
white space chars key	33
width key	204, 209, 327
winter key	332

— X —

x key	31, 210, 270, 306
\x In Coordinate Expressions	34
x axis line style key	193, 276
x coord inv trafo key	296
x coord trafo key	295
x descriptions at key	336
x dir key	213, 233, 241
x error key	32
x error expr key	32
x error index key	32
x explicit key	233
x explicit relative key	233
x expr key	31
x filter key	299
x fixed key	233
x fixed relative key	233
x grid style key	279
x index key	31
x label style key	274
x post scale key	218
x SI prefix key	401
x tick label as interval key	254
x tick label style key	277
x tick scale label style key	277
x unit key	400
x unit prefix key	400
xbar key	59, 61
xbar interval key	67
xbar interval legend key	183
xbar interval stacked key	74
xbar legend key	183
xbar stacked key	74
xcomb key	68
xgrid each nth passes y key	370
xgrid each nth passes y start key	372
xgrid stop at y key	372
xlabel	
Line break	172
Multiline	172
xlabel key	172
xlabel absolute key	173
xlabel near ticks key	173
xlabel shift key	173
xlabel style key	274
xlables at key	335

xmajorgrids key	262
xmajorticks key	255
xmax key	44, 239, 327
xmin key	44, 239, 327
xminorgrids key	262
xminorticks key	255
xmode key	213, 241, 306
xshift key	420
xtick key	245
xtick align key	256
xtick placement tolerance key	261
xtick pos key	256
xtick scale label code key	260
xtick style key	277
xticklabel key	252
xticklabel cs coordinate system	163
xticklabel interval boundaries key	67
xticklabel pos key	256
xticklabel shift key	257
xticklabel style key	277
xticklabel* cs coordinate system	163
xticklabels key	251
xticklabels at key	336
xticklabels from table key	254
xtickmax key	196, 256
xtickmin key	196, 255
xtickten key	250

— Y —

y key	31, 210, 270, 306
y axis line style key	193, 276
y coord inv trafo key	296
y coord trafo key	296
y descriptions at key	336
y dir key	213, 233, 241
y domain key	36
y error key	32
y error expr key	32
y error index key	32
y explicit key	233
y explicit relative key	233
y expr key	31
y filter key	299
y fixed key	233
y fixed relative key	233
y grid style key	279
y index key	31
y label style key	274
y post scale key	218
y SI prefix key	401
y tick label as interval key	254
y tick label style key	277
y tick scale label style key	277
y unit key	400
y unit prefix key	400
ybar key	61, 64
ybar interval key	66f.
ybar interval legend key	183
ybar interval stacked key	74
ybar legend key	183
ybar stacked key	74
ycomb key	68
ygrid each nth passes x key	371
ygrid each nth passes x start key	372

ygrid stop at x key	372
ylabel key	172
ylabel absolute key	173
ylabel near ticks key	173
ylabel shift key	173
ylabel style key	274
ylabels at key	335
ymajorgrids key	262
ymajorticks key	255
ymax key	44, 240, 327
ymin key	44, 239, 327
yminorgrids key	262
yminorticks key	255
ymode key	213, 241, 307
yshift key	420
ytick key	245
ytick align key	256
ytick placement tolerance key	261
ytick pos key	256
ytick scale label code key	260
ytick style key	278
yticklabel key	252
yticklabel around circle key	369
yticklabel around circle* key	369
yticklabel cs coordinate system	163
yticklabel in circle key	368
yticklabel interval boundaries key	67
yticklabel pos key	256
yticklabel shift key	257
yticklabel style key	277
yticklabel* cs coordinate system	163
yticklabels key	251
yticklabels at key	336
yticklabels from table key	254
ytickmax key	196, 256
ytickmin key	196, 255
ytickten key	250

— Z —

z key	31, 210, 270
z axis line style key	193, 276
z buffer key	101
z coord inv trafo key	296
z coord trafo key	296
z dir key	213, 233, 241
z error key	32
z error expr key	32
z error index key	32
z explicit key	233
z explicit relative key	233
z expr key	31
z filter key	299
z fixed key	233
z fixed relative key	233
z grid style key	279
z index key	31
z label style key	274
z post scale key	218
z SI prefix key	401
z tick label as interval key	254
z tick label style key	277
z tick scale label style key	277
z unit key	400
z unit prefix key	400

zbar interval legend key	183
zbar legend key	183
zlabel key	172
zlabel absolute key	173
zlabel near ticks key	173
zlabel shift key	173
zlabel style key	274
zmajorgrids key	262
zmajorticks key	255
zmax key	44, 240
zmin key	44, 239
zminorgrids key	262
zminorticks key	255
zmode key	213, 241
ztick key	246
ztick align key	256
ztick placement tolerance key	261
ztick pos key	256
ztick scale label code key	260
ztick style key	278
zticklabel key	252
zticklabel cs coordinate system	163
zticklabel interval boundaries key	67
zticklabel pos key	256
zticklabel shift key	257
zticklabel style key	277
zticklabel* cs coordinate system	163
zticklabels key	251
zticklabels from table key	254
ztickmax key	196, 256
ztickmin key	196, 255
ztickten key	250

Bibliography

- [1] C. Feuersänger. `PGFPLOTS``TABLE` package – Loading, rounding and formatting tables in LaTeX. Available as separate package `\usepackage{pgfplotstable}`, as part of `PGFPLOTS`.
- [2] C. Feuersänger. Programming in `TEX` and Library Functions from `PGF` and `PGFPLOTS`. Available as part of `PGFPLOTS`, [TeX-programming-notes.pdf](#), March 17, 2013.
- [3] U. Kern. Extending `LATEX`’s color facilities: the `xcolor` package.
- [4] D. P. Story. The `AcroTEX` eEducation Bundle. <http://www.ctan.org/tex-archive/macros/latex/contrib/acrotex>. Sub packages `insdljs` and `eforms` are required for the clickable library.
- [5] T. Tantau. `TikZ` and `PGF` manual. <http://sourceforge.net/projects/pgf>. *v.* ≥ 2.00 .
- [6] K. van Zonneveld. PhP to javascript conversion project (GPL). http://kevin.vanzonneveld.net/techblog/article/phpjs_licensing.
- [7] J. Wright and C. Feuersänger. Implementing `keyval` input: an introduction. <http://pgfplots.sourceforge.net> as `.pdf`, 2008.